GIS · سنجش از دور · SQL · PostGIS مکانی

پایگاه داده مکانی ابری

مرجع جامع سامانه‌ی اطلاعات مکانی، سنجش از دور و SQL مکانی؛ از صفر تا متخصص

مبانی علمی

SQL مکانی گام‌به‌گام

تصویر ماهواره‌ای

نقشه و انتشار وب

برداشت میدانی

تمرین و پروژه

ویرایش نخست · ۱۴۰۵

پایگاه داده مکانی ابری

مرجع جامع سامانه‌ی اطلاعات مکانی، سنجش از دور و SQL مکانی؛ از صفر تا متخصص

ویرایش نخست · ۱۴۰۵

عنوان	پایگاه داده مکانی ابری: مرجع جامع سامانه‌ی اطلاعات مکانی، سنجش از دور و SQL مکانی
پدیدآور	—
ناشر	—
نوبت و سال چاپ	نخست، ۱۴۰۵
قطع	وزیری (۱۷ × ۲۴ سانتی‌متر)
شابک	—
داده‌ها و تصویرها	تصویرهای رابط از محیط نمایشی سامانه؛ داده‌ی OpenStreetMap با مجوز ODbL؛ تصویرهای Sentinel از برنامه‌ی Copernicus (ESA) و Landsat از USGS
حق نشر	همه‌ی حقوق برای پدیدآور محفوظ است. استفاده‌ی آموزشی از تمرین‌ها و کدها با ذکر منبع آزاد است.

فهرست مطالب

پیشگفتار ۱۷

راهنمای استفاده از کتاب ۲۰

بخش ۱: مبانی علم اطلاعات مکانی

۱ سامانه‌ی اطلاعات مکانی چیست

۱-۱ مکان در همه‌چیز هست ۲۷

۱-۲ تعریف سامانه‌ی اطلاعات مکانی ۲۷

۱-۳ پنج پرسشی که GIS به آن‌ها پاسخ می‌دهد ۲۹

۱-۴ نگاهی کوتاه به تاریخ ۳۰

۱-۵ مفهوم لایه ۳۰

۱-۶ کاربردهای GIS

۱-۷ ابری و پایگاه داده مکانی ابری GIS ۳۲

۱-۸ جریان کار GIS در این کتاب ۳۵

۲ داده‌ی مکانی؛ مدل برداری و رستری ۳۷

۲-۱ جهان واقعی و مدل آن ۳۹

۲-۲ مدل برداری ۳۹

۲-۳ مدل رستری ۴۳

۲-۴ کدام مدل، برای کدام مسئله

۲-۵ ویژگی‌ها و سطح اندازه‌گیری ۴۵

۲-۶ قالب‌های فایل مکانی ۴۶

۲-۷ مقیاس، دقت و صحت

۲-۸ کیفیت داده و فراداده ۴۸

۳ شکل زمین، سیستم مختصات و تصویر نقشه ۵۰

۳-۱ چرا این فصل مهم‌ترین فصل مبانی است ۵۲

۳-۲ شکل زمین: ژئوئید و بیضوی ۵۲

۳-۳ داده‌نما (Datum) ۵۳

۳-۴ مختصات جغرافیایی ۵۴

۳-۵ تصویر نقشه ۵۵
۳-۶ سیستم UTM ۵۷
۳-۷ شناسنامه‌ی سیستم مختصات: EPSG و SRID ۵۹
۳-۸ دو تابعی که نباید با هم اشتباه شوند ۶۰
۳-۹ قاعده‌ی طلایی کار با مختصات ۶۱

۴ پایگاه داده‌ی مکانی و معماری WebGIS در ابر ۶۳

۴-۱ از فایل تا پایگاه داده ۶۵
۴-۲ پایگاه داده‌ی رابطه‌ای در یک نگاه ۶۵
۴-۳ PostGIS و PostgreSQL ۶۶
۴-۴ افزونه‌ها ۶۸
۴-۵ معماری یک سامانه‌ی WebGIS
۴-۶ پایگاه داده مکانی ابری چه کاری برای شما می‌کند
۴-۷ مسیر داده در این کتاب ۷۲

بخش ۲: آغاز کار با پایگاه داده مکانی ابری

۵ حساب کاربری، ساخت پایگاه داده و فضای کار

۵-۱ ورود به سامانه ۷۷
۵-۲ پایگاه‌های داده‌ی من ۷۸
۵-۳ ساخت پایگاه داده ۷۸
۵-۴ صفحه‌ی «نمای کلی و اتصال»
۵-۵ اولین پرسش ۸۲
۵-۶ فضای کار ۸۲
۵-۷ رمز تازه و افزونه‌ی تازه ۸۴
۵-۸ چرخه‌ی عمر پایگاه داده ۸۴

۶ اتصال از pgAdmin، ArcGIS، QGIS و برنامه‌ها ۸۶

۶-۱ یک پایگاه، همه‌ی نرم‌افزارها ۸۸
۶-۲ اتصال از QGIS
۶-۳ اتصال از ArcGIS Pro ۹۰
۶-۴ اتصال از pgAdmin، DBeaver و psql ۹۰
۶-۵ رشته‌ی اتصال در قالب‌های مختلف ۹۱

۶-۶ اتصال از برنامه‌ها: یک نگاه سریع ۹۲
۶-۷ اتصال از GeoServer ۹۳
۶-۸ چرا از مرورگر مستقیم وصل نشویم ۹۳
۶-۹ خطاهای رایج اتصال ۹۴

۷ ورود داده و سلامت داده ۹۶

۷-۱ داده از کجا می‌آید ۹۸
۷-۲ ورود فایل در سه گام ۹۸
۷-۳ افزودن فایل به جدول موجود
۷-۴ فایل‌های بزرگ: ogr2ogr، QGIS و shp2pgsql ۱۰۲
۷-۵ سلامت داده ۱۰۳

۸ مرور، جست‌وجو و خروجی گرفتن از داده ۱۰۸

۸-۱ مرورگر داده ۱۱۰
۸-۲ صفحه‌ی هر جدول ۱۱۱
۸-۳ یافتن عارضه ۱۱۳
۸-۴ خروجی گرفتن

۹ ساخت جدول، ویرایش داده و تغییر ساختار ۱۱۸

۹-۱ ساخت جدول تازه ۱۲۰
۹-۲ ویرایش داده با پیش‌نویس ۱۲۲
۹-۳ تغییر ساختار جدول ۱۲۵
۹-۴ حذف جدول

بخش ۳: زبان SQL از صفر

۱۰ پایگاه داده‌ی رابطه‌ای و زبان SQL

۱۰-۱ چرا SQL ۱۳۱
۱۰-۲ مدل رابطه‌ای ۱۳۱
۱۰-۳ انواع داده ۱۳۳
۱۰-۴ دسته‌های دستورهای SQL ۱۳۵
۱۰-۵ کنسول SQL ۱۳۵
۱۰-۶ طراحی جدول: نرمال‌سازی در یک نگاه ۱۳۸

۱۱ پرسش از یک جدول؛ SELECT، WHERE و ORDER BY ۱۴۰

۱۱-۱ ساختار یک پرسش ۱۴۲
۱۱-۲ انتخاب ستون‌ها ۱۴۲
۱۱-۳ فیلتر کردن با WHERE ۱۴۴
۱۱-۴ مرتب‌سازی، محدود کردن و صفحه‌بندی
۱۱-۵ دسته‌بندی با CASE
۱۱-۶ توابع پرکاربرد
۱۱-۷ جست‌وجوی تقریبی نام‌ها ۱۵۰

۱۲ تجمیع، گروه‌بندی و توابع پنجره‌ای ۱۵۲

۱۲-۱ از سطر تا خلاصه ۱۵۴
۱۲-۲ خلاصه برای هر دسته: GROUP BY
۱۲-۳ فیلتر روی دسته‌ها: HAVING ۱۵۶
۱۲-۴ چند شمارش در یک پرسش: FILTER ۱۵۶
۱۲-۵ آمار توصیفی ۱۵۷
۱۲-۶ توابع پنجره‌ای ۱۵۸
۱۲-۷ تجمیع مکانی: نگاهی به فصل‌های بعد ۱۶۰

۱۳ پیوند جدول‌ها، زیرپرسش و CTE ۱۶۳

۱۳-۱ چرا پیوند ۱۶۵
۱۳-۲ فقط جفت‌های جور: INNER JOIN ۱۶۵
۱۳-۳ همه‌ی سطرهای سمت چپ: LEFT JOIN ۱۶۶
۱۳-۴ RIGHT، FULL و CROSS ۱۶۷
۱۳-۵ نخستین پیوند مکانی ۱۶۸
۱۳-۶ پیوند با خود ۱۶۹
۱۳-۷ زیرپرسش ۱۷۰
۱۳-۸ پرسش‌های گام‌به‌گام: CTE ۱۷۱
۱۳-۹ ترکیب مجموعه‌ها
۱۳-۱۰ پیوند با یک زیرپرسش برای هر سطر: LATERAL ۱۷۳

۱۴ ساخت و تغییر داده؛ DML، DDL، قید و تراکنش ۱۷۶

۱۴-۱ پیش از شروع: حالت نوشتن ۱۷۸
۱۴-۲ ساختن جدول با قید ۱۷۸
۱۴-۳ ذخیره‌ی نتیجه‌ی تحلیل: CREATE TABLE AS

۱۴-۴ افزودن سطر :INSERT
۱۴-۵ تغییر سطرها :UPDATE ۱۸۲
۱۴-۶ حذف سطرها :DELETE
۱۴-۷ درج یا به‌روزرسانی :UPSERT ۱۸۴
۱۴-۸ تراکنش: همه یا هیچ
۱۴-۹ نما و نمای مادی‌شده
۱۴-۱۰ فراداده روی جدول ۱۸۶
۱۴-۱۱ ساختن بقیه‌ی داده‌ی نمونه ۱۸۷

بخش ۴: SQL مکانی با PostGIS

۱۵ هندسه در SQL؛ ساختن، خواندن و قالب‌ها
۱۵-۱ هندسه، یک نوع داده ۱۹۲
۱۵-۲ ساختن هندسه ۱۹۲
۱۵-۳ خروجی گرفتن: قالب‌ها ۱۹۵
۱۵-۴ خواندن اجزای هندسه ۱۹۶
۱۵-۵ مرکز، نقطه‌ی روی سطح و محدوده ۱۹۸
۱۵-۶ geography یا geometry؟ ۱۹۹
۱۶ اندازه‌گیری؛ فاصله، طول، مساحت و جهت ۲۰۲
۱۶-۱ چهار پرسش اندازه‌گیری ۲۰۴
۱۶-۲ سه روش، سه دقت ۲۰۵
۱۶-۳ جدول تصمیم ۲۰۸
۱۶-۴ طول خط‌ها ۲۰۸
۱۶-۵ جهت و مسیریابی هوایی
۱۶-۶ فاصله بین چندضلعی‌ها و خط‌ها ۲۰۹
۱۷ روابط مکانی و پیوند مکانی ۲۱۲
۱۷-۱ گزاره‌ی مکانی ۲۱۴
۱۷-۲ مدل DE-9IM ۲۱۵
۱۷-۳ Covers یا Contains؟ ۲۱۷
۱۷-۴ الگوهای پیوند مکانی ۲۱۸
۱۷-۵ سرعت پرسش‌های مکانی ۲۲۱

۱۸ پردازش هندسه؛ حریم، ادغام، برش و ساده‌سازی ۲۲۴

۱۸-۱ هندسه‌ی تازه از هندسه ۲۲۶

۱۸-۲ حریم (بافر) ۲۲۶

۱۸-۳ ادغام (Union / Dissolve) ۲۲۸

۱۸-۴ عملیات همپوشانی ۲۲۹

۱۸-۵ ساده‌سازی، گرد کردن و چسباندن ۲۳۱

۱۸-۶ پوسته و تقسیم فضا

۱۸-۷ خرد کردن ۲۳۴

۱۸-۸ ارجاع خطی ۲۳۴

۱۸-۹ تبدیل‌های کمکی

۱۹ نزدیک‌ترین همسایه (KNN) ۲۳۷

۱۹-۱ پرسش «نزدیک‌ترین» ۲۳۹

۱۹-۲ عملگر <-> ۲۳۹

۱۹-۳ برای هر سطر، نزدیک‌ترین: LATERAL ۲۴۰

۱۹-۴ نزدیک‌ترین با شرط ۲۴۲

۱۹-۵ الگوی دو مرحله‌ای برای دقت ۲۴۴

۱۹-۶ نزدیک‌ترین روی خط

۲۰ شاخص مکانی و کارایی پرسش‌ها ۲۴۶

۲۰-۱ چرا شاخص ۲۴۸

۲۰-۲ درخت R ۲۴۸

۲۰-۳ فیلتر دو مرحله‌ای ۲۴۹

۲۰-۴ آزمایش: با شاخص و بی‌شاخص ۲۴۹

۲۰-۵ خطاهایی که شاخص را بی‌اثر می‌کنند ۲۵۳

۲۰-۶ انواع دیگر شاخص ۲۵۴

۲۰-۷ هندسه‌های بزرگ: ST_Subdivide ۲۵۴

۲۰-۸ نگه‌داری ۲۵۵

۲۰-۹ فهرست کنترل کارایی ۲۵۵

۲۱ آمار فضایی با SQL؛ تراکم، خوشه و کانون داغ ۲۵۷

۲۱-۱ از «کجا» تا «چرا اینجا» ۲۵۹

۲۱-۲ الگوی پراکنش: شاخص نزدیک‌ترین همسایه ۲۵۹

۲۱-۳ شبکه‌ی منظم و تراکم ۲۶۰
۲۱-۴ خوشه‌بندی ۲۶۲
۲۱-۵ خودهمبستگی فضایی: شاخص موران ۲۶۴
۲۱-۶ کانون داغ: آماره‌ی Gi* گتیس-ارد ۲۶۶
۲۲ اعتبار هندسه، توپولوژی و هندسه‌ی سه‌بعدی ۲۶۹
۲۲-۱ هندسه‌ی معتبر چیست ۲۷۱
۲۲-۲ اصلاح: ST_MakeValid ۲۷۲
۲۲-۳ قواعد توپولوژی با SQL ۲۷۳
۲۲-۴ توپولوژی ذخیره‌شده: postgis_topology
۲۲-۵ هندسه‌ی سه‌بعدی ۲۷۶
۲۳ رستر در PostGIS ۲۷۸
۲۳-۱ رستر کنار برداری ۲۸۰
۲۳-۲ ورود رستر
۲۳-۳ آمار و مقدار پیکسل ۲۸۲
۲۳-۴ آمار ناحیه‌ای ۲۸۳
۲۳-۵ تحلیل ارتفاع: شیب، جهت و سایه‌روشن ۲۸۴
۲۳-۶ جبر نقشه و طبقه‌بندی دوباره ۲۸۵
۲۳-۷ از رستر به برداری ۲۸۶
۲۳-۸ خروجی رستر ۲۸۷
۲۳-۹ رستر در پایگاه یا بیرون از آن؟ ۲۸۷
۲۴ تحلیل شبکه و مسیریابی با pgRouting ۲۸۹
۲۴-۱ از فاصله‌ی هوایی تا فاصله‌ی شبکه‌ای ۲۹۱
۲۴-۲ شبکه به‌عنوان گراف ۲۹۱
۲۴-۳ شبکه‌ی نمونه
۲۴-۴ کوتاه‌ترین (سریع‌ترین) مسیر ۲۹۴
۲۴-۵ چسباندن نقطه به شبکه ۲۹۶
۲۴-۶ حوزه‌ی دسترسی
۲۴-۷ ماتریس هزینه و نزدیک‌ترین مرکز روی شبکه ۲۹۷
۲۴-۸ توابع دیگر pgRouting

بخش ۵: تحلیل مکانی بدون کد

۲۵ ابزارهای تحلیل بدون کد؛ مجاورت و فاصله

۲۵-۱ ابزارها: SQL مکانی پشت یک فرم ۳۰۴

۲۵-۲ چرخه‌ی کار با هر ابزار

۲۵-۳ حریم (بافر)

۲۵-۴ نزدیک‌ترین عارضه ۳۱۰

۲۵-۵ انتخاب بر اساس فاصله

۲۵-۶ حوزه‌ی نفوذ (تیسن) ۳۱۱

۲۵-۷ پس از اجرا ۳۱۲

۲۶ شمارش، تراکم و خوشه بدون کد ۳۱۳

۲۶-۱ شمارش در چندضلعی ۳۱۵

۲۶-۲ نقشه‌ی تراکم شش‌ضلعی ۳۱۶

۲۶-۳ خوشه‌بندی نقطه‌ها

۲۶-۴ طول و مساحت در چندضلعی ۳۱۷

۲۷ همپوشانی و تغییر بدون کد ۳۱۸

۲۷-۱ همپوشانی دو لایه‌ی چندضلعی ۳۲۰

۲۷-۲ سهم همپوشانی ۳۲۰

۲۷-۳ تغییر بین دو نسخه ۳۲۱

۲۷-۴ برش با مرز ۳۲۲

۲۷-۵ حذف ناحیه ۳۲۳

۲۸ تحلیل شبکه‌ی معابر بدون کد ۳۲۴

۲۸-۱ پیش‌نیاز: شبکه‌ی مسیریابی ۳۲۶

۲۸-۲ حوزه‌ی دسترسی ۳۲۶

۲۸-۳ کوتاه‌ترین مسیر ۳۲۷

۲۸-۴ نزدیک‌ترین روی شبکه ۳۲۷

۲۸-۵ پوشش چند مرکز ۳۲۸

۲۹ پرسش فارسی از داده ۳۲۹

۲۹-۱ از جمله به SQL ۳۳۱

۲۹-۲ راه‌اندازی

۲۹-۳ نمونه‌ی پرسش‌ها ۳۳۲
۲۹-۴ را بخوانید SQL ۳۳۲
۲۹-۵ تاریخچه‌ی پرسش‌ها ۳۳۳

۳۰ داده‌ی آماده‌ی ایران از OpenStreetMap ۳۳۴

۳۰-۱ OpenStreetMap ۳۳۶
۳۰-۲ ورود با یک کلیک ۳۳۶
۳۰-۳ جدول‌ها
۳۰-۴ پرسش از داده‌ی OSM
۳۰-۵ کامل بودن و تازگی ۳۳۹

بخش ۶: سنجش از دور

۳۱ سنجش از دور؛ نور، طیف و امضای طیفی

۳۱-۱ دیدن بدون لمس ۳۴۴
۳۱-۲ تابش الکترومغناطیس ۳۴۵
۳۱-۳ تابش و جو ۳۴۶
۳۱-۴ برهم‌کنش با سطح زمین ۳۴۷
۳۱-۵ امضای طیفی ۳۴۷
۳۱-۶ ترکیب رنگی: دیدن نامرئی‌ها ۳۴۹

۳۲ ماهواره، سنجنده و چهار تفکیک ۳۵۱

۳۲-۱ چهار تفکیک ۳۵۳
۳۲-۲ مدار ماهواره
۳۲-۳ Sentinel-2 ۳۵۴
۳۲-۴ Landsat 8 و 9 ۳۵۵
۳۲-۵ مدل رقومی ارتفاع Copernicus ۳۵۶
۳۲-۶ سنجنده‌های دیگری که باید بشناسید ۳۵۷
۳۲-۷ کدام سنجنده برای کدام کار

۳۳ پیش‌پردازش؛ از عدد خام تا بازتاب سطح ۳۵۹

۳۳-۱ چرا پیش‌پردازش ۳۶۱
۳۳-۲ سطوح پردازش ۳۶۱
۳۳-۳ از عدد دیجیتال به بازتاب ۳۶۲

۳۳-۴ ابر و سایه ۳۶۳
۳۳-۵ ترکیب بی‌ابر ۳۶۴
۳۳-۶ هندسه: ارتو و بازنمونه‌گیری ۳۶۵
۳۳-۷ چگونه فقط تکه‌ی لازم دانلود می‌شود: COG و STAC
۳۴ دریافت تصویر ماهواره‌ای در پایگاه داده مکانی ابری ۳۶۷
۳۴-۱ بخش «تصویر ماهواره‌ای» ۳۶۹
۳۴-۲ گام‌به‌گام: دریافت تصویر تازه ۳۶۹
۳۴-۳ پس از ثبت ۳۷۴
۳۴-۴ در QGIS
۳۴-۵ سقف‌ها و سهمیه ۳۷۷
۳۵ شاخص‌های طیفی ۳۷۹
۳۵-۱ از امضای طیفی تا یک عدد ۳۸۱
۳۵-۲ شاخص تفاضل نرمال‌شده‌ی پوشش گیاهی: NDVI ۳۸۱
۳۵-۳ شاخص‌های آب ۳۸۲
۳۵-۴ رطوبت پوشش گیاهی: NDMI ۳۸۳
۳۵-۵ و شدت آتش‌سوزی NBR ۳۸۳
۳۵-۶ سطوح ساخته‌شده: NDBI ۳۸۴
۳۵-۷ شاخص‌های پیشرفته‌تر ۳۸۴
۳۵-۸ انتخاب شاخص ۳۸۵
۳۵-۹ محاسبه در سامانه
۳۵-۱۰ محاسبه با SQL روی رستر ۳۸۶
۳۶ آمار هر قطعه؛ از تصویر به جدول ۳۸۸
۳۶-۱ عدد هر قطعه، نه عکس ۳۹۰
۳۶-۲ گام‌به‌گام: تحلیل تازه ۳۹۰
۳۶-۳ جدول نتیجه ۳۹۲
۳۶-۴ پرسیدن از نتیجه ۳۹۳
۳۶-۵ نقشه‌ی سلامت قطعه‌ها ۳۹۶
۳۷ سری زمانی، فنولوژی و پایش خودکار ۳۹۸
۳۷-۱ زمان، بُعد چهارم ۴۰۰
۳۷-۲ فنولوژی ۴۰۰

۳۷-۳ نمودار روند در سامانه ۴۰۰
۳۷-۴ سری زمانی با SQL ۴۰۱
۳۷-۵ پایش خودکار ۴۰۴
۳۸ مدل رقومی ارتفاع و تحلیل زمین ۴۰۸
۳۸-۱ سه مدل ارتفاعی ۴۱۰
۳۸-۲ دریافت DEM در سامانه ۴۱۱
۳۸-۳ مشتق‌های زمین ۴۱۱
۳۸-۴ پروفیل ارتفاعی ۴۱۲
۳۸-۵ منحنی میزان ۴۱۳
۳۸-۶ هیدرولوژی: آب کجا می‌رود ۴۱۴
۳۹ طبقه‌بندی تصویر و آشکارسازی تغییر ۴۱۷
۳۹-۱ از پیکسل تا کلاس ۴۱۹
۳۹-۲ سه رویکرد
۳۹-۳ نمونه‌ی آموزشی در پایگاه داده
۳۹-۴ جریان کار در QGIS ۴۲۲
۳۹-۵ ارزیابی صحت ۴۲۳
۳۹-۶ آشکارسازی تغییر ۴۲۶
۴۰ رادار و سنجش حرارتی ۴۲۸
۴۰-۱ چرا رادار ۴۳۰
۴۰-۲ رادار روزنه‌ی مصنوعی ۴۳۰
۴۰-۳ Sentinel-1 ۴۳۲
۴۰-۴ تداخل‌سنجی و فرونشست ۴۳۲
۴۰-۵ سنجش حرارتی ۴۳۳

بخش ۷: کارتوگرافی و انتشار

۴۱ اصول کارتوگرافی
۴۱-۱ نقشه، استدلال بصری ۴۳۹
۴۱-۲ اجزای نقشه و سلسله‌مراتب بصری ۴۳۹
۴۱-۳ متغیرهای بصری
۴۱-۴ نقشه‌ی کروپلت و نرمال‌سازی ۴۴۰

۴۱-۵ طبقه‌بندی داده ۴۴۱
۴۱-۶ رنگ ۴۴۳
۴۱-۷ نماد، برچسب و تعمیم ۴۴۴
۴۱-۸ چک‌لیست نقشه‌ی خوب ۴۴۴
۴۲ استودیوی نقشه‌ی چندلایه ۴۴۶
۴۲-۱ نمایش با هم و نقشه‌ی ذخیره‌شده ۴۴۸
۴۲-۲ استودیو ۴۴۸
۴۲-۳ کارایی: کاشی برداری ۴۵۱
۴۳ نمودار و داشبورد روی نقشه ۴۵۲
۴۳-۱ نقشه و نمودار، دو چشم یک داده ۴۵۴
۴۳-۲ ساختن نمودار ۴۵۴
۴۳-۳ کدام نمودار، برای کدام پرسش
۴۳-۴ داشبورد و نقشه‌ی عمومی ۴۵۷
۴۴ اشتراک، ارائه، چاپ و مقایسه ۴۵۸
۴۴-۱ اشتراک و خروجی ۴۶۰
۴۴-۲ ارائه با اسلاید ۴۶۱
۴۴-۳ چاپ، PDF و PNG
۴۴-۴ مقایسه‌ی دو لایه
۴۴-۵ WMS/WFS و QGIS ۴۶۳
۴۵ انتشار داده؛ کاشی برداری، OGC API و WMS/WFS ۴۶۴
۴۵-۱ چرا انتشار ۴۶۶
۴۵-۲ کدام قالب، برای کدام کار ۴۶۶
۴۵-۳ انتشار لایه ۴۶۶
۴۵-۴ کلید API
۴۵-۵ QGIS Server با WFS و WMS ۴۶۹
۴۵-۶ سقف درخواست روزانه
۴۶ ساخت برنامه‌ی نقشه‌ی وب؛ REST API، Leaflet، MapLibre و فرم میدانی ۴۷۰
۴۶-۱ صفحه‌ی وب با MapLibre ۴۷۲
۴۶-۲ Leaflet و OpenLayers ۴۷۴
۴۶-۳ REST API ۴۷۴

۴۶-۴ فرم جمع‌آوری میدانی ۴۷۵

بخش ۸: برداشت میدانی و نقشه‌برداری

۴۷ GNSS؛ موقعیت‌یابی ماهواره‌ای، RTK و NTRIP

۴۷-۱ ماهواره‌ها چگونه موقعیت می‌دهند ۴۷۹

۴۷-۲ منابع خطا ۴۷۹

۴۷-۳ سطوح دقت ۴۸۰

۴۷-۴ RTCM و NTRIP

۴۷-۵ جمله‌های NMEA ۴۸۱

۴۷-۶ کدام دقت، برای کدام کار ۴۸۲

۴۸ پروژه‌ی برداشت میدانی ۴۸۳

۴۸-۱ برداشت میدانی در پایگاه داده ۴۸۵

۴۸-۲ ساختن پروژه ۴۸۵

۴۸-۳ برداشت ۴۸۶

۴۸-۴ کار بدون اینترنت

۴۸-۵ ورود و خروج ۴۸۷

۴۸-۶ داده‌ی برداشت در بقیه‌ی سامانه ۴۸۷

۴۹ محاسبات نقشه‌برداری؛ COGO، سطح، منحنی میزان و حجم ۴۸۸

۴۹-۱ داده‌ی این فصل ۴۹۰

۴۹-۲ هندسه‌ی مختصاتی :COGO ۴۹۰

۴۹-۳ پیاده‌کردن ۴۹۱

۴۹-۴ سطح TIN ۴۹۱

۴۹-۵ منحنی میزان از TIN ۴۹۲

۴۹-۶ حجم خاک‌برداری و خاک‌ریزی ۴۹۲

بخش ۹: حرفه‌ای و سازمانی

۵۰ تیم، نقش‌ها و امنیت داده

۵۰-۱ دو نوع دسترسی ۴۹۷

۵۰-۲ کمترین دسترسی با SQL ۴۹۸

۵۰-۳ حریم خصوصی داده‌ی مکانی ۴۹۹

۵۰-۴ فهرست کنترل امنیت ۴۹۹

۵۱ بکاپ، بازگردانی، مصرف و کارایی ۵۰۰

۵۱-۱ بکاپ ۵۰۲

۵۱-۲ مصرف و کارایی

۵۱-۳ اتصال‌ها و PgBouncer ۵۰۵

۵۲ امکانات سازمانی ۵۰۶

۵۲-۱ امکانات سازمانی در یک نگاه ۵۰۸

۵۲-۲ اختصاصی GeoServer ۵۰۸

۵۲-۳ بکاپ ساعتی و کپی آزمایشی

۵۲-۴ سرویس رستر ۵۰۹

۵۳ برنامه‌نویسی با پایگاه داده‌ی مکانی؛ تابع، تریگر، Python و خودکارسازی ۵۱۰

۵۳-۱ منطق در پایگاه داده ۵۱۲

۵۳-۲ تابع SQL ۵۱۲

۵۳-۳ تریگر: کار خودکار پس از هر تغییر

۵۳-۴ Python ۵۱۴

۵۳-۵ خودکارسازی ۵۱۵

۵۴ پایگاه داده مکانی ابری در کلاس ۵۱۷

۵۴-۱ چرا کلاس ۵۱۹

۵۴-۲ چرخه‌ی کلاس ۵۱۹

۵۴-۳ طراحی تمرین خوب ۵۲۰

۵۴-۴ این کتاب به‌عنوان درس

۵۵ پروژه‌های جامع ۵۲۱

۵۵-۱ ساختار هر پروژه ۵۲۳

۵۵-۲ پروژه‌ی ۱: دسترسی دانش‌آموزان به مدرسه ۵۲۳

۵۵-۳ پروژه‌ی ۲: پایش خشکسالی کشاورزی یک دشت ۵۲۳

۵۵-۴ پروژه‌ی ۳: تغییر کاربری در حریم رودخانه ۵۲۴

۵۵-۵ پروژه‌ی ۴: پوشش اورژانس و آتش‌نشانی ۵۲۴

۵۵-۶ پروژه‌ی ۵: کانون‌های تصادف و پیشنهاد اصلاح

۵۵-۷ پروژه‌ی ۶: برداشت و حجم‌سنجی یک محوطه‌ی عمرانی ۵۲۵

۵۵-۸ معیار ارزیابی پروژه (برای مدرس)

۹-۵۵ پایان سخن ۵۲۶
پیوست الف ـ واژه‌نامه ۵۲۶
پیوست ب ـ مرجع توابع پرکاربرد PostGIS ۵۲۹
پیوست ج ـ سیستم‌های مختصات ایران ۵۳۹

پیشگفتار

هر چیزی که در جهان رخ می‌دهد «جایی» رخ می‌دهد. مدرسه در محله‌ای است، مزرعه در دشتی، خط لوله از کنار رودی می‌گذرد و آمبولانس باید از کوتاه‌ترین مسیر به بیمار برسد. پرسش «کجا؟» در کنار «چه؟» و «چه وقت؟» یکی از پایه‌ای‌ترین پرسش‌های تصمیم‌گیری است، و دانشی که به این پرسش پاسخ می‌دهد امروز در همه‌ی رشته‌ها حضور دارد: از شهرسازی و کشاورزی تا بهداشت، بیمه، مخابرات، امداد و نجات و مدیریت منابع آب.

این دانش تا چند سال پیش در انحصار نرم‌افزارهای سنگین رومیزی، فایل‌های پراکنده و کارشناسانی بود که سال‌ها با آن نرم‌افزارها کار کرده بودند. سه تحول این وضع را عوض کرد. نخست، **پایگاه داده‌ی مکانی** که داده‌ی مکانی را مثل هر داده‌ی دیگری در جدول نگه می‌دارد و با زبان استاندارد SQL پرسش‌پذیر می‌کند. دوم، **رایانش ابری** که ساختن و نگه‌داشتن چنین پایگاهی را از یک پروژه‌ی زیرساختی چندهفته‌ای به چند کلیک تبدیل کرد. سوم، **تصویرهای ماهواره‌ای رایگان** مثل Sentinel-2 و Landsat که هر چند روز یک بار از هر نقطه‌ی زمین عکس می‌گیرند و در دسترس همه‌اند.

کتابی که در دست دارید حاصل کنار هم گذاشتن این سه تحول است. موضوع آن **پایگاه داده مکانی ابری** است: سامانه‌ای تحت وب که در آن یک پایگاه داده‌ی PostgreSQL با افزونه‌ی PostGIS می‌سازید، داده وارد می‌کنید، با SQL یا ابزارهای آماده تحلیل می‌کنید، تصویر ماهواره‌ای سفارش می‌دهید، نقشه می‌سازید و نتیجه را منتشر می‌کنید؛ بدون نصب هیچ نرم‌افزاری.

چرا این کتاب

کتاب‌های خوبی درباره‌ی سامانه‌ی اطلاعات مکانی (GIS)، سنجش از دور و PostGIS نوشته شده است، اما معمولاً یکی از این سه را پوشش می‌دهند و یا آن‌قدر نظری‌اند که خواننده نمی‌داند فردا صبح در محل کار با آن‌ها چه کند، یا آن‌قدر نرم‌افزارمحورند که با عوض شدن یک دکمه بی‌اعتبار می‌شوند. هدف این کتاب پر کردن این فاصله است. هر فصل سه لایه دارد:

1. **مفهوم:** نظریه‌ی علمی موضوع، مستقل از هر نرم‌افزار. این لایه همان چیزی است که در کلاس دانشگاه تدریس می‌شود و ده سال دیگر هم معتبر است.
2. **در پایگاه داده مکانی ابری:** اجرای گام‌به‌گام همان مفهوم در سامانه، با تصویر واقعی رابط کاربری.
3. **زیر کاپوت:** همان کار با SQL. این لایه شما را از «کاربر یک نرم‌افزار» به «متخصص داده‌ی مکانی» تبدیل می‌کند، چون SQL مکانی زبان مشترک PostGIS، QGIS، GeoServer و تقریباً همه‌ی سامانه‌های مکانی حرفه‌ای است.

تأکید این کتاب بر **SQL و به‌ویژه SQL مکانی** آگاهانه است. ابزارهای گرافیکی می‌آیند و می‌روند، اما کسی که بتواند پرسش مکانی خود را به SQL بنویسد، به هر پایگاه داده‌ی مکانی در هر جای دنیا که وصل شود می‌تواند کارش را انجام دهد. به همین دلیل دو بخش کامل کتاب (بخش‌های سوم و چهارم) به SQL اختصاص دارد و در فصل‌های دیگر هم معادل SQL هر کار آمده است.

همه‌ی کدها اجرا شده‌اند

کدهای SQL این کتاب روی **داده‌ی نمونه**‌ای اجرا شده‌اند که در فصل ۵ آن را در پایگاه خودتان می‌سازید: مدرسه‌ها، ایستگاه‌های مترو، پارک‌ها، محله‌ها و مراکز درمانی بخشی از تهران، به‌علاوه‌ی قطعه‌های یک دشت کشاورزی. جدول نتیجه‌ای که زیر بسیاری از کدها می‌بینید خروجی واقعی همان کد روی همان داده است، نه عددی که دستی نوشته شده باشد. اگر نتیجه‌ی شما با کتاب فرق داشت، یعنی داده یا کدتان با کتاب فرق دارد و همین تفاوت خودش درس خوبی است.

برای مدرسان

این کتاب از ابتدا برای تدریس طراحی شده است. هر فصل اهداف یادگیری قابل سنجش، تمرین در سه سطح، مینی‌پروژه و کادر «برای مدرس» دارد که زمان پیشنهادی کلاس و نکته‌های تدریس را می‌گوید. سامانه بخشی به نام «کلاس» دارد که در آن استاد تمرین می‌دهد، دانشجو در پایگاه داده‌ی خودش کار می‌کند و تحویل می‌دهد و استاد نمره می‌دهد؛ فصل ۵۴ این چرخه را کامل

شرح می‌دهد. پیشنهاد سرفصل‌های ترمی و کارگاهی در بخش «راهنمای استفاده از کتاب» آمده است.

یک یادآوری

رابط کاربری نرم‌افزارهای ابری مدام بهتر می‌شود. ممکن است جای یک دکمه یا رنگ یک صفحه با تصویر کتاب کمی فرق داشته باشد؛ در این صورت به لایه‌ی «مفهوم» و «زیر کاپوت» تکیه کنید که تغییر نمی‌کنند. پایگاه دانش سامانه و ویدیوهای راهنمای هر بخش همیشه با آخرین نسخه‌ی رابط هماهنگ‌اند.

امیدواریم این کتاب برای شما همان چیزی باشد که برای ما در آغاز راه کم بود: یک همراه کامل، از اولین نقشه تا حرفه‌ای‌ترین پرسش مکانی.

راهنمای استفاده از کتاب

سطح هر فصل

کنار عنوان هر فصل سه دایره می‌بینید. یک دایره‌ی رنگی یعنی **مبتدی** (بدون هیچ پیش‌زمینه)، دو دایره یعنی **متوسط** (فصل‌های پیش‌نیاز را خوانده‌اید) و سه دایره یعنی **پیشرفته** (برای متخصص و برنامه‌نویس). لازم نیست کتاب را از اول تا آخر بخوانید؛ مسیر مطالعه‌ی خودتان را از جدول زیر انتخاب کنید.

مسیرهای مطالعه

شما کیستید	فصل‌های اصلی	فصل‌های تکمیلی
تازه‌کار، می‌خواهم GIS یاد بگیرم	۱ تا ۹، ۲۵ تا ۲۷، ۴۱ تا ۴۴	۱۰ تا ۱۳، ۳۱ و ۳۵
دانشجوی GIS یا جغرافیا	همه‌ی بخش‌های ۱ تا ۶	۴۱ تا ۴۶
کارشناس شهرداری و برنامه‌ریزی شهری	۱ تا ۹، ۱۵ تا ۱۹، ۲۵ تا ۳۰	۴۱ تا ۴۵
کارشناس کشاورزی، منابع طبیعی و آب	۱ تا ۸، ۳۱ تا ۳۸	۱۵ تا ۱۸، ۲۳ و ۳۹
نقشه‌بردار	۱ تا ۷، ۴۷ تا ۴۹	۱۵ تا ۱۸ و ۳۸
توسعه‌دهنده‌ی نرم‌افزار و وب	۴، ۶، ۱۰ تا ۲۴، ۴۵ و ۴۶	۵۰ تا ۵۳
مدیر داده و پایگاه داده	۴ تا ۷، ۱۴، ۲۰ و ۲۲، ۵۰ تا ۵۲	۲۴
مدرس و استاد	کل کتاب، به‌ویژه کادرهای «برای مدرس» و فصل ۵۴	—

کادرهای کتاب

نکته

نکته: راهی کوتاه‌تر، ترفندی کاربردی یا توضیحی که فهم موضوع را آسان‌تر می‌کند.

هشدار

هشدار: کاری که اگر اشتباه انجام شود داده را خراب یا نتیجه را غلط می‌کند.

بدانید

بدانید: پیش‌زمینه‌ی نظری یا تاریخی که برای انجام کار لازم نیست، اما برای «فهمیدن» آن لازم است.

سطح حرفه‌ای

سطح حرفه‌ای: جزئیات فنی برای خواننده‌ی پیشرفته؛ در خواندن اول می‌توانید از آن بگذرید.

در دنیای واقعی

در دنیای واقعی: نمونه‌ای از کاربرد موضوع در یک پروژه یا سازمان واقعی.

خودتان امتحان کنید

خودتان امتحان کنید: کار کوتاهی که همان‌جا در سامانه انجام می‌دهید تا مطلب تثبیت شود.

برای مدرس

برای مدرس: زمان پیشنهادی کلاس، نکته‌ی تدریس و تمرین کلاسی.

قراردادهای نگارشی

- نام منوها، دکمه‌ها و زبانه‌های رابط داخل گیومه می‌آید: «اجرای SQL»، «ورود فایل».
- مسیر رسیدن به یک صفحه با علامت ← نوشته می‌شود: منوی کناری ← داده ← جدول‌ها.
- کد، نام جدول و ستون با قلم ثابت‌عرض و چپ‌به‌راست نوشته می‌شود: `schools`، `ST_Buffer`.
- کلیدواژه‌های SQL با حروف بزرگ (`SELECT`، `WHERE`) و نام جدول و ستون با حروف کوچک نوشته شده است؛ SQL به بزرگی و کوچکی کلیدواژه‌ها حساس نیست و این فقط برای

خوانایی است.

- زیر بسیاری از کدها جدول کوچکی با قلم ثابت‌عرض آمده است: **نتیجه‌ی واقعی اجرای همان کد روی داده‌ی نمونه.** اگر نتیجه بیش از چند سطر باشد، فقط سطرهای اول و تعداد کل نشان داده شده است.
- اصطلاح انگلیسی بار اول کنار معادل فارسی آمده است و فهرست کامل آن‌ها در واژه‌نامه‌ی پایان کتاب است.

داده‌ی نمونه

همه‌ی مثال‌ها روی یک پایگاه داده با نام `tehran_gis` اجرا شده‌اند. این پایگاه شش جدول دارد:

جدول	نوع هندسه	تعداد	ستون‌های مهم
`schools`	نقطه	۱۸۰	`name`، `level`، `students` (مقطع)
`metro_stations`	نقطه	۸۰	`name`، `line`
`parks`	چندضلعی	۳۵	`name`، `area_ha`
`neighborhoods`	چندضلعی	۶	`name`، `population`
`health_centers`	نقطه	۴۰	`name`، `type`، `beds`
`parcels`	چندضلعی	۱۲	`id`، `name` (قطعه‌های کشاورزی)

همه‌ی جدول‌ها در سیستم مختصات جغرافیایی WGS84 (کد ۴۳۲۶) ذخیره شده‌اند. این داده‌ها برای آموزش ساخته شده‌اند و جای واقعی مدرسه یا ایستگاهی را نشان نمی‌دهند. فصل ۷ نشان می‌دهد چگونه این فایل‌ها را وارد پایگاه خود کنید.

پیشنهاد سرفصل تدریس

درس ترمی «سامانه‌ی اطلاعات مکانی تحت وب» (۱۶ جلسه‌ی سه‌ساعته):

جلسه	موضوع	فصل‌ها
۱	GIS چیست؛ داده‌ی مکانی	۱ و ۲
۲	سیستم مختصات و تصویر نقشه	۳
۳	پایگاه داده‌ی مکانی؛ ساخت پایگاه و اتصال QGIS	۴ تا ۶
۴	ورود، مرور و ویرایش داده	۷ تا ۹
۵	SQL پایه	۱۰ و ۱۱
۶	تجمیع و پیوند جدول‌ها	۱۲ و ۱۳
۷	هندسه و اندازه‌گیری	۱۵ و ۱۶
۸	روابط مکانی و اتصال مکانی	۱۷
۹	پردازش هندسه و نزدیک‌ترین همسایه	۱۸ و ۱۹
۱۰	آزمون میان‌ترم؛ شاخص و کارایی	۲۰
۱۱	ابزارهای تحلیل؛ تحلیل شبکه	۲۵ تا ۲۸
۱۲	مبانی سنجش از دور	۳۱ و ۳۲
۱۳	تصویر ماهواره‌ای و شاخص‌های طیفی	۳۴ و ۳۵
۱۴	آمار هر قطعه و پایش زمانی	۳۶ و ۳۷
۱۵	کارتوگرافی و انتشار	۴۱ تا ۴۵
۱۶	ارائه‌ی پروژه‌ی پایانی	۵۵

کارگاه فشرده‌ی «SQL مکانی» (۸ جلسه): فصل‌های ۴، ۱۰ تا ۱۳، ۱۵ تا ۲۰ و یک پروژه از فصل ۵۵.

کارگاه «سنجش از دور کاربردی» (۶ جلسه): فصل‌های ۳، ۳۱ تا ۳۸.

بخش ۱

مبانی علم اطلاعات مکانی

پیش از هر کلیک و هر خط کد، باید بدانیم «مکان» چگونه به عدد تبدیل می‌شود. این بخش زبان مشترک همه‌ی فصل‌های بعد را می‌سازد: داده‌ی مکانی چیست، زمین چگونه روی صفحه می‌آید و چرا پایگاه داده‌ی مکانی در فضای ابری جای فایل‌های پراکنده را گرفته است.

۱ سامانه‌ی اطلاعات مکانی چیست

۲ داده‌ی مکانی؛ مدل برداری و رستری

۳ شکل زمین، سیستم مختصات و تصویر نقشه

۴ پایگاه داده‌ی مکانی و معماری WebGIS در ابر

فصل

۱

سامانه‌ی اطلاعات مکانی چیست

سطح: **مبتدی** | زمان مطالعه: **۴۵ دقیقه** | پیش‌نیاز: **ندارد**

در این فصل یاد می‌گیرید

- تعریف سامانه‌ی اطلاعات مکانی و پنج جزء آن را بیان کنید.
- پنج پرسش اصلی که GIS به آن‌ها پاسخ می‌دهد را با مثال توضیح دهید.
- مفهوم «لایه» و قانون اول جغرافیا را شرح دهید.
- جایگاه پایگاه داده مکانی ابری را در زنجیره‌ی کار GIS بشناسید و بخش‌های اصلی آن را نام ببرید.

سرفصل‌ها

۱-۱ مکان در همه‌چیز هست

۱-۲ تعریف سامانه‌ی اطلاعات مکانی

۱-۳ پنج پرسشی که GIS به آن‌ها پاسخ می‌دهد

۱-۴ نگاهی کوتاه به تاریخ

۱-۵ مفهوم لایه

۱-۶ کاربردهای GIS

۱-۷ ابری و پایگاه داده مکانی ابری GIS

۱-۸ جریان کار GIS در این کتاب

۱-۱ مکان در همه‌چیز هست

در تابستان ۱۸۵۴ وبا در محله‌ی سوهوی لندن ظرف ده روز بیش از پانصد نفر را کشت. نظریه‌ی رایج آن زمان این بود که بیماری از «هوای بد» منتقل می‌شود. پزشکی به نام جان اسنو کار ساده‌ای کرد: محل خانه‌ی هر قربانی را با یک خط کوتاه روی نقشه‌ی محله علامت زد و محل تلمبه‌های آب عمومی را هم روی همان نقشه گذاشت. الگو بلافاصله پیدا شد: مرگ‌ها دور یک تلمبه در خیابان برود جمع شده بودند. دسته‌ی تلمبه را برداشتند و همه‌گیری فروکش کرد.

اسنو رایانه نداشت، اما همان کاری را کرد که امروز هر سامانه‌ی اطلاعات مکانی انجام می‌دهد: **داده‌ای را که مکان دارد روی هم گذاشت تا الگویی دیده شود که در جدول دیده نمی‌شد.** در فهرست نام و نشانی قربانیان هیچ چیز غیرعادی نبود؛ روی نقشه، پاسخ فریاد می‌زد.

تخمین زده می‌شود بخش بزرگی از داده‌هایی که سازمان‌ها نگه می‌دارند به نوعی مؤلفه‌ی مکانی دارند: نشانی مشتری، محل پروژه، مسیر ناوگان، مرز ملک، محل تیر برق، مزرعه‌ی تحت پوشش بیمه. هر جا این مؤلفه وجود دارد، پرسش «کجا؟» هم وجود دارد.

بدانید

قانون اول جغرافیا. والدو تابلر، جغرافی‌دان آمریکایی، در سال ۱۹۷۰ جمله‌ای نوشت که به «قانون اول جغرافیا» مشهور شد: «همه چیز به همه چیز مربوط است، اما چیزهای نزدیک بیشتر از چیزهای دور به هم مربوط‌اند.» قیمت خانه‌های یک محله به هم نزدیک است، آلودگی هوای دو ایستگاه مجاور شبیه است و بیماری واگیر از نزدیک‌ترین‌ها پخش می‌شود. بسیاری از روش‌های این کتاب، از نزدیک‌ترین همسایه تا درون‌یابی و آمار فضایی، روی همین قانون بنا شده‌اند.

۱-۲ تعریف سامانه‌ی اطلاعات مکانی

سامانه‌ی اطلاعات مکانی (Geographic Information System یا به اختصار **GIS**) مجموعه‌ای از داده، نرم‌افزار، سخت‌افزار، روش و انسان است که برای **گردآوری، ذخیره، مدیریت، تحلیل و نمایش** داده‌هایی به کار می‌رود که به موقعیتی روی زمین وابسته‌اند.

در این تعریف دو نکته مهم است. اول اینکه GIS فقط «نرم‌افزار نقشه» نیست؛ نقشه یکی از خروجی‌های آن است. قلب GIS **تحلیل** است: پیدا کردن رابطه‌ها و الگوهایی که بدون در نظر گرفتن مکان دیده نمی‌شوند. دوم اینکه GIS یک «سامانه» است و فقط وقتی کار می‌کند که همه‌ی اجزایش حاضر باشند.

پنج جزء هر سامانه‌ی اطلاعات مکانی

پنج جزء سامانه‌ی اطلاعات مکانی. نبود هر کدام سامانه را ناقص می‌کند.

داده مهم‌ترین و گران‌ترین جزء است. داده‌ی مکانی سه بُعد دارد: **مکان** (کجاست؟)، **ویژگی** (چیست و چه مشخصاتی دارد؟) و **زمان** (کی؟). یک مدرسه مکانی دارد (طول و عرض جغرافیایی)، ویژگی‌هایی دارد (نام، مقطع، تعداد دانش‌آموز) و در زمانی ثبت شده است.

نرم‌افزار امکان ذخیره، پرسش، تحلیل و نمایش را فراهم می‌کند. در این کتاب نرم‌افزار اصلی ما پایگاه داده مکانی ابری است که روی PostgreSQL و PostGIS ساخته شده است، و در کنارش از QGIS (نرم‌افزار رومیزی متن‌باز) هم استفاده می‌کنیم.

سخت‌افزار از سرورهایی که داده را نگه می‌دارند تا گیرنده‌ی GNSS نقشه‌بردار و ماهواره‌ای که از زمین عکس می‌گیرد. در رایانش ابری، بخش بزرگی از این جزء از دوش شما برداشته شده است.

روش مجموعه‌ی استانداردها، الگوریتم‌ها و فرایندهای کاری است: کدام سیستم مختصات؟ کدام روش طبقه‌بندی؟ داده چگونه کنترل کیفیت شود؟

انسان هم کارشناسی است که سامانه را می‌سازد و هم مدیری که با خروجی آن تصمیم می‌گیرد. بهترین داده و نرم‌افزار، بدون کسی که پرسش درست را بپرسد، بی‌فایده است.

۱-۳ پنج پرسشی که GIS به آن‌ها پاسخ می‌دهد

بیشتر کارهای GIS را می‌توان در پنج دسته پرسش جا داد. شناختن این دسته‌ها کمک می‌کند پرسش مدیر یا کارفرما را به یک تحلیل مشخص تبدیل کنید.

نوع پرسش	شکل کلی	مثال	ابزار در این کتاب
مکان	فلان چیز کجاست؟	نزدیک‌ترین بیمارستان به این تصادف کجاست؟	یافتن عارضه، نزدیک‌ترین (فصل‌های ۸ و ۱۹)
وضعیت	در این مکان چه چیزی هست؟	این قطعه زمین در کدام محله و چه کاربری است؟	روابط مکانی (فصل ۱۷)
روند	از فلان زمان چه چیزی تغییر کرده؟	پوشش گیاهی این دشت در پنج سال چه تغییری کرده؟	سری زمانی ماهواره‌ای، تغییر بین دو نسخه (فصل‌های ۲۷ و ۳۷)
الگو	چه الگوی مکانی وجود دارد؟	تصادف‌ها در کجا متمرکزند؟	تراکم شش‌ضلعی، خوشه‌بندی (فصل‌های ۲۱ و ۲۶)
مدل‌سازی	اگر ...، چه می‌شود؟	اگر ایستگاه آتش‌نشانی تازه اینجا ساخته شود، چند خانه زیر پوشش پنج‌دقیقه‌ای می‌رود؟	حوزه‌ی دسترسی، پوشش مراکز (فصل ۲۸)

در دنیای واقعی

مکان‌یابی شعبه. بانکی می‌خواهد شعبه‌ی تازه باز کند. پرسش «کجا شعبه بزنیم؟» در GIS به چند پرسش جزئی شکسته می‌شود: جمعیت هر محله چقدر است (وضعیت)؟ شعبه‌های فعلی ما و رقبا کجا هستند (مکان)؟ کدام محله‌ها بیشتر از ۱۵۰۰ متر از نزدیک‌ترین شعبه فاصله دارند (الگو)؟ اگر شعبه را در نقطه‌ی الف بزنیم چند نفر در فاصله‌ی ده دقیقه پیاده‌روی آن قرار می‌گیرند (مدل‌سازی)؟ همه‌ی این پرسش‌ها را تا پایان بخش پنجم این کتاب می‌توانید پاسخ دهید.

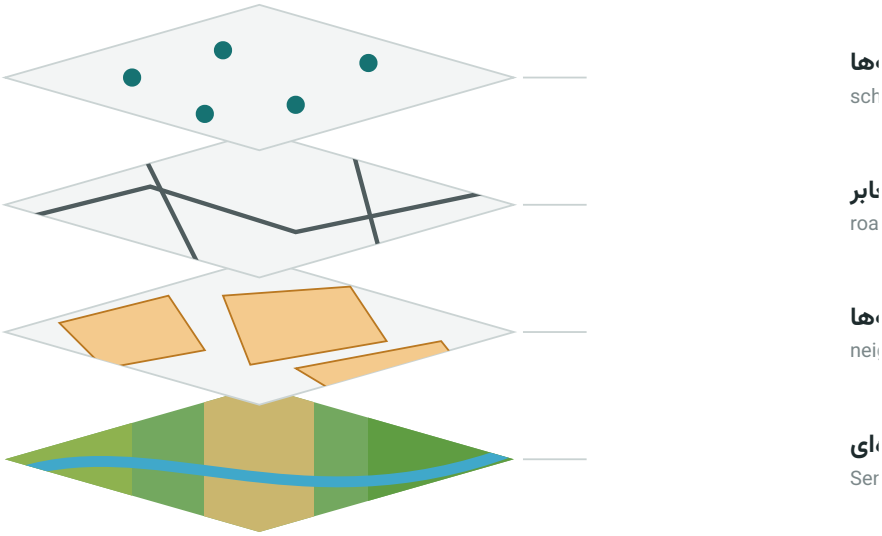
۱-۴ نگاهی کوتاه به تاریخ

- **۱۸۵۴:** نقشه‌ی وبای جان اسنو، نمونه‌ی کلاسیک تحلیل مکانی.
- **دهه‌ی ۱۹۶۰:** راجر تاملینسون در کانادا نخستین سامانه‌ی رایانه‌ای GIS را برای سیاهه‌ی اراضی کشور ساخت. اصطلاح «سامانه‌ی اطلاعات جغرافیایی» از همین پروژه آمد.
- **۱۹۷۲:** پرتاب نخستین ماهواره‌ی Landsat و آغاز سنجش از دور غیرنظامی.
- **دهه‌ی ۱۹۸۰ و ۱۹۹۰:** نرم‌افزارهای تجاری رومیزی و همه‌گیر شدن GIS در سازمان‌ها؛ تشکیل کنسرسیوم مکانی باز (OGC) و استانداردهایی که امروز همه از آن‌ها استفاده می‌کنند.
- **۲۰۰۱:** نخستین نسخه‌ی PostGIS؛ داده‌ی مکانی به پایگاه داده‌ی رابطه‌ای متن‌باز آمد.
- **۲۰۰۲ تا ۲۰۰۴:** آغاز QGIS و OpenStreetMap، نقشه‌ی آزاد جهان که با مشارکت داوطلبان ساخته می‌شود.
- **۲۰۰۸:** آرشیو Landsat رایگان شد.
- **۲۰۱۵:** پرتاب Sentinel-2A اروپا؛ تصویر رایگان با تفکیک ده متر و تکرار چندروزه.
- **دهه‌ی ۲۰۲۰:** GIS ابری؛ پایگاه داده، تحلیل و تصویر ماهواره‌ای همه در مرورگر.

۱-۵ مفهوم لایه

نقشه‌ی کاغذی همه چیز را با هم نشان می‌دهد. GIS داده را در **لایه‌ها** نگه می‌دارد: هر لایه یک نوع پدیده است و همه‌ی لایه‌ها روی یک سیستم مختصات مشترک قرار می‌گیرند تا بتوان آن‌ها را روی هم گذاشت. لایه‌ی مدرسه‌ها، لایه‌ی معابر، لایه‌ی محله‌ها و لایه‌ی تصویر ماهواره‌ای جدا

از هم ذخیره می‌شوند، اما چون همه می‌دانند «کجا» هستند، می‌توانیم بپرسیم «کدام مدرسه‌ها در محله‌ی ونک‌اند» یا «پوشش گیاهی در حریم ۵۰۰ متری رودخانه چقدر است».



مفهوم لایه: هر نوع پدیده در یک لایه؛ همه روی یک دستگاه مختصات. سه لایه‌ی بالا برداری و لایه‌ی پایین رستری است.

در پایگاه داده‌ی مکانی، **هر لایه یک جدول است.** هر سطر جدول یک **عارضه** (feature) است، مثلاً یک مدرسه، و هر ستون یک ویژگی. یکی از ستون‌ها، که معمولاً `geom` نام دارد، شکل و مکان عارضه را نگه می‌دارد. این ایده‌ی ساده، یعنی «لایه = جدول»، پایه‌ی همه‌ی فصل‌های بعدی است و در فصل ۴ مفصل به آن برمی‌گردیم.

۱-۶ کاربردهای GIS

حوزه	نمونه‌ی کاربرد
شهرسازی و مدیریت شهری	طرح تفصیلی، کاربری زمین، مکان‌یابی خدمات، عوارض نوسازی، مدیریت پسماند
کشاورزی	پایش سلامت مزرعه با NDVI، برآورد سطح زیر کشت، بیمه‌ی محصول، کشاورزی دقیق
آب و محیط زیست	حوضه‌ی آبریز، پایش دریاچه و تالاب، خشکسالی، آلودگی، مناطق حفاظت‌شده
منابع طبیعی	پایش جنگل و مرتع، آتش‌سوزی و برآورد سطح سوخته، فرسایش
بهداشت و درمان	دسترسی به مراکز درمانی، پایش همه‌گیری، مکان‌یابی پایگاه اورژانس
امداد و بحران	سیل، زلزله، مسیر امدادرسانی، برآورد خسارت با تصویر ماهواره‌ای
حمل‌ونقل و لجستیک	مسیریابی ناوگان، حوزه‌ی توزیع، تحلیل تصادف
زیرساخت و تأسیسات	شبکه‌ی برق، آب، گاز و مخابرات؛ مدیریت دارایی؛ پوشش آنتن
املاک، بانک و بیمه	ارزش‌گذاری ملک، مکان‌یابی شعبه، ریسک بیمه
نقشه‌برداری و عمران	برداشت میدانی، محاسبه‌ی حجم خاک‌برداری، پیاده‌کردن نقشه

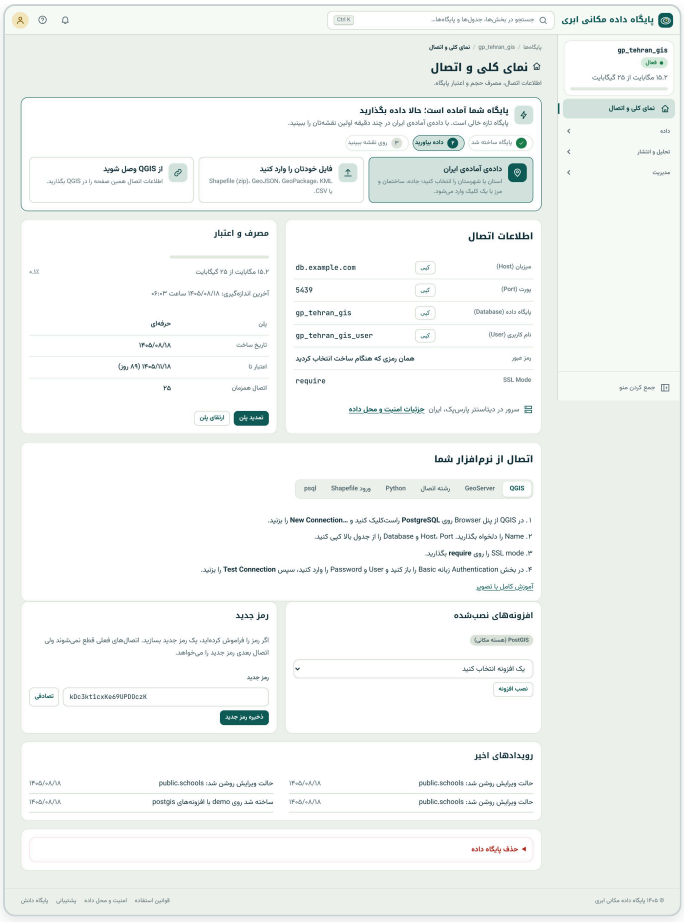
۱-۷ ابری و پایگاه داده مکانی ابری GIS

در شیوه‌ی سنتی، هر کارشناس داده را در چند فایل روی رایانه‌ی خودش نگه می‌داشت: یک Shapefile برای مدرسه‌ها، یک فایل اکسل برای جمعیت و پوشه‌ای از تصویرهای ماهواره‌ای. همکار دیگر نسخه‌ی دیگری از همان فایل‌ها داشت و هیچ‌کس نمی‌دانست کدام نسخه تازه‌تر است. برای ساختن یک سرور پایگاه داده‌ی مکانی هم باید سرور تهیه، سیستم‌عامل و

PostgreSQL نصب، PostGIS پیکربندی، امنیت تنظیم و بکاپ‌گیری راه‌اندازی می‌شد؛ کاری که از عهده‌ی بیشتر کارشناسان GIS خارج است.

پایگاه داده مکانی ابری این زنجیره را ساده می‌کند. ثبت‌نام می‌کنید، نامی برای پایگاه داده انتخاب می‌کنید و چند ثانیه بعد یک پایگاه داده‌ی PostgreSQL با PostGIS در اختیار دارید که:

- از هر جایی با مرورگر در دسترس است و نیازی به نصب ندارد؛
- از GeoServer، ArcGIS Pro، QGIS، پایتون و هر نرم‌افزار دیگری که PostgreSQL را می‌شناسد هم قابل اتصال است؛
- بکاپ خودکار، کاربران و نقش‌ها، و پایش مصرف دارد؛
- ابزارهای تحلیل، تصویر ماهواره‌ای، نقشه‌سازی، انتشار وب و برداشت میدانی را کنار داده‌ی شما گذاشته است.



صفحه‌ی اول یک پایگاه داده در پایگاه داده مکانی ابری: اطلاعات اتصال، مصرف و راهنمای قدم بعد. منوی کناری سمت راست همه‌ی بخش‌های سامانه را در چهار گروه نشان می‌دهد.

منوی کناری هر پایگاه داده چهار گروه دارد که ساختار این کتاب هم تا حد زیادی از آن پیروی می‌کند:

گروه منو	بخش‌ها	فصل‌های کتاب
نمای کلی و اتصال	اطلاعات اتصال، مصرف، اعتبار	۵ و ۶
داده	جدول‌ها، یافتن عارضه، اجرای SQL، ورود فایل، داده‌ی آماده‌ی ایران، تصویر ماهواره‌ای، سلامت داده	۷ تا ۲۴، ۳۰ تا ۴۰
تحلیل و انتشار	ابزارهای تحلیل، نقشه‌های چندلایه، انتشار لایه، WMS/WFS، برداشت میدانی	۲۵ تا ۲۹، ۴۱ تا ۴۹
مدیریت	تیم، بکاپ، مصرف و کارایی، افزونه‌ها، امکانات سازمانی	۵۰ تا ۵۲

نکته

آیا GIS ابری جای نرم‌افزار رومیزی را می‌گیرد؟ نه کاملاً، و لازم هم نیست. پایگاه داده مکانی ابری «منبع واحد حقیقت» است: داده یک جا نگه داشته می‌شود و همه به همان وصل می‌شوند. هر کسی می‌تواند با ابزار دلخواهش، از مرورگر تا QGIS، روی همان داده کار کند. در این کتاب هر جا QGIS کاری را بهتر انجام دهد، آن را هم آموزش می‌دهیم.

۱-۸ جریان کار GIS در این کتاب

تقریباً هر پروژه‌ی GIS این مراحل را طی می‌کند و هر مرحله در یک بخش از کتاب آمده است:

۱. **پرسش:** مسئله را به پرسش مکانی تبدیل کنید (همین فصل).

۲. **داده:** داده را گردآوری کنید: از فایل، از داده‌ی آماده، از برداشت میدانی یا از تصویر ماهواره‌ای (بخش‌های ۲، ۵، ۶ و ۸).

۳. **ذخیره و آماده‌سازی:** داده را در پایگاه داده وارد، سیستم مختصات را یکدست و هندسه‌ها را اصلاح کنید (بخش ۲ و فصل ۲۲).

۴. **تحلیل:** با SQL مکانی یا ابزارهای آماده به پرسش پاسخ دهید (بخش‌های ۳ تا ۶).

۵. **نمایش و انتشار:** نتیجه را به نقشه، نمودار، گزارش یا سرویس وب تبدیل کنید (بخش ۷).

۶ **نگه‌داری:** داده را به‌روز، امن و پشتیبان‌دار نگه دارید (بخش ۹).

خلاصه‌ی فصل

- GIS سامانه‌ای از داده، نرم‌افزار، سخت‌افزار، روش و انسان برای پاسخ به پرسش «کجا؟» است؛ نقشه یکی از خروجی‌های آن است، نه همه‌ی آن.
- داده‌ی مکانی سه بُعد دارد: مکان، ویژگی و زمان.
- پنج نوع پرسش GIS: مکان، وضعیت، روند، الگو و مدل‌سازی.
- داده در لایه‌ها نگه داشته می‌شود و در پایگاه داده‌ی مکانی هر لایه یک جدول و هر عارضه یک سطر است.
- پایگاه داده مکانی ابری یک PostgreSQL/PostGIS آماده است که داده، تحلیل، تصویر ماهواره‌ای و انتشار را یک‌جا جمع کرده است.

برای مدرس

زمان پیشنهادی: یک جلسه‌ی ۹۰ دقیقه‌ای. داستان جان اسنو را با نقشه‌ی اصلی او شروع کنید و از دانشجوها بخواهید پیش از دیدن نقشه حدس بزنند علت چیست. در نیمه‌ی دوم، هر گروه سه‌نفره یک مسئله از رشته‌ی خودش را به پنج نوع پرسش GIS بشکنند. تمرین ۴ را به‌عنوان تکلیف بدهید تا جلسه‌ی بعد هر دانشجو حساب کاربری داشته باشد.

تمرین‌ها

۱. سطح ۱ پنج جزء GIS را نام ببرید و برای هر کدام یک مثال از محیط کار یا تحصیل خودتان بزنید.

۲. سطح ۱ سه داده‌ی روزمره (مثلاً قبض برق) نام ببرید که مؤلفه‌ی مکانی دارند. مؤلفه‌ی مکانی هر کدام چیست؟

۳. سطح ۱ برای هر یک از پنج نوع پرسش GIS یک پرسش در شهر خودتان بنویسید.

۴. سطح ۱ در پایگاه داده مکانی ابری حساب کاربری بسازید (فصل ۵) و منوی کناری را مرور کنید. کدام بخش‌ها برای رشته‌ی شما مفیدترند؟

۵. سطح ۲ قانون اول جغرافیا را با یک مثال توضیح دهید. آیا موردی می‌شناسید که این قانون در آن صدق نکند؟

۶. سطح ۲ مسئله‌ی «جای مناسب برای یک داروخانه‌ی تازه در محله‌ی شما» را به مراحل جریان کار GIS بشکنید: چه داده‌هایی لازم است و از کجا؟ چه تحلیلی؟ خروجی چه باشد؟

۷. سطح ۳ درباره‌ی یکی از رویدادهای تاریخ GIS که در این فصل آمد تحقیق کنید و در یک صفحه بنویسید چه مسئله‌ای را حل کرد و چه تأثیری بر امروز گذاشت.

فصل

۲

داده‌ی مکانی؛ مدل برداری و رستری

سطح: **مبتدی** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل ۱**

در این فصل یاد می‌گیرید

- عارضه، هندسه و ویژگی را از هم تشخیص دهید.
- مدل برداری و رستری را مقایسه کنید و برای هر مسئله مدل مناسب را برگزینید.
- هفت نوع هندسه‌ی استاندارد را بشناسید و با WKT بخوانید.
- سطح اندازه‌گیری هر ویژگی (اسمی، ترتیبی، فاصله‌ای، نسبی) را تعیین کنید.
- قالب‌های رایج فایل مکانی و کاربرد هر کدام را بدانید.

سرفصل‌ها

۲-۱ جهان واقعی و مدل آن

۲-۲ مدل برداری

۲-۳ مدل رستری

۲-۴ کدام مدل، برای کدام مسئله

۲-۵ ویژگی‌ها و سطح اندازه‌گیری

۲-۶ قالب‌های فایل مکانی

۲-۷ مقیاس، دقت و صحت

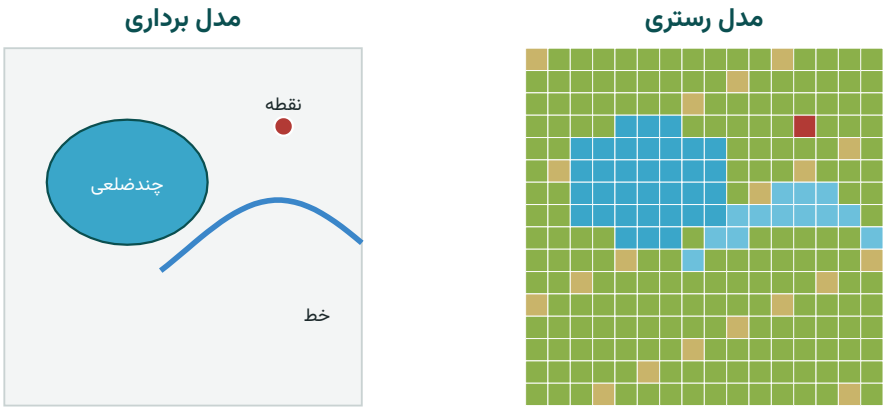
۲-۸ کیفیت داده و فراداده

۲-۱ جهان واقعی و مدل آن

جهان واقعی پیوسته و بی‌نهایت پیچیده است. خیابان عرض دارد، لبه‌ی جدول دارد، درخت‌های کنارش هر روز رشد می‌کنند. هیچ رایانه‌ای نمی‌تواند همه‌ی این جزئیات را ذخیره کند؛ پس هر GIS جهان را **ساده می‌کند** و به یک **مدل داده** تبدیل می‌کند. اینکه خیابان را یک خط ببینیم یا یک چندضلعی با عرض واقعی، به هدف ما بستگی دارد: برای مسیریابی خط کافی است، برای محاسبه‌ی مساحت آسفالت چندضلعی لازم است.

دو مدل اصلی برای نمایش جهان در GIS وجود دارد:

- **مدل برداری** (vector): جهان از **اشیای گسسته** ساخته شده است که مرز مشخص دارند؛ مثل چاه، جاده یا قطعه‌ی زمین.
- **مدل رستری** (raster): جهان یک **میدان پیوسته** است که در یک شبکه‌ی منظم از سلول‌ها نمونه‌برداری شده است؛ مثل ارتفاع، دما یا بازتاب نور در تصویر ماهواره‌ای.



مرز دقیق، هندسه با مختصات رأس‌ها — شبکه‌ی سلول‌ها، هر سلول یک مقدار

یک صحنه در دو مدل. در مدل برداری دریاچه یک چندضلعی، رود یک خط و چاه یک نقطه است؛ در مدل رستری همه‌چیز سلول‌هایی با یک مقدار است.

۲-۲ مدل برداری

در مدل برداری هر پدیده یک **عارضه** (feature) است. عارضه دو بخش دارد:

1. **هندسه** (geometry): شکل و مکان، که با مختصات یک یا چند نقطه بیان می‌شود.
2. **ویژگی‌ها** (attributes): اطلاعات توصیفی مثل نام، نوع، تاریخ یا جمعیت.

در پایگاه داده‌ی مکانی، عارضه یک سطر جدول است؛ هندسه در یک ستون از نوع `geometry` و هر ویژگی در یک ستون معمولی ذخیره می‌شود. مثلاً سه سطر اول جدول مدرسه‌ها در داده‌ی نمونه این است:

```sql
SELECT school_id, name, level, students, ST_AsText(geom) AS geom
FROM schools
ORDER BY school_id
LIMIT 3;
```

school_id	name	level	students	geom
1	مدرسه 1	متوسطه اول	411	POINT(51.532393 35.655237)
2	مدرسه 2	متوسطه اول	795	POINT(51.456527 35.702091)
3	مدرسه 3	متوسطه اول	799	POINT(51.486526 35.681792)

ستون آخر هندسه را به صورت متن نشان می‌دهد: `POINT(طول عرض)`. فعلاً نگران کد بالا نباشید؛ در بخش سوم SQL را از صفر یاد می‌گیرید. نکته‌ی مهم این است که **هندسه هم یک ستون جدول است**، کنار نام و تعداد دانش‌آموز.

انواع هندسه

استاندارد «ویژگی‌های ساده» (Simple Features) که کنسرسیوم مکانی باز (OGC) و سازمان استاندارد ISO تدوین کرده‌اند، هفت نوع هندسه‌ی پایه تعریف می‌کند که PostGIS و تقریباً همه‌ی نرم‌افزارهای GIS از آن پیروی می‌کنند:

Point
نقطه

LineString
خط

Polygon
چندضلعی (با حفره)

MultiPolygon
چندبخشی

GeometryCollection
مجموعه‌ی ناهمگن

انواع هندسه در استاندارد Simple Features. انواع چندبخشی MultiPoint و MultiLineString هم مانند MultiPolygonاند.

نوع	تعریف	نمونه‌ی پدیده	نمایش متنی (WKT)
`POINT`	یک جفت مختصات	چاه، مدرسه، تیر برق	`POINT(51.39 35.70)`
`LINESTRING`	دنباله‌ای از دست‌کم دو نقطه	جاده، رود، لوله	`LINESTRING(0 0, 1 1, 2 1)`
`POLYGON`	یک حلقه‌ی بیرونی بسته و صفر یا چند حفره	قطعه‌ی زمین، دریاچه، محله	`POLYGON((0 0, 4 0, 4 4, 0 4, 0 0))`
`MULTIPOINT`	چند نقطه در یک عارضه	شعبه‌های یک بانک	`MULTIPOINT((0 0), (1 2))`
`MULTILINESTRING`	چند خط در یک عارضه	بزرگراهی با دو باند جدا	`MULTILINESTRING((0 0, 1 1), (2 2, 3 3))`
`MULTIPOLYGON`	چند چندضلعی در یک عارضه	کشوری با جزیره، مثل ایران و جزیره‌های خلیج فارس	`MULTIPOLYGON(((…)), ((…)))`
`GEOMETRYCOLLECTION`	مجموعه‌ای ناهمگن	نتیجه‌ی بعضی عملیات هندسی	`GEOMETRYCOLLECTION(POINT(…), LINESTRING(…))`

چند قاعده‌ی مهم دارد که در عمل زیاد به آن برمی‌خوریم:

- حلقه‌ی چندضلعی باید **بسته** باشد: نقطه‌ی آخر دقیقاً همان نقطه‌ی اول است.
- حلقه‌ی چندضلعی نباید **خودش را قطع کند**؛ چندضلعی «پاپیونی» نامعتبر است و بسیاری از محاسبات را خراب می‌کند. فصل ۲۲ روش یافتن و اصلاح هندسه‌ی نامعتبر را آموزش می‌دهد.

- در WKT ابتدا **طول** (x، شرقی‌غربی) و سپس **عرض** (y، شمالی‌جنوبی) می‌آید. این ترتیب برعکس عادت روزمره است که «عرض، طول» می‌گوییم و یکی از رایج‌ترین منشأهای خطاست.

> **هشدار**
>
> **ترتیب طول و عرض.** تهران در حدود طول ۵۱٫۴ و عرض ۳۵٫۷ درجه است. اگر نقطه‌ای با `POINT(35.7 51.4)` بسازید، آن را در شمال روسیه می‌گذارید! هر وقت لایه‌ای «در جای عجیبی» نمایش داده شد، اول ترتیب مختصات را بررسی کنید.

هندسه با ارتفاع و اندازه

هر رأس هندسه می‌تواند علاوه بر x و y، مختصات **Z** (ارتفاع) و مقدار **M** (اندازه، مثلاً فاصله‌ی پیموده‌شده روی خط) هم داشته باشد: `POINT Z (51.39 35.70 1210)`. نقطه‌های برداشت میدانی با گیرنده‌ی GNSS معمولاً Z دارند و فصل ۴۹ از همین Z برای ساختن سطح و منحنی میزان استفاده می‌کند.

توپولوژی؛ رابطه‌ها، نه فقط شکل‌ها

دو قطعه‌ی زمین مجاور یک مرز مشترک دارند. اگر هر قطعه جداگانه رقومی شود، احتمال دارد مرز مشترک دو بار و با اندکی اختلاف کشیده شود و بین دو قطعه شکاف یا همپوشانی باریکی (sliver) ایجاد شود. **توپولوژی** مجموعه‌ی قواعدی است که رابطه‌ی هندسه‌ها را تضمین می‌کند: قطعه‌ها روی هم نیفتند، بین‌شان شکاف نباشد، خط‌های شبکه در گره‌ها به هم برسند. مدل «ویژگی‌های ساده» که بیشتر نرم‌افزارها به کار می‌برند هر عارضه را مستقل نگه می‌دارد و توپولوژی را ذخیره نمی‌کند؛ اما می‌توان قواعد آن را با SQL بررسی کرد یا از افزونه‌ی `postgis_topology` استفاده کرد (فصل ۲۲).

۲-۳ مدل رستری

در مدل رستری، محدوده‌ی مطالعه با یک **شبکه‌ی منظم** از سلول‌های هم‌اندازه پوشانده می‌شود. هر سلول، که در تصویر **پیکسل** نامیده می‌شود، یک مقدار عددی دارد: ارتفاع زمین، دمای سطح، نوع کاربری یا میزان بازتاب نور.

مفاهیم کلیدی رستر:

- **تفکیک مکانی** (resolution): اندازه‌ی ضلع هر سلول روی زمین. تصویر Sentinel-2 در باندهای مرئی تفکیک ده متر دارد، یعنی هر پیکسل مربعی ۱۰ × ۱۰ متر از زمین است. مدل ارتفاعی Copernicus تفکیک حدود ۳۰ متر دارد.
- **باند** (band): رستر می‌تواند چند لایه‌ی هم‌شبکه داشته باشد. تصویر رنگی سه باند دارد (قرمز، سبز، آبی) و تصویر Sentinel-2 سیزده باند طیفی.
- **نوع داده‌ی پیکسل:** عدد صحیح ۸ بیتی (۰ تا ۲۵۵) برای تصویر نمایشی، عدد صحیح ۱۶ بیتی برای بازتاب، عدد اعشاری برای شاخص‌هایی مثل NDVI که بین منفی یک و یک‌اند.
- **مقدار بی‌داده** (NoData): مقداری قراردادی برای سلول‌هایی که داده ندارند، مثلاً بیرون از مرز برش یا زیر ابر. فراموش کردن NoData میانگین‌ها را خراب می‌کند.
- **مرجع مکانی:** مختصات گوشه‌ی رستر، اندازه‌ی سلول و سیستم مختصات. بدون این‌ها رستر فقط یک عکس است و نمی‌داند کجای زمین است.

رستر دو نوع اصلی دارد: **پیوسته** که مقدارش به تدریج تغییر می‌کند (ارتفاع، دما، بازتاب) و **گسسته یا طبقه‌ای** که هر مقدار یک کلاس است (۱ = آب، ۲ = جنگل، ۳ = شهر). نقشه‌ی کاربری اراضی حاصل طبقه‌بندی تصویر ماهواره‌ای یک رستر گسسته است.

بدانید

اندازه‌ی رستر به سرعت بزرگ می‌شود. محدوده‌ای ۱۰۰ × ۱۰۰ کیلومتری با تفکیک ۱۰ متر، ۱۰٬۰۰۰ × ۱۰٬۰۰۰ یعنی صد میلیون پیکسل در هر باند دارد. با ۱۳ باند و دو بایت برای هر پیکسل، حدود ۲٫۶ گیگابایت می‌شود. به همین دلیل سامانه تصویر ماهواره‌ای را فقط برای محدوده‌ی مورد نیاز شما برش می‌دهد و در قالب COG (فصل ۳۴) ذخیره می‌کند.

۲-۴ کدام مدل، برای کدام مسئله

معیار	برداری	رستری
مناسب برای	اشیای گسسته با مرز مشخص	پدیده‌های پیوسته و تصویر
دقت مرز	بالا؛ به اندازه‌ی دقت مختصات	محدود به اندازه‌ی سلول
حجم داده برای محدوده‌ی بزرگ	کم	زیاد
ویژگی‌های متعدد برای هر عارضه	آسان (ستون‌های جدول)	دشوار (هر ویژگی یک باند)
تحلیل شبکه و مسیریابی	بسیار مناسب	نامناسب
تحلیل همپوشانی و جبر نقشه	ممکن، گاهی کند	سریع و ساده
نمونه	قطعه‌ی ثبتی، شبکه‌ی آب، مدرسه	تصویر ماهواره‌ای، DEM، نقشه‌ی دما

در عمل بیشتر پروژه‌ها از هر دو استفاده می‌کنند و تبدیل بین آن‌ها هم رایج است. فصل ۳۶ نمونه‌ی مهمی را نشان می‌دهد: محاسبه‌ی میانگین NDVI (رستر) برای هر قطعه‌ی کشاورزی (برداری)، که به آن **آمار ناحیه‌ای** می‌گویند.

۲-۵ ویژگی‌ها و سطح اندازه‌گیری

ستون‌های توصیفی هم نوع دارند و نوع آن‌ها تعیین می‌کند چه محاسبه‌ای مجاز و چه نمادی روی نقشه مناسب است. روان‌شناسی به نام استنلی استیونس در ۱۹۴۶ چهار **سطح اندازه‌گیری** تعریف کرد که در کارتوگرافی و آمار مکانی پایه است:

سطح	ویژگی	مثال	محاسبه‌ی مجاز	نماد مناسب روی نقشه
اسمی	فقط نام یا دسته؛ ترتیب ندارد	کاربری زمین، نوع مرکز درمانی، خط مترو	شمارش، مُد	رنگ‌های متفاوت (فام)، شکل نماد
ترتیبی	ترتیب دارد ولی فاصله‌ها معنا ندارند	مقطع تحصیلی، درجه‌ی جاده، کم/متوسط/زیاد	میانه، رتبه	روشنی رنگ، ضخامت خط
فاصله‌ای	فاصله معنا دارد، صفر قراردادی است	دما به سلسیوس، سال	میانگین، تفاضل	طیف رنگی پیوسته
نسبی	صفر واقعی دارد، نسبت معنا دارد	جمعیت، مساحت، تعداد دانش‌آموز، بارش	همه‌ی محاسبات	اندازه‌ی نماد، طیف رنگی

هشدار

میانگینِ کد معنا ندارد. اگر کاربری‌ها با عدد کدگذاری شده باشند (۱ = مسکونی، ۲ = تجاری)، میانگین ۱٫۵ هیچ معنایی ندارد. همین‌طور نمایش «جمعیت» (یک مقدار مطلق) با طیف رنگی روی محله‌هایی با مساحت متفاوت گمراه‌کننده است؛ باید آن را به **تراکم** (جمعیت بر کیلومتر مربع) تبدیل کرد. فصل ۴۱ این قواعد را مفصل‌تر بررسی می‌کند.

۲-۶ قالب‌های فایل مکانی

داده‌ی مکانی پیش از ورود به پایگاه داده معمولاً در یکی از این قالب‌هاست. سامانه بیشتر آن‌ها را مستقیم می‌پذیرد (فصل ۷):

قالب	نوع	توضیح
Shapefile (`.shp` و فایل‌های همراه)	برداری	قدیمی‌ترین و رایج‌ترین؛ دست‌کم سه فایل (`.shp`، `.shx`، `.dbf`) و معمولاً `.prj`. محدودیت ده‌حرفی نام ستون، حداکثر دو گیگابایت و مشکل رایج کدگذاری متن فارسی. برای ورود باید همه‌ی فایل‌ها را zip کنید.
GeoPackage (`.gpkg`)	برداری و رستری	جانشین مدرن Shapefile؛ یک فایل واحد که در واقع یک پایگاه داده‌ی SQLite است؛ چند لایه، نام ستون بلند و UTF-8.
GeoJSON (`.geojson`)	برداری	متنی و خوانا، استاندارد وب؛ همیشه در WGS84. برای حجم‌های بزرگ کند است.
KML / KMZ	برداری	قالب Google Earth؛ همراه سبک نمایش.
CSV با ستون طول و عرض یا WKT	برداری (نقطه)	ساده‌ترین راه برای داده‌ی نقطه‌ای از اکسل.
DXF / DWG	برداری	قالب نقشه‌کشی (CAD)؛ رایج در نقشه‌برداری و عمران.
GeoTIFF / COG	رستری	تصویر TIFF با مرجع مکانی؛ COG نسخه‌ی بهینه‌شده برای خواندن تکه‌ای از اینترنت است.
پایگاه داده‌ی PostGIS	برداری و رستری	جایی که داده‌ها پس از ورود زندگی می‌کنند: چندکاربره، پرسش‌پذیر با SQL و امن.

۲-۷ مقیاس، دقت و صحت

«مقیاس» در نقشه‌ی کاغذی نسبت فاصله روی نقشه به فاصله روی زمین است: در مقیاس ۱:۲۵,۰۰۰، یک سانتی‌متر نقشه ۲۵۰ متر زمین است. داده‌ی رقومی مقیاس ثابت ندارد و می‌توان آن را تا هر اندازه بزرگ کرد، اما **دقت آن ثابت می‌ماند.** داده‌ای که از نقشه‌ی ۱:۲۵۰,۰۰۰ رقومی شده، با بزرگ‌نمایی دقیق‌تر نمی‌شود؛ فقط خطایش بیشتر دیده می‌شود.

دو مفهوم را نباید با هم اشتباه گرفت:

- **دقت** (precision): میزان جزئیات یا تکرارپذیری؛ مثلاً تعداد رقم اعشار مختصات.
- **صحت** (accuracy): نزدیکی به مقدار واقعی.

مختصاتی با هشت رقم اعشار (دقت میلی‌متری) که از GPS گوشی با خطای پنج متری آمده، دقیق است اما صحیح نیست. فصل ۴۷ منابع خطای موقعیت‌یابی و راه کاهش آن با RTK را توضیح می‌دهد.

> **سطح حرفه‌ای**
>
> **رقم اعشار چه دقتی دارد؟** در عرض‌های جغرافیایی ایران، یک درجه‌ی عرض حدود ۱۱۱ کیلومتر است. پس رقم پنجم اعشار در مختصات درجه‌ای حدود یک متر و رقم ششم حدود ده سانتی‌متر است. ذخیره‌ی پانزده رقم اعشار برای داده‌ای با صحت چند متری فقط حجم را زیاد می‌کند. تابع `ST_ReducePrecision` در فصل ۱۸ برای همین کار است.

۲-۸ کیفیت داده و فراداده

داده‌ی بدون شناسنامه داده‌ی خطرناکی است. **فراداده** (metadata) «داده درباره‌ی داده» است: چه کسی، کی، با چه روشی و در چه سیستم مختصاتی آن را تهیه کرده، صحتش چقدر است و با چه مجوزی می‌توان از آن استفاده کرد. استاندارد ISO 19115 عناصر فراداده را تعریف می‌کند و استاندارد ISO 19157 کیفیت داده را در پنج بُعد می‌سنجد:

1. **کامل بودن:** آیا همه‌ی عارضه‌ها هستند؟ عارضه‌ی اضافه نداریم؟
2. **سازگاری منطقی:** قواعد رعایت شده‌اند؟ (هندسه معتبر، توپولوژی، دامنه‌ی مقادیر)

3. **صحت مکانی:** موقعیت‌ها چقدر به واقعیت نزدیک‌اند؟
4. **صحت زمانی:** داده چقدر به‌روز است؟
5. **صحت موضوعی:** ویژگی‌ها و دسته‌بندی‌ها درست‌اند؟

بخش «سلامت داده»ی سامانه (فصل ۷) بخشی از این بررسی‌ها، مثل هندسه‌ی نامعتبر، نبود شاخص مکانی و نبود سیستم مختصات را خودکار انجام می‌دهد. در پایگاه داده هم می‌توانید روی هر جدول و ستون **توضیح** (comment) بگذارید که در عمل همان فراداده‌ی ساده است (فصل ۱۴).

خودتان امتحان کنید

در داده‌ی نمونه، جدول `schools` را در نظر بگیرید. ستون‌های `name`، `level` و `students` هر کدام در کدام سطح اندازه‌گیری‌اند؟ برای نمایش هر کدام روی نقشه چه نمادی مناسب است؟ پاسخ را با جدول «سطح اندازه‌گیری» مقایسه کنید.

خلاصه‌ی فصل

- GIS جهان را در قالب مدل داده ساده می‌کند: **برداری** برای اشیای گسسته و **رستری** برای میدان‌های پیوسته و تصویر.
- عارضه‌ی برداری = هندسه + ویژگی‌ها؛ در پایگاه داده یک سطر با یک ستون هندسه.
- هفت نوع هندسه‌ی استاندارد: نقطه، خط، چندضلعی، سه نوع چندبخشی و مجموعه. در WKT اول طول (x) و بعد عرض (y) می‌آید.
- رستر با تفکیک مکانی، باند، نوع داده‌ی پیکسل، مقدار بی‌داده و مرجع مکانی تعریف می‌شود.
- سطح اندازه‌گیری (اسمی، ترتیبی، فاصله‌ای، نسبی) محاسبه‌ی مجاز و نماد مناسب را تعیین می‌کند.
- دقت با صحت فرق دارد، و داده بدون فراداده قابل اعتماد نیست.

برای مدرس

زمان پیشنهادی: دو جلسه‌ی ۹۰ دقیقه‌ای. در جلسه‌ی اول، یک عکس هوایی از محله‌ی دانشگاه نشان دهید و از دانشجوها بخواهید روی کاغذ شفاف آن را به نقطه، خط و چندضلعی تبدیل کنند؛ بعد همان را روی کاغذ شطرنجی رستری کنند و درباره‌ی از دست رفتن جزئیات بحث کنید. در جلسه‌ی دوم، تمرین ۶ را در کلاس انجام دهید.

تمرین‌ها

۱. سطح ۱ برای هر پدیده مدل مناسب (برداری یا رستری) و در صورت برداری، نوع هندسه را تعیین کنید: ایستگاه هواشناسی، دمای سطح زمین، مرز استان، خط لوله‌ی گاز، بارش سالانه، ساختمان، جزیره‌های یک کشور.

۲. سطح ۱ WKT یک چندضلعی مربعی با رأس‌های (۰، ۰) و (۱۰، ۱۰) بنویسید. چرا باید پنج جفت مختصات داشته باشد؟

۳. سطح ۱ تفاوت دقت و صحت را با یک مثال از زندگی روزمره توضیح دهید.

۴. سطح ۲ تصویری با تفکیک ۳۰ متر یک محدوده‌ی ۶۰ × ۹۰ کیلومتری را پوشش می‌دهد. چند پیکسل دارد؟ اگر هفت باند ۱۶ بیتی داشته باشد، حجم خام آن چند مگابایت است؟

۵. سطح ۲ چرا نمایش «تعداد جمعیت» محله‌ها با طیف رنگی گمراه‌کننده است؟ چه چیزی باید نمایش داده شود؟

۶. سطح ۲ یک فایل Shapefile با نام ستون‌های فارسی و متن به‌هم‌ریخته به شما داده‌اند. دو علت محتمل و راه حل هر کدام را بنویسید (راهنمایی: فصل ۷).

۷. سطح ۳ برای یک لایه‌ی واقعی که با آن کار می‌کنید یک فرادادهٔ یک‌صفحه‌ای بر اساس پنج بُعد کیفیت ISO 19157 بنویسید.

فصل

۳

شکل زمین، سیستم مختصات و تصویر نقشه

سطح: **مبتدی** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۲**

در این فصل یاد می‌گیرید

- تفاوت ژئوئید، بیضوی و داده‌نما (datum) را توضیح دهید.
- مختصات جغرافیایی را بخوانید و درجه‌ی دقیقه‌ثانیه را به درجه‌ی اعشاری تبدیل کنید.
- خانواده‌های تصویر نقشه و اعوجاج هر کدام را بشناسید.
- سیستم UTM و ناحیه‌های آن در ایران را بشناسید و SRID مناسب هر کار را انتخاب کنید.
- با SQL سیستم مختصات یک لایه را بخوانید و آن را تبدیل کنید.

سرفصل‌ها

۳-۱ چرا این فصل مهم‌ترین فصل مبانی است

۳-۲ شکل زمین: ژئوئید و بیضوی

۳-۳ داده‌نما (Datum)

۳-۴ مختصات جغرافیایی

۳-۵ تصویر نقشه

۳-۶ سیستم UTM

۳-۷ شناسنامه‌ی سیستم EPSG: و SRID مختصات

۳-۸ دو تابعی که نباید با هم اشتباه شوند

۳-۹ قاعده‌ی طلایی کار با مختصات

۳-۱ چرا این فصل مهم‌ترین فصل مبانی است

بیشترین خطاهایی که کارشناسان GIS در عمل با آن روبه‌رو می‌شوند به سیستم مختصات برمی‌گردد: لایه‌ای که وسط اقیانوس اطلس نمایش داده می‌شود، دو لایه که چند صد متر از هم فاصله دارند در حالی که باید روی هم بیفتند، مساحتی که به «درجه‌ی مربع» محاسبه شده و فاصله‌ای که دو برابر مقدار واقعی است. همه‌ی این‌ها با فهم چند مفهوم ساده که در این فصل می‌آید قابل پیشگیری‌اند.

پرسش اصلی این فصل این است: **مکان یک نقطه روی زمین را چگونه با عدد بیان کنیم؟** پاسخ سه لایه دارد: شکل زمین را چگونه مدل کنیم (بیضوی و داده‌نما)، نقطه را روی آن مدل چگونه نام‌گذاری کنیم (مختصات جغرافیایی)، و سطح خمیده را چگونه روی صفحه‌ی تخت بیاوریم (تصویر نقشه).

۳-۲ شکل زمین: ژئوئید و بیضوی

زمین کره‌ی کامل نیست. چرخش زمین آن را در استوا اندکی پهن کرده است: شعاع استوایی حدود ۶۳۷۸ کیلومتر و شعاع قطبی حدود ۶۳۵۷ کیلومتر است، یعنی حدود ۲۱ کیلومتر اختلاف. علاوه بر این، توزیع ناهمگون جرم در درون زمین باعث می‌شود میدان گرانش در همه‌جا یکسان نباشد.

- **ژئوئید** (geoid) سطحی است که سطح متوسط آب دریاها اگر زیر خشکی‌ها هم ادامه پیدا می‌کرد تشکیل می‌داد. ژئوئید سطح هم‌پتانسیل گرانش است و شکلی ناهموار و «سیب‌زمینی‌مانند» دارد. ارتفاع‌هایی که روی نقشه می‌بینیم (ارتفاع از سطح دریا) نسبت به ژئوئید است.
- **بیضوی** (ellipsoid) شکل ریاضی ساده‌ای است که ژئوئید را تقریب می‌زند: یک بیضی که حول محور کوتاهش چرخیده است. بیضوی با دو عدد تعریف می‌شود: نیم‌قطر بزرگ a و پهن‌شدگی f.

$$f = \frac{a - b}{a}$$

بیضوی WGS84 که GPS از آن استفاده می‌کند نیم‌قطر بزرگ ۶٬۳۷۸٬۱۳۷ متر و پهن‌شدگی حدود ۱ به ۲۹۸٫۲۶ دارد. بیضوی GRS80 تقریباً همان است (اختلاف آن‌ها کسری از میلی‌متر است).

بدانید

ارتفاع ارتومتریک و ارتفاع بیضوی. گیرنده‌ی GNSS ارتفاع را نسبت به بیضوی می‌دهد (ارتفاع بیضوی، h)، اما ارتفاع روی نقشه‌ها نسبت به ژئوئید است (ارتفاع ارتومتریک، H). اختلاف این دو، **ارتفاع ژئوئید** (N) نام دارد: $H = h - N$. در ایران N بسته به منطقه چند ده متر است؛ برای همین ارتفاع خام GPS با ارتفاع نقشه‌ی توپوگرافی یکی نیست. فصل ۴۷ این موضوع را در کار نقشه‌برداری دنبال می‌کند.

۳-۳ داده‌نما (Datum)

بیضوی فقط یک شکل است؛ باید مشخص کنیم این شکل **کجا و چگونه** روی زمین واقعی قرار گرفته است: مرکزش کجاست و محورهایش به کدام سمت‌اند. **داده‌نمای افقی** (datum) همین را تعریف می‌کند: یک بیضوی به‌علاوه‌ی جهت و موقعیتش نسبت به زمین.

دو نوع داده‌نما داریم:

- **داده‌نمای محلی** که بیضوی را طوری می‌گذارد که در یک منطقه‌ی خاص بهترین تطابق را با ژئوئید داشته باشد. داده‌نمای اروپایی ۱۹۵۰ (ED50) نمونه‌ای از آن است که در بسیاری از نقشه‌های قدیمی ایران به کار رفته است.
- **داده‌نمای زمین‌مرکز** که مرکز بیضوی را بر مرکز جرم زمین منطبق می‌کند و برای همه‌ی جهان مناسب است؛ مثل WGS84 و ITRF. داده‌های امروزی، GPS و نقشه‌های وب همه زمین‌مرکزند.

یک نقطه‌ی ثابت روی زمین در دو داده‌نمای مختلف مختصات متفاوتی دارد. اختلاف ED50 و WGS84 در ایران از مرتبه‌ی صد تا دویست متر است. پس اگر لایه‌ای در ED50 باشد و آن را بدون تبدیل کنار لایه‌ای در WGS84 بگذارید، دو لایه روی هم نمی‌افتند؛ هیچ خطایی هم نمی‌بینید، فقط نتیجه غلط است.

هشدار

داده‌نما را حدس نزنید. وقتی داده‌ای از سازمانی دیگر می‌گیرید، همیشه سیستم مختصات و داده‌نمای آن را بپرسید. فایل `.prj` همراه Shapefile معمولاً این اطلاعات را دارد؛ اگر نبود، از تهیه‌کننده بپرسید. مختصاتی که «تقریباً درست» به نظر می‌رسد ممکن است دویست متر خطا داشته باشد.

۳-۴ مختصات جغرافیایی

روی بیضوی، هر نقطه با دو زاویه مشخص می‌شود:

- **عرض جغرافیایی** (latitude، φ): زاویه‌ی شمال یا جنوب استوا، از ۹۰- (قطب جنوب) تا ۹۰+ (قطب شمال). خط‌های هم‌عرض را **مدار** می‌گویند.
- **طول جغرافیایی** (longitude، λ): زاویه‌ی شرق یا غرب نصف‌النهار مبدأ (گرینویچ)، از ۱۸۰- تا ۱۸۰+. خط‌های هم‌طول را **نصف‌النهار** می‌گویند.

ایران تقریباً بین عرض ۲۵ تا ۴۰ درجه‌ی شمالی و طول ۴۴ تا ۶۳٫۵ درجه‌ی شرقی قرار دارد. میدان آزادی تهران حدود عرض ۳۵٫۶۹۹ و طول ۵۱٫۳۳۸ است.

مختصات به دو شکل نوشته می‌شود: **درجه، دقیقه، ثانیه** (DMS) مثل `51° 20′ 17″ E` و **درجه‌ی اعشاری** (DD) مثل `51.338`. پایگاه داده و نرم‌افزارها با درجه‌ی اعشاری کار می‌کنند. تبدیل ساده است:

$$DD = D + \frac{M}{60} + \frac{S}{3600}$$

```sql
SELECT round(51 + 20/60.0 + 17/3600.0, 6) AS lon_dd,
       round(35 + 41/60.0 + 56/3600.0, 6) AS lat_dd;
```

lon_dd	lat_dd
51.338056	35.698889

نکته

طول یک درجه چقدر است؟ یک درجه‌ی عرض همه‌جا حدود ۱۱۱ کیلومتر است. اما یک درجه‌ی طول با نزدیک شدن به قطب کوتاه‌تر می‌شود: در استوا ۱۱۱ کیلومتر، در عرض تهران (حدود ۳۵٫۷ درجه) حدود ۹۰ کیلومتر و در قطب صفر. به همین دلیل **درجه واحد مناسبی برای اندازه‌گیری فاصله و مساحت نیست** و «درجه‌ی مربع» هیچ مساحت ثابتی ندارد.

همین را با SQL می‌بینیم. تابع `ST_Distance` روی هندسه‌ای در سیستم درجه‌ای، فاصله را به درجه می‌دهد که عدد بی‌معنایی است؛ اما اگر هندسه را به نوع `geography` تبدیل کنیم، فاصله را روی بیضوی و به متر حساب می‌کند:

```sql
SELECT ST_Distance(a.geom, b.geom)                         AS
dist_degree,
       ST_Distance(a.geom::geography, b.geom::geography) AS dist_meter
FROM metro_stations a, metro_stations b
WHERE a.station_id = 1 AND b.station_id = 2;
```

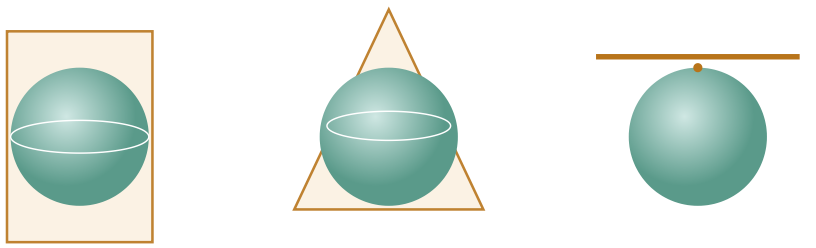
dist_degree	dist_meter
0.0123	1366.3021

۳-۵ تصویر نقشه

نقشه و صفحه‌ی نمایش تخت‌اند و زمین خمیده. **تصویر نقشه** (map projection) قاعده‌ای ریاضی است که هر نقطه‌ی بیضوی (φ, λ) را به یک نقطه روی صفحه (x, y) می‌برد. قضیه‌ای از گاوس (قضیه‌ی شگفت‌انگیز، ۱۸۲۷) نشان می‌دهد که تخت کردن کره **بدون اعوجاج ممکن نیست.** هر تصویری دست‌کم یکی از این ویژگی‌ها را خراب می‌کند:

ویژگی که حفظ می‌شود	نام	کاربرد
زاویه و شکل محلی	**همدیس** (conformal)	ناوبری، نقشه‌برداری، نقشه‌ی بزرگ‌مقیاس؛ UTM، مرکاتور، لامبرت همدیس
مساحت	**هم‌مساحت** (equal-area)	نقشه‌ی موضوعی تراکم، آمار مساحت؛ آلبرز، لامبرت هم‌مساحت
فاصله از یک یا دو نقطه	**هم‌فاصله** (equidistant)	فاصله از یک مرکز؛ آزیموتی هم‌فاصله
هیچ‌کدام دقیق؛ سازش	**سازشی**	نقشه‌ی جهان‌نما؛ رابینسون، وینکل

تصویرها بر اساس سطحی که کره روی آن «تابانده» می‌شود هم دسته‌بندی می‌شوند:



آزیموتی (صفحه‌ای)
نقشه‌ی قطبی، فاصله از یک مرکز

مخروطی
لامبرت؛ کشورهای عرض میانه

استوانه‌ای
مرکاتور، UTM، 3857

سه خانواده‌ی تصویر نقشه. کره روی استوانه، مخروط یا صفحه تابانده می‌شود؛ اعوجاج در نزدیکی خط تماس کمترین است.

Web Mercator، تصویر نقشه‌های وب

تقریباً همه‌ی نقشه‌های وب (OpenStreetMap و نقشه‌های تجاری) از تصویر **Web Mercator** با کد `3857` استفاده می‌کنند. این تصویر همدیس است و برای نمایش کاشی‌کاشی نقشه بسیار مناسب است، اما مساحت را به شدت بزرگ نشان می‌دهد: در عرض تهران مساحت‌ها حدود ۱٫۵ برابر و در عرض‌های بالا چند برابر بزرگ‌تر از واقعیت دیده می‌شوند (به همین دلیل گرینلند

روی نقشه‌ی وب هم‌اندازه‌ی آفریقا به نظر می‌رسد، در حالی که حدود چهارده برابر کوچک‌تر است).

هشدار

هرگز در 3857 مساحت یا فاصله حساب نکنید. این سیستم فقط برای نمایش است. مساحت قطعه‌ای در تهران که در 3857 محاسبه شود حدود ۵۰ درصد بیشتر از مقدار واقعی است.

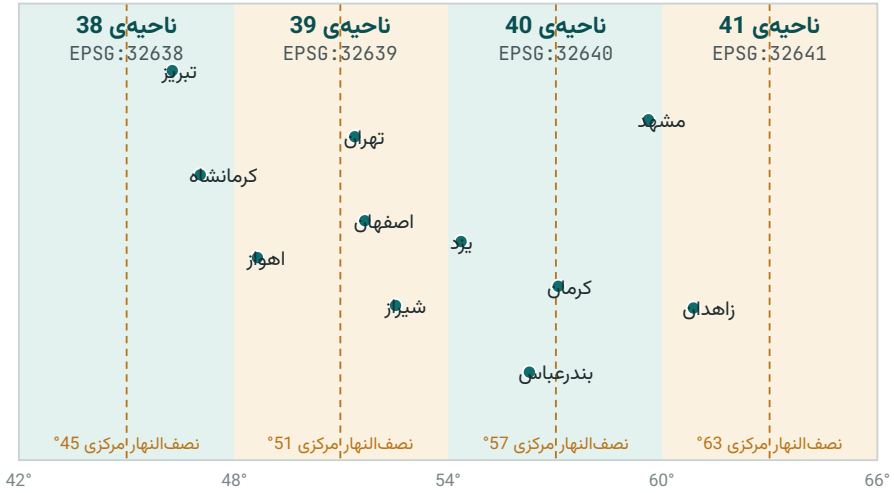
۳-۶ سیستم UTM

ترانسورس مرکاتور جهانی (Universal Transverse Mercator یا **UTM**) پرکاربردترین سیستم تصویر برای نقشه‌برداری و تحلیل در مقیاس شهری و استانی است. ایده‌ی آن ساده است: زمین را به ۶۰ **ناحیه**ی شش‌درجه‌ای تقسیم می‌کنیم و هر ناحیه را جداگانه با یک تصویر ترانسورس مرکاتور (استوانه‌ی افقی که در امتداد نصف‌النهار مرکزی ناحیه بر زمین مماس است) تصویر می‌کنیم. چون هر ناحیه باریک است، اعوجاج در آن ناچیز است.

مشخصات UTM:

- ناحیه‌ی ۱ از طول ۱۸۰- تا ۱۷۴- است و شماره‌ها به سمت شرق زیاد می‌شوند. ناحیه‌ی هر طول: $\lfloor(\lambda + 180)/6\rfloor + 1$.
- مختصات به **متر** است: x را «شرقی» (Easting) و y را «شمالی» (Northing) می‌نامند.
- برای اینکه x هیچ‌وقت منفی نشود، به نصف‌النهار مرکزی مقدار ۵۰۰٬۰۰۰ متر (شرقی کاذب) داده می‌شود. در نیمکره‌ی شمالی، y فاصله از استواست.
- ضریب مقیاس روی نصف‌النهار مرکزی ۰٫۹۹۹۶ است تا اعوجاج در کل ناحیه متعادل شود.

ایران در چهار ناحیه‌ی UTM قرار می‌گیرد:



ناحیه‌های UTM در ایران. هر ناحیه شش درجه‌ی طول را می‌پوشاند؛ خط‌چین نصف‌النهار مرکزی است که اعوجاج روی آن کمینه است.

در پایگاه داده‌ی مکانی، تبدیل از مختصات جغرافیایی به UTM یک تابع است:

```sql
SELECT name,
       ST_AsText(geom)                        AS wgs84_degree,
       ST_AsText(ST_Transform(geom, 32639)) AS utm39_meter
FROM schools
ORDER BY school_id
LIMIT 3;
```

name	wgs84_degree	utm39_meter
مدرسه 1	POINT(51.532393 35.655237)	POINT(548191.5190154058 3945840.8108380623)
مدرسه 2	POINT(51.456527 35.702091)	POINT(541300.0269403309 3951002.83231394)
مدرسه 3	POINT(51.486526 35.681792)	POINT(544025.0812104389 3948764.500113559)

مختصات UTM مدرسه‌ها حدود ۵۴۰٬۰۰۰ شرقی و ۳٬۹۵۰٬۰۰۰ شمالی است، یعنی حدود ۴۰ کیلومتر شرق نصف‌النهار مرکزی ۵۱ درجه و ۳۹۵۰ کیلومتر شمال استوا.

سطح حرفه‌ای

لایه‌ای که دو ناحیه را می‌پوشاند. برای کار در مقیاس کشور، UTM مناسب نیست چون ایران چهار ناحیه دارد و انتقال داده به ناحیه‌ای دور از محدوده‌اش اعوجاج زیادی ایجاد می‌کند. برای محاسبه‌ی مساحت در سطح کشور یا از نوع `geography` استفاده کنید که روی خود بیضوی حساب می‌کند (فصل ۱۶)، یا از یک تصویر مخروطی هم‌مساحت آلبرز یا لامبرت که مدارهای استاندارد آن داخل ایران تعریف شده‌اند. فصل ۱۶ این دو راه را با هم مقایسه می‌کند.

۳-۷ شناسنامه‌ی سیستم مختصات: EPSG و SRID

برای اینکه نرم‌افزارها بدانند هر عدد در کدام سیستم مختصات است، هر سیستم یک کد دارد. سازمانی که فهرست جهانی این کدها را نگه می‌دارد **EPSG** است (در اصل گروه نقشه‌برداری نفتی اروپا). در PostGIS به این کد **SRID** (شناسه‌ی سیستم مرجع مکانی) می‌گویند و هر هندسه SRID خودش را همراه دارد.

SRID	نام	واحد	کاربرد
`4326`	WGS 84 جغرافیایی	درجه	ذخیره‌ی عمومی، GPS، GeoJSON، تبادل داده
`3857`	WGS 84 / Pseudo-Mercator	متر (اعوجاج‌دار)	فقط نمایش روی نقشه‌ی وب
`32638`	WGS 84 / UTM 38N	متر	غرب ایران؛ تبریز، کرمانشاه
`32639`	WGS 84 / UTM 39N	متر	مرکز ایران؛ تهران، اصفهان، شیراز
`32640`	WGS 84 / UTM 40N	متر	شرق ایران؛ مشهد، کرمان
`32641`	WGS 84 / UTM 41N	متر	منتهی‌الیه شرق؛ زاهدان
`23038` تا `23040`	ED50 / UTM 38 تا 40	متر	نقشه‌های قدیمی؛ پیش از هر تحلیل به WGS84 تبدیل شود

فهرست همه‌ی سیستم‌هایی که PostGIS می‌شناسد در جدول `spatial_ref_sys` هر پایگاه داده‌ی مکانی هست:

```sql
SELECT srid, auth_name, auth_srid, split_part(srtext, '"', 2) AS
name
FROM spatial_ref_sys
WHERE srid IN (4326, 3857, 32639, 23039)
ORDER BY srid;
```

srid	auth_name	auth_srid	name
3857	EPSG	3857	WGS 84 / Pseudo-Mercator
4326	EPSG	4326	WGS 84
32639	EPSG	32639	WGS 84 / UTM zone 39N

و برای دیدن SRID یک لایه، یا SRID یک هندسه:

```sql
SELECT Find_SRID('public', 'schools', 'geom') AS layer_srid,
       (SELECT ST_SRID(geom) FROM parks LIMIT 1) AS one_park_srid;
```

layer_srid	one_park_srid
4326	4326

۳-۸ دو تابعی که نباید با هم اشتباه شوند

تفاوت این دو تابع را باید از بر باشید، چون اشتباه گرفتن آن‌ها رایج‌ترین خطای کار با مختصات است:

- `ST_Transform(geom, srid)` مختصات را واقعاً **تبدیل** می‌کند: عددها عوض می‌شوند تا همان نقطه‌ی روی زمین را در سیستم دیگر بیان کنند. از درجه به متر، یا از ED50 به WGS84.
- `ST_SetSRID(geom, srid)` فقط **برچسب** را عوض می‌کند و به عددها دست نمی‌زند. فقط وقتی به کار می‌رود که عددها از اول در آن سیستم بوده‌اند ولی برچسب نداشته‌اند (SRID صفر) یا برچسب اشتباه خورده‌اند.

```sql
SELECT ST_AsText(ST_SetSRID(ST_MakePoint(51.3890, 35.6892), 4326))
AS labeled,
       ST_AsText(ST_Transform(ST_SetSRID(ST_MakePoint(51.3890,
35.6892), 4326), 32639)) AS transformed;
```

labeled	transformed
POINT(51.389 35.6892)	POINT(535196.7818289505 3949546.7888362613)

در دنیای واقعی

لایه‌ی وسط اقیانوس. کارشناسی فایل CSV نقطه‌های برداشت را با ستون‌های x و y وارد می‌کند و SRID را ۴۳۲۶ می‌گذارد. نقطه‌ها روی نقشه دیده نمی‌شوند و بعد از جست‌وجو، گوشه‌ی نقشه‌ی جهان و بیرون از محدوده‌ی مجاز پیدا می‌شوند. علت: مختصات به UTM (مثلاً x = ۵۳۸۲۱۰، y = ۳۹۴۹۸۰۰) بوده و نرم‌افزار آن را درجه خوانده است. راه درست: `ST_SetSRID(geom, 32639)` تا برچسب درست شود و سپس اگر لازم است `ST_Transform(…, 4326)` . قاعده‌ی سرانگشتی: عدد بین ۴۴ تا ۶۴ و ۲۵ تا ۴۰ یعنی درجه در ایران؛ عددهای صدها هزار و میلیونی یعنی متر.

۳-۹ قاعده‌ی طلایی کار با مختصات

۱ **ذخیره:** داده را با SRID اصلی و درستش ذخیره کنید؛ برای داده‌ی تازه، `4326` انتخاب پیش‌فرض خوبی است.

۲ **محاسبه:** برای فاصله، طول و مساحت، یا هندسه را به UTM همان ناحیه ببرید، یا با `:: geography` روی بیضوی حساب کنید.

۳ **نمایش:** برای وب به `4326` یا `3857` (سامانه این را خودکار انجام می‌دهد).

۴ **یکدستی:** پیش از هر تحلیل دولایه‌ای مطمئن شوید هر دو لایه یک SRID دارند. PostGIS اگر SRID دو هندسه فرق کند خطا می‌دهد؛ این خطا دوست شماست، نه دشمن.

در فصل ۷ می‌بینید که سامانه هنگام ورود فایل، سیستم مختصات را از فایل `.prj` می‌خواند و اگر نبود، از شما می‌پرسد. بخش «سلامت داده» هم لایه‌های بدون SRID را پیدا می‌کند و امکان تعیین آن را می‌دهد.

خلاصه‌ی فصل

- ژئوئید سطح واقعی گرانشی و بیضوی تقریب ریاضی آن است؛ داده‌نما جای بیضوی را نسبت به زمین تعیین می‌کند.
- یک نقطه در دو داده‌نمای مختلف مختصات متفاوتی دارد (ED50 و WGS84 در ایران صد تا دویست متر).
- مختصات جغرافیایی (طول و عرض) به درجه است و برای اندازه‌گیری مناسب نیست.
- هر تصویر نقشه چیزی را خراب می‌کند: همدیس زاویه را حفظ می‌کند، هم‌مساحت مساحت را. Web Mercator فقط برای نمایش است.
- UTM ناحیه‌های شش‌درجه‌ای متری دارد؛ ایران در ناحیه‌های ۳۸ تا ۴۱ (SRID ۳۲۶۳۸ تا ۳۲۶۴۱) است.
- `ST_Transform` مختصات را تبدیل می‌کند و `ST_SetSRID` فقط برچسب را عوض می‌کند.

برای مدرس

زمان پیشنهادی: دو جلسه. یک پرتقال و چاقو به کلاس ببرید: پوست پرتقال را بکنید و سعی کنید آن را بدون پاره شدن روی میز تخت کنید؛ بهترین توضیح قضیه‌ی گاوس است. در جلسه‌ی دوم، تمرین ۵ و ۶ را در کنسول SQL انجام دهید. خطای «لایه‌ی وسط اقیانوس» را عمداً بسازید (CSV با مختصات UTM و SRID ۴۳۲۶) و از دانشجوها بخواهید علت را پیدا کنند.

تمرین‌ها

۱. سطح ۱ مختصات `۳۲° ۳۹′ ۲۴″ N` و `۵۱° ۴۰′ ۱۲″ E` را به درجه‌ی اعشاری تبدیل کنید. این نقطه در کدام شهر است؟

۲. سطح ۱ شهرهای رشت (طول ۴۹٫۶)، یزد (۵۴٫۴)، بیرجند (۵۹٫۲) و سنندج (۴۷٫۰) در کدام ناحیه‌ی UTMاند؟ SRID هر کدام چیست؟

۳. سطح ۱ چرا مساحت را نباید در SRID ۴۳۲۶ یا ۳۸۵۷ حساب کرد؟

۴. سطح ۲ تفاوت `ST_Transform` و `ST_SetSRID` را با یک مثال توضیح دهید. در چه موقعیتی هر کدام درست است؟

۵. سطح ۲ در کنسول SQL، مختصات میدان آزادی (`POINT(51.338 35.699)`) را به UTM ناحیه‌ی ۳۹ و ۴۰ تبدیل کنید. کدام شرقی‌اش به ۵۰۰٬۰۰۰ نزدیک‌تر است و چرا؟

۶. سطح ۲ فاصله‌ی دو ایستگاه مترو را یک بار با `:: geography` و یک بار پس از تبدیل به ۳۲۶۳۹ حساب کنید. اختلاف چقدر است؟

۷. سطح ۳ با فرمول ناحیه‌ی UTM، تابعی SQL بنویسید که برای هر نقطه‌ی WGS84 در نیمکره‌ی شمالی، SRID ناحیه‌ی UTM آن (۳۲۶۰۰ + شماره‌ی ناحیه) را برگرداند و آن را روی جدول `schools` آزمایش کنید.

فصل

۴

پایگاه داده‌ی مکانی و معماری WebGIS در ابر

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۱ تا ۳**

در این فصل یاد می‌گیرید

- مشکلات نگه‌داری داده‌ی مکانی در فایل را بشناسید و مزیت پایگاه داده را توضیح دهید.
- اجزای PostgreSQL و PostGIS و افزونه‌های رایج آن را نام ببرید.
- معماری یک سامانه‌ی WebGIS و جای هر جزء در آن را ترسیم کنید.
- استانداردهای OGC (OGC API، WFS، WMS، کاشی برداری) را از هم تشخیص دهید.
- بدانید پایگاه داده مکانی ابری کدام کارها را به جای شما انجام می‌دهد.

سرفصل‌ها

۴-۱ از فایل تا پایگاه داده

۴-۲ پایگاه داده‌ی رابطه‌ای در یک نگاه

۴-۳ PostGIS و PostgreSQL

۴-۴ افزونه‌ها

۴-۵ معماری یک سامانه‌ی WebGIS

۴-۶ پایگاه داده مکانی ابری چه کاری برای شما می‌کند

۴-۷ مسیر داده در این کتاب

۴-۱ از فایل تا پایگاه داده

تصور کنید شهرداری یک شهر متوسط داده‌های مکانی‌اش را در فایل نگه می‌دارد. کارشناس طرح تفصیلی یک Shapefile از کاربری زمین دارد، کارشناس درآمد نسخه‌ای دیگر از همان فایل که ستون عوارض به آن اضافه کرده، و کارشناس ترافیک نسخه‌ی سومی که شش ماه پیش از کارشناس طرح تفصیلی گرفته است. روزی مدیر می‌پرسد: «مساحت کاربری تجاری در منطقه‌ی ۳ چقدر است؟» و سه کارشناس سه عدد متفاوت می‌دهند. این ماجرا آشناست، چون مشکلات کار با فایل ذاتی‌اند:

مشکل کار با فایل	پایگاه داده‌ی مکانی چگونه حل می‌کند
نسخه‌های متعدد و ناهماهنگ	یک منبع واحد؛ همه به همان وصل می‌شوند
یک نفر در هر لحظه می‌تواند ویرایش کند	ویرایش هم‌زمان چندکاربره با کنترل تعارض
ویرایش نیمه‌کاره فایل را خراب می‌کند	**تراکنش**: یا همه‌ی تغییر ذخیره می‌شود یا هیچ‌کدام
هر کس فایل را دارد همه چیز را می‌بیند	**نقش و مجوز**: چه کسی چه جدولی را ببیند یا ویرایش کند
قواعد داده رعایت نمی‌شود	**قید**: نوع ستون، مقدار مجاز، کلید یکتا، هندسه‌ی معتبر
پرسش‌ها فقط با نرم‌افزار خاص	**SQL** استاندارد از هر نرم‌افزار و زبان برنامه‌نویسی
حجم زیاد کند می‌شود	**شاخص**؛ میلیون‌ها عارضه در کسری از ثانیه
بکاپ دستی و فراموش‌شدنی	بکاپ خودکار شبانه و بازگردانی با یک کلیک

به همین دلیل از حدود دو دهه پیش، روند حرفه‌ای در GIS این است که داده‌ی اصلی در پایگاه داده‌ی مکانی نگه داشته شود و فایل‌ها فقط برای تبادل به کار روند.

۴-۲ پایگاه داده‌ی رابطه‌ای در یک نگاه

پایگاه داده‌ی رابطه‌ای داده را در **جدول**‌ها نگه می‌دارد. هر جدول تعدادی **ستون** با نام و نوع مشخص دارد و هر **سطر** یک رکورد است. هر جدول معمولاً یک **کلید اصلی** دارد، ستونی که هر

سطر را یکتا مشخص می‌کند. جدول‌ها از طریق ستون‌های مشترک به هم **رابطه** دارند، و زبان **SQL** برای ساختن، پر کردن و پرسیدن از این جدول‌ها به کار می‌رود.

پایگاه داده‌ی مکانی همان پایگاه داده‌ی رابطه‌ای است که سه چیز به آن اضافه شده است:

1. **نوع داده‌ی مکانی:** ستونی که هندسه یا رستر نگه می‌دارد.
2. **توابع مکانی:** محاسبه‌ی فاصله، مساحت، تقاطع، بافر و صدها عمل دیگر داخل SQL.
3. **شاخص مکانی:** ساختاری که پرسش‌های مکانی روی داده‌ی حجیم را سریع می‌کند.

بخش سوم کتاب (فصل‌های ۱۰ تا ۱۴) پایگاه داده‌ی رابطه‌ای و SQL را از صفر آموزش می‌دهد. اینجا فقط می‌خواهیم تصویر کلی داشته باشیم.

۳-۴ PostgreSQL و PostGIS

PostgreSQL پایگاه داده‌ی رابطه‌ای متن‌باز و رایگانی است که ریشه‌ی آن به پروژه‌ای در دانشگاه برکلی در دهه‌ی ۱۹۸۰ برمی‌گردد. پایداری، پایبندی دقیق به استاندارد SQL و امکان افزودن نوع و تابع تازه آن را محبوب‌ترین پایگاه داده‌ی متن‌باز در کارهای حرفه‌ای کرده است.

PostGIS افزونه‌ای (extension) برای PostgreSQL است که آن را به پایگاه داده‌ی مکانی تبدیل می‌کند. PostGIS از سال ۲۰۰۱ توسعه می‌یابد و امروز مرجع عملی SQL مکانی در جهان است؛ بسیاری از نرم‌افزارها و سرویس‌های مکانی، از QGIS و GeoServer تا سرویس‌های نقشه‌ی بزرگ، روی آن ساخته شده‌اند. آنچه PostGIS اضافه می‌کند:

- **انواع داده:** `geometry` (هندسه روی صفحه‌ی مختصات)، `geography` (هندسه روی بیضوی زمین، با محاسبه به متر)، و `raster` (با افزونه‌ی `postgis_raster`).
- **توابع:** صدها تابع با پیشوند `ST_` (مخفف Spatial Type) مثل `ST_Distance`، `ST_Intersects`، `ST_Buffer` و `ST_Area`. نام و رفتار بیشتر آن‌ها از استاندارد OGC Simple Features و ISO SQL/MM آمده است، پس در پایگاه‌های داده‌ی مکانی دیگر هم آشناست.
- **شاخص مکانی** از نوع GiST بر پایه‌ی درخت R (فصل ۲۰).

- **جدول‌های فراداده:** `spatial_ref_sys` (فهرست سیستم‌های مختصات) و نمای `geometry_columns` (فهرست ستون‌های هندسه‌ی هر جدول با نوع و SRID).

نسخه‌ی PostGIS و کتابخانه‌هایی که با آن کار می‌کند (GEOS برای هندسه، PROJ برای تبدیل مختصات) را این‌طور می‌بینید:

```sql
SELECT postgis_lib_version() AS postgis,
       postgis_geos_version() AS geos,
       postgis_proj_version() AS proj;
```

postgis	geos	proj
3.4.2	3.12.1-CAPI-1.18.1	9.4.0 NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn.proj.org USER_WRITABLE_DIRECTORY=/tmp…

و فهرست لایه‌های مکانی پایگاه داده، با نوع هندسه و SRID، در یک پرسش:

```sql
SELECT f_table_name AS layer, f_geometry_column AS column_name,
type, srid
FROM geometry_columns
ORDER BY f_table_name;
```

layer	column_name	type	srid
health_centers	geom	POINT	4326
metro_stations	geom	GEOMETRY	4326
ndvi_parcels_latest	geom	POLYGON	4326
neighborhoods	geom	POLYGON	4326
parcels	geom	POLYGON	4326
parks	geom	GEOMETRY	4326
… (7 rows)			

بدانید

تعداد توابع. پایگاه داده‌ی مکانی واقعاً صدها تابع مکانی دارد. شمردنش با SQL ساده است: `SELECT count(DISTINCT proname) FROM pg_proc WHERE proname ILIKE 'st\_%';`. لازم نیست همه را حفظ کنید؛ حدود سی تابع کار روزمره‌ی بیشتر کارشناسان را پوشش می‌دهد و پیوست «مرجع توابع پرکاربرد» آن‌ها را در یک جا جمع کرده است.

۴-۴ افزونه‌ها

PostgreSQL با **افزونه**‌ها گسترش می‌یابد. در پایگاه داده مکانی ابری، PostGIS همیشه نصب است و بقیه را هنگام ساخت پایگاه یا بعداً از صفحه‌ی «نمای کلی» با یک کلیک نصب می‌کنید (فصل ۵):

افزونه	چه می‌کند	فصل
`postgis`	هندسه، geography، توابع `ST_` ، شاخص مکانی	۱۵ تا ۲۲
`postgis_raster`	ذخیره و تحلیل رستر داخل پایگاه داده	۲۳
`postgis_topology`	توپولوژی: مرز مشترک، همسایگی، کاداستر	۲۲
`postgis_sfcgal`	عملیات سه‌بعدی و هندسه‌ی دقیق	۲۲
`pgrouting`	تحلیل شبکه: کوتاه‌ترین مسیر، حوزه‌ی دسترسی	۲۴
`h3`	شبکه‌ی شش‌ضلعی سلسله‌مراتبی جهانی	۲۱
`pg_trgm`	جست‌وجوی سریع و تشابه متنی (برای نام مکان‌ها)	۱۱
`fuzzystrmatch`	شباهت آوایی و فاصله‌ی ویرایشی کلمات	۱۱
`hstore`	ذخیره‌ی ویژگی‌های کلید-مقدار متغیر (مثل داده‌ی OSM)	۳۰

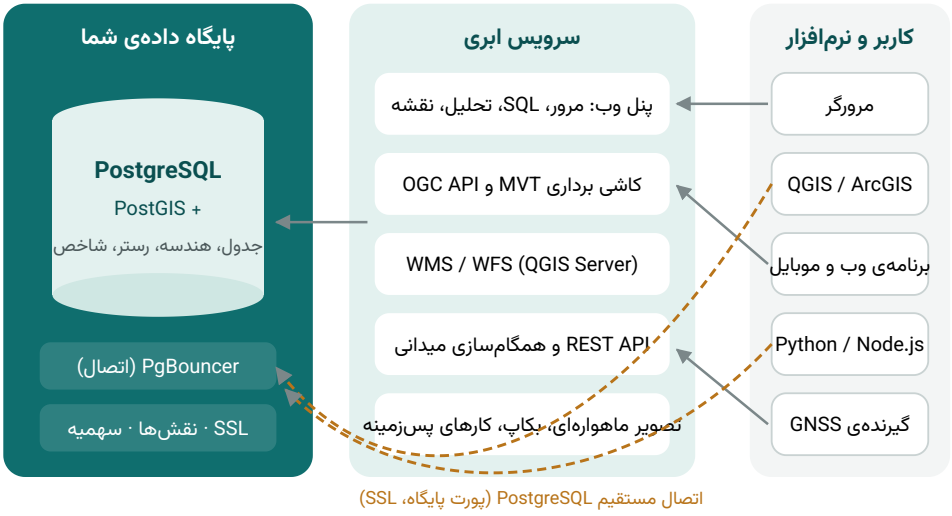
نکته

افزونه‌ای را نصب کنید که واقعاً لازم دارید. هر افزونه جدول‌ها و توابع تازه‌ای به پایگاه اضافه می‌کند؛ مثلاً `postgis_raster` اگر رستر بزرگ در آن ذخیره کنید، حجم زیادی از سهمیه‌ی پایگاه را می‌گیرد.

۴-۵ معماری یک سامانه‌ی WebGIS

WebGIS یعنی سامانه‌ی GIS که از طریق وب در دسترس است. یک سامانه‌ی WebGIS معمولاً سه لایه دارد:

1. **لایه‌ی داده:** پایگاه داده‌ی مکانی که داده‌ی اصلی را نگه می‌دارد.
2. **لایه‌ی سرویس:** نرم‌افزارهایی که داده را به شکل استاندارد به بیرون می‌دهند: نقشه‌ی تصویری (WMS)، عارضه (WFS، OGC API Features)، کاشی برداری (MVT)، API برنامه‌نویسی.
3. **لایه‌ی کاربر:** مرورگر با یک کتابخانه‌ی نقشه (MapLibre، OpenLayers، Leaflet)، نرم‌افزار رومیزی (QGIS) یا برنامه‌ی موبایل.



معماری پایگاه داده مکانی ابری. نرم‌افزارهای رومیزی و برنامه‌ها می‌توانند هم از طریق سرویس‌های وب و هم مستقیم با پروتکل PostgreSQL به پایگاه داده وصل شوند.

در شیوه‌ی سنتی، سازمان باید هر سه لایه را خودش بسازد و نگه دارد: سرور، PostgreSQL، PostGIS، GeoServer، وب‌سرور، گواهی SSL، بکاپ و پایش. در پایگاه داده مکانی ابری، لایه‌ی داده و لایه‌ی سرویس آماده است و شما فقط داده و پرسش‌تان را می‌آورید.

استانداردهای OGC

کنسرسیوم مکانی باز (Open Geospatial Consortium یا OGC) سازمانی بین‌المللی است که استانداردهای تبادل داده‌ی مکانی را تدوین می‌کند. به لطف این استانداردها، لایه‌ای که در یک سامانه منتشر می‌شود در هر نرم‌افزار دیگری قابل استفاده است.

استاندارد	چه چیزی می‌دهد	مثال استفاده
WMS (Web Map Service)	تصویر نقشه (PNG/JPEG) برای یک محدوده	لایه‌ی زمینه در QGIS؛ نقشه‌ی ثابت در وب‌سایت
WFS (Web Feature Service)	خود عارضه‌ها با هندسه و ویژگی (GML/GeoJSON)	دریافت داده برای تحلیل در QGIS
OGC API – Features	نسل جدید WFS؛ RESTful با JSON	برنامه‌های وب مدرن
WMTS / کاشی برداری (MVT)	نقشه در کاشی‌های پیش‌برش‌خورده	نقشه‌های سریع وب با هزاران عارضه
GeoPackage، GeoTIFF، KML	قالب‌های فایل استاندارد	تبادل داده
Simple Features، SQL/MM	مدل هندسه و توابع SQL	همان توابع `ST_` در PostGIS

بخش هفتم کتاب انتشار داده با هر یک از این استانداردها را گام‌به‌گام نشان می‌دهد.

۴-۶ پایگاه داده مکانی ابری چه کاری برای شما می‌کند

کار	در شیوه‌ی سنتی	در پایگاه داده مکانی ابری
ساخت پایگاه داده با PostGIS	نصب و پیکربندی سرور (ساعت‌ها تا روزها)	یک فرم و چند ثانیه
امنیت اتصال	تهیه‌ی گواهی و تنظیم SSL	SSL پیش‌فرض و اجباری
کاربران و دسترسی	دستورهای `GRANT`	صفحه‌ی «تیم» با نقش‌های آماده
اتصال‌های زیاد از برنامه‌ی وب	راه‌اندازی PgBouncer	اتصال PgBouncer آماده
بکاپ	اسکریپت و زمان‌بندی دستی	بکاپ خودکار روزانه و بازگردانی با یک کلیک
پایش حجم و کارایی	ابزارهای جانبی	صفحه‌ی «مصرف و کارایی»
مرور و ویرایش داده	نرم‌افزار رومیزی	در مرورگر، با پیش‌نویس و تأیید
تحلیل مکانی	نوشتن SQL یا نرم‌افزار رومیزی	SQL در مرورگر، ابزارهای آماده، پرسش فارسی
تصویر ماهواره‌ای	جست‌وجو، دانلود چندگیگابایتی، برش	انتخاب محدوده و تاریخ؛ تحویل برش‌خورده
انتشار نقشه و سرویس	GeoServer و وب‌سرور	چند کلیک؛ کاشی، OGC API، WMS/WFS

هشدار

ابری بودن به معنای بی‌مسئولیتی نیست. سرویس ابری زیرساخت را نگه می‌دارد، اما **طراحی داده، کیفیت آن، مجوز دسترسی کاربران و درستی تحلیل** همچنان با شماست. رمز پایگاه داده را مثل رمز بانک نگه دارید، دسترسی ویرایش را فقط به کسانی بدهید که لازم دارند، و پیش از تغییرهای بزرگ بکاپ دستی بگیرید (فصل ۵۱).

امنیت در یک نگاه

چند اصل امنیتی که پایگاه داده مکانی ابری رعایت می‌کند و دانستنش برای شما مفید است:

- **جداسازی:** هر پایگاه داده نقش (کاربر) جداگانه دارد و هیچ کاربری به پایگاه داده‌ی دیگران دسترسی ندارد. نقش شما «ابرکاربر» (superuser) نیست.
- **مرورگر هیچ‌وقت مستقیم به پایگاه داده وصل نمی‌شود.** صفحه‌های وب سامانه با یک نقش محدود جداگانه به پایگاه وصل می‌شوند و کنسول SQL به‌طور پیش‌فرض فقط‌خواندنی است؛ حداکثر ۵۰۰ سطر نمایش می‌دهد و هر پرسش حداکثر ۲۰ ثانیه وقت دارد.
- **رمز فقط یک بار نمایش داده می‌شود** و سامانه آن را به صورت خوانا نگه نمی‌دارد. اگر فراموش شد، رمز تازه می‌سازید (فصل ۵).
- **کلید API** برای لایه‌های منتشرشده‌ی خصوصی جدا از رمز پایگاه داده است و هر وقت لازم شد باطل می‌شود (فصل ۴۵).

۴-۷ مسیر داده در این کتاب

حالا که تصویر کلی را دارید، بخش بعد کار عملی را شروع می‌کند: پایگاه داده‌ی خودتان را می‌سازید (فصل ۵)، از QGIS به آن وصل می‌شوید (فصل ۶)، داده‌ی نمونه را وارد می‌کنید (فصل ۷)، آن را مرور و جست‌وجو می‌کنید (فصل ۸) و جدول تازه می‌سازید و ویرایش می‌کنید (فصل ۹). از بخش سوم به بعد، همین پایگاه داده آزمایشگاه شما برای یادگیری SQL و تحلیل مکانی است.

خلاصه‌ی فصل

- نگه‌داری داده در فایل به نسخه‌های ناهماهنگ، ویرایش تک‌کاربره و نبود قید و امنیت می‌انجامد؛ پایگاه داده این‌ها را حل می‌کند.
- پایگاه داده‌ی مکانی = پایگاه داده‌ی رابطه‌ای + نوع داده‌ی مکانی + توابع مکانی + شاخص مکانی.
- PostGIS افزونه‌ی مکانی PostgreSQL است: `geometry` ، `geography` ، `raster` ، توابع `ST_` و شاخص GiST.
- WebGIS سه لایه دارد: داده، سرویس و کاربر؛ استانداردهای OGC (WMS، WFS، API)، کاشی) پیوند آن‌ها را ممکن می‌کنند.
- پایگاه داده مکانی ابری زیرساخت، امنیت اتصال، بکاپ و سرویس‌ها را آماده می‌کند؛ طراحی و کیفیت داده با شماست.

برای مدرس

زمان پیشنهادی: یک جلسه. داستان «سه کارشناس، سه عدد» را به صورت نمایش کوتاه در کلاس اجرا کنید: سه گروه یک فایل اکسل مشترک را جداگانه ویرایش کنند و بعد سعی کنند نسخه‌ها را یکی کنند. سپس همان کار را در یک جدول مشترک در پایگاه داده انجام دهید. تمرین ۵ برای جمع‌بندی مناسب است.

تمرین‌ها

۱. سطح ۱ سه مشکل نگه‌داری داده‌ی مکانی در فایل و راه حل پایگاه داده برای هر کدام را بنویسید.

۲. سطح ۱ سه چیزی که PostGIS به PostgreSQL اضافه می‌کند کدام‌اند؟

۳. سطح ۱ تفاوت WMS و WFS چیست؟ برای هر کدام یک مورد استفاده بنویسید.

۴. سطح ۲ برای یک پروژه‌ی مسیریابی آمبولانس با نام خیابان‌های فارسی، کدام افزونه‌ها را نصب می‌کنید و چرا؟

۵. سطح ۲ معماری WebGIS سازمان خودتان (یا یک سازمان فرضی) را ترسیم کنید: داده کجاست، با چه سرویسی منتشر می‌شود و چه کسانی با چه نرم‌افزاری از آن استفاده می‌کنند؟

۶. سطح ۳ پس از ساختن پایگاه داده در فصل ۵، پرسش شمارش توابع `ST_` را اجرا کنید. سپس فهرست ده تابعی را که نامشان را می‌فهمید بنویسید و حدس بزنید هر کدام چه می‌کند؛ حدس‌تان را با پیوست کتاب مقایسه کنید.

بخش ۲

آغاز کار با پایگاه داده مکانی ابری

در این بخش پایگاه داده‌ی خودتان را می‌سازید، از QGIS و نرم‌افزارهای دیگر به آن وصل می‌شوید، داده وارد می‌کنید، آن را مرور و جست‌وجو می‌کنید و جدول تازه می‌سازید. پایان این بخش، یک پایگاه داده‌ی مکانی کارا در اختیار دارید.

۵ حساب کاربری، ساخت پایگاه داده و فضای کار

۶ اتصال از QGIS، ArcGIS، pgAdmin و برنامه‌ها

۷ ورود داده و سلامت داده

۸ مرور، جست‌وجو و خروجی گرفتن از داده

۹ ساخت جدول، ویرایش داده و تغییر ساختار

فصل

۵

حساب کاربری، ساخت پایگاه داده و فضای کار

سطح: **مبتدی** | زمان مطالعه: **۴۵ دقیقه** | پیش‌نیاز: **فصل ۴**

در این فصل یاد می‌گیرید

- با موبایل حساب بسازید و وارد سامانه شوید.
- یک پایگاه داده‌ی مکانی با نام، رمز و افزونه‌های مناسب بسازید.
- اطلاعات اتصال را بخوانید و بدانید هر مقدار چه معنایی دارد.
- فضای کار، منوی کناری، جستجوی سراسری و اعلان‌ها را بشناسید.
- رمز تازه بسازید و افزونه‌ی تازه نصب کنید.

سرفصل‌ها

۵-۱ ورود به سامانه

۵-۲ پایگاه‌های داده‌ی من

۵-۳ ساخت پایگاه داده

۵-۴ صفحه‌ی «نمای کلی و اتصال»

۵-۵ اولین پرسش

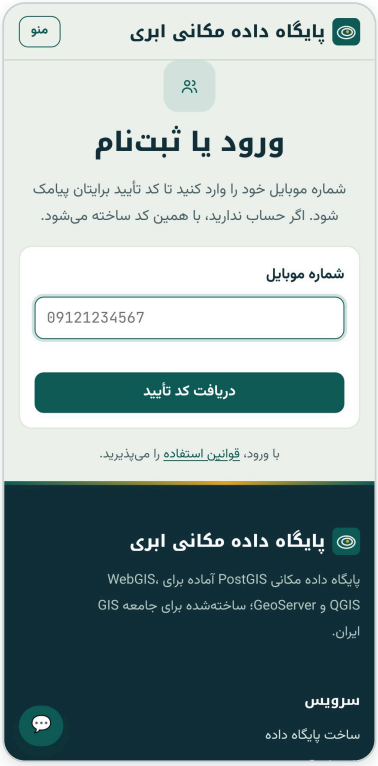
۵-۶ فضای کار

۵-۷ رمز تازه و افزونه‌ی تازه

۵-۸ چرخه‌ی عمر پایگاه داده

۵-۱ ورود به سامانه

ورود به پایگاه داده مکانی ابری **رمز عبور ندارد.** شماره‌ی موبایل خود را وارد می‌کنید، یک کد یک‌بارمصرف پیامک می‌شود و با وارد کردن آن وارد می‌شوید. ثبت‌نام و ورود یک صفحه است: اگر بار اول باشد حساب ساخته می‌شود، وگرنه وارد حساب قبلی می‌شوید.



صفحه‌ی ورود روی گوشی. همه‌ی صفحه‌های سامانه روی گوشی هم کار می‌کنند.

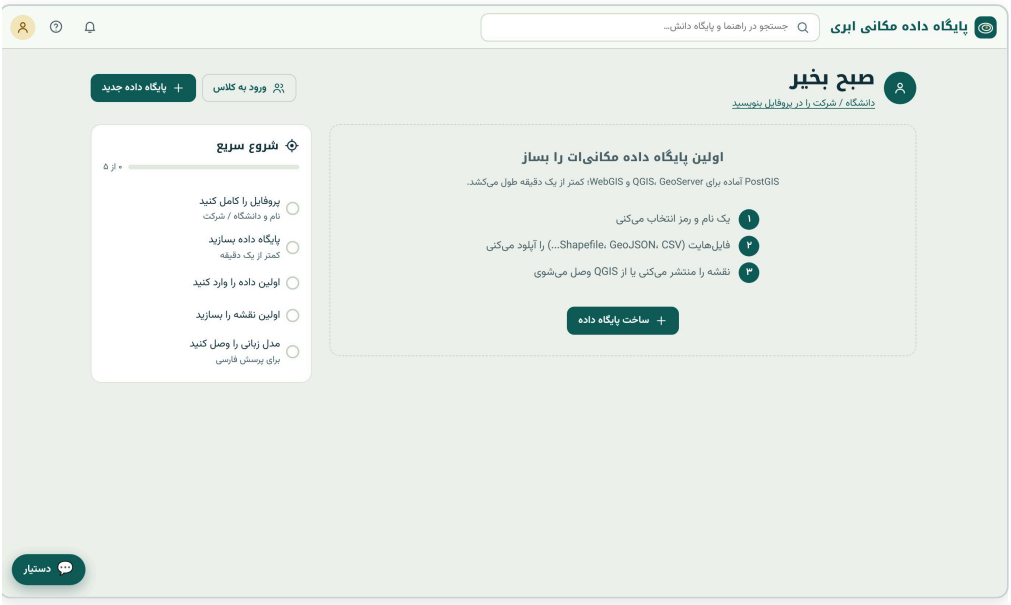
نکته

دو رمز متفاوت. ورود به **سایت** رمز ندارد و با کد پیامکی است. اما هر **پایگاه داده** رمز جداگانه‌ای دارد که هنگام ساختنش تعیین می‌کنید و QGIS و نرم‌افزارهای دیگر با آن به پایگاه وصل می‌شوند. این دو را با هم اشتباه نگیرید.

هر شخص یک حساب رایگان دارد. اگر بعداً شماره‌ی موبایل‌تان عوض شد، از بخش «پشتیبانی» درخواست تغییر بدهید.

۵-۲ پایگاه‌های داده‌ی من

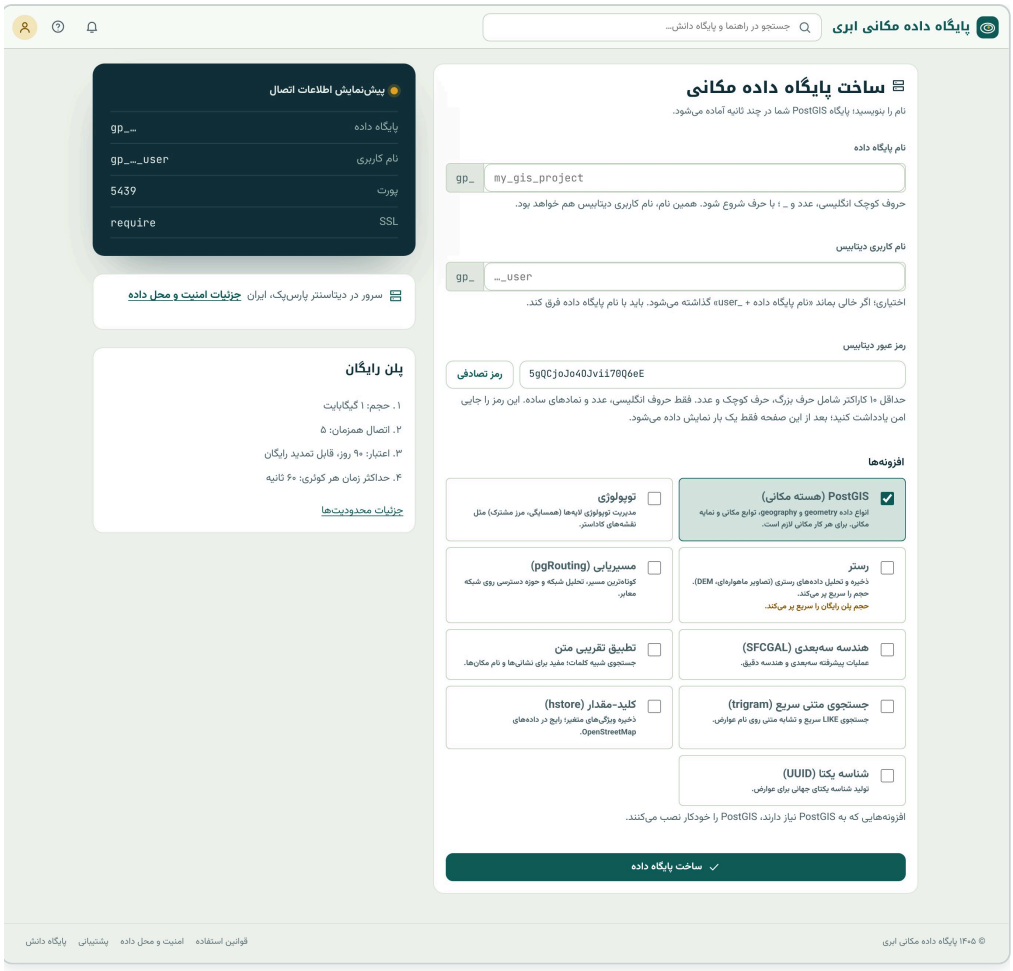
پس از ورود، صفحه‌ی «پایگاه‌های داده» را می‌بینید. بار اول این صفحه خالی است و یک دکمه‌ی اصلی دارد: «ساخت پایگاه داده».



صفحه‌ی پایگاه‌های داده برای کاربر تازه. حالت خالی همیشه کار بعدی را نشان می‌دهد.

۵-۳ ساخت پایگاه داده

فرم ساخت پایگاه داده چهار بخش دارد و در سمت چپ پیش‌نمایش زنده‌ی اطلاعات اتصال را نشان می‌دهد.



فرم ساخت پایگاه داده. نام، نام کاربری، رمز و افزونه‌ها؛ در سمت چپ پیش‌نمایش اتصال و مشخصات پلن.

۱ **نام پایگاه داده:** با حرف کوچک انگلیسی شروع شود و فقط حروف کوچک انگلیسی، عدد و زیرخط داشته باشد (۳ تا ۴۱ نویسه). پیشوند `gp_` خودکار اضافه می‌شود. برای داده‌ی نمونه‌ی این کتاب نام `tehran_gis` را بنویسید تا پایگاه `gp_tehran_gis` ساخته شود.

۲ **نام کاربری پایگاه داده:** اختیاری است؛ اگر خالی بماند `<نام پایگاه>_user` گذاشته می‌شود. نام کاربری باید با نام پایگاه فرق کند.

۳ **رمز:** دست‌کم ده نویسه شامل حرف بزرگ، حرف کوچک و عدد. دکمه‌ی «رمز تصادفی» یک رمز قوی می‌سازد.

۴ **افزونه‌ها:** PostGIS همیشه تیک خورده است. برای این کتاب `pgRouting` (فصل ۲۴) و `جستجوی متنی سریع (trigram)` را هم تیک بزنید. «رستر» را فقط اگر قصد دارید رستر را **داخل** پایگاه ذخیره کنید انتخاب کنید (فصل ۲۳)؛ برای تصویر ماهواره‌ای سامانه لازم نیست.

۵ «ساخت پایگاه داده» را بزنید. کمتر از یک دقیقه بعد پایگاه آماده است.

هشدار

رمز فقط یک بار نمایش داده می‌شود. بلافاصله پس از ساخت، رمز را در یک مدیر رمز یا جای امن ذخیره کنید. سامانه رمز را به شکل خوانا نگه نمی‌دارد و نمی‌تواند آن را دوباره به شما نشان دهد. اگر گم شد، رمز تازه بسازید (پایین‌تر در همین فصل)؛ داده‌ها از بین نمی‌روند. نام پایگاه داده را هم بعداً نمی‌توان عوض کرد.

بدانید

پشت صحنه‌ی ساخت پایگاه. وقتی «ساخت» را می‌زنید سامانه روی یک سرور PostgreSQL این کارها را می‌کند: یک **نقش** (کاربر) با نام کاربری شما می‌سازد، یک **پایگاه داده** با مالکیت همان نقش می‌سازد، افزونه‌های انتخابی را نصب می‌کند، **سقف حجم و تعداد اتصال** پلن را اعمال می‌کند و دسترسی دیگران را می‌بندد. معادل SQL این کارها تقریباً این است (فقط برای فهم؛ لازم نیست اجرا کنید):

```sql
CREATE ROLE gp_tehran_gis_user LOGIN PASSWORD '…' CONNECTION
LIMIT 5;
CREATE DATABASE gp_tehran_gis OWNER gp_tehran_gis_user;
\c gp_tehran_gis
CREATE EXTENSION postgis;
CREATE EXTENSION pgrouting;
REVOKE ALL ON DATABASE gp_tehran_gis FROM PUBLIC;
```

۵-۴ صفحه‌ی «نمای کلی و اتصال»

هر پایگاه داده یک صفحه‌ی اصلی دارد که در فصل ۱ تصویرش را دیدید. بالای صفحه، برای پایگاه تازه، کارت «حالا داده بگذارید» سه راه شروع را پیشنهاد می‌کند: داده‌ی آماده‌ی ایران، ورود فایل خودتان، یا اتصال از QGIS. زیر آن دو کارت اصلی است:

اطلاعات اتصال. پنج مقداری که هر نرم‌افزاری برای اتصال لازم دارد، هر کدام با دکمه‌ی «کپی»:

مقدار	معنا	نکته
میزبان (Host)	نشانی سرور پایگاه داده	نام دامنه‌ی سرور
پورت (Port)	درگاه شبکه‌ی سرور	پورت سامانه ۵۴۳۲ (پیش‌فرض PostgreSQL) نیست؛ همان را که نشان داده می‌شود بنویسید
پایگاه داده (Database)	نام پایگاه	با پیشوند `gp_`
نام کاربری (User)	نقش شما	با پسوند `_user` (اگر عوضش نکرده باشید)
رمز	همان رمز هنگام ساخت	نمایش داده نمی‌شود
SSL Mode	رمزنگاری اتصال	همیشه `require`؛ بدون SSL اتصال پذیرفته نمی‌شود

مصرف و اعتبار. حجم مصرفی از سهمیه، تاریخ ساخت و پایان اعتبار، تعداد اتصال هم‌زمان مجاز و دکمه‌های تمدید و ارتقای پلن.

پایین‌تر، کارت **«اتصال از نرم‌افزار شما»** برای هر نرم‌افزار (QGIS، GeoServer، رشته‌ی اتصال، Python، ورود psql، Shapefile) راهنمای کوتاه با همان مقادیر پایگاه شما دارد، و کارت‌های **«افزونه‌های نصب‌شده»**، **«رمز جدید»** و **«رویدادهای اخیر»** (تاریخچه‌ی کارهای مهم روی پایگاه) قرار دارند.

سطح حرفه‌ای

محدودیت‌های پلن در سطح پایگاه داده. «اتصال هم‌زمان» یعنی حداکثر چند نشست هم‌زمان می‌تواند با نقش شما باز باشد؛ هر پنجره‌ی QGIS، هر برنامه و هر اسکریپت پایتون حداقل یک اتصال می‌گیرد. «حداکثر زمان هر کوئری» (statement timeout) پرسش‌هایی را که بیش از حد طول بکشند متوقف می‌کند. هر دو را در SQL هم می‌توانید ببینید: `SHOW statement_timeout;` و `SELECT rolconnlimit FROM pg_roles WHERE rolname = current_user;`. فصل ۵۱ مدیریت اتصال با PgBouncer را توضیح می‌دهد.

۵-۵ اولین پرسش

پیش از ورود داده، مطمئن شویم پایگاه کار می‌کند. در منوی کناری ← داده ← «اجرای SQL» را باز کنید، این دستور را بنویسید و «اجرا» را بزنید:

```sql
SELECT current_database() AS database,
       postgis_lib_version() AS postgis,
       now()::date AS today;
```

database	postgis	today
gp_tehran_gis	3.4.2	2026-11-09

اگر نام پایگاه و نسخه‌ی PostGIS را دیدید، پایگاه آماده است. فصل ۱۰ کنسول SQL را کامل معرفی می‌کند.

۵-۶ فضای کار

رابط سامانه یک **نوار بالا** و یک **ریل کناری** دارد و جز این دو، منوی دیگری نیست:

- **نوار بالا:** نام سامانه (بازگشت به فهرست پایگاه‌ها)، **جستجوی سراسری**، راهنما، اعلان‌ها و منوی حساب.
- **ریل کناری** (سمت راست): نام و وضعیت پایگاه، نوار مصرف حجم و چهار گروه منو: «نمای کلی و اتصال»، «داده»، «تحلیل و انتشار» و «مدیریت». با «جمع کردن منو» پایین ریل،

فضای کار بزرگ‌تر می‌شود.

جستجوی سراسری (کلید میانبر `Ctrl+K`) بخش‌های سامانه، جدول‌های پایگاه‌ها و مقاله‌های راهنما را با هم جست‌وجو می‌کند. نوشتن «بکاپ» شما را به صفحه‌ی بکاپ می‌برد و نوشتن `schools` به جدول مدرسه‌ها.

اعلان‌ها (زنگوله) کارهای پس‌زمینه را خبر می‌دهند: ورود فایل بزرگ تمام شد، تحلیل آماده است، تصویر ماهواره‌ای رسید، پایش ماهواره‌ای هشدار داد یا اعتبار پایگاه رو به پایان است.

همان صفحه روی گوشی. ریل کناری به کشو تبدیل می‌شود و کارت‌ها زیر هم قرار می‌گیرند.

نکته

دستیار هوشمند. دکمه‌ی «دستیار» پایین ریل، پرسش فارسی شما درباره‌ی کار با سامانه را از روی پایگاه دانش پاسخ می‌دهد و منبع پاسخ را هم نشان می‌دهد. دستیار به داده‌ها و رمزهای شما دسترسی ندارد؛ برای پرسش از **داده‌ها**، «پرسش فارسی» در کنسول SQL را به کار ببرید (فصل ۲۹).

۵-۷ رمز تازه و افزونه‌ی تازه

رمز تازه: در صفحه‌ی «نمای کلی»، کارت «رمز جدید» یک رمز تصادفی پیشنهاد می‌کند یا رمز دلخواه شما را می‌گیرد. با «ذخیره‌ی رمز جدید»، اتصال‌های فعلی قطع نمی‌شوند ولی اتصال بعدی رمز تازه را می‌خواهد؛ پس رمز ذخیره‌شده در QGIS و برنامه‌ها را هم عوض کنید.

افزونه‌ی تازه: در کارت «افزونه‌های نصب‌شده»، افزونه را از فهرست انتخاب کنید و «نصب افزونه» را بزنید. حذف افزونه از رابط ممکن نیست، چون ممکن است جدول‌هایی به آن وابسته باشند.

خودتان امتحان کنید

یک پایگاه داده با نام `tehran_gis` و افزونه‌های PostGIS، pgRouting و trigram بسازید. رمز را ذخیره کنید، پرسش «اولین پرسش» را اجرا کنید و سپس از کارت «اتصال از نرم‌افزار شما» زبانه‌ی «رشته اتصال» را ببینید. این رشته را در فصل ۶ لازم دارید.

۵-۸ چرخه‌ی عمر پایگاه داده

هر پایگاه داده اعتبار زمانی دارد که در کارت «مصرف و اعتبار» نمایش داده می‌شود. چند روز پیش از پایان اعتبار با پیامک و اعلان خبر داده می‌شود. اگر تمدید نشود، **ورود به پایگاه غیرفعال می‌شود ولی داده می‌ماند** و چند روز فرصت تمدید دارید؛ پس از آن پایگاه و داده‌هایش حذف می‌شوند. پس بکاپ مهم را همیشه روی رایانه‌ی خودتان هم نگه دارید (فصل ۵۱).

سهمیه‌ی **حجم** جداگانه پایش می‌شود: در ۸۰ درصد هشدار می‌گیرید و اگر به سقف برسد، پایگاه **فقط‌خواندنی** می‌شود؛ خواندن و دانلود ممکن است ولی نوشتن نه. با پاک کردن داده‌ی

اضافه تا زیر ۹۵ درصد سقف یا ارتقای پلن، پایگاه دوباره فعال می‌شود.

خلاصه‌ی فصل

- ورود به سایت با موبایل و کد پیامکی است؛ هر پایگاه داده رمز جداگانه‌ی خودش را دارد.
- نام پایگاه داده (با پیشوند `gp_`) قابل تغییر نیست؛ رمز فقط یک بار نمایش داده می‌شود ولی می‌توان رمز تازه ساخت.
- اطلاعات اتصال پنج مقدار به‌علاوه‌ی `sslmode=require` است؛ پورت سامانه پیش‌فرض نیست.
- فضای کار یک نوار بالا (جستجوی `Ctrl+K`، اعلان‌ها) و یک ریل کناری با چهار گروه منو دارد.
- پر شدن حجم پایگاه را فقط‌خواندنی می‌کند؛ پایان اعتبار ورود را می‌بندد و پس از مهلت تمدید، داده حذف می‌شود.

برای مدرس

زمان پیشنهادی: ۳۰ دقیقه در آغاز جلسه‌ی سوم. از دانشجوها بخواهید پیش از کلاس حساب بسازند تا وقت کلاس صرف پیامک نشود. اگر از بخش «کلاس» استفاده می‌کنید (فصل ۵۴)، پایگاه داده‌ی دانشجوها خودکار ساخته و داده‌ی نمونه در آن گذاشته می‌شود.

تمرین‌ها

۱. سطح ۱ کدام نام‌ها برای پایگاه داده مجازند و چرا؟ `Tehran`، `2024_data`، `my-project`، `farm_ndvi_1405`، `ab`.

۲. سطح ۱ پنج مقدار اتصال پایگاه خود را بنویسید (به جز رمز). کدام‌یک با پیش‌فرض PostgreSQL فرق دارد؟

۳. سطح ۱ با جستجوی سراسری، صفحه‌ی «سلامت داده» و مقاله‌ی راهنمای «SRID» را پیدا کنید.

۴. سطح ۲ رمز پایگاه را عوض کنید و بلافاصله پرسش «اولین پرسش» را در کنسول SQL اجرا کنید. آیا کنسول هنوز کار می‌کند؟ چرا؟ (راهنمایی: فصل ۴، بخش امنیت)

۵. سطح ۲ با دو پرسش SQL بخش «سطح حرفه‌ای»، سقف اتصال و حداکثر زمان پرسش پایگاه خود را بخوانید.

فصل

۶

اتصال از QGIS، ArcGIS، pgAdmin و برنامه‌ها

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل ۵**

در این فصل یاد می‌گیرید

- پایگاه داده را در QGIS به‌عنوان اتصال PostGIS اضافه کنید و لایه‌ها را باز کنید.
- از pgAdmin، DBeaver و psql و ArcGIS Pro وصل شوید.
- قالب‌های رشته‌ی اتصال (libpq، URL، SQLAlchemy، JDBC، ogr2ogr) را بنویسید.
- خطاهای رایج اتصال را تشخیص دهید و رفع کنید.
- بدانید چرا نباید از کد مرورگر مستقیم به پایگاه داده وصل شد.

سرفصل‌ها

۶-۱ یک پایگاه، همه‌ی نرم‌افزارها

۶-۲ اتصال از QGIS

۶-۳ اتصال از ArcGIS Pro

۶-۴ اتصال از pgAdmin، DBeaver و psql

۶-۵ رشته‌ی اتصال در قالب‌های مختلف

۶-۶ اتصال از برنامه‌ها: یک نگاه سریع

۶-۷ اتصال از GeoServer

۶-۸ چرا از مرورگر مستقیم وصل نشویم

۶-۹ خطاهای رایج اتصال

۶-۱ یک پایگاه، همه‌ی نرم‌افزارها

بزرگ‌ترین مزیت پایگاه داده‌ی PostgreSQL/PostGIS این است که **همه** آن را می‌شناسند. هر نرم‌افزاری که بتواند به PostgreSQL وصل شود، با همان پنج مقدار اتصال به پایگاه داده مکانی ابری شما هم وصل می‌شود: QGIS، ArcGIS Pro، GeoServer، pgAdmin، DBeaver، FME، Power BI، اسکریپت پایتون، برنامه‌ی Node.js و ده‌ها ابزار دیگر. داده یک جا می‌ماند و هر کس با ابزار دلخواهش روی آن کار می‌کند.

کارت «اطلاعات اتصال» در صفحه‌ی «نمای کلی». هر مقدار دکمه‌ی کپی دارد.

۶-۲ اتصال از QGIS

QGIS نرم‌افزار رومیزی GIS متن‌باز و رایگان است و بهترین همراه پایگاه داده مکانی ابری برای ویرایش پیشرفته، کارتوگرافی چاپی و تحلیل‌های رستری. اتصال یک بار ساخته می‌شود و QGIS آن را به خاطر می‌سپارد.

۱. در QGIS، در پنل **Browser** روی **PostgreSQL** راست‌کلیک کنید و **New Connection…** را بزنید.

۲. در **Name** یک نام دلخواه بنویسید، مثلاً «تهران ابری».

۳. **Host**، **Port** و **Database** را از کارت «اطلاعات اتصال» کپی کنید. پورت را دقیقاً همان بنویسید که نمایش داده می‌شود.

۴. **SSL mode** را روی **require** بگذارید.

۵. در بخش **Authentication**، زبانه‌ی **Basic** را باز کنید، **User name** و **Password** را وارد و تیک **Store** را بزنید.

۶. **Test Connection** را بزنید. پیام موفقیت را که دیدید، **OK** کنید.

۷. اتصال در Browser زیر PostgreSQL ظاهر می‌شود. آن را باز کنید، schemaی `public` را باز کنید و جدول `schools` را روی بوم نقشه بکشید.

> **نکته**
>
> **Store کردن رمز.** اگر تیک Store را نزنید، QGIS هر بار رمز را می‌پرسد. اگر بزنید، رمز در پروفایل QGIS ذخیره می‌شود؛ برای امنیت بیشتر از **Authentication Configuration** (پیکربندی احراز هویت رمزنگاری‌شده با رمز اصلی) استفاده کنید. فایل پروژه‌ی QGIS که با دیگران به اشتراک می‌گذارید نباید رمز داشته باشد.

کار با لایه‌ها در QGIS

- **باز کردن:** کشیدن جدول از Browser به بوم یا دوبار کلیک. جدول‌های بدون ستون هندسه به‌عنوان جدول ویژگی باز می‌شوند.

- **ویرایش:** آیکون مداد (Toggle Editing). تغییرها با Save Layer Edits مستقیم در پایگاه داده نوشته می‌شوند و همکاران بلافاصله می‌بینند. برای ویرایش، جدول باید کلید اصلی داشته باشد (فصل ۷، سلامت داده).
- **پرسش SQL:** منوی **DB Manager** ← **Database**، اتصال را انتخاب و **SQL Window** را باز کنید. نتیجه‌ی هر پرسشی که ستون هندسه داشته باشد را با **Load as new layer** روی نقشه بیاورید.
- **ورود لایه:** در DB Manager، **Import Layer/File** هر لایه‌ی باز در QGIS را به جدول تازه در پایگاه تبدیل می‌کند (فصل ۷).

در دنیای واقعی

تیم پنج‌نفره، یک داده. در یک شرکت مشاور، پنج کارشناس روی لایه‌ی کاربری اراضی یک شهر کار می‌کنند. هر نفر از QGIS به همان جدول در پایگاه داده مکانی ابری وصل است و محدوده‌ی خودش را ویرایش می‌کند. مدیر پروژه در مرورگر و بدون نصب QGIS پیشرفت کار را روی نقشه می‌بیند و با یک پرسش SQL مساحت هر کاربری را لحظه‌ای گزارش می‌گیرد. نسخه‌ی «نهایی_۳_اصلاح‌شده.shp» دیگر وجود ندارد.

۶-۳ اتصال از ArcGIS Pro

ArcGIS Pro هم به PostgreSQL/PostGIS وصل می‌شود. در پنل **Catalog** روی **Databases** راست‌کلیک و **New Database Connection** را بزنید؛ **Database Platform** را PostgreSQL، **Instance** را به شکل `host,port` (با ویرگول) و نام کاربری، رمز و نام پایگاه را وارد کنید. جزئیات رابط بسته به نسخه‌ی ArcGIS فرق می‌کند؛ از مستندات Esri برای نسخه‌ی خودتان کمک بگیرید.

۶-۴ اتصال از pgAdmin، DBeaver و psql

pgAdmin ابزار رسمی مدیریت PostgreSQL و **DBeaver** ابزار عمومی پایگاه داده است که نمایش هندسه روی نقشه را هم دارد. در هر دو یک **Server** یا **Connection** تازه از نوع PostgreSQL بسازید، پنج مقدار را وارد کنید و در زبانه‌ی SSL حالت را **require** بگذارید.

psql خط فرمان PostgreSQL است و برای اسکریپت‌نویسی و کارهای مدیریتی بهترین ابزار است:

```
psql "host=db.example.com port=5439 dbname=gp_tehran_gis
user=gp_tehran_gis_user sslmode=require"
```

psql رمز را می‌پرسد. دستورهای مفید psql که با بک‌اسلش شروع می‌شوند: `\dt` (فهرست جدول‌ها)، `\d schools` (ساختار یک جدول)، `\dx` (افزونه‌های نصب‌شده) و `\q` (خروج).

۶-۵ رشته‌ی اتصال در قالب‌های مختلف

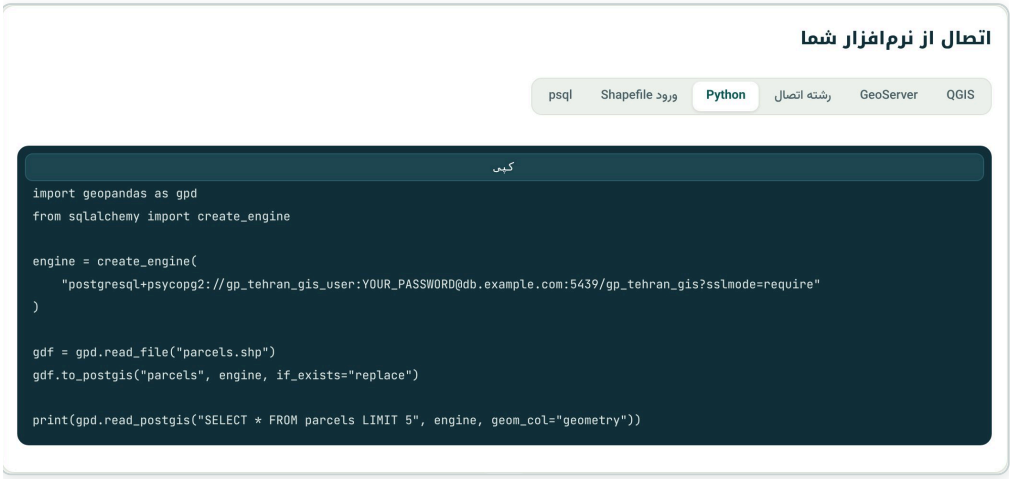
برنامه‌ها اطلاعات اتصال را در قالب یک رشته می‌گیرند. همه‌ی قالب‌ها همان پنج مقدار به‌علاوه‌ی `sslmode=require` را دارند:

قالب	نمونه
کلید-مقدار (libpq)	`.com port=5439 dbname=gp_tehran_gis user=gp_tehran_gis_user sslmode=require`
نشانی URL	`/gp_tehran_gis_user:PASS@db.example.com:5439/gp_tehran_gis?sslmode=require`
SQLAlchemy (پایتون)	`/gp_tehran_gis_user:PASS@db.example.com:5439/gp_tehran_gis?sslmode=require`
JDBC (جاوا، GeoServer)	`:gresql://db.example.com:5439/gp_tehran_gis?sslmode=require`
GDAL/ogr2ogr	`.com port=5439 dbname=gp_tehran_gis user=gp_tehran_gis_user sslmode=require"`

هشدار

نویسه‌های خاص رمز در URL. اگر رمز شما نویسه‌هایی مثل `@`، `:`، `/` یا `#` دارد، در قالب URL باید آن‌ها را کدگذاری کنید (`@` می‌شود `%40`). وگرنه برنامه بخشی از رمز را نام سرور می‌پندارد. قالب کلید-مقدار این مشکل را ندارد.

کارت «اتصال از نرم‌افزار شما» در صفحه‌ی «نمای کلی» همین قالب‌ها و نمونه‌ی کد آماده را با مقادیر پایگاه شما نشان می‌دهد:



زبانه‌ی Python در کارت «اتصال از نرم‌افزار شما»: خواندن یک Shapefile با GeoPandas، نوشتن در پایگاه و خواندن با SQL.

۶-۶ اتصال از برنامه‌ها: یک نگاه سریع

فصل ۵۳ برنامه‌نویسی با پایگاه داده‌ی مکانی را مفصل آموزش می‌دهد. اینجا فقط کوتاه‌ترین نمونه‌ها را می‌آوریم.

پایتون با GeoPandas:

```
import geopandas as gpd
from sqlalchemy import create_engine

engine = create_engine("postgresql+psycopg2://USER:PASS@HOST:PORT/DB?sslmode=require")
schools = gpd.read_postgis("SELECT * FROM schools WHERE students > 500", engine, geom_col="geom")
print(schools.head())
```

Node.js با بسته‌ی pg:

```
import pg from "pg";
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL,
ssl: { rejectUnauthorized: false }, max: 3 });
const { rows } = await pool.query(
  "SELECT name, ST_AsGeoJSON(geom) AS geom FROM schools WHERE level =
$1", ["ابتدایی"]);
```

دو قاعده را همیشه رعایت کنید: **اتصال را ببندید یا از pool با سقف کم استفاده کنید** تا از سقف اتصال هم‌زمان پلن عبور نکنید، و **مقدار ورودی کاربر را هرگز داخل متن SQL نچسبانید**؛ آن را مثل نمونه‌ی بالا به‌صورت پارامتر (`$1`) بفرستید تا از حمله‌ی تزریق SQL در امان باشید.

۶-۷ اتصال از GeoServer

GeoServer سرور نقشه‌ی متن‌باز و پرکاربرد برای انتشار WMS و WFS است. در GeoServer از منوی **Stores** گزینه‌ی **Add new Store** و سپس **PostGIS** را انتخاب کنید؛ host، port، database، user و passwd را وارد کنید، schema را `public` و SSL را **REQUIRE** بگذارید و **max connections** را کم (مثلاً ۳) نگه دارید. سپس کنار هر جدول **Publish** بزنید. برای کاربرانی که نمی‌خواهند GeoServer نگه دارند، سامانه انتشار WMS/WFS و GeoServer اختصاصی را هم آماده دارد (فصل‌های ۴۵ و ۵۲).

۶-۸ چرا از مرورگر مستقیم وصل نشویم

ممکن است وسوسه شوید در یک صفحه‌ی وب با جاوااسکریپت مستقیم به پایگاه داده وصل شوید. **این کار را نکنید.** هر کسی کد صفحه را ببیند رمز پایگاه را می‌بیند و می‌تواند همه‌ی داده‌ها را بخواند یا پاک کند. برای نمایش داده در وب:

- لایه را **منتشر کنید** (کاشی برداری یا OGC API) و در صورت نیاز با **کلید API** فقط‌خواندنی محافظت کنید (فصل ۴۵)؛
- یا از **REST API** سامانه با کلید محدود استفاده کنید (فصل ۴۶)؛
- یا یک سرور میانی خودتان بسازید که رمز را نگه دارد و فقط پرسش‌های مجاز را اجرا کند.

۶-۹ خطاهای رایج اتصال

پیام خطا	علت محتمل	راه حل
`password authentication failed`	رمز یا نام کاربری اشتباه	نام کاربری را دقیق کپی کنید؛ اگر رمز را نمی‌دانید رمز تازه بسازید (فصل ۵)
`no pg_hba.conf entry … SSL off`	اتصال بدون SSL	`sslmode=require` را تنظیم کنید
`connection refused` یا `timeout`	پورت اشتباه یا فایروال شبکه‌ی شما	پورت را از کارت اتصال کپی کنید (۵۴۳۲ نیست)؛ با شبکه‌ی دیگری امتحان کنید
`too many connections for role`	عبور از سقف اتصال هم‌زمان پلن	پنجره‌ها و اسکریپت‌های باز را ببندید؛ برای برنامه از PgBouncer استفاده کنید (فصل ۵۱)
`database … does not exist`	نام پایگاه بدون پیشوند `gp_`	نام کامل را کپی کنید
`permission denied for table`	نقش استفاده‌شده دسترسی ندارد	با نقش مالک وصل شوید یا در «تیم» دسترسی بدهید (فصل ۵۰)

خودتان امتحان کنید

از QGIS به پایگاه خود وصل شوید و جدول `schools` را باز کنید (اگر هنوز داده وارد نکرده‌اید، پس از فصل ۷ برگردید). در DB Manager پرسش `SELECT * FROM schools WHERE students > 600` را اجرا و با **Load as new layer** روی نقشه بیاورید. چند مدرسه پیدا شد؟

خلاصه‌ی فصل

- هر نرم‌افزاری که PostgreSQL را بشناسد با پنج مقدار اتصال و `sslmode=require` به پایگاه وصل می‌شود.
- در QGIS: Browser ← PostgreSQL ← New Connection؛ DB Manager برای SQL و ورود لایه.
- رشته‌ی اتصال قالب‌های کلید-مقدار، JDBC، SQLAlchemy، URL و GDAL دارد؛ نویسه‌های خاص رمز را در URL کدگذاری کنید.
- در برنامه‌ها اتصال را ببندید یا pool کوچک بسازید و ورودی کاربر را پارامتری بفرستید.
- رمز پایگاه داده هرگز نباید در کد مرورگر باشد؛ برای وب لایه را منتشر کنید یا از API استفاده کنید.

برای مدرس

زمان پیشنهادی: ۶۰ دقیقه‌ی عملی. QGIS را از پیش روی رایانه‌های کلاس نصب کنید. یک خطای عمدی (SSL خاموش یا پورت ۵۴۳۲) در اتصال یکی از رایانه‌ها بگذارید و از دانشجوها بخواهید با جدول خطاها علت را پیدا کنند.

تمرین‌ها

۱. سطح ۱ از QGIS به پایگاه خود وصل شوید و همه‌ی لایه‌های داده‌ی نمونه را روی یک نقشه بیاورید.

۲. سطح ۱ رشته‌ی اتصال پایگاه خود را در قالب URL بنویسید، با فرض اینکه رمز `Ab@12345cd` است.

۳. سطح ۲ با psql وصل شوید و با `\d schools` ساختار جدول را ببینید. ستون هندسه چه نوع و SRID دارد؟

۴. سطح ۲ در QGIS یک مدرسه را جابه‌جا و ذخیره کنید، سپس در مرورگر سامانه (فصل ۸) ببینید تغییر اعمال شده است.

۵. سطح ۳ با نمونه‌ی پایتون این فصل، تعداد مدرسه‌های هر مقطع را با GeoPandas بخوانید و یک نمودار میله‌ای بکشید.

فصل

۷

ورود داده و سلامت داده

سطح: **مبتدی** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل‌های ۲، ۳ و ۵**

در این فصل یاد می‌گیرید

- فایل‌های CSV، KML، GeoJSON، GeoPackage، Shapefile و DXF را وارد پایگاه کنید.
- سیستم مختصات، کدگذاری متن فارسی و حالت ورود (جدید، جایگزینی، افزودن) را درست تنظیم کنید.
- فایل را به جدول موجود با نگاشت ستون‌ها اضافه کنید.
- فایل‌های بزرگ را با ogr2ogr، QGIS و shp2pgsql وارد کنید.
- مشکلات سلامت داده را با «سلامت داده» و معادل SQL آن پیدا و رفع کنید.

سرفصل‌ها

۷-۱ داده از کجا می‌آید

۷-۲ ورود فایل در سه گام

۷-۳ افزودن فایل به جدول موجود

۷-۴ فایل‌های بزرگ: ogr2ogr، QGIS و shp2pgsql

۷-۵ سلامت داده

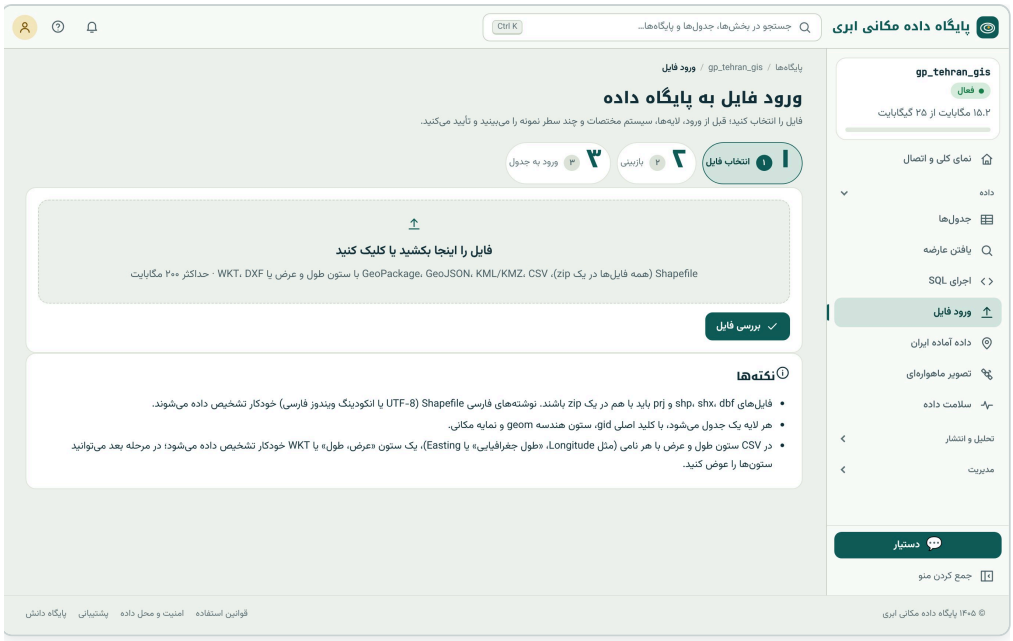
۷-۱ داده از کجا می‌آید

پیش از تحلیل باید داده داشته باشید. چهار راه اصلی برای آوردن داده به پایگاه داده مکانی ابری هست و هر کدام در یک فصل آمده است:

راه	مناسب برای	فصل
ورود فایل در مرورگر	فایل‌های موجود تا ۲۰۰ مگابایت	همین فصل
ابزارهای رومیزی (QGIS، ogr2ogr)	فایل‌های بزرگ و کار خودکار	همین فصل
داده‌ی آماده‌ی ایران	معابر، ساختمان‌ها، کاربری‌ها و مرزها از OpenStreetMap	۳۰
تصویر ماهواره‌ای	Landsat، Sentinel-2 و DEM	۳۴
برداشت میدانی	نقطه‌های GNSS و فرم میدانی	۴۸
ساخت جدول و ویرایش دستی	داده‌ی کم یا تازه	۹

۷-۲ ورود فایل در سه گام

منوی کناری ← داده ← «ورود فایل». این صفحه یک فرایند سه‌مرحله‌ای است: **انتخاب فایل**، **بازبینی** و **ورود به جدول**.



گام اول: انتخاب فایل. قالب‌های پشتیبانی‌شده و حداکثر حجم زیر کادر آمده است.

قالب‌های پذیرفته‌شده

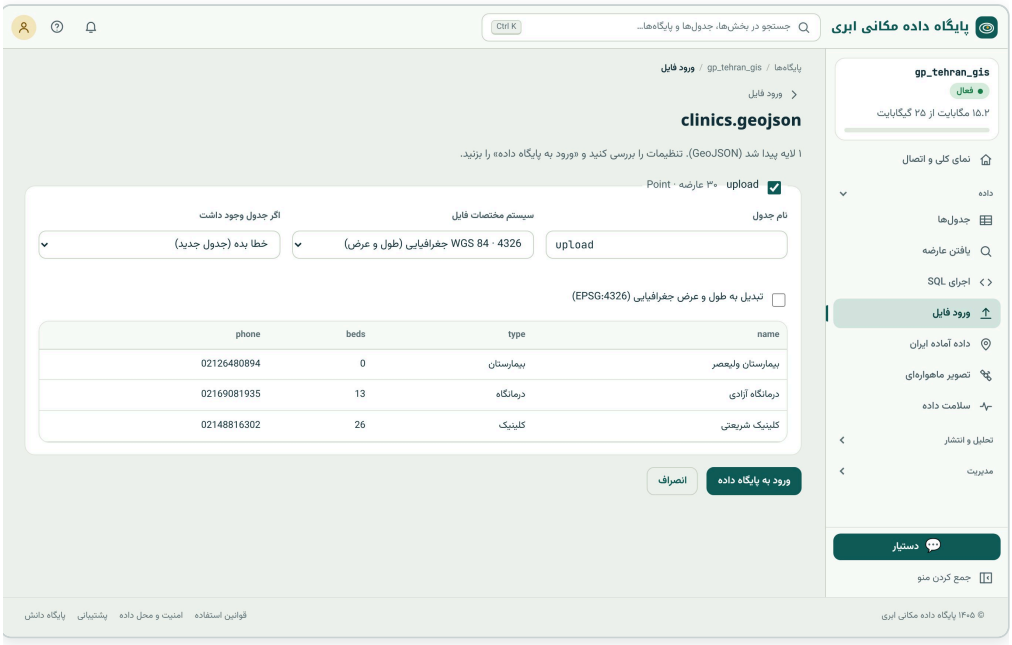
- **Shapefile:** همه‌ی فایل‌های همراه (`.shp`، `.shx`، `.dbf` و در صورت وجود `.prj`، `.cpg`) را کنار هم در **یک فایل zip** بگذارید و zip را بفرستید. نبودن `.prj` یعنی سیستم مختصات تشخیص داده نمی‌شود.
- **GeoPackage** (`.gpkg`): همه‌ی لایه‌های داخلش فهرست می‌شوند.
- **GeoJSON، KML/KMZ:** مستقیم.
- **CSV:** با ستون طول و عرض یا با یک ستون WKT (فصل ۲).
- **DXF:** نقشه‌های CAD؛ لایه‌های DXF جدا می‌شوند.

حداکثر حجم هر فایل ۲۰۰ مگابایت است و حجم نهایی جدول‌ها باید در سهمیه‌ی پلن جا شود.

بازبینی پیش از ورود

پس از «بررسی فایل»، سامانه فایل را باز می‌کند و برای هر لایه‌ی داخلش این‌ها را نشان می‌دهد: تعداد عارضه، نوع هندسه، سیستم مختصات تشخیص‌داده‌شده و چند سطر نمونه.

اینجا جایی است که بیشترین خطاها را می‌توان پیش از ورود گرفت.



گام دوم: بازبینی. نام جدول، سیستم مختصات، حالت ورود و نمونه‌ی سطرها پیش از نوشتن در پایگاه.

۱ **نام جدول:** با حروف کوچک انگلیسی، عدد و زیرخط. نام‌های پیش‌فرض (مثل `upload`) را به نام معنادار عوض کنید: `health_clinics`.

۲ **سیستم مختصات فایل:** اگر از `.prj` خوانده شده، بررسی کنید منطقی باشد؛ اگر نه، سیستم درست را انتخاب کنید. یادتان باشد که این گزینه سیستم **فایل** را می‌گوید، نه سیستم مقصد (فصل ۳).

۳ **«تبدیل به طول و عرض جغرافیایی (EPSG:4326)»:** اگر داده‌ی شما UTM است و می‌خواهید در ۴۳۲۶ ذخیره شود، این تیک را بزنید.

۴ **اگر جدول وجود داشت:** «خطا بده» (پیش‌فرض و امن)، «جایگزین کن» یا «به انتهایش اضافه کن».

۵ **کدگذاری نوشته‌ها:** اگر متن فارسی در نمونه‌ها به‌هم‌ریخته است (مثل `ÇíÑÇä`)، کدگذاری را روی «ویندوز فارسی/عربی (CP1256)» بگذارید. Shapefileهای قدیمی معمولاً همین کدگذاری را دارند.

۶ «ورود به پایگاه داده» را بزنید. فایل‌های بزرگ در پس‌زمینه وارد می‌شوند و پایان کار با اعلان خبر داده می‌شود.

هشدار

«جایگزین کن» برگشت‌پذیر نیست. این گزینه جدول قبلی را حذف و جدول تازه را جایش می‌سازد. اگر جدول مهم است، پیش از جایگزینی یک بکاپ دستی بگیرید (فصل ۵۱).

CSV نقطه‌ای

برای فایل CSV، سامانه ستون‌های طول و عرض را از روی نام (مثل `lon`، `Longitude`، `X`، «طول جغرافیایی»، `Easting`) یا از روی مقدارها تشخیص می‌دهد. اگر درست تشخیص داده نشد، در صفحه‌ی بازبینی ستون طول و عرض را خودتان انتخاب کنید. دو خطای رایج:

- **جابه‌جایی طول و عرض:** نقطه‌ها در جای آینه‌ای و دور می‌افتند. ستون‌ها را جابه‌جا انتخاب کنید.
- **مختصات متری با SRID درجه‌ای:** اگر مقدارها صدها هزار و میلیونی‌اند، سیستم را UTM ناحیه‌ی درست بگذارید (فصل ۳).

پشت صحنه‌ی ورود

ورود فایل در سامانه با کتابخانه‌ی **GDAL** انجام می‌شود که استاندارد جهانی خواندن قالب‌های مکانی است. برای هر لایه:

- یک جدول با ستون کلید اصلی `gid` (عدد صحیح خودافزا) و ستون هندسه با نام `geom` ساخته می‌شود؛
- نام ستون‌ها به حروف کوچک برگردانده می‌شود؛
- **شاخص مکانی** روی `geom` ساخته می‌شود؛
- سطرها در یک تراکنش نوشته می‌شوند: اگر وسط کار خطایی رخ دهد، هیچ سطری نوشته نمی‌شود.

۷-۳ افزودن فایل به جدول موجود

گاهی جدول را از قبل دارید و فقط داده‌ی تازه، مثلاً برداشت این ماه، باید به آن اضافه شود. در صفحه‌ی همان جدول (فصل ۸) دکمه‌ی «ورود فایل به این جدول» را بزنید. پس از انتخاب فایل:

- هر ستون جدول را به یک ستون فایل **نگاشت** می‌کنید (ستون‌های هم‌نام خودکار وصل می‌شوند)؛
- «افزودن» یا «جایگزینی کل داده‌ها» را انتخاب می‌کنید؛
- هندسه خودکار به SRID جدول تبدیل می‌شود؛
- ورود **همه‌یاهیچ** است: اگر یک سطر با قیدهای جدول (مثلاً نوع ستون یا کلید یکتا) جور نباشد، هیچ سطری اضافه نمی‌شود و سطر مشکل‌دار گزارش می‌شود.

۷-۴ فایل‌های بزرگ: QGIS، ogr2ogr و shp2pgsql

برای فایل‌های بزرگ‌تر از ۲۰۰ مگابایت، یا وقتی ورود باید خودکار و تکراری باشد، از رایانه‌ی خودتان مستقیم به پایگاه وصل شوید.

QGIS DB Manager: لایه را در QGIS باز کنید، **DB Manager** ← **Database**، اتصال را انتخاب و **Import Layer/File** را بزنید. نام جدول را بنویسید و تیک‌های **Create spatial index** و **Convert field names to lowercase** را بزنید. اگر می‌خواهید SRID عوض شود، **Target SRID** را تعیین کنید.

ogr2ogr ابزار خط فرمان GDAL است و سریع‌ترین و انعطاف‌پذیرترین راه:

```
ogr2ogr -f PostgreSQL \
  PG:"host=db.example.com port=5439 dbname=gp_tehran_gis
user=gp_tehran_gis_user sslmode=require" \
  roads.gpkg -nln roads \
  -nlt PROMOTE_TO_MULTI \
  -lco GEOMETRY_NAME=geom -lco FID=gid \
  -t_srs EPSG:4326 \
  -progress
```

گزینه	معنا
`-nln roads`	نام جدول مقصد
`-nlt PROMOTE_TO_MULTI`	همه‌ی هندسه‌ها چندبخشی شوند تا لایه‌ای که هم Polygon و هم MultiPolygon دارد خطا ندهد
`-lco GEOMETRY_NAME=geom`	نام ستون هندسه
`-lco FID=gid`	نام ستون کلید اصلی
`-t_srs EPSG:4326`	سیستم مختصات مقصد (تبدیل در حین ورود)
`-s_srs EPSG:32639`	سیستم مختصات مبدأ، اگر فایل `.prj` ندارد
`-append` یا `-overwrite`	افزودن به جدول موجود یا جایگزینی آن
`--config SHAPE_ENCODING CP1256`	کدگذاری Shapefile فارسی قدیمی

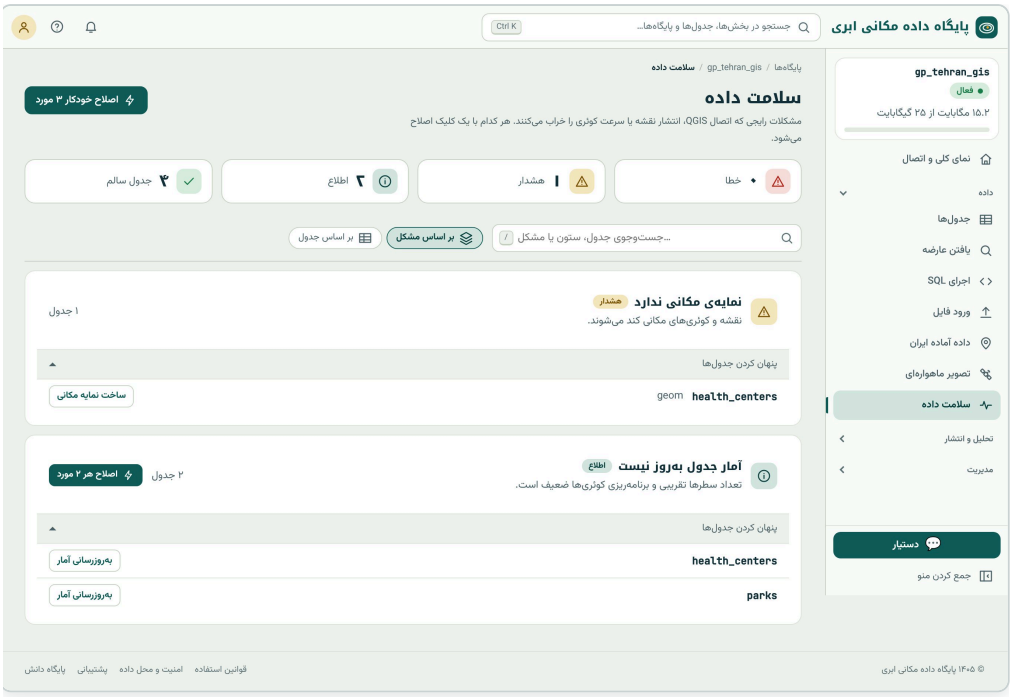
shp2pgsql ابزار همراه PostGIS است که Shapefile را به دستورهای SQL تبدیل می‌کند و خروجی‌اش را به psql می‌دهیم:

```
shp2pgsql -s 32639:4326 -I -W CP1256 parcels.shp parcels | psql "host=… sslmode=require"
```

`-s 32639:4326` یعنی فایل در UTM ۳۹ است و در ۴۳۲۶ ذخیره شود، `-I` شاخص مکانی می‌سازد و `-W` کدگذاری را تعیین می‌کند.

۷-۵ سلامت داده

داده‌ای که وارد می‌شود همیشه سالم نیست. منوی کناری ← داده ← «سلامت داده» همه‌ی جدول‌ها را بررسی و مشکلات را بر اساس شدت دسته‌بندی می‌کند: **خطا**، **هشدار** و **اطلاع**. کنار هر مشکل دکمه‌ی رفع آن هست و «اصلاح خودکار» همه‌ی موارد قابل رفع را یک‌جا اصلاح می‌کند.



صفحه‌ی «سلامت داده». در داده‌ی نمونه، جدول مراکز درمانی شاخص مکانی ندارد و آمار دو جدول به‌روز نیست.

هر بررسی «سلامت داده» یک پرسش SQL پشت صحنه است و دانستن آن کمک می‌کند در هر پایگاه داده‌ای همین بررسی‌ها را انجام دهید:

۱. جدول بدون کلید اصلی

بدون کلید اصلی، QGIS و مرورگر سامانه نمی‌توانند سطرها را ویرایش کنند و انتشار لایه هم مشکل پیدا می‌کند.

```sql
SELECT c.relname AS table_without_pk
FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r' AND n.nspname = 'public' AND c.relname <>
'spatial_ref_sys'
  AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conrelid = c.oid
AND contype = 'p');
```

table_without_pk

نتیجه‌ی خالی یعنی همه‌ی جدول‌ها کلید اصلی دارند. رفع: `ALTER TABLE t ADD COLUMN gid serial PRIMARY KEY;`

۲. ستون هندسه بدون شاخص مکانی

بدون شاخص مکانی، هر پرسش مکانی همه‌ی سطرها را یکی‌یکی بررسی می‌کند و نقشه کند بارگذاری می‌شود (فصل ۲۰).

```sql
SELECT g.f_table_name AS layer
FROM geometry_columns g
WHERE NOT EXISTS (
  SELECT 1 FROM pg_indexes i
  WHERE i.tablename = g.f_table_name AND i.indexdef ILIKE '%gist%' ||
g.f_geometry_column || '%');
```

layer
health_centers
ndvi_parcels_latest

رفع:

```sql
CREATE INDEX ON health_centers USING gist (geom);
```

۳. هندسه‌ی نامعتبر

چندضلعی‌های خودتقاطع یا حلقه‌های باز باعث می‌شوند توابعی مثل `ST_Intersection` و `ST_Union` خطا بدهند یا نتیجه‌ی غلط برگردانند.

```sql
SELECT 'parks' AS layer, count(*) FILTER (WHERE NOT
ST_IsValid(geom)) AS invalid, count(*) AS total
FROM parks
UNION ALL
SELECT 'neighborhoods', count(*) FILTER (WHERE NOT ST_IsValid(geom)),
count(*)
FROM neighborhoods;
```

layer	invalid	total
parks	0	35
neighborhoods	0	6

رفع: `UPDATE parks SET geom = ST_MakeValid(geom) WHERE NOT ST_IsValid(geom);` (فصل ۲۲ جزئیات و احتیاط‌های آن را می‌گوید).

۴. SRID تعریف‌نشده

هندسه‌ای با SRID صفر نمی‌داند کجای زمین است؛ نه روی نقشه درست می‌نشیند و نه با لایه‌های دیگر قابل تحلیل است.

```sql
SELECT f_table_name, srid FROM geometry_columns WHERE srid = 0;
```

f_table_name	srid

رفع (اگر می‌دانید داده در ۴۳۲۶ است): `SELECT UpdateGeometrySRID('mytable', 'geom', 4326);`

۵. آمار قدیمی

PostgreSQL برای انتخاب بهترین روش اجرای هر پرسش از **آمار** جدول (تعداد سطر، توزیع مقادیر) استفاده می‌کند. پس از ورود یا تغییر حجم زیادی داده، آمار کهنه می‌شود. رفع: `ANALYZE health_centers;`

سطح حرفه‌ای

قید برای پیشگیری. به جای اینکه هر بار سلامت داده را بررسی کنید، می‌توانید با **قید** (constraint) جلوی ورود داده‌ی بد را بگیرید: `ALTER TABLE parks ADD CONSTRAINT parks_valid CHECK (ST_IsValid(geom));` از این پس هر درج یا ویرایشی که هندسه‌ی نامعتبر بسازد رد می‌شود. ستون هندسه را هم با نوع و SRID مشخص تعریف کنید، مثلاً `geometry(MultiPolygon, 4326)` ، تا هندسه‌ی نوع دیگر یا SRID دیگر پذیرفته نشود (فصل ۱۴).

خودتان امتحان کنید

داده‌ی نمونه‌ی کتاب را وارد پایگاه خود کنید. سه فایل `schools.geojson`، `metro_stations.geojson` و `parks.geojson` همراه کتاب است. هر کدام را با همان نام جدول وارد کنید. سپس به «سلامت داده» بروید و همه‌ی مشکلات را با «اصلاح خودکار» رفع کنید. جدول‌های `neighborhoods`، `health_centers` و `parcels` را در فصل ۱۴ با SQL خودتان می‌سازید.

خلاصه‌ی فصل

- ورود فایل سه گام دارد: انتخاب، بازبینی (نام، سیستم مختصات، حالت، کدگذاری) و ورود؛ حداکثر ۲۰۰ مگابایت.
- Shapefile را با همه‌ی فایل‌های همراهش zip کنید؛ متن فارسی به‌هم‌ریخته یعنی کدگذاری CP1256.
- «جایگزین کن» جدول قبلی را حذف می‌کند؛ پیش از آن بکاپ بگیرید.
- فایل‌های بزرگ را با QGIS DB Manager، ogr2ogr یا shp2pgsql مستقیم وارد کنید.
- «سلامت داده» نبود کلید اصلی، نبود شاخص مکانی، هندسه‌ی نامعتبر، SRID صفر و آمار کهنه را پیدا و رفع می‌کند.

برای مدرس

زمان پیشنهادی: ۷۵ دقیقه. یک Shapefile فارسی قدیمی (با کدگذاری CP1256) و یک CSV با مختصات UTM آماده کنید و بگذارید دانشجوها خودشان به خطا بخورند و با بخش «بازبینی» رفعش کنند. این دو خطا رایج‌ترین مشکلات دنیای واقعی‌اند.

تمرین‌ها

۱. سطح ۱ داده‌ی نمونه را وارد کنید و تعداد عارضه‌ی هر جدول را با جدول «داده‌ی نمونه» در راهنمای کتاب مقایسه کنید.

۲. سطح ۱ یک فایل CSV با سه ستون `name`، `lat` و `lon` و پنج مکان شهر خودتان در اکسل بسازید و وارد کنید.

۳. سطح ۲ همان CSV را با طول و عرض جابه‌جا وارد کنید. نقطه‌ها کجا افتادند؟ چرا؟

۴. سطح ۲ پرسش «هندسه‌ی نامعتبر» را برای جدول `parcels` بنویسید و اجرا کنید.

۵. سطح ۳ یک دستور ogr2ogr بنویسید که فایل GeoPackage با لایه‌ای در UTM ناحیه‌ی ۴۰ را به جدول `buildings` در ۴۳۲۶ وارد و به جدول موجود اضافه کند.

فصل

۸

مرور، جست‌وجو و خروجی گرفتن از داده

سطح: **مبتدی** ●●○ | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل ۷**

در این فصل یاد می‌گیرید

- فهرست جدول‌ها را با جست‌وجو و فیلتر مرور کنید و ویژگی‌های هر جدول را بخوانید.
- داده، ستون‌ها و نقشه‌ی هر جدول را ببینید و چند جدول را با هم روی نقشه نمایش دهید.
- با «یافتن عارضه» شرط ویژگی و شرط مکانی بگذارید و نتیجه را به لایه‌ی تازه تبدیل کنید.
- از جدول‌ها در قالب‌های CSV، Shapefile، GeoPackage، GeoJSON و KML خروجی بگیرید.

سرفصل‌ها

۸-۱ مرورگر داده

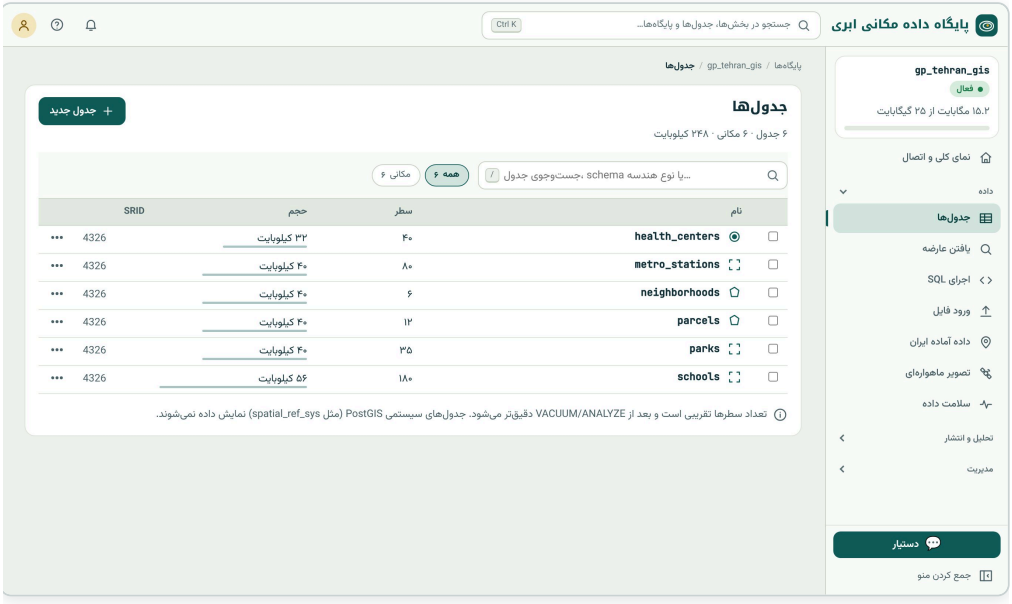
۸-۲ صفحه‌ی هر جدول

۸-۳ یافتن عارضه

۸-۴ خروجی گرفتن

۸-۱ مرورگر داده

بخش «جدول‌ها» در گروه «داده» همان کاری را می‌کند که پنل Browser و جدول ویژگی‌ها در QGIS، اما در مرورگر و بدون نصب هیچ نرم‌افزاری. اینجا جای اولی است که بعد از ورود داده به آن سر می‌زنید.



فهرست جدول‌ها: نام، تعداد سطر، حجم و SRID هر جدول. آیکون کنار نام نوع هندسه را نشان می‌دهد.

در این صفحه:

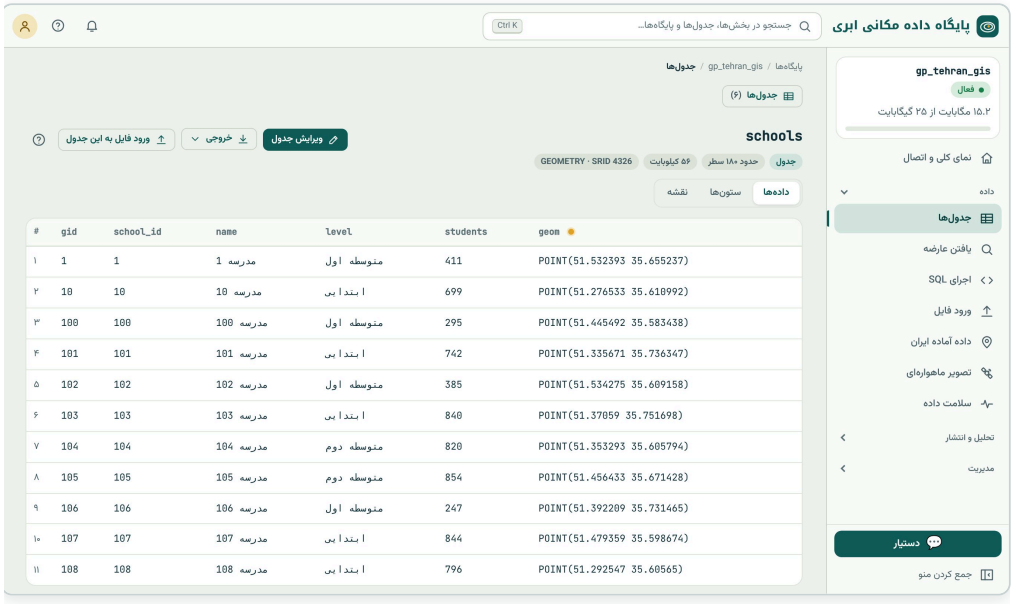
- **جست‌وجو** بر اساس نام جدول، schema یا نوع هندسه؛
- **چیپ‌های فیلتر:** همه، مکانی، غیرمکانی، نما؛
- **مرتب‌سازی** با کلیک روی سربرگ ستون‌ها (مثلاً بزرگ‌ترین جدول‌ها)؛
- **تیک چند جدول** و «نمایش با هم روی نقشه» برای دیدن چند لایه کنار هم، هر کدام با رنگ جدا؛
- **منوی ⋯** هر جدول: خروجی، افزودن به نقشه‌ی چندلایه، انتشار، حذف؛
- دکمه‌ی اصلی «جدول جدید» (فصل ۹).

بدانید

تعداد سطر تقریبی است. ستون «سطر» از آمار PostgreSQL خوانده می‌شود تا صفحه حتی برای جدول‌های میلیونی فوری باز شود. پس از `ANALYZE` (یا رفع «آمار به‌روز نیست» در «سلامت داده») دقیق می‌شود. تعداد دقیق را همیشه با `SELECT count(*) FROM schools;` می‌گیرید.

۸-۲ صفحه‌ی هر جدول

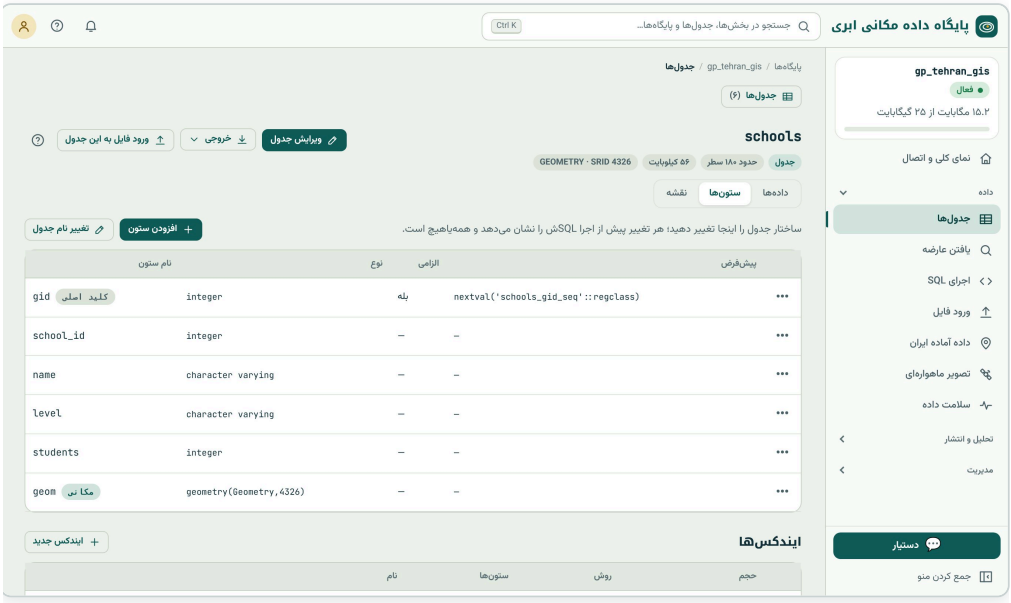
با کلیک روی نام جدول، صفحه‌ی آن باز می‌شود. سربرگ صفحه نوع (جدول یا نما)، تعداد تقریبی سطر، حجم و نوع هندسه با SRID را نشان می‌دهد و سه زبانه دارد: **داده‌ها**، **ستون‌ها** و **نقشه**. دکمه‌های «ویرایش جدول»، «خروجی» و «ورود فایل به این جدول» بالای صفحه‌اند.



زبانه‌ی «داده‌ها»: سطرها ۵۰تا ۵۰تا، هندسه به صورت متن کوتاه WKT.

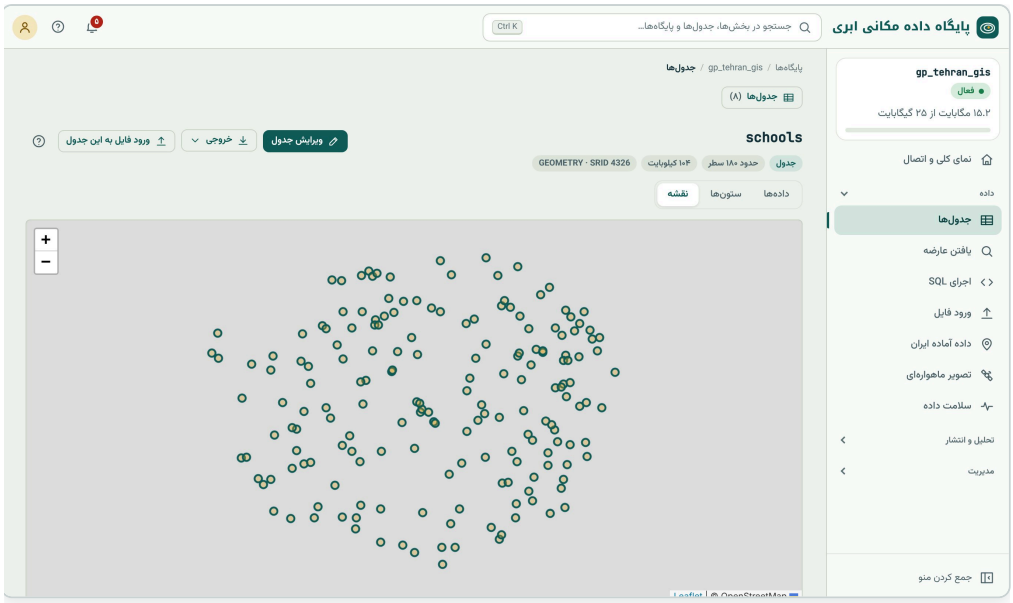
داده‌ها سطرها را در صفحه‌های ۵۰ سطری نشان می‌دهد؛ هندسه‌ها به صورت WKT کوتاه‌شده نمایش داده می‌شوند. با کلیک روی سربرگ هر ستون، مرتب‌سازی عوض می‌شود.

ستون‌ها ساختار جدول را نشان می‌دهد: نام، نوع داده، الزامی بودن و مقدار پیش‌فرض هر ستون، کلید اصلی و ستون مکانی، و فهرست شاخص‌ها. تغییر ساختار (افزودن ستون، تغییر نوع، ساخت شاخص) هم از همین‌جاست (فصل ۹).



زبانه‌ی «ستون‌ها»: ساختار جدول، کلید اصلی و ستون مکانی، و شاخص‌ها.

نقشه عارضه‌های جدول را روی نقشه‌ی پایه نشان می‌دهد. با کلیک روی هر عارضه، ویژگی‌هایش در پنجره‌ی کوچکی می‌آید. داده‌ی هر SRIDی خودکار به ۴۳۲۶ تبدیل می‌شود تا روی نقشه‌ی وب بنشیند.



زبانه‌ی «نقشه»: ۱۸۰ مدرسه‌ی داده‌ی نمونه. (در محیطی که تصویرها گرفته شده نقشه‌ی پایه بارگذاری نشده است.)

> **نکته**
>
> **سقف ۲۰۰۰ عارضه روی نقشه.** برای سریع ماندن مرورگر، زبانه‌ی نقشه حداکثر ۲۰۰۰ عارضه را نشان می‌دهد؛ جدول شما کامل است. برای دیدن همه‌ی عارضه‌های یک جدول بزرگ، آن را در QGIS باز کنید یا در یک نقشه‌ی چندلایه بگذارید که با کاشی برداری کار می‌کند و محدودیت تعداد ندارد (فصل ۴۲).

۸-۳ یافتن عارضه

پرسش «کدام عارضه‌ها چنین شرطی دارند؟» رایج‌ترین پرسش GIS است. بخش «یافتن عارضه» آن را بدون نوشتن SQL پاسخ می‌دهد و ترکیبی از **شرط ویژگی** و **شرط مکانی** را می‌پذیرد.

۱. منوی کناری ← داده ← «یافتن عارضه». در «کدام لایه؟» جدول را انتخاب کنید؛ مثلاً `schools`.

۲. **شرط ویژگی** (اختیاری): «افزودن شرط»، ستون، عملگر و مقدار؛ مثلاً `students` بزرگ‌تر از `500`. چند شرط را می‌توان با «و» یا «یا» ترکیب کرد. برای ستون‌های متنی، مقدارهای

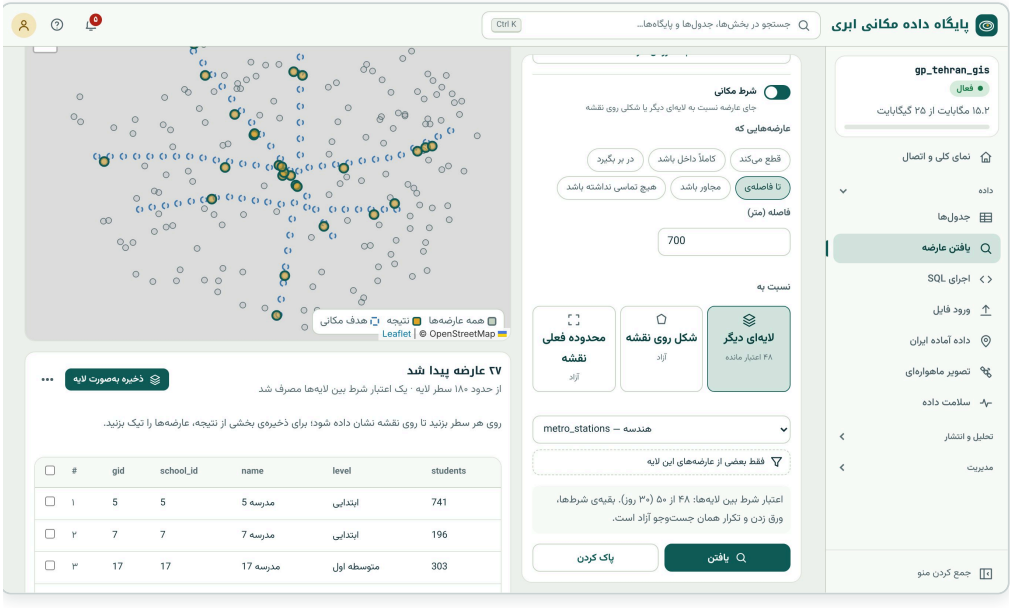
موجود پیشنهاد می‌شوند.

۳ **شرط مکانی** (اختیاری): کلید «شرط مکانی» را روشن کنید و رابطه را انتخاب کنید: قطع می‌کند، کاملاً داخل باشد، در بر بگیرد، تا فاصله‌ی...، مجاور باشد، هیچ تماسی نداشته باشد.

۴ هدف شرط مکانی را تعیین کنید: **لایه‌ای دیگر** (مثلاً ایستگاه‌های مترو، با امکان فیلتر روی آن لایه)، **شکلی که روی نقشه می‌کشید**، یا **محدوده‌ی فعلی نقشه**.

۵ «یافتن» را بزنید. نتیجه روی نقشه برجسته و زیر آن فهرست می‌شود.

۶ برای نگه داشتن نتیجه، «ذخیره به‌صورت لایه» آن را به جدول تازه‌ای در همان پایگاه تبدیل می‌کند. می‌توانید فقط سطرهای تیک‌خورده را ذخیره کنید.



مدرسه‌هایی که تا ۷۰۰ متری یک ایستگاه مترو قرار دارند: ۲۷ مدرسه از ۱۸۰. نتیجه روی نقشه برجسته و زیر آن فهرست شده است.

نکته

اعتبار شرط بین لایه‌ای. شرط مکانی با «لایه‌ای دیگر» سنگین‌تر از بقیه است و هر پلن تعداد مشخصی از آن را در یک دوره‌ی زمانی دارد که روی کاشی مربوط نمایش داده می‌شود. بقیه‌ی شرط‌ها، ورق زدن و تکرار همان جست‌وجو آزاد است. اگر این نوع پرسش را زیاد لازم دارید، معادل SQL آن را یاد بگیرید (پایین همین بخش و فصل ۱۷)؛ کنسول SQL چنین محدودیتی ندارد.

زیر کاپوت: همان پرسش با SQL

«یافتن عارضه» در واقع یک پرسش SQL می‌سازد. همان جست‌وجوی شکل بالا، یعنی «مدرسه‌هایی که تا ۷۰۰ متری دست‌کم یک ایستگاه مترو هستند»، به SQL این است:

```sql
SELECT s.gid, s.name, s.level, s.students
FROM schools s
WHERE EXISTS (
  SELECT 1 FROM metro_stations m
  WHERE ST_DWithin(s.geom::geography, m.geom::geography, 700)
)
ORDER BY s.gid;
```

gid	name	level	students
5	مدرسه 5	ابتدایی	741
7	مدرسه 7	ابتدایی	196
17	مدرسه 17	متوسطه اول	303
20	مدرسه 20	ابتدایی	495
… (27 rows)			

شمار سطرها همان ۲۷ است. تابع `ST_DWithin(a, b, d)` می‌پرسد آیا فاصله‌ی دو هندسه حداکثر `d` است، و تبدیل به `geography` باعث می‌شود فاصله به متر حساب شود (فصل ۳). در فصل ۱۷ همه‌ی رابطه‌های مکانی را با جزئیات یاد می‌گیرید؛ اینجا کافی است بدانید که **هر چیزی که در رابط ساخته می‌شود یک معادل SQL دارد**، و نسخه‌ی SQL هیچ سقف و محدودیتی جز سقف زمان پرسش ندارد.

۸-۴ خروجی گرفتن

دکمه‌ی «خروجی» در صفحه‌ی هر جدول (یا منوی ⋯ در فهرست جدول‌ها) جدول را در یکی از این قالب‌ها دانلود می‌کند:

قالب	مناسب برای	نکته
GeoPackage	تبادل کامل با QGIS و ArcGIS	بهترین انتخاب عمومی؛ نام ستون بلند و متن فارسی سالم
GeoJSON	وب، برنامه‌نویسی	همیشه در ۴۳۲۶
Shapefile (zip)	نرم‌افزارهای قدیمی	نام ستون‌ها به ده نویسه کوتاه می‌شود
CSV	اکسل، گزارش	هندسه به صورت WKT در یک ستون
KML	Google Earth	همراه نام عارضه

هر خروجی حداکثر ۱۰۰٬۰۰۰ سطر دارد. برای جدول‌های بزرگ‌تر، از QGIS (راست‌کلیک روی لایه ← Export) یا ogr2ogr استفاده کنید:

```
ogr2ogr -f GPKG schools.gpkg PG:"host=… sslmode=require" -sql "SELECT * FROM schools WHERE students > 500"
```

نمونه‌ی بالا یک نکته‌ی مهم را هم نشان می‌دهد: با `-sql` می‌توانید **نتیجه‌ی هر پرسشی** را خروجی بگیرید، نه فقط یک جدول کامل.

خودتان امتحان کنید

با «یافتن عارضه» مدرسه‌های ابتدایی با بیش از ۶۰۰ دانش‌آموز را پیدا کنید که **بیش از** یک کیلومتر از نزدیک‌ترین مرکز درمانی فاصله دارند (راهنمایی: رابطه‌ی «تا فاصله‌ی…» را با «هیچ تماسی نداشته باشد» مقایسه کنید؛ کدام برای «بیش از» مناسب است؟). نتیجه را با نام `far_primary_schools` ذخیره کنید و از آن خروجی GeoPackage بگیرید.

خلاصه‌ی فصل

- «جدول‌ها» فهرست جدول‌ها را با تعداد سطر (تقریبی)، حجم و SRID نشان می‌دهد؛ چند جدول را می‌توان با هم روی نقشه دید.
- صفحه‌ی هر جدول سه زبانه دارد: داده‌ها (۵۰ سطری)، ستون‌ها (ساختار و شاخص) و نقشه (تا ۲۰۰۰ عارضه).
- «یافتن عارضه» شرط ویژگی و شرط مکانی (نسبت به لایه، شکل یا محدوده‌ی نقشه) را ترکیب می‌کند و نتیجه را لایه می‌کند.
- هر جست‌وجو یک معادل SQL دارد؛ `ST_DWithin` روی `geography` یعنی «تا فاصله‌ی چند متر».
- خروجی تا ۱۰۰٬۰۰۰ سطر در CSV، Shapefile، GeoJSON، GeoPackage و KML؛ برای بیشتر، QGIS یا ogr2ogr.

برای مدرس

زمان پیشنهادی: ۴۵ دقیقه. پرسش‌های «یافتن عارضه» را به صورت مسابقه‌ی گروهی طرح کنید: هر گروه پرسشی از گروه دیگر بپرسد و جواب را با شمار نتیجه بسنجید. این فصل پل خوبی به SQL است: برای هر جست‌وجو از دانشجوها بخواهید حدس بزنند SQL آن چه شکلی است.

تمرین‌ها

۱. سطح ۱ بزرگ‌ترین جدول پایگاه نمونه از نظر حجم کدام است؟ از نظر تعداد سطر؟

۲. سطح ۱ جدول‌های `parks`، `schools` و `metro_stations` را با «نمایش با هم روی نقشه» کنار هم ببینید.

۳. سطح ۱ با «یافتن عارضه» مراکز درمانی از نوع «بیمارستان» را پیدا کنید. چند مورد است؟

۴. سطح ۲ مدرسه‌هایی را پیدا کنید که داخل یک پارک‌اند. چرا ممکن است نتیجه خالی باشد؟ رابطه‌ی «تا فاصله‌ی ۲۰۰ متر» را امتحان کنید.

۵. سطح ۲ روی نقشه یک چندضلعی دور شمال شهر بکشید و مدرسه‌های داخل آن را پیدا کنید.

۶. سطح ۳ پرسش SQL «زیر کاپوت» را طوری تغییر دهید که فقط مدرسه‌های ابتدایی را برگرداند و فاصله را ۱۰۰۰ متر کنید. حاصل را با «یافتن عارضه» مقایسه کنید.

فصل

۹

ساخت جدول، ویرایش داده و تغییر ساختار

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل ۸**

در این فصل یاد می‌گیرید

- بدون نوشتن SQL جدول مکانی تازه با نوع هندسه، SRID و ستون‌های مناسب بسازید.
- داده را در حالت ویرایش امن (پیش‌نویس، بازبینی، ذخیره‌ی همه‌یاهیچ، بازگردانی) ویرایش کنید.
- هندسه را روی نقشه بکشید و ویرایش کنید.
- ستون اضافه، تغییر نام و نوع، و شاخص بسازید و SQL هر تغییر را بخوانید.

سرفصل‌ها

۹-۱ ساخت جدول تازه

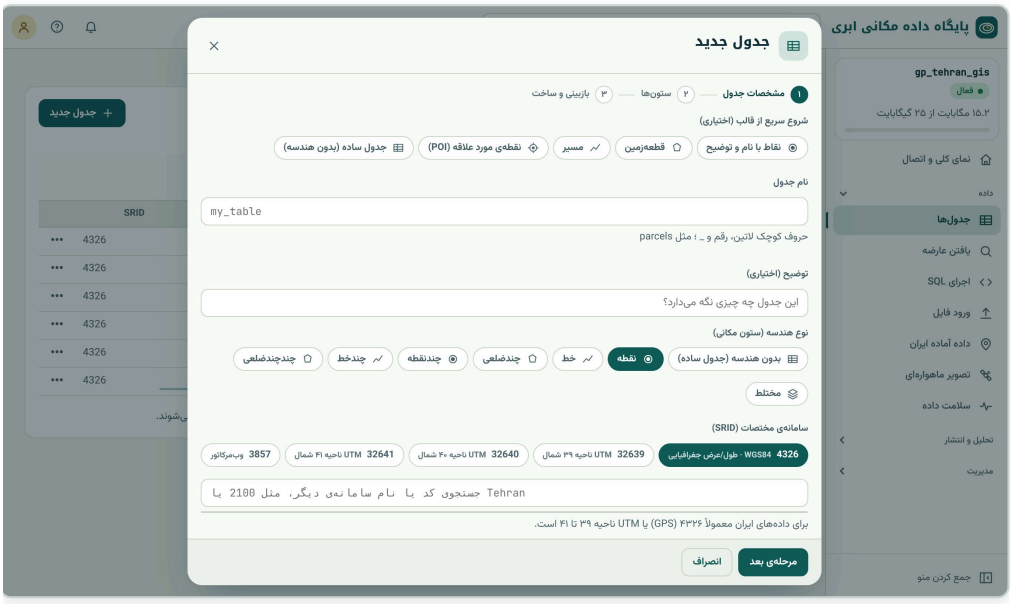
۹-۲ ویرایش داده با پیش‌نویس

۹-۳ تغییر ساختار جدول

۹-۴ حذف جدول

۹-۱ ساخت جدول تازه

گاهی داده از جای دیگری نمی‌آید و باید آن را از صفر بسازید: فهرست چاه‌های یک دشت، محل دوربین‌های ترافیکی یا محدوده‌ی پروژه‌های یک شرکت. دکمه‌ی «جدول جدید» در صفحه‌ی «جدول‌ها» یک مودال سه‌مرحله‌ای باز می‌کند: **مشخصات جدول**، **ستون‌ها** و **بازبینی و ساخت**.



مودال «جدول جدید». قالب‌های آماده، نام، نوع هندسه و سیستم مختصات در گام اول.

۱ **شروع سریع از قالب** (اختیاری): «نقاط با نام و توضیح»، «قطعه‌زمین»، «مسیر»، «نقطه‌ی مورد علاقه (POI)» یا «جدول ساده». هر قالب ستون‌های معمول آن نوع داده را آماده می‌کند.

۲ **نام جدول** با حروف کوچک انگلیسی و زیرخط، مثلاً `wells`، و یک **توضیح** کوتاه که به‌عنوان فراداده روی جدول می‌نشیند.

۳ **نوع هندسه:** بدون هندسه، نقطه، خط، چندضلعی، چندنقطه، چندخط، چندچندضلعی یا مختلط. نوع درست را انتخاب کنید؛ پایگاه هندسه‌ی نوع دیگر را رد می‌کند و این خودش یک کنترل کیفیت است.

۴ **سیستم مختصات:** ۴۳۲۶ برای داده‌ی GPS و عمومی؛ UTM ناحیه‌ی مربوط (۳۲۶۳۸ تا ۳۲۶۴۱) اگر داده‌تان متری است؛ هر کد دیگری با جست‌وجو.

۵ **ستون‌ها:** برای هر ستون نام، نوع (متن، عدد صحیح، عدد اعشاری، تاریخ، بله/خیر و ...)، الزامی بودن و مقدار پیش‌فرض.

۶ **بازبینی و ساخت:** دستور SQLای که اجرا خواهد شد نمایش داده می‌شود. «ساخت» را بزنید.

زیر کاپوت

جدولی که مودال می‌سازد با این SQL معادل است. کد زیر روی پایگاه نمونه اجرا شده، اما چون کتاب هیچ چیزی را در پایگاه نمونه نگه نمی‌دارد، در پایان برگردانده شده است:

```sql
CREATE TABLE wells (
  gid        serial PRIMARY KEY,
  name       text NOT NULL,
  depth_m    numeric(6,1),
  drilled_on date,
  active     boolean DEFAULT true,
  geom       geometry(Point, 4326)
);
CREATE INDEX ON wells USING gist (geom);
COMMENT ON TABLE wells IS 'چاه‌های بهره‌برداری دشت نمونه';

INSERT INTO wells (name, depth_m, drilled_on, geom)
VALUES ('01-05-2019' ,120.5 ,'چاه ۱', ST_SetSRID(ST_MakePoint(51.30,
35.62), 4326));

SELECT gid, name, depth_m, active, ST_AsText(geom) FROM wells;
```

gid	name	depth_m	active	st_astext
1	چاه ۱	120.5	True	POINT(51.3 35.62)

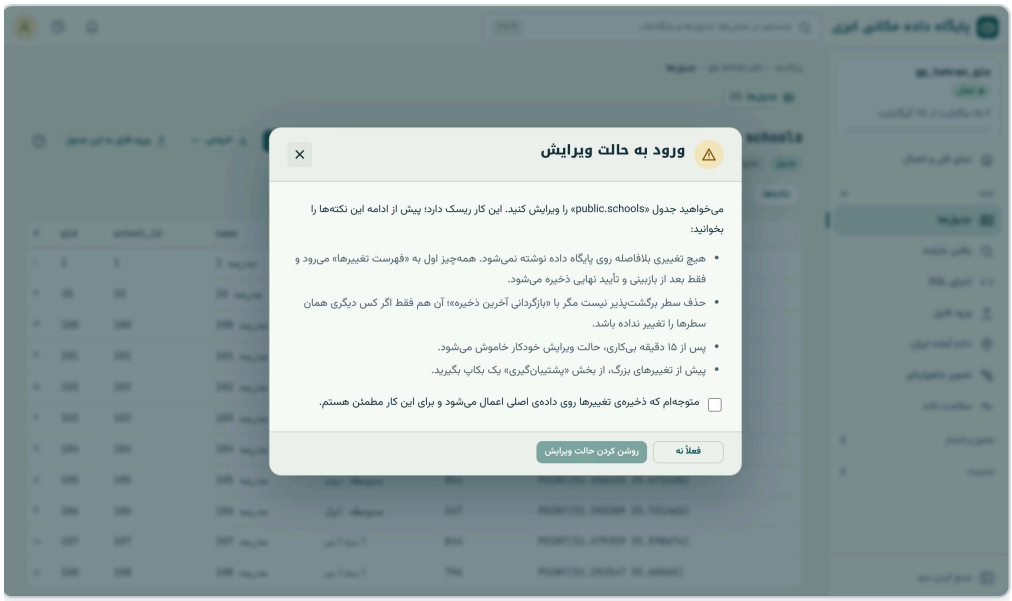
چند نکته در این کد هست که در فصل ۱۴ مفصل یاد می‌گیرید: `serial PRIMARY KEY` کلید اصلی خودافزا می‌سازد؛ `NOT NULL` ستون را الزامی می‌کند؛ `geometry(Point, 4326)` ستون

هندسه را به نوع نقطه و SRID ۴۳۲۶ محدود می‌کند؛ و شاخص GiST برای سرعت پرسش‌های مکانی است.

۹-۲ ویرایش داده با پیش‌نویس

ویرایش مستقیم پایگاه داده‌ای که چند نفر روی آن کار می‌کنند پرخطر است: یک کلیک اشتباه می‌تواند صدها سطر را عوض کند. ویرایشگر سامانه برای همین با چند لایه‌ی ایمنی طراحی شده است:

1. **حالت ویرایش همیشه خاموش شروع می‌شود** و روشن کردنش تأیید می‌خواهد.
2. **هر تغییر اول پیش‌نویس است** و در «فهرست تغییرها» جمع می‌شود؛ هیچ چیز فوری در پایگاه نوشته نمی‌شود.
3. **پیش از ذخیره، بازبینی** مقدار قبلی و جدید هر تغییر را نشان می‌دهد؛ حذف سطر تأیید جداگانه می‌خواهد.
4. **ذخیره همه‌یاهیچ است:** همه‌ی تغییرها در یک تراکنش نوشته می‌شوند؛ اگر یکی رد شود، هیچ‌کدام نوشته نمی‌شود.
5. **تعارض با همکار** تشخیص داده می‌شود: اگر کسی در این فاصله همان سطر را عوض کرده باشد، ذخیره متوقف می‌شود تا کار او از بین نرود.
6. **آخرین ذخیره قابل بازگردانی است.**
7. پس از ۱۵ دقیقه بی‌کاری، حالت ویرایش خودکار خاموش می‌شود.



تأیید ورود به حالت ویرایش. نکته‌های ریسک پیش از شروع گفته می‌شود.

۱ در صفحه‌ی جدول «ویرایش جدول» را بزنید، نکته‌ها را بخوانید، کادر تأیید را تیک بزنید و «روشن کردن حالت ویرایش» را بزنید. نوار «حالت ویرایش» پایین صفحه ظاهر می‌شود.

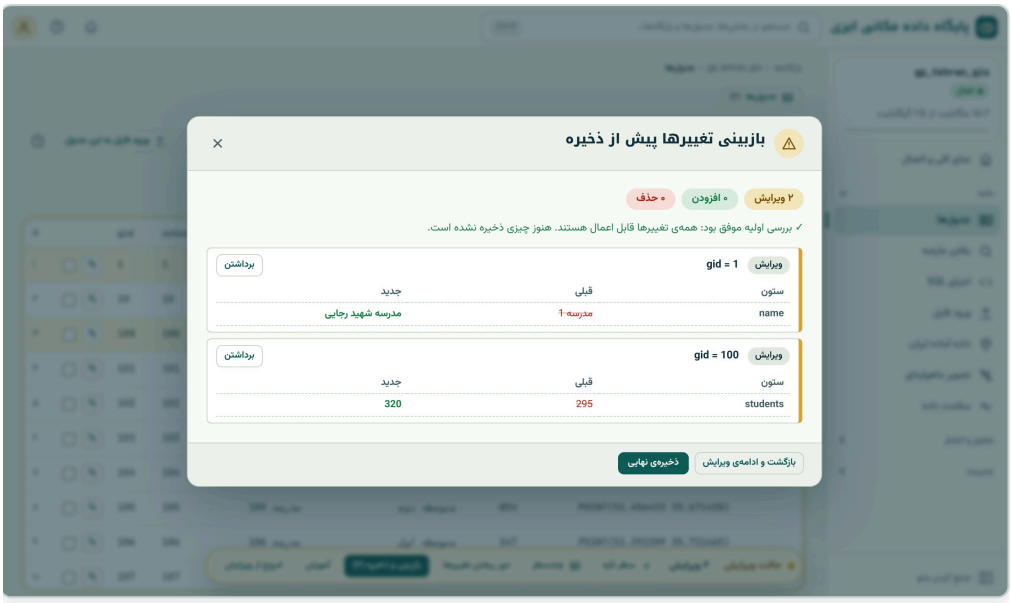
۲ در زبانه‌ی «داده‌ها»، کنار هر سطر آیکون ویرایش آمده است. آن را بزنید، مقدار تازه را بنویسید و «ثبت در فهرست تغییرها» را بزنید. کلید اصلی قابل ویرایش نیست.

۳ برای سطر تازه «افزودن سطر» و برای حذف، آیکون حذف کنار سطر را بزنید.

۴ در زبانه‌ی «نقشه» با ابزارهای کنار نقشه شکل تازه بکشید، گره‌های یک شکل را جابه‌جا کنید یا عارضه را حذف کنید. تغییرهای نقشه هم در همان فهرست جمع می‌شوند و با عوض کردن زبانه از بین نمی‌روند.

۵ «بازبینی و ذخیره» را بزنید. هر تغییر با مقدار قبلی (خط‌خورده) و جدید نشان داده می‌شود. اگر همه درست است «ذخیره‌ی نهایی».

۶ اگر پس از ذخیره پشیمان شدید، «بازگردانی آخرین ذخیره» آن را برمی‌گرداند (به شرطی که کسی سطرها را بعد از شما عوض نکرده باشد).



بازبینی پیش از ذخیره: دو ویرایش با مقدار قبلی و جدید. «بررسی اولیه موفق بود» یعنی قیدهای جدول رعایت شده‌اند.

هشدار

ویرایش فقط برای جدول دارای کلید اصلی. ویرایشگر برای شناختن هر سطر به کلید اصلی نیاز دارد. اگر جدولی کلید اصلی ندارد، اول از «سلامت داده» آن را اضافه کنید (فصل ۷). ویرایشگر هندسه را هم پیش از ذخیره بررسی می‌کند و هندسه‌ی نامعتبر را نمی‌پذیرد.

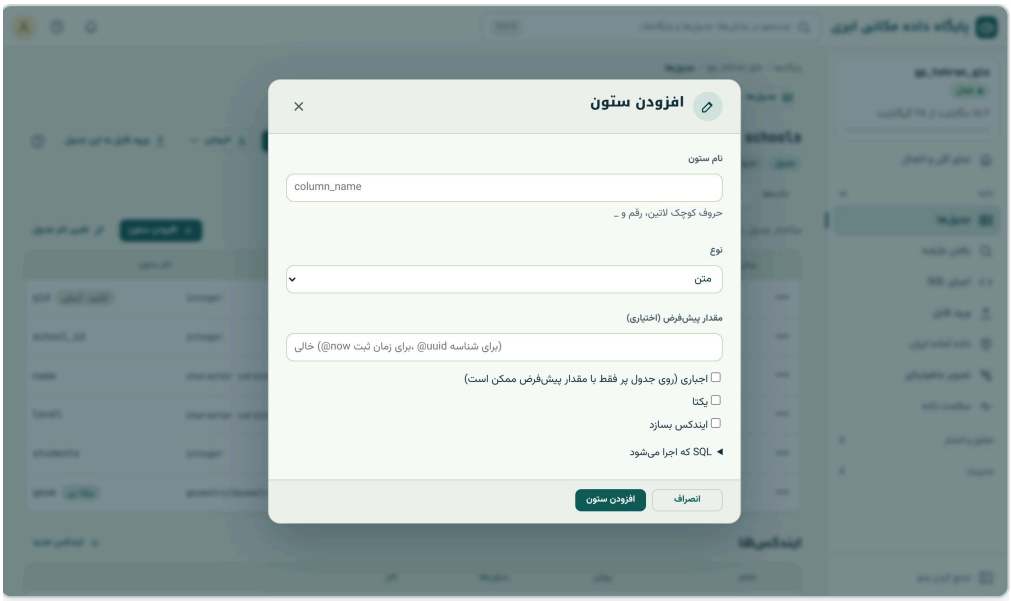
بدانید

تشخیص تعارض چگونه کار می‌کند؟ وقتی سطری را برای ویرایش باز می‌کنید، ویرایشگر «اثر انگشت» (چکیده‌ی md5) محتوای آن سطر را نگه می‌دارد. هنگام ذخیره، دوباره اثر انگشت سطر را در پایگاه حساب می‌کند. اگر فرق کرده باشد یعنی کس دیگری در این فاصله سطر را عوض کرده است، و ذخیره متوقف می‌شود. به این روش **قفل خوش‌بینانه** (optimistic locking) می‌گویند؛ در مقابلِ قفل بدبینانه که سطر را هنگام باز شدن برای دیگران قفل می‌کند.

۹-۳ تغییر ساختار جدول

زبانه‌ی «ستون‌ها» علاوه بر نمایش ساختار، امکان تغییر آن را هم دارد. منوی ⋯ کنار هر ستون و دکمه‌های بالای جدول این کارها را انجام می‌دهند:

کار	توضیح	معادل SQL
افزودن ستون	نام، نوع، پیش‌فرض، اجباری، یکتا، شاخص	`ALTER TABLE … ADD COLUMN …`
تغییر نام ستون		`ALTER TABLE … RENAME COLUMN … TO …`
تغییر نوع	مثلاً متن به عدد؛ مقدارهای ناسازگار گزارش می‌شوند	`ALTER TABLE … ALTER COLUMN … TYPE … USING …`
مقدار پیش‌فرض		`ALTER TABLE … ALTER COLUMN … SET DEFAULT …`
حذف ستون	تأیید تایپ‌شده می‌خواهد	`ALTER TABLE … DROP COLUMN …`
تغییر نام جدول		`ALTER TABLE … RENAME TO …`
ساخت شاخص	روی یک یا چند ستون؛ GiST برای هندسه	`CREATE INDEX …`
تغییر SRID یا نوع هندسه	با تبدیل واقعی مختصات	`ALTER TABLE … ALTER COLUMN geom TYPE geometry(…, srid) USING ST_Transform(…)`



مودال «افزودن ستون». بخش «SQL که اجرا می‌شود» دستور دقیق را پیش از اجرا نشان می‌دهد.

هر تغییر پیش از اجرا SQL خودش را نشان می‌دهد، همه‌یاهیچ است، و کارهای پرخطر مثل حذف ستون **تأیید تایپ‌شده** می‌خواهند: باید نام ستون را دوباره بنویسید. این بهترین فرصت برای یاد گرفتن SQL است؛ هر بار پیش از زدن دکمه، SQL را بخوانید.

مثلاً افزودن ستون «مساحت به هکتار» به پارک‌ها و پر کردن آن از روی هندسه:

```sql
ALTER TABLE parks ADD COLUMN area_calc_ha numeric(10,2);
UPDATE parks SET area_calc_ha = ST_Area(geom::geography) / 10000;
SELECT name, area_ha, area_calc_ha FROM parks ORDER BY park_id LIMIT 3;
```

name	area_ha	area_calc_ha
پارک 1	13.0	12.90
پارک 2	11.1	11.02
پارک 3	11.1	11.00

ستون `area_ha` از فایل اصلی آمده و `area_calc_ha` را خودمان از هندسه حساب کرده‌ایم. اختلاف این دو نمونه‌ی خوبی از مسئله‌ی کیفیت داده است (فصل ۲): کدام درست است؟ فصل ۱۶ اندازه‌گیری دقیق مساحت را آموزش می‌دهد.

۹-۴ حذف جدول

حذف جدول از رابط عمداً سخت است: باید دستور `DROP TABLE` را در کنسول SQL با تیک «اجازه تغییر داده» اجرا کنید، یا از منوی ⋯ جدول با تأیید تایپ‌شده. **حذف جدول برگشت‌پذیر نیست** مگر از بکاپ؛ پس پیش از آن یک بکاپ دستی بگیرید (فصل ۵۱). حذف جدول فضای سهمیه را آزاد می‌کند و لایه‌هایی که از آن در نقشه‌ها ساخته شده بودند هم برداشته می‌شوند.

خودتان امتحان کنید

جدول `wells` را با مودال «جدول جدید» بسازید (نقطه، ۴۳۲۶، ستون‌های `name`، `depth_m`، `drilled_on`). در حالت ویرایش، روی نقشه سه چاه در غرب تهران بکشید، نام و عمق بدهید و ذخیره کنید. سپس در کنسول SQL اجرا کنید: `SELECT name, ST_AsText(geom) FROM wells;`

خلاصه‌ی فصل

- «جدول جدید» در سه گام (مشخصات، ستون‌ها، بازبینی) جدول مکانی می‌سازد و SQL آن را نشان می‌دهد.
- نوع هندسه و SRID ستون را محدود کنید تا پایگاه داده‌ی ناسازگار را نپذیرد.
- ویرایش: حالت خاموش، پیش‌نویس، بازبینی، ذخیره‌ی همه‌یاهیچ، تشخیص تعارض (قفل خوش‌بینانه) و بازگردانی.
- زبانه‌ی «ستون‌ها» ساختار را تغییر می‌دهد؛ هر تغییر پیش از اجرا SQL خودش را نشان می‌دهد.
- حذف جدول برگشت‌ناپذیر است؛ پیش از آن بکاپ بگیرید.

برای مدرس

زمان پیشنهادی: ۶۰ دقیقه. دو دانشجو را هم‌زمان روی یک سطر به ویرایش وادارید تا تشخیص تعارض را ببینند؛ تجربه‌ی بهتری از هر توضیحی درباره‌ی هم‌زمانی است. تمرین ۵ مقدمه‌ی خوبی برای بخش SQL است.

تمرین‌ها

۱. سطح ۱ جدولی برای ثبت ایستگاه‌های اتوبوس طراحی کنید: چه ستون‌هایی، با چه نوعی، و چه نوع هندسه‌ای؟

۲. سطح ۱ در حالت ویرایش، نام سه مدرسه را عوض کنید ولی پیش از ذخیره «بازبینی» را ببینید و یکی را برگردانید.

۳. سطح ۲ یک ستون `capacity` از نوع عدد صحیح با پیش‌فرض صفر به `metro_stations` اضافه کنید. SQL نمایش‌داده‌شده را یادداشت کنید.

۴. سطح ۲ تفاوت قفل خوش‌بینانه و بدبینانه را بنویسید. کدام برای ویرایش وب مناسب‌تر است و چرا؟

۵. سطح ۳ کد «زیر کاپوت» را تغییر دهید تا جدول `cameras` با هندسه‌ی نقطه در SRID ۳۲۶۳۹ و ستون `installed_on` از نوع تاریخ بسازد و یک دوربین با مختصات UTM درج کند.

بخش ۳

زبان SQL از صفر

SQL زبان گفت‌وگو با پایگاه داده است و قلب این کتاب. این بخش بدون هیچ پیش‌فرض برنامه‌نویسی شروع می‌کند و تا پیوند جدول‌ها، زیرپرسش و تراکنش پیش می‌رود؛ همه با داده‌ی مکانی واقعی.

۱۰ پایگاه داده‌ی رابطه‌ای و زبان SQL

۱۱ پرسش از یک جدول؛ SELECT، WHERE و ORDER BY

۱۲ تجمیع، گروه‌بندی و توابع پنجره‌ای

۱۳ پیوند جدول‌ها، زیرپرسش و CTE

۱۴ ساخت و تغییر داده؛ DML، DDL، قید و تراکنش

فصل

۱۰

پایگاه داده‌ی رابطه‌ای و زبان SQL

سطح: **مبتدی** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل ۴**

در این فصل یاد می‌گیرید

- مفاهیم جدول، سطر، ستون، کلید اصلی، کلید خارجی و NULL را دقیق تعریف کنید.
- انواع داده‌ی پرکاربرد PostgreSQL را بشناسید و برای هر ستون نوع مناسب برگزینید.
- پنج دسته‌ی دستورهای SQL را از هم تشخیص دهید.
- با کنسول SQL سامانه کار کنید و نخستین پرسش‌ها را بنویسید.
- اصول نرمال‌سازی را در طراحی جدول به کار ببرید.

سرفصل‌ها

۱۰-۱ چرا SQL

۱۰-۲ مدل رابطه‌ای

۱۰-۳ انواع داده

۱۰-۴ دسته‌های دستورهای SQL

۱۰-۵ کنسول SQL

۱۰-۶ طراحی جدول: نرمال‌سازی در یک نگاه

۱۰-۱ چرا SQL

SQL (زبان پرسش ساخت‌یافته، Structured Query Language) زبانی است که تقریباً همه‌ی پایگاه‌های داده‌ی رابطه‌ای جهان با آن کار می‌کنند. SQL در دهه‌ی ۱۹۷۰ در شرکت IBM بر پایه‌ی مدل رابطه‌ای ادگار کاد طراحی شد و از ۱۹۸۶ استاندارد ANSI و ISO است. پنجاه سال بعد، SQL هنوز زبان اصلی داده است؛ نه چون کسی چیز بهتری نساخته، بلکه چون ایده‌ی آن بسیار قدرتمند است:

> **در SQL می‌گویید چه می‌خواهید، نه چگونه آن را پیدا کنید.**

وقتی می‌نویسید «مدرسه‌هایی را بده که بیش از ۵۰۰ دانش‌آموز دارند»، نمی‌گویید جدول را از کجا بخوان، به چه ترتیبی، با کدام شاخص. پایگاه داده خودش بهترین روش را انتخاب می‌کند. به این زبان‌ها **اعلانی** (declarative) می‌گویند، در مقابلِ زبان‌های **دستوری** (imperative) مثل پایتون که گام‌به‌گام می‌گویید چه کنی.

برای متخصص GIS، SQL سه مزیت دارد: **یک بار یاد می‌گیرید و همه‌جا استفاده می‌کنید** (PostGIS، QGIS، GeoServer، BigQuery، Snowflake، DuckDB، SpatiaLite و…)؛ **تکرارپذیر است** (یک پرسش را ذخیره می‌کنید و ماه بعد روی داده‌ی تازه دوباره اجرا می‌کنید)؛ و **روی داده‌ی بزرگ سریع است**، چون کار در خود پایگاه داده انجام می‌شود، نه در رایانه‌ی شما.

۱۰-۲ مدل رابطه‌ای

در مدل رابطه‌ای، داده در **رابطه**‌ها یا همان **جدول**‌ها نگه داشته می‌شود.

اصطلاح	معنا	مثال در داده‌ی نمونه
جدول (table، relation)	مجموعه‌ای از سطرها با ساختار یکسان؛ یک نوع موجودیت	`schools`
ستون (column، attribute)	یک ویژگی با نام و نوع مشخص	`students` از نوع عدد صحیح
سطر (row، tuple، record)	یک نمونه از موجودیت	مدرسه‌ی ۱
کلید اصلی (primary key)	ستونی که هر سطر را یکتا مشخص می‌کند؛ تکراری و خالی نمی‌شود	`gid`
کلید خارجی (foreign key)	ستونی که به کلید اصلی جدول دیگر اشاره می‌کند	`neighborhood_id` در جدول مدرسه‌ها
schema	پوشه‌ای برای دسته‌بندی جدول‌ها داخل پایگاه	`public`

دو نکته‌ی مهم درباره‌ی مدل رابطه‌ای: **ترتیب سطرها معنا ندارد** (جدول یک مجموعه است، نه یک فهرست؛ اگر ترتیب می‌خواهید باید صریحاً بخواهید) و **هر خانه یک مقدار دارد** (نه فهرستی از مقدارها).

NULL: مقدار ناموجود

`NULL` یعنی «مقدار نامعلوم یا ناموجود». NULL با صفر یا متن خالی فرق دارد: مدرسه‌ای که تعداد دانش‌آموزش NULL است یعنی تعدادش را نمی‌دانیم، نه اینکه دانش‌آموز ندارد. NULL رفتار خاصی دارد که منشأ خطاهای رایج است:

```sql
SELECT NULL = NULL       AS null_equals_null,
       NULL + 5          AS null_plus_5,
       NULL IS NULL      AS is_null,
       coalesce(NULL, 0) AS coalesce_zero;
```

null_equals_null	null_plus_5	is_null	coalesce_zero
NULL	NULL	True	0

هر محاسبه یا مقایسه با NULL نتیجه‌ی NULL (نامعلوم) دارد، حتی `NULL = NULL`! برای آزمودن NULL باید `IS NULL` یا `IS NOT NULL` بنویسید، و تابع `coalesce` اولین مقدار غیر NULL را برمی‌گرداند که برای جایگزینی مقدار پیش‌فرض به کار می‌رود.

۳-۱۰ انواع داده

هر ستون یک **نوع داده** دارد که تعیین می‌کند چه مقدارهایی مجازند و چه عملیاتی روی آن‌ها ممکن است.

دسته	نوع	کاربرد	مثال
عدد صحیح	`integer` (`int`)، `bigint` ، `smallint`	شمارش، شناسه	۴۱۱
عدد اعشاری دقیق	`numeric(p,s)`	پول، مقادیری که گرد کردن نمی‌پذیرند	۱۲۰٫۵
عدد اعشاری تقریبی	`double precision`، `real`	اندازه‌گیری، محاسبات علمی	۳۵٫۶۹۹۲
متن	`text` ، `varchar(n)`	نام، توضیح	«مدرسه ۱»
منطقی	`boolean`	بله/خیر	`true`
تاریخ و زمان	`date` ، `timestamp` ، `timestamptz`	تاریخ برداشت، زمان رویداد	`2026-11-09`
شناسه‌ی یکتا	`uuid`	شناسه‌ی جهانی	
ساخت‌یافته	`jsonb` ، `hstore`	ویژگی‌های متغیر	`{"lanes": 2}`
آرایه	`integer[]` ، `text[]`	فهرست مقدار (با احتیاط)	
مکانی	`geometry` ، `geography` ، `raster`	شکل و مکان	`POINT(51.4 35.7)`

نکته

در PostgreSQL، `text` کافی است. برخلاف بعضی پایگاه‌های داده، در PostgreSQL نوع `text` هیچ کندی یا حجم اضافه‌ای نسبت به `varchar(n)` ندارد. فقط وقتی `varchar(n)` بگذارید که واقعاً محدودیت طول یک قاعده‌ی کسب‌وکار است.

هشدار

تاریخ را متن نگذارید. اگر تاریخ را به صورت متن (`'1405/08/18'`) ذخیره کنید، مرتب‌سازی، مقایسه و محاسبه‌ی فاصله‌ی زمانی ممکن نیست. تاریخ را در ستون `date` (میلادی) ذخیره کنید و هنگام نمایش به شمسی تبدیل کنید؛ سامانه در نمودارها و چاپ نقشه همین کار را خودکار انجام می‌دهد.

۱۰-۴ دسته‌های دستورهای SQL

دستورهای SQL در پنج دسته جا می‌گیرند. در این کتاب همه را یاد می‌گیرید:

دسته	نام کامل	دستورها	کاربرد	فصل
DQL	زبان پرسش داده	`SELECT`	خواندن داده	۱۱ تا ۱۳
DML	زبان دست‌کاری داده	`INSERT`، `UPDATE`، `DELETE`	افزودن، تغییر و حذف سطر	۱۴
DDL	زبان تعریف داده	`CREATE`، `ALTER`، `DROP`	ساخت و تغییر ساختار	۱۴
TCL	زبان کنترل تراکنش	`BEGIN`، `COMMIT`، `ROLLBACK`	همه‌یاهیچ	۱۴
DCL	زبان کنترل دسترسی	`GRANT`، `REVOKE`	مجوز کاربران	۵۰

۱۰-۵ کنسول SQL

منوی کناری ← داده ← «اجرای SQL» کنسول SQL سامانه را باز می‌کند. این کنسول آزمایشگاه شما در سراسر بخش‌های سوم و چهارم است.

کنسول SQL. پرسش در کادر نوشته می‌شود و نتیجه زیر آن می‌آید. تیک «اجازه تغییر داده» برای دستورهای تغییردهنده لازم است.

قاعده‌های کنسول:

- در هر اجرا **یک دستور** اجرا می‌شود (برای چند دستور، هر کدام را جدا اجرا کنید یا از QGIS و psql استفاده کنید).
- به‌طور پیش‌فرض **فقط‌خواندنی** است: `INSERT`، `UPDATE`، `DELETE`، `CREATE` و `DROP` فقط با تیک «اجازه تغییر داده» اجرا می‌شوند. این تیک را فقط وقتی بزنید که واقعاً می‌خواهید چیزی را عوض کنید.
- حداکثر **۵۰۰ سطر** نتیجه نمایش داده می‌شود و هر پرسش حداکثر **۲۰ ثانیه** وقت دارد.
- **چیپ‌های آماده** زیر کادر، پرسش‌های پرکاربرد را می‌نویسند: «۲۰ سطر اول»، «انواع هندسه»، «محدوده‌ی لایه»، «نسخه‌ی PostGIS» و «حجم پایگاه داده».
- زبانه‌ی **«پرسش فارسی»** پرسش شما را به فارسی می‌گیرد و SQL آن را می‌سازد تا پیش از اجرا ببینید (فصل ۲۹). بهترین راه برای یاد گرفتن SQL این است که SQL ساخته‌شده را بخوانید و بفهمید.

اولین پرسش‌ها

ساده‌ترین پرسش، همه‌ی ستون‌های چند سطر اول یک جدول است. علامت `*` یعنی «همه‌ی ستون‌ها» و `LIMIT` تعداد سطر را محدود می‌کند:

```sql
SELECT * FROM metro_stations LIMIT 3;
```

gid	station_id	name	line	geom
1	1	ایستگاه 1	خط ۱	POINT(51.43 35.8)
2	2	ایستگاه 2	خط ۱	POINT(51.43 35.787686)
3	3	ایستگاه 3	خط ۱	POINT(51.43 35.775372)

SQL به بزرگی و کوچکی کلیدواژه‌ها حساس نیست (`select` و `SELECT` یکی‌اند)، ولی متن داخل گیومه‌ی تکی حساس است. فاصله و رفتن به خط تازه هم اهمیتی ندارد؛ پرسش‌های بلند را برای خوانایی در چند خط بنویسید. هر دستور با `;` تمام می‌شود.

توضیح (comment) در SQL با دو خط تیره شروع می‌شود و تا پایان خط ادامه دارد. از توضیح زیاد استفاده کنید؛ شش ماه بعد خودتان از آن تشکر می‌کنید:

```sql
-- شمار سطرهای هر جدول نمونه
SELECT 'schools' AS layer, count(*) FROM schools
UNION ALL SELECT 'metro_stations', count(*) FROM metro_stations
UNION ALL SELECT 'parks', count(*) FROM parks;
```

layer	count
schools	180
metro_stations	80
parks	35

SQL به‌عنوان ماشین حساب

`SELECT` بدون `FROM` هم کار می‌کند و برای آزمودن توابع بسیار مفید است:

```sql
SELECT 2 + 3 * 4              AS arithmetic,
       round(22 / 7.0, 3)     AS pi_approx,
       upper('postgis')       AS upper_text,
       length('پایگاه داده') AS persian_length,
       current_date           AS today;
```

arithmetic	pi_approx	upper_text	persian_length	today
14	3.143	POSTGIS	11	2026-11-09

بدانید

تقسیم عدد صحیح. `SELECT 7 / 2;` نتیجه‌ی `3` می‌دهد، نه `3.5`! وقتی هر دو طرف تقسیم عدد صحیح باشند، نتیجه هم عدد صحیح است. برای تقسیم اعشاری دست‌کم یکی را اعشاری کنید: `7 / 2.0` یا `7::numeric / 2`. این خطا در محاسبه‌ی درصد و نسبت بسیار رایج است.

علامت `::` در `7::numeric` **تبدیل نوع** (cast) است و در PostgreSQL و این کتاب زیاد می‌بینیدش؛ از جمله `geom::geography` که در فصل ۳ دیدید. شکل استاندارد آن `CAST(7 AS numeric)` است.

۱۰-۶ طراحی جدول: نرمال‌سازی در یک نگاه

فرض کنید جدول مدرسه‌ها ستون‌های «نام محله» و «جمعیت محله» هم داشته باشد. جمعیت ونک برای همه‌ی مدرسه‌های ونک تکرار می‌شود و اگر جمعیت را به‌روز کنید، باید همه‌ی آن سطرها را عوض کنید؛ کافی است یکی جا بماند تا داده ناسازگار شود. **نرمال‌سازی** یعنی طراحی جدول‌ها طوری که هر واقعیت فقط یک جا ذخیره شود. سه قاعده‌ی عملی:

1. **هر خانه یک مقدار.** به جای ستون «تلفن‌ها» با مقدار «۰۲۱۱، ۰۲۱۲»، یا چند ستون جدا یا جدول جداگانه‌ی تلفن.
2. **هر جدول یک موضوع.** ویژگی‌های محله در جدول محله، ویژگی‌های مدرسه در جدول مدرسه.

3. **رابطه با کلید، نه با تکرار.** مدرسه فقط شناسه‌ی محله را نگه می‌دارد (کلید خارجی) و بقیه‌ی اطلاعات محله با **پیوند** (JOIN، فصل ۱۳) خوانده می‌شود.

در داده‌ی مکانی یک ترفند دیگر هم داریم: لازم نیست شناسه‌ی محله را در جدول مدرسه نگه داریم، چون **هندسه خودش کلید پیوند است**. «مدرسه در کدام محله است؟» با رابطه‌ی مکانی `ST_Contains` پاسخ داده می‌شود؛ همان پرسشی که روی جلد پشت کتاب آمده است. این **پیوند مکانی** قدرتمندترین ابزار SQL مکانی است و فصل ۱۷ به آن اختصاص دارد.

خودتان امتحان کنید

در کنسول SQL، چیپ «انواع هندسه» را بزنید و پرسش ساخته‌شده را بخوانید. سپس پرسش‌های این فصل را یکی‌یکی اجرا کنید و نتیجه را با جدول‌های کتاب مقایسه کنید. یک پرسش هم خودتان بنویسید که شمار سطرهای `health_centers` و `neighborhoods` را بدهد.

خلاصه‌ی فصل

- SQL زبان اعلانی است: «چه» می‌خواهید، نه «چگونه».
- جدول مجموعه‌ای از سطرهاست؛ ترتیب سطر معنا ندارد. کلید اصلی سطر را یکتا می‌کند و کلید خارجی به جدول دیگر اشاره می‌کند.
- NULL یعنی نامعلوم؛ با `IS NULL` آزموده می‌شود و `coalesce` جایگزینش می‌کند.
- نوع داده را درست انتخاب کنید: تاریخ در `date`، پول در `numeric`، هندسه در `geometry`.
- پنج دسته‌ی دستور: DDL، DML، TCL، DQL و DCL.
- کنسول سامانه یک دستور در هر اجرا، پیش‌فرض فقط‌خواندنی، ۵۰۰ سطر و ۲۰ ثانیه دارد.
- نرمال‌سازی: هر واقعیت یک جا؛ در داده‌ی مکانی، هندسه هم کلید پیوند است.

برای مدرس

زمان پیشنهادی: یک جلسه. برای دانشجوهای غیرفنی، پیش از کنسول یک «جدول کاغذی» روی تخته بکشید و پرسش‌ها را با دست اجرا کنید («کدام سطرها شرط را دارند؟ خط بکشید»). رفتار NULL را حتماً با مثال واقعی نشان دهید؛ منشأ نیمی از خطاهای بعدی است.

تمرین‌ها

۱. سطح ۱ برای هر ستون نوع داده‌ی مناسب پیشنهاد کنید: کد ملی، مساحت قطعه به متر مربع، تاریخ تولد، فعال بودن کاربر، نام خیابان، مبلغ عوارض به ریال، محل چاه.

۲. سطح ۱ نتیجه‌ی این‌ها چیست؟ `10 / 4`، `10 / 4.0`، `NULL * 0`، `coalesce(NULL, NULL, 'x')`. با کنسول بیازمایید.

۳. سطح ۱ پنج سطر اول جدول `parks` را ببینید. کدام ستون کلید اصلی است؟

۴. سطح ۲ جدول «پروانه‌های ساختمانی» با ستون‌های شماره‌ی پروانه، نام مالک، تلفن مالک، نشانی ملک، منطقه‌ی شهرداری، نام شهردار منطقه و هندسه‌ی ملک را نرمال کنید. چند جدول لازم است؟

۵. سطح ۲ چرا `WHERE students = NULL` هیچ سطری برنمی‌گرداند، حتی اگر سطرهایی با تعداد نامعلوم وجود داشته باشد؟

۶. سطح ۳ با پرسش روی `information_schema.columns` نام و نوع همه‌ی ستون‌های جدول `schools` را فهرست کنید.

فصل

۱۱

پرسش از یک جدول: SELECT، WHERE و ORDER BY

سطح: **مبتدی** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۱۰**

در این فصل یاد می‌گیرید

- ستون‌های دلخواه را انتخاب، نام‌گذاری و محاسبه کنید.
- سطرها را با عملگرهای مقایسه، منطقی، IN، BETWEEN و LIKE فیلتر کنید.
- نتیجه را مرتب، محدود و صفحه‌بندی کنید.
- با CASE دسته‌بندی بسازید و توابع متنی، عددی و تاریخی را به کار ببرید.
- با جست‌وجوی تقریبی (pg_trgm) نام‌های فارسی را پیدا کنید.

سرفصل‌ها

۱۱-۱ ساختار یک پرسش

۱۱-۲ انتخاب ستون‌ها

۱۱-۳ فیلتر کردن با WHERE

۱۱-۴ مرتب‌سازی، محدود کردن و صفحه‌بندی

۱۱-۵ دسته‌بندی با CASE

۱۱-۶ توابع پرکاربرد

۱۱-۷ جست‌وجوی تقریبی نام‌ها

۱۱-۱ ساختار یک پرسش

هر پرسش `SELECT` از چند **بند** (clause) ساخته می‌شود که ترتیب نوشتن‌شان ثابت است:

```sql
SELECT    -- چه چیزی برگردد        ستون‌ها و محاسبه‌ها
FROM      -- از کجا                جدول
WHERE     -- کدام سطرها            شرط روی سطرها
GROUP BY  -- (فصل ۱۲)              دسته‌بندی
HAVING    -- (فصل ۱۲)              شرط روی دسته‌ها
ORDER BY  -- به چه ترتیبی          مرتب‌سازی
LIMIT     -- چند سطر               تعداد
OFFSET    -- از سطر چندم           پرش
```

اما پایگاه داده آن‌ها را به ترتیب دیگری **اجرا** می‌کند و دانستن این ترتیب بسیاری از خطاها را توضیح می‌دهد:

`FROM` ← `WHERE` ← `GROUP BY` ← `HAVING` ← `SELECT` ← `ORDER BY` ← `LIMIT`

یعنی اول جدول خوانده می‌شود، سطرها فیلتر می‌شوند، دسته‌بندی می‌شوند، ستون‌های خروجی ساخته می‌شوند و در آخر مرتب و محدود می‌شوند. به همین دلیل نمی‌توانید در `WHERE` از نام مستعاری که در `SELECT` ساخته‌اید استفاده کنید (هنوز ساخته نشده)، ولی در `ORDER BY` می‌توانید.

۱۱-۲ انتخاب ستون‌ها

به جای `*` بهتر است فقط ستون‌هایی را بخواهید که لازم دارید؛ پرسش خواناتر و سریع‌تر می‌شود:

```sql
SELECT name, level, students
FROM schools
LIMIT 4;
```

name	level	students
مدرسه 1	متوسطه اول	411
مدرسه 2	متوسطه اول	795
مدرسه 3	متوسطه اول	799
مدرسه 4	متوسطه اول	709

نام مستعار و محاسبه

با `AS` به ستون نام تازه می‌دهید و می‌توانید هر محاسبه‌ای را ستون کنید:

```sql
SELECT name                    AS school_name,
       students,
       round(students / 30.0)  AS classes_needed,
       students ⩾ 500          AS is_large
FROM schools
LIMIT 4;
```

school_name	students	classes_needed	is_large
مدرسه 1	411	14	False
مدرسه 2	795	27	True
مدرسه 3	799	27	True
مدرسه 4	709	24	True

نکته

نام مستعار فارسی؟ می‌توانید نام ستون را فارسی بگذارید، به شرطی که داخل گیومه‌ی دوتایی باشد: `AS "تعداد کلاس"` . اما نام‌های فارسی در کد و نرم‌افزارهای دیگر دردسر درست می‌کنند؛ پیشنهاد این کتاب نام انگلیسی برای ستون و فارسی برای **برچسب** روی نقشه و نمودار است.

DISTINCT: مقدارهای یکتا

```sql
SELECT DISTINCT line FROM metro_stations ORDER BY line;
```

line
خط ۱
خط ۲
خط ۳
خط ۴

۳-۱۱ فیلتر کردن با WHERE

`WHERE` شرطی می‌گیرد که برای هر سطر درست یا نادرست است؛ فقط سطرهایی که شرط برایشان **درست** است باقی می‌مانند.

عملگرهای مقایسه

عملگر	معنا
`=`	برابر
`<>` یا `!=`	نابرابر
`<` `>` `⩽` `⩾`	کوچک‌تر، بزرگ‌تر
`BETWEEN a AND b`	بین a و b (شامل خود a و b)
`IN (…)`	یکی از این مقدارها
`IS NULL` / `IS NOT NULL`	خالی / غیرخالی
`LIKE` / `ILIKE`	تطابق الگوی متنی (ILIKE بدون حساسیت به بزرگی حروف)

```sql
SELECT name, level, students
FROM schools
WHERE students > 850
ORDER BY students DESC;
```

name	level	students
مدرسه 16	ابتدایی	898
مدرسه 97	ابتدایی	896
مدرسه 163	ابتدایی	887
مدرسه 53	ابتدایی	876
مدرسه 164	ابتدایی	873
… (7 rows)		

متن داخل **گیومه‌ی تکی** نوشته می‌شود. گیومه‌ی دوتایی برای **نام** ستون و جدول است، نه مقدار:

```sql
SELECT count(*) AS primary_schools
FROM schools
WHERE level = 'ابتدایی';
```

primary_schools
88

ترکیب شرط‌ها

`AND` (هر دو)، `OR` (دست‌کم یکی) و `NOT` (نقیض). `AND` از `OR` اولویت بیشتری دارد؛ همیشه با پرانتز منظورتان را روشن کنید:

```sql
SELECT name, level, students
FROM schools
WHERE (level = 'ابتدایی' OR level = 'متوسطه اول')
  AND students BETWEEN 400 AND 450
ORDER BY students;
```

name	level	students
مدرسه 34	ابتدایی	405
مدرسه 146	ابتدایی	407
مدرسه 1	متوسطه اول	411
مدرسه 90	متوسطه اول	416
… (7 rows)		

همان شرط با `IN` کوتاه‌تر است:

```sql
SELECT name, level, students
FROM schools
WHERE level IN ('متوسطه اول' ,'ابتدایی')
  AND students BETWEEN 400 AND 450
ORDER BY students;
```

name	level	students
مدرسه 34	ابتدایی	405
مدرسه 146	ابتدایی	407
مدرسه 1	متوسطه اول	411
مدرسه 90	متوسطه اول	416
… (7 rows)		

هشدار

دام پرانتز. پرسش `WHERE level = 'ابتدایی' OR level = 'متوسطه اول' AND students > 800` یعنی «همه‌ی مدرسه‌های ابتدایی، به‌علاوه‌ی مدرسه‌های متوسطه‌ی اول بالای ۸۰۰ دانش‌آموز»، چون `AND` اول اجرا می‌شود. احتمالاً منظورتان این نبوده است.

الگوی متنی با LIKE

در `LIKE`، علامت `%` یعنی «هر تعداد نویسه» و `_` یعنی «دقیقاً یک نویسه»:

```sql
SELECT name FROM schools WHERE name LIKE '%12%' ORDER BY name;
```

name
مدرسه 112
مدرسه 12
مدرسه 120
مدرسه 121
… (12 rows)

۴-۱۱ مرتب‌سازی، محدود کردن و صفحه‌بندی

`ORDER BY` با `ASC` (صعودی، پیش‌فرض) و `DESC` (نزولی). چند ستون را می‌توان پشت هم آورد: اول بر اساس اولی، و در سطرهای برابر بر اساس دومی.

```sql
SELECT level, name, students
FROM schools
ORDER BY level, students DESC
LIMIT 6;
```

level	name	students
ابتدایی	مدرسه 16	898
ابتدایی	مدرسه 97	896
ابتدایی	مدرسه 163	887
ابتدایی	مدرسه 53	876
ابتدایی	مدرسه 164	873
ابتدایی	مدرسه 124	865

`LIMIT n OFFSET m` یعنی «از سطر m+1، n سطر بده»؛ این همان صفحه‌بندی ۵۰ سطری مرورگر داده است (صفحه‌ی سوم: `LIMIT 50 OFFSET 100`).

سطح حرفه‌ای

صفحه‌بندی بدون OFFSET. `OFFSET` بزرگ کند است، چون پایگاه باید همه‌ی سطرهای قبلی را بخواند و دور بریزد. برای جدول‌های بزرگ، صفحه‌بندی **کلیدی** (keyset) سریع‌تر است: به جای `OFFSET 100000` بنویسید `WHERE gid > آخرین_gid_صفحه‌ی_قبل ORDER BY gid LIMIT 50`.

`NULL`ها در مرتب‌سازی صعودی آخر می‌آیند؛ با `NULLS FIRST` یا `NULLS LAST` آن را کنترل کنید.

۵-۱۱ دسته‌بندی با CASE

`CASE` شرط «اگر … آنگاه …» SQL است و برای ساختن دسته از مقدار عددی (طبقه‌بندی) کاربرد زیادی دارد؛ همان کاری که در نقشه‌ی موضوعی انجام می‌دهیم:

```sql
SELECT name, students,
       CASE
         WHEN students < 300 THEN 'کوچک'
         WHEN students < 700 THEN 'متوسط'
         ELSE 'بزرگ'
       END AS size_class
FROM schools
ORDER BY gid
LIMIT 6;
```

name	students	size_class
مدرسه 1	411	متوسط
مدرسه 2	795	بزرگ
مدرسه 3	799	بزرگ
مدرسه 4	709	بزرگ
مدرسه 5	741	بزرگ
مدرسه 6	637	متوسط

شرط‌ها به ترتیب بررسی می‌شوند و اولین شرط درست برنده است؛ پس ترتیب `WHEN`ها مهم است.

۱۱-۶ توابع پرکاربرد

متنی

```sql
SELECT length('۱۲ مدرسه')                    AS len,
       replace('دبستان ' ,'مدرسه ' ,'12 مدرسه') AS replaced,
       split_part('2 ,' ' ,'۳ خط)             AS line_no,
       '۴'||  ' '||  'خط'                     AS concat_op,
       trim('  فاصله  ')                      AS trimmed,
       translate('۱۲۳', '۰۱۲۳۴۵۶۷۸۹', '0123456789')::int + 1 AS
persian_digits;
```

len	replaced	line_no	concat_op	trimmed	persian_digits
8	دبستان 12	۳	خط ۴	فاصله	124

آخرین ستون یک ترفند کاربردی برای داده‌ی ایرانی است: رقم‌های فارسی را با `translate` به لاتین تبدیل می‌کند تا بتوان آن را عدد کرد. در داده‌ی واقعی گاهی هر دو نوع رقم (و حتی رقم عربی ٤٥٦) را با هم می‌بینید؛ یکدست کردن آن‌ها پیش از تحلیل لازم است.

> **هشدار**
>
> **«ی» و «ک» عربی.** متن فارسی که از منابع مختلف می‌آید ممکن است «ي» و «ك» عربی داشته باشد. «علي» و «على» برای پایگاه داده دو متن متفاوت‌اند و جست‌وجوی یکی دیگری را پیدا نمی‌کند. پیش از تحلیل یکدست کنید: `UPDATE t SET name = translate(name, 'يك', 'یک');`

عددی

`round(x, n)`، `ceil(x)`، `floor(x)`، `abs(x)`، `power(x, y)`، `sqrt(x)`، `greatest(a, b, …)`، `least(a, b, …)` و `random()`.

تاریخ و زمان

```sql
SELECT current_date                                  AS today,
       current_date - date '2025-03-21'              AS
days_since_nowruz_1404,
       extract(year FROM current_date)               AS year,
       date_trunc('month', now())::date              AS month_start,
       (current_date + interval '30 days')::date     AS in_30_days;
```

today	days_since_nowruz_1404	year	month_start	in_30_days
2026-11-09	598	2026	2026-11-01	2026-12-09

تفاضل دو تاریخ، تعداد روز است. `interval` بازه‌ی زمانی است و `extract` و `date_trunc` بخشی از تاریخ را جدا یا تاریخ را به آغاز ماه و سال گرد می‌کنند؛ پایه‌ی همه‌ی گزارش‌های زمانی و سری‌های زمانی ماهواره‌ای در فصل ۳۷.

۷-۱۱ جست‌وجوی تقریبی نام‌ها

کاربر می‌نویسد «درمانگا» و انتظار دارد «درمانگاه» را پیدا کند. افزونه‌ی `pg_trgm` (که در فصل ۵ نصب کردیم) شباهت دو متن را بر اساس **سه‌نویسه‌ای**‌های مشترکشان می‌سنجد:

```sql
SELECT name, type, similarity(type, 'درمانگا') AS sim
FROM health_centers
WHERE type % 'درمانگا'
ORDER BY sim DESC, name
LIMIT 4;
```

name	type	sim
مرکز 11	درمانگاه	0.7
مرکز 14	درمانگاه	0.7
مرکز 17	درمانگاه	0.7
مرکز 2	درمانگاه	0.7

عملگر `%` یعنی «به اندازه‌ی کافی شبیه» (پیش‌فرض شباهت ۰٫۳). با یک شاخص از نوع GIN روی ستون، این جست‌وجو روی میلیون‌ها نام هم سریع است: `CREATE INDEX ON`

`health_centers USING gin (name gin_trgm_ops);` این روش پایه‌ی ساختن جست‌وجوی نشانی و نام مکان است.

خودتان امتحان کنید

پرسشی بنویسید که مراکز درمانی با بیش از ۵۰ تخت را، از نوع بیمارستان یا درمانگاه، به ترتیب تعداد تخت (نزولی) برگرداند و برای هر کدام ستون «ظرفیت» با مقدار «زیاد» (۷۰ تخت و بیشتر) یا «متوسط» بسازد.

خلاصه‌ی فصل

- ترتیب نوشتن: SELECT، FROM، WHERE، GROUP BY، HAVING، ORDER BY، LIMIT؛ ترتیب اجرا: FROM، WHERE، GROUP BY، HAVING، SELECT، ORDER BY، LIMIT.
- مقدار متنی در گیومه‌ی تکی؛ نام ستون در گیومه‌ی دوتایی.
- `AND` پیش از `OR` اجرا می‌شود؛ پرانتز بگذارید. `IN` و `BETWEEN` شرط‌ها را کوتاه می‌کنند.
- `CASE` برای دسته‌بندی؛ اولین شرط درست برنده است.
- توابع متنی، عددی و تاریخی؛ رقم فارسی و «ی/ک» عربی را پیش از تحلیل یکدست کنید.
- `pg_trgm` جست‌وجوی تقریبی نام را ممکن می‌کند.

برای مدرس

زمان پیشنهادی: دو جلسه. هر مفهوم را با یک «پرسش از مدیر» شروع کنید: «مدرسه‌های بزرگ کدام‌اند؟» و بگذارید دانشجوها آن را به SQL ترجمه کنند. ترتیب اجرای بندها را روی تخته بکشید و خطای استفاده از نام مستعار در WHERE را عمداً نشان دهید.

تمرین‌ها

۱. سطح ۱ نام و تعداد دانش‌آموز ده مدرسه‌ی کوچک‌تر را بدهید.

۲. سطح ۱ ایستگاه‌های خط ۲ را به ترتیب نام فهرست کنید.

۳. سطح ۱ مراکز درمانی که تعداد تختشان صفر یا NULL است کدام‌اند؟

۴. سطح ۲ مدرسه‌های متوسطه‌ی دوم با ۳۰۰ تا ۶۰۰ دانش‌آموز را بدهید؛ صفحه‌ی دوم نتیجه با صفحه‌های ۵ سطری چیست؟

۵. سطح ۲ برای هر پارک ستونی بسازید که «کوچک» (زیر ۵ هکتار)، «متوسط» (۵ تا ۱۵) یا «بزرگ» باشد.

۶. سطح ۲ چرا `SELECT students * 2 AS d FROM schools WHERE d > 1000` خطا می‌دهد؟ آن را درست کنید.

۷. سطح ۳ ستونی از نوع متن با مقدارهای «۱۲۰»، «120» و «۱۲۰» دارید. پرسشی بنویسید که همه را به عدد ۱۲۰ تبدیل کند.

فصل

۱۲

تجمیع، گروه‌بندی و توابع پنجره‌ای

سطح: **متوسط** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۱۱**

در این فصل یاد می‌گیرید

- با توابع تجمیعی شمارش، جمع، میانگین، کمینه و بیشینه بگیرید.
- داده را با GROUP BY دسته‌بندی و با HAVING دسته‌ها را فیلتر کنید.
- با FILTER چند شمارش شرطی را در یک پرسش بگیرید.
- میانه، صدک و انحراف معیار را حساب کنید.
- با توابع پنجره‌ای رتبه، سهم از کل و مقدار تجمعی بسازید.
- تجمیع مکانی (ST_Extent، ST_Union، ST_Collect) را بشناسید.

سرفصل‌ها

۱۲-۱ از سطر تا خلاصه

۱۲-۲ خلاصه برای هر دسته: GROUP BY

۱۲-۳ فیلتر روی دسته‌ها: HAVING

۱۲-۴ چند شمارش در یک پرسش: FILTER

۱۲-۵ آمار توصیفی

۱۲-۶ توابع پنجره‌ای

۱۲-۷ تجمیع مکانی: نگاهی به فصل‌های بعد

۱۲-۱ از سطر تا خلاصه

پرسش‌های فصل قبل سطر به سطر جواب می‌دادند: «کدام مدرسه‌ها؟». اما مدیران معمولاً **خلاصه** می‌خواهند: «چند مدرسه؟»، «میانگین دانش‌آموز هر مقطع؟»، «کدام خط مترو بیشترین ایستگاه را دارد؟». **تابع تجمیعی** (aggregate) چند سطر را می‌گیرد و یک مقدار برمی‌گرداند.

تابع	نتیجه	توجه
`count(*)`	تعداد سطرها	NULLها را هم می‌شمارد
`count(col)`	تعداد مقدارهای غیر NULL ستون	
`count(DISTINCT col)`	تعداد مقدارهای یکتا	
`sum(col)`	جمع	NULLها نادیده گرفته می‌شوند
`avg(col)`	میانگین	NULLها در مخرج حساب نمی‌شوند
`min(col)` ، `max(col)`	کمینه و بیشینه	روی متن و تاریخ هم کار می‌کند
`string_agg(col, ', ')`	چسباندن متن‌ها	
`array_agg(col)`	آرایه‌ی مقدارها	

```sql
SELECT count(*)                  AS schools,
       sum(students)             AS total_students,
       round(avg(students), 1)   AS avg_students,
       min(students)             AS smallest,
       max(students)             AS largest,
       count(DISTINCT level)     AS levels
FROM schools;
```

schools	total_students	avg_students	smallest	largest	levels
180	91520	508.4	151	898	3

۱۲-۲ خلاصه برای هر دسته: GROUP BY

`GROUP BY` سطرها را بر اساس مقدار یک یا چند ستون دسته‌بندی می‌کند و تابع تجمیعی را برای هر دسته جداگانه حساب می‌کند:

```sql
SELECT level,
       count(*)               AS schools,
       sum(students)          AS students,
       round(avg(students))   AS avg_students
FROM schools
GROUP BY level
ORDER BY students DESC;
```

level	schools	students	avg_students
ابتدایی	88	43404	493
متوسطه اول	57	30175	529
متوسطه دوم	35	17941	513

قاعده‌ی طلایی GROUP BY: هر ستونی در `SELECT` یا باید در `GROUP BY` باشد یا داخل یک تابع تجمیعی. اگر بنویسید `SELECT level, name, count(*) … GROUP BY level` خطا می‌گیرید، چون هر دسته چند `name` دارد و پایگاه نمی‌داند کدام را نشان دهد.

می‌توان بر اساس چند ستون یا یک عبارت محاسباتی هم گروه‌بندی کرد:

```sql
SELECT CASE WHEN students < 300 THEN '۱. کوچک'
            WHEN students < 700 THEN '۲. متوسط'
            ELSE '۳. بزرگ' END AS size_class,
       count(*) AS schools
FROM schools
GROUP BY 1
ORDER BY 1;
```

size_class	schools
۱. کوچک	43
۲. متوسط	90
۳. بزرگ	47

`GROUP BY 1` یعنی «بر اساس ستون اول SELECT»؛ کوتاه و مفید وقتی ستون اول عبارت بلندی است.

۱۲-۳ فیلتر روی دسته‌ها: HAVING

`WHERE` سطرها را **پیش از** گروه‌بندی فیلتر می‌کند و `HAVING` دسته‌ها را **پس از** آن. شرط روی نتیجه‌ی تابع تجمیعی فقط در `HAVING` ممکن است:

```sql
SELECT line, count(*) AS stations
FROM metro_stations
GROUP BY line
HAVING count(*) ⩾ 19
ORDER BY stations DESC;
```

line	stations
خط ۲	27
خط ۱	19
خط ۴	19

۱۲-۴ چند شمارش در یک پرسش: FILTER

گاهی در یک جدول خلاصه، چند شمارش شرطی کنار هم لازم است؛ مثلاً برای هر مقطع، تعداد مدرسه‌های کوچک و بزرگ. بند `FILTER` در PostgreSQL این کار را بسیار خوانا می‌کند:

```sql
SELECT level,
       count(*)                                  AS total,
       count(*) FILTER (WHERE students < 300)    AS small,
       count(*) FILTER (WHERE students ⩾ 700)    AS large,
       round(100.0 * count(*) FILTER (WHERE students ⩾ 700) /
count(*), 1) AS large_pct
FROM schools
GROUP BY level
ORDER BY level;
```

level	total	small	large	large_pct
ابتدایی	88	27	23	26.1
متوسطه اول	57	12	17	29.8
متوسطه دوم	35	4	7	20.0

به `100.0` در محاسبه‌ی درصد دقت کنید: بدون اعشار، تقسیم عدد صحیح نتیجه را صفر می‌کرد (فصل ۱۰).

۱۲-۵ آمار توصیفی

میانگین به تنهایی گمراه‌کننده است، به‌ویژه وقتی داده چوله باشد (مثل قیمت ملک یا درآمد). **میانه** و **صدک‌ها** تصویر واقعی‌تری می‌دهند. PostgreSQL توابع آماری کاملی دارد:

```sql
SELECT round(avg(students), 1)
AS mean,
       percentile_cont(0.5)  WITHIN GROUP (ORDER BY students)
AS median,
       percentile_cont(0.25) WITHIN GROUP (ORDER BY students)
AS q1,
       percentile_cont(0.75) WITHIN GROUP (ORDER BY students)
AS q3,
       round(stddev_samp(students), 1)
AS std_dev,
       mode() WITHIN GROUP (ORDER BY level)
AS most_common_level
FROM schools;
```

mean	median	q1	q3	std_dev	most_common_level
508.4	512.0	308.75	706.0	224.3	ابتدایی

بدانید

چارک و طبقه‌بندی نقشه. چارک‌ها (q1، میانه، q3) مرزهای طبقه‌بندی «چندک» (quantile) را می‌دهند که در نقشه‌ی موضوعی هر طبقه را با تعداد برابری عارضه پر می‌کند. وقتی در استودیوی نقشه طبقه‌بندی عددی می‌سازید (فصل ۴۲)، پشت صحنه دقیقاً همین محاسبه انجام می‌شود. `corr(x, y)` (همبستگی) و `regr_slope(y, x)` (شیب رگرسیون) هم برای تحلیل رابطه‌ی دو متغیر هست.

۱۲-۶ توابع پنجره‌ای

تابع تجمیعی سطرها را در یک سطر **خلاصه** می‌کند و سطرهای اصلی از دست می‌روند. **تابع پنجره‌ای** (window function) محاسبه‌ی تجمیعی را روی گروهی از سطرها انجام می‌دهد اما **همه‌ی سطرها را نگه می‌دارد** و نتیجه را کنار هر سطر می‌گذارد. نشانه‌اش بند `OVER (…)` است.

سهم از کل و مقایسه با میانگین دسته

```sql
SELECT name, level, students,
       round(avg(students) OVER (PARTITION BY level))             AS level_avg,
       students - round(avg(students) OVER (PARTITION BY level))  AS diff_from_avg,
       round(100.0 * students / sum(students) OVER (), 2)         AS pct_of_all
FROM schools
ORDER BY gid
LIMIT 5;
```

name	level	students	level_avg	diff_from_avg	pct_of_all
مدرسه 1	متوسطه اول	411	529	-118	0.45
مدرسه 2	متوسطه اول	795	529	266	0.87
مدرسه 3	متوسطه اول	799	529	270	0.87
مدرسه 4	متوسطه اول	709	529	180	0.77
مدرسه 5	ابتدایی	741	493	248	0.81

`PARTITION BY level` یعنی «پنجره = همه‌ی مدرسه‌های هم‌مقطع» و `OVER ()` یعنی «پنجره = کل جدول». هر سطر سر جایش ماند و میانگین مقطع و سهمش از کل کنارش آمد؛ کاری که با `GROUP BY` به تنهایی ممکن نبود.

رتبه‌بندی: «سه مدرسه‌ی بزرگ هر مقطع»

```sql
SELECT level, name, students, rnk
FROM (
  SELECT level, name, students,
         rank() OVER (PARTITION BY level ORDER BY students DESC) AS rnk
  FROM schools
) ranked
WHERE rnk ⩽ 3
ORDER BY level, rnk;
```

level	name	students	rnk
ابتدایی	مدرسه 16	898	1
ابتدایی	مدرسه 97	896	2
ابتدایی	مدرسه 163	887	3
متوسطه اول	مدرسه 79	845	1
متوسطه اول	مدرسه 19	839	2
متوسطه اول	مدرسه 15	835	3
متوسطه دوم	مدرسه 105	854	1
متوسطه دوم	مدرسه 104	820	2
متوسطه دوم	مدرسه 125	808	3

این الگو، یعنی «n تای برتر هر گروه»، یکی از پرکاربردترین الگوهای SQL است. چون تابع پنجره‌ای در مرحله‌ی `SELECT` حساب می‌شود و `WHERE` پیش از آن اجرا می‌شود، باید پرسش را داخل یک **زیرپرسش** بگذاریم و بیرون از آن فیلتر کنیم (فصل ۱۳).

تابع	رفتار در مقدار برابر
`row_number()`	۱، ۲، ۳، ۴ (همیشه یکتا)
`rank()`	۱، ۲، ۲، ۴ (برابرها هم‌رتبه، شکاف بعدش)
`dense_rank()`	۱، ۲، ۲، ۳ (بدون شکاف)
`ntile(4)`	شماره‌ی چارک (۱ تا ۴)
`lag(col)` ، `lead(col)`	مقدار سطر قبلی و بعدی در ترتیب پنجره
`sum(col) OVER (ORDER BY …)`	جمع تجمعی (running total)

در دنیای واقعی

سری زمانی NDVI. در فصل ۳۷ برای هر قطعه‌ی کشاورزی یک سطر NDVI در هر تاریخ تصویر داریم. «تغییر NDVI نسبت به تصویر قبلی» با `ndvi - lag(ndvi) OVER (PARTITION BY parcel_id ORDER BY scene_date)` و «میانگین متحرک سه‌تصویره» با `avg(ndvi) OVER (PARTITION BY parcel_id ORDER BY scene_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)` به دست می‌آید. پایش افت ناگهانی سلامت مزرعه روی همین دو خط بنا شده است.

۱۲-۷ تجمیع مکانی: نگاهی به فصل‌های بعد

PostGIS توابع تجمیعی مکانی هم دارد که دقیقاً مثل `sum` و `count` با `GROUP BY` کار می‌کنند، ولی به جای عدد، **هندسه** برمی‌گردانند:

تابع	نتیجه
`ST_Extent(geom)`	مستطیل محدوده‌ی همه‌ی هندسه‌ها
`ST_Collect(geom)`	یک هندسه‌ی چندبخشی از همه (بدون ادغام)
`ST_Union(geom)`	ادغام هندسه‌ها و حذف مرزهای داخلی
`ST_Centroid(ST_Collect(geom))`	مرکز ثقل یک گروه

```sql
SELECT line,
       count(*)                                    AS stations,
       ST_AsText(ST_Centroid(ST_Collect(geom)), 4) AS line_center,
       ST_Extent(geom)                             AS bbox
FROM metro_stations
GROUP BY line
ORDER BY line;
```

line	stations	line_center	bbox
خط ۱	19	POINT(51.4243 35.6896)	BOX(51.41 35.58,51.43 35.8)
خط ۲	27	POINT(51.4059 35.6995)	BOX(51.24 35.690762,51.57 35.72)
خط ۳	15	POINT(51.3945 35.7136)	BOX(51.33 35.64,51.46 35.78)
خط ۴	19	POINT(51.396 35.6636)	BOX(51.28 35.66,51.52 35.669582)

با یک پرسش، مرکز ثقل و محدوده‌ی ایستگاه‌های هر خط مترو را داریم. فصل ۱۸ ادغام (`ST_Union`) را با جزئیات بررسی می‌کند.

خودتان امتحان کنید

گزارشی بسازید که برای هر نوع مرکز درمانی تعداد مراکز، مجموع تخت، میانگین تخت، و تعداد مراکز بی‌تخت (تخت صفر) را بدهد. سپس برای هر مرکز رتبه‌اش از نظر تعداد تخت در میان مراکز هم‌نوع را کنارش بگذارید.

خلاصه‌ی فصل

- توابع تجمیعی (`count` ، `sum` ، `avg` ، `min` ، `max`) چند سطر را به یک مقدار خلاصه می‌کنند؛ NULLها جز در `count(*)` نادیده گرفته می‌شوند.
- `GROUP BY` برای هر دسته خلاصه می‌سازد؛ هر ستون SELECT یا در GROUP BY است یا در تابع تجمیعی.
- `WHERE` پیش از گروه‌بندی و `HAVING` پس از آن فیلتر می‌کند. `FILTER` شمارش‌های شرطی را کنار هم می‌گذارد.
- میانه و صدک با `percentile_cont` ؛ میانگین در داده‌ی چوله گمراه‌کننده است.
- تابع پنجره‌ای (`OVER`) محاسبه‌ی گروهی را کنار هر سطر می‌گذارد: سهم از کل، رتبه، `lag` ، جمع تجمعی.
- `ST_Extent` ، `ST_Collect` و `ST_Union` تجمیع مکانی‌اند.

برای مدرس

زمان پیشنهادی: دو جلسه. تفاوت `GROUP BY` و تابع پنجره‌ای را با کارت‌های کاغذی نشان دهید: در GROUP BY کارت‌های هر دسته روی هم گذاشته و به یک کارت خلاصه تبدیل می‌شوند؛ در تابع پنجره‌ای کارت‌ها سر جایشان می‌مانند و روی هر کدام عدد خلاصه‌ی دسته‌اش نوشته می‌شود.

تمرین‌ها

۱. سطح ۱ تعداد ایستگاه هر خط مترو را به ترتیب نزولی بدهید.

۲. سطح ۱ مجموع و میانگین مساحت پارک‌ها (ستون `area_ha`) چقدر است؟ بزرگ‌ترین و کوچک‌ترین؟

۳. سطح ۲ مقطع‌هایی را بدهید که میانگین دانش‌آموزشان بیش از ۵۰۰ است.

۴. سطح ۲ برای هر مقطع، تعداد مدرسه‌ها در هر چارک تعداد دانش‌آموز را با `ntile(4)` و `GROUP BY` بشمارید.

۵. سطح ۲ میانگین و میانه‌ی تخت مراکز درمانی را مقایسه کنید. کدام نماینده‌ی بهتری است و چرا؟

۶. سطح ۳ برای هر خط مترو، ایستگاه‌ها را به ترتیب `station_id` مرتب کنید و با `lag` فاصله‌ی هر ایستگاه تا ایستگاه قبلی را به متر حساب کنید (راهنمایی: `ST_Distance` روی `geography`). طول تقریبی هر خط چقدر است؟

فصل

۱۳

پیوند جدول‌ها، زیرپرسش و CTE

سطح: **متوسط** | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل ۱۲**

در این فصل یاد می‌گیرید

- انواع پیوند (CROSS ،FULL ،RIGHT ،LEFT ،INNER و پیوند با خود) را بشناسید و درست به کار ببرید.
- نخستین پیوند مکانی را بنویسید.
- زیرپرسش‌های اسکالر، IN، EXISTS و همبسته را بنویسید.
- پرسش‌های پیچیده را با CTE (بند WITH) گام‌به‌گام و خوانا بسازید.
- مجموعه‌ها را با UNION، INTERSECT و EXCEPT ترکیب کنید و با LATERAL آشنا شوید.

سرفصل‌ها

۱۳-۱ چرا پیوند

۱۳-۲ فقط جفت‌های جور: INNER JOIN

۱۳-۳ همه‌ی سطرهای سمت چپ: LEFT JOIN

۱۳-۴ CROSS و FULL، RIGHT

۱۳-۵ نخستین پیوند مکانی

۱۳-۶ پیوند با خود

۱۳-۷ زیرپرسش

۱۳-۸ پرسش‌های گام‌به‌گام: CTE

۱۳-۹ ترکیب مجموعه‌ها

۱۳-۱ چرا پیوند

در فصل ۱۰ دیدیم که در طراحی خوب، هر موضوع جدول خودش را دارد. پس پاسخ بیشتر پرسش‌های واقعی در **چند جدول** پخش است: «نام محله‌ی هر مدرسه» در جدول محله‌هاست و «تعداد دانش‌آموز» در جدول مدرسه‌ها. **پیوند** (JOIN) سطرهای دو جدول را بر اساس یک **شرط پیوند** کنار هم می‌گذارد.

برای مثال‌های غیرمکانی این فصل، دو جدول کوچک را با `VALUES` همان‌جا در پرسش می‌سازیم: مشخصات خط‌های مترو و ظرفیت استاندارد کلاس هر مقطع. (در عمل این‌ها جدول‌های واقعی‌اند؛ فصل ۱۴ ساختنشان را نشان می‌دهد.)

```sql
SELECT * FROM (VALUES ('1378 ,'قرمز' ,'خط ۱'),
                      ('1379  ,'آبی' ,'خط ۲'),
                      ('1391 ,'آبی روشن' ,'خط ۳'),
                      ('1378  ,'سبز' ,'خط ۵')) AS lines(line, color,
opened);
```

line	color	opened
خط ۱	قرمز	1378
خط ۲	آبی	1379
خط ۳	آبی روشن	1391
خط ۵	سبز	1378

به داده دقت کنید: جدول خط‌ها «خط ۴» ندارد و در عوض «خط ۵» دارد که در جدول ایستگاه‌ها نیست. این ناهمخوانی عمدی است تا تفاوت انواع پیوند را ببینیم.

۱۳-۲ فقط جفت‌های جور: INNER JOIN

`INNER JOIN` (یا فقط `JOIN`) سطرهایی را نگه می‌دارد که در **هر دو** جدول جفت دارند:

```sql
WITH lines(line, color, opened) AS (
  VALUES ('1378 ,'قرمز' ,'خط ۱'), ('1379 ,'آبی' ,'خط ۲'),
         ('1391 ,'آبی روشن' ,'خط ۳'), ('1378 ,'سبز' ,'خط ۵'))
SELECT l.line, l.color, count(m.*) AS stations
FROM metro_stations m
JOIN lines l ON l.line = m.line
GROUP BY l.line, l.color
ORDER BY l.line;
```

line	color	stations
خط ۱	قرمز	19
خط ۲	آبی	27
خط ۳	آبی روشن	15

خط ۴ (که در جدول خط‌ها نیست) و خط ۵ (که ایستگاهی ندارد) هر دو حذف شدند. حرف‌های `m` و `l` **نام مستعار جدول**‌اند و نوشتن را کوتاه و منظور را روشن می‌کنند؛ وقتی دو جدول ستون هم‌نام دارند (اینجا `line`) باید بگویید منظورتان ستون کدام جدول است.

۱۳-۳ همه‌ی سطرهای سمت چپ: LEFT JOIN

`LEFT JOIN` همه‌ی سطرهای جدول **اول** را نگه می‌دارد؛ اگر جفتی در جدول دوم نبود، ستون‌های آن NULL می‌شوند:

```sql
WITH lines(line, color, opened) AS (
  VALUES ('1378 ,'قرمز' ,'خط ۱'), ('1379 ,'آبی' ,'خط ۲'),
         ('1391 ,'آبی روشن' ,'خط ۳'), ('1378 ,'سبز' ,'خط ۵'))
SELECT l.line, l.color, count(m.gid) AS stations
FROM lines l
LEFT JOIN metro_stations m ON m.line = l.line
GROUP BY l.line, l.color
ORDER BY l.line;
```

line	color	stations
خط ۱	قرمز	19
خط ۲	آبی	27
خط ۳	آبی روشن	15
خط ۵	سبز	0

خط ۵ با صفر ایستگاه آمد. به `count(m.gid)` دقت کنید: `count(*)` برای خط ۵ عدد ۱ می‌داد (یک سطر با NULLها)، اما `count(m.gid)` فقط مقدارهای غیر NULL را می‌شمارد.

LEFT JOIN برای یافتن «کمبودها» هم بسیار مفید است: «کدام خط ایستگاهی ندارد؟» یعنی `LEFT JOIN … WHERE m.gid IS NULL`.

۱۳-۴ RIGHT، FULL و CROSS

پیوند	چه سطرهایی می‌مانند
`INNER JOIN`	فقط جفت‌های جور در هر دو
`LEFT JOIN`	همه‌ی سطرهای جدول اول + جفت‌ها
`RIGHT JOIN`	همه‌ی سطرهای جدول دوم + جفت‌ها (همان LEFT با جای جدول‌ها عوض)
`FULL JOIN`	همه‌ی سطرهای هر دو؛ هر جا جفت نبود NULL
`CROSS JOIN`	هر سطر اول با هر سطر دوم (حاصل‌ضرب؛ m × n سطر)

```sql
WITH lines(line, color) AS (VALUES ('خط ۱','قرمز'), ('خط ۲','آبی'),
('خط ۳','آبی روشن'), ('خط ۵','سبز')),
     st AS (SELECT line, count(*) AS n FROM metro_stations GROUP BY
line)
SELECT coalesce(l.line, st.line) AS line, l.color, st.n AS stations
FROM lines l
FULL JOIN st ON st.line = l.line
ORDER BY 1;
```

line	color	stations
خط ۱	قرمز	19
خط ۲	آبی	27
خط ۳	آبی روشن	15
خط ۴	NULL	19
خط ۵	سبز	NULL

هشدار

پیوند بدون شرط. اگر شرط `ON` را فراموش کنید یا اشتباه بنویسید، ممکن است ناخواسته CROSS JOIN بسازید: پیوند ۱۸۰ مدرسه با ۴۰ مرکز درمانی، ۷۲۰۰ سطر می‌دهد و برای جدول‌های بزرگ میلیون‌ها. اگر نتیجه‌ی پیوند بسیار بیشتر از انتظار بود، شرط پیوند را بررسی کنید.

۱۳-۵ نخستین پیوند مکانی

تا اینجا شرط پیوند **برابری** دو ستون بود. اما شرط پیوند می‌تواند هر عبارت درست/نادرستی باشد، از جمله یک **رابطه‌ی مکانی**. این پرسش، که روی جلد پشت کتاب هم آمده، هر مدرسه را به محله‌ای که آن را در بر دارد پیوند می‌دهد:

```sql
SELECT n.name AS neighborhood,
       count(s.*) AS schools,
       sum(s.students) AS students
FROM neighborhoods n
JOIN schools s ON ST_Contains(n.geom, s.geom)
GROUP BY n.name
ORDER BY schools DESC;
```

neighborhood	schools	students
شهرک غرب	7	3747
نارمک	7	3942
ونک	6	3721
جمهوری	5	1656
تجریش	2	889
هفت‌تیر	1	697

بدون اینکه هیچ ستون مشترکی بین دو جدول باشد، فقط با **مکان** آن‌ها را پیوند دادیم. این همان «هندسه خودش کلید پیوند است» که در فصل ۱۰ گفتیم، و قدرتمندترین ایده‌ی SQL مکانی است. اگر بخواهیم محله‌های بی‌مدرسه هم بیایند، `LEFT JOIN` می‌نویسیم. فصل ۱۷ همه‌ی رابطه‌های مکانی (`ST_Intersects`، `ST_Within`، `ST_DWithin`، …) و نکته‌های کارایی پیوند مکانی را آموزش می‌دهد.

> **سطح حرفه‌ای**
>
> **شش محله، ۱۸۰ مدرسه، چند پیوند؟** مجموع ستون `schools` در نتیجه‌ی بالا کمتر از ۱۸۰ است، چون محله‌های داده‌ی نمونه همه‌ی شهر را نمی‌پوشانند. همیشه این را بسنجید: «چند عارضه پیوند نخوردند؟» با `LEFT JOIN` از سمت مدرسه‌ها و شمردن `n.gid IS NULL` پیدا می‌شود. در داده‌ی واقعی، عارضه‌هایی که درست روی مرز دو محله‌اند یا بیرون از همه‌ی محله‌ها افتاده‌اند، منشأ اختلاف آمارها هستند.

۱۳-۶ پیوند با خود

گاهی جدول با **خودش** پیوند می‌خورد؛ مثلاً «جفت ایستگاه‌هایی که کمتر از ۳۰۰ متر از هم فاصله دارند»:

```sql
SELECT a.name AS station_a, b.name AS station_b,
       round(ST_Distance(a.geom::geography, b.geom::geography)) AS
meters
FROM metro_stations a
JOIN metro_stations b
  ON a.gid < b.gid
 AND ST_DWithin(a.geom::geography, b.geom::geography, 300)
ORDER BY meters
LIMIT 5;
```

station_a	station_b	meters

شرط `a.gid < b.gid` دو کار می‌کند: ایستگاه را با خودش جفت نمی‌کند، و هر جفت را فقط یک بار می‌آورد (نه هم الف-ب و هم ب-الف).

۱۳-۷ زیرپرسش

زیرپرسش (subquery) پرسشی داخل پرسش دیگر است، داخل پرانتز.

زیرپرسش اسکالر یک مقدار برمی‌گرداند و هر جا یک مقدار مجاز است می‌نشیند:

```sql
SELECT name, students
FROM schools
WHERE students > (SELECT avg(students) * 1.6 FROM schools)
ORDER BY students DESC
LIMIT 5;
```

name	students
مدرسه 16	898
مدرسه 97	896
مدرسه 163	887
مدرسه 53	876
مدرسه 164	873

زیرپرسش با IN فهرستی از مقدارها می‌دهد:

```sql
SELECT name, line FROM metro_stations
WHERE line IN (SELECT line FROM metro_stations GROUP BY line HAVING
count(*) < 19)
ORDER BY gid LIMIT 3;
```

name	line
ایستگاه 47	خط ۳
ایستگاه 48	خط ۳
ایستگاه 49	خط ۳

زیرپرسش با EXISTS فقط می‌پرسد «آیا دست‌کم یک سطر هست؟». این همان پرسشی بود که «یافتن عارضه» در فصل ۸ ساخت. EXISTS معمولاً سریع‌ترین راه برای «داشتن» یا «نداشتن» یک رابطه است:

```sql
SELECT count(*) AS parks_without_school_within_300m
FROM parks p
WHERE NOT EXISTS (
  SELECT 1 FROM schools s
  WHERE ST_DWithin(p.geom::geography, s.geom::geography, 300)
);
```

parks_without_school_within_300m
25

زیرپرسش بالا **همبسته** (correlated) است: به `p` از پرسش بیرونی اشاره می‌کند و برای هر پارک جداگانه ارزیابی می‌شود.

۱۳-۸ پرسش‌های گام‌به‌گام: CTE

پرسش‌های پیچیده با زیرپرسش‌های تودرتو به سرعت ناخوانا می‌شوند. **CTE** (عبارت جدول مشترک، Common Table Expression) با بند `WITH` اجازه می‌دهد پرسش را در چند **گام نام‌دار** بنویسید، مثل متغیر در برنامه‌نویسی:

```sql
WITH school_stats AS (            -- گام ۱: آمار مدرسه‌های هر محله
  SELECT n.gid, n.name, n.population,
         count(s.*) AS schools, coalesce(sum(s.students), 0) AS
students
  FROM neighborhoods n
  LEFT JOIN schools s ON ST_Contains(n.geom, s.geom)
  GROUP BY n.gid, n.name, n.population
),
ratios AS (                       -- گام ۲: شاخص‌ها
  SELECT *, round(population::numeric / nullif(schools, 0)) AS
people_per_school
  FROM school_stats
)
SELECT name, population, schools, people_per_school   -- گام ۳: گزارش
FROM ratios
ORDER BY people_per_school DESC NULLS FIRST;
```

name	population	schools	people_per_school
هفت‌تیر	65500	1	65500
تجریش	27500	2	13750
جمهوری	56000	5	11200
شهرک غرب	46500	7	6643
نارمک	37000	7	5286
ونک	18000	6	3000

هر گام از گام قبلی می‌خواند و پرسش از بالا به پایین مثل یک متن خوانده می‌شود. `nullif(schools, 0)` اگر تعداد مدرسه صفر بود NULL می‌گذارد تا خطای «تقسیم بر صفر» رخ ندهد؛ ترفندی که در هر محاسبه‌ی نسبت لازم دارید.

بدانید

CTE بازگشتی. `WITH RECURSIVE` می‌تواند خودش را صدا بزند و برای داده‌ی سلسله‌مراتبی (استان ← شهرستان ← بخش) یا پیمایش شبکه به کار می‌رود. برای تحلیل شبکه‌ی معابر ابزار تخصصی‌تری داریم (pgRouting، فصل ۲۴).

۱۳-۹ ترکیب مجموعه‌ها

عملگر	نتیجه
`UNION`	سطرهای هر دو پرسش، بدون تکرار
`UNION ALL`	سطرهای هر دو پرسش، با تکرار (سریع‌تر)
`INTERSECT`	سطرهایی که در هر دو هستند
`EXCEPT`	سطرهای پرسش اول که در دومی نیستند

دو پرسش باید تعداد و نوع ستون یکسان داشته باشند. (اینجا برای خوانایی هندسه را به متن تبدیل کرده‌ایم؛ در عمل خود `geom` را نگه می‌دارید تا نتیجه لایه‌ی مکانی باشد.) یک کاربرد رایج، ساختن یک لایه‌ی واحد «خدمات» از چند جدول است:

```sql
SELECT 'مدرسه' AS kind, name, ST_AsText(geom) AS geom FROM schools
UNION ALL
SELECT 'درمان', name, ST_AsText(geom) FROM health_centers
UNION ALL
SELECT 'مترو', name, ST_AsText(geom) FROM metro_stations
ORDER BY kind, name
LIMIT 4;
```

kind	name	geom
درمان	مرکز 1	POINT(51.33 35.68)
درمان	مرکز 10	POINT(51.358 35.71)
درمان	مرکز 11	POINT(51.385999999999996 35.71)
درمان	مرکز 12	POINT(51.414 35.71)

۱۳-۱۰ پیوند با یک زیرپرسش برای هر سطر: LATERAL

`JOIN LATERAL` به زیرپرسش سمت راست اجازه می‌دهد به سطر جاری سمت چپ اشاره کند؛ یعنی «برای هر سطر، این پرسش را اجرا کن». این ابزار پرسش «نزدیک‌ترین‌ها» را ممکن می‌کند:

```sql
SELECT s.name AS school, h.name AS nearest_center, h.meters
FROM schools s
CROSS JOIN LATERAL (
  SELECT name, round(ST_Distance(s.geom::geography,
hc.geom::geography)) AS meters
  FROM health_centers hc
  ORDER BY s.geom <-> hc.geom
  LIMIT 1
) h
ORDER BY s.gid
LIMIT 5;
```

school	nearest_center	meters
مدرسه 1	مرکز 8	2808.0
مدرسه 2	مرکز 14	1502.0
مدرسه 3	مرکز 7	1058.0
مدرسه 4	مرکز 17	9574.0
مدرسه 5	مرکز 2	1316.0

عملگر `<->` فاصله است و با شاخص مکانی، یافتن نزدیک‌ترین را حتی روی میلیون‌ها نقطه سریع می‌کند. این الگو آن‌قدر مهم است که فصل ۱۹ کاملاً به آن اختصاص دارد.

خودتان امتحان کنید

با یک CTE گزارشی بسازید که برای هر محله تعداد مدرسه، تعداد مرکز درمانی و تعداد ایستگاه مترو داخل آن را کنار هم بگذارد. (راهنمایی: سه CTE جدا با LEFT JOIN مکانی، و یک پیوند نهایی روی `gid` محله.)

خلاصه‌ی فصل

- `INNER JOIN` فقط جفت‌ها؛ `LEFT JOIN` همه‌ی سطرهای اول؛ `FULL JOIN` همه‌ی هر دو؛ `CROSS JOIN` حاصل‌ضرب.
- شرط پیوند می‌تواند رابطه‌ی مکانی باشد: `JOIN … ON ST_Contains(n.geom, s.geom)`؛ پیوند مکانی.
- `LEFT JOIN … WHERE b.id IS NULL` و `NOT EXISTS` برای یافتن «نداشتن‌ها».
- زیرپرسش: اسکالر، EXISTS، IN، همبسته. CTE (`WITH`) پرسش را گام‌به‌گام و خوانا می‌کند.
- `nullif(x, 0)` جلوی تقسیم بر صفر را می‌گیرد.
- `UNION ALL` لایه‌ها را روی هم می‌گذارد؛ `LATERAL` برای «برای هر سطر، نزدیک‌ترین…».

برای مدرس

زمان پیشنهادی: دو جلسه. نمودار ون برای انواع پیوند رایج است اما گمراه‌کننده؛ به جای آن دو جدول کوچک روی تخته بنویسید و خروجی هر نوع پیوند را سطر به سطر بسازید. پیوند مکانی را آخر جلسه‌ی اول نشان دهید؛ معمولاً لحظه‌ای است که دانشجوها قدرت SQL مکانی را درک می‌کنند.

تمرین‌ها

۱. سطح ۱ با جدول کوچک ظرفیت کلاس هر مقطع (ابتدایی ۲۸، متوسطه اول ۳۲، متوسطه دوم ۳۵) و `JOIN`، تعداد کلاس لازم هر مدرسه را حساب کنید.

۲. سطح ۱ تعداد مرکز درمانی هر محله را با پیوند مکانی بدهید. محله‌های بی‌مرکز هم بیایند.

۳. سطح ۲ چند مدرسه در هیچ محله‌ای نیستند؟ با `LEFT JOIN` و با `NOT EXISTS` دو راه بنویسید.

۴. سطح ۲ مدرسه‌هایی که تعداد دانش‌آموزشان از میانگین مقطع خودشان بیشتر است (زیرپرسش همبسته، یا تابع پنجره‌ای از فصل ۱۲).

۵. سطح ۲ نام ایستگاه‌هایی که تا ۵۰۰ متری‌شان هم مدرسه هست و هم مرکز درمانی (`INTERSECT` یا دو `EXISTS`).

۶. سطح ۳ برای هر مرکز درمانی، سه مدرسه‌ی نزدیک و فاصله‌شان را با `LATERAL` بدهید.

فصل

۱۴

ساخت و تغییر داده؛ DML، DDL، قید و تراکنش

سطح: **متوسط** | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل ۱۳**

در این فصل یاد می‌گیرید

- جدول را با کلید، قید و ستون هندسه‌ی محدودشده بسازید.
- با CREATE TABLE AS نتیجه‌ی هر تحلیل را به جدول ماندگار تبدیل کنید.
- با INSERT، UPDATE و DELETE داده را تغییر دهید؛ از جمله به‌روزرسانی مکانی با UPDATE ... FROM
- با ON CONFLICT درج-یا-به‌روزرسانی (upsert) بنویسید.
- تراکنش را بفهمید و تغییرهای خطرناک را ایمن انجام دهید.
- نما و نمای مادی‌شده بسازید و روی جدول‌ها فراداده بگذارید.

سرفصل‌ها

۱۴-۱ پیش از شروع: حالت نوشتن

۱۴-۲ ساختن جدول با قید

۱۴-۳ ذخیره‌ی نتیجه‌ی تحلیل با CREATE TABLE AS:

۱۴-۴ افزودن سطر با INSERT:

۱۴-۵ تغییر سطرها با UPDATE:

۱۴-۶ حذف سطرها با DELETE:

۱۴-۷ درج یا به‌روزرسانی با UPSERT:

۱۴-۸ تراکنش: همه یا هیچ
۱۴-۹ وقتی تراکنش ملغی می‌شود
۱۴-۱۰ فراداده روی جدول
۱۴-۱۱ ساختن بقیه‌ی داده‌ی نمونه

۱۴-۱ پیش از شروع: حالت نوشتن

دستورهای این فصل پایگاه را **تغییر می‌دهند.** در کنسول SQL سامانه باید تیک «اجازه تغییر داده» را بزنید. پیش از هر تغییر بزرگ روی داده‌ی واقعی یک بکاپ دستی بگیرید (فصل ۵۱). کدهای این فصل که زیرشان نتیجه آمده روی پایگاه نمونه اجرا شده‌اند، اما در پایان **برگردانده** شده‌اند تا داده‌ی نمونه برای فصل‌های بعد دست‌نخورده بماند.

۱۴-۲ ساختن جدول با قید

در فصل ۹ جدول ساده‌ای ساختیم. اینجا یک جدول کامل‌تر با همه‌ی ابزارهای پاسداری از کیفیت داده می‌سازیم. **قید** (constraint) قاعده‌ای است که پایگاه داده همیشه و برای همه‌ی کاربران و نرم‌افزارها اعمال می‌کند؛ مهم نیست داده از QGIS بیاید، از مرورگر یا از اسکریپت.

```sql
CREATE TABLE districts (
  id    smallint PRIMARY KEY,
  name  text NOT NULL UNIQUE
);

CREATE TABLE bus_stops (
  gid         serial PRIMARY KEY,                        -- کلید اصلی خودافزا
  code        text NOT NULL UNIQUE,                      -- کد یکتای ایستگاه
  name        text NOT NULL,
  district_id smallint REFERENCES districts(id),         -- کلید خارجی
  shelters    smallint DEFAULT 0 CHECK (shelters >= 0),  -- مقدار پیش‌فرض و بازه‌ی مجاز
  status      text DEFAULT 'active' CHECK (status IN ('active', 'closed', 'planned')),
  created_at  timestamptz DEFAULT now(),
  geom        geometry(Point, 4326) NOT NULL
              CHECK (ST_Within(geom, ST_MakeEnvelope(44, 25, 64, 40, 4326)))  -- فقط داخل محدوده‌ی ایران
);
CREATE INDEX ON bus_stops USING gist (geom);

INSERT INTO districts VALUES (1, 'منطقه ۱'), (2 ,'منطقه ۲');
INSERT INTO bus_stops (code, name, district_id, geom)
VALUES ('BS-001', '1 ,'میدان تجریش, ST_SetSRID(ST_MakePoint(51.4335, 35.8044), 4326))
RETURNING gid, code, shelters, status;
```

gid	code	shelters	status
1	BS-001	0	active

قید	چه چیزی را تضمین می‌کند
`PRIMARY KEY`	یکتا و غیرخالی؛ هویت سطر
`NOT NULL`	ستون الزامی
`UNIQUE`	مقدار تکراری ممنوع
`REFERENCES` (کلید خارجی)	مقدار باید در جدول مرجع وجود داشته باشد
`CHECK`	هر شرط دلخواه؛ حتی شرط مکانی
`DEFAULT`	مقدار پیش‌فرض اگر چیزی داده نشود
`geometry(Point, 4326)`	نوع هندسه و SRID ثابت

`RETURNING` در انتهای `INSERT` سطرهای درج‌شده را برمی‌گرداند؛ برای دیدن `gid` ساخته‌شده و مقدارهای پیش‌فرض بسیار مفید است.

حالا ببینیم قیدها چه می‌کنند. درج ایستگاهی با مختصات جابه‌جا (عرض به جای طول) رد می‌شود:

```sql
INSERT INTO bus_stops (code, name, geom)
VALUES ('BS-002', 'خطای عمدی', ST_SetSRID(ST_MakePoint(35.80, 51.43),
4326));
-- ERROR: new row for relation "bus_stops" violates check constraint
"bus_stops_geom_check"
```

این خطا دوست شماست: بدون آن، نقطه‌ای در شمال روسیه وارد داده می‌شد و شاید ماه‌ها کسی متوجه نمی‌شد.

سطح حرفه‌ای

قید روی هندسه، با احتیاط. قید `CHECK (ST_IsValid(geom))` جلوی هندسه‌ی نامعتبر را می‌گیرد و قید محدوده جلوی مختصات جابه‌جا را. اما قید سنگین (مثلاً `ST_Within` روی مرز پیچیده‌ی کشور با هزاران رأس) هر درج را کند می‌کند. برای مرزهای پیچیده، قید ساده‌ی مستطیلی (مثل بالا) را در پایگاه بگذارید و کنترل دقیق را در «سلامت داده» یا پرسش دوره‌ای انجام دهید.

۱۴-۳ ذخیره‌ی نتیجه‌ی تحلیل: CREATE TABLE AS

مهم‌ترین دستور DDL برای متخصص GIS این است: **هر پرسش SELECT را می‌توان به جدول تبدیل کرد.** نتیجه‌ی تحلیل، که تا الان فقط روی صفحه دیده می‌شد، جدول ماندگاری می‌شود که روی نقشه می‌آید، منتشر می‌شود و ورودی تحلیل بعدی است:

```sql
CREATE TABLE neighborhood_school_stats AS
SELECT n.gid, n.name, n.population,
       count(s.*)                      AS schools,
       coalesce(sum(s.students), 0)    AS students,
       n.geom
FROM neighborhoods n
LEFT JOIN schools s ON ST_Contains(n.geom, s.geom)
GROUP BY n.gid, n.name, n.population, n.geom;

ALTER TABLE neighborhood_school_stats ADD PRIMARY KEY (gid);
CREATE INDEX ON neighborhood_school_stats USING gist (geom);

SELECT name, schools, students FROM neighborhood_school_stats ORDER BY
schools DESC LIMIT 3;
```

name	schools	students
نارمک	7	3942
شهرک غرب	7	3747
ونک	6	3721

جدول ساخته‌شده با `CREATE TABLE AS` کلید اصلی و شاخص ندارد؛ همیشه این دو را بلافاصله اضافه کنید، وگرنه «سلامت داده» به شما یادآوری می‌کند. همه‌ی ابزارهای تحلیل سامانه (بخش پنجم) پشت صحنه همین الگو را اجرا می‌کنند.

۱۴-۴ افزودن سطر: INSERT

```sql
-- یک سطر
INSERT INTO districts (id, name) VALUES (3, 'منطقه ۳');

-- چند سطر
INSERT INTO districts (id, name) VALUES (4, 'منطقه ۴'), (5, 'منطقه ۵');

-- از نتیجه‌ی یک پرسش
INSERT INTO bus_stops (code, name, geom)
SELECT 'M-' || gid, 'ایستگاه مترو ' || name, geom
FROM metro_stations WHERE line = 'خط ۱';
```

همیشه **نام ستون‌ها را بنویسید** (`INSERT INTO t (a, b) …`). اگر ننویسید و روزی ستونی به جدول اضافه شود، درج‌های قدیمی خطا می‌دهند یا بدتر، مقدار را در ستون اشتباه می‌گذارند.

۱۴-۵ تغییر سطرها: UPDATE

```sql
UPDATE schools
SET students = students + 20
WHERE level = 'ابتدایی' AND students < 200
RETURNING name, students;
```

name	students
مدرسه 7	216
مدرسه 30	190
مدرسه 33	172
مدرسه 55	178
… (13 rows)	

هشدار

UPDATE و DELETE بدون WHERE. `UPDATE schools SET students = 0;` بدون `WHERE` **همه‌ی** سطرها را عوض می‌کند. پیش از هر UPDATE یا DELETE، همان شرط را با `SELECT count(*) … WHERE …` بیازمایید تا بدانید چند سطر تغییر می‌کند. بهتر از آن، داخل تراکنش کار کنید (پایین‌تر).

به‌روزرسانی مکانی با UPDATE … FROM

یکی از پرکاربردترین کارها در GIS، **نسبت دادن ویژگی یک لایه به لایه‌ی دیگر بر اساس مکان** است؛ مثلاً نوشتن نام محله در جدول مدرسه‌ها. در PostgreSQL، `UPDATE … FROM` پیوند را داخل UPDATE ممکن می‌کند:

```sql
ALTER TABLE schools ADD COLUMN neighborhood text;

UPDATE schools s
SET neighborhood = n.name
FROM neighborhoods n
WHERE ST_Contains(n.geom, s.geom);

SELECT neighborhood, count(*) FROM schools GROUP BY neighborhood ORDER
BY count(*) DESC LIMIT 4;
```

neighborhood	count
NULL	154
نارمک	7
ونک	6
شهرک غرب	5

مدرسه‌هایی که در هیچ محله‌ای نیستند `NULL` ماندند. این کار در نرم‌افزارهای رومیزی «Join attributes by location» نام دارد.

۱۴-۶ حذف سطرها: DELETE

```sql
DELETE FROM bus_stops WHERE status = 'closed';
DELETE FROM bus_stops b USING districts d
WHERE b.district_id = d.id AND d.name = 'منطقه ۵';     --حذف بر اساس جدول دیگر
```

برای خالی کردن کامل یک جدول، `TRUNCATE bus_stops;` بسیار سریع‌تر از `DELETE` بدون شرط است (ولی برگشت‌پذیر نیست مگر داخل تراکنش).

۱۴-۷ درج یا به‌روزرسانی: UPSERT

داده‌ی برداشت میدانی هر روز همگام می‌شود: سطرهای تازه باید درج شوند و سطرهایی که قبلاً بوده‌اند به‌روز. `ON CONFLICT` این کار را در یک دستور انجام می‌دهد:

```sql
CREATE TABLE station_counts (station_code text PRIMARY KEY,
passengers int, updated date);
INSERT INTO station_counts VALUES ('ST-1', 1200, '2026-11-01');

INSERT INTO station_counts (station_code, passengers, updated)
VALUES ('ST-1', 1350, '2026-11-08'), ('ST-2', 900, '2026-11-08')
ON CONFLICT (station_code)
DO UPDATE SET passengers = EXCLUDED.passengers, updated =
EXCLUDED.updated;

SELECT * FROM station_counts ORDER BY station_code;
```

station_code	passengers	updated
ST-1	1350	2026-11-08
ST-2	900	2026-11-08

`EXCLUDED` به سطری اشاره می‌کند که می‌خواستیم درج کنیم. با `DO NOTHING` به جای `DO UPDATE`، سطرهای تکراری بی‌صدا نادیده گرفته می‌شوند.

۱۴-۸ تراکنش: همه یا هیچ

تراکنش (transaction) مجموعه‌ای از دستورهاست که یا **همه** اعمال می‌شوند یا **هیچ‌کدام**. مثال کلاسیک، انتقال پول است: کم کردن از یک حساب و افزودن به حساب دیگر نباید نیمه‌کاره بماند. در GIS، تفکیک یک قطعه‌ی زمین به دو قطعه (حذف قطعه‌ی قدیم، درج دو قطعه‌ی تازه) همین‌طور است.

```sql
BEGIN;                                          -- آغاز تراکنش
  UPDATE schools SET students = 0 WHERE level = 'ابتدایی';
  SELECT count(*) FROM schools WHERE students = 0;   -- بررسی نتیجه
ROLLBACK;                                       -- پشیمان شدیم: همه چیز برمی‌گردد
--  یا COMMIT; برای ماندگار کردن
```

ویژگی‌های تراکنش را با کلمه‌ی **ACID** به خاطر می‌سپارند:

حرف	ویژگی	معنا
A	اتمی بودن (Atomicity)	همه یا هیچ
C	سازگاری (Consistency)	پس از تراکنش همه‌ی قیدها برقرارند
I	جدایی (Isolation)	تراکنش‌های هم‌زمان در کار هم دخالت نمی‌کنند
D	ماندگاری (Durability)	پس از COMMIT، حتی با قطع برق هم داده می‌ماند

نکته

تراکنش در سامانه. کنسول SQL سامانه هر دستور را جداگانه و خودکار در تراکنش خودش اجرا می‌کند؛ برای تراکنش چنددستوری از DBeaver ،psql یا DB Manager در QGIS استفاده کنید. ورود فایل، ویرایشگر جدول، تغییر ساختار و ابزارهای تحلیل سامانه همه همه‌یاهیچ‌اند. DDL در PostgreSQL هم تراکنشی است: حتی `DROP TABLE` داخل تراکنش با `ROLLBACK` برمی‌گردد؛ ویژگی‌ای که همه‌ی پایگاه‌های داده ندارند.

۱۴-۹ نما و نمای مادی‌شده

نما (view) یک پرسش ذخیره‌شده است که مثل جدول خوانده می‌شود ولی داده‌ای نگه نمی‌دارد؛ هر بار که خوانده می‌شود پرسش اجرا می‌شود و همیشه به‌روز است:

```sql
CREATE VIEW large_schools AS
SELECT gid, name, level, students, geom FROM schools WHERE students ⩾
700;

SELECT name, students FROM large_schools ORDER BY students DESC LIMIT
3;
```

name	students
مدرسه 16	898
مدرسه 97	896
مدرسه 163	887

نما در QGIS و مرورگر سامانه مثل یک لایه باز می‌شود و برای منتشر کردن «زیرمجموعه‌ای» از داده (مثلاً فقط ستون‌های عمومی) بسیار مناسب است.

نمای مادی‌شده (materialized view) نتیجه را **ذخیره** می‌کند؛ خواندنش سریع است ولی تا `REFRESH` نشود به‌روز نمی‌شود. برای تحلیل‌های سنگینی که روزی یک بار لازم‌اند مناسب است:

```sql
CREATE MATERIALIZED VIEW mv_neighborhood_stats AS
SELECT n.name, count(s.*) AS schools, n.geom
FROM neighborhoods n LEFT JOIN schools s ON ST_Contains(n.geom, s.geom)
GROUP BY n.name, n.geom;

REFRESH MATERIALIZED VIEW mv_neighborhood_stats;   -- به‌روزرسانی
```

۱۴-۱۰ فراداده روی جدول

`COMMENT ON` توضیح را روی خود جدول و ستون می‌نشاند؛ ساده‌ترین شکل فراداده که همراه داده می‌ماند و QGIS و مرورگر سامانه نمایشش می‌دهند:

```sql
COMMENT ON TABLE schools IS 'مدرسه‌های نمونه؛ منبع: داده‌ی آموزشی کتاب؛ ۱۴۰۵';
COMMENT ON COLUMN schools.students IS 'تعداد دانش‌آموز سال تحصیلی ۱۴۰۴-۱۴۰۵';
```

۱۴-۱۱ ساختن بقیه‌ی داده‌ی نمونه

در فصل ۷ سه جدول نمونه را از فایل وارد کردید. سه جدول دیگر، یعنی `neighborhoods`، `health_centers` و `parcels`، در فایل `sample.sql` همراه کتاب به صورت دستورهای `CREATE TABLE` و `INSERT` آمده‌اند. حالا که این دستورها را می‌شناسید، فایل را باز کنید، بخوانید و هر دستور را با تیک «اجازه تغییر داده» در کنسول اجرا کنید (یا کل فایل را یک‌جا با psql: `psql "…" -f sample.sql`). پس از آن پایگاه شما دقیقاً همان داده‌ی کتاب را دارد.

خودتان امتحان کنید

جدولی برای ثبت شکایت‌های شهروندان طراحی و بسازید: کد یکتا، متن، دسته (یکی از «پسماند»، «روشنایی»، «آسفالت»)، تاریخ ثبت با پیش‌فرض امروز، وضعیت با پیش‌فرض «باز» و هندسه‌ی نقطه در ۴۳۲۶ با قید محدوده‌ی ایران. سه شکایت درج کنید، سپس با `UPDATE … FROM` نام محله را به هر شکایت نسبت دهید.

خلاصه‌ی فصل

- قیدها (`PRIMARY KEY`، `NOT NULL`، `UNIQUE`، `REFERENCES`، `CHECK`، نوع و SRID هندسه) کیفیت داده را برای همه‌ی نرم‌افزارها تضمین می‌کنند.
- `CREATE TABLE AS SELECT …` نتیجه‌ی تحلیل را جدول می‌کند؛ بلافاصله کلید اصلی و شاخص مکانی بگذارید.
- `INSERT` (با نام ستون‌ها)، `UPDATE` و `DELETE` (همیشه با WHERE)؛ `RETURNING` سطرهای تغییرکرده را برمی‌گرداند.
- `UPDATE … FROM … WHERE ST_Contains(…)` ویژگی را بر اساس مکان منتقل می‌کند.
- `ON CONFLICT` درج-یا-به‌روزرسانی است. تراکنش (ACID) تغییرها را همه‌یاهیچ می‌کند؛ DDL هم در PostgreSQL تراکنشی است.
- نما همیشه به‌روز و بدون داده؛ نمای مادی‌شده سریع ولی با REFRESH.

برای مدرس

زمان پیشنهادی: دو جلسه. در جلسه‌ی اول قیدها را با «حمله» آموزش دهید: یک جدول با قید بسازید و از دانشجوها بخواهید داده‌ی بد واردش کنند. در جلسه‌ی دوم تراکنش را با psql نشان دهید: `UPDATE` بدون WHERE داخل `BEGIN` و بعد `ROLLBACK`؛ آرامشی که این ابزار می‌دهد را دانشجوها فراموش نمی‌کنند.

تمرین‌ها

۱. سطح ۱ جدول `trees` (درخت‌های خیابانی) با نوع گونه، ارتفاع به متر (مثبت)، سال کاشت و هندسه‌ی نقطه بسازید.

۲. سطح ۱ با `CREATE TABLE AS`، جدول `big_parks` از پارک‌های بزرگ‌تر از ۲۰ هکتار بسازید و کلید و شاخص اضافه کنید.

۳. سطح ۲ به `health_centers` ستون `neighborhood` اضافه و با `UPDATE … FROM` پر کنید.

۴. سطح ۲ نمایی بسازید که فقط نام، نوع و هندسه‌ی بیمارستان‌ها را نشان دهد. این نما را در مرورگر سامانه باز کنید.

۵. سطح ۲ داخل یک تراکنش در psql همه‌ی مدرسه‌های بالای ۸۰۰ دانش‌آموز را حذف کنید، تعداد باقی‌مانده را بشمارید و `ROLLBACK` کنید.

۶. سطح ۳ برای جدول `bus_stops` یک upsert بنویسید که اگر `code` تکراری بود، فقط وقتی نام را عوض کند که نام تازه خالی نباشد (راهنمایی: `WHERE` در `DO UPDATE`).

بخش ۴

SQL مکانی با PostGIS

اینجا SQL «مکان» را می‌فهمد: هندسه می‌سازیم، فاصله و مساحت را درست اندازه می‌گیریم، روابط مکانی را می‌پرسیم، لایه‌ها را برش و ادغام می‌کنیم، نزدیک‌ترین همسایه را پیدا می‌کنیم و پرسش‌ها را با شاخص مکانی سریع می‌کنیم.

۱۵ هندسه در SQL؛ ساختن، خواندن و قالب‌ها

۱۶ اندازه‌گیری؛ فاصله، طول، مساحت و جهت

۱۷ روابط مکانی و پیوند مکانی

۱۸ پردازش هندسه؛ حریم، ادغام، برش و ساده‌سازی

۱۹ نزدیک‌ترین همسایه (KNN)

۲۰ شاخص مکانی و کارایی پرسش‌ها

۲۱ آمار فضایی با SQL؛ تراکم، خوشه و کانون داغ

۲۲ اعتبار هندسه، توپولوژی و هندسه‌ی سه‌بعدی

۲۳ رستر در PostGIS

۲۴ تحلیل شبکه و مسیریابی با pgRouting

فصل

۱۵

هندسه در SQL؛ ساختن، خواندن و قالب‌ها

سطح: **متوسط** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل‌های ۲، ۳ و ۱۴**

در این فصل یاد می‌گیرید

- نقطه، خط و چندضلعی را با توابع سازنده بسازید.
- هندسه را به قالب‌های KML، GeoJSON، WKB، EWKT، WKT و GML تبدیل کنید و از آن‌ها بخوانید.
- اجزای هندسه (نوع، مختصات، تعداد رأس، حلقه‌ها، اجزای چندبخشی) را استخراج کنید.
- تفاوت geometry و geography را دقیق بدانید و برای هر کار نوع مناسب را انتخاب کنید.
- مرکز، نقطه‌ی روی سطح و محدوده‌ی هندسه را بگیرید.

سرفصل‌ها

۱۵-۱ هندسه، یک نوع داده

۱۵-۲ ساختن هندسه

۱۵-۳ خروجی گرفتن: قالب‌ها

۱۵-۴ خواندن اجزای هندسه

۱۵-۵ مرکز، نقطه‌ی روی سطح و محدوده

۱۵-۶ geography یا geometry؟

۱۵-۱ هندسه، یک نوع داده

در PostGIS، هندسه یک **نوع داده** است، درست مثل عدد و متن. می‌توان آن را در ستون ذخیره کرد، به تابع داد، از تابع گرفت، مقایسه کرد و در `WHERE` و `JOIN` به کار برد. توابع PostGIS را می‌توان در پنج دسته دید و این بخش به ترتیب همین دسته‌ها پیش می‌رود:

دسته	کار	نمونه
سازنده	ساختن هندسه	`ST_MakePoint`، `ST_GeomFromText`
خروجی	تبدیل به قالب متنی یا دودویی	`ST_AsText`، `ST_AsGeoJSON`
دسترسی	خواندن اجزا	`ST_X`، `GeometryType`، `ST_NPoints`
اندازه‌گیری و رابطه	عدد یا درست/نادرست	`ST_Area`، `ST_Intersects` (فصل‌های ۱۶ و ۱۷)
پردازش	هندسه‌ی تازه از هندسه	`ST_Buffer`، `ST_Union` (فصل ۱۸)

۱۵-۲ ساختن هندسه

نقطه

```sql
SELECT ST_AsText(ST_MakePoint(51.389, 35.689))
AS no_srid,
       ST_SRID(ST_MakePoint(51.389, 35.689))                    AS
srid_0,
       ST_AsEWKT(ST_SetSRID(ST_MakePoint(51.389, 35.689), 4326))   AS
with_srid,
       ST_AsEWKT(ST_Point(51.389, 35.689, 4326))                AS
shortcut;
```

no_srid	srid_0	with_srid	shortcut
POINT(51.389 35.689)	0	SRID=4326;POINT(51.389 35.689)	SRID=4326;POINT(51.389 35.689)

`ST_MakePoint(x, y)` نقطه‌ای **بدون SRID** (صفر) می‌سازد؛ تقریباً همیشه باید با `ST_SetSRID` آن را برچسب بزنید. در PostGIS ۳٫۲ به بعد، `ST_Point(x, y, srid)` هر دو کار را یک‌جا انجام می‌دهد. برای نقطه‌ی سه‌بعدی `ST_MakePoint(x, y, z)` .

> **هشدار**
>
> **باز هم ترتیب x و y.** همه‌ی سازنده‌ها اول طول (x) و بعد عرض (y) می‌گیرند. اگر داده‌ی شما ستون‌های `lat` و `lon` دارد، بنویسید `ST_MakePoint(lon, lat)` . این خطا آن‌قدر رایج است که ارزش دارد دوباره گفته شود.

ساختن ستون هندسه از دو ستون عددی، الگوی پرکاربردی برای داده‌ای است که از اکسل آمده:

```sql
ALTER TABLE customers ADD COLUMN geom geometry(Point, 4326);
UPDATE customers SET geom = ST_SetSRID(ST_MakePoint(lon, lat), 4326);
CREATE INDEX ON customers USING gist (geom);
```

از متن و قالب‌های دیگر

```sql
SELECT ST_AsText(ST_GeomFromText('LINESTRING(51.38 35.70, 51.40
35.71, 51.42 35.70)', 4326)) AS from_wkt,
       ST_AsText(ST_GeomFromGeoJSON('{"type":"Point","coordinates":
[51.42,35.73]}'))           AS from_geojson,
       ST_SRID('SRID=4326;POINT(51.4 35.7)'::geometry)
AS from_ewkt;
```

from_wkt	from_geojson	from_ewkt
LINESTRING(51.38 35.7,51.4 35.71,51.42 35.7)	POINT(51.42 35.73)	4326

آخرین ستون نشان می‌دهد متن EWKT را می‌توان مستقیم با `::geometry` به هندسه تبدیل کرد. `ST_GeomFromGeoJSON` SRID را صفر می‌گذارد (GeoJSON طبق استاندارد همیشه WGS84 است) و باید آن را با `ST_SetSRID(…, 4326)` تکمیل کنید.

خط و چندضلعی از نقطه‌ها

```sql
SELECT ST_AsText(ST_MakeLine(ST_Point(51.38, 35.70), ST_Point(51.42, 35.72)))  AS line,
       ST_AsText(ST_MakeEnvelope(51.38, 35.70, 51.42, 35.72, 4326))  AS envelope,
       ST_AsText(ST_MakePolygon(ST_GeomFromText(
         'LINESTRING(51.38 35.70, 51.42 35.70, 51.40 35.73, 51.38 35.70)')))   AS triangle;
```

line	envelope	triangle
LINESTRING(51.38 35.7,51.42 35.72)	POLYGON((51.38 35.7,51.38 35.72,51.42 35.72,51.42 35.7,51.38 35.7))	POLYGON((51.38 35.7,51.42 35.7,51.4 35.73,51.38 35.7))

`ST_MakeLine` به‌عنوان **تابع تجمیعی** هم کار می‌کند و این یکی از کاربردی‌ترین ترفندهاست: ساختن **مسیر** از نقطه‌های پشت‌سرهم (مثلاً ردپای GPS یک خودرو). برای مثال، خط هر مسیر مترو از ایستگاه‌هایش:

```sql
SELECT line,
       ST_NPoints(ST_MakeLine(geom ORDER BY station_id))  AS vertices,
       round((ST_Length(ST_MakeLine(geom ORDER BY station_id)::geography) / 1000)::numeric, 1) AS length_km
FROM metro_stations
GROUP BY line
ORDER BY line;
```

line	vertices	length_km
خط ۱	19	24.5
خط ۲	27	30.5
خط ۳	15	19.5
خط ۴	19	21.9

`ORDER BY` داخل تابع تجمیعی ترتیب نقطه‌ها را در خط تعیین می‌کند. بدون آن، خط زیگزاگ می‌زند.

بدانید

خطای `round(double precision, integer) does not exist`. توابع اندازه‌گیری PostGIS عدد از نوع `double precision` برمی‌گردانند، اما `round` با تعداد رقم اعشار فقط روی `numeric` تعریف شده است. اگر بنویسید `round(ST_Length(g), 1)` این خطا را می‌گیرید. راه حل، تبدیل نوع است: `round(ST_Length(g)::numeric, 1)`. تک‌آرگومانی (گرد کردن به عدد صحیح) روی `double precision` هم کار می‌کند.

۱۵-۳ خروجی گرفتن: قالب‌ها

تابع	قالب	کاربرد
`ST_AsText`	WKT	خواندن انسانی، دیباگ
`ST_AsEWKT`	WKT با SRID	تبادل بین پایگاه‌های PostGIS
`ST_AsBinary` ، `ST_AsEWKB`	WKB دودویی	تبادل سریع بین نرم‌افزارها
`ST_AsGeoJSON`	GeoJSON	وب و برنامه‌نویسی
`ST_AsKML`	KML	Google Earth
`ST_AsGML`	GML	سرویس‌های OGC (WFS)
`ST_AsSVG`	مسیر SVG	ترسیم مستقیم در صفحه‌ی وب
`ST_AsMVT`	کاشی برداری	نقشه‌ی وب سریع (فصل ۴۵)
`ST_GeoHash`	رشته‌ی geohash	شناسه‌ی مکانی کوتاه

```sql
SELECT ST_AsGeoJSON(geom, 5)  AS geojson,
       ST_AsKML(geom, 5)      AS kml,
       ST_GeoHash(geom, 8)    AS geohash
FROM metro_stations WHERE gid = 1;
```

eojson	kml	geohash
"type":"Point","coordinates": 51.43,35.8]}	<Point> <coordinates>51.43,35.8</coordinates> </Point>	tnkedm4n

عدد دوم در `ST_AsGeoJSON(geom, 5)` تعداد رقم اعشار است. پنج رقم در درجه حدود یک متر دقت دارد (فصل ۲) و حجم خروجی را بسیار کم می‌کند.

در دنیای واقعی

یک FeatureCollection کامل با یک پرسش. برنامه‌ی وبی به یک فایل GeoJSON از مدرسه‌های بزرگ نیاز دارد. PostgreSQL می‌تواند کل FeatureCollection را مستقیم بسازد، بدون هیچ کد واسطی:

```sql
SELECT json_build_object(
  'type', 'FeatureCollection',
  'features', json_agg(ST_AsGeoJSON(s.*, 'geom', 5)::json)
)
FROM (SELECT name, students, geom FROM schools WHERE students >
800) s;
```

`ST_AsGeoJSON` وقتی یک سطر کامل بگیرد، ستون هندسه را geometry و بقیه را `properties` می‌کند. انتشار لایه در سامانه (فصل ۴۵) همین کار را با کش و کلید API انجام می‌دهد.

۴-۱۵ خواندن اجزای هندسه

```sql
SELECT name,
       GeometryType(geom)      AS type,
       ST_NPoints(geom)        AS vertices,
       ST_NumInteriorRings(geom) AS holes,
       ST_Dimension(geom)      AS dim,
       round(ST_X(ST_Centroid(geom))::numeric, 4) AS cx
FROM parks
ORDER BY park_id
LIMIT 3;
```

name	type	vertices	holes	dim	cx
پارک 1	POLYGON	6	0	2	51.4641
پارک 2	POLYGON	7	0	2	51.3830
پارک 3	POLYGON	7	0	2	51.2839

تابع	نتیجه
`GeometryType(g)` ، `ST_GeometryType(g)`	نام نوع (`POLYGON` یا `ST_Polygon`)
`ST_X(g)` ، `ST_Y(g)` ، `ST_Z(g)`	مختصات نقطه
`ST_NPoints(g)`	تعداد رأس‌ها
`ST_NumGeometries(g)`	تعداد اجزای هندسه‌ی چندبخشی
`ST_GeometryN(g, n)`	جزء nام
`ST_ExteriorRing(g)` ، `ST_InteriorRingN(g, n)`	حلقه‌ی بیرونی و حفره‌ها
`ST_StartPoint(g)` ، `ST_EndPoint(g)` ، `ST_PointN(g, n)`	نقطه‌های خط
`ST_Dimension(g)`	بعد: ۰ نقطه، ۱ خط، ۲ سطح
`ST_Envelope(g)`	مستطیل محدوده
`ST_XMin` ، `ST_YMax` ، …	مختصات محدوده

شکافتن هندسه: ST_Dump و ST_DumpPoints

`ST_Dump` هندسه‌ی چندبخشی را به اجزایش می‌شکند و برای هر جزء یک **سطر** برمی‌گرداند؛ برای «جدا کردن جزیره‌ها» یا تبدیل MultiPolygon به Polygon به کار می‌رود. `ST_DumpPoints` همین کار را با رأس‌ها می‌کند:

```sql
SELECT (d).path[2] AS vertex_no,
       ST_AsText((d).geom) AS vertex
FROM (SELECT ST_DumpPoints(geom) AS d FROM neighborhoods WHERE name =
'ونک') x;
```

vertex_no	vertex
1	POINT(51.375 35.739999999999995)
2	POINT(51.375 35.78)
3	POINT(51.415000000000006 35.78)
4	POINT(51.415000000000006 35.739999999999995)
5	POINT(51.375 35.739999999999995)

پنج رأس برای یک مربع: چهار گوشه و تکرار نقطه‌ی اول که حلقه را می‌بندد (فصل ۲).

۱۵-۵ مرکز، نقطه‌ی روی سطح و محدوده

برای گذاشتن برچسب یا نماد روی چندضلعی به یک نقطه نیاز داریم. دو انتخاب داریم:

- `ST_Centroid` مرکز ثقل هندسی است. برای شکل‌های محدب درست است، اما برای شکل هلالی یا «U» شکل ممکن است **بیرون** چندضلعی بیفتد.
- `ST_PointOnSurface` نقطه‌ای را برمی‌گرداند که **تضمین می‌شود** داخل چندضلعی باشد.

```sql
WITH u AS (SELECT ST_GeomFromText('POLYGON((0 0, 10 0, 10 10, 8 10,
8 2, 2 2, 2 10, 0 10, 0 0))') AS g)
SELECT ST_AsText(ST_Centroid(g))                    AS centroid,
       ST_Contains(g, ST_Centroid(g))               AS centroid_inside,
       ST_AsText(ST_PointOnSurface(g))              AS
point_on_surface,
       ST_Contains(g, ST_PointOnSurface(g))         AS pos_inside
FROM u;
```

centroid	centroid_inside	point_on_surface	pos_inside
POINT(5 4.076923076923077)	False	POINT(1 6)	True

مرکز ثقل شکل U بیرون آن افتاد. برای برچسب‌گذاری و پیوند «نقطه در چندضلعی» همیشه `ST_PointOnSurface` امن‌تر است.

۱۵-۶ geography یا geometry؟

PostGIS دو نوع مکانی برداری دارد و انتخاب بین آن‌ها یکی از تصمیم‌های مهم طراحی است.

- `geometry` مختصات را روی یک **صفحه‌ی تخت** (دکارتی) می‌بیند. همه‌ی محاسبات ساده و سریع‌اند و واحد نتیجه همان واحد SRID است: در ۴۳۲۶ درجه (بی‌معنا برای اندازه‌گیری)، در UTM متر.
- `geography` مختصات را روی **بیضوی زمین** می‌بیند. فاصله و مساحت همیشه به متر و متر مربع و روی سطح خمیده‌ی زمین حساب می‌شود، در هر جای جهان، بدون نیاز به انتخاب تصویر نقشه. اما توابع کمتری دارد و کندتر است.

معیار	geometry	geography
مدل زمین	صفحه‌ی تخت	بیضوی (WGS84)
واحد اندازه‌گیری	واحد SRID	همیشه متر
تعداد توابع	صدها	ده‌ها (مهم‌ترین‌ها)
سرعت	سریع‌تر	کندتر
محدوده‌ی مناسب	محلی و منطقه‌ای (با SRID متری)	کشوری، قاره‌ای، جهانی
SRID	هر سیستمی	جغرافیایی (معمولاً ۴۳۲۶)

```sql
SELECT ST_Distance('SRID=4326;POINT(51.39 35.69)'::geometry,
'SRID=4326;POINT(59.61 36.30)'::geometry)  AS tehran_mashhad_geom,
       round(ST_Distance('SRID=4326;POINT(51.39 35.69)'::geography,
'SRID=4326;POINT(59.61 36.30)'::geography) / 1000) AS
tehran_mashhad_km;
```

tehran_mashhad_geom	tehran_mashhad_km
8.2426	744.0

فاصله‌ی تهران تا مشهد در `geometry` با SRID ۴۳۲۶ یک عدد بی‌معنا به درجه است؛ در `geography` حدود ۷۴۰ کیلومتر، که فاصله‌ی ژئودزیک (کوتاه‌ترین مسیر روی سطح زمین) است.

راهبرد پیشنهادی این کتاب: ستون را `geometry` با SRID ۴۳۲۶ ذخیره کنید (بیشترین سازگاری با نرم‌افزارها و توابع) و هر جا اندازه‌گیری لازم بود، یا موقتاً به `:: geography` تبدیل کنید یا با `ST_Transform` به UTM ببرید. فصل ۱۶ این دو راه را از نظر دقت و سرعت مقایسه می‌کند.

سطح حرفه‌ای

شاخص روی عبارت. اگر پرسش‌هایتان مدام `geom::geography` را در `ST_DWithin` به کار می‌برند، شاخص معمولی روی `geom` استفاده نمی‌شود. یک شاخص روی خود عبارت بسازید: `CREATE INDEX ON schools USING gist ((geom::geography));`. فصل ۲۰ شاخص‌ها را مفصل بررسی می‌کند.

خودتان امتحان کنید

ردپای یک پیاده‌روی را بسازید: جدولی از پنج نقطه با ستون زمان درج کنید (`WITH` و `VALUES` کافی است)، با `ST_MakeLine(geom ORDER BY t)` خط مسیر را بسازید و طولش را به متر حساب کنید. سپس خروجی GeoJSON مسیر را بگیرید و در geojson.io یا QGIS ببینید.

خلاصه‌ی فصل

- سازنده‌ها: `ST_Point(x, y, srid)`، `ST_SetSRID + ST_MakePoint`، `ST_GeomFromText`، `ST_GeomFromGeoJSON`، `ST_MakeLine`، `ST_MakeEnvelope`.
- `ST_MakeLine(geom ORDER BY t)` به‌عنوان تجمیعی، نقطه‌ها را مسیر می‌کند.
- خروجی: GeoJSON، WKB، EWKT، WKT (با تعداد رقم)، MVT، SVG، GML، KML؛ `ST_AsGeoJSON(row.*)` عارضه‌ی کامل می‌سازد.
- دسترسی: `GeometryType`، `ST_X/Y`، `ST_NPoints`، `ST_Dump` و `ST_DumpPoints` برای شکافتن.
- `ST_PointOnSurface` برخلاف `ST_Centroid` همیشه داخل چندضلعی است.
- `geometry` تخت و پرتابع؛ `geography` روی بیضوی و همیشه به متر. ذخیره با geometry ۴۳۲۶، اندازه‌گیری با geography یا UTM.

برای مدرس

زمان پیشنهادی: یک و نیم جلسه. مثال شکل U را حتماً روی تخته بکشید و بپرسید «برچسب را کجا بگذاریم؟». تمرین «ردپای پیاده‌روی» را با GPS گوشی خود دانشجوها انجام دهید (صادرات GPX از یک برنامه‌ی ردیابی و ورود با فصل ۷).

تمرین‌ها

۱. سطح ۱ نقطه‌ای در مختصات میدان نقش جهان اصفهان (طول ۵۱٫۶۷۸، عرض ۳۲٫۶۵۷) با SRID ۴۳۲۶ بسازید و EWKT آن را ببینید.

۲. سطح ۱ GeoJSON پنج مدرسه‌ی اول را با سه رقم اعشار بگیرید.

۳. سطح ۲ تعداد رأس هر محله را بدهید. چرا همه یکسان‌اند؟

۴. سطح ۲ فاصله‌ی تهران تا تبریز (۴۶٫۲۹، ۳۸٫۰۸) را با `geography` و پس از تبدیل هر دو نقطه به UTM ۳۹ حساب کنید. اختلاف چقدر است و چرا؟

۵. سطح ۳ یک FeatureCollection از ایستگاه‌های خط ۲ بسازید که در `properties` فقط نام و خط باشد.

فصل

۱۶

اندازه‌گیری: فاصله، طول، مساحت و جهت

سطح: **متوسط** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۱۵**

در این فصل یاد می‌گیرید

- فاصله، طول، محیط و مساحت را با واحد درست (متر، کیلومتر، هکتار) حساب کنید.
- دقت سه روش geography، UTM و Web Mercator را با هم مقایسه کنید و منبع خطای هر کدام را توضیح دهید.
- برای محاسبه در مقیاس کشور از تصویر هم‌مساحت سفارشی استفاده کنید.
- جهت (آزیموت) بین دو نقطه را بگیرید و نقطه‌ای را در فاصله و جهت معین بسازید.
- مساحت گزارش‌شده در داده را با مساحت هندسی بسنجید.

سرفصل‌ها

۱۶-۱ چهار پرسش اندازه‌گیری

۱۶-۲ سه روش، سه دقت

۱۶-۳ جدول تصمیم

۱۶-۴ طول خط‌ها

۱۶-۵ جهت و مسیریابی هوایی

۱۶-۶ فاصله بین چندضلعی‌ها و خط‌ها

۱۶-۱ چهار پرسش اندازه‌گیری

پرسش	تابع	نتیجه برای geography	نتیجه برای geometry
فاصله‌ی دو عارضه چقدر است؟	`ST_Distance(a, b)`	متر	واحد SRID
طول این خط چقدر است؟	`ST_Length(g)`	متر	واحد SRID
محیط این چندضلعی چقدر است؟	`ST_Perimeter(g)`	متر	واحد SRID
مساحت این چندضلعی چقدر است؟	`ST_Area(g)`	متر مربع	مربع واحد SRID

قاعده‌ی فصل ۳ را اینجا به کار می‌بندیم: **روی geometry با SRID ۴۳۲۶ هرگز اندازه نگیرید.** یا به `geography` تبدیل کنید، یا به یک SRID متری مناسب.

```sql
SELECT name,
       area_ha                                          AS
area_reported_ha,
       round((ST_Area(geom::geography) / 10000)::numeric, 2) AS
area_ha,
       round(ST_Perimeter(geom::geography))             AS
perimeter_m
FROM parks
ORDER BY area_ha DESC
LIMIT 4;
```

name	area_reported_ha	area_ha	perimeter_m
پارک 15	81.8	81.27	3901.0
پارک 27	68.2	67.84	3509.0
پارک 25	63.2	62.86	3444.0
پارک 8	45.8	45.55	2950.0

تبدیل واحد را در خاطر داشته باشید: یک هکتار ۱۰٬۰۰۰ متر مربع و یک کیلومتر مربع ۱۰۰ هکتار (یک میلیون متر مربع) است.

۱۶-۲ سه روش، سه دقت

برای اندازه‌گیری با geometry باید به یک سیستم متری ببریم. اما کدام؟ این پرسش را با آزمایش پاسخ می‌دهیم: مساحت محله‌ی ونک را با چهار روش حساب می‌کنیم.

```sql
SELECT round(ST_Area(geom::geography))                    AS
geography_m2,
       round(ST_Area(ST_Transform(geom, 32639)))          AS
utm39_m2,
       round(ST_Area(ST_Transform(geom, 3857)))           AS
webmercator_m2,
       ST_Area(geom)                                      AS
degree2
FROM neighborhoods
WHERE name = 'ونک';
```

geography_m2	utm39_m2	webmercator_m2	degree2
16054873.0	16042536.0	24433680.0	0.0016

- **geography** مساحت را روی بیضوی WGS84 حساب می‌کند؛ این را مرجع می‌گیریم.
- **UTM ۳۹** با مرجع کمتر از یک‌دهم درصد اختلاف دارد؛ برای بیشتر کارهای مهندسی کافی است.
- **Web Mercator** حدود **۵۰ درصد** بیشتر است! (ضریب مساحت در عرض تهران حدود $1/\cos^2 35.7° \approx 1.52$ است.)
- **درجه‌ی مربع** عدد بی‌معنایی است.

همین آزمایش را برای فاصله‌ی دو مدرسه‌ی دور از هم تکرار کنیم:

```sql
SELECT round(ST_Distance(a.geom::geography, b.geom::geography))
AS geography_m,
       round(ST_Distance(ST_Transform(a.geom, 32639),
ST_Transform(b.geom, 32639))) AS utm39_m,
       round(ST_Distance(ST_Transform(a.geom, 3857),
ST_Transform(b.geom, 3857)))   AS webmercator_m,
       round(ST_DistanceSphere(a.geom, b.geom))
AS sphere_m
FROM schools a, schools b
WHERE a.school_id = 1 AND b.school_id = 10;
```

geography_m	utm39_m	webmercator_m	sphere_m
23690.0	23681.0	29120.0	23641.0

`ST_DistanceSphere` زمین را **کره** با شعاع متوسط فرض می‌کند؛ سریع‌تر از geography و با خطای کمتر از نیم درصد. Web Mercator فاصله را هم حدود ۲۳ درصد بیشتر نشان می‌دهد (ضریب خطی $1/\cos\varphi$).

هشدار

«نقشه‌ی وب نشان می‌دهد...» اگر کسی فاصله یا مساحت را با ابزار اندازه‌گیری یک نقشه‌ی وب ساده یا در QGIS با پروژه‌ای در ۳۸۵۷ و بدون اندازه‌گیری بیضوی حساب کرده، عددش در ایران ۲۰ تا ۵۰ درصد خطا دارد. QGIS به‌طور پیش‌فرض اندازه‌گیری بیضوی دارد (Project Properties ← General ← Ellipsoid)؛ مطمئن شوید خاموش نیست.

UTM در لبه‌ی ناحیه

UTM در نزدیکی نصف‌النهار مرکزی بسیار دقیق است و به سمت لبه‌ی ناحیه خطایش بیشتر می‌شود. اگر داده‌ای را به ناحیه‌ی **اشتباه** ببرید (مثلاً مشهد را به ناحیه‌ی ۳۹)، خطا چندین برابر می‌شود؛ در مثال زیر از حدود ۰٫۰۵ درصد به حدود ۱٫۴ درصد. برای داده‌ای که چند ناحیه را می‌پوشاند (مثلاً کل ایران) UTM مناسب نیست.

```sql
WITH m AS (SELECT ST_Buffer('SRID=4326;POINT(59.6 36.3)'::geography,
5000)::geometry AS g)   -- دایره‌ی ۵ کیلومتری در مشهد
SELECT round(ST_Area(g::geography))           AS geography_m2,
       round(ST_Area(ST_Transform(g, 32640)))  AS utm40_correct_zone,
       round(ST_Area(ST_Transform(g, 32639)))  AS utm39_wrong_zone
FROM m;
```

geography_m2	utm40_correct_zone	utm39_wrong_zone
77993742.0	78036129.0	79084933.0

مساحت در مقیاس کشور: تصویر هم‌مساحت

برای آمار مساحت در سطح کل کشور (مثلاً مساحت جنگل‌های هر استان) دو راه داریم: `geography`، که ساده و دقیق است ولی روی چندضلعی‌های بسیار بزرگ و پرجزئیات کند است، یا یک **تصویر هم‌مساحت** که برای ایران تنظیم شده باشد. `ST_Transform` می‌تواند به جای کد EPSG، تعریف کامل یک تصویر را به صورت متن PROJ بگیرد. تصویر مخروطی هم‌مساحت آلبرز با دو مدار استاندارد درون ایران:

```sql
SELECT round(ST_Area(geom::geography)) AS geography_m2,
       round(ST_Area(ST_Transform(geom,
         '+proj=aea +lat_1=27 +lat_2=38 +lat_0=32 +lon_0=54 +x_0=0
+y_0=0 +datum=WGS84 +units=m +no_defs'))) AS albers_iran_m2
FROM neighborhoods
WHERE name = 'ونک';
```

geography_m2	albers_iran_m2
16054873.0	16054873.0

دو عدد تقریباً یکسان‌اند. در تصویر آلبرز، مساحت در سراسر ایران حفظ می‌شود و محاسبه روی هندسه‌ی تخت انجام می‌گیرد که از محاسبه‌ی بیضوی سریع‌تر است.

۱۶-۳ جدول تصمیم

موقعیت	روش پیشنهادی
یک شهر یا استان در یک ناحیه‌ی UTM	`ST_Transform(geom, 326xx)` ؛ یا ستون geometry ذخیره‌شده در همان UTM
داده‌ی پراکنده در چند ناحیه یا کل کشور	`::geography`
آمار مساحت کشوری روی داده‌ی حجیم	تصویر هم‌مساحت آلبرز یا لامبرت برای ایران
فاصله‌ی سریع تقریبی بین نقطه‌ها	`ST_DistanceSphere`
نمایش روی نقشه‌ی وب	۳۸۵۷؛ ولی هرگز برای اندازه‌گیری

۱۶-۴ طول خط‌ها

```sql
WITH routes AS (
  SELECT line, ST_MakeLine(geom ORDER BY station_id) AS g
  FROM metro_stations GROUP BY line
)
SELECT line,
       round((ST_Length(g::geography) / 1000)::numeric, 2)
AS km_geography,
       round((ST_Length(ST_Transform(g, 32639)) / 1000)::numeric, 2)
AS km_utm39
FROM routes
ORDER BY line;
```

line	km_geography	km_utm39
خط ۱	24.53	24.52
خط ۲	30.49	30.48
خط ۳	19.51	19.50
خط ۴	21.88	21.87

طول خط چندرأسی جمع طول پاره‌خط‌های آن است. یادتان باشد که این «طول هوایی» بین ایستگاه‌هاست؛ طول واقعی مسیر روی شبکه‌ی خیابان‌ها موضوع فصل ۲۴ است.

۱۶-۵ جهت و مسیریابی هوایی

آزیموت زاویه‌ی جهت از شمال، در جهت حرکت عقربه‌های ساعت است (۰ شمال، ۹۰ شرق، ۱۸۰ جنوب، ۲۷۰ غرب). `ST_Azimuth` آن را به رادیان برمی‌گرداند و `degrees` به درجه تبدیلش می‌کند:

```sql
SELECT round(degrees(ST_Azimuth('SRID=4326;POINT(51.39 35.69)'::geography,
                                'SRID=4326;POINT(59.61 36.30)'::geography))::numeric, 1) AS tehran_to_mashhad_deg,
       ST_AsText(ST_Project('SRID=4326;POINT(51.39 35.69)'::geography, 10000, radians(45))::geometry) AS point_10km_northeast;
```

tehran_to_mashhad_deg	point_10km_northeast
82.4	POINT(51.468182397657564 35.75370442875681)

`ST_Project(نقطه، فاصله، آزیموت)` نقطه‌ای در فاصله و جهت معین می‌سازد؛ برای ترسیم پرتو دید دوربین، جهت آنتن یا محدوده‌ی جست‌وجو.

> **بدانید**
>
> **ژئودزیک چیست؟** کوتاه‌ترین مسیر بین دو نقطه روی سطح زمین را **ژئودزیک** (روی کره، دایره‌ی عظیمه) می‌گویند. روی نقشه‌ی مرکاتور، ژئودزیک یک **منحنی** است، نه خط راست؛ به همین دلیل مسیر پروازهای طولانی روی نقشه‌ی تخت خمیده دیده می‌شود. فاصله‌ای که `geography` می‌دهد طول همین ژئودزیک روی بیضوی است.

۱۶-۶ فاصله بین چندضلعی‌ها و خط‌ها

`ST_Distance` بین دو چندضلعی، **کوتاه‌ترین** فاصله‌ی لبه‌هایشان است (صفر اگر هم‌پوشانی یا تماس داشته باشند)، نه فاصله‌ی مرکزها. برای «دورترین فاصله» `ST_MaxDistance` و برای «میزان شباهت دو شکل» `ST_HausdorffDistance` هست. خود پاره‌خط کوتاه‌ترین فاصله را `ST_ShortestLine` می‌سازد که برای ترسیم «خط اتصال به نزدیک‌ترین» روی نقشه مفید است:

```sql
SELECT p.name,
       round(ST_Distance(p.geom::geography, s.geom::geography)) AS
edge_distance_m,
       round(ST_Distance(ST_Centroid(p.geom)::geography,
s.geom::geography)) AS center_distance_m,
       ST_AsText(ST_ShortestLine(p.geom, s.geom)) AS connector
FROM parks p, schools s
WHERE s.school_id = 1
ORDER BY edge_distance_m
LIMIT 3;
```

name	edge_distance_m	center_distance_m	connector
پارک 25	1326.0	1675.0	LINESTRING(51.53559977106712 35.64350828576881,51.532393 35.655237)
پارک 27	2865.0	3226.0	LINESTRING(51.514865006395375 35.67683762083082,51.532393 35.655237)
پارک 8	3573.0	4058.0	LINESTRING(51.51399643949921 35.683725710300294,51.532393 35.655237)

در دنیای واقعی

بررسی مساحت ثبتی. در پروژه‌ای کاداستری، ستون «مساحت سندی» هر قطعه با مساحت هندسی آن مقایسه می‌شود تا قطعه‌هایی که اختلاف بیش از ۵ درصد دارند برای بازبینی میدانی فهرست شوند: `SELECT id FROM parcels_cadastre WHERE abs(ST_Area(geom::geography) - deed_area) / deed_area > 0.05;`. همین منطق را روی پارک‌های داده‌ی نمونه در تمرین ۴ امتحان کنید.

خلاصه‌ی فصل

- `ST_Distance`، `ST_Length`، `ST_Perimeter` و `ST_Area` روی geography به متر و متر مربع‌اند؛ روی geometry به واحد SRID.
- در ایران، Web Mercator مساحت را حدود ۵۰٪ و فاصله را حدود ۲۳٪ بیشتر نشان می‌دهد؛ هرگز برای اندازه‌گیری به کار نرود.
- UTM ناحیه‌ی درست تقریباً هم‌دقت geography است؛ ناحیه‌ی اشتباه خطا می‌سازد.
- برای کل کشور: geography یا تصویر هم‌مساحت آلبرز با متن PROJ در `ST_Transform`.
- `ST_Azimuth` جهت و `ST_Project` نقطه در فاصله و جهت معین را می‌دهد.
- فاصله‌ی دو چندضلعی، فاصله‌ی لبه‌هاست؛ `ST_ShortestLine` خط آن را می‌سازد.

برای مدرس

زمان پیشنهادی: یک جلسه. آزمایش «چهار روش» را به صورت گروهی انجام دهید: هر گروه یک روش را روی یک عارضه حساب کند و نتیجه‌ها را روی تخته کنار هم بنویسید. سپس بپرسید «اگر عوارض نوسازی بر اساس مساحت باشد، خطای ۵۰٪ یعنی چه؟».

تمرین‌ها

۱. سطح ۱ مساحت هر محله را به کیلومتر مربع و تراکم جمعیت (نفر بر کیلومتر مربع) را حساب کنید.

۲. سطح ۱ طول هوایی هر خط مترو را به کیلومتر بدهید. کدام خط بلندتر است؟

۳. سطح ۲ آزمایش «سه روش، سه دقت» را برای یک پارک تکرار و درصد خطای هر روش را نسبت به geography حساب کنید.

۴. سطح ۲ پارک‌هایی را که مساحت گزارش‌شده (`area_ha`) با مساحت هندسی بیش از ۱ درصد اختلاف دارد پیدا کنید.

۵. سطح ۲ فاصله‌ی هر ایستگاه خط ۱ تا ایستگاه بعدی (به ترتیب `station_id`) را با `lag` و geography حساب کنید.

۶. سطح ۳ با `ST_Project`، یک «بادبزن» دید ۶۰ درجه و ۵۰۰ متری برای دوربینی در یک نقطه با جهت ۳۰ درجه بسازید (راهنمایی: چند نقطه با `ST_Project` در زاویه‌های ۰ تا ۶۰ و `ST_MakePolygon`).

فصل

۱۷

روابط مکانی و پیوند مکانی

سطح: **متوسط** | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل‌های ۱۳ و ۱۶**

در این فصل یاد می‌گیرید

- گزاره‌های مکانی (Overlaps، Crosses، Touches، Covers، Within، Contains، Intersects، Disjoint، Equals) را دقیق بشناسید.
- مدل نُه‌خانه‌ای DE-9IM را توضیح دهید و با ST_Relate رابطه‌ی دلخواه بسازید.
- تفاوت Contains و Covers را در عارضه‌های روی مرز بدانید.
- الگوهای پیوند مکانی (شمارش، انتقال ویژگی، یک‌به‌چند) را بنویسید و از شمارش مضاعف بپرهیزید.
- پرسش فاصله‌ای را با ST_DWithin به‌جای ST_Distance سریع بنویسید.

سرفصل‌ها

۱۷-۱ گزاره‌ی مکانی

۱۷-۲ مدل DE-9IM

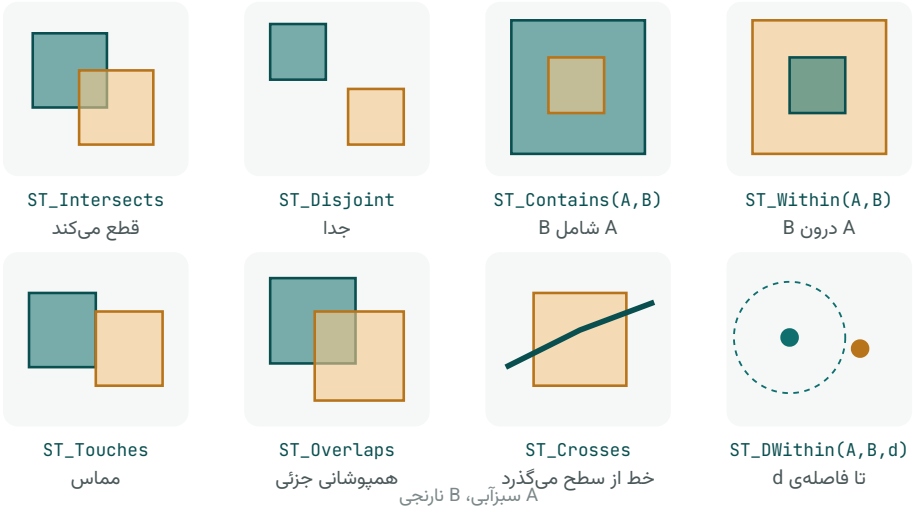
۱۷-۳ Contains یا Covers؟

۱۷-۴ الگوهای پیوند مکانی

۱۷-۵ سرعت پرسش‌های مکانی

۱۷-۱ گزاره‌ی مکانی

گزاره‌ی مکانی (spatial predicate) تابعی است که دو هندسه می‌گیرد و **درست یا نادرست** برمی‌گرداند: «آیا این مدرسه داخل این محله است؟»، «آیا این جاده از این رود می‌گذرد؟». چون نتیجه درست/نادرست است، گزاره‌ها جای طبیعی‌شان در `WHERE` و در شرط `JOIN … ON` است.



گزاره‌های مکانی پرکاربرد. همه‌ی آن‌ها از مدل DE-9IM مشتق شده‌اند.

گزاره	درست است وقتی…	نمونه‌ی پرسش
`ST_Intersects(a, b)`	هر نقطه‌ی مشترکی داشته باشند (نقیض Disjoint)	کدام قطعه‌ها با محدوده‌ی طرح برخورد دارند؟
`ST_Disjoint(a, b)`	هیچ نقطه‌ی مشترکی نداشته باشند	
`ST_Contains(a, b)`	b کاملاً داخل a باشد و دست‌کم یک نقطه از درون b داخل درون a	مدرسه‌های داخل محله
`ST_Within(a, b)`	a کاملاً داخل b باشد (= `ST_Contains(b, a)`)	
`ST_Covers(a, b)`	هیچ نقطه‌ای از b بیرون a نباشد (مرز هم قبول)	مثل Contains، امن برای عارضه‌ی روی مرز
`ST_CoveredBy(a, b)`	عکس Covers	
`ST_Touches(a, b)`	فقط مرزهایشان تماس داشته باشد، درونشان نه	قطعه‌های هم‌مرز
`ST_Crosses(a, b)`	از هم بگذرند و بعد اشتراک کمتر از بعد بزرگ‌تر باشد	جاده‌هایی که از رود می‌گذرند
`ST_Overlaps(a, b)`	هم‌بعد باشند، بخشی را مشترک داشته باشند ولی هیچ‌کدام شامل دیگری نباشد	دو محدوده‌ی طرح که بخشی روی هم افتاده‌اند
`ST_Equals(a, b)`	از نظر مکانی یکسان باشند (ترتیب رأس مهم نیست)	یافتن عارضه‌ی تکراری
`ST_DWithin(a, b, d)`	فاصله‌شان حداکثر d باشد	تا ۵۰۰ متری ایستگاه

۱۷-۲ مدل DE-9IM

همه‌ی این گزاره‌ها از یک مدل ریاضی واحد می‌آیند: **مدل نُه‌تقاطعی ابعادی‌شده** (Dimensionally Extended 9-Intersection Model). هر هندسه سه بخش دارد: **درون** (I)، **مرز** (B) و **بیرون** (E). برای دو هندسه، اشتراک هر بخش از اولی با هر بخش از دومی را بررسی

می‌کنیم؛ نُه خانه. در هر خانه، **بعد** اشتراک نوشته می‌شود: `F` (اشتراکی نیست)، `0` (نقطه)، `1` (خط) یا `2` (سطح).

	درونِ B	مرزِ B	بیرونِ B
درونِ A	I∩I	I∩B	I∩E
مرزِ A	B∩I	B∩B	B∩E
بیرونِ A	E∩I	E∩B	E∩E

`ST_Relate(a, b)` این ماتریس را به صورت رشته‌ای نُه‌حرفی برمی‌گرداند (سطر به سطر):

```sql
SELECT ST_Relate(n.geom, s.geom) AS matrix_neighborhood_school,
       ST_Relate(a.geom, b.geom) AS matrix_two_neighborhoods
FROM neighborhoods n, schools s, neighborhoods a, neighborhoods b
WHERE ST_Contains(n.geom, s.geom) AND a.name = 'ونک' AND b.name = 'شهرک غرب'
LIMIT 1;
```

matrix_neighborhood_school	matrix_two_neighborhoods
0F2FF1FF2	212101212

رشته‌ی اول، `0F2FF1FF2`، یعنی درون محله با درون مدرسه اشتراک صفربعدی (نقطه) دارد و مدرسه هیچ بخشی بیرون محله ندارد؛ یعنی «محله شامل مدرسه است». رشته‌ی دوم، `212101212`، پیام مهمی دارد: خانه‌ی اول `2` است، یعنی **درون دو محله اشتراک دوبعدی دارد؛ دو محله روی هم افتاده‌اند!** هر گزاره در واقع یک **الگو** روی این ماتریس است. مثلاً `ST_Within` یعنی الگوی `T*F**F***` (`T` = هر اشتراکی، `*` = مهم نیست)، و با `ST_Relate(a, b, 'الگو')` می‌توانید رابطه‌ای بسازید که هیچ تابع آماده‌ای ندارد.

یک کشف کیفیت داده

محله‌های یک شهر نباید روی هم بیفتند. بیایید همه‌ی همپوشانی‌ها را پیدا کنیم:

```sql
SELECT a.name AS neighborhood_a, b.name AS neighborhood_b,
       ST_Relate(a.geom, b.geom) AS matrix,
       round((ST_Area(ST_Intersection(a.geom, b.geom)::geography) /
10000)::numeric, 1) AS overlap_ha
FROM neighborhoods a
JOIN neighborhoods b ON a.gid < b.gid AND ST_Overlaps(a.geom, b.geom)
ORDER BY overlap_ha DESC;
```

neighborhood_a	neighborhood_b	matrix	overlap_ha
جمهوری	هفت‌تیر	212101212	451.8
ونک	شهرک غرب	212101212	351.2
ونک	هفت‌تیر	212101212	50.2

سه جفت محله همپوشانی دارند. محله‌های داده‌ی نمونه مربع‌های ساده‌ای‌اند که دور یک نقطه کشیده شده‌اند و این همپوشانی‌ها عمداً اصلاح نشده‌اند تا نشان دهند چنین خطایی در داده‌ی واقعی چقدر بی‌صدا می‌ماند: هیچ نرم‌افزاری خطا نمی‌دهد، فقط آمارها غلط می‌شوند. مثلاً مدرسه‌ای که در بخش همپوشان است در **دو** محله شمرده می‌شود:

```sql
SELECT count(*) AS join_rows, count(DISTINCT s.gid) AS
distinct_schools
FROM schools s JOIN neighborhoods n ON ST_Contains(n.geom, s.geom);
```

join_rows	distinct_schools
28	26

۲۸ جفت برای ۲۶ مدرسه: دو مدرسه دو بار شمرده شده‌اند. در لایه‌ی تقسیمات درست، این دو عدد باید برابر باشند و همسایه‌ها فقط در یک **خط** مرز مشترک داشته باشند؛ الگوی DE-9IM آن `FF*F1****` است (درون‌ها اشتراک ندارند، مرزها در یک خط مشترک‌اند). فصل ۲۲ ابزار اصلاح چنین لایه‌ای را آموزش می‌دهد.

۱۷-۳ Contains یا Covers؟

تفاوت ظریف ولی مهم: نقطه‌ای که **دقیقاً روی مرز** چندضلعی است، طبق تعریف DE-9IM «درون» آن نیست. پس `ST_Contains` آن را **نادرست** می‌داند، ولی `ST_Covers` درست:

```sql
WITH sq AS (SELECT ST_MakeEnvelope(0, 0, 10, 10) AS g)
SELECT ST_Contains(g, ST_Point(5, 5))    AS contains_center,
       ST_Contains(g, ST_Point(10, 5))   AS contains_on_edge,
       ST_Covers(g, ST_Point(10, 5))     AS covers_on_edge,
       ST_Intersects(g, ST_Point(10, 5)) AS intersects_on_edge
FROM sq;
```

contains_center	contains_on_edge	covers_on_edge	intersects_on_edge
True	False	True	True

در داده‌ی واقعی، عارضه‌هایی که روی مرز افتاده‌اند (چون با همان لایه‌ی مرز رقومی شده‌اند) کم نیستند. اگر می‌خواهید آن‌ها هم شمرده شوند، `ST_Covers` یا `ST_Intersects` به کار ببرید؛ اما مراقب باشید که نقطه‌ی روی مرز مشترک دو محله در **هر دو** شمرده می‌شود (پایین‌تر).

۱۷-۴ الگوهای پیوند مکانی

۱. شمارش و جمع در چندضلعی

رایج‌ترین تحلیل GIS: «چند نقطه در هر چندضلعی؟». از LEFT JOIN استفاده کنید تا چندضلعی‌های خالی هم بیایند:

```sql
SELECT n.name,
       count(h.gid)                AS centers,
       coalesce(sum(h.beds), 0)    AS beds,
       round(n.population::numeric / nullif(sum(h.beds), 0)) AS
people_per_bed
FROM neighborhoods n
LEFT JOIN health_centers h ON ST_Contains(n.geom, h.geom)
GROUP BY n.gid, n.name, n.population
ORDER BY centers DESC;
```

name	centers	beds	people_per_bed
ونک	4	90	200
هفت‌تیر	4	254	258
تجریش	2	137	201
نارمک	2	135	274
شهرک غرب	1	85	547
جمهوری	1	77	727

۲. انتقال ویژگی (Join by location)

در فصل ۱۴ با `UPDATE … FROM` نام محله را در جدول مدرسه‌ها نوشتیم. اگر نمی‌خواهید جدول را تغییر دهید، همان را در یک پرسش یا نما بیاورید:

```sql
SELECT s.name, s.students, n.name AS neighborhood
FROM schools s
LEFT JOIN neighborhoods n ON ST_Within(s.geom, n.geom)
ORDER BY n.name NULLS LAST, s.gid
LIMIT 4;
```

name	students	neighborhood
مدرسه 80	378	تجریش
مدرسه 134	511	تجریش
مدرسه 83	193	جمهوری
مدرسه 85	440	جمهوری

۳. یک‌به‌چند و شمارش مضاعف

وقتی شرط پیوند یک سطر را به **چند** سطر جفت می‌کند، نتیجه بیشتر از جدول اول سطر دارد. این در تحلیل فاصله‌ای رایج است:

```sql
SELECT count(*)                 AS join_rows,
       count(DISTINCT s.gid)    AS distinct_schools
FROM schools s
JOIN metro_stations m ON ST_DWithin(s.geom::geography,
m.geom::geography, 700);
```

join_rows	distinct_schools
31	27

۲۷ مدرسه در فاصله‌ی ۷۰۰ متری دست‌کم یک ایستگاه‌اند (همان عدد فصل ۸)، ولی پیوند سطرهای بیشتری داد چون بعضی مدرسه‌ها نزدیک چند ایستگاه‌اند. اگر روی نتیجه‌ی این پیوند `sum(students)` بگیرید، دانش‌آموزان آن مدرسه‌ها **چند بار** شمرده می‌شوند. سه راه درست:

- برای پرسش «آیا دارد؟» از `EXISTS` استفاده کنید (فصل ۸ و ۱۳)؛
- `count(DISTINCT …)`؛
- اول یکتا کنید، بعد جمع بزنید (با زیرپرسش یا CTE).

هشدار

نقطه‌ی روی مرز مشترک. در لایه‌ی محله‌هایی که مرز مشترک دارند، نقطه‌ای که دقیقاً روی مرز است با `ST_Intersects` یا `ST_Covers` در دو محله شمرده می‌شود و مجموع شمارش محله‌ها از کل نقطه‌ها بیشتر می‌شود. اگر مجموع باید با کل برابر باشد، از `ST_Contains` استفاده کنید و نقطه‌های روی مرز را جداگانه بررسی کنید، یا هر نقطه را با `DISTINCT ON` فقط به یک محله بدهید.

۴. همپوشانی دو لایه‌ی چندضلعی

«کدام پارک‌ها با کدام محله‌ها برخورد دارند و چه مقدار؟» با `ST_Intersects` در پیوند و `ST_Intersection` برای ساختن بخش مشترک (فصل ۱۸):

```sql
SELECT n.name AS neighborhood, p.name AS park,
       round((ST_Area(ST_Intersection(n.geom, p.geom)::geography) /
10000)::numeric, 2) AS shared_ha
FROM neighborhoods n
JOIN parks p ON ST_Intersects(n.geom, p.geom)
ORDER BY shared_ha DESC;
```

neighborhood	park	shared_ha
تجریش	پارک 5	28.14
ونک	پارک 30	21.45
شهرک غرب	پارک 24	12.09
جمهوری	پارک 31	11.12
شهرک غرب	پارک 2	6.21

۱۷-۵ سرعت پرسش‌های مکانی

گزاره‌های مکانی روی هندسه‌های پیچیده سنگین‌اند. PostGIS برای سرعت، هر گزاره را در **دو مرحله** اجرا می‌کند:

1. **فیلتر اولیه با شاخص:** با عملگر `&&` (آیا مستطیل‌های محدوده‌ی دو هندسه هم‌پوشانی دارند؟) و شاخص GiST، بیشتر جفت‌های ناممکن را فوری کنار می‌گذارد. این مرحله خیلی سریع ولی تقریبی است.
2. **بررسی دقیق:** فقط برای جفت‌های باقی‌مانده، گزاره‌ی واقعی روی هندسه‌ی کامل اجرا می‌شود.

توابع `ST_Intersects`، `ST_Contains`، `ST_Within`، `ST_Covers`، `ST_DWithin` و بیشتر گزاره‌ها مرحله‌ی اول را **خودکار** دارند، به شرطی که ستون شاخص داشته باشد. فصل ۲۰ این سازوکار را با `EXPLAIN` نشان می‌دهد.

هشدار

ST_DWithin، نه ST_Distance < d. دو پرسش زیر نتیجه‌ی یکسان دارند ولی سرعتشان صدها برابر فرق می‌کند:

```sql
-- کند: فاصله‌ی همه‌ی جفت‌ها حساب می‌شود؛ شاخص استفاده نمی‌شود
SELECT … WHERE ST_Distance(a.geom::geography, b.geom::geography) < 700;
-- سریع: شاخص جفت‌های دور را از اول کنار می‌گذارد
SELECT … WHERE ST_DWithin(a.geom::geography, b.geom::geography, 700);
```

همین قاعده برای «نزدیک‌ترین» هم هست: `ORDER BY ST_Distance(…) LIMIT 1` کند است و `ORDER BY a.geom ⟷ b.geom LIMIT 1` سریع (فصل ۱۹).

سطح حرفه‌ای

ST_Intersects روی geography. گزاره‌های geography کمترند: `ST_Intersects`، `ST_Covers`، `ST_CoveredBy` و `ST_DWithin`. برای `ST_Contains` و بقیه باید با geometry کار کنید. چون گزاره‌ها (برخلاف اندازه‌گیری) به تصویر نقشه حساس نیستند (نقطه‌ای که داخل محله است، در هر SRIDی داخل است)، استفاده از geometry در ۴۳۲۶ برای گزاره‌ها کاملاً درست است. فقط ST_DWithin است که چون «فاصله» دارد به واحد حساس است.

خودتان امتحان کنید

گزارشی بنویسید که برای هر محله نشان دهد: تعداد مدرسه، تعداد ایستگاه مترو، مساحت پارک داخل محله به هکتار، و اینکه آیا دست‌کم یک بیمارستان تا یک کیلومتری مرز محله هست یا نه. (راهنمایی: CTE برای هر بخش، `EXISTS` برای بیمارستان.)

خلاصه‌ی فصل

- گزاره‌ی مکانی درست/نادرست برمی‌گرداند و در `WHERE` و `JOIN … ON` می‌نشیند.
- همه‌ی گزاره‌ها از DE-9IM (درون، مرز، بیرون؛ نُه خانه) می‌آیند؛ `ST_Relate` ماتریس و الگوی دلخواه را می‌دهد و خطاهایی مثل همپوشانی محله‌ها را آشکار می‌کند.
- نقطه‌ی روی مرز در Contains نیست ولی در Covers و Intersects هست.
- پیوند مکانی: شمارش در چندضلعی با LEFT JOIN، انتقال ویژگی، همپوشانی با ST_Intersection.
- پیوند یک‌به‌چند شمارش مضاعف می‌سازد؛ EXISTS یا DISTINCT.
- شاخص + `&&`مرحله‌ی اول هر گزاره است؛ `ST_DWithin` به‌جای `ST_Distance < d` .

برای مدرس

زمان پیشنهادی: دو جلسه. با دو برگه‌ی کاغذ شفاف رنگی روی پروژکتور، هر گزاره را فیزیکی نشان دهید. مسئله‌ی «نقطه روی مرز» را با یک نقطه‌ی ماژیکی دقیقاً روی خط مرز مطرح کنید و بپرسید «این مدرسه مال کدام محله است؟»؛ بحث جالبی درباره‌ی تفاوت داده و واقعیت درمی‌گیرد.

تمرین‌ها

۱. سطح ۱ تعداد مدرسه‌ی هر مقطع در هر محله را بدهید (دو ستون گروه‌بندی).

۲. سطح ۱ کدام ایستگاه‌های مترو داخل یک پارک‌اند؟ کدام تا ۲۰۰ متری یک پارک؟

۳. سطح ۲ تعداد مدرسه‌های داخل محله‌ها را با `ST_Contains`، `ST_Covers` و `ST_Intersects` بشمارید. آیا فرقی هست؟ چرا؟

۴. سطح ۲ مجموع دانش‌آموزان مدرسه‌هایی که تا ۷۰۰ متری مترو هستند را یک بار با JOIN ساده و یک بار درست حساب کنید.

۵. سطح ۲ بخش همپوشان محله‌های «جمهوری» و «هفت‌تیر» را با `ST_Intersection` بسازید و ببینید کدام مدرسه‌ها در آن‌اند.

۶. سطح ۲ الگوی DE-9IM برای «چندضلعی A کاملاً داخل B بدون تماس با مرز آن» چیست؟ با `ST_Relate` بیازمایید.

۷. سطح ۳ با `EXPLAIN ANALYZE` (فصل ۲۰)، زمان دو پرسش ST_Distance و ST_DWithin در کادر هشدار را برای پیوند مدرسه‌ها و مراکز درمانی مقایسه کنید.

فصل

۱۸

پردازش هندسه؛ حریم، ادغام، برش و ساده‌سازی

سطح: **متوسط** | زمان مطالعه: **۱۲۰ دقیقه** | پیش‌نیاز: **فصل ۱۷**

در این فصل یاد می‌گیرید

- حریم (بافر) را با واحد درست و گزینه‌های شکل بسازید.
- هندسه‌ها را ادغام (dissolve) و بر اساس ویژگی گروه‌بندی کنید.
- با اشتراک، تفاضل و تفاضل متقارن، همپوشانی دو لایه را بسازید.
- هندسه را ساده، گرد، به هم چسبانده و خرد کنید.
- پوسته‌ی محدب و مقعر، دیاگرام ورونوی و نقطه‌ی تصادفی بسازید.
- با ارجاع خطی روی خط‌ها نقطه بیابید و بخشی از خط را جدا کنید.

سرفصل‌ها

۱۸-۱ هندسه‌ی تازه از هندسه

۱۸-۲ حریم (بافر)

۱۸-۳ ادغام (Union / Dissolve)

۱۸-۴ عملیات همپوشانی

۱۸-۵ ساده‌سازی، گرد کردن و چسباندن

۱۸-۶ پوسته و تقسیم فضا

۱۸-۷ خرد کردن

۱۸-۸ ارجاع خطی

۱۸-۹ تبدیل‌های کمکی

۱۸-۱ هندسه‌ی تازه از هندسه

توابع این فصل هندسه می‌گیرند و **هندسه‌ی تازه** برمی‌گردانند. این‌ها همان ابزارهای «Geoprocessing» نرم‌افزارهای رومیزی‌اند، اما داخل SQL؛ پس می‌توان آن‌ها را با فیلتر، پیوند و تجمیع ترکیب کرد و نتیجه را با `CREATE TABLE AS` ذخیره کرد.

۱۸-۲ حریم (بافر)

حریم (buffer) همه‌ی نقطه‌هایی است که تا فاصله‌ی معینی از یک هندسه قرار دارند: حریم ۵۰۰ متری ایستگاه، حریم ۵۰ متری رودخانه، حریم ۱۵ متری خط لوله.

```sql
SELECT name,
       GeometryType(ST_Buffer(geom::geography, 500)::geometry)
AS type,
       round(ST_Area(ST_Buffer(geom::geography, 500)))
AS area_m2,
       round(pi() * 500 ^ 2)
AS ideal_circle_m2
FROM metro_stations
ORDER BY gid
LIMIT 3;
```

name	type	area_m2	ideal_circle_m2
ایستگاه 1	POLYGON	780957.0	785398.0
ایستگاه 2	POLYGON	780957.0	785398.0
ایستگاه 3	POLYGON	780957.0	785398.0

نکته‌های حریم:

- **واحد فاصله:** روی geography به متر؛ روی geometry به واحد SRID. `ST_Buffer(geom, 500)` روی ۴۳۲۶ یعنی ۵۰۰ **درجه**! یا به geography تبدیل کنید (مثل بالا) یا به UTM: `ST_Transform(ST_Buffer(ST_Transform(geom, 32639), 500), 4326)`.

- **دایره‌ی چندضلعی:** حریم نقطه یک چندضلعی منتظم است که دایره را تقریب می‌زند (پیش‌فرض ۸ پاره‌خط در هر ربع، یعنی ۳۲ ضلع)؛ برای همین مساحتش اندکی از دایره‌ی ایده‌آل کمتر است. با `'quad_segs=16'` دقیق‌تر می‌شود.
- **شکل انتهای خط:** برای حریم خط، `'endcap=round|flat|square'` و برای گوشه‌ها `'join=round|mitre|bevel'`.
- **حریم یک‌طرفه:** `'side=left'` یا `'side=right'`؛ مثلاً فقط سمت راست جاده.
- **حریم منفی:** روی چندضلعی، فاصله‌ی منفی آن را **کوچک** می‌کند (عقب‌نشینی ۱۰ متری از مرز قطعه برای ساخت‌وساز).

```sql
SELECT round(ST_Area(ST_Buffer(geom, 0.001))::numeric * 1e6, 2)
AS deg_area_x1e6,
       round(ST_Area(ST_Transform(ST_Buffer(ST_Transform(geom, 32639),
100), 32639))) AS utm_buffer_100m_m2,
       round(ST_Area(ST_Transform(ST_Buffer(ST_Transform(geom, 32639),
-100), 32639))) AS utm_shrink_100m_m2,
       round(ST_Area(ST_Transform(geom, 32639)))
AS original_m2
FROM parks WHERE park_id = 1;
```

deg_area_x1e6	utm_buffer_100m_m2	utm_shrink_100m_m2	original_m2
32.83	326479.0	17220.0	128925.0

ادغام حریم‌ها

حریم ایستگاه‌های نزدیک روی هم می‌افتند. برای «کل محدوده‌ای که تا ۵۰۰ متری **یک** ایستگاه است» حریم‌ها را **ادغام** می‌کنیم:

```sql
SELECT count(*)
AS buffers,
       round((sum(ST_Area(ST_Buffer(geom::geography, 500))) /
1e6)::numeric, 2)            AS sum_of_areas_km2,
       round((ST_Area(ST_Union(ST_Buffer(geom::geography,
500)::geometry)::geography) / 1e6)::numeric, 2) AS union_area_km2
FROM metro_stations;
```

buffers	sum_of_areas_km2	union_area_km2
80	62.48	62.01

جمع مساحت حریم‌ها از مساحت ادغام‌شده بیشتر است، چون بخش‌های همپوشان چند بار شمرده شده‌اند. پرسش «چند درصد شهر در پوشش مترو است؟» فقط با نسخه‌ی ادغام‌شده درست جواب داده می‌شود.

۱۸-۳ ادغام (Union / Dissolve)

`ST_Union` به‌عنوان **تابع تجمیعی** همه‌ی هندسه‌های یک گروه را یکی و مرزهای داخلی را حذف می‌کند؛ این همان «Dissolve» در نرم‌افزارهای رومیزی است. با `GROUP BY` بر اساس ویژگی ادغام می‌کنید:

```sql
SELECT line,
       GeometryType(ST_Union(ST_Buffer(geom::geography,
400)::geometry))      AS type,
       ST_NumGeometries(ST_Union(ST_Buffer(geom::geography,
400)::geometry))  AS parts
FROM metro_stations
GROUP BY line
ORDER BY line;
```

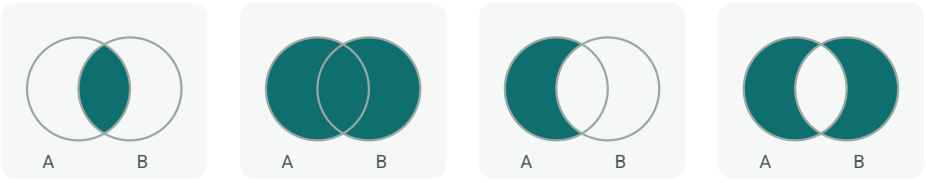
line	type	parts
خط ۱	MULTIPOLYGON	19
خط ۲	MULTIPOLYGON	27
خط ۳	MULTIPOLYGON	15
خط ۴	MULTIPOLYGON	19

اگر نتیجه چند بخش جدا داشته باشد، MultiPolygon است و `ST_NumGeometries` تعداد بخش‌ها را می‌دهد. برای ادغام **دو** هندسه `ST_Union(a, b)` هم هست.

سطح حرفه‌ای

ادغام سریع برای داده‌ی حجیم. `ST_Union` روی ده‌ها هزار چندضلعی کند است. دو ترفند: اگر هندسه‌ها هم‌پوشانی ندارند و فقط می‌خواهید آن‌ها را یک عارضه کنید، `ST_Collect` (بدون حذف مرز) هزاران برابر سریع‌تر است؛ و برای ادغام واقعی داده‌ی بزرگ، اول با `ST_Subdivide` چندضلعی‌های غول‌پیکر را خرد کنید.

۱۸-۴ عملیات همپوشانی



`ST_Intersection` اشتراک — `ST_Union` اجتماع — `ST_Difference(A,B)` تفاضل A منهای B — `ST_SymDifference` تفاضل متقارن

چهار عمل همپوشانی بین دو هندسه. نتیجه همیشه یک هندسه‌ی تازه است.

تابع	نتیجه	معادل در نرم‌افزارهای رومیزی
`ST_Intersection(a, b)`	بخش مشترک	Intersect / Clip
`ST_Union(a, b)`	همه‌ی بخش‌ها، یکپارچه	Union / Merge
`ST_Difference(a, b)`	a منهای بخشی که زیر b است	Erase / Difference
`ST_SymDifference(a, b)`	بخش‌های غیرمشترک هر دو	Symmetrical Difference

برش با مرز (Clip)

«پارک‌های داخل محله‌ی ونک، فقط بخشی که داخل محله است»:

```sql
SELECT p.name,
       round((ST_Area(p.geom::geography) / 10000)::numeric, 2)
AS park_ha,
       round((ST_Area(ST_Intersection(p.geom, n.geom)::geography) /
10000)::numeric, 2) AS inside_ha
FROM parks p JOIN neighborhoods n ON ST_Intersects(p.geom, n.geom)
WHERE n.name = 'ونک';
```

name	park_ha	inside_ha
پارک 30	21.45	21.45

حذف ناحیه (Erase)

«بخش‌هایی از محله‌ی ونک که پارک نیست» یعنی محله منهای ادغام پارک‌ها:

```sql
SELECT round((ST_Area(n.geom::geography) / 1e6)::numeric, 3)
AS neighborhood_km2,
       round((ST_Area(ST_Difference(n.geom,
ST_Union(p.geom))::geography) / 1e6)::numeric, 3)        AS
without_parks_km2
FROM neighborhoods n
JOIN parks p ON ST_Intersects(n.geom, p.geom)
WHERE n.name = 'ونک'
GROUP BY n.geom;
```

neighborhood_km2	without_parks_km2
16.055	15.840

اصلاح همپوشانی محله‌ها

در فصل ۱۷ کشف کردیم که محله‌های نمونه همپوشانی دارند. یک راه ساده‌ی اصلاح: از هر محله، بخش‌هایی را که زیر محله‌هایی با شناسه‌ی کوچک‌تر است کم کنیم؛ یعنی محله‌ی زودتر ثبت‌شده «برنده»ی بخش همپوشان است:

```sql
WITH fixed AS (
  SELECT a.gid, a.name,
         coalesce(ST_Difference(a.geom, (SELECT ST_Union(b.geom) FROM
neighborhoods b
                                         WHERE b.gid < a.gid AND
ST_Intersects(a.geom, b.geom))), a.geom) AS geom
  FROM neighborhoods a
)
SELECT count(*) AS overlapping_pairs_after_fix
FROM fixed a JOIN fixed b ON a.gid < b.gid AND ST_Overlaps(a.geom,
b.geom);
```

overlapping_pairs_after_fix
0

صفر جفت همپوشان. قاعده‌ی «برنده» به تصمیم کارشناسی وابسته است؛ SQL فقط اجرایش را ممکن می‌کند. `coalesce` برای محله‌ای لازم بود که هیچ همپوشانی با قبلی‌ها نداشت و زیرپرسش برایش NULL برمی‌گرداند.

> **هشدار**
>
> **بعد نتیجه را بررسی کنید.** `ST_Intersection` دو چندضلعی ممکن است فقط یک **خط** یا **نقطه** برگرداند (اگر فقط در مرز تماس داشته باشند) یا یک GeometryCollection مختلط. اگر نتیجه را در ستونی از نوع `MultiPolygon` می‌ریزید، با `ST_CollectionExtract(geom, 3)` فقط بخش‌های چندضلعی را نگه دارید (۱ نقطه، ۲ خط، ۳ چندضلعی) و سطرهای خالی را با `WHERE NOT ST_IsEmpty(…)` حذف کنید.

۱۸-۵ ساده‌سازی، گرد کردن و چسباندن

داده‌ای که از نقشه‌ی بزرگ‌مقیاس آمده برای نمایش در مقیاس کوچک یا انتشار وب بیش از حد پرجزئیات است. **ساده‌سازی** رأس‌های اضافه را حذف می‌کند:

تابع	روش	نکته
`ST_Simplify(g, tol)`	داگلاس-پوکر	سریع؛ ممکن است هندسه‌ی نامعتبر بسازد
`ST_SimplifyPreserveTopology(g, tol)`	داگلاس-پوکر با حفظ اعتبار	امن‌تر برای چندضلعی
`ST_SimplifyVW(g, area)`	ویسوالینگام-وایات	شکل طبیعی‌تر برای کارتوگرافی
`ST_ChaikinSmoothing(g)`	هموارسازی	گوشه‌های تیز را گرد می‌کند
`ST_SnapToGrid(g, size)`	گرد کردن مختصات به شبکه	کاهش دقت ظاهری و حجم
`ST_ReducePrecision(g, size)`	گرد کردن با حفظ اعتبار	(PostGIS ۳٫۱+)
`ST_Snap(a, b, tol)`	چسباندن رأس‌های a به b	بستن شکاف‌های باریک بین لایه‌ها

```sql
WITH c AS (SELECT ST_Buffer('SRID=4326;POINT(51.4 35.7)'::geography, 1000, 'quad_segs=32')::geometry AS g)
SELECT ST_NPoints(g)                                        AS original_vertices,
       ST_NPoints(ST_SimplifyPreserveTopology(g, 0.0005))   AS simplified_vertices,
       round((100 * (1 - ST_Area(ST_SimplifyPreserveTopology(g, 0.0005)::geography) / ST_Area(g::geography)))::numeric, 2) AS area_loss_pct
FROM c;
```

original_vertices	simplified_vertices	area_loss_pct
129	17	2.51

تلورانس (`0.0005`) به واحد SRID است؛ در ۴۳۲۶ حدود ۵۰ متر. رأس‌ها از ۱۲۹ به ده‌ها کاهش یافت و مساحت چند درصد کم شد؛ بسته به مقیاس نمایش، این تبادل حجم و دقت را باید آگاهانه انتخاب کنید. کاشی‌های برداری سامانه (فصل ۴۵) در هر سطح زوم خودکار ساده‌سازی می‌کنند.

۱۸-۶ پوسته و تقسیم فضا

```sql
SELECT round((ST_Area(ST_ConvexHull(ST_Collect(geom))::geography) / 1e6)::numeric, 1)       AS convex_hull_km2,
       round((ST_Area(ST_ConcaveHull(ST_Collect(geom), 0.3)::geography) / 1e6)::numeric, 1) AS concave_hull_km2,
       round((ST_Area(ST_Envelope(ST_Collect(geom))::geography) / 1e6)::numeric, 1)       AS envelope_km2
FROM schools;
```

convex_hull_km2	concave_hull_km2	envelope_km2
638.9	576.4	871.6

- **پوسته‌ی محدب** (`ST_ConvexHull`): کوچک‌ترین چندضلعی محدبی که همه را بپوشاند؛ مثل کش دور میخ‌ها.
- **پوسته‌ی مقعر** (`ST_ConcaveHull(g, ratio)`): پوسته‌ی تنگ‌تر که فرورفتگی دارد؛ ratio بین ۰ (تنگ‌ترین) و ۱ (محدب). برای «محدوده‌ی واقعی پراکنش» نقطه‌ها.
- **مستطیل محدوده** (`ST_Envelope`): ساده‌ترین تقریب.

دیاگرام ورونوی (`ST_VoronoiPolygons`) صفحه را طوری تقسیم می‌کند که هر بخش به نزدیک‌ترین نقطه‌اش تعلق داشته باشد؛ همان «حوزه‌ی نفوذ» یا «چندضلعی تیسن» که در اقلیم‌شناسی (بارش ایستگاه‌ها) و برنامه‌ریزی خدمات (حوزه‌ی هر مرکز) کاربرد دارد. ابزار «حوزه‌ی نفوذ (تیسن)» سامانه همین را می‌سازد (فصل ۲۶):

```sql
WITH v AS (
  SELECT (ST_Dump(ST_VoronoiPolygons(ST_Collect(geom)))).geom AS cell
FROM health_centers WHERE type = 'بیمارستان'
)
SELECT h.name, round((ST_Area(v.cell::geography) / 1e6)::numeric, 2) AS service_area_km2
FROM v JOIN health_centers h ON h.type = 'بیمارستان' AND
ST_Contains(v.cell, h.geom)
ORDER BY service_area_km2 DESC
LIMIT 4;
```

name	service_area_km2
مرکز 1	559.65
مرکز 40	557.84
مرکز 7	360.49
مرکز 34	358.60

مساحت سلول‌های لبه بسیار بزرگ است، چون ورونوی تا محدوده‌ای وسیع‌تر از خود نقطه‌ها امتداد می‌یابد. در عمل سلول‌ها را با مرز شهر یا محدوده‌ی مطالعه برش می‌دهید (`ST_Intersection`؛ تمرین ۴). این نکته در تفسیر «حوزه‌ی خدمات» مهم است.

`ST_DelaunayTriangles` مثلث‌بندی دلونی (پایه‌ی سطح TIN در فصل ۴۹) و `ST_GeneratePoints(polygon, n)` نقطه‌های تصادفی داخل چندضلعی (برای نمونه‌برداری میدانی) می‌سازد.

۱۸-۷ خرد کردن

`ST_Subdivide(g, max_vertices)` یک چندضلعی پیچیده را به قطعه‌های کوچک با حداکثر رأس معین می‌شکند. این عجیب به نظر می‌رسد، اما **یکی از مؤثرترین بهینه‌سازی‌های کارایی** است: پرسش «کدام نقطه داخل مرز ایران است؟» روی یک چندضلعی با ۵۰ هزار رأس بسیار کندتر از همان پرسش روی ۵۰۰ قطعه‌ی ۱۰۰ رأسی با شاخص مکانی است (فصل ۲۰).

`ST_Split(g, blade)` هندسه را با یک خط (یا نقطه) می‌بُرد؛ مثلاً تقسیم قطعه‌ی زمین با خط تفکیک.

۱۸-۸ ارجاع خطی

ارجاع خطی (linear referencing) مکان را با «فاصله در امتداد یک خط» بیان می‌کند: «کیلومتر ۳۲ جاده‌ی تهران-قم»، «۲۰۰ متر بعد از شیر شماره‌ی ۵ خط لوله». توابع آن با کسری از طول خط (۰ تا ۱) کار می‌کنند:

تابع	کار
`ST_LineInterpolatePoint(line, f)`	نقطه در کسر f طول خط
`ST_LineLocatePoint(line, pt)`	کسر طول نزدیک‌ترین نقطه‌ی خط به pt
`ST_LineSubstring(line, f1, f2)`	بخشی از خط بین دو کسر
`ST_ClosestPoint(line, pt)`	نزدیک‌ترین نقطه روی خط

```sql
WITH route AS (SELECT ST_MakeLine(geom ORDER BY station_id) AS g
FROM metro_stations WHERE line = 'خط ۱')
SELECT round((ST_Length(g::geography) / 1000)::numeric, 2)
AS line_km,
       ST_AsText(ST_LineInterpolatePoint(g, 0.5))
AS midpoint,
       round(ST_LineLocatePoint(g, (SELECT geom FROM schools WHERE
school_id = 1))::numeric, 3) AS school1_fraction,
       round((ST_Length(ST_LineSubstring(g, 0.25, 0.75)::geography) /
1000)::numeric, 2) AS middle_half_km
FROM route;
```

line_km	midpoint	school1_fraction	middle_half_km
24.53	POINT(51.42821848640793 35.68931091620407)	0.574	12.26

در دنیای واقعی

نقطه‌ی حادثه روی جاده. پلیس راه حادثه‌ها را با «کیلومتر جاده» ثبت می‌کند، نه با مختصات. با جدول جاده‌ها و ستون کیلومتر، مختصات هر حادثه با `ST_LineInterpolatePoint(road.geom, km * 1000 / ST_Length(road.geom::geography))` ساخته می‌شود و می‌توان نقاط حادثه‌خیز را روی نقشه دید. برعکس، برای ثبت یک برداشت GPS به کیلومتر جاده، `ST_LineLocatePoint` به کار می‌رود.

۱۸-۹ تبدیل‌های کمکی

`ST_Multi` (تبدیل به چندبخشی)، `ST_Force2D` (حذف Z)، `ST_Boundary` (مرز چندضلعی به صورت خط)، `ST_Polygonize` (چندضلعی از خط‌های بسته)، `ST_Translate`، `ST_Rotate` و `ST_Scale` (جابه‌جایی، چرخش و مقیاس؛ برای تصحیح جابه‌جایی ثابت یک لایه) و `ST_FlipCoordinates` (جابه‌جایی x و y؛ درمان قطعی خطای «طول و عرض جابه‌جا»).

خودتان امتحان کنید

محدوده‌ای را بسازید که «تا ۶۰۰ متری یک ایستگاه مترو» باشد ولی «تا ۲۰۰ متری هیچ بیمارستانی» نباشد. مساحتش را به کیلومتر مربع بدهید و نتیجه را با `CREATE TABLE AS` ذخیره و روی نقشه ببینید. (راهنمایی: ادغام حریم‌ها، سپس `ST_Difference` .)

خلاصه‌ی فصل

- `ST_Buffer` روی geography به متر؛ روی ۴۳۲۶ درجه! گزینه‌ها: quad_segs، endcap، join، side؛ فاصله‌ی منفی چندضلعی را کوچک می‌کند.
- `ST_Union` تجمیعی همان Dissolve است؛ جمع مساحت حریم‌های همپوشان با مساحت ادغام‌شده فرق دارد.
- همپوشانی: `ST_Intersection` ، `ST_Difference` ، `ST_SymDifference` ؛ نوع نتیجه را با `ST_CollectionExtract` کنترل کنید.
- ساده‌سازی (`ST_SimplifyPreserveTopology` ، `ST_SimplifyVW`)، گرد کردن (`ST_SnapToGrid` ، `ST_ReducePrecision`)، چسباندن (`ST_Snap`).
- پوسته‌ی محدب و مقعر، ورونوی (حوزه‌ی نفوذ)، دلونی، نقطه‌ی تصادفی.
- `ST_Subdivide` برای کارایی؛ ارجاع خطی با `ST_LineInterpolatePoint` و `ST_LineLocatePoint` .

برای مدرس

زمان پیشنهادی: دو جلسه. هر تابع را با یک «پرسش برنامه‌ریز» معرفی کنید و خروجی را در QGIS (DB Manager ← Load as new layer) نشان دهید؛ دیدن حریم‌ها و ورونوی روی نقشه بسیار گویاتر از عدد است. تمرین «خودتان امتحان کنید» یک مسئله‌ی کامل مکان‌یابی کوچک است.

تمرین‌ها

۱. سطح ۱ حریم ۳۰۰ متری همه‌ی مدرسه‌های ابتدایی را بسازید و ادغام کنید. مساحت کل چقدر است؟

۲. سطح ۱ مساحت هر پارک را که داخل هر محله است (اشتراک) بدهید.

۳. سطح ۲ پوسته‌ی مقعر ایستگاه‌های هر خط مترو را بسازید و مساحتش را مقایسه کنید.

۴. سطح ۲ دیاگرام ورونوی همه‌ی مراکز درمانی را بسازید و به محدوده‌ی پوسته‌ی محدب مدرسه‌ها برش دهید (`ST_Intersection`).

۵. سطح ۲ نقطه‌ی هر ۲ کیلومتر روی خط ۲ مترو را با `ST_LineInterpolatePoint` و `generate_series` بسازید.

۶. سطح ۳ یک پارک را با `ST_SimplifyPreserveTopology` در سه تلورانس ساده کنید و جدولی از تعداد رأس و درصد تغییر مساحت بسازید. کدام تلورانس برای نقشه‌ی کل شهر مناسب است؟

فصل

۱۹

نزدیک‌ترین همسایه (KNN)

سطح: **متوسط** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل‌های ۱۳ و ۱۷**

در این فصل یاد می‌گیرید

- با عملگر <-> و شاخص مکانی، نزدیک‌ترین عارضه‌ها را سریع پیدا کنید.
- برای هر سطر یک جدول، k نزدیک‌ترین عارضه‌ی جدول دیگر را با LATERAL بیابید.
- نزدیک‌ترین را با فیلتر ویژگی و سقف فاصله ترکیب کنید.
- تفاوت فاصله‌ی شاخص و فاصله‌ی دقیق را بدانید.
- شاخص‌های دسترسی ساده (فاصله تا نزدیک‌ترین خدمت) بسازید.

سرفصل‌ها

۱۹-۱ پرسش «نزدیک‌ترین»

۱۹-۲ عملگر <->

۱۹-۳ برای هر سطر، نزدیک‌ترین: LATERAL

۱۹-۴ نزدیک‌ترین با شرط

۱۹-۵ الگوی دو مرحله‌ای برای دقت

۱۹-۶ نزدیک‌ترین روی خط

۱۹-۱ پرسش «نزدیک‌ترین»

«نزدیک‌ترین بیمارستان به محل حادثه کجاست؟»، «برای هر مدرسه، نزدیک‌ترین ایستگاه مترو و فاصله‌اش؟»، «پنج شعبه‌ی نزدیک به این مشتری؟». این‌ها پرسش‌های **نزدیک‌ترین همسایه** (k-Nearest Neighbors یا **KNN**) هستند و در هر سامانه‌ی خدمات مکانی هزاران بار در روز اجرا می‌شوند.

راه ساده‌لوحانه این است که فاصله‌ی همه‌ی جفت‌ها را حساب و مرتب کنیم:

```sql
-- درست ولی کند: فاصله تا همه‌ی مراکز حساب و بعد مرتب می‌شود
SELECT name FROM health_centers
ORDER BY ST_Distance(geom, 'SRID=4326;POINT(51.40 35.72)'::geometry)
LIMIT 1;
```

برای ۴۰ مرکز مهم نیست؛ برای ۴۰ میلیون نقطه یعنی چند دقیقه. PostGIS عملگر ویژه‌ای دارد که این کار را با **شاخص** انجام می‌دهد.

۱۹-۲ عملگر <->

عملگر `<->` فاصله‌ی دو هندسه را برمی‌گرداند، اما وقتی در `ORDER BY` با `LIMIT` بیاید و یک طرفش ستون شاخص‌دار باشد، PostGIS به جای محاسبه‌ی همه‌ی فاصله‌ها، درخت شاخص را **از نزدیک به دور** پیمایش می‌کند و به محض یافتن k مورد متوقف می‌شود:

```sql
SELECT name, type,
       round(ST_Distance(geom::geography, 'SRID=4326;POINT(51.40
35.72)'::geography)) AS meters
FROM health_centers
ORDER BY geom <-> 'SRID=4326;POINT(51.40 35.72)'::geometry
LIMIT 3;
```

name	type	meters
مرکز 11	درمانگاه	1684.0
مرکز 12	کلینیک	1684.0
مرکز 19	بیمارستان	2555.0

الگوی کلی را به خاطر بسپارید: **مرتب‌سازی با `<->`، نمایش فاصله با `ST_Distance` روی `geography`.** `<->` روی geometry با SRID ۴۳۲۶ فاصله را به درجه می‌دهد؛ برای **ترتیب** کافی است (در فاصله‌های کوتاه ترتیب تقریباً درست است)، ولی برای **نمایش** باید فاصله‌ی متری را جداگانه حساب کرد.

بدانید

درخت R و پیمایش «بهترین-اول». شاخص GiST فضا را به مستطیل‌های تودرتو تقسیم می‌کند (فصل ۲۰). برای یافتن نزدیک‌ترین، الگوریتم از ریشه شروع می‌کند و همیشه مستطیلی را باز می‌کند که کمترین فاصله را تا نقطه‌ی هدف دارد. مستطیل‌های دور هرگز باز نمی‌شوند. برای همین زمان یافتن نزدیک‌ترین تقریباً مستقل از حجم جدول است.

سطح حرفه‌ای

دقت ترتیب در geometry درجه‌ای. در عرض تهران، یک درجه‌ی طول حدود ۹۰ و یک درجه‌ی عرض حدود ۱۱۱ کیلومتر است؛ پس `<->` روی ۴۳۲۶ فاصله‌ی شمالی-جنوبی را کمتر از واقعیت وزن می‌دهد. وقتی دو نامزد فاصله‌ی نزدیک به هم در جهت‌های مختلف دارند، ترتیب ممکن است جابه‌جا شود. راه مطمئن: چند نامزد بیشتر با `<->` بگیرید و آن‌ها را با فاصله‌ی دقیق دوباره مرتب کنید (الگوی «دو مرحله‌ای» پایین)، یا مستقیم روی geography مرتب کنید: `ORDER BY geom::geography <-> point::geography` که با شاخص روی `(geom::geography)` سریع است.

۱۹-۳ برای هر سطر، نزدیک‌ترین: LATERAL

پرسش رایج‌تر این است: «برای **هر** مدرسه، نزدیک‌ترین ایستگاه مترو». در فصل ۱۳ دیدیم که `JOIN LATERAL` زیرپرسش را برای هر سطر اجرا می‌کند:

```sql
SELECT s.name AS school,
       m.name AS nearest_station, m.line,
       round(ST_Distance(s.geom::geography, m.geom::geography)) AS
meters
FROM schools s
CROSS JOIN LATERAL (
  SELECT name, line, geom
  FROM metro_stations
  ORDER BY metro_stations.geom <-> s.geom
  LIMIT 1
) m
ORDER BY meters DESC
LIMIT 5;
```

school	nearest_station	line	meters
مدرسه 162	ایستگاه 45	خط ۲	7499.0
مدرسه 34	ایستگاه 44	خط ۲	7338.0
مدرسه 8	ایستگاه 1	خط ۱	6696.0
مدرسه 71	ایستگاه 66	خط ۴	6668.0
مدرسه 165	ایستگاه 41	خط ۲	6587.0

دورترین مدرسه‌ها از مترو بالای فهرست‌اند. برای **k نزدیک‌ترین**، فقط `LIMIT 1` را به `LIMIT k` تغییر دهید. اگر جدول سمت راست ممکن است برای سطری نتیجه نداشته باشد (مثلاً با فیلتر)، `LEFT JOIN LATERAL (…) ON true` بنویسید تا آن سطر حذف نشود.

از نتیجه تا شاخص دسترسی

با همین الگو می‌توان یک **شاخص دسترسی** ساخت: میانگین فاصله تا نزدیک‌ترین خدمت، و درصد جمعیت (اینجا دانش‌آموز) در فاصله‌ی قابل پیاده‌روی:

```sql
WITH nearest AS (
  SELECT s.level, s.students,
         ST_Distance(s.geom::geography, h.geom::geography) AS meters
  FROM schools s
  CROSS JOIN LATERAL (SELECT geom FROM health_centers ORDER BY geom <->
s.geom LIMIT 1) h
)
SELECT level,
       round(avg(meters))
AS avg_m_to_clinic,
       round(percentile_cont(0.9) WITHIN GROUP (ORDER BY meters))
AS p90_m,
       round(100.0 * sum(students) FILTER (WHERE meters <= 1000) /
sum(students), 1) AS students_within_1km_pct
FROM nearest
GROUP BY level
ORDER BY level;
```

level	avg_m_to_clinic	p90_m	students_within_1km_pct
ابتدایی	3286.0	7808.0	21.6
متوسطه اول	3024.0	7870.0	22.0
متوسطه دوم	4840.0	9343.0	11.2

این پرسش در یک خط می‌گوید «چند درصد دانش‌آموزان هر مقطع تا یک کیلومتری یک مرکز درمانی درس می‌خوانند»؛ از آن نوع عددهایی که در گزارش‌های برنامه‌ریزی خواسته می‌شود.

۱۹-۴ نزدیک‌ترین با شرط

فیلتر ویژگی: «نزدیک‌ترین بیمارستان»، نه هر مرکزی:

```sql
SELECT s.name, h.name AS nearest_hospital, h.meters
FROM schools s
CROSS JOIN LATERAL (
  SELECT name, round(ST_Distance(s.geom::geography, geom::geography))
AS meters
  FROM health_centers
  WHERE type = 'بیمارستان'
  ORDER BY geom <-> s.geom
  LIMIT 1
) h
ORDER BY s.gid
LIMIT 3;
```

name	nearest_hospital	meters
مدرسه 1	مرکز 7	4153.0
مدرسه 2	مرکز 13	1581.0
مدرسه 3	مرکز 7	1058.0

سقف فاصله: «نزدیک‌ترین ایستگاه، اگر تا ۵۰۰ متر باشد؛ وگرنه هیچ». شرط فاصله را با `ST_DWithin` در `WHERE` زیرپرسش بگذارید و از `LEFT JOIN LATERAL` استفاده کنید تا مدرسه‌های بی‌ایستگاه با NULL بیایند:

```sql
SELECT count(*) FILTER (WHERE m.name IS NOT NULL) AS
with_station_500m,
       count(*) FILTER (WHERE m.name IS NULL)     AS
without_station_500m
FROM schools s
LEFT JOIN LATERAL (
  SELECT name FROM metro_stations
  WHERE ST_DWithin(geom::geography, s.geom::geography, 500)
  ORDER BY geom <-> s.geom
  LIMIT 1
) m ON true;
```

with_station_500m	without_station_500m
14	166

عارضه‌ی هم‌جدول: نزدیک‌ترین مدرسه‌ی **دیگر** به هر مدرسه (پیوند با خود؛ با شرط `gid <>` تا خودش انتخاب نشود). این پرسش پایه‌ی **تحلیل نزدیک‌ترین همسایه** (Nearest Neighbor Index) در آمار فضایی است که نشان می‌دهد نقطه‌ها خوشه‌ای، تصادفی یا منظم پخش شده‌اند (فصل ۲۱).

۱۹-۵ الگوی دو مرحله‌ای برای دقت

وقتی دقت ترتیب مهم است (مثلاً هندسه‌های غیرنقطه‌ای بزرگ یا عرض جغرافیایی بالا)، با شاخص چند نامزد بگیرید و با فاصله‌ی دقیق دوباره مرتب کنید:

```sql
SELECT name, round(d) AS meters
FROM (
  SELECT name, ST_Distance(geom::geography, 'SRID=4326;POINT(51.40 35.72)'::geography) AS d
  FROM parks
  ORDER BY geom <-> 'SRID=4326;POINT(51.40 35.72)'::geometry
  LIMIT 10                  -- مرحله‌ی ۱: ده نامزد با شاخص
) candidates
ORDER BY d                  -- مرحله‌ی ۲: ترتیب دقیق
LIMIT 3;
```

name	meters
پارک 30	2646.0
پارک 31	2799.0
پارک 28	4117.0

برای چندضلعی‌ها، `<->` فاصله‌ی واقعی هندسه را برمی‌گرداند (نه فقط مستطیل محدوده)، ولی پیمایش شاخص بر پایه‌ی مستطیل‌هاست؛ الگوی دو مرحله‌ای هم سرعت را حفظ می‌کند و هم ترتیب را تضمین. عملگر `<#>` فاصله‌ی **مستطیل‌های محدوده** را می‌دهد که از همه سریع‌تر ولی تقریبی‌تر است.

۱۹-۶ نزدیک‌ترین روی خط

گاهی «نزدیک‌ترین نقطه روی یک خط» لازم است، نه نزدیک‌ترین عارضه؛ مثلاً اتصال هر ساختمان به نزدیک‌ترین نقطه‌ی خیابان برای نشانی‌دهی یا اتصال به شبکه‌ی مسیریابی. `ST_ClosestPoint(line, point)` و `ST_ShortestLine(line, point)` همین را می‌دهند (فصل ۱۶ و ۱۸) و در فصل ۲۴ برای «چسباندن» مبدأ و مقصد به شبکه‌ی معابر از آن استفاده می‌کنیم.

> **هشدار**
>
> **فاصله‌ی هوایی، نه واقعی.** همه‌ی فاصله‌های این فصل **هوایی** (خط مستقیم) هستند. نزدیک‌ترین بیمارستان هوایی ممکن است آن سوی بزرگراه یا رود باشد و با خودرو دورتر از دومی. برای دسترسی واقعی، فاصله را روی **شبکه‌ی معابر** حساب کنید (فصل‌های ۲۴ و ۲۸). فاصله‌ی هوایی برای غربال اولیه و تحلیل‌های کلان مناسب است.

> **خودتان امتحان کنید**
>
> برای هر ایستگاه مترو، سه مدرسه‌ی نزدیک و فاصله‌شان را بدهید. سپس ایستگاه‌هایی را پیدا کنید که نزدیک‌ترین مدرسه‌شان بیش از ۸۰۰ متر فاصله دارد؛ این ایستگاه‌ها برای ساخت مدرسه‌ی تازه کاندیدند.

خلاصه‌ی فصل

- `ORDER BY geom <-> X LIMIT k` با شاخص GiST نزدیک‌ترین‌ها را بدون محاسبه‌ی همه‌ی فاصله‌ها می‌یابد.
- ترتیب با `<->` ، نمایش فاصله با `ST_Distance` روی geography.
- «برای هر سطر، نزدیک‌ترین» = `CROSS JOIN LATERAL (… ORDER BY <-> LIMIT k)`؛ با `LEFT JOIN LATERAL … ON true` سطرهای بی‌نتیجه می‌مانند.
- فیلتر ویژگی و سقف فاصله (`ST_DWithin`) داخل زیرپرسش LATERAL.
- الگوی دو مرحله‌ای: نامزدها با شاخص، ترتیب نهایی با فاصله‌ی دقیق.
- فاصله‌ی KNN هوایی است؛ دسترسی واقعی روی شبکه.

برای مدرس

زمان پیشنهادی: یک جلسه. با `EXPLAIN ANALYZE` (فصل ۲۰) زمان پرسش ساده‌لوحانه و `<->` را روی جدولی بزرگ (مثلاً ساختمان‌های OSM از فصل ۳۰) مقایسه کنید؛ اختلاف چند صد برابری بهترین انگیزه برای یادگیری شاخص است.

تمرین‌ها

۱. سطح ۱ پنج مدرسه‌ی نزدیک به نقطه‌ی `(51.42, 35.70)` را با فاصله به متر بدهید.

۲. سطح ۱ برای هر پارک نزدیک‌ترین ایستگاه مترو و فاصله‌اش را بدهید.

۳. سطح ۲ برای هر مدرسه‌ی ابتدایی، نزدیک‌ترین مدرسه‌ی ابتدایی **دیگر** و فاصله‌اش را بدهید. میانگین این فاصله چقدر است؟

۴. سطح ۲ شاخص دسترسی این فصل را برای فاصله تا نزدیک‌ترین ایستگاه مترو بسازید.

۵. سطح ۳ با یک CTE، برای هر مرکز درمانی تعداد مدرسه‌هایی را بشمارید که این مرکز «نزدیک‌ترین مرکز» آن‌هاست (بار خدمت هر مرکز). کدام مرکز بیشترین بار را دارد؟

فصل

۲۰

شاخص مکانی و کارایی پرسش‌ها

سطح: **پیشرفته** ●●●

زمان مطالعه: **۹۰ دقیقه**

پیش‌نیاز: **فصل‌های ۱۷ و ۱۹**

در این فصل یاد می‌گیرید

- سازوکار شاخص GiST و درخت R را توضیح دهید.
- برنامه‌ی اجرای پرسش را با EXPLAIN و EXPLAIN ANALYZE بخوانید.
- پرسش‌هایی بنویسید که از شاخص استفاده می‌کنند و خطاهایی که شاخص را بی‌اثر می‌کنند بشناسید.
- شاخص روی عبارت، شاخص جزئی و BRIN را در جای خود به کار ببرید.
- با VACUUM، ANALYZE، ST_Subdivide و CLUSTER کارایی را بهبود دهید.

سرفصل‌ها

۲۰-۱ چرا شاخص

۲۰-۲ درخت R

۲۰-۳ فیلتر دو مرحله‌ای

۲۰-۴ آزمایش: با شاخص و بی‌شاخص

۲۰-۵ خطاهایی که شاخص را بی‌اثر می‌کنند

۲۰-۶ انواع دیگر شاخص

۲۰-۷ هندسه‌های بزرگ: ST_Subdivide

۲۰-۸ نگه‌داری

۲۰-۹ فهرست کنترل کارایی

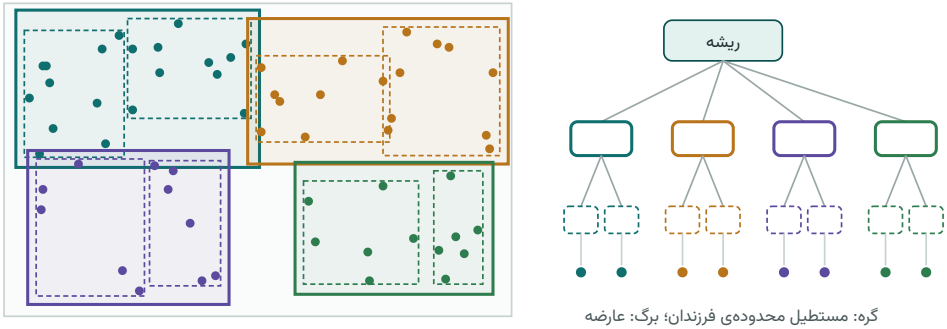
۲۰-۱ چرا شاخص

پرسش «کدام نقطه‌ها داخل این چندضلعی‌اند؟» بدون شاخص یعنی بررسی **تک‌تک** نقطه‌ها. برای ۱۸۰ مدرسه فوری است؛ برای ۲۰ میلیون نقطه‌ی ردیابی خودرو، دقیقه‌ها طول می‌کشد. **شاخص** (index) ساختاری کمکی است که مثل فهرست الفبایی پایان یک کتاب، اجازه می‌دهد بدون خواندن همه‌ی صفحه‌ها به بخش مورد نظر برسیم.

شاخص معمولی پایگاه داده (B-tree) برای مقدارهای **مرتب‌شدنی** مثل عدد و متن ساخته شده است. اما هندسه ترتیب طبیعی ندارد: کدام نقطه «بزرگ‌تر» است؟ پس PostGIS از نوع دیگری استفاده می‌کند: **GiST** (درخت جست‌وجوی تعمیم‌یافته) که روی آن **درخت R** پیاده شده است.

۲۰-۲ درخت R

ایده‌ی درخت R ساده است: هر هندسه با **مستطیل محدوده**‌اش (bounding box) جایگزین می‌شود. مستطیل‌های نزدیک به هم در یک گروه قرار می‌گیرند و هر گروه مستطیل محدوده‌ی خودش را دارد؛ گروه‌ها دوباره گروه می‌شوند تا به یک ریشه برسیم.



درخت R. چهار گروه (رنگ‌ها) هر کدام دو زیرگروه (خط‌چین) دارند. برای پرسشی در گوشه‌ی بالا-راست، فقط گره‌ی سبز و فرزندانش بررسی می‌شوند.

برای یافتن عارضه‌هایی که با یک محدوده برخورد دارند، الگوریتم از ریشه شروع می‌کند و فقط وارد گره‌هایی می‌شود که مستطیلشان با محدوده‌ی پرسش برخورد دارد. بیشتر درخت هرگز

دیده نمی‌شود. به همین دلیل پرسش مکانی روی جدول بزرگ با شاخص، به جای خطی (n)، تقریباً لگاریتمی (log n) رشد می‌کند.

ساختن شاخص یک خط است و همیشه، برای هر ستون هندسه‌ای که در پرسش‌ها به کار می‌رود، لازم است:

```sql
CREATE INDEX schools_geom_idx ON schools USING gist (geom);
```

۲۰-۳ فیلتر دو مرحله‌ای

چون شاخص فقط مستطیل‌ها را می‌شناسد، پاسخش **تقریبی** است: «این‌ها **شاید** برخورد دارند». پس هر گزاره‌ی مکانی دو مرحله دارد (فصل ۱۷):

1. **فیلتر اولیه** با عملگر `&&` (برخورد مستطیل‌ها) روی شاخص؛ سریع، تقریبی، بیشتر نامزدها را حذف می‌کند.
2. **بررسی دقیق** با هندسه‌ی کامل، فقط روی نامزدهای باقی‌مانده.

توابعی مثل `ST_Intersects`، `ST_Contains`، `ST_Within`، `ST_Covers` و `ST_DWithin` مرحله‌ی اول را در خودشان دارند. یعنی پرسش `WHERE ST_Intersects(a.geom, b.geom)` به طور خودکار به `WHERE a.geom && b.geom AND _ST_Intersects(a.geom, b.geom)` تبدیل می‌شود.

۲۰-۴ آزمایش: با شاخص و بی‌شاخص

برای دیدن اثر شاخص به جدول بزرگ‌تری از داده‌ی نمونه نیاز داریم. یک جدول ۲۰۰ هزار نقطه‌ای تصادفی در محدوده‌ی تهران می‌سازیم و زمان یک پرسش «نقطه در محله» را پیش و پس از شاخص می‌سنجیم. `EXPLAIN ANALYZE` پرسش را اجرا و **برنامه‌ی اجرا** (query plan) و زمان واقعی هر مرحله را گزارش می‌کند:

```sql
CREATE TABLE pts AS
SELECT g AS id, ST_SetSRID(ST_MakePoint(51.2 + random() * 0.4, 35.55 +
random() * 0.3), 4326) AS geom
FROM generate_series(1, 200000) g;
ANALYZE pts;

EXPLAIN ANALYZE
SELECT count(*) FROM pts p JOIN neighborhoods n ON ST_Contains(n.geom,
p.geom) WHERE n.name = 'ونک';
```

QUERY PLAN
Finalize Aggregate (cost=1602139.37..1602139.38 rows=1 width=8) (actual time=211.517..2…
→ Gather (cost=1602139.26..1602139.37 rows=1 width=8) (actual time=211.292..216.756…
Workers Planned: 1
Workers Launched: 1
→ Partial Aggregate (cost=1601139.26..1601139.27 rows=1 width=8) (actual time…
→ Nested Loop (cost=0.00..1601138.97 rows=118 width=0) (actual time=93.…
Join Filter: st_contains(n.geom, p.geom)
Rows Removed by Join Filter: 98601
→ Parallel Seq Scan on pts p (cost=0.00..2904.47 rows=117647 widt…
→ Seq Scan on neighborhoods n (cost=0.00..1.07 rows=1 width=32) (…
Filter: (name = 'ونک'::text)
Rows Removed by Filter: 5
Planning Time: 0.486 ms
JIT:
Functions: 18
Options: Inlining true, Optimization true, Expressions true, Deforming true
Timing: Generation 0.546 ms, Inlining 89.280 ms, Optimization 55.731 ms, Emission 40.9…
Execution Time: 232.508 ms

برنامه را از پایین به بالا بخوانید. **Parallel Seq Scan on pts** یعنی همه‌ی ۲۰۰ هزار سطر (با دو پردازه‌ی موازی) یکی‌یکی خوانده و با `ST_Contains` بررسی شده‌اند؛ «Rows Removed by Join Filter» نشان می‌دهد تقریباً همه‌ی آن‌ها بیهوده بررسی شده‌اند. حالا شاخص می‌سازیم و همان پرسش را تکرار می‌کنیم:

```sql
CREATE TABLE pts AS
SELECT g AS id, ST_SetSRID(ST_MakePoint(51.2 + random() * 0.4, 35.55 +
random() * 0.3), 4326) AS geom
FROM generate_series(1, 200000) g;
CREATE INDEX ON pts USING gist (geom);
ANALYZE pts;

EXPLAIN ANALYZE
SELECT count(*) FROM pts p JOIN neighborhoods n ON ST_Contains(n.geom,
p.geom) WHERE n.name = 'ونک';
```

QUERY PLAN
Aggregate (cost=329.96..329.97 rows=1 width=8) (actual time=2.892..2.893 rows=1 loops=1)
→ Nested Loop (cost=4.44..329.46 rows=200 width=0) (actual time=0.508..2.790 rows=2…
→ Seq Scan on neighborhoods n (cost=0.00..1.07 rows=1 width=32) (actual time=…
Filter: (name = 'ونک'::text)
Rows Removed by Filter: 5
→ Bitmap Heap Scan on pts p (cost=4.44..328.18 rows=20 width=32) (actual time…
Filter: st_contains(n.geom, geom)
Heap Blocks: exact=1334
→ Bitmap Index Scan on pts_geom_idx (cost=0.00..4.43 rows=20 width=0) (…
Index Cond: (geom @ n.geom)
Planning Time: 0.444 ms
Execution Time: 2.965 ms

این بار **Bitmap Index Scan** روی شاخص `pts_geom_idx` دیده می‌شود و شرط مستطیلی در «Index Cond» آمده است (برای `ST_Contains` عملگر `@` یعنی «مستطیل داخل مستطیل»؛ برای `ST_Intersects` همان `&&`). فقط سطرهای نامزد خوانده شدند و «Rows Removed by Filter» تقریباً صفر است. زمان کل («Execution Time») در سطر آخر) را در دو برنامه مقایسه کنید: در اجرای کتاب بیش از ۷۰ برابر سریع‌تر (حدود ۲۳۰ در برابر ۳ میلی‌ثانیه). روی جدول‌های میلیونی، اختلاف از چند ثانیه به چند میلی‌ثانیه می‌رسد. (چون نقطه‌ها تصادفی‌اند، عددهای شما کمی فرق می‌کند.)

واژه‌نامه‌ی برنامه‌ی اجرا

گره	معنا
`Seq Scan`	خواندن کل جدول؛ برای جدول کوچک یا وقتی بیشتر سطرها لازم‌اند طبیعی است
`Index Scan`	پیمایش شاخص و خواندن سطرهای منطبق
`Bitmap Index Scan` + `Bitmap Heap Scan`	ابتدا فهرست صفحه‌های نامزد از شاخص، سپس خواندن آن صفحه‌ها؛ برای تعداد متوسط سطر
`Index Cond`	شرطی که روی شاخص بررسی شد (مرحله‌ی اول)
`Filter` / `Rows Removed by Filter`	شرط دقیق پس از خواندن (مرحله‌ی دوم) و تعداد سطرهای رد شده
`Nested Loop`	برای هر سطر بیرونی، جست‌وجو در جدول درونی؛ با شاخص درونی سریع است
`Hash Join` ، `Merge Join`	روش‌های پیوند برای برابری؛ در پیوند مکانی کمتر
`cost=… rows=…`	تخمین برنامه‌ریز (بدون ANALYZE)
`actual time=… rows=…`	مقدار واقعی (با ANALYZE)

نکته

EXPLAIN بدون ANALYZE پرسش را اجرا نمی‌کند و فقط برنامه‌ی تخمینی را نشان می‌دهد؛ برای پرسش‌های سنگین یا تغییردهنده (`DELETE`) امن‌تر است. سامانه در کنسول SQL، برای پرسش‌هایی که کند اجرا می‌شوند، دکمه‌ی «چرا کند است؟» را نشان می‌دهد که برنامه را خوانا توضیح می‌دهد.

۲۰-۵ خطاهایی که شاخص را بی‌اثر می‌کنند

۱. **تابع روی ستون شاخص‌دار.** شاخص روی `geom` ساخته شده، نه روی `ST_Transform(geom, 32639)` . این پرسش شاخص را **استفاده نمی‌کند:**

```sql
-- کند: برای هر سطر تبدیل انجام و شاخص کنار گذاشته می‌شود
WHERE ST_DWithin(ST_Transform(geom, 32639), ST_Transform(:pt, 32639),
500)
```

دو راه درست: یا **ثابت** را تبدیل کنید و ستون را دست نزنید، یا روی عبارت شاخص بسازید:

```sql
-- راه ۱: شاخص روی عبارت geography
CREATE INDEX ON schools USING gist ((geom::geography));
WHERE ST_DWithin(geom::geography, :pt::geography, 500)

-- راه ۲: ستون ذخیره‌شده در UTM (اگر همه‌ی کارها متری است)
ALTER TABLE schools ADD COLUMN geom_utm geometry(Point, 32639)
  GENERATED ALWAYS AS (ST_Transform(geom, 32639)) STORED;
CREATE INDEX ON schools USING gist (geom_utm);
```

۲. **`ST_Distance(…) < d` به جای `ST_DWithin`** . `ST_Distance` شاخص ندارد (فصل ۱۷).

۳. **`ORDER BY ST_Distance(…)` به جای `<->`** برای نزدیک‌ترین (فصل ۱۹).

۴. **SRID ناهمسان.** اگر ستون ۴۳۲۶ است و ثابت پرسش ۳۲۶۳۹، یکی باید تبدیل شود؛ آن یکی ثابت باشد، نه ستون.

۵. آمار کهنه. برنامه‌ریز بر اساس آمار تصمیم می‌گیرد. پس از ورود حجم زیادی داده، `ANALYZE جدول;` را فراموش نکنید (یا «سلامت داده»).

۲۰-۶ انواع دیگر شاخص

نوع	کجا
GiST روی هندسه	پیش‌فرض و همه‌کاره؛ همیشه بسازید
SP-GiST روی هندسه	گاهی برای نقطه‌های بسیار زیاد و بی‌همپوشانی سریع‌تر
BRIN روی هندسه	جدول‌های عظیم افزایشی که داده با ترتیب مکانی یا زمانی وارد می‌شود (ردپای GPS)؛ بسیار کم‌حجم
B-tree روی ستون ویژگی	فیلتر روی `level` ، `type` ، تاریخ
GIN با `gin_trgm_ops`	جست‌وجوی متنی تقریبی (فصل ۱۱)
شاخص روی عبارت	`((geom::geography))` ، `(lower(name))`
شاخص جزئی	`CREATE INDEX … WHERE status = 'active'`؛ فقط سطرهای پرکاربرد
شاخص چندستونی	`(type, created_at)` برای فیلتر ترکیبی

۲۰-۷ هندسه‌های بزرگ: ST_Subdivide

شاخص فقط مستطیل محدوده را می‌شناسد. مستطیل محدوده‌ی مرز یک استان پهناور، بخش بزرگی از خارج استان را هم می‌پوشاند؛ پس فیلتر اولیه تقریباً هیچ نقطه‌ای را حذف نمی‌کند و بار سنگین روی بررسی دقیق با چندضلعی هزاران‌رأسی می‌افتد. راه حل، خرد کردن هندسه است:

```sql
CREATE TABLE provinces_sub AS
SELECT province_id, ST_Subdivide(geom, 256) AS geom FROM provinces;
CREATE INDEX ON provinces_sub USING gist (geom);
-- پرسش نقطه-در-استان حالا روی قطعه‌های کوچک با مستطیل‌های تنگ اجرا می‌شود
```

این ترفند در عمل پرسش‌های «کدام استان/شهرستان؟» روی میلیون‌ها نقطه را ده‌ها برابر سریع‌تر می‌کند.

۲۰-۸ نگه‌داری

- **`ANALYZE`**: به‌روزرسانی آمار؛ پس از ورود یا حذف انبوه.
- **`VACUUM`**: بازیافت فضای سطرهای حذف یا به‌روزشده؛ PostgreSQL آن را خودکار (autovacuum) انجام می‌دهد، ولی پس از `DELETE` یا `UPDATE` بزرگ یک `VACUUM ANALYZE جدول;` دستی کمک می‌کند. `VACUUM` فضا را به سیستم‌عامل برنمی‌گرداند؛ برای کاهش حجم واقعی جدول `VACUUM FULL` لازم است که جدول را قفل می‌کند.
- **`CLUSTER جدول USING شاخص_مکانی;`**: سطرها را روی دیسک به ترتیب مکانی می‌چیند تا عارضه‌های نزدیک در صفحه‌های نزدیک باشند؛ برای جدول‌های بزرگی که بیشتر خوانده می‌شوند تا نوشته.

۲۰-۹ فهرست کنترل کارایی

۱. هر ستون هندسه شاخص GiST دارد؟ («سلامت داده» بررسی می‌کند.)

۲. پرسش‌های فاصله با `ST_DWithin`، نزدیک‌ترین با `<->`؟

۳. تابعی روی ستون شاخص‌دار در `WHERE` نیست؟ اگر هست، شاخص روی عبارت یا ستون محاسباتی.

۴. فقط ستون‌های لازم در `SELECT`؟ (`SELECT *` با هندسه‌ی سنگین، داده‌ی زیادی جابه‌جا می‌کند.)

۵. هندسه‌های بسیار پیچیده خرد (`ST_Subdivide`) یا برای نمایش ساده شده‌اند؟

۶. آمار به‌روز است؟ (`ANALYZE`)

۷. برنامه را با `EXPLAIN ANALYZE` دیده‌اید؟

در دنیای واقعی

ردپای ناوگان. شرکتی پخش هر روز ۵ میلیون نقطه‌ی GPS از خودروهایش ذخیره می‌کند و گزارش «توقف‌های بیش از ده دقیقه در هر منطقه» ۴۰ ثانیه طول می‌کشید و از سقف زمان پرسش عبور می‌کرد. سه تغییر آن را به زیر یک ثانیه رساند: شاخص BRIN روی زمان ثبت (داده به ترتیب زمان وارد می‌شود)، شاخص GiST روی هندسه، و `ST_Subdivide` روی مرز منطقه‌ها. هیچ سخت‌افزار تازه‌ای لازم نشد.

خلاصه‌ی فصل

- شاخص GiST درخت R از مستطیل‌های محدوده می‌سازد و پرسش مکانی را از خطی به تقریباً لگاریتمی می‌رساند.
- گزاره‌ها دو مرحله دارند: &&روی شاخص (تقریبی)، سپس بررسی دقیق.
- `EXPLAIN ANALYZE` برنامه و زمان واقعی را نشان می‌دهد: Seq Scan در برابر Index/Bitmap Scan.
- تابع روی ستون، `ST_Distance < d` ، `ORDER BY ST_Distance` و آمار کهنه شاخص را بی‌اثر می‌کنند.
- شاخص روی عبارت، ستون محاسباتی، شاخص جزئی، BRIN و GIN هر کدام جای خود را دارند.
- `ST_Subdivide` برای هندسه‌های بزرگ؛ `ANALYZE` ، `VACUUM` و `CLUSTER` برای نگه‌داری.

برای مدرس

زمان پیشنهادی: یک جلسه. آزمایش «با شاخص و بی‌شاخص» را با یک میلیون نقطه زنده اجرا کنید و زمان را روی تخته بنویسید. سپس هر خطای بخش «خطاهایی که شاخص را بی‌اثر می‌کنند» را عمداً بسازید و برنامه‌ی اجرا را با دانشجوها بخوانید.

تمرین‌ها

۱. سطح ۲ برنامه‌ی اجرای پرسش «مدرسه‌های داخل محله‌ی ونک» را با `EXPLAIN` ببینید. از شاخص استفاده شد؟ چرا شاید نه؟ (راهنمایی: اندازه‌ی جدول)

۲. سطح ۲ شاخص مکانی `health_centers` را (که «سلامت داده» نبودش را گزارش کرد) بسازید و برنامه‌ی یک پرسش KNN را پیش و پس از آن مقایسه کنید.

۳. سطح ۳ با جدول ۲۰۰ هزار نقطه‌ای این فصل، زمان `ST_DWithin` روی geography را با `(geom::geography)` و بی شاخص روی بسنجید.

۴. سطح ۳ ستون محاسباتی `geom_utm` را به جدول نقطه‌ها اضافه کنید و حریم ۲۰۰ متری را با آن و با geography مقایسه کنید (زمان و نتیجه).

۵. سطح ۳ یک چندضلعی بزرگ با `ST_Buffer` یک نقطه به شعاع ۲۰ کیلومتر و `quad_segs=256` بسازید و پرسش نقطه-در-چندضلعی را روی نسخه‌ی کامل و خردشده (`ST_Subdivide`) مقایسه کنید.

فصل

۲۱

آمار فضایی با SQL؛ تراکم، خوشه و کانون داغ

سطح: **پیشرفته** ●●●

زمان مطالعه: **۱۲۰ دقیقه**

پیش‌نیاز: **فصل‌های ۱۲، ۱۸ و ۱۹**

در این فصل یاد می‌گیرید

- الگوی پراکنش نقطه‌ها را با شاخص نزدیک‌ترین همسایه بسنجید.
- شبکه‌ی شش‌ضلعی و مربعی بسازید و تراکم را روی آن حساب کنید.
- نقطه‌ها را با DBSCAN و k-means خوشه‌بندی کنید.
- خودهمبستگی فضایی را با شاخص موران اندازه بگیرید و تفسیر کنید.
- کانون‌های داغ را با آماره‌ی Gi* گتیس-ارد پیدا کنید.

سرفصل‌ها

۲۱-۱ از «کجا» تا «چرا اینجا»

۲۱-۲ الگوی پراکنش: شاخص نزدیک‌ترین همسایه

۲۱-۳ شبکه‌ی منظم و تراکم

۲۱-۴ خوشه‌بندی

۲۱-۵ خودهمبستگی فضایی: شاخص موران

۲۱-۶ کانون داغ: آماره‌ی Gi* گتیس-ارد

۲۱-۱ از «کجا» تا «چرا اینجا»

تا اینجا پرسیدیم «کجاست؟» و «چند تا؟». آمار فضایی یک قدم جلوتر می‌رود: «آیا این الگو **تصادفی** است یا معنادار؟». ده تصادف در یک تقاطع ممکن است اتفاق باشد یا نشانه‌ی یک نقص طراحی. آمار فضایی به ما می‌گوید کدام.

نقطه‌ی شروع همه‌ی این روش‌ها همان قانون اول جغرافیا است (فصل ۱): چیزهای نزدیک بیشتر به هم شبیه‌اند. اگر داده‌ی ما از این قانون پیروی کند، می‌گوییم **خودهمبستگی فضایی** دارد. بیشتر آزمون‌های آماری کلاسیک فرض می‌کنند مشاهده‌ها مستقل‌اند؛ در داده‌ی مکانی این فرض تقریباً همیشه نقض می‌شود، و این دلیل وجود رشته‌ای به نام آمار فضایی است.

۲۱-۲ الگوی پراکنش: شاخص نزدیک‌ترین همسایه

نقطه‌ها سه الگوی پایه دارند: **خوشه‌ای** (نزدیک هم جمع شده‌اند)، **تصادفی** و **منظم** (با فاصله‌ی یکنواخت از هم). **شاخص نزدیک‌ترین همسایه** (Nearest Neighbor Index)، کلارک و ایوانز، ۱۹۵۴) میانگین فاصله‌ی هر نقطه تا نزدیک‌ترین همسایه‌اش را با مقداری که برای پراکنش کاملاً تصادفی انتظار می‌رود مقایسه می‌کند:

$$R = \frac{\bar{d}_{obs}}{\bar{d}_{exp}}, \qquad \bar{d}_{exp} = \frac{0.5}{\sqrt{n/A}}$$

که n تعداد نقطه‌ها و A مساحت محدوده است. $R < 1$ یعنی خوشه‌ای، $R \approx 1$ تصادفی و $R > 1$ منظم.

```sql
WITH nn AS (
  SELECT ST_Distance(s.geom::geography, n.geom::geography) AS d
  FROM schools s
  CROSS JOIN LATERAL (SELECT geom FROM schools o WHERE o.gid <> s.gid
ORDER BY o.geom <-> s.geom LIMIT 1) n
),
area AS (SELECT ST_Area(ST_ConvexHull(ST_Collect(geom))::geography) AS
a, count(*) AS n FROM schools)
SELECT round(avg(d))                                    AS
observed_m,
       round(0.5 / sqrt(area.n / area.a))               AS
expected_m,
       round((avg(d) / (0.5 / sqrt(area.n / area.a)))::numeric, 3) AS
nni
FROM nn, area
GROUP BY area.n, area.a;
```

observed_m	expected_m	nni
1006.0	942.0	1.068

`R` کمی بیشتر از ۱ است: پراکنش مدرسه‌ها نزدیک به تصادفی است، با گرایشی اندک به منظم. اما آیا این انحراف از ۱ **معنادار** است؟ برای پاسخ، آماره‌ی z آن را حساب می‌کنیم ($z = (\bar{d}_{obs} - \bar{d}_{exp})/SE$ با $SE = 0.26136/\sqrt{n^2/A}$). تمرین ۳ این را به شما می‌سپارد.

> **هشدار**
>
> **حساسیت به محدوده.** A (مساحت محدوده) نتیجه را شدیداً تحت تأثیر قرار می‌دهد: همان نقطه‌ها در محدوده‌ای بزرگ‌تر «خوشه‌ای‌تر» به نظر می‌رسند. پوسته‌ی محدب یک انتخاب رایج است، اما بهتر است محدوده‌ی واقعی مطالعه (مرز شهر) را به کار ببرید و آن را در گزارش ذکر کنید.

۲۱-۳ شبکه‌ی منظم و تراکم

نقشه‌ی هزاران نقطه خوانا نیست. راه رایج، **تجمیع روی شبکه‌ی منظم** است: محدوده را با شش‌ضلعی یا مربع‌های هم‌اندازه می‌پوشانیم و نقطه‌های هر خانه را می‌شمریم. شش‌ضلعی

معمولاً بهتر است: فاصله‌ی مرکز تا همه‌ی همسایه‌ها یکسان است و شکلش به دایره نزدیک‌تر است، پس اثر جهت کمتر است.

PostGIS دو تابع برای ساختن شبکه دارد: `ST_HexagonGrid(اندازه، محدوده)` و `ST_SquareGrid(اندازه، محدوده)`. اندازه به واحد SRID محدوده است؛ پس شبکه را در UTM می‌سازیم تا اندازه به متر باشد:

```sql
WITH bounds AS (SELECT ST_Transform(ST_SetSRID(ST_Extent(geom),
4326), 32639) AS b FROM schools),
grid AS (SELECT h.i, h.j, h.geom FROM bounds, LATERAL
ST_HexagonGrid(1500, bounds.b) h)
SELECT g.i, g.j, count(s.gid) AS schools, sum(s.students) AS students
FROM grid g
JOIN schools s ON ST_Intersects(g.geom, ST_Transform(s.geom, 32639))
GROUP BY g.i, g.j
ORDER BY schools DESC
LIMIT 5;
```

i	j	schools	students
244	1522	5	2628
245	1522	5	2391
244	1521	5	2425
239	1520	5	1656
244	1519	4	2081

`i` و `j` شناسه‌ی هر خانه در شبکه‌اند. با `JOIN` فقط خانه‌های پر آمدند؛ برای نقشه‌ی کامل (با خانه‌های صفر) از `LEFT JOIN` استفاده کنید. ابزار «نقشه‌ی تراکم شش‌ضلعی» سامانه (فصل ۲۶) دقیقاً همین را با یک فرم انجام می‌دهد. نتیجه را با `CREATE TABLE AS` ذخیره و در استودیوی نقشه با طبقه‌بندی رنگی نمایش دهید (فصل ۴۲).

بدانید

مسئله‌ی واحد مساحتی قابل تغییر (MAUP). نتیجه‌ی هر تحلیل تجمیعی به **اندازه** و **جای** خانه‌ها بستگی دارد. همان نقطه‌ها با شش‌ضلعی ۵۰۰ متری یک الگو و با ۲ کیلومتری الگوی دیگری نشان می‌دهند؛ حتی جابه‌جا کردن شبکه به اندازه‌ی نصف یک خانه می‌تواند کانون‌ها را عوض کند. این پدیده که اوپنشا در ۱۹۸۴ صورت‌بندی کرد، یکی از مهم‌ترین هشدارهای GIS است: همیشه چند اندازه را امتحان کنید و اندازه‌ی منتخب را با دلیل گزارش کنید.

H3: شبکه‌ی شش‌ضلعی جهانی

H3 سامانه‌ی شبکه‌ی شش‌ضلعی سلسله‌مراتبی است که کره‌ی زمین را در ۱۶ سطح تفکیک با شش‌ضلعی‌هایی با شناسه‌ی ثابت می‌پوشاند. مزیتش این است که شناسه‌ی هر خانه همه‌جا و برای همه یکسان است و داده‌های مختلف را می‌توان بدون پیوند مکانی، با شناسه، روی هم گذاشت. اگر افزونه‌ی `h3` (و `h3_postgis`) روی سرور شما فعال باشد:

```sql
SELECT h3_lat_lng_to_cell(geom, 8) AS cell, count(*)          -- سطح ۸: خانه‌های حدود ۰٫۷ کیلومتر مربع
FROM schools GROUP BY 1 ORDER BY 2 DESC;
SELECT h3_cell_to_boundary_geometry('88…'::h3index);          -- هندسه‌ی یک خانه
```

۲۱-۴ خوشه‌بندی

خوشه‌بندی نقطه‌ها را به گروه‌هایی تقسیم می‌کند که اعضایشان به هم نزدیک‌اند. PostGIS سه تابع پنجره‌ای خوشه‌بندی دارد.

DBSCAN: خوشه‌های چگال

DBSCAN (۱۹۹۶) نقطه‌ای را «هسته» می‌داند که دست‌کم `minpoints` همسایه در فاصله‌ی `eps` داشته باشد؛ هسته‌های به‌هم‌پیوسته یک خوشه می‌سازند و نقطه‌های پرت در هیچ خوشه‌ای نیستند (NULL). مزیتش این است که **تعداد خوشه را از پیش نمی‌خواهد** و شکل خوشه‌ها آزاد است؛ برای یافتن کانون‌های واقعی (تصادف، جرم، شکایت) بسیار مناسب است.

```sql
WITH c AS (
  SELECT gid, students, geom,
         ST_ClusterDBSCAN(ST_Transform(geom, 32639), eps := 800,
minpoints := 4) OVER () AS cid
  FROM schools
)
SELECT cid,
       count(*)                                                AS schools,
       sum(students)                                           AS
students,
       ST_AsText(ST_Centroid(ST_Collect(geom)), 4)             AS center
FROM c
WHERE cid IS NOT NULL
GROUP BY cid
ORDER BY schools DESC;
```

cid	schools	students	center
1	5	1656	POINT(51.4142 35.6892)
0	4	1810	POINT(51.5456 35.7059)
2	4	2688	POINT(51.3661 35.79)
3	4	2337	POINT(51.374 35.7543)

چند خوشه‌ی کوچک پیدا شد و بیشتر مدرسه‌ها «پرت» ماندند؛ یعنی مدرسه‌ها نسبتاً پراکنده‌اند و فقط چند کانون کوچک دارند. با تغییر `eps` و `minpoints` حساسیت را تنظیم کنید. `ST_ClusterWithin(geom, d)` ساده‌تر است: همه‌ی نقطه‌هایی را که با زنجیره‌ای از فاصله‌های کمتر از d به هم وصل‌اند یک خوشه می‌کند.

k-means: تعداد خوشه‌ی معین

k-means نقطه‌ها را به **k** گروه تقسیم می‌کند طوری که مجموع فاصله‌ی هر نقطه تا مرکز گروهش کمینه شود. وقتی تعداد گروه از پیش معلوم است کاربرد دارد؛ مثلاً تقسیم مدرسه‌ها بین ۵ بازرس یا منطقه‌بندی توزیع بین ۶ انبار:

```sql
WITH k AS (
  SELECT students, geom, ST_ClusterKMeans(ST_Transform(geom, 32639), 5)
OVER () AS zone
  FROM schools
)
SELECT zone, count(*) AS schools, sum(students) AS students,
       round(ST_Area(ST_ConvexHull(ST_Collect(geom))::geography) / 1e6)
AS hull_km2
FROM k GROUP BY zone ORDER BY zone;
```

zone	schools	students	hull_km2
0	27	12992	85.0
1	46	22884	101.0
2	24	12481	93.0
3	42	22492	123.0
4	41	20671	103.0

نکته

k-means با وزن. در PostGIS ۳٫۲ به بعد، اگر به هندسه‌ی نقطه مقدار M (اندازه) بدهید، k-means آن را وزن نقطه می‌گیرد؛ مثلاً `ST_Force3DM(geom, students)` تا مدرسه‌های بزرگ‌تر مرکز خوشه را به سمت خود بکشند. نتیجه‌ی k-means به نقطه‌ی شروع تصادفی وابسته است و ممکن است در هر اجرا کمی فرق کند.

۲۱-۵ خودهمبستگی فضایی: شاخص موران

شاخص موران (Moran's I، ۱۹۵۰) می‌پرسد: آیا خانه‌های مجاور مقدار مشابه دارند؟

$$I = \frac{n}{W} \cdot \frac{\sum_i \sum_j w_{ij}(x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2}$$

که w_{ij} وزن فضایی (اینجا ۱ برای خانه‌های مجاور و ۰ برای بقیه) و W مجموع وزن‌هاست. I بین حدود ۱- و ۱ است: مثبت یعنی مقدارهای مشابه کنار هم‌اند (خوشه‌ای)، نزدیک به مقدار انتظاری $-1/(n-1)$ یعنی تصادفی، و منفی یعنی مقدارهای متضاد کنار هم‌اند (شطرنجی).

محاسبه‌ی آن با SQL روی شبکه‌ی شش‌ضلعی، با تعریف همسایگی «هم‌مرز» (`ST_Touches`):

```sql
WITH bounds AS (SELECT ST_Transform(ST_SetSRID(ST_Extent(geom),
4326), 32639) AS b FROM schools),
grid AS (SELECT h.i, h.j, h.geom FROM bounds, LATERAL
ST_HexagonGrid(1500, bounds.b) h),
cells AS (                                   -- تعداد مدرسه در هر خانه (با
خانه‌های صفر)
  SELECT g.i, g.j, g.geom, count(s.gid) AS x
  FROM grid g LEFT JOIN schools s ON ST_Intersects(g.geom,
ST_Transform(s.geom, 32639))
  GROUP BY g.i, g.j, g.geom
),
st AS (SELECT count(*) AS n, avg(x) AS m FROM cells),
pairs AS (                                   -- جفت‌های همسایه (وزن ۱)
  SELECT a.x AS xi, b.x AS xj
  FROM cells a JOIN cells b ON (a.i, a.j) <> (b.i, b.j) AND
ST_Touches(a.geom, b.geom)
)
SELECT st.n                                                    AS
cells,
       count(*)                                                AS
neighbor_pairs,
       round((st.n::numeric / count(*)) * sum((xi - st.m) * (xj -
st.m))
             / (SELECT sum((x - st.m) ^ 2) FROM cells), 4)      AS
morans_i,
       round(-1.0 / (st.n - 1), 4)                             AS
expected_i
FROM pairs, st
GROUP BY st.n, st.m;
```

cells	neighbor_pairs	morans_i	expected_i
176	950	0.2551	-0.0057

/ به‌روشنی بزرگ‌تر از مقدار انتظاری است: خانه‌های پرمدرسه کنار هم‌اند. این با نتیجه‌ی شاخص نزدیک‌ترین همسایه (نزدیک به تصادفی) تناقض ندارد و تفاوت ظریف و مهمی را نشان می‌دهد: **اثر مرتبه‌ی اول** یعنی تغییر تراکم در فضا (مرکز شهر مدرسه‌ی بیشتری دارد تا حاشیه)، و **اثر مرتبه‌ی دوم** یعنی برهم‌کنش نقطه‌ها (آیا مدرسه‌ها کنار هم جمع می‌شوند یا از هم دوری

می‌کنند). موران روی شمارش خانه‌ها اولی را می‌بیند و شاخص نزدیک‌ترین همسایه بیشتر دومی را؛ گزارش خوب هر دو را می‌سنجد. برای آزمون معناداری، معمولاً از **آزمون جایگشت** استفاده می‌شود: مقدارها صدها بار به‌طور تصادفی بین خانه‌ها جابه‌جا و *I* دوباره حساب می‌شود؛ اگر *I* واقعی از تقریباً همه‌ی *I*های تصادفی بزرگ‌تر بود، الگو معنادار است. این کار در SQL ممکن ولی سنگین است و ابزارهای تخصصی (کتابخانه‌ی PySAL در پایتون یا GeoDa) برایش مناسب‌ترند؛ داده را با اتصال مستقیم (فصل ۶) از همان پایگاه می‌خوانند.

۲۱-۶ کانون داغ: آماره‌ی Gi* گتیس-ارد

شاخص موران یک عدد برای کل نقشه می‌دهد. **آماره‌ی Gi*** (گتیس و ارد، ۱۹۹۲) برای **هر خانه** می‌گوید آیا آن خانه و همسایه‌هایش با هم به‌طور معناداری بالاتر (کانون داغ) یا پایین‌تر (کانون سرد) از میانگین‌اند. خروجی آن یک **امتیاز z** است:

$$G_i^* = \frac{\sum_j w_{ij} x_j - \bar{X} \sum_j w_{ij}}{S \sqrt{\frac{n \sum_j w_{ij}^2 - \left(\sum_j w_{ij}\right)^2}{n-1}}}$$

که جمع روی خانه‌ی i و همسایه‌هایش (شامل خودش) است و S انحراف معیار. $z > 1.96$ یعنی کانون داغ با اطمینان ۹۵ درصد و $z > 2.58$ با اطمینان ۹۹ درصد.

```sql
WITH bounds AS (SELECT ST_Transform(ST_SetSRID(ST_Extent(geom),
4326), 32639) AS b FROM schools),
grid AS (SELECT h.i, h.j, h.geom FROM bounds, LATERAL
ST_HexagonGrid(1500, bounds.b) h),
cells AS (
  SELECT g.i, g.j, g.geom, count(s.gid) AS x
  FROM grid g LEFT JOIN schools s ON ST_Intersects(g.geom,
ST_Transform(s.geom, 32639))
  GROUP BY g.i, g.j, g.geom
),
st AS (SELECT count(*)::numeric AS n, avg(x) AS m, sqrt(avg(x * x) -
avg(x) ^ 2) AS s FROM cells),
nb AS (                                    -- جمع مقدار خانه و
همسایه‌هایش (ST_Intersects خود خانه را هم دارد)
  SELECT a.i, a.j, sum(b.x) AS sx, count(*) AS wn
  FROM cells a JOIN cells b ON ST_Intersects(a.geom, b.geom)
  GROUP BY a.i, a.j
)
SELECT nb.i, nb.j, c.x AS schools,
       round(((nb.sx - st.m * nb.wn) / (st.s * sqrt((st.n * nb.wn -
nb.wn ^ 2) / (st.n - 1))))::numeric, 2) AS gi_z,
       CASE WHEN (nb.sx - st.m * nb.wn) / (st.s * sqrt((st.n * nb.wn -
nb.wn ^ 2) / (st.n - 1))) > 2.58 THEN '٪داغ ۹۹'
            WHEN (nb.sx - st.m * nb.wn) / (st.s * sqrt((st.n * nb.wn -
nb.wn ^ 2) / (st.n - 1))) > 1.96 THEN '٪داغ ۹۵'
            ELSE 'معنادار نیست' END AS class
FROM nb JOIN cells c USING (i, j), st
ORDER BY gi_z DESC
LIMIT 5;
```

i	j	schools	gi_z	class
244	1522	5	5.39	٪داغ ۹۹
245	1521	1	4.11	٪داغ ۹۹
243	1522	2	3.79	٪داغ ۹۹
244	1523	4	3.79	٪داغ ۹۹
237	1523	2	3.47	٪داغ ۹۹

خانه‌هایی که بالای فهرست‌اند، با همسایه‌هایشان کانون معنادار مدرسه‌اند. اگر این پرسش را با `CREATE TABLE AS` (همراه هندسه‌ی خانه‌ها) ذخیره و در استودیوی نقشه با طیف رنگی واگرا

(آبی برای سرد، قرمز برای داغ) نمایش دهید، **نقشه‌ی کانون‌های داغ** کامل دارید.

هشدار

آزمون‌های چندگانه. وقتی برای صدها خانه هم‌زمان آزمون می‌کنید، با سطح ۹۵ درصد انتظار می‌رود حدود ۵ درصد خانه‌ها به‌طور تصادفی «معنادار» شوند. برای گزارش رسمی، آستانه را سخت‌تر کنید (مثلاً اصلاح بونفرونی یا روش FDR) و همیشه نتیجه را با دانش میدانی بسنجید. نقشه‌ی کانون داغ یک **فرضیه** است، نه حکم.

در دنیای واقعی

کانون تصادف‌ها. پلیس راهور یک شهر نقطه‌ی تصادف‌های سه سال را در پایگاه داده دارد. تحلیل Gi* روی شش‌ضلعی‌های ۳۰۰ متری، ده کانون معنادار را نشان داد که هفت تا از آن‌ها تقاطع‌هایی بودند که از پیش شناخته شده بودند، اما سه تا روی بزرگراهی بودند که کسی گمانش را نمی‌برد. بازدید میدانی نشان داد خروجی‌های آن بخش تابلوی هشدار ندارند. یک پرسش SQL و یک بازدید، سه کانون خطر را آشکار کرد.

خلاصه‌ی فصل

- شاخص نزدیک‌ترین همسایه (R) الگو را خوشه‌ای ($R < 1$)، تصادفی ($R \approx 1$) یا منظم ($R > 1$) می‌سنجد؛ به محدوده حساس است.
- `ST_HexagonGrid` و `ST_SquareGrid` در UTM شبکه‌ی متری می‌سازند؛ نتیجه به اندازه‌ی خانه وابسته است (MAUP).
- `ST_ClusterDBSCAN` خوشه‌های چگال بدون تعداد از پیش، `ST_ClusterKMeans` k گروه، `ST_ClusterWithin` زنجیره‌ی فاصله.
- شاخص موران خودهمبستگی کلی و Gi* کانون‌های داغ و سرد محلی را می‌دهد؛ هر دو با SQL روی شبکه ممکن‌اند.
- معناداری را با احتیاط تفسیر کنید؛ کانون داغ فرضیه است، نه حکم.

برای مدرس

زمان پیشنهادی: دو جلسه. پیش از فرمول‌ها، سه نقشه‌ی نقطه‌ای (خوشه‌ای، تصادفی، منظم) نشان دهید و بگذارید دانشجوها حدس بزنند؛ معمولاً الگوی تصادفی را «خوشه‌ای» می‌بینند، چون ذهن انسان در دیدن الگو زیاده‌روی می‌کند؛ بهترین دلیل برای آمار فضایی. MAUP را با تغییر اندازه‌ی شش‌ضلعی زنده نشان دهید.

تمرین‌ها

۱. سطح ۲ شاخص نزدیک‌ترین همسایه را برای مراکز درمانی حساب کنید. الگوی آن‌ها چیست؟ نگاهی به نحوه‌ی ساختن این داده در `sample.sql` نتیجه را توضیح می‌دهد.

۲. سطح ۲ شبکه‌ی شش‌ضلعی ۱ کیلومتری بسازید و مجموع دانش‌آموزان هر خانه را حساب کنید؛ نتیجه را ذخیره و در استودیوی نقشه نمایش دهید.

۳. سطح ۳ امتیاز z شاخص نزدیک‌ترین همسایه‌ی مدرسه‌ها را حساب کنید. آیا خوشه‌ای بودن معنادار است؟

۴. سطح ۳ شاخص موران را برای اندازه‌ی خانه‌ی ۱۰۰۰ و ۳۰۰۰ متر تکرار کنید. نتیجه چگونه با MAUP توضیح داده می‌شود؟

۵. سطح ۳ مدرسه‌ها را با k-means به ۵ منطقه تقسیم کنید و برای هر منطقه یک چندضلعی (پوسته‌ی مقعر) بسازید. منطقه‌ها از نظر تعداد دانش‌آموز متعادل‌اند؟ چگونه متعادل‌ترشان می‌کنید؟

فصل

۲۲

اعتبار هندسه، توپولوژی و هندسه‌ی سه‌بعدی

سطح: **پیشرفته** ●●● زمان مطالعه: **۹۰ دقیقه** پیش‌نیاز: **فصل ۱۸**

در این فصل یاد می‌گیرید

- قواعد اعتبار هندسه را بشناسید و علت نامعتبر بودن را با SQL تشخیص دهید.
- هندسه‌ی نامعتبر را با ST_MakeValid اصلاح و نتیجه را کنترل کنید.
- قواعد توپولوژی (همپوشانی، شکاف، گره‌ی آویزان) را با SQL بررسی کنید.
- با افزونه‌ی postgis_topology توپولوژی ذخیره‌شده بسازید.
- با مختصات Z کار کنید و عملیات سه‌بعدی SFCGAL را بشناسید.

سرفصل‌ها

۲۲-۱ هندسه‌ی معتبر چیست

۲۲-۲ اصلاح: ST_MakeValid

۲۲-۳ قواعد توپولوژی با SQL

۲۲-۴ توپولوژی ذخیره‌شده: postgis_topology

۲۲-۵ هندسه‌ی سه‌بعدی

۲۲-۱ هندسه‌ی معتبر چیست

استاندارد OGC برای هر نوع هندسه قواعدی دارد. هندسه‌ای که این قواعد را رعایت نکند **نامعتبر** (invalid) است. PostGIS هندسه‌ی نامعتبر را می‌پذیرد و ذخیره می‌کند، اما توابعی مثل `ST_Intersection`، `ST_Union` و `ST_Area` روی آن ممکن است خطا بدهند یا نتیجه‌ی **بی‌صدا و غلط** برگردانند. پس اعتبار هندسه پیش‌شرط هر تحلیل است.

قاعده	نقض آن	نام رایج
حلقه بسته باشد	نقطه‌ی آخر ≠ نقطه‌ی اول	حلقه‌ی باز
حلقه خودش را قطع نکند	دو ضلع حلقه از هم بگذرند	پاپیون (bowtie)
حلقه حداقل چهار نقطه داشته باشد	خط تبهگن	
حفره داخل حلقه‌ی بیرونی باشد	حفره بیرون زده	
حفره‌ها هم‌پوشانی نداشته باشند	دو حفره روی هم	
اجزای MultiPolygon هم‌پوشانی نداشته باشند	دو بخش روی هم	
حلقه در بیش از یک نقطه با خودش تماس نداشته باشد	گره‌ی خودتماس	

```sql
SELECT name, ST_IsValid(g) AS valid, ST_IsValidReason(g) AS reason
FROM (VALUES
  ('مربع سالم', 'POLYGON((0 0, 4 0, 4 4, 0 4, 0 0))'::geometry),
  ('پاپیون',    'POLYGON((0 0, 4 4, 4 0, 0 4, 0 0))'::geometry),
  ('حفره‌ی بیرون‌زده', 'POLYGON((0 0, 4 0, 4 4, 0 4, 0 0), (3 3, 6 3, 6 6, 3 6, 3 3))'::geometry),
  ('دو بخش هم‌پوشان', 'MULTIPOLYGON(((0 0, 4 0, 4 4, 0 4, 0 0)), ((2 2, 6 2, 6 6, 2 6, 2 2)))'::geometry)
) t(name, g);
```

name	valid	reason
مربع سالم	True	Valid Geometry
پاپیون	False	Self-intersection[2 2]
حفره‌ی بیرون‌زده	False	Self-intersection[3 4]
دو بخش هم‌پوشان	False	Self-intersection[2 4]

`ST_IsValidReason` محل دقیق خطا را می‌گوید و `ST_IsValidDetail` همان را به صورت سطر (علت و نقطه‌ی هندسی محل خطا) برمی‌گرداند که می‌توان روی نقشه نشانش داد.

> **بدانید**
>
> **هندسه‌ی نامعتبر از کجا می‌آید؟** رقومی‌سازی دستی بی‌دقت، تبدیل از قالب CAD (DXF)، ساده‌سازی بدون حفظ توپولوژی (`ST_Simplify`)، گرد کردن مختصات و تبدیل بین سیستم‌های مختصات. Shapefile و بسیاری از نرم‌افزارهای قدیمی اعتبار را بررسی نمی‌کنند؛ برای همین داده‌ی وارداتی اغلب نامعتبر است.

۲۲-۲ اصلاح: ST_MakeValid

`ST_MakeValid` هندسه را بدون حذف هیچ رأسی معتبر می‌کند. نتیجه ممکن است **نوع دیگری** داشته باشد: پاپیون به MultiPolygon دوبخشی تبدیل می‌شود.

```sql
SELECT ST_AsText(ST_MakeValid('POLYGON((0 0, 4 4, 4 0, 0 4, 0 0))'))
AS bowtie_fixed,
       ST_Area(ST_MakeValid('POLYGON((0 0, 4 4, 4 0, 0 4, 0 0))'))
AS fixed_area,
       ST_Area('POLYGON((0 0, 4 4, 4 0, 0 4, 0 0))'::geometry)
AS invalid_area,
       ST_AsText(ST_MakeValid('MULTIPOLYGON(((0 0,4 0,4 4,0 4,0 0)),((2
2,6 2,6 6,2 6,2 2)))',
                              'method=structure'))
AS overlap_fixed;
```

bowtie_fixed	fixed_area	invalid_area	overlap_fixed
MULTIPOLYGON(((0 4,2 2,0 0,0 4)),((4 0,2 2,4 4,4 0)))	8.0	0.0	MULTIPOLYGON(((0 4,2 4,2 6,6 6,6 2,4 2,4 0,0 0,0 4)))

مساحت پاپیون نامعتبر **صفر** گزارش شد (دو مثلث با جهت مخالف هم را خنثی کردند) در حالی که مساحت واقعی ۸ است؛ نمونه‌ی کامل «نتیجه‌ی بی‌صدا و غلط». از PostGIS ۳٫۲، `ST_MakeValid` دو روش دارد: `method=linework` (پیش‌فرض؛ همه‌ی خط‌ها را نگه می‌دارد)

و `method=structure` (حلقه‌ها را بر اساس ساختار بیرونی/حفره بازسازی می‌کند و برای اجزای هم‌پوشان، اجتماعشان را می‌گیرد).

دستور اصلاح یک لایه، با حفظ نوع ستون:

```sql
UPDATE parks
SET geom = ST_Multi(ST_CollectionExtract(ST_MakeValid(geom), 3))
WHERE NOT ST_IsValid(geom);
```

`ST_CollectionExtract(…, 3)` فقط بخش‌های چندضلعی را نگه می‌دارد (اگر اصلاح، خط یا نقطه‌ی اضافه ساخته باشد) و `ST_Multi` همه را چندبخشی می‌کند تا با نوع ستون `MultiPolygon` جور شود. دکمه‌ی «اصلاح هندسه» در «سلامت داده» (فصل ۷) همین کار را می‌کند.

> **هشدار**
>
> **پس از اصلاح، بسنجید.** `ST_MakeValid` معتبر می‌کند، اما «درست» نمی‌کند. پاپیونی که از خطای رقومی‌سازی آمده، با اصلاح خودکار به دو مثلث تبدیل می‌شود، در حالی که منظور رقومی‌کننده احتمالاً یک چهارضلعی بوده است. پس از اصلاح انبوه، تعداد عارضه‌هایی را که نوع یا مساحتشان محسوس عوض شده بشمارید و آن‌ها را دستی بازبینی کنید. ترفند قدیمی `ST_Buffer(geom, 0)` هم گاهی معتبر می‌کند، اما ممکن است بخشی از هندسه را بی‌صدا حذف کند؛ `ST_MakeValid` امن‌تر است.

۳-۲۲ قواعد توپولوژی با SQL

اعتبار هر عارضه به تنهایی کافی نیست؛ **رابطه‌ی** عارضه‌ها هم قاعده دارد. لایه‌ی قطعه‌های ثبتی نباید همپوشانی یا شکاف داشته باشد؛ خط‌های شبکه‌ی آب باید در گره‌ها به هم برسند. این قواعد را می‌توان با SQL بررسی کرد:

همپوشانی (فصل ۱۷):

```sql
SELECT count(*) AS overlapping_parcel_pairs
FROM parcels a JOIN parcels b ON a.id < b.id AND ST_Overlaps(a.geom,
b.geom);
```

overlapping_parcel_pairs
0

شکاف بین چندضلعی‌هایی که باید پیوسته باشند: حفره‌های اجتماع آن‌ها. اگر اجتماع لایه حفره داشته باشد، یا شکاف واقعی است (مثلاً معبر) یا خطای رقومی‌سازی:

```sql
SELECT ST_MakePolygon(ST_InteriorRingN(u, n)) AS gap,
ST_Area(ST_MakePolygon(ST_InteriorRingN(u, n))::geography) AS m2
FROM (SELECT (ST_Dump(ST_Union(geom))).geom AS u FROM parcels_cadastre)
x,
     generate_series(1, ST_NumInteriorRings(u)) n
ORDER BY m2;   -- شکاف‌های بسیار کوچک (sliver) معمولاً خطا هستند
```

گره‌ی آویزان در شبکه‌ی خطی: انتهای خطی که به هیچ خط دیگری نرسیده است. در شبکه‌ی معابر، آویزان‌ها مسیریابی را خراب می‌کنند (فصل ۲۴):

```sql
WITH ends AS (
  SELECT gid, ST_StartPoint(geom) AS p FROM roads UNION ALL
  SELECT gid, ST_EndPoint(geom) FROM roads
)
SELECT e.gid, e.p
FROM ends e
WHERE NOT EXISTS (SELECT 1 FROM roads r WHERE r.gid <> e.gid AND
ST_DWithin(r.geom, e.p, 0.000001));
```

`ST_Snap(a, b, tolerance)` برای بستن شکاف‌های باریک بین دو لایه یا دو عارضه، رأس‌های a را به b می‌چسباند (فصل ۱۸).

۲۲-۴ توپولوژی ذخیره‌شده: postgis_topology

مدل «ویژگی‌های ساده» هر چندضلعی را مستقل نگه می‌دارد: مرز مشترک دو قطعه **دو بار** ذخیره می‌شود. افزونه‌ی `postgis_topology` مدل دیگری دارد: داده را به **گره** (node)، **یال** (edge) و **وجه** (face) می‌شکند و هر مرز مشترک فقط **یک بار** ذخیره می‌شود. اگر آن مرز را ویرایش کنید، هر دو قطعه با هم عوض می‌شوند؛ شکاف و همپوشانی از اساس ممکن نیست. این مدل برای کاداستر، تقسیمات کشوری و شبکه‌ها مناسب است.

```sql
SELECT topology.CreateTopology('parcels_topo', 4326, 0.0000001);
SELECT topology.AddTopoGeometryColumn('parcels_topo', 'public',
'parcels', 'topo', 'POLYGON');
UPDATE parcels SET topo = topology.toTopoGeom(geom, 'parcels_topo', 1,
0.0000001);

SELECT (SELECT count(*) FROM parcels_topo.node) AS nodes,
       (SELECT count(*) FROM parcels_topo.edge) AS edges,
       (SELECT count(*) FROM parcels_topo.face) AS faces;
```

nodes	edges	faces
12	12	13

۱۲ قطعه‌ی نمونه به ۱۲ یال و ۱۳ وجه تبدیل شدند (وجه شماره‌ی صفر، «جهان بیرونی» است). چون قطعه‌های نمونه از هم جدا هستند، هیچ یالی بین دو قطعه مشترک نیست؛ در لایه‌ی کاداستر واقعی، تعداد یال‌ها بسیار کمتر از مجموع اضلاع قطعه‌هاست و همین «صرفه‌جویی» ضامن سازگاری است. ستون `topo` هندسه‌ی هر قطعه را از روی یال‌ها می‌سازد (`topo::geometry`) و توابعی مثل `topology.ST_ChangeEdgeGeom` یک یال مشترک را برای همه‌ی قطعه‌های مجاور ویرایش می‌کنند.

سطح حرفه‌ای

تلورانس توپولوژی. آخرین عدد در `CreateTopology` و `toTopoGeom` تلورانس است: رأس‌هایی نزدیک‌تر از این فاصله یکی شمرده می‌شوند. در SRID ۴۳۲۶، `0.0000001` درجه حدود یک سانتی‌متر است. تلورانس بزرگ رأس‌های واقعی را ادغام می‌کند و تلورانس خیلی کوچک شکاف‌های میکروسکوپی را باقی می‌گذارد. برای داده‌ی کاداستر در UTM، تلورانس را به متر (مثلاً ۰٫۰۱) بگذارید.

۲۲-۵ هندسه‌ی سه‌بعدی

مختصات Z

هر رأس می‌تواند ارتفاع (Z) داشته باشد (فصل ۲). توابع دوبعدی Z را **نادیده می‌گیرند**؛ برای محاسبات سه‌بعدی توابع `3D` هست:

```sql
WITH p AS (SELECT 'POINT Z (0 0 0)'::geometry AS a, 'POINT Z (30 40 120)'::geometry AS b,
                  'LINESTRING Z (0 0 1200, 300 400 1350, 600 800 1290)'::geometry AS road)
SELECT ST_Distance(a, b)    AS distance_2d,
       ST_3DDistance(a, b)  AS distance_3d,
       ST_Length(road)      AS road_2d,
       round(ST_3DLength(road)::numeric, 1) AS road_3d,
       ST_ZMax(road) - ST_ZMin(road)        AS elevation_range
FROM p;
```

distance_2d	distance_3d	road_2d	road_3d	elevation_range
50.0	130.0	1000.0	1025.6	150.0

طول سه‌بعدی جاده‌ی کوهستانی (طول واقعی که خودرو می‌پیماید) از طول افقی بیشتر است. `ST_Force2D` و `ST_Force3D` بُعد را حذف یا اضافه می‌کنند. برای افزودن ارتفاع از DEM به نقطه‌ها، فصل ۲۳ را ببینید.

SFCGAL: عملیات سه‌بعدی واقعی

افزونه‌ی `postgis_sfcgal` عملیات هندسی سه‌بعدی را اضافه می‌کند. پرکاربردترینشان **اکسترود** است: ساختن حجم سه‌بعدی ساختمان از ردپای دوبعدی و ارتفاع آن:

```sql
WITH b AS (SELECT ST_Extrude('POLYGON((0 0, 20 0, 20 12, 0 12, 0 0))'::geometry, 0, 0, 15) AS g)
SELECT ST_GeometryType(g)                                  AS building_3d,
       ST_NumGeometries(g)                                 AS faces,
       round(ST_Volume(ST_MakeSolid(g))::numeric)          AS volume_m3,
       (SELECT round(sum(ST_3DArea((d).geom))::numeric) FROM ST_Dump(g) d) AS surface_m2
FROM b;
```

building_3d	faces	volume_m3	surface_m2
ST_PolyhedralSurface	6	3600	1440

سطح بیرونی را با جمع مساحت سه‌بعدی تک‌تک وجه‌ها (`ST_Dump`) حساب کرده‌ایم؛ در بعضی نسخه‌ها `ST_3DArea` روی کل سطح چندوجهی صفر برمی‌گرداند و این روش مطمئن‌تر است.

ساختمانی ۲۰ در ۱۲ متر با ارتفاع ۱۵ متر: ۳۶۰۰ متر مکعب حجم و ۱۴۴۰ متر مربع سطح بیرونی (برای برآورد نما یا عایق‌کاری). `ST_3DIntersection`، `ST_3DUnion` و `ST_StraightSkeleton` (اسکلت سقف) هم در این افزونه‌اند. نمایش سه‌بعدی نتیجه در QGIS (نمای سه‌بعدی) یا کتابخانه‌های وب سه‌بعدی انجام می‌شود.

در دنیای واقعی

ساختمان‌های OSM در سه‌بعد. داده‌ی آماده‌ی ایران (فصل ۳۰) ردپای ساختمان‌ها را از OpenStreetMap می‌آورد و برای بعضی، تعداد طبقه (`building:levels`). با فرض سه متر برای هر طبقه، `ST_Extrude(geom, 0, 0, levels * 3)` مدل سه‌بعدی ساده‌ی شهر را می‌سازد؛ پایه‌ی تحلیل سایه، دید و تراکم ساختمانی.

خلاصه‌ی فصل

- هندسه‌ی نامعتبر (پاپیون، حفره‌ی بیرون‌زده، اجزای هم‌پوشان) نتیجه‌ی بی‌صدا و غلط می‌دهد؛ `ST_IsValid` و `ST_IsValidReason` آن را می‌یابند.
- `ST_MakeValid` (با روش linework یا structure) معتبر می‌کند؛ با `ST_CollectionExtract` و `ST_Multi` نوع را حفظ و نتیجه را بازبینی کنید.
- قواعد توپولوژی (هم‌پوشانی، شکاف، آویزان) با SQL قابل بررسی‌اند.
- `postgis_topology` مرزهای مشترک را یک بار ذخیره می‌کند (گره، یال، وجه) و سازگاری را تضمین می‌کند.
- توابع `3D` ارتفاع را در فاصله و طول حساب می‌کنند؛ SFCGAL اکسترود، حجم و سطح سه‌بعدی می‌دهد.

برای مدرس

زمان پیشنهادی: یک و نیم جلسه. یک Shapefile واقعی و قدیمی با هندسه‌ی نامعتبر به کلاس بیاورید و ابتدا با آن `ST_Union` بگیرید تا خطا ببینند؛ سپس بشمارند، علت را پیدا و اصلاح کنند. «مساحت صفر پاپیون» را حتماً نشان دهید.

تمرین‌ها

۱. سطح ۲ یک چندضلعی با حلقه‌ی خودتماس بسازید (`POLYGON((0 0, 4 0, 4 4, 2 0, 0 4, 0 0))`). علت نامعتبر بودن و نتیجه‌ی `ST_MakeValid` چیست؟

۲. سطح ۲ تعداد هندسه‌های نامعتبر همه‌ی جدول‌های چندضلعی داده‌ی نمونه را با یک پرسش `UNION ALL` گزارش کنید.

۳. سطح ۳ محله‌های نمونه را به یک توپولوژی تبدیل کنید. چون همپوشانی دارند (فصل ۱۷)، تعداد وجه‌ها چقدر می‌شود؟ چرا؟

۴. سطح ۳ ساختمانی L شکل با ارتفاع ۲۴ متر اکسترود و حجمش را حساب کنید.

فصل

۲۳

رستر در PostGIS

سطح: **پیشرفته** ●●● | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل‌های ۲ و ۱۸**

در این فصل یاد می‌گیرید

- ساختار نوع raster (کاشی، باند، نوع پیکسل، بی‌داده، مرجع مکانی) را بشناسید.
- رستر را با raster2pgsql وارد کنید و فراداده‌ی آن را بخوانید.
- مقدار پیکسل را در نقطه‌ها بخوانید و آمار ناحیه‌ای بگیرید.
- از مدل ارتفاعی، شیب، جهت و سایه‌روشن بسازید.
- با جبر نقشه و طبقه‌بندی دوباره رستر تازه تولید کنید و رستر را به برداری تبدیل کنید.
- بدانید کی رستر را در پایگاه داده نگه دارید و کی به صورت فایل.

سرفصل‌ها

۲۳-۱ رستر کنار برداری

۲۳-۲ ورود رستر

۲۳-۳ آمار و مقدار پیکسل

۲۳-۴ آمار ناحیه‌ای

۲۳-۵ تحلیل ارتفاع: شیب، جهت و سایه‌روشن

۲۳-۶ جبر نقشه و طبقه‌بندی دوباره

۲۳-۷ از رستر به برداری

۲۳-۸ خروجی رستر

۲۳-۹ رستر در پایگاه یا بیرون از آن؟

۲۳-۱ رستر کنار برداری

در فصل ۲ دیدیم که رستر شبکه‌ای از پیکسل‌هاست. افزونه‌ی `postgis_raster` نوع داده‌ی `raster` را اضافه می‌کند تا رستر هم مثل هندسه در جدول بنشیند و با SQL پرسیده شود؛ از همه مهم‌تر، **کنار داده‌ی برداری**: «میانگین ارتفاع هر قطعه»، «شیب محل هر چاه»، «درصد پوشش گیاهی هر محله».

مفهوم	در PostGIS
کاشی	رستر بزرگ معمولاً به **کاشی**‌های کوچک (مثلاً ۲۵۶×۲۵۶) شکسته و هر کاشی یک سطر جدول می‌شود
باند	هر کاشی یک یا چند باند دارد؛ `ST_NumBands(rast)`
نوع پیکسل	`8BUI` (صحیح بی‌علامت ۸ بیتی)، `16BSI` ، `32BF` (اعشاری ۳۲ بیتی)، …
بی‌داده	مقدار NoData هر باند؛ `ST_BandNoDataValue(rast, 1)`
مرجع مکانی	گوشه‌ی بالا-چپ، اندازه‌ی پیکسل، چرخش و SRID؛ `ST_MetaData(rast)`

نکته

در پایگاه داده مکانی ابری، رستر کجاست؟ تصویرهای ماهواره‌ای و DEM که بخش «تصویر ماهواره‌ای» تهیه می‌کند (بخش ششم کتاب) به صورت **فایل COG** نگه داشته می‌شوند، نه داخل پایگاه داده؛ و «آمار هر قطعه» آن‌ها را بدون نیاز به این افزونه محاسبه می‌کند. افزونه‌ی رستر وقتی لازم است که بخواهید رستر خودتان را داخل پایگاه بیاورید و با SQL تحلیل کنید. یادتان باشد رستر حجم زیادی از سهمیه را می‌گیرد؛ پیش از ورود، آن را به محدوده‌ی مطالعه برش دهید.

۲۳-۲ ورود رستر

ابزار `raster2pgsql` (همراه PostGIS) فایل رستر را به دستورهای SQL تبدیل می‌کند:

```
raster2pgsql -s 32639 -I -C -M -t 256x256 -N -9999 dem.tif public.dem |
psql "host=… sslmode=require"
```

گزینه	معنا
`-s 32639`	SRID رستر
`-I`	ساخت شاخص مکانی روی محدوده‌ی کاشی‌ها
`-C`	افزودن قیدهای رستر (همه‌ی کاشی‌ها هم‌SRID، هم‌اندازه‌ی پیکسل)
`-M`	اجرای VACUUM ANALYZE پس از ورود
`-t 256x256`	اندازه‌ی کاشی؛ کاشی کوچک برای پرسش‌های محلی سریع‌تر است
`-N -9999`	مقدار بی‌داده
`-R`	رستر **بیرون از پایگاه** (out-db): فقط مسیر فایل ذخیره می‌شود

برای این فصل، یک مدل ارتفاعی ساختگی (دو تپه روی یک دامنه‌ی ملایم) در غرب تهران با پیکسل ۳۰ متری می‌سازیم تا بدون فایل خارجی کار کنیم. دستور زیر را یک بار در کنسول با «اجازه تغییر داده» اجرا کنید؛ کدهای بعدی این فصل روی همین جدول اجرا شده‌اند:

```sql
CREATE TABLE dem AS
SELECT 1 AS rid,
       ST_MapAlgebra(
         ST_AddBand(ST_MakeEmptyRaster(200, 200, 530000, 3955000, 30,
-30, 0, 0, 32639), '32BF'::text, 0, -9999),
         1, '32BF',
         '1200 + 400 * exp(-((([rast.x] - 70)^2 + ([rast.y] - 90)^2) /
1800.0))
               + 150 * exp(-((([rast.x] - 150)^2 + ([rast.y] - 140)^2)
/ 900.0)) + 0.8 * [rast.y]'
       ) AS rast;

SELECT (ST_MetaData(rast)).* FROM dem;
```

upperlefty	width	height	scalex	scaley	skewx	skewy	srid	numbands
3955000.0	200	200	30.0	-30.0	0.0	0.0	32639	1

`ST_MakeEmptyRaster(سطر، ستون، x گوشه، y گوشه، اندازه‌ی پیکسل x، اندازه‌ی پیکسل y، چرخش، چرخش، SRID)` یک رستر خالی ۲۰۰×۲۰۰ می‌سازد. اندازه‌ی پیکسل y منفی است چون سطرها از بالا به پایین شماره می‌خورند. `ST_MapAlgebra` مقدار هر پیکسل را از یک عبارت حساب می‌کند؛ `[rast.x]` و `[rast.y]` شماره‌ی ستون و سطر پیکسل‌اند.

۲۳-۳ آمار و مقدار پیکسل

```sql
SELECT (stats).count, round((stats).mean::numeric, 1) AS mean_m,
       round((stats).min::numeric, 1) AS min_m,
round((stats).max::numeric, 1) AS max_m,
       round((stats).stddev::numeric, 1) AS std_m
FROM (SELECT ST_SummaryStats(rast) AS stats FROM dem) s;
```

count	mean_m	min_m	max_m	std_m
40000	1346.8	1200.8	1672.7	98.8

مقدار در نقطه: «ارتفاع محل هر مدرسه» با `ST_Value`. نقطه باید در SRID رستر باشد؛ پس آن را تبدیل می‌کنیم (نه رستر را؛ همان قاعده‌ی فصل ۲۰):

```sql
SELECT s.name, round(ST_Value(d.rast, ST_Transform(s.geom,
32639))::numeric, 1) AS elevation_m
FROM schools s
JOIN dem d ON ST_Intersects(d.rast, ST_Transform(s.geom, 32639))
ORDER BY elevation_m DESC
LIMIT 5;
```

name	elevation_m
مدرسه 137	1594.4
مدرسه 77	1547.0
مدرسه 5	1346.5
مدرسه 159	1297.4
مدرسه 50	1269.4

`ST_Intersects(rast, geom)` فقط مدرسه‌هایی را نگه می‌دارد که روی رستر افتاده‌اند (و با شاخص روی کاشی‌ها سریع است). اگر جدول ستون ارتفاع داشته باشد، با `UPDATE … SET elev = ST_Value(…)` پر می‌شود؛ یا نقطه را سه‌بعدی می‌کنید: `ST_MakePoint(ST_X(g), ST_Y(g), ST_Value(…))` (فصل ۲۲).

۲۳-۴ آمار ناحیه‌ای

آمار ناحیه‌ای (zonal statistics) آمار پیکسل‌های داخل هر چندضلعی است و پرکاربردترین تلفیق رستر و برداری: میانگین ارتفاع هر قطعه، میانگین NDVI هر مزرعه، بیشینه‌ی دمای هر محله. `ST_Clip` رستر را با چندضلعی می‌بُرد و `ST_SummaryStats` آمار را می‌گیرد:

```sql
SELECT n.name,
       (s).count                       AS pixels,
       round((s).mean::numeric, 1)     AS mean_elev_m,
       round((s).max::numeric, 1)      AS max_elev_m
FROM (
  SELECT n.name, ST_SummaryStats(ST_Clip(d.rast, ST_Transform(n.geom,
32639))) AS s
  FROM neighborhoods n
  JOIN dem d ON ST_Intersects(d.rast, ST_Transform(n.geom, 32639))
) n
ORDER BY mean_elev_m DESC;
```

name	pixels	mean_elev_m	max_elev_m
جمهوری	1188	1313.5	1360.4

فقط محله‌هایی آمدند که با محدوده‌ی DEM برخورد دارند. برای رستری که چند کاشی دارد، آمار هر کاشی جدا حساب می‌شود و باید با `ST_Union` کاشی‌ها را پیش از برش یکی کنید یا آمار را با وزن تعداد پیکسل تجمیع کنید. «آمار هر قطعه» در بخش تصویر ماهواره‌ای (فصل ۳۶) همین تحلیل را روی NDVI و برای سری زمانی انجام می‌دهد.

۲۳-۵ تحلیل ارتفاع: شیب، جهت و سایه‌روشن

از مدل رقومی ارتفاع (DEM) سه رستر مشتق پرکاربرد ساخته می‌شود:

- **شیب** (slope): تندی دامنه به درجه یا درصد؛ برای مکان‌یابی ساخت‌وساز، فرسایش و کشاورزی.
- **جهت** (aspect): سمتی که دامنه رو به آن است (۰ تا ۳۶۰، شمال = ۰)؛ برای تابش خورشید، کشاورزی و انرژی خورشیدی.
- **سایه‌روشن** (hillshade): تصویری که برجستگی را با نور فرضی از یک جهت نشان می‌دهد؛ زمینه‌ی نقشه‌های توپوگرافی.

```sql
WITH d AS (SELECT rast FROM dem)
SELECT 'شیب (درجه)' AS layer, round((ST_SummaryStats(ST_Slope(rast, 1,
'32BF', 'DEGREES'))).mean::numeric, 2) AS mean,
       round((ST_SummaryStats(ST_Slope(rast, 1, '32BF',
'DEGREES'))).max::numeric, 2) AS max FROM d
UNION ALL
SELECT 'جهت (درجه)', round((ST_SummaryStats(ST_Aspect(rast, 1, '32BF',
'DEGREES'))).mean::numeric, 1),
       round((ST_SummaryStats(ST_Aspect(rast, 1, '32BF',
'DEGREES'))).max::numeric, 1) FROM d
UNION ALL
SELECT 'سایه‌روشن (۰ تا ۲۵۵)', round((ST_SummaryStats(ST_HillShade(rast,
1, '32BF', 315, 45))).mean::numeric, 1),
       round((ST_SummaryStats(ST_HillShade(rast, 1, '32BF', 315,
45))).max::numeric, 1) FROM d;
```

layer	mean	max
شیب (درجه)	5.47	16.49
جهت (درجه)	158.2	360.0
سایه‌روشن (۰ تا ۲۵۵)	182.1	223.2

`ST_HillShade(rast, ارتفاع نور، آزیموت نور، نوع باند)` با نور از شمال‌غرب (۳۱۵ درجه) و زاویه‌ی ۴۵ درجه، قرارداد رایج کارتوگرافی است.

بدانید

شیب چگونه حساب می‌شود؟ برای هر پیکسل، از ارتفاع هشت همسایه‌اش، تغییر ارتفاع در جهت x و y ($\partial z/\partial x$ و $\partial z/\partial y$) با روش هورن تخمین زده می‌شود و شیب برابر است با $\arctan\sqrt{(\partial z/\partial x)^2 + (\partial z/\partial y)^2}$. برای همین DEM باید در سیستمی متری (مثل UTM) باشد: اگر اندازه‌ی پیکسل به درجه و ارتفاع به متر باشد، شیب بی‌معنا می‌شود.

۲۳-۶ جبر نقشه و طبقه‌بندی دوباره

جبر نقشه (map algebra) یعنی ساختن رستر تازه با عملیات ریاضی روی پیکسل‌ها؛ مثلاً NDVI از دو باند قرمز و فروسرخ (فصل ۳۵)، یا «مناطق مناسب ساخت‌وساز = شیب کمتر از ۱۰ درجه و

ارتفاع کمتر از ۱۴۰۰ متر». **طبقه‌بندی دوباره** (reclass) بازه‌های مقدار را به کلاس تبدیل می‌کند:

```sql
WITH s AS (SELECT ST_Slope(rast, 1, '32BF', 'DEGREES') AS slope FROM dem),
     c AS (SELECT ST_Reclass(slope, 1, '[0-5):1, [5-10):2, [10-90]:3', '8BUI', 0) AS cls FROM s)
SELECT (vc).value AS slope_class,
       CASE (vc).value WHEN 1 THEN 'هموار (۰ تا ۵)' WHEN 2 THEN 'ملایم (۵ تا ۱۰)' ELSE 'تند (۱۰ و بیشتر)' END AS label,
       (vc).count AS pixels,
       round((vc).count * 900 / 10000.0, 1) AS hectares
FROM (SELECT ST_ValueCount(cls) AS vc FROM c) x
ORDER BY 1;
```

slope_class	label	pixels	hectares
1.0	هموار (۰ تا ۵)	22739	2046.5
2.0	ملایم (۵ تا ۱۰)	9678	871.0
3.0	تند (۱۰ و بیشتر)	7583	682.5

`ST_ValueCount` تعداد پیکسل هر مقدار را می‌دهد و چون هر پیکسل ۳۰×۳۰ = ۹۰۰ متر مربع است، مساحت هر کلاس به دست می‌آید. با `ST_MapAlgebra` دوباندی هم می‌توان دو رستر را ترکیب کرد: `ST_MapAlgebra(slope, 1, dem, 1, 'CASE WHEN [rast1] < 10 AND [rast2] < 1400 THEN 1 ELSE 0 END')`.

۲۳-۷ از رستر به برداری

`ST_DumpAsPolygons` پیکسل‌های هم‌مقدار مجاور را به چندضلعی تبدیل می‌کند؛ کاربرد اصلی‌اش تبدیل نقشه‌ی طبقه‌بندی‌شده به لایه‌ی برداری است:

```sql
WITH c AS (SELECT ST_Reclass(ST_Slope(rast, 1, '32BF', 'DEGREES'), 1, '[0-5):1, [5-10):2, [10-90]:3', '8BUI', 0) AS cls FROM dem)
SELECT (p).val AS slope_class,
       count(*) AS polygons,
       round(sum(ST_Area((p).geom)) / 10000) AS hectares
FROM (SELECT ST_DumpAsPolygons(cls) AS p FROM c) x
GROUP BY 1 ORDER BY 1;
```

slope_class	polygons	hectares
1.0	3	2047.0
2.0	2	871.0
3.0	1	682.0

جهت مخالف، **برداری به رستر**، با `ST_AsRaster(geom, …)` انجام می‌شود؛ مثلاً برای تبدیل مرز مناطق حفاظت‌شده به ماسک رستری.

۲۳-۸ خروجی رستر

- `ST_AsPNG(ST_ColorMap(rast, 'elevation'))` تصویر رنگی PNG برای نمایش یا گزارش.
- `ST_AsGDALRaster(rast, 'GTiff')` فایل GeoTIFF (برای دانلود از برنامه).
- `gdal_translate` با `PG:"… table=dem"` رستر را مستقیم از پایگاه به هر قالب GDAL می‌برد.
- QGIS لایه‌های رستری PostGIS را مستقیم باز می‌کند (DB Manager یا Browser).

۲۳-۹ رستر در پایگاه یا بیرون از آن؟

	درون پایگاه (in-db)	بیرون پایگاه (out-db) یا فایل COG
پرسش و تلفیق با SQL	بسیار راحت	با `-R` ممکن، کندتر
حجم پایگاه و بکاپ	بزرگ	کوچک
مناسب برای	رسترهای کوچک و متوسط، نتیجه‌ی تحلیل، ماسک‌ها	تصویرهای بزرگ و سری زمانی ماهواره‌ای
در سامانه	افزونه‌ی رستر	بخش «تصویر ماهواره‌ای» و «رستر سازمانی» (فصل‌های ۳۴ و ۵۲)

در دنیای واقعی

مکان‌یابی پنل خورشیدی. شرکتی برای یافتن زمین مناسب نیروگاه خورشیدی در یک استان، سه معیار دارد: شیب کمتر از ۵ درجه، جهت جنوبی (بین ۱۳۵ و ۲۲۵ درجه) و فاصله‌ی کمتر از ۵ کیلومتر از خط انتقال برق. با DEM استان در پایگاه، شیب و جهت با `ST_Slope` و `ST_Aspect`، ترکیب با `ST_MapAlgebra`، تبدیل به چندضلعی با `ST_DumpAsPolygons` و فیلتر فاصله با `ST_DWithin` روی لایه‌ی خطوط برق، همه در یک اسکریپت SQL تکرارپذیر انجام شد.

خلاصه‌ی فصل

- نوع `raster` (افزونه‌ی postgis_raster) رستر را کاشی‌کاشی در جدول نگه می‌دارد؛ `ST_MetaData` مرجع مکانی را می‌دهد.
- `raster2pgsql -s -I -C -M -t -R` رستر را وارد می‌کند؛ آن را بیرون پایگاه نگه می‌دارد.
- `ST_Value` مقدار در نقطه، `ST_Clip` + `ST_SummaryStats` آمار ناحیه‌ای.
- `ST_Slope`، `ST_Aspect` و `ST_HillShade` تحلیل ارتفاع؛ DEM باید متری باشد.
- `ST_MapAlgebra` جبر نقشه، `ST_Reclass` طبقه‌بندی دوباره، `ST_ValueCount` مساحت کلاس‌ها، `ST_DumpAsPolygons` رستر به برداری.
- تصویرهای ماهواره‌ای سامانه فایل COGاند؛ افزونه‌ی رستر برای رسترهای خودتان در پایگاه.

برای مدرس

زمان پیشنهادی: دو جلسه. اگر DEM واقعی یک منطقه را دارید (مثلاً از بخش «تصویر ماهواره‌ای» با محصول «ارتفاع (DEM)» و ورود با raster2pgsql)، همین کدها را روی آن اجرا کنید؛ دیدن شیب و سایه‌روشن منطقه‌ی آشنا در QGIS بسیار جذاب است.

تمرین‌ها

۱. سطح ۲ ارتفاع محل همه‌ی مراکز درمانی روی DEM را بدهید. بلندترین کدام است؟

۲. سطح ۲ میانگین شیب داخل هر محله را با `ST_Clip` روی رستر شیب حساب کنید.

۳. سطح ۳ رستری بسازید که پیکسل‌های «شیب کمتر از ۵ درجه و جهت جنوبی» را ۱ و بقیه را صفر کند. مساحت مناسب چند هکتار است؟

۴. سطح ۳ DEM را با `ST_Reclass` به کلاس‌های ارتفاعی ۱۰۰ متری تقسیم و با `ST_DumpAsPolygons` به لایه‌ی برداری تبدیل کنید. این لایه شبیه چیست؟ (راهنمایی: منحنی میزان، فصل ۴۹)

فصل ۲۴

تحلیل شبکه و مسیریابی با pgRouting

سطح: **پیشرفته** ●●● | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل‌های ۱۳ و ۱۹**

در این فصل یاد می‌گیرید

- شبکه را به‌عنوان گراف (گره، یال، وزن، جهت) مدل کنید.
- جدول شبکه را برای pgRouting آماده کنید (source، target، cost، reverse_cost).
- کوتاه‌ترین و سریع‌ترین مسیر را با الگوریتم دایکسترا بیابید.
- نقطه‌های دلخواه را به شبکه بچسبانید.
- حوزه‌ی دسترسی زمانی، ماتریس هزینه و نزدیک‌ترین مرکز روی شبکه را حساب کنید.

سرفصل‌ها

۲۴-۱ از فاصله‌ی هوایی تا فاصله‌ی شبکه‌ای

۲۴-۲ شبکه به‌عنوان گراف

۲۴-۳ شبکه‌ی نمونه

۲۴-۴ کوتاه‌ترین (سریع‌ترین) مسیر

۲۴-۵ چسباندن نقطه به شبکه

۲۴-۶ حوزه‌ی دسترسی

۲۴-۷ ماتریس هزینه و نزدیک‌ترین مرکز روی شبکه

۲۴-۸ توابع دیگر pgRouting

۲۴-۱ از فاصله‌ی هوایی تا فاصله‌ی شبکه‌ای

در فصل ۱۹ هشدار دادیم که نزدیک‌ترین بیمارستان هوایی، شاید آن سوی بزرگراه باشد. حرکت واقعی روی **شبکه** انجام می‌شود: خیابان‌ها، لوله‌ها، کابل‌ها، رودها. **تحلیل شبکه** فاصله و زمان را روی همین شبکه حساب می‌کند و افزونه‌ی **pgRouting** آن را به PostGIS می‌آورد.

۲۴-۲ شبکه به‌عنوان گراف

در نظریه‌ی گراف، شبکه از دو جزء ساخته می‌شود:

- **گره** (node، vertex): تقاطع‌ها و انتهای خیابان‌ها.
- **یال** (edge): هر پاره‌خیابان بین دو گره.

هر یال یک **هزینه** (cost) دارد: طول به متر، زمان پیمودن به ثانیه، یا هر معیار دیگر (هزینه‌ی سوخت، خطر). یال ممکن است **جهت‌دار** باشد: خیابان یک‌طرفه در یک جهت هزینه دارد و در جهت دیگر ممنوع است. pgRouting برای هر یال چهار ستون می‌خواهد:

ستون	معنا
`id`	شناسه‌ی یال
`source` ، `target`	شناسه‌ی گره‌ی آغاز و پایان
`cost`	هزینه‌ی پیمودن از source به target؛ منفی یعنی ممنوع
`reverse_cost`	هزینه از target به source؛ منفی یعنی یک‌طرفه

بدانید

الگوریتم دایکسترا (۱۹۵۹) کوتاه‌ترین مسیر را در گرافی با وزن‌های نامنفی می‌یابد. از مبدأ شروع می‌کند و مثل موجی که روی آب پخش می‌شود، گره‌ها را به ترتیب فاصله‌شان از مبدأ «قطعی» می‌کند تا به مقصد برسد. **A*** همان است با یک «حدس» (معمولاً فاصله‌ی هوایی تا مقصد) که موج را به سمت مقصد هدایت می‌کند و در شبکه‌های بزرگ سریع‌تر است. هر دو در pgRouting هستند: `pgr_dijkstra` و `pgr_aStar` .

۳-۲۴ شبکه‌ی نمونه

داده‌ی نمونه شبکه‌ی معابر ندارد؛ در فصل ۳۰ شبکه‌ی واقعی معابر ایران را از OpenStreetMap وارد می‌کنید. برای این فصل یک **شبکه‌ی شطرنجی** ساختگی می‌سازیم: خیابان‌هایی با فاصله‌ی حدود یک کیلومتر، که هر پنجمی «اصلی» با سرعت ۶۰ و بقیه «محلی» با سرعت ۳۰ کیلومتر بر ساعت‌اند. این SQL را یک بار با «اجازه تغییر داده» اجرا کنید (و پیش از آن افزونه‌ی pgRouting را نصب کنید؛ فصل ۵):

```sql
CREATE TABLE roads AS
WITH h AS (   -- خیابان‌های شرقی-غربی
  SELECT i, j, ST_SetSRID(ST_MakeLine(ST_MakePoint(51.25 + 0.01 * i,
35.60 + 0.01 * j),
                                      ST_MakePoint(51.26 + 0.01 * i,
35.60 + 0.01 * j)), 4326) AS geom,
         i * 100 + j AS source, (i + 1) * 100 + j AS target, (j % 5 =
0) AS main
  FROM generate_series(0, 29) i, generate_series(0, 20) j
), v AS (     -- خیابان‌های شمالی-جنوبی
  SELECT i, j, ST_SetSRID(ST_MakeLine(ST_MakePoint(51.25 + 0.01 * i,
35.60 + 0.01 * j),
                                      ST_MakePoint(51.25 + 0.01 * i,
35.61 + 0.01 * j)), 4326),
         i * 100 + j, i * 100 + j + 1, (i % 5 = 0)
  FROM generate_series(0, 30) i, generate_series(0, 19) j
)
SELECT row_number() OVER ()::int AS id, source, target,
       CASE WHEN main THEN 'اصلی' ELSE 'محلی' END AS road_class,
       CASE WHEN main THEN 60 ELSE 30 END AS speed_kmh,
       ST_Length(geom::geography) AS length_m, geom
FROM (SELECT * FROM h UNION ALL SELECT * FROM v) x;

ALTER TABLE roads ADD COLUMN cost double precision, ADD COLUMN
reverse_cost double precision;
UPDATE roads SET cost = length_m / (speed_kmh / 3.6),           -- ثانیه
                 reverse_cost = length_m / (speed_kmh / 3.6);   --
دوطرفه

CREATE TABLE roads_vertices AS
SELECT DISTINCT ON (id) id, geom FROM (
  SELECT source AS id, ST_StartPoint(geom) AS geom FROM roads
  UNION ALL SELECT target, ST_EndPoint(geom) FROM roads) p;
CREATE INDEX ON roads USING gist (geom);
CREATE INDEX ON roads_vertices USING gist (geom);

SELECT road_class, count(*) AS edges, round(sum(length_m) / 1000) AS km
FROM roads GROUP BY road_class;
```

road_class	edges	km
اصلی	290	291.0
محلی	960	967.0

هزینه را **زمان به ثانیه** گذاشته‌ایم (طول تقسیم بر سرعت به متر بر ثانیه)؛ پس «کوتاه‌ترین» مسیر یعنی **سریع‌ترین**. اگر cost را `length_m` بگذارید، کوتاه‌ترین مسیر از نظر طول به دست می‌آید. جدول `roads_vertices` گره‌ها را با هندسه نگه می‌دارد تا نقطه‌ها را به شبکه بچسبانیم.

> **سطح حرفه‌ای**
>
> **توپولوژی شبکه‌ی واقعی.** در شبکه‌ی ساختگی ما شناسه‌ی گره‌ها از اول معلوم بود. در داده‌ی واقعی (مثلاً OSM)، خیابان‌ها خط‌هایی بلندند که باید در تقاطع‌ها **شکسته** شوند و برای هر گره شناسه ساخته شود. pgRouting برای این کار `pgr_extractVertices` (و در نسخه‌های قدیمی‌تر `pgr_createTopology` و `pgr_analyzeGraph`) یا `pgr_connectedComponents` گره‌های آویزان و بخش‌های جدا از شبکه را پیدا می‌کند (فصل ۲۲). داده‌ی آماده‌ی ایران در سامانه شبکه را آماده‌ی مسیریابی تحویل می‌دهد (فصل ۳۰).

۲۴-۴ کوتاه‌ترین (سریع‌ترین) مسیر

`pgr_dijkstra(پرسش یال‌ها، گره مبدأ، گره مقصد)` مسیر را به صورت فهرست یال‌ها، با هزینه‌ی هر قدم و هزینه‌ی تجمعی، برمی‌گرداند:

```sql
SELECT seq, node, edge, round(cost) AS step_s, round(agg_cost) AS
total_s
FROM pgr_dijkstra('SELECT id, source, target, cost, reverse_cost FROM
roads', 312, 2615);
```

seq	node	edge	step_s	total_s
1	312	364	109.0	0.0
2	412	365	109.0	109.0
3	512	1008	67.0	217.0
4	513	1039	67.0	284.0
5	514	1070	67.0	350.0
6	515	456	54.0	417.0
… (27 rows)				

نخستین آرگومان یک **متن SQL** است که یال‌ها را می‌دهد. این طراحی بسیار انعطاف‌پذیر است: می‌توانید شبکه را همان‌جا فیلتر کنید (مثلاً «فقط خیابان‌های اصلی» یا «بدون خیابان‌های مسدود امروز») بدون اینکه جدول را تغییر دهید. برای خلاصه و هندسه‌ی مسیر، نتیجه را با جدول یال‌ها پیوند می‌دهیم:

```sql
SELECT round((max(d.agg_cost) / 60)::numeric, 1)
AS minutes,
       round((sum(r.length_m) / 1000)::numeric, 2)
AS km,
       count(*)                                          AS edges,
       round(sum(r.length_m) FILTER (WHERE r.road_class = 'اصلی') * 100
/ sum(r.length_m)) AS main_road_pct,
       round((ST_Length(ST_LineMerge(ST_Collect(r.geom ORDER BY
d.seq))::geography) / 1000)::numeric, 2) AS route_line_km
FROM pgr_dijkstra('SELECT id, source, target, cost, reverse_cost FROM
roads', 312, 2615) d
JOIN roads r ON r.id = d.edge;
```

minutes	km	edges	main_road_pct	route_line_km
25.0	24.13	26	93.0	24.13

بیشتر مسیر روی خیابان‌های اصلی است، چون با وجود طول بیشتر، سریع‌ترند. `ST_LineMerge(ST_Collect(…))` یال‌های مسیر را به یک خط پیوسته تبدیل می‌کند که روی نقشه نشان داده می‌شود. اگر cost را طول بگذارید، مسیر کوتاه‌تر ولی کندتری می‌گیرید؛ تمرین ۲.

نکته

نام‌های ابهام‌آمیز. خروجی `pgr_dijkstra` ستون‌هایی به نام `cost` و `edge` دارد و جدول `roads` هم ستون `cost` دارد. در پیوند، همیشه نام جدول یا نام مستعار را بنویسید (`d.cost`، `r.length_m`)؛ وگرنه خطای «column reference "cost" is ambiguous» می‌گیرید.

۲۴-۵ چسباندن نقطه به شبکه

مبدأ و مقصد واقعی (یک مدرسه، یک خانه) معمولاً روی گره نیستند. ساده‌ترین روش، **نزدیک‌ترین گره** است (فصل ۱۹):

```sql
WITH a AS (SELECT v.id FROM roads_vertices v, schools s WHERE
s.school_id = 1  ORDER BY v.geom <-> s.geom LIMIT 1),
     b AS (SELECT v.id FROM roads_vertices v, schools s WHERE
s.school_id = 10 ORDER BY v.geom <-> s.geom LIMIT 1)
SELECT a.id AS from_node, b.id AS to_node,
       round((max(d.agg_cost) / 60)::numeric, 1) AS drive_minutes,
       (SELECT round((ST_Distance(s1.geom::geography,
s2.geom::geography) / 1000)::numeric, 2)
          FROM schools s1, schools s2 WHERE s1.school_id = 1 AND
s2.school_id = 10) AS air_km
FROM a, b, pgr_dijkstra('SELECT id, source, target, cost, reverse_cost
FROM roads', a.id, b.id) d
GROUP BY a.id, b.id;
```

from_node	to_node	drive_minutes	air_km
2806	301	31.1	23.69

برای دقت بیشتر (وقتی یال‌ها بلندند)، `pgr_withPoints` نقطه را روی **نزدیک‌ترین نقطه‌ی یال** (با `ST_LineLocatePoint`، فصل ۱۸) قرار می‌دهد، نه روی نزدیک‌ترین گره.

۲۴-۶ حوزه‌ی دسترسی

حوزه‌ی دسترسی (isochrone، service area) همه‌ی جاهایی است که از یک نقطه در زمان یا فاصله‌ی معین می‌توان رسید: «تا ۵ دقیقه رانندگی از ایستگاه آتش‌نشانی». `pgr_drivingDistance` همه‌ی گره‌هایی را که هزینه‌ی رسیدن به آن‌ها کمتر از حد است برمی‌گرداند و با پوسته‌ی مقعر (فصل ۱۸) به چندضلعی تبدیل می‌شود:

```sql
WITH reach AS (
  SELECT d.node, d.agg_cost
  FROM pgr_drivingDistance('SELECT id, source, target, cost,
reverse_cost FROM roads', 1510, 300) d   -- ۳۰۰ ثانیه = ۵ دقیقه
)
SELECT count(*)
AS reachable_nodes,
       round(max(agg_cost))
AS max_s,
       round((ST_Area(ST_ConcaveHull(ST_Collect(v.geom),
0.5)::geography) / 1e6)::numeric, 1)      AS isochrone_km2
FROM reach JOIN roads_vertices v ON v.id = reach.node;
```

reachable_nodes	max_s	isochrone_km2
35	296.0	28.1

با تکرار برای ۵، ۱۰ و ۱۵ دقیقه، حلقه‌های هم‌زمان دسترسی ساخته می‌شوند؛ ابزار «حوزه‌ی دسترسی» سامانه (فصل ۲۸) همین را با یک فرم و روی شبکه‌ی واقعی انجام می‌دهد.

۲۴-۷ ماتریس هزینه و نزدیک‌ترین مرکز روی شبکه

`pgr_dijkstraCost` هزینه‌ی همه‌ی جفت‌های چند مبدأ و چند مقصد را یک‌جا حساب می‌کند (**ماتریس مبدأ-مقصد**). با آن می‌توان «نزدیک‌ترین مرکز روی شبکه» را برای هر مبدأ یافت؛ همان ابزار «نزدیک‌ترین روی شبکه» سامانه:

```sql
WITH s AS (      -- هر مدرسه‌ی داخل محدوده‌ی شبکه، چسبیده به نزدیک‌ترین گره
  SELECT sc.gid, sc.name, (SELECT id FROM roads_vertices v ORDER BY
v.geom <-> sc.geom LIMIT 1) AS node
  FROM schools sc WHERE sc.geom && (SELECT ST_Extent(geom) FROM roads)
), h AS (        -- بیمارستان‌ها
  SELECT hc.name, (SELECT id FROM roads_vertices v ORDER BY v.geom <->
hc.geom LIMIT 1) AS node
  FROM health_centers hc WHERE hc.type = 'بیمارستان'
), m AS (
  SELECT * FROM pgr_dijkstraCost('SELECT id, source, target, cost,
reverse_cost FROM roads',
                                 ARRAY(SELECT DISTINCT node FROM s),
ARRAY(SELECT DISTINCT node FROM h))
)
SELECT DISTINCT ON (s.gid) s.name AS school, h.name AS
nearest_hospital, round((m.agg_cost / 60)::numeric, 1) AS minutes
FROM s JOIN m ON m.start_vid = s.node JOIN h ON h.node = m.end_vid
ORDER BY s.gid, m.agg_cost
LIMIT 5;
```

school	nearest_hospital	minutes
مدرسه 1	مرکز 7	7.7
مدرسه 2	مرکز 13	3.8
مدرسه 3	مرکز 7	1.8
مدرسه 5	مرکز 10	4.4
مدرسه 6	مرکز 34	4.9

`DISTINCT ON (s.gid) … ORDER BY s.gid, m.agg_cost` برای هر مدرسه فقط سطری با کمترین هزینه را نگه می‌دارد؛ الگوی پرکاربرد «کمینه‌ی هر گروه» در PostgreSQL.

۲۴-۸ توابع دیگر pgRouting

تابع	کار
`pgr_aStar`	کوتاه‌ترین مسیر با حدس فاصله؛ سریع‌تر در شبکه‌ی بزرگ
`pgr_bdDijkstra`	دایکسترای دوطرفه
`pgr_KSP`	k مسیر کوتاه‌تر (مسیرهای جایگزین)
`pgr_withPoints`	مسیریابی با نقطه‌های روی یال
`pgr_drivingDistance`	حوزه‌ی دسترسی
`pgr_dijkstraCost`، `pgr_dijkstraCostMatrix`	ماتریس هزینه
`pgr_TSP`	مسئله‌ی فروشنده‌ی دوره‌گرد: بهترین ترتیب بازدید از چند نقطه
`pgr_connectedComponents`	بخش‌های جدا از شبکه
`pgr_trsp`	مسیریابی با محدودیت گردش (مثلاً گردش به چپ ممنوع)

در دنیای واقعی

مسیر سرویس مدرسه. یک اداره‌ی آموزش و پرورش می‌خواهد بداند چند دانش‌آموز بیش از ۲۰ دقیقه با سرویس تا مدرسه فاصله دارند. با شبکه‌ی معابر OSM، نشانی دانش‌آموزان (نقطه) و مدرسه‌ها، یک `pgr_dijkstraCost` چندبه‌چند و پیوند با محل تحصیل هر دانش‌آموز، گزارش در چند ثانیه آماده شد و برای سه منطقه مسیر سرویس تازه با `pgr_TSP` طراحی شد.

خلاصه‌ی فصل

- شبکه = گراف: گره، یال، هزینه، جهت. pgRouting ستون‌های `id` ، `source` ، `target` ، `cost` و `reverse_cost` می‌خواهد؛ هزینه‌ی منفی یعنی ممنوع.
- هزینه‌ی زمانی (طول ÷ سرعت) مسیر سریع‌ترین و هزینه‌ی طولی مسیر کوتاه‌ترین را می‌دهد.
- `pgr_dijkstra` مسیر را یال به یال می‌دهد؛ پرسش یال‌ها متن SQL است و می‌توان شبکه را همان‌جا فیلتر کرد.
- نقطه را به نزدیک‌ترین گره (یا با `pgr_withPoints` به یال) بچسبانید.
- `pgr_drivingDistance` + پوسته‌ی مقعر = حوزه‌ی دسترسی؛ `pgr_dijkstraCost` + `DISTINCT ON` = نزدیک‌ترین مرکز روی شبکه.

برای مدرس

زمان پیشنهادی: دو جلسه. الگوریتم دایکسترا را روی یک گراف کوچک شش‌گرهی روی تخته اجرا کنید تا دانشجوها «موج» را ببینند. سپس مسیر را در QGIS نمایش دهید (پرسش مسیر در DB Manager ← Load as new layer) و هزینه را از زمان به طول عوض کنید تا تفاوت مسیر را ببینند.

تمرین‌ها

۱. سطح ۲ زمان سفر از گره ۵۰۵ به ۲۰۱۸ چقدر است؟ چند درصد مسیر روی خیابان‌های اصلی است؟

۲. سطح ۲ همان مسیر را با هزینه‌ی `length_m` بگیرید. طول و زمان دو مسیر را مقایسه کنید.

۳. سطح ۲ خیابان‌های افقی ردیف ۱۰ (`j = 10`) را یک‌طرفه‌ی شرق به غرب کنید (`reverse_cost = -1` یا عکس) و اثرش را روی یک مسیر ببینید.

۴. سطح ۳ حلقه‌های دسترسی ۳، ۶ و ۹ دقیقه‌ای از گره ۱۵۱۰ را بسازید و ذخیره کنید؛ در استودیوی نقشه با سه رنگ نمایش دهید.

۵. سطح ۳ با `pgr_TSP` بهترین ترتیب بازدید از پنج مدرسه‌ی داخل شبکه را پیدا کنید (راهنمایی: ماتریس هزینه با `pgr_dijkstraCostMatrix`).

بخش ۵

تحلیل مکانی بدون کد

ابزارهای آماده‌ی تحلیل همان SQL مکانی را پشت یک فرم ساده پنهان کرده‌اند. در این بخش هر ابزار را با پرسش واقعی، نظریه‌ی پشتش و SQL معادلش یاد می‌گیرید.

۲۵ ابزارهای تحلیل بدون کد؛ مجاورت و فاصله

۲۶ شمارش، تراکم و خوشه بدون کد

۲۷ همپوشانی و تغییر بدون کد

۲۸ تحلیل شبکه‌ی معابر بدون کد

۲۹ پرسش فارسی از داده

۳۰ داده‌ی آماده‌ی ایران از OpenStreetMap

فصل

۲۵

ابزارهای تحلیل بدون کد؛ مجاورت و فاصله

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۸ و ۱۸**

در این فصل یاد می‌گیرید

- صفحه‌ی ابزارها، فرم هر ابزار و چرخه‌ی اجرا و نتیجه را بشناسید.
- با ابزارهای حریم، نزدیک‌ترین عارضه، انتخاب بر اساس فاصله و حوزه‌ی نفوذ کار کنید.
- بدانید هر ابزار پشت صحنه چه SQLای اجرا می‌کند.
- نتیجه‌ی ابزار را روی نقشه ببینید و ورودی تحلیل بعدی کنید.

سرفصل‌ها

۲۵-۱ ابزارها: SQL مکانی پشت یک فرم

۲۵-۲ چرخه‌ی کار با هر ابزار

۲۵-۳ حریم (بافر)

۲۵-۴ نزدیک‌ترین عارضه

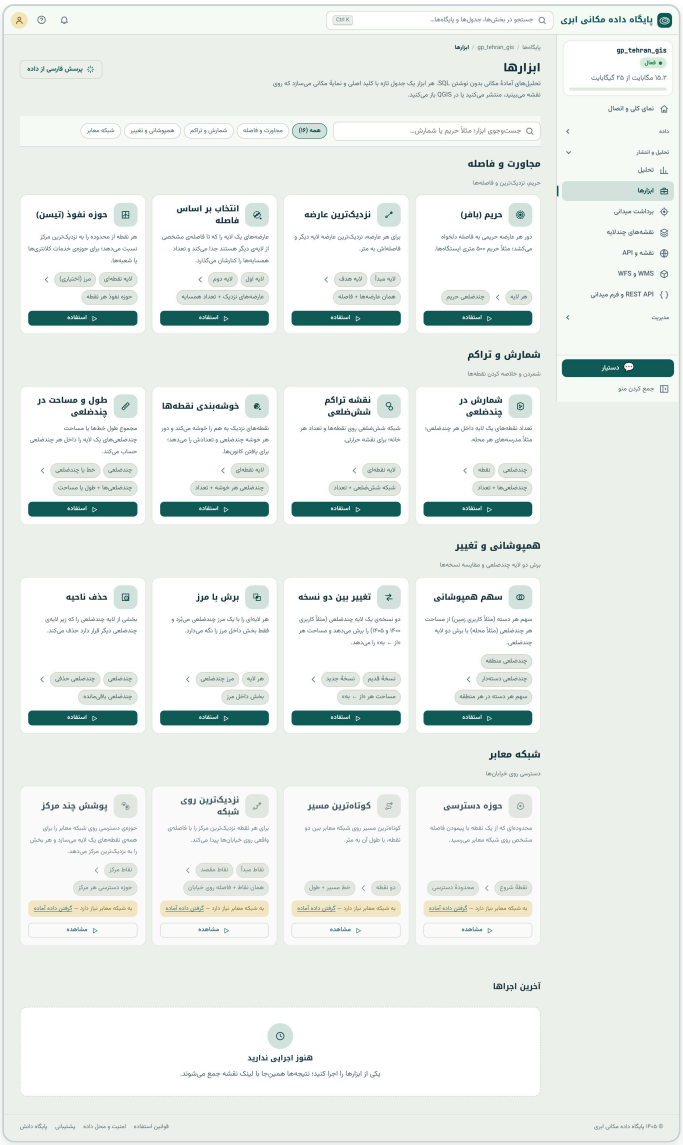
۲۵-۵ انتخاب بر اساس فاصله

۲۵-۶ حوزه‌ی نفوذ (تیسن)

۲۵-۷ پس از اجرا

۲۵-۱ ابزارها: SQL مکانی پشت یک فرم

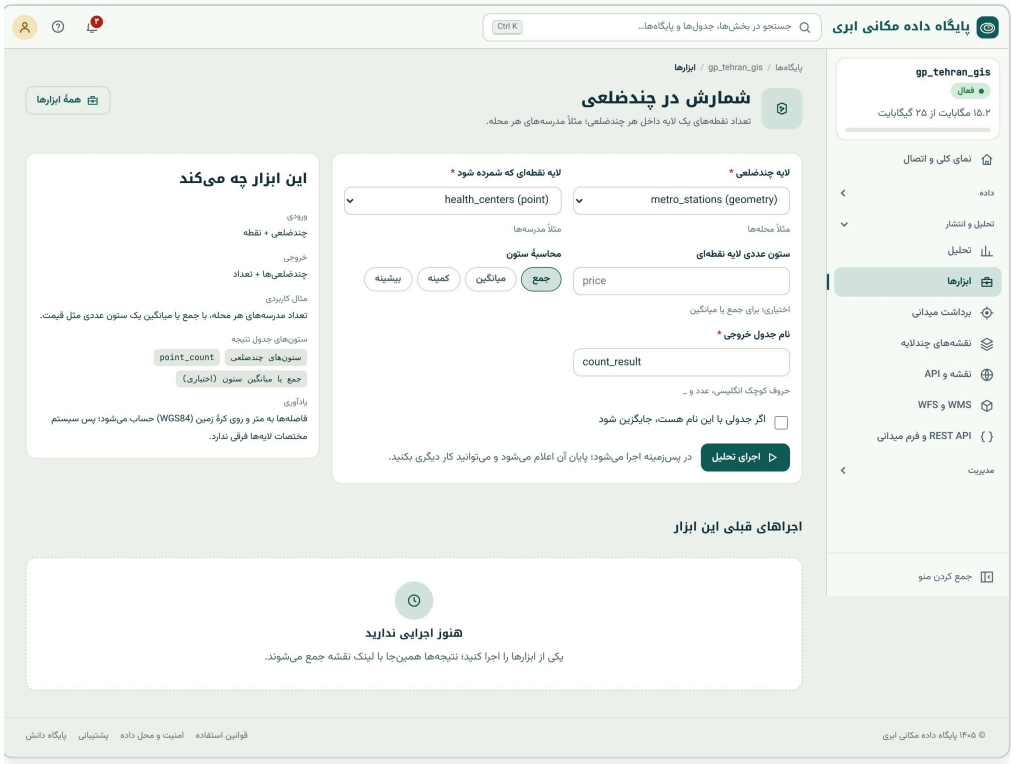
بخش‌های سوم و چهارم کتاب SQL مکانی را آموزش داد. اما همه‌ی کاربران SQL نمی‌نویسند و همه‌ی تحلیل‌ها هم ارزش نوشتن پرسش از صفر را ندارند. منوی کناری ← تحلیل و انتشار ← **«ابزارها»** شانزده تحلیل پرکاربرد را پشت فرم‌های ساده گذاشته است. هر ابزار یک **جدول تازه** با کلید اصلی و شاخص مکانی در پایگاه شما می‌سازد که روی نقشه دیده می‌شود، منتشر می‌شود، در QGIS باز می‌شود و ورودی تحلیل بعدی است.



صفحه‌ی ابزارها: شانزده ابزار در چهار گروه. هر کارت ورودی‌ها، خروجی و دکمه‌ی «استفاده» دارد. ابزارهای شبکه‌ی معابر به داده‌ی آماده‌ی ایران نیاز دارند.

گروه	ابزارها	فصل
مجاورت و فاصله	حریم، نزدیک‌ترین عارضه، انتخاب بر اساس فاصله، حوزه‌ی نفوذ (تیسن)	همین فصل
شمارش و تراکم	شمارش در چندضلعی، نقشه‌ی تراکم شش‌ضلعی، خوشه‌بندی نقطه‌ها، طول و مساحت در چندضلعی	۲۶
همپوشانی و تغییر	سهم همپوشانی، تغییر بین دو نسخه، برش با مرز، حذف ناحیه	۲۷
شبکه‌ی معابر	حوزه‌ی دسترسی، کوتاه‌ترین مسیر، نزدیک‌ترین روی شبکه، پوشش چند مرکز	۲۸

۲۵-۲ چرخه‌ی کار با هر ابزار



صفحه‌ی ابزار «شمارش در چندضلعی». فرم در وسط، توضیح ابزار (ورودی، خروجی، مثال، ستون‌های نتیجه، یادآوری) در کنار، و «اجراهای قبلی» پایین.

۱. روی کارت ابزار «استفاده» را بزنید.

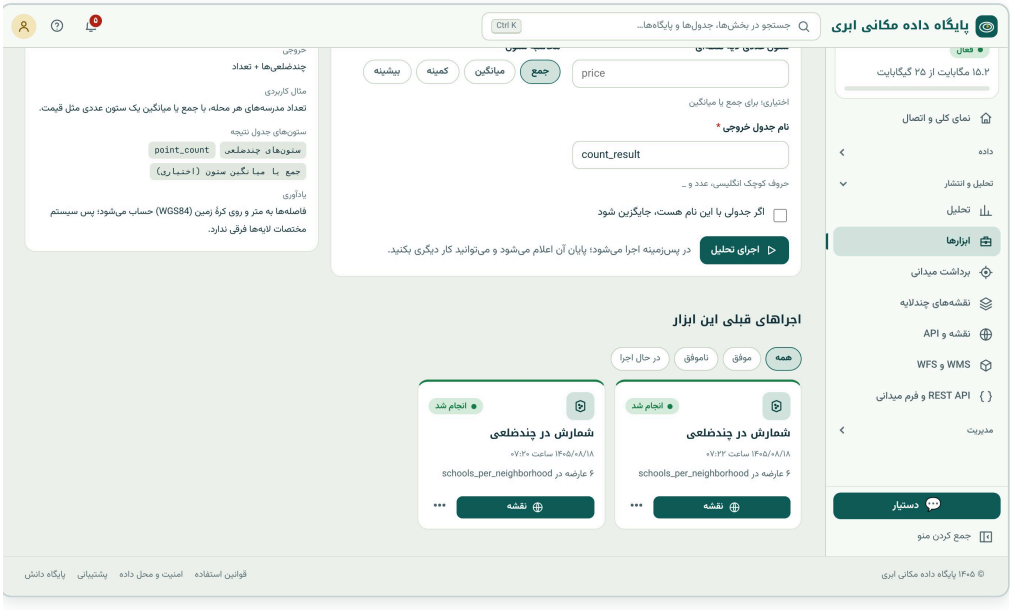
۲. **لایه‌های ورودی** را انتخاب کنید. فهرست فقط لایه‌هایی را نشان می‌دهد که نوع هندسه‌شان به کار ابزار می‌آید (مثلاً فقط چندضلعی برای «لایه‌ی مناطق»).

۳. **پارامترها** را تنظیم کنید: فاصله به متر، ستون نام، نوع محاسبه (جمع، میانگین، …) با چیپ‌ها.

۴. **نام جدول خروجی** را با حروف کوچک انگلیسی، عدد و زیرخط بنویسید. اگر جدولی با این نام هست، «جایگزین شود» را تیک بزنید یا نام دیگری بدهید.

۵. «اجرای تحلیل». تحلیل در **پس‌زمینه** اجرا می‌شود؛ پایان کار با اعلان خبر داده می‌شود و می‌توانید در این فاصله کار دیگری بکنید.

۶ نتیجه در «اجراهای قبلی» همان ابزار و در «آخرین اجراها»ی صفحه‌ی ابزارها می‌آید؛ دکمه‌ی **«نقشه»** آن را روی نقشه باز می‌کند و منوی ⋯ کارهای دیگر را دارد.



اجرای ابزار «شمارش در چندضلعی» تمام شده است: ۶ عارضه در جدول schools_per_neighborhood. دکمه‌ی «نقشه» نتیجه را باز می‌کند.

ستون‌های جدول نتیجه را کارت «این ابزار چه می‌کند» پیش از اجرا می‌گوید. مثلاً خروجی همین تحلیل ستون‌های محله را دارد به‌علاوه‌ی `point_count` و `avg_students`؛ با همان عددهایی که در فصل ۱۳ با پیوند مکانی به دست آوردیم (نارمک ۷، شهرک غرب ۷، ونک ۶، …). حالا می‌دانید ابزار و SQL به یک جواب می‌رسند.

نکته

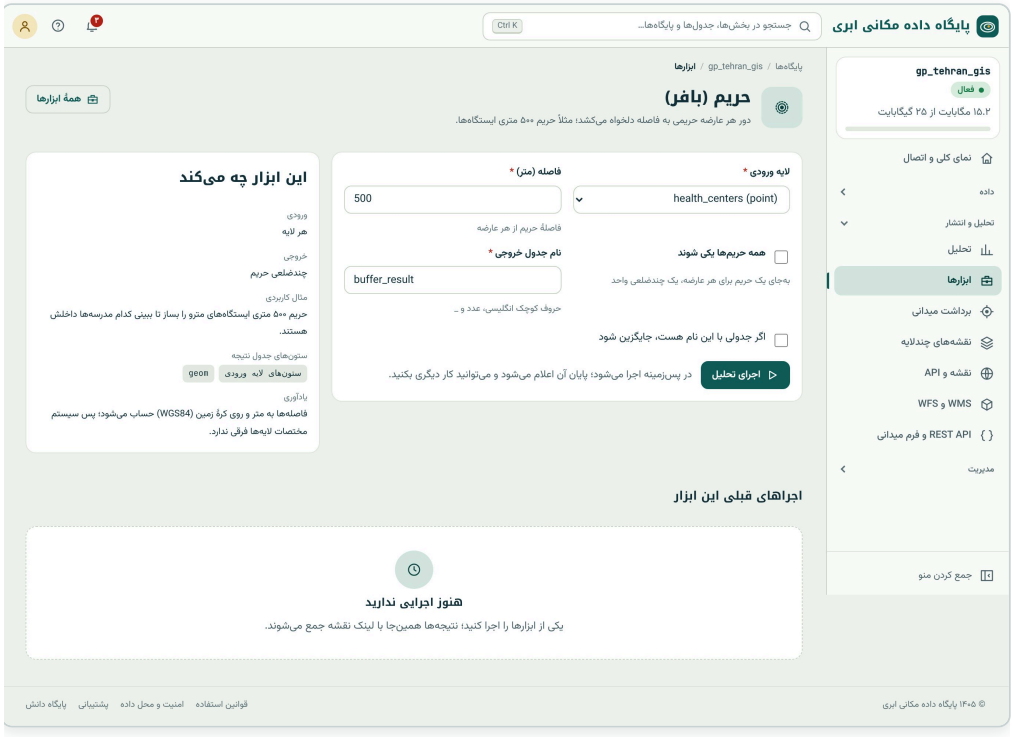
همه‌ی فاصله‌ها به متر و روی کره‌ی زمین. ابزارها فاصله را با `geography` (فصل‌های ۳ و ۱۶) حساب می‌کنند؛ پس سیستم مختصات لایه‌ها مهم نیست و لازم نیست چیزی را به UTM ببرید. اگر لایه‌ها SRID متفاوت دارند، ابزار خودش تبدیل را انجام می‌دهد. خروجی همیشه در ۴۳۲۶ ذخیره می‌شود.

۳-۲۵ حریم (بافر)

پرسش: «محدوده‌ی ۵۰۰ متری ایستگاه‌های مترو کجاست؟»

ورودی: هر لایه (نقطه، خط یا چندضلعی). **پارامترها:** فاصله به متر و «همه‌ی حریم‌ها یکی شوند».

خروجی: برای هر عارضه یک چندضلعی حریم با ستون‌های عارضه‌ی اصلی؛ یا اگر «یکی شوند» تیک بخورد، یک چندضلعی واحد (MultiPolygon) که همپوشانی‌ها در آن ادغام شده‌اند.



فرم ابزار «حریم».

زیر کاپوت (فصل ۱۸):

```sql
-- حریم جداگانه
SELECT <ستون‌ها>, ST_Buffer(geom::geography, 500)::geometry(Polygon,
4326) AS geom FROM metro_stations;
-- حریم یکپارچه
SELECT ST_Multi(ST_Union(ST_Buffer(geom::geography,
500)::geometry))::geometry(MultiPolygon, 4326) AS geom FROM
metro_stations;
```

چه وقت «یکی شوند»؟ وقتی پرسش درباره‌ی **مساحت پوشش** است («چند درصد شهر تا ۵۰۰ متری مترو است؟»): بدون ادغام، بخش‌های همپوشان چند بار شمرده می‌شوند (فصل ۱۸). وقتی پرسش درباره‌ی **هر عارضه** است («حریم کدام ایستگاه با کدام مدرسه برخورد دارد؟»)، حریم جداگانه.

۲۵-۴ نزدیک‌ترین عارضه

پرسش: «برای هر مدرسه، نزدیک‌ترین پارک و فاصله‌اش چقدر است؟»

ورودی: لایه‌ی مبدأ و لایه‌ی هدف. **پارامتر:** ستون نام در لایه‌ی هدف (اختیاری).

خروجی: همان عارضه‌های مبدأ، به‌علاوه‌ی `distance_m` (فاصله به متر) و نام نزدیک‌ترین عارضه‌ی هدف.

زیر کاپوت: الگوی `LATERAL` با `<->` (فصل ۱۹)، با فاصله‌ی دقیق روی geography:

```sql
SELECT s.*, n.name AS nearest_name,
round(ST_Distance(s.geom::geography, n.geom::geography)::numeric, 1) AS
distance_m
FROM schools s
CROSS JOIN LATERAL (SELECT name, geom FROM parks p ORDER BY p.geom ⟷
s.geom LIMIT 1) n;
```

نتیجه فاصله‌ی **هوایی** است. برای فاصله‌ی واقعی روی خیابان، ابزار «نزدیک‌ترین روی شبکه» (فصل ۲۸).

۲۵-۵ انتخاب بر اساس فاصله

پرسش: «کدام مدرسه‌ها تا ۷۰۰ متری یک ایستگاه مترو هستند و چند ایستگاه در این فاصله دارند؟»

ورودی: لایه‌ی اول (عارضه‌هایی که انتخاب می‌شوند) و لایه‌ی دوم. **پارامتر:** فاصله به متر.

خروجی: فقط عارضه‌هایی از لایه‌ی اول که دست‌کم یک عارضه‌ی لایه‌ی دوم در آن فاصله دارند، به‌علاوه‌ی ستون **تعداد همسایه**.

این همان پرسش «یافتن عارضه» در فصل ۸ است، با این تفاوت که نتیجه مستقیم جدول می‌شود و تعداد همسایه هم دارد. **زیر کاپوت:** `ST_DWithin` روی geography با پیش‌فیلتر شاخص (فصل‌های ۱۷ و ۲۰):

```sql
SELECT s.*, c.neighbors
FROM schools s
CROSS JOIN LATERAL (SELECT count(*) AS neighbors FROM metro_stations m
                    WHERE ST_DWithin(s.geom::geography,
m.geom::geography, 700)) c
WHERE c.neighbors > 0;
```

۲۵-۶ حوزه‌ی نفوذ (تیسن)

پرسش: «حوزه‌ی خدمات هر بیمارستان، به شرط اینکه هر نقطه به نزدیک‌ترین بیمارستان مراجعه کند، کجاست؟»

ورودی: لایه‌ی نقطه‌ای مراکز. **پارامتر:** لایه‌ی مرز (اختیاری) برای برش حوزه‌ها.

خروجی: یک چندضلعی برای هر مرکز، با ستون‌های آن.

زیر کاپوت: `ST_VoronoiPolygons` و برش با مرز (فصل ۱۸). بدون لایه‌ی مرز، حوزه‌های لبه تا محدوده‌ای بسیار بزرگ امتداد می‌یابند؛ همیشه یک مرز (مثلاً محدوده‌ی شهر) بدهید.

بدانید

حوزه‌ی نفوذ و فرض‌هایش. تیسن فرض می‌کند مردم به **نزدیک‌ترین** مرکز (به خط مستقیم) می‌روند و همه‌ی مراکز یکسان‌اند. در واقعیت، شبکه‌ی خیابان، ظرفیت و کیفیت مراکز و مرزهای اداری اثر دارند. تیسن پاسخ اولیه‌ی خوبی است؛ برای برنامه‌ریزی دقیق، «پوشش چند مرکز» روی شبکه (فصل ۲۸) و مدل‌های جاذبه را به کار ببرید. تیسن در اقلیم‌شناسی هم کاربرد کلاسیک دارد: برآورد بارش منطقه‌ای از ایستگاه‌ها با وزن مساحت حوزه‌ی هر ایستگاه.

۲۵-۷ پس از اجرا

- **نقشه:** دکمه‌ی «نقشه» روی کارت اجرا، نتیجه را باز می‌کند.
- **کنار ورودی‌ها:** نتیجه را روی لایه‌های ورودی در «نمایش با هم» باز می‌کند تا رابطه‌شان را ببینید.
- **نقشه و نمودار:** برای ابزارهایی که خروجی‌شان آمار دارد (سهم همپوشانی، شمارش با جمع یا میانگین)، نمودار پیشنهادی را در نقشه‌ی چندلایه می‌سازد (فصل ۴۳).
- **ورودی تحلیل بعدی:** خروجی یک جدول معمولی است؛ مثلاً حریم یکپارچه‌ی مترو را ورودی «برش با مرز» کنید تا مدرسه‌های داخلش جدا شوند.
- **SQL:** می‌توانید جدول نتیجه را در کنسول SQL بپرسید یا با بقیه‌ی جدول‌ها پیوند دهید.

خودتان امتحان کنید

با ابزار «انتخاب بر اساس فاصله»، مدرسه‌های تا ۷۰۰ متری مترو را جدا کنید و نتیجه را با پرسش SQL فصل ۸ مقایسه کنید (۲۷ مدرسه). سپس با «نزدیک‌ترین عارضه» برای هر مدرسه نزدیک‌ترین مرکز درمانی را بیابید و با کنسول SQL، میانگین `distance_m` را برای هر مقطع حساب کنید.

خلاصه‌ی فصل

- «ابزارها» شانزده تحلیل را پشت فرم دارد؛ هر ابزار جدول تازه با کلید و شاخص مکانی می‌سازد.
- چرخه: لایه‌ها، پارامترها، نام خروجی، اجرا در پس‌زمینه، اعلان، نتیجه در «اجراهای قبلی» با دکمه‌ی نقشه.
- فاصله‌ها به متر روی geography؛ خروجی در ۴۳۲۶.
- حریم (جداگانه یا یکپارچه)، نزدیک‌ترین عارضه (فاصله‌ی هوایی)، انتخاب بر اساس فاصله (با تعداد همسایه)، حوزه‌ی نفوذ (با برش مرز).
- هر ابزار یک الگوی SQL فصل‌های ۱۷ تا ۱۹ است؛ نتیجه‌ی ابزار و SQL یکی است.

برای مدرس

زمان پیشنهادی: یک جلسه. برای کلاس‌های غیرفنی، این فصل می‌تواند جای بخش‌های سوم و چهارم بنشیند. برای کلاس‌های فنی، بگذارید دانشجوها نتیجه‌ی هر ابزار را با SQL خودشان بازتولید کنند؛ بهترین تمرین برای دیدن «زیر کاپوت».

تمرین‌ها

۱. سطح ۱ حریم ۳۰۰ متری پارک‌ها را جداگانه و یکپارچه بسازید. مساحت کل دو نتیجه را مقایسه کنید.

۲. سطح ۱ برای هر ایستگاه مترو نزدیک‌ترین مدرسه را بیابید. کدام ایستگاه دورترین مدرسه را دارد؟

۳. سطح ۲ حوزه‌ی نفوذ بیمارستان‌ها را با و بدون لایه‌ی مرز (مثلاً پوسته‌ی محدب مدرسه‌ها که با SQL ساخته‌اید) بسازید و تفاوت را توضیح دهید.

۴. سطح ۲ نتیجه‌ی «انتخاب بر اساس فاصله» را ورودی «شمارش در چندضلعی» کنید: در هر محله چند مدرسه‌ی نزدیک مترو هست؟

فصل

۲۶

شمارش، تراکم و خوشه بدون کد

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل ۲۵**

در این فصل یاد می‌گیرید

- با «شمارش در چندضلعی» تعداد و جمع یا میانگین نقطه‌ها را برای هر منطقه بگیرید.
- با «نقشه‌ی تراکم شش‌ضلعی» نقشه‌ی حرارتی بسازید و اندازه‌ی خانه را آگاهانه انتخاب کنید.
- با «خوشه‌بندی نقطه‌ها» کانون‌ها را پیدا کنید.
- با «طول و مساحت در چندضلعی» طول شبکه یا مساحت پوشش هر منطقه را حساب کنید.

سرفصل‌ها

۲۶-۱ شمارش در چندضلعی

۲۶-۲ نقشه‌ی تراکم شش‌ضلعی

۲۶-۳ خوشه‌بندی نقطه‌ها

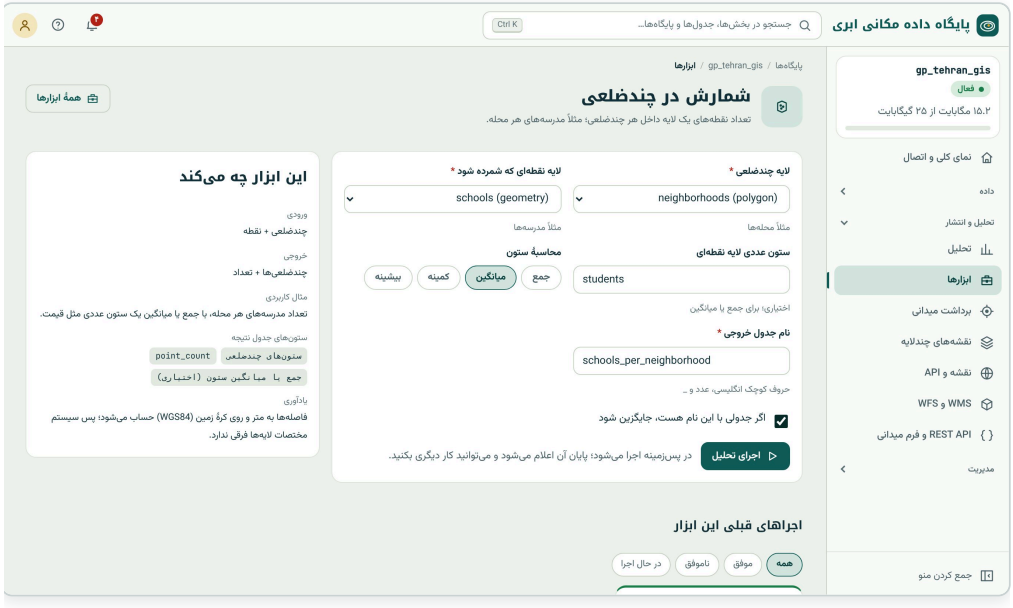
۲۶-۴ طول و مساحت در چندضلعی

۲۶-۱ شمارش در چندضلعی

پرسش: «هر محله چند مدرسه دارد و میانگین دانش‌آموز مدرسه‌هایش چقدر است؟»

ورودی: لایه‌ی چندضلعی (مناطق) و لایه‌ی نقطه‌ای (آنچه شمرده می‌شود). **پارامتر اختیاری:** ستون عددی لایه‌ی نقطه‌ای و نوع محاسبه (جمع، میانگین، کمینه، بیشینه).

خروجی: همه‌ی چندضلعی‌ها (حتی خالی‌ها با صفر) با ستون‌های خودشان، به‌علاوه‌ی `point_count` و در صورت انتخاب، ستونی مثل `avg_students` .



فرم «شمارش در چندضلعی» پر شده: محله‌ها، مدرسه‌ها، ستون students و محاسبه‌ی میانگین.

زیر کاپوت: زیرپرسش همبسته‌ی شمارش با `ST_Intersects` برای هر چندضلعی (فصل‌های ۱۳ و ۱۷). نقطه‌ی روی مرز مشترک دو منطقه در هر دو شمرده می‌شود (فصل ۱۷).

هشدار

نسبت، نه عدد خام. «محله‌ی الف ۷ مدرسه دارد و محله‌ی ب ۲» به تنهایی معنایی ندارد اگر جمعیت یا مساحتشان فرق کند. پس از اجرای ابزار، در کنسول SQL نسبت بسازید: `SELECT name, point_count, round(population::numeric / nullif(point_count, 0)) AS people_per_school FROM نتیجه;` و برای نقشه، نسبت را نمایش دهید (فصل ۴۱).

۲۶-۲ نقشه‌ی تراکم شش‌ضلعی

پرسش: «تصادف‌ها یا مدرسه‌ها کجا متراکم‌ترند؟»

ورودی: لایه‌ی نقطه‌ای. **پارامتر:** اندازه‌ی هر خانه.

خروجی: شش‌ضلعی‌هایی که دست‌کم یک نقطه دارند، با ستون `count`. خانه‌های خالی در نتیجه نیستند.

زیر کاپوت: `ST_HexagonGrid` روی محدوده‌ی نقطه‌ها و شمارش با پیوند مکانی (فصل ۲۱). شبکه در سیستم Web Mercator (۳۸۵۷) ساخته می‌شود.

سطح حرفه‌ای

اندازه‌ی واقعی خانه. چون شبکه در Web Mercator ساخته می‌شود و این سیستم در عرض‌های جغرافیایی میانه طول‌ها را بزرگ‌تر نشان می‌دهد (فصل ۱۶)، اندازه‌ای که در فرم می‌نویسید در تهران حدود ۸۱ درصد اندازه‌ی واقعی روی زمین است (ضریب $\cos 35.7°$): خانه‌ی «۵۰۰ متری» در واقع حدود ۴۰۵ متر است. برای مقایسه‌ی دو شهر با عرض متفاوت یا گزارش رسمی، شبکه را با SQL فصل ۲۱ در UTM بسازید تا اندازه دقیقاً متری باشد.

انتخاب اندازه‌ی خانه همان مسئله‌ی MAUP است (فصل ۲۱): خانه‌ی کوچک نویز و خانه‌های خالی زیاد می‌دهد، خانه‌ی بزرگ جزئیات را محو می‌کند. قاعده‌ی سرانگشتی: اندازه‌ای که بیشتر خانه‌های پر، بیش از چند نقطه داشته باشند؛ و همیشه دو یا سه اندازه را امتحان کنید.

۲۶-۳ خوشه‌بندی نقطه‌ها

پرسش: «کانون‌های شکایت شهروندان کجاست؟»

ورودی: لایه‌ی نقطه‌ای. **پارامترها:** شعاع همسایگی (متر) و حداقل نقطه در هر خوشه.

خروجی: یک چندضلعی (پوسته‌ی محدب با حاشیه‌ی کم) برای هر خوشه، با `cluster_id` و `point_count`. نقطه‌های پراکنده در هیچ خوشه‌ای نیستند.

زیر کاپوت: `ST_ClusterDBSCAN` (فصل ۲۱)؛ شعاع همسایگی برای عرض جغرافیایی تصحیح می‌شود تا واقعاً متری باشد. تنظیم پارامترها:

- شعاع کم و حداقل زیاد: فقط کانون‌های بسیار فشرده.
- شعاع زیاد و حداقل کم: خوشه‌های بزرگ و پراکنده که ممکن است به هم بپیوندند.

۲۶-۴ طول و مساحت در چندضلعی

پرسش: «هر محله چند کیلومتر خیابان دارد؟» یا «چند درصد هر محله پارک است؟»

ورودی: لایه‌ی مناطق (چندضلعی) و لایه‌ی خطی یا چندضلعی.

خروجی: مناطق با `length_m` و تراکم (کیلومتر در کیلومتر مربع) برای خط‌ها، یا `area_m2` و درصد پوشش برای چندضلعی‌ها.

زیر کاپوت: `ST_Intersection` هر عارضه با منطقه و جمع طول یا ادغام و مساحت روی geography (فصل‌های ۱۶ و ۱۸). برای چندضلعی‌ها اجتماع گرفته می‌شود تا همپوشانی‌ها دو بار شمرده نشوند.

> **در دنیای واقعی**
>
> **سرانه‌ی فضای سبز.** معاونت شهرسازی یک شهر با «طول و مساحت در چندضلعی» (لایه‌ی محله‌ها و لایه‌ی پارک‌ها) درصد فضای سبز هر محله را گرفت و با یک پرسش SQL آن را به سرانه‌ی متر مربع برای هر نفر تبدیل کرد. نقشه‌ی سرانه، که در استودیوی نقشه با پنج طبقه نمایش داده شد، مبنای اولویت‌بندی بودجه‌ی پارک‌سازی محله‌ای قرار گرفت.

خلاصه‌ی فصل

- «شمارش در چندضلعی» تعداد و جمع/میانگین/کمینه/بیشینه‌ی ستون نقطه‌ها را برای همه‌ی مناطق می‌دهد؛ برای نقشه، نسبت بسازید.
- «نقشه‌ی تراکم شش‌ضلعی» شبکه را در Web Mercator می‌سازد؛ اندازه در تهران حدود ۸۱٪ اندازه‌ی واقعی است؛ برای دقت متری، SQL فصل ۲۱.
- «خوشه‌بندی نقطه‌ها» DBSCAN با شعاع متری و حداقل نقطه؛ پوسته‌ی هر خوشه را می‌دهد.
- «طول و مساحت در چندضلعی» طول خط و مساحت/درصد پوشش هر منطقه را می‌دهد.

تمرین‌ها

۱. سطح ۱ با «شمارش در چندضلعی» تعداد مراکز درمانی و مجموع تخت هر محله را بگیرید.

۲. سطح ۱ نقشه‌ی تراکم مدرسه‌ها را با اندازه‌ی ۵۰۰ و ۱۵۰۰ بسازید. کدام برای نقشه‌ی کل شهر مناسب‌تر است؟

۳. سطح ۲ مدرسه‌ها را با شعاع ۸۰۰ متر و حداقل ۴ نقطه خوشه‌بندی کنید و نتیجه را با پرسش DBSCAN فصل ۲۱ مقایسه کنید.

۴. سطح ۲ درصد پارک هر محله را بگیرید و با SQL به سرانه (متر مربع برای هر نفر) تبدیل کنید.

فصل

۲۷

همپوشانی و تغییر بدون کد

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل ۲۵**

در این فصل یاد می‌گیرید

- با «سهم همپوشانی» سهم هر دسته از مساحت هر منطقه را بگیرید.
- با «تغییر بین دو نسخه» ماتریس «از ← به» دو لایه‌ی دسته‌دار را بسازید.
- با «برش با مرز» و «حذف ناحیه» لایه‌ها را به محدوده‌ی مطالعه محدود کنید.
- نتیجه را به نمودار و نقشه تبدیل کنید.

سرفصل‌ها

۲۷-۱ همپوشانی دو لایه‌ی چندضلعی

۲۷-۲ سهم همپوشانی

۲۷-۳ تغییر بین دو نسخه

۲۷-۴ برش با مرز

۲۷-۵ حذف ناحیه

۲۷-۱ همپوشانی دو لایه‌ی چندضلعی

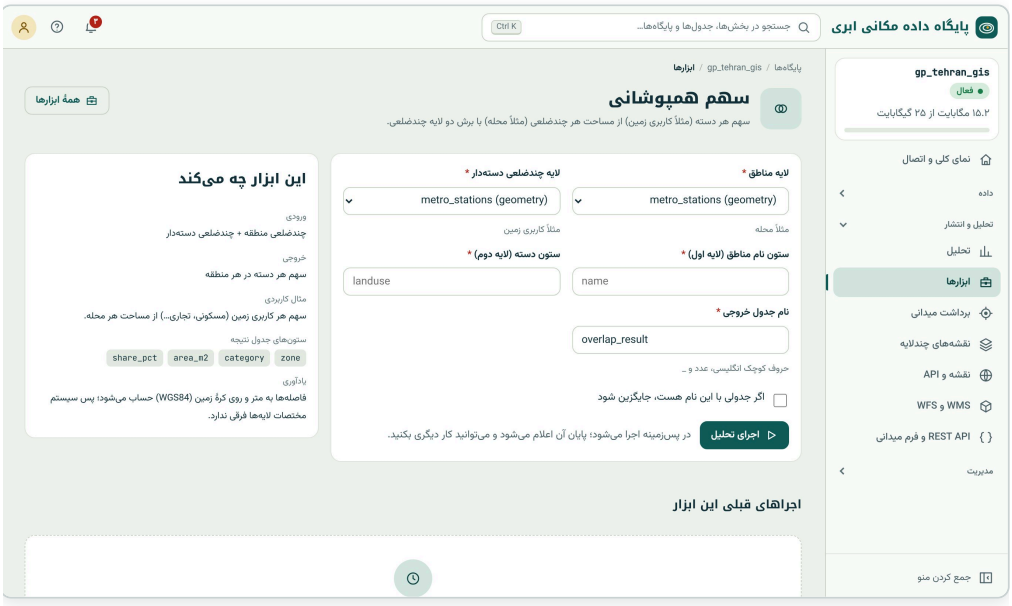
بسیاری از پرسش‌های برنامه‌ریزی، پرسش **همپوشانی** دو لایه است: کاربری زمین در محله‌ها، پوشش جنگل در حوضه‌ها، طرح تفصیلی در محدوده‌ی یک پروژه. چهار ابزار این گروه همه از اشتراک یا تفاضل هندسه‌ها (فصل ۱۸) ساخته شده‌اند و پیش از محاسبه هندسه‌ها را معتبر می‌کنند (`ST_MakeValid`، فصل ۲۲) تا داده‌ی ناقص خطا ندهد.

۲۷-۲ سهم همپوشانی

پرسش: «سهم هر کاربری (مسکونی، تجاری، سبز، ...) از مساحت هر محله چقدر است؟»

ورودی: لایه‌ی مناطق (مثلاً محله) و لایه‌ی چندضلعی دسته‌دار (مثلاً کاربری زمین با ستون نوع).
پارامترها: ستون نام منطقه و ستون دسته.

خروجی: یک سطر برای هر جفت منطقه و دسته: `zone`، `category`، `area_m2`، `share_pct` و هندسه‌ی بخش مشترک.



فرم ابزار «سهم همپوشانی».

زیر کاپوت:

```sql
WITH j AS (
  SELECT p.name AS zone, q.landuse AS category,
         ST_Area(ST_MakeValid(p.geom)::geography) AS zone_area,
         ST_CollectionExtract(ST_Intersection(ST_MakeValid(p.geom),
ST_MakeValid(q.geom)), 3) AS g
  FROM neighborhoods p JOIN landuse q ON ST_Intersects(p.geom, q.geom)
)
SELECT zone, category,
       round(sum(ST_Area(g::geography))::numeric, 1)
AS area_m2,
       round((100 * sum(ST_Area(g::geography)) /
max(zone_area))::numeric, 2)     AS share_pct
FROM j GROUP BY zone, category;
```

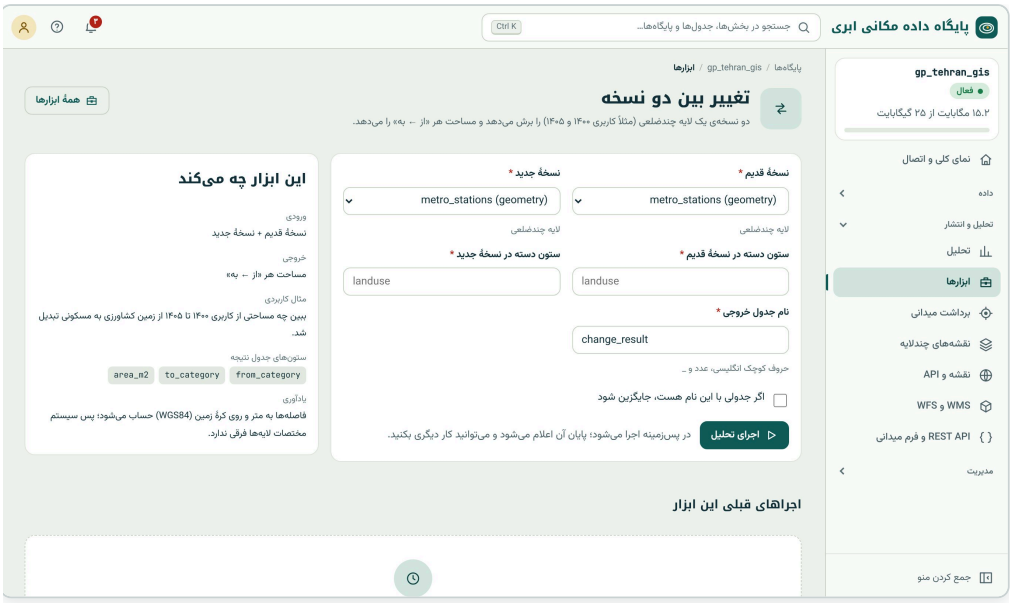
دکمه‌ی «نقشه و نمودار» روی کارت اجرا، نمودار ستونی انباشته (۱۰۰ درصد) سهم دسته‌ها در هر منطقه را پیشنهاد می‌کند (فصل ۴۳).

۲۷-۳ تغییر بین دو نسخه

پرسش: «از ۱۴۰۰ تا ۱۴۰۵ چند هکتار باغ به مسکونی تبدیل شده؟»

ورودی: نسخه‌ی قدیم و نسخه‌ی جدید یک لایه‌ی چندضلعی دسته‌دار (مثلاً کاربری ۱۴۰۰ و ۱۴۰۵). **پارامترها:** ستون دسته در هر نسخه.

خروجی: یک سطر برای هر جفت «از ← به»: `from_category`، `to_category`، `area_m2` و هندسه‌ی بخش‌هایی که این تغییر را داشته‌اند. سطرهایی که «از» و «به» یکی دارند، بخش‌های **بدون تغییر**اند.



فرم ابزار «تغییر بین دو نسخه».

این همان **ماتریس تغییر** مقایسه‌ی پس از طبقه‌بندی است (فصل ۳۹) و نتیجه‌اش در استودیوی نقشه با نمودار **Sankey** (جریان) بهترین نمایش را دارد (فصل ۴۳). هندسه‌ی هر جفت تغییر هم ذخیره می‌شود؛ پس می‌توانید فقط «باغ ← مسکونی» را روی نقشه نشان دهید.

هشدار

دو نسخه باید هم‌مرز باشند. اگر دو نسخه با دقت یا روش رقومی‌سازی متفاوت تهیه شده باشند، مرزهای یکسان کمی جابه‌جا رقومی می‌شوند و در نتیجه نوارهای باریک «تغییر کاذب» (sliver) پدید می‌آید. پیش از گزارش، سطرهای با مساحت بسیار کم را کنار بگذارید یا نسخه‌ها را به هم بچسبانید (`ST_Snap` ، فصل ۱۸).

۲۷-۴ برش با مرز

پرسش: «فقط بخشی از شبکه‌ی آب که داخل محدوده‌ی طرح است.»

ورودی: هر لایه و یک لایه‌ی مرز (چندضلعی). **خروجی:** عارضه‌ها، بریده‌شده به داخل مرز؛ برای چندضلعی‌ها با `area_m2` تازه.

زیر کاپوت: `ST_Intersection` با اجتماع مرزها (فصل ۱۸). عارضه‌هایی که کاملاً بیرون‌اند حذف و آن‌هایی که روی مرزند بریده می‌شوند.

۲۷-۵ حذف ناحیه

پرسش: «زمین‌های محدوده‌ی طرح، منهای حریم رودخانه.»

ورودی: لایه‌ی چندضلعی و لایه‌ی چندضلعی حذفی. **خروجی:** چندضلعی‌های باقی‌مانده با `area_m2` تازه.

زیر کاپوت: `ST_Difference` با اجتماع چندضلعی‌های حذفی (فصل ۱۸).

در دنیای واقعی

زمین قابل ساخت. یک شرکت مشاور برای برآورد زمین قابل ساخت در یک شهرک، این زنجیره را با چهار ابزار اجرا کرد: «حریم» ۵۰ متری رودخانه و ۳۰ متری خط فشار قوی (یکپارچه)، «حذف ناحیه» آن حریم‌ها از قطعه‌ها، «برش با مرز» به محدوده‌ی طرح، و «سهم همپوشانی» نتیجه با پهنه‌های کاربری طرح تفصیلی. خروجی جدول هکتار زمین قابل ساخت هر پهنه بود؛ بدون یک خط SQL.

خلاصه‌ی فصل

- «سهم همپوشانی»: سهم هر دسته از مساحت هر منطقه (`share_pct`)؛ نمودار ستونی انباشته.
- «تغییر بین دو نسخه»: ماتریس «از ← به» با مساحت و هندسه؛ نمودار Sankey؛ مراقب تغییرهای کاذب باریک.
- «برش با مرز» (اشتراک) و «حذف ناحیه» (تفاضل) لایه‌ها را به محدوده‌ی مطالعه محدود می‌کنند.
- هندسه‌ها پیش از محاسبه معتبر می‌شوند؛ مساحت‌ها روی geography به متر مربع‌اند.

تمرین‌ها

۱. سطح ۱ پارک‌ها را با مرز محله‌ها ببرید. مساحت کل پارک داخل محله‌ها چقدر است؟

۲. سطح ۱ محله‌ها منهای پارک‌ها: مساحت باقی‌مانده‌ی هر محله را بگیرید.

۳. سطح ۲ با SQL یک لایه‌ی «کاربری» آزمایشی بسازید (مثلاً پارک‌ها با دسته‌ی «سبز» و بقیه‌ی هر محله با دسته‌ی «شهری») و «سهم همپوشانی» را روی محله‌ها اجرا کنید.

۴. سطح ۳ دو نسخه از لایه‌ی کاربری بسازید که در یکی چند پارک به «شهری» تبدیل شده باشد و «تغییر بین دو نسخه» را اجرا کنید؛ نتیجه را با نمودار Sankey نمایش دهید.

فصل

۲۸

تحلیل شبکه‌ی معابر بدون کد

سطح: **متوسط** زمان مطالعه: **۶۰ دقیقه**

پیش‌نیاز: **فصل‌های ۲۴، ۲۵ و ۳۰**

در این فصل یاد می‌گیرید

- پیش‌نیاز ابزارهای شبکه (شبکه‌ی مسیریابی داده‌ی آماده‌ی ایران) را فراهم کنید.
- حوزه‌ی دسترسی و کوتاه‌ترین مسیر را بسازید.
- نزدیک‌ترین مرکز روی شبکه و پوشش چند مرکز را حساب کنید.
- تفاوت فاصله‌ی شبکه‌ای و هوایی را در تصمیم‌گیری به کار ببرید.

سرفصل‌ها

۲۸-۱ پیش‌نیاز: شبکه‌ی مسیریابی

۲۸-۲ حوزه‌ی دسترسی

۲۸-۳ کوتاه‌ترین مسیر

۲۸-۴ نزدیک‌ترین روی شبکه

۲۸-۵ پوشش چند مرکز

۲۸-۱ پیش‌نیاز: شبکه‌ی مسیریابی

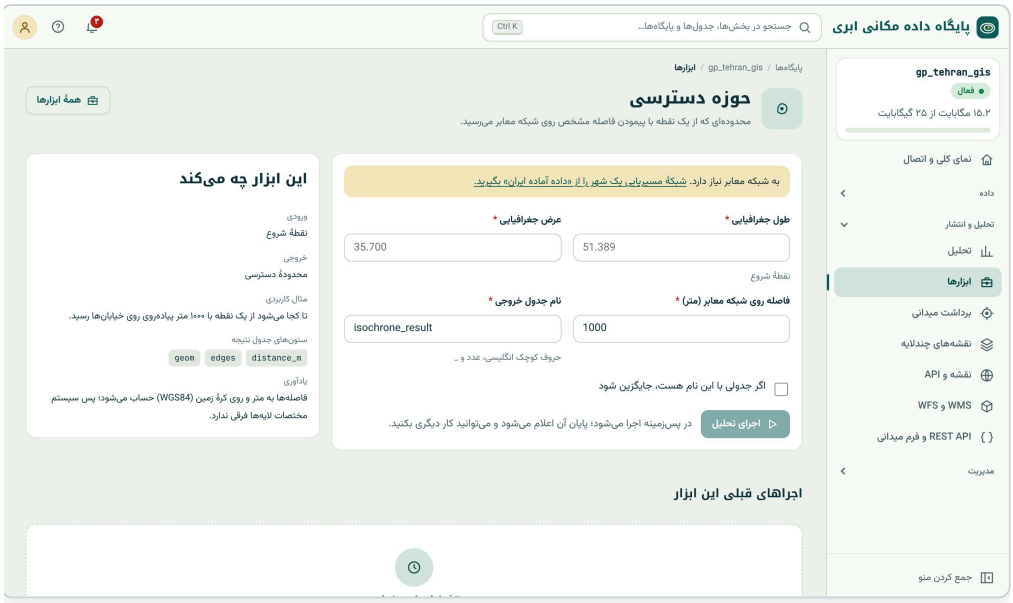
چهار ابزار گروه «شبکه‌ی معابر» روی **شبکه‌ی مسیریابی** واقعی خیابان‌ها کار می‌کنند. این شبکه را از بخش «داده‌ی آماده‌ی ایران» (فصل ۳۰) با گزینه‌ی «شبکه‌ی آماده‌ی مسیریابی» برای یک شهر یا شهرستان وارد می‌کنید. نتیجه در schemaای به نام `routing` دو جدول است: `routing.ways` (یال‌ها با طول، جهت و هزینه) و `routing.ways_vertices_pgr` (گره‌ها). تا این شبکه نباشد، کارت این ابزارها برچسب «به شبکه معابر نیاز دارد» و پیوند «گرفتن داده آماده» دارد.

ابزارها **طول به متر** را هزینه می‌گیرند و جهت خیابان‌های یک‌طرفه را رعایت می‌کنند؛ پس «کوتاه‌ترین» یعنی کوتاه‌ترین مسیر مجاز رانندگی از نظر طول، نه لزوماً سریع‌ترین. برای مسیر سریع‌ترین (با سرعت مجاز هر خیابان)، SQL فصل ۲۴ را با هزینه‌ی زمانی بنویسید.

۲۸-۲ حوزه‌ی دسترسی

پرسش: «از این ایستگاه آتش‌نشانی، تا ۲ کیلومتر روی خیابان‌ها کجا می‌رسیم؟»

ورودی: نقطه‌ی شروع (با کلیک روی نقشه) و فاصله روی شبکه به متر. **خروجی:** چندضلعی محدوده‌ی دسترسی.



فرم ابزار «حوزه‌ی دسترسی». نقطه‌ی شروع روی نقشه انتخاب می‌شود.

زیر کاپوت: `pgr_drivingDistance` از نزدیک‌ترین گره به نقطه و پوسته‌ی مقعر گره‌های رسیده (فصل ۲۴). حوزه‌ی دسترسی شبکه‌ای معمولاً شکلی ستاره‌ای دارد که در امتداد خیابان‌های اصلی کشیده شده است، برخلاف حریم دایره‌ای (فصل ۲۵). اختلاف این دو شکل همان اختلاف «دسترسی واقعی» و «فاصله‌ی هوایی» است.

۲۸-۳ کوتاه‌ترین مسیر

پرسش: «کوتاه‌ترین مسیر از انبار تا این مشتری؟»

ورودی: دو نقطه روی نقشه. **خروجی:** خط مسیر با طول به متر.

زیر کاپوت: `pgr_dijkstra` بین نزدیک‌ترین گره‌های دو نقطه و ادغام یال‌های مسیر (فصل ۲۴).

۲۸-۴ نزدیک‌ترین روی شبکه

پرسش: «برای هر مدرسه، نزدیک‌ترین درمانگاه **با رانندگی** کدام است و چند متر؟»

ورودی: لایه‌ی مبدأ (نقطه‌ها) و لایه‌ی مقصد (مراکز). **پارامتر:** حداکثر فاصله روی شبکه؛ مبدأهایی که دورترند بدون مقدار می‌مانند.

خروجی: همان نقطه‌های مبدأ با نام نزدیک‌ترین مرکز و فاصله‌ی شبکه‌ای.

زیر کاپوت: ماتریس هزینه‌ی چندبه‌چند (`pgr_dijkstraCost`) و انتخاب کمینه برای هر مبدأ با `DISTINCT ON` (فصل ۲۴). نتیجه را با «نزدیک‌ترین عارضه» (هوایی، فصل ۲۵) مقایسه کنید: هر جا نزدیک‌ترین مرکز هوایی و شبکه‌ای فرق دارند، مانعی (بزرگراه، رود، بلوک بسته) در کار است.

۲۸-۵ پوشش چند مرکز

پرسش: «با ۸ ایستگاه آتش‌نشانی فعلی، کدام بخش‌های شهر تا ۳ کیلومتر رانندگی پوشش داده می‌شوند و هر بخش مال کدام ایستگاه است؟»

ورودی: لایه‌ی نقطه‌ای مراکز. **پارامتر:** فاصله روی شبکه. **خروجی:** حوزه‌ی دسترسی هر مرکز، که بخش‌های مشترک به نزدیک‌ترین مرکز داده شده‌اند.

این ابزار پاسخ «حوزه‌ی نفوذ» (تیسن، فصل ۲۵) را روی شبکه‌ی واقعی می‌دهد. با آن می‌توان **نقطه‌های کور** (جاهایی که هیچ مرکزی پوشش نمی‌دهد) را پیدا کرد: نتیجه را با «حذف ناحیه» (فصل ۲۷) از محدوده‌ی شهر کم کنید.

در دنیای واقعی

مکان‌یابی پایگاه اورژانس. برای انتخاب بین سه زمین پیشنهادی برای پایگاه اورژانس تازه، هر کدام یک بار به لایه‌ی پایگاه‌های فعلی اضافه و «پوشش چند مرکز» با ۴ کیلومتر اجرا شد. سپس با «شمارش در چندضلعی»، جمعیت (نقطه‌های بلوک آماری) داخل پوشش در هر حالت شمرده شد. زمینی که بیشترین جمعیت تازه را زیر پوشش می‌برد انتخاب شد؛ سه اجرای ابزار و یک جدول مقایسه.

خلاصه‌ی فصل

- ابزارهای شبکه به `routing.ways` از «داده‌ی آماده‌ی ایران» (فقط شهر یا شهرستان) نیاز دارند.
- هزینه طول به متر است و یک‌طرفه‌ها رعایت می‌شوند؛ برای سریع‌ترین مسیر، SQL فصل ۲۴ با هزینه‌ی زمانی.
- حوزه‌ی دسترسی (`pgr_drivingDistance`) + پوسته‌ی مقعر)، کوتاه‌ترین مسیر (`pgr_dijkstra`)، نزدیک‌ترین روی شبکه (ماتریس هزینه)، پوشش چند مرکز.
- تفاوت نتیجه‌ی شبکه‌ای و هوایی، موانع واقعی را آشکار می‌کند.

تمرین‌ها

۱. سطح ۱ برای شهر خود شبکه‌ی مسیریابی را وارد کنید (فصل ۳۰) و حوزه‌ی دسترسی ۱ کیلومتری یک مدرسه را بسازید. آن را با حریم ۱ کیلومتری مقایسه کنید.

۲. سطح ۲ برای ده مدرسه، نزدیک‌ترین بیمارستان هوایی و شبکه‌ای را بگیرید. در چند مورد فرق دارند؟ چرا؟

۳. سطح ۳ پوشش ۳ کیلومتری بیمارستان‌ها را بسازید و با «حذف ناحیه»، نقطه‌های کور شهر را پیدا کنید.

فصل

۲۹

پرسش فارسی از داده

سطح: **مبتدی** ●●○

زمان مطالعه: **۴۵ دقیقه**

پیش‌نیاز: **فصل‌های ۸ و ۱۰**

در این فصل یاد می‌گیرید

- با جمله‌ی فارسی از داده‌ی خودتان پرسش مکانی بپرسید.
- SQL ساخته‌شده را بخوانید، بسنجید و در صورت نیاز اصلاح کنید.
- محدودیت‌ها و نکته‌های حریم خصوصی این قابلیت را بدانید.
- پرسش فارسی را به‌عنوان ابزار یادگیری SQL به کار ببرید.

سرفصل‌ها

۲۹-۱ از جمله به SQL

۲۹-۲ راه‌اندازی

۲۹-۳ نمونه‌ی پرسش‌ها

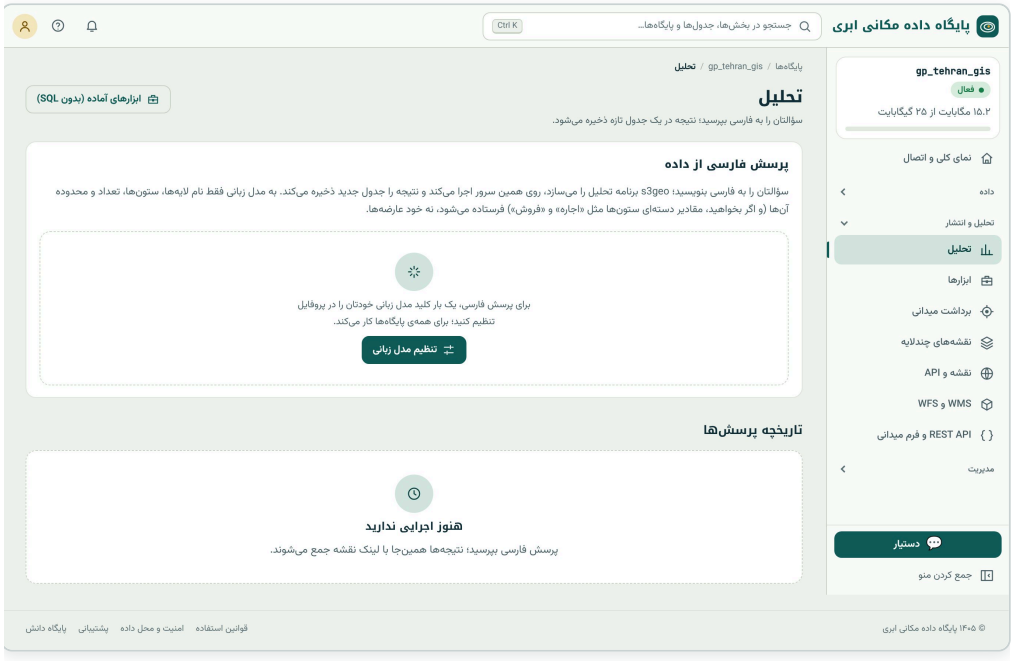
۲۹-۴ را بخوانید SQL

۲۹-۵ تاریخچه‌ی پرسش‌ها

۲۹-۱ از جمله به SQL

مدل‌های زبانی بزرگ امروز می‌توانند جمله‌ی فارسی را به SQL ترجمه کنند. پایگاه داده مکانی ابری این امکان را در دو جا دارد:

- **بخش «تحلیل»** (منوی تحلیل و انتشار): پرسش فارسی با انتخاب لایه‌ها؛ نتیجه یک **جدول تازه** در پایگاه است و در «تاریخچه‌ی پرسش‌ها» با پیوند نقشه می‌ماند.
- **زبانه‌ی «پرسش فارسی» در کنسول SQL** (فصل ۱۰): جمله را می‌نویسید و «ساخت SQL» را می‌زنید. پرسش خواندنی بی‌درنگ اجرا می‌شود؛ دستوری که جدول می‌سازد یا داده را تغییر می‌دهد **اول نشان داده می‌شود** تا خودتان اجرا کنید.



بخش «تحلیل»: پرسش فارسی از داده و تاریخچه‌ی پرسش‌ها. برای استفاده، کلید مدل زبانی خودتان را یک بار در پروفایل تنظیم می‌کنید.

۲۹-۲ راه‌اندازی

این قابلیت از **کلید API مدل زبانی خودتان** استفاده می‌کند (سرویس سازگار با OpenAI). کلید را یک بار در پروفایل تنظیم می‌کنید و برای همه‌ی پایگاه‌های شما کار می‌کند. آنچه به مدل فرستاده می‌شود محدود است: **نام لایه‌ها، ستون‌ها، نوع هندسه، تعداد عارضه و محدوده**، و اگر بخواهید، چند مقدار دسته‌ای ستون‌ها (مثل «اجاره» و «فروش») تا پرسش را دقیق‌تر بفهمد. **خود عارضه‌ها و داده‌ها فرستاده نمی‌شوند.** SQL ساخته‌شده روی همین سرور اجرا می‌شود.

۲۹-۳ نمونه‌ی پرسش‌ها

پرسش فارسی	SQL که باید ساخته شود (فصل)
«مدرسه‌های ابتدایی با بیش از ۶۰۰ دانش‌آموز»	`WHERE level = … AND students > 600` (۱۱)
«تعداد مدرسه‌های هر محله»	پیوند مکانی با `ST_Contains` و `GROUP BY` (۱۳ و ۱۷)
«مدرسه‌هایی که تا ۵۰۰ متری مترو هستند را در لایه‌ی جدید ذخیره کن»	`CREATE TABLE … AS` با `ST_DWithin` روی geography (۱۴ و ۱۷)
«نزدیک‌ترین بیمارستان به هر مدرسه و فاصله‌اش»	`LATERAL` با `<->` (۱۹)
«مساحت هر پارک به هکتار»	`ST_Area(geom::geography) / 10000` (۱۶)

۲۹-۴ را بخوانید SQL

مدل زبانی گاهی اشتباه می‌کند: ستون اشتباه انتخاب می‌کند، فاصله را به درجه حساب می‌کند، `ST_Contains` را برعکس می‌نویسد یا شرطی را جا می‌اندازد. پس همیشه پیش از تکیه بر نتیجه، SQL را بخوانید. فهرست کنترل:

۱. **جدول‌ها و ستون‌ها** همان‌هایی‌اند که منظور شما بود؟

۲. **واحد فاصله:** برای متر، `::geography` یا تبدیل به UTM دارد؟ (فصل ۱۶)

۳. **ترتیب آرگومان‌ها** در گزاره‌ها درست است؟ `ST_Contains(محله, مدرسه)`، نه برعکس (فصل ۱۷).

۴. **شمارش مضاعف:** پیوند یک‌به‌چند جمع را دو بار حساب نمی‌کند؟ (فصل ۱۷)

۵. **تعداد سطرهای نتیجه** با انتظار شما جور است؟ یک پرسش ساده‌ی کنترلی بزنید (مثلاً `count(*)`).

دکمه‌ی «چرا کند است؟» در کنسول، برای پرسش‌های کند برنامه‌ی اجرا را خوانا توضیح می‌دهد (فصل ۲۰).

> **نکته**
>
> **بهترین معلم SQL.** پرسش فارسی را ابزار یادگیری ببینید: پرسش را فارسی بپرسید، SQL را بخوانید و سعی کنید هر خط را با فصل‌های بخش سوم و چهارم بفهمید؛ بعد آن را تغییر دهید (فاصله‌ی دیگر، مقطع دیگر) و دوباره اجرا کنید. چند هفته بعد خودتان SQL می‌نویسید.

۲۹-۵ تاریخچه‌ی پرسش‌ها

هر پرسش بخش «تحلیل» در «تاریخچه‌ی پرسش‌ها» با متن پرسش، SQL، جدول نتیجه و پیوند نقشه می‌ماند. دکمه‌ی **«کنار ورودی‌ها»** نتیجه را روی لایه‌های ورودی در «نمایش با هم» باز می‌کند و **«نقشه و نمودار»** برای نتیجه‌هایی که آمار دارند، نمودار پیشنهادی می‌سازد (فصل ۴۳).

خلاصه‌ی فصل

- پرسش فارسی در «تحلیل» (نتیجه = جدول تازه) و در کنسول SQL (پرسش خواندنی فوری؛ تغییردهنده پس از نمایش).
- کلید مدل زبانی از خود کاربر است؛ فقط ساختار لایه‌ها (و در صورت تمایل چند مقدار دسته‌ای) فرستاده می‌شود، نه داده.
- SQL ساخته‌شده را با فهرست کنترل بسنجید: جدول و ستون، واحد فاصله، ترتیب آرگومان، شمارش مضاعف، تعداد سطر.
- پرسش فارسی ابزار یادگیری SQL است.

تمرین‌ها

۱. سطح ۱ پنج پرسش جدول بالا را بپرسید و SQL هر کدام را با پاسخ کتاب مقایسه کنید.

۲. سطح ۲ پرسشی بپرسید که مدل در آن اشتباه کند (مثلاً «مدرسه‌های نزدیک پارک») و SQL را خودتان اصلاح کنید. «نزدیک» را چگونه تعریف کردید؟

۳. سطح ۲ برای یک پرسش تحلیلی، نتیجه‌ی پرسش فارسی، ابزار آماده و SQL دستی خودتان را مقایسه کنید.

فصل

۳۰

داده‌ی آماده‌ی ایران از OpenStreetMap

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۷ و ۸**

در این فصل یاد می‌گیرید

- OpenStreetMap، مدل داده و مجوز ODbL آن را بشناسید.
- داده‌ی یک شهر یا شهرستان ایران را با یک کلیک وارد پایگاه کنید.
- هشت جدول داده و ستون‌های اصلی هر کدام را بشناسید و با SQL از آن‌ها پرسش کنید.
- کامل بودن و تازگی داده را ارزیابی کنید.

سرفصل‌ها

۳۰-۱ OpenStreetMap

۳۰-۲ ورود با یک کلیک

۳۰-۳ جدول‌ها

۳۰-۴ پرسش از داده‌ی OSM

۳۰-۵ کامل بودن و تازگی

۳۰-۱ OpenStreetMap

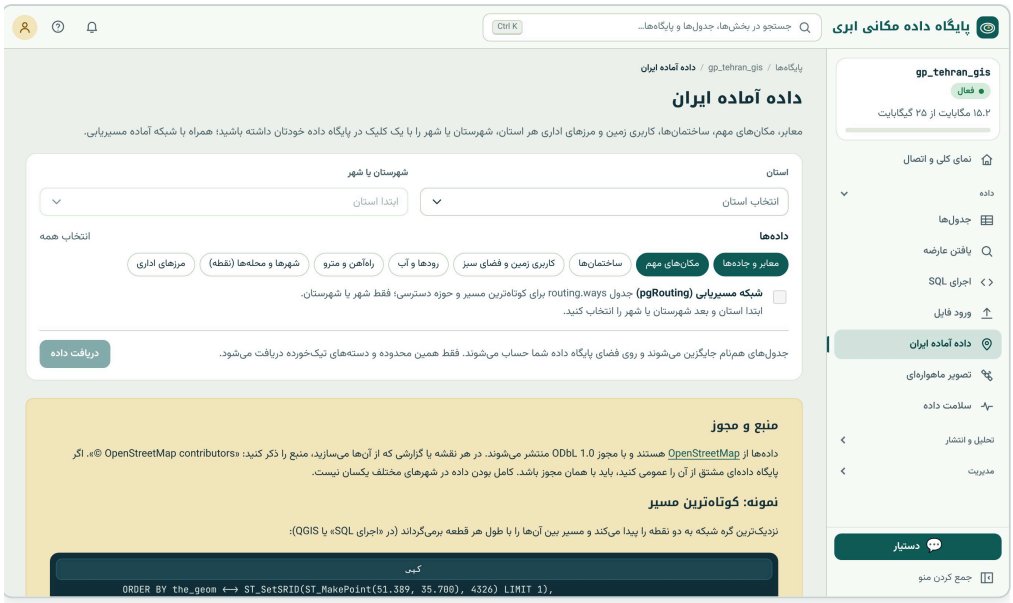
(OSM) **OpenStreetMap** نقشه‌ی آزاد جهان است که از ۲۰۰۴ با مشارکت میلیون‌ها داوطلب ساخته می‌شود؛ مثل ویکی‌پدیا برای نقشه. هر کسی می‌تواند خیابان، ساختمان یا مغازه‌ای را اضافه یا اصلاح کند. در بسیاری از شهرهای ایران، OSM کامل‌ترین داده‌ی آزاد معابر و مکان‌هاست.

مدل داده‌ی OSM سه جزء دارد: **گره** (node؛ نقطه)، **راه** (way؛ دنباله‌ی گره‌ها، خط یا چندضلعی) و **رابطه** (relation؛ گروهی از اجزا، مثل مرز استان). ویژگی‌ها به صورت **برچسب**‌های کلید=مقدار نگه داشته می‌شوند: `highway=primary` ، `amenity=hospital` ، `name=`بیمارستان امام. این ساختار آزاد، دلیل اینکه افزونه‌ی `hstore` (فصل ۴) در داده‌ی OSM کاربرد دارد.

> **هشدار**
>
> **مجوز ODbL.** داده‌ی OSM تحت مجوز **Open Database License** است: استفاده‌ی تجاری آزاد است، اما در هر نقشه یا گزارش باید «© OpenStreetMap contributors» بیاید، و اگر پایگاه داده‌ی **مشتق** از آن را عمومی منتشر کنید، باید با همان مجوز باشد. نقشه‌ها و تحلیل‌های شما (نه خود پایگاه داده) این شرط دوم را ندارند.

۳۰-۲ ورود با یک کلیک

منوی کناری ← داده ← **«داده‌ی آماده‌ی ایران»**:



صفحه‌ی «داده‌ی آماده‌ی ایران»: انتخاب محدوده (استان، شهرستان یا شهر)، دسته‌های داده و گزینه‌ی شبکه‌ی مسیریابی.

۱ **محدوده** را انتخاب کنید: استان، شهرستان یا شهر. محدوده‌ی کوچک‌تر یعنی ورود سریع‌تر و حجم کمتر؛ برای کار شهری، شهر یا شهرستان.

۲ **دسته‌ها** را تیک بزنید: معابر، مکان‌های مهم، ساختمان‌ها، کاربری زمین، آب‌ها، راه‌آهن، سکونتگاه‌ها و مرزهای اداری.

۳ در صورت نیاز **شبکه‌ی آماده‌ی مسیریابی** را هم بخواهید (فقط برای شهر یا شهرستان؛ پیش‌نیاز ابزارهای شبکه در فصل ۲۸ و SQL فصل ۲۴).

۴ «ورود» را بزنید. کار در پس‌زمینه انجام و پایانش با اعلان خبر داده می‌شود. تاریخ نسخه‌ی داده در صفحه نمایش داده می‌شود.

۳۰-۳ جدول‌ها

جدول	هندسه	ستون‌های اصلی
`osm_roads`	خط	`name`، `name_en`، `highway`، `oneway`، `maxspeed`، `lanes`، `surface`، `bridge`، `tunnel`
`osm_pois`	نقطه	`name`، `amenity`، `shop`، `tourism`، `leisure`، `office`، `historic`، `religion`، `healthcare`، `phone`، `opening_hours`
`osm_buildings`	چندضلعی	`name`، `building`، `levels`، `amenity`
`osm_landuse`	چندضلعی	`name`، `landuse`، `nature`، `leisure`
`osm_water`	خط	`name`، `waterway`
`osm_railways`	خط	`name`، `railway`
`osm_places`	نقطه	`name`، `place`، `population`
`osm_boundaries`	چندضلعی	`name`، `admin_level`

همه‌ی جدول‌ها `osm_id` دارند، هندسه در `geom` با SRID ۴۳۲۶ است، و `name` نام فارسی (اگر ثبت شده باشد) است. با شبکه‌ی مسیریابی، `routing.ways` و `routing.ways_vertices_pgr` هم ساخته می‌شوند.

۳۰-۴ پرسش از داده‌ی OSM

```sql
-- بیمارستان‌ها و داروخانه‌ها
SELECT name, amenity FROM osm_pois WHERE amenity IN ('hospital',
'pharmacy');

-- طول بزرگراه‌ها و خیابان‌های اصلی به کیلومتر
SELECT highway, round(sum(ST_Length(geom::geography)) / 1000) AS km
FROM osm_roads WHERE highway IN ('motorway', 'trunk', 'primary') GROUP
BY highway;

-- داروخانه‌هایی که تا ۵۰۰ متری یک بیمارستان‌اند
SELECT p.name FROM osm_pois p
WHERE p.amenity = 'pharmacy'
  AND EXISTS (SELECT 1 FROM osm_pois h WHERE h.amenity = 'hospital'
              AND ST_DWithin(p.geom::geography, h.geom::geography,
500));

-- مساحت کاربری‌های زمین به هکتار
SELECT landuse, round(sum(ST_Area(geom::geography)) / 10000) AS ha
FROM osm_landuse WHERE landuse IS NOT NULL GROUP BY landuse ORDER BY ha
DESC;
```

مقدارهای برچسب‌ها انگلیسی و استاندارد OSMاند (`hospital`، `school`، `residential`)؛ فهرست کامل را در ویکی OSM ببینید. برای جست‌وجوی نام فارسی، `pg_trgm` (فصل ۱۱) و یکدست کردن «ی» و «ک» را فراموش نکنید.

۳۰-۵ کامل بودن و تازگی

OSM داده‌ی داوطلبانه است و **کامل بودن آن در همه‌جا یکسان نیست**: معابر اصلی شهرهای بزرگ معمولاً کامل‌اند، اما ساختمان‌ها، تعداد طبقه، سرعت مجاز و تعداد خط اغلب ناقص. پیش از تحلیل، کیفیت را بسنجید (فصل ۲):

```sql
-- چند درصد معابر سرعت مجاز و چند درصد نام دارند؟
SELECT round(100.0 * count(*) FILTER (WHERE maxspeed IS NOT NULL) /
count(*), 1) AS with_speed_pct,
       round(100.0 * count(*) FILTER (WHERE name IS NOT NULL) /
count(*), 1)     AS with_name_pct
FROM osm_roads;
```

برای داده‌ی تازه‌تر، ورود را دوباره درخواست کنید. اگر خطایی در OSM می‌بینید، آن را در خود OpenStreetMap اصلاح کنید؛ هم به خودتان کمک کرده‌اید و هم به همه.

در دنیای واقعی

نقشه‌ی پایه‌ی فارسی. نقشه‌ی پایه‌ی برداری فارسی که زیر نقشه‌های سامانه قرار می‌گیرد هم از OpenStreetMap ساخته شده است، با برچسب‌های فارسی و بدون وابستگی به سرویس خارجی. یعنی همان داده‌ای که در این فصل وارد پایگاه کردید، زمینه‌ی نقشه‌های شما هم هست.

خلاصه‌ی فصل

- OSM نقشه‌ی آزاد داوطلبانه است: گره، راه و رابطه با برچسب‌های کلید=مقدار؛ مجوز ODbL با ذکر «© OpenStreetMap contributors».
- «داده‌ی آماده‌ی ایران» داده‌ی یک استان، شهرستان یا شهر را در هشت جدول `osm_*` و در صورت نیاز شبکه‌ی `routing` وارد می‌کند.
- برچسب‌ها مقدار انگلیسی استاندارد دارند؛ نام فارسی در `name` .
- کامل بودن یکسان نیست؛ پیش از تحلیل، کیفیت را با SQL بسنجید.

تمرین‌ها

۱. سطح ۱ داده‌ی شهر خود را وارد کنید. چند بیمارستان، مدرسه و داروخانه دارد؟

۲. سطح ۱ طول معابر هر رده‌ی `highway` را به کیلومتر بدهید.

۳. سطح ۲ درصد ساختمان‌هایی که تعداد طبقه دارند را حساب کنید. برای برآورد جمعیت از ساختمان‌ها کافی است؟

۴. سطح ۲ محله‌ای از شهر را که می‌شناسید با داده‌ی OSM مقایسه کنید. چه چیزهایی کم است؟ یکی را در OSM اضافه کنید.

بخش ۶

سنجش از دور

از فیزیک نور تا پایش خودکار مزرعه: طیف الکترومغناطیس، سنجنده‌ها، تفکیک‌ها، شاخص‌های طیفی، آمار هر قطعه، سری زمانی و مدل رقومی ارتفاع؛ همه با تصویرهای Sentinel-2 و Landsat.

۳۱ سنجش از دور؛ نور، طیف و امضای طیفی

۳۲ ماهواره، سنجنده و چهار تفکیک

۳۳ پیش‌پردازش؛ از عدد خام تا بازتاب سطح

۳۴ دریافت تصویر ماهواره‌ای در پایگاه داده مکانی ابری

۳۵ شاخص‌های طیفی

۳۶ آمار هر قطعه؛ از تصویر به جدول

۳۷ سری زمانی، فنولوژی و پایش خودکار

۳۸ مدل رقومی ارتفاع و تحلیل زمین

۳۹ طبقه‌بندی تصویر و آشکارسازی تغییر

۴۰ رادار و سنجش حرارتی

فصل

۳۱

سنجش از دور؛ نور، طیف و امضای طیفی

سطح: **مبتدی** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۲**

در این فصل یاد می‌گیرید

- سنجش از دور را تعریف و سامانه‌های فعال و غیرفعال را از هم جدا کنید.
- طیف الکترومغناطیس و رابطه‌ی طول موج، بسامد و انرژی را توضیح دهید.
- برهم‌کنش تابش با جو (پراکندگی، جذب، پنجره‌های جوی) و با سطح زمین را شرح دهید.
- امضای طیفی گیاه، خاک و آب را بخوانید و علت شکل هر کدام را بگویید.
- بدانید چرا در تصویر فروسرخ کاذب گیاه قرمز دیده می‌شود.

سرفصل‌ها

۳۱-۱ دیدن بدون لمس

۳۱-۲ تابش الکترومغناطیس

۳۱-۳ تابش و جو

۳۱-۴ برهم‌کنش با سطح زمین

۳۱-۵ امضای طیفی

۳۱-۶ ترکیب رنگی: دیدن نامرئی‌ها

۳۱-۱ دیدن بدون لمس

سنجش از دور (Remote Sensing یا **RS**) یعنی گردآوری اطلاعات درباره‌ی یک پدیده **بدون تماس فیزیکی** با آن. چشم شما هم یک سنجنده‌ی از دور است: نوری را که از اشیا بازتاب می‌شود می‌گیرد و مغز آن را تفسیر می‌کند. سنجش از دور ماهواره‌ای همین کار را از ارتفاع صدها کیلومتری، با سنجنده‌هایی انجام می‌دهد که نورهایی را هم «می‌بینند» که چشم ما نمی‌بیند، و هر چند روز یک بار از همه‌جای زمین تصویر برمی‌دارند.

اهمیت سنجش از دور در پاسخ به پرسش‌هایی است که با بازدید میدانی جواب دادنشان یا ممکن نیست یا بسیار گران است: سلامت گیاه در یک میلیون هکتار مزرعه، وسعت سیل در لحظه‌ی وقوع، روند خشک شدن یک دریاچه در سی سال، گسترش شهر در یک دهه. امروز تصویرهای Sentinel-2 و Landsat رایگان‌اند و پایگاه داده مکانی ابری آن‌ها را برای محدوده‌ی شما، بدون دانلود فریم‌های چندگیگابایتی، تحویل می‌دهد (فصل ۳۴).

اجزای یک سامانه‌ی سنجش از دور

1. **منبع انرژی:** خورشید، یا خود سنجنده.
2. **تابش و جو:** انرژی از جو می‌گذرد و بخشی از آن پراکنده یا جذب می‌شود.
3. **برهم‌کنش با هدف:** سطح زمین بخشی از انرژی را بازتاب، بخشی را جذب و بخشی را عبور می‌دهد؛ این نسبت‌ها به جنس سطح بستگی دارد.
4. **ثبت در سنجنده:** سنجنده‌ی روی ماهواره انرژی بازگشتی را در باندهای مختلف اندازه می‌گیرد.
5. **ارسال و پردازش:** داده به ایستگاه زمینی می‌رسد و به تصویر و محصول تبدیل می‌شود.
6. **تفسیر و تحلیل:** همان کاری که در این بخش از کتاب یاد می‌گیرید.
7. **کاربرد:** تصمیم‌گیری.

سنجش غیرفعال (passive) از انرژی خورشید یا گرمای خود زمین استفاده می‌کند: سنجنده‌های نوری مثل Sentinel-2 و Landsat و سنجنده‌های حرارتی. فقط در روز (برای نوری) و هوای صاف

کار می‌کنند. **سنجش فعال** (active) خودش انرژی می‌فرستد و بازگشت آن را می‌گیرد: رادار (Sentinel-1) و لیدار. شب و روز و از پشت ابر کار می‌کنند (فصل ۴۰).

۳۱-۲ تابش الکترومغناطیس

نور یک **موج الکترومغناطیس** است و با **طول موج** (λ، فاصله‌ی دو قله) و **بسامد** (ν، تعداد نوسان در ثانیه) توصیف می‌شود. حاصل‌ضرب این دو، سرعت نور است:

$$c = \lambda \nu \qquad (c \approx 3 \times 10^{8}\ \text{m/s})$$

و انرژی هر بسته‌ی نور (فوتون) با بسامد نسبت مستقیم و با طول موج نسبت عکس دارد ($E = h\nu = hc/\lambda$). یعنی طول موج‌های کوتاه پرانرژی‌ترند. این رابطه یک نتیجه‌ی عملی دارد: انرژی‌ای که از سطح در طول موج‌های بلند (مثل فروسرخ حرارتی) می‌رسد کم است، پس سنجنده باید پیکسل‌های بزرگ‌تری را جمع کند تا سیگنال کافی بگیرد؛ دلیل اینکه باند حرارتی Landsat تفکیک درشت‌تری دارد.

طیف الکترومغناطیس کل گستره‌ی طول موج‌هاست. چشم انسان فقط بخش بسیار کوچکی از آن را می‌بیند (حدود ۴۰۰ تا ۷۰۰ نانومتر):

سنجنده‌های نوری (Sentinel-2، Landsat): مرئی تا فروسرخ موج کوتاه · حرارتی: فروسرخ ۸ تا ۱۴ μm · رادار (Sentinel-1): ریزموج

طیف الکترومغناطیس و بخش‌هایی که در سنجش از دور به کار می‌روند. یک نانومتر یک میلیاردیم متر و یک میکرومتر (μm) هزار نانومتر است.

ناحیه	طول موج تقریبی	کاربرد در سنجش از دور
آبی	۴۵۰ تا ۵۲۰ nm	عمق آب کم‌عمق، تفکیک خاک و گیاه، رنگ طبیعی
سبز	۵۲۰ تا ۶۰۰ nm	قله‌ی بازتاب گیاه در مرئی، آب
قرمز	۶۳۰ تا ۶۹۰ nm	جذب کلروفیل؛ تفکیک گونه‌ها
لبه‌ی قرمز	۷۰۰ تا ۷۸۰ nm	حساس به مقدار کلروفیل و تنش گیاه
فروسرخ نزدیک (NIR)	۷۸۰ تا ۹۰۰ nm	ساختار برگ؛ زیست‌توده؛ مرز آب و خشکی
فروسرخ موج کوتاه (SWIR)	۱۵۵۰ تا ۲۳۵۰ nm	رطوبت گیاه و خاک، کانی‌ها، سوختگی
فروسرخ حرارتی (TIR)	۸ تا ۱۴ µm	دمای سطح
ریزموج	۱ میلی‌متر تا ۱ متر	رادار؛ زبری سطح، رطوبت، از پشت ابر

۳-۲۱ تابش و جو

پیش از رسیدن به سنجنده، تابش دو بار از جو می‌گذرد (رفت از خورشید و برگشت از زمین). جو دو اثر دارد:

- **پراکندگی** (scattering): ذرات و مولکول‌های جو نور را در جهت‌های مختلف پخش می‌کنند. **پراکندگی ریلی** که مولکول‌های گاز بسیار کوچک‌تر از طول موج ایجاد می‌کنند، با توان چهارم طول موج نسبت عکس دارد؛ پس آبی بسیار بیشتر از قرمز پراکنده می‌شود. دلیل آبی بودن آسمان همین است، و دلیل اینکه باند آبی تصویرها «مه‌آلودتر» از بقیه است. گرد و غبار و دود (پراکندگی مای) همه‌ی طول موج‌های مرئی را کم‌وبیش یکسان پخش و تصویر را کدر می‌کنند؛ پدیده‌ای آشنا در روزهای گردوغباری ایران.
- **جذب** (absorption): گازهایی مثل بخار آب، دی‌اکسید کربن و ازن بعضی طول موج‌ها را جذب می‌کنند. طول موج‌هایی که جو در آن‌ها شفاف است **پنجره‌های جوی** نام دارند و سنجنده‌ها باندهایشان را در همین پنجره‌ها می‌گذارند. به همین دلیل در نمودار امضای

طیفی پایین، حوالی ۱۴۰۰ و ۱۹۰۰ نانومتر باندی نمی‌بینید: بخار آب جو تقریباً همه‌ی انرژی آن‌جا را جذب می‌کند.

حذف اثر جو از تصویر **تصحیح اتمسفری** نام دارد و فصل ۳۳ آن را توضیح می‌دهد. تصویرهایی که سامانه تحویل می‌دهد از پیش تصحیح‌شده‌اند (سطح پردازش L2A).

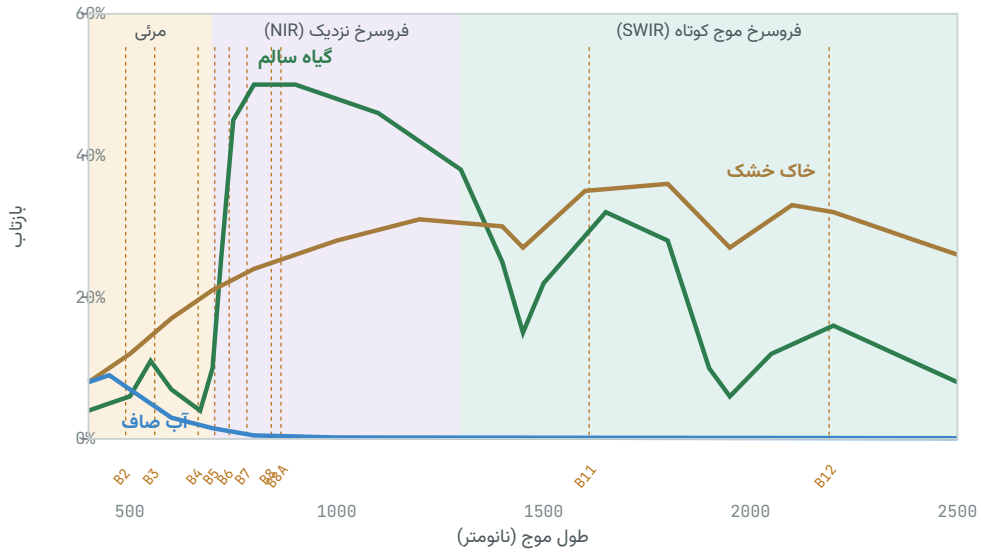
۳۱-۴ برهم‌کنش با سطح زمین

انرژی که به سطح می‌رسد سه سرنوشت دارد: **بازتاب**، **جذب** یا **عبور**. جمع این سه برابر انرژی ورودی است و نسبت آن‌ها در هر طول موج به جنس سطح بستگی دارد. سنجنده‌ی نوری بازتاب را می‌گیرد. کسری از انرژی ورودی که بازتاب می‌شود **بازتاب طیفی** (reflectance، ρ) نام دارد و عددی بین ۰ و ۱ (یا ۰ تا ۱۰۰ درصد) است.

شکل بازتاب هم مهم است. سطح صاف مثل آب آرام بازتاب **آینه‌ای** دارد: انرژی در یک جهت برمی‌گردد و اگر سنجنده در آن جهت نباشد، سطح تیره دیده می‌شود (یا اگر باشد، درخشش خیره‌کننده). سطح زبر مثل خاک شخم‌خورده بازتاب **پخشی** دارد و از همه‌ی جهت‌ها تقریباً یکسان دیده می‌شود.

۳۱-۵ امضای طیفی

نمودار بازتاب یک ماده در طول موج‌های مختلف را **امضای طیفی** (spectral signature) آن ماده می‌گویند؛ مثل اثر انگشت، هر ماده امضای خودش را دارد و همین اساس تشخیص پدیده‌ها در تصویر ماهواره‌ای است.



امضای طیفی سه سطح رایج. خط‌چین‌ها جای باندهای Sentinel-2 است. بین ۱۴۰۰ و ۱۹۰۰ نانومتر، بخار آب جو انرژی را جذب می‌کند و باندی آن‌جا نیست.

گیاه سالم

امضای گیاه سالم از همه‌ی سطوح طبیعی جالب‌تر است و تقریباً همه‌ی شاخص‌های گیاهی از آن می‌آیند:

1. **مرئی، بازتاب کم با قله‌ی سبز:** کلروفیل نور آبی و قرمز را برای فتوسنتز به شدت **جذب** می‌کند و سبز را بیشتر بازتاب می‌دهد؛ برای همین گیاه سبز دیده می‌شود. بازتاب در قرمز حدود ۴ تا ۵ درصد است.
2. **لبه‌ی قرمز:** بین ۶۸۰ و ۷۵۰ نانومتر بازتاب ناگهان چند برابر می‌شود. جای دقیق این «لبه» به مقدار کلروفیل وابسته است و با تنش گیاه به سمت طول موج کوتاه‌تر می‌رود؛ دلیل اینکه Sentinel-2 سه باند در این ناحیه دارد.
3. **فروسرخ نزدیک، فلات بازتاب بالا:** ساختار داخلی برگ (دیواره‌ی سلول‌ها و فضای هوای بین آن‌ها، بافت اسفنجی) فروسرخ نزدیک را پراکنده و تا ۴۰ تا ۵۰ درصد آن را بازتاب می‌کند. گیاه پرشاخ‌وبرگ و سالم فروسرخ بیشتری بازتاب می‌دهد؛ گیاه تحت تنش یا رو به خشکی کمتر.

4. **فروسرخ موج کوتاه:** آب داخل برگ این طول موج‌ها را جذب می‌کند. هرچه برگ آب بیشتری داشته باشد، بازتاب SWIR کمتر است؛ پایه‌ی شاخص رطوبت NDMI (فصل ۳۵).

اختلاف بزرگ بین قرمز (جذب شدید) و فروسرخ نزدیک (بازتاب شدید) **مهم‌ترین نشانه‌ی پوشش گیاهی** در سنجش از دور است و شاخص NDVI دقیقاً همین اختلاف را اندازه می‌گیرد.

خاک

بازتاب خاک با افزایش طول موج کم‌کم زیاد می‌شود و منحنی صاف‌تری دارد. سه عامل بر آن اثر دارند: **رطوبت** (خاک مرطوب در همه‌ی طول موج‌ها تیره‌تر است)، **مواد آلی** (خاک پرماده‌ی آلی تیره‌تر) و **بافت و کانی‌ها** (اکسید آهن در خاک‌های قرمز، فرورفتگی‌هایی در منحنی ایجاد می‌کند).

آب

آب صاف کمی آبی و سبز را بازتاب می‌دهد و فروسرخ را تقریباً کامل **جذب** می‌کند؛ پس در باند فروسرخ نزدیک تقریباً سیاه است. این ویژگی تشخیص مرز آب و خشکی را بسیار آسان می‌کند و پایه‌ی شاخص آب NDWI است. آب گل‌آلود یا دارای جلبک، بازتاب بیشتری در سبز و قرمز دارد؛ برای همین کیفیت آب هم تا حدی از روی رنگ آن در تصویر قابل برآورد است.

> **بدانید**
>
> **برف و ابر.** برف در مرئی بسیار روشن است (بازتاب ۸۰ تا ۹۰ درصد) ولی در SWIR تیره؛ ابر در هر دو روشن است. این تفاوت برای جدا کردن برف از ابر به کار می‌رود. لایه‌ی طبقه‌بندی صحنه‌ی Sentinel-2 که سامانه برای حذف ابر به کار می‌برد (فصل ۳۳) از همین منطق استفاده می‌کند.

۳۱-۶ ترکیب رنگی: دیدن نامرئی‌ها

صفحه‌ی نمایش سه رنگ دارد: قرمز، سبز و آبی. اگر سه باند تصویر را به این سه رنگ بدهیم، یک **ترکیب رنگی** ساخته‌ایم.

- **رنگ طبیعی** (true color): باند قرمز به قرمز، سبز به سبز، آبی به آبی؛ همان چیزی که چشم می‌بیند. در Sentinel-2 یعنی B4، B3، B2.

- **فروسرخ کاذب** (false color): باند فروسرخ نزدیک به قرمز، قرمز به سبز، سبز به آبی (در Sentinel-2: B8، B4، B3). چون گیاه سالم در فروسرخ نزدیک بسیار روشن است و این باند به کانال قرمز صفحه رفته، **گیاه به رنگ قرمز** دیده می‌شود؛ هرچه قرمزتر، پرشاخ‌وبرگ‌تر. آب سیاه یا آبی تیره، شهر خاکستری مایل به آبی و خاک لخت قهوه‌ای یا خاکستری دیده می‌شود.

فروسرخ کاذب از نخستین روزهای عکاسی هوایی برای تشخیص پوشش گیاهی به کار رفته است و هنوز هم سریع‌ترین راه برای «دیدن» سلامت گیاه بدون محاسبه است. هر دو ترکیب در سامانه محصول آماده‌اند (فصل ۳۴). ترکیب‌های دیگر هم کاربرد دارند؛ مثلاً SWIR2، NIR، قرمز (B12، B8، B4) برای مناطق سوخته و زمین‌شناسی که با محصول «همه‌ی باندها» در QGIS ساخته می‌شود.

در دنیای واقعی

دریاچه‌ی ارومیه. امضای طیفی آب (جذب کامل فروسرخ) باعث می‌شود مرز آب دریاچه در باند فروسرخ نزدیک با کنتراست بسیار بالا دیده شود. با همین ویژگی و تصویرهای Landsat از دهه‌ی ۱۹۸۰ تا امروز، پژوهشگران روند کاهش سطح آب دریاچه را سال به سال اندازه گرفته‌اند. در فصل ۳۷ خواهید دید که با شاخص NDWI و سری زمانی، همین کار را برای هر آبگیر دلخواه انجام می‌دهید.

خلاصه‌ی فصل

- سنجش از دور گردآوری اطلاعات بدون تماس است؛ غیرفعال (نور خورشید، گرما) یا فعال (رادار، لیدار).
- $c = \lambda\nu$ و انرژی فوتون با طول موج نسبت عکس دارد. سنجنده‌های نوری از مرئی تا SWIR، حرارتی ۸ تا ۱۴ µm و رادار ریزموج را می‌گیرند.
- جو نور را پراکنده (آبی بیشتر) و جذب (بخار آب) می‌کند؛ باندها در پنجره‌های جوی‌اند.
- بازتاب طیفی کسر انرژی بازتاب‌شده است؛ امضای طیفی نمودار آن در طول موج‌هاست.
- گیاه سالم: جذب قرمز (کلروفیل)، بازتاب بالای NIR (ساختار برگ)، جذب SWIR (آب برگ). آب: جذب کامل NIR. خاک: افزایش تدریجی.
- فروسرخ کاذب (NIR، قرمز، سبز) گیاه را قرمز نشان می‌دهد.

برای مدرس

زمان پیشنهادی: دو جلسه. یک برگ تازه و یک برگ خشک به کلاس بیاورید و با دوربین گوشی‌ای که فیلتر فروسرخ ضعیفی دارد (یا یک دوربین تبدیل‌شده) تفاوت را نشان دهید. نمودار امضای طیفی را روی تخته بکشید و از دانشجوها بخواهید جای قله‌ها و دره‌ها را با دلیل فیزیکی توضیح دهند.

تمرین‌ها

۱. سطح ۱ سنجنده‌های فعال و غیرفعال را با دو مثال مقایسه کنید. کدام برای پایش سیل در روز ابری مناسب‌تر است؟

۲. سطح ۱ چرا گیاه در تصویر رنگ طبیعی سبز و در فروسرخ کاذب قرمز دیده می‌شود؟

۳. سطح ۱ بسامد نور قرمز با طول موج ۶۶۵ نانومتر چقدر است؟

۴. سطح ۲ از روی نمودار امضای طیفی، برای تفکیک آب از خاک کدام باند بهترین است؟ برای تفکیک گیاه از خاک؟ چرا؟

۵. سطح ۲ یک گندم‌زار سالم و همان مزرعه پس از دو هفته بی‌آبی، در کدام باندها بیشترین تفاوت را نشان می‌دهند؟

۶. سطح ۳ درباره‌ی «پراکندگی مای» و اثر گرد و غبار بر تصویرهای ماهواره‌ای ایران تحقیق کنید. چه پیامدی برای انتخاب تصویر در فصل‌های گردوغباری دارد؟

فصل

۳۲

ماهواره، سنجنده و چهار تفکیک

سطح: **مبتدی** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل ۳۱**

در این فصل یاد می‌گیرید

- چهار نوع تفکیک (مکانی، طیفی، زمانی، رادیومتری) و بده‌بستان میان آن‌ها را توضیح دهید.
- مدار خورشیدآهنگ و زمین‌آهنگ و پهنای پوشش را بشناسید.
- مشخصات Landsat 8/9، Sentinel-2 و Copernicus DEM را بدانید و باندهای هر کدام را بخوانید.
- برای یک مسئله‌ی مشخص، سنجنده‌ی مناسب را انتخاب کنید.
- مفهوم پیکسل مختلط را بفهمید.

سرفصل‌ها

۳۲-۱ چهار تفکیک

۳۲-۲ مدار ماهواره

۳۲-۳ Sentinel-2

۳۲-۴ Landsat 8 و 9

۳۲-۵ مدل رقومی ارتفاع Copernicus

۳۲-۶ سنجنده‌های دیگری که باید بشناسید

۳۲-۷ کدام سنجنده برای کدام کار

۳۲-۱ چهار تفکیک

هر سنجنده با چهار نوع **تفکیک** (resolution) توصیف می‌شود و انتخاب سنجنده برای هر کار، در واقع انتخاب بین همین چهار است:

تفکیک	پرسش	نمونه
مکانی	کوچک‌ترین چیزی که می‌بیند چقدر است؟	پیکسل ۱۰ متری Sentinel-2، ۳۰ متری Landsat
طیفی	چند باند و چقدر باریک؟	۱۳ باند Sentinel-2، صدها باند سنجنده‌ی ابرطیفی
زمانی	هر چند روز یک بار از یک نقطه تصویر می‌گیرد؟	۵ روز Sentinel-2 (با دو ماهواره)، ۸ روز Landsat 8 و 9 با هم
رادیومتری	چند سطح روشنایی را از هم تشخیص می‌دهد؟	۱۲ بیت (۴۰۹۶ سطح) Sentinel-2؛ ۱۴ بیت Landsat 9

این چهار با هم **بده‌بستان** دارند. پیکسل کوچک‌تر یعنی انرژی کمتر به هر آشکارساز می‌رسد، پس یا باید باند پهن‌تر باشد (تفکیک طیفی کمتر) یا نوار تصویربرداری باریک‌تر (و در نتیجه بازگشت دیرتر به همان نقطه). ماهواره‌های تجاری با پیکسل نیم‌متری معمولاً پهنای پوشش چنده‌کیلومتری دارند و فقط به سفارش از یک نقطه تصویر می‌گیرند؛ Sentinel-2 با پیکسل ۱۰ متری نواری ۲۹۰ کیلومتری را می‌پوشاند و هر ۵ روز همه‌جای خشکی‌ها را تصویر می‌کند.

بدانید

پیکسل مختلط. پیکسل ۱۰ متری روی مرز یک مزرعه و جاده، بازتاب هر دو را با هم می‌گیرد. چنین **پیکسل مختلطی** (mixed pixel) مقداری میان امضای گیاه و آسفالت دارد. برای همین در آمار هر قطعه (فصل ۳۶) پیکسل‌های لبه‌ی قطعه‌های کوچک نتیجه را منحرف می‌کنند و قطعه‌ای که از چند پیکسل کوچک‌تر است اصلاً قابل تحلیل قابل‌اعتماد نیست. قاعده‌ی سرانگشتی: قطعه دست‌کم ده تا بیست برابر مساحت یک پیکسل باشد (برای Sentinel-2 حدود ۰٫۱ تا ۰٫۲ هکتار به بالا).

۳۲-۲ مدار ماهواره

- **مدار خورشیدآهنگ** (sun-synchronous): ماهواره در ارتفاع حدود ۷۰۰ تا ۸۰۰ کیلومتر، تقریباً از قطب تا قطب می‌چرخد و هر بار در **ساعت محلی ثابتی** (مثلاً حدود ۱۰:۳۰ صبح) از روی هر نقطه می‌گذرد. نور خورشید در تصویرهای روزهای مختلف مشابه است و مقایسه‌ی زمانی ممکن می‌شود. Sentinel-2 و Landsat خورشیدآهنگ‌اند.
- **مدار زمین‌آهنگ** (geostationary): در ارتفاع حدود ۳۶ هزار کیلومتر، هم‌گام با چرخش زمین؛ همیشه یک نیمکره را می‌بیند و هر چند دقیقه تصویر می‌گیرد، با پیکسل‌های کیلومتری. ماهواره‌های هواشناسی از این نوع‌اند.

۳۲-۳ Sentinel-2

Sentinel-2 بخشی از برنامه‌ی **کوپرنیک** اتحادیه‌ی اروپا است که آژانس فضایی اروپا (ESA) اجرا می‌کند. دو ماهواره‌ی Sentinel-2A (۲۰۱۵) و Sentinel-2B (۲۰۱۷) با اختلاف فاز ۱۸۰ درجه در یک مدار می‌چرخند و Sentinel-2C (۲۰۲۴) جانشین 2A شده است. سنجنده‌ی آن **MSI** (چندطیفی) ۱۳ باند دارد. داده‌ها رایگان و آزادند.

باند	نام	مرکز (nm)	تفکیک	کاربرد
B1	هواویز ساحلی	۴۴۳	۶۰ m	تصحیح جو
B2	آبی	۴۹۰	۱۰ m	رنگ طبیعی، آب
B3	سبز	۵۶۰	۱۰ m	رنگ طبیعی، NDWI
B4	قرمز	۶۶۵	۱۰ m	کلروفیل، NDVI
B5	لبه‌ی قرمز ۱	۷۰۵	۲۰ m	تنش گیاه
B6	لبه‌ی قرمز ۲	۷۴۰	۲۰ m	تنش گیاه
B7	لبه‌ی قرمز ۳	۷۸۳	۲۰ m	تنش گیاه
B8	فروسرخ نزدیک	۸۴۲	۱۰ m	زیست‌توده، NDVI
B8A	فروسرخ نزدیک باریک	۸۶۵	۲۰ m	پوشش گیاهی
B9	بخار آب	۹۴۵	۶۰ m	تصحیح جو
B10	SWIR ابر سیروس	۱۳۷۵	۶۰ m	تشخیص ابر (فقط در L1C)
B11	SWIR ۱	۱۶۱۰	۲۰ m	رطوبت، NDBI، NDMI
B12	SWIR ۲	۲۱۹۰	۲۰ m	سوختگی، NBR، زمین‌شناسی

محصول «همه‌ی باندها» در سامانه ده باند سطحی (B2 تا B11، B8A، B8 و B12) را تحویل می‌دهد که برای همه‌ی شاخص‌ها و طبقه‌بندی کافی است؛ باندهای ۶۰ متری که برای تصحیح جو به کار می‌روند در آن نیستند.

۳۲-۴ Landsat 8 و 9

برنامه‌ی **لندست** قدیمی‌ترین برنامه‌ی پیوسته‌ی تصویربرداری زمین است (از ۱۹۷۲) که سازمان زمین‌شناسی آمریکا (USGS) و ناسا اجرا می‌کنند. آرشیو پنجاه‌ساله‌ی آن برای مطالعه‌ی تغییرات بلندمدت بی‌رقیب است. Landsat 8 (۲۰۱۳) و Landsat 9 (۲۰۲۱) هر کدام هر ۱۶ روز از یک نقطه تصویر می‌گیرند و با هم هر ۸ روز. دو سنجنده دارند: **OLI** (نوری، ۹ باند) و **TIRS** (حرارتی، ۲ باند).

باند	نام	تفکیک	معادل Sentinel-2
B1	هواویز ساحلی	۳۰ m	B1
B2	آبی	۳۰ m	B2
B3	سبز	۳۰ m	B3
B4	قرمز	۳۰ m	B4
B5	فروسرخ نزدیک	۳۰ m	B8A
B6	SWIR ۱	۳۰ m	B11
B7	SWIR ۲	۳۰ m	B12
B8	پانکروماتیک	۱۵ m	—
B9	سیروس	۳۰ m	B10
B10، B11	فروسرخ حرارتی	۱۰۰ m (بازنمونه به ۳۰)	—

هشدار

شماره‌ی باندها یکسان نیست. «باند ۵» در Landsat 8 فروسرخ نزدیک است ولی در Sentinel-2 لبه‌ی قرمز. اگر فرمول شاخصی را از جایی کپی می‌کنید، همیشه بپرسید منظور کدام ماهواره است. سامانه برای جلوگیری از این خطا باندها را با **نام منطقی** (red، nir، swir16) نگه می‌دارد و برای هر ماهواره به باند درست نگاشت می‌کند.

در سامانه، برای Landsat محصول «همه‌ی باندها» شش باند (قرمز، سبز، آبی، NIR، SWIR1، SWIR2) دارد و چون پیکسل ۳۰ متری است، فایل برای همان محدوده حدود نُه برابر کوچک‌تر از Sentinel-2 است.

۳۲-۵ مدل رقومی ارتفاع Copernicus

Copernicus DEM GLO-30 مدل ارتفاعی جهانی با پیکسل حدود ۳۰ متری است که از داده‌ی راداری ماموریت TanDEM-X ساخته شده است. این یک مدل **سطح** (DSM) است: ارتفاع بالای ساختمان‌ها و تاج درختان را می‌دهد، نه لزوماً زمین لخت را. برای ارتفاع، شیب، جهت و

حوضه‌ی آبریز در مقیاس منطقه‌ای مناسب است (فصل ۳۸) و در سامانه محصول «ارتفاع (DEM)» است.

۳۲-۶ سنجنده‌های دیگری که باید بشناسید

سنجنده	تفکیک مکانی	بازگشت	ویژگی	دسترسی
MODIS (Terra، Aqua)	۲۵۰ تا ۱۰۰۰ m	روزانه	پایش جهانی، آتش، پوشش برف	رایگان
Sentinel-1	حدود ۱۰ m	۶ تا ۱۲ روز	رادار؛ از پشت ابر، شب و روز (فصل ۴۰)	رایگان
Sentinel-3	۳۰۰ m	۱ تا ۲ روز	رنگ اقیانوس، دمای سطح	رایگان
PlanetScope	حدود ۳ m	روزانه	ده‌ها ماهواره‌ی کوچک	تجاری
WorldView، Pléiades	۰٫۳ تا ۰٫۵ m	به سفارش	جزئیات شهری و ساختمانی	تجاری
پهپاد	چند سانتی‌متر	هر وقت لازم	محدوده‌ی کوچک، عکس‌برداری و فتوگرامتری	تجهیزات خود

۳۲-۷ کدام سنجنده برای کدام کار

مسئله	انتخاب پیشنهادی	دلیل
پایش سلامت مزرعه‌ها در یک دشت	Sentinel-2	۱۰ متر، ۵ روز، باندهای لبه‌ی قرمز
روند ۲۰ ساله‌ی سطح یک دریاچه	Landsat (و Sentinel-2 برای سال‌های اخیر)	آرشیو طولانی
گسترش شهر در یک دهه	Landsat یا Sentinel-2	بسته به دوره و دقت
سیل در روز ابری	Sentinel-1 (رادار)	عبور از ابر
شیب و ارتفاع منطقه	Copernicus DEM	جهانی و رایگان
شمارش درخت یا ساختمان	تصویر تجاری یا پهپاد	تفکیک زیرمتری
آتش‌سوزی‌های فعال کشور	MODIS / VIIRS	روزانه

در دنیای واقعی

بیمه‌ی محصول. شرکت بیمه‌ای برای تأیید ادعای خسارت سرمازدگی باغ‌های یک شهرستان، به جای بازدید از هزاران باغ، سری زمانی NDVI از Sentinel-2 را برای قطعه‌های بیمه‌شده می‌سنجد (فصل ۳۷): باغ‌هایی که NDVI آن‌ها پس از تاریخ سرما افتی محسوس داشته برای بازدید میدانی انتخاب می‌شوند. تفکیک ۱۰ متری برای قطعه‌های چند هکتاری کافی است و بازگشت ۵ روزه، حتی با چند روز ابری، چند تصویر پیش و پس از رویداد را تضمین می‌کند.

خلاصه‌ی فصل

- چهار تفکیک: مکانی، طیفی، زمانی، رادیومتری؛ با هم بده‌بستان دارند.
- پیکسل مختلط روی مرزها؛ قطعه‌ی کوچک‌تر از ده‌ها پیکسل تحلیل قابل‌اعتمادی ندارد.
- مدار خورشیدآهنگ ساعت ثابت عبور و مقایسه‌پذیری زمانی می‌دهد.
- Sentinel-2: ۱۳ باند، ۱۰ تا ۶۰ متر، ۵ روز، از ۲۰۱۵. Landsat 8/9: ۳۰ متر، ۸ روز با هم، آرشیو از ۱۹۷۲، باند حرارتی.
- شماره‌ی باندها بین ماهواره‌ها فرق می‌کند؛ با نام منطقی کار کنید.
- Copernicus DEM GLO-30 مدل سطح ۳۰ متری جهانی است.

برای مدرس

زمان پیشنهادی: یک جلسه. جدول «کدام سنجنده برای کدام کار» را خالی به دانشجوها بدهید و بگذارید خودشان با استدلال پر کنند. سپس یک محدوده را در سامانه با Sentinel-2 و Landsat دریافت کنید و پیکسل‌ها را در بزرگ‌نمایی زیاد کنار هم ببینید.

تمرین‌ها

۱. سطح ۱ چهار تفکیک را برای Sentinel-2 و Landsat 9 در یک جدول مقایسه کنید.

۲. سطح ۱ قطعه‌ای ۵۰ × ۴۰ متری در تصویر Sentinel-2 چند پیکسل ۱۰ متری و در Landsat چند پیکسل ۳۰ متری دارد؟

۳. سطح ۲ فرمول NDVI برای Landsat 8 با شماره‌ی باندها چیست؟ برای Sentinel-2؟

۴. سطح ۲ برای پایش هفتگی آبگیری کوچک (۲ هکتار) در منطقه‌ای کوهستانی که زمستان‌ها ابری است، چه ترکیبی از سنجنده‌ها پیشنهاد می‌کنید؟

۵. سطح ۳ چرا DEM حاصل از رادار در جنگل یا شهر «ارتفاع زمین» را بیش از واقع نشان می‌دهد؟ تفاوت DSM و DTM را توضیح دهید.

فصل

۳۳

پیش‌پردازش؛ از عدد خام تا بازتاب سطح

سطح: **متوسط** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل ۳۲**

در این فصل یاد می‌گیرید

- سطوح پردازش تصویر (L1C، L2A) و تفاوت بازتاب بالای جو و سطح را توضیح دهید.
- عدد دیجیتال را با ضریب مقیاس و جابه‌جایی به بازتاب تبدیل کنید.
- ماسک ابر و سایه را با لایه‌ی SCL و QA بشناسید.
- روش ترکیب بی‌ابر با میانه را توضیح دهید و بدانید کی به کار ببرید.
- با STAC، COG و روش‌های بازنمونه‌گیری آشنا شوید.

سرفصل‌ها

۳۳-۱ چرا پیش‌پردازش

۳۳-۲ سطوح پردازش

۳۳-۳ از عدد دیجیتال به بازتاب

۳۳-۴ ابر و سایه

۳۳-۵ ترکیب بی‌ابر

۳۳-۶ هندسه: ارتو و بازنمونه‌گیری

۳۳-۷ چگونه فقط تکه‌ی لازم از COG و STAC دانلود می‌شود

۳۳-۱ چرا پیش‌پردازش

عددی که سنجنده ثبت می‌کند هنوز «بازتاب سطح زمین» نیست. این عدد تحت تأثیر زاویه‌ی خورشید، جو، ابر، شکل زمین و ویژگی‌های خود سنجنده است. **پیش‌پردازش** این اثرها را حذف می‌کند تا عدد هر پیکسل تا حد ممکن فقط به جنس سطح بستگی داشته باشد؛ شرط لازم برای مقایسه‌ی دو تاریخ یا دو منطقه و برای استفاده از شاخص‌ها.

۳۳-۲ سطوح پردازش

سطح	محتوا	تصحیح‌های انجام‌شده
L0	داده‌ی خام سنجنده	هیچ
L1C (Sentinel-2) / L1 (Landsat)	**بازتاب بالای جو** (TOA)	رادیومتری، هندسی و ارتوی زمین (با DEM)
L2A (Sentinel-2) / L2 (Landsat)	**بازتاب سطح** (BOA یا SR)	به‌علاوه‌ی تصحیح اتمسفری و لایه‌ی طبقه‌بندی صحنه

بازتاب بالای جو (Top Of Atmosphere) همان چیزی است که سنجنده بالای جو می‌بیند: سهمی از نور که جو پراکنده کرده (به‌ویژه در آبی) هنوز در آن هست. **بازتاب سطح** (Bottom Of Atmosphere یا Surface Reflectance) پس از حذف اثر جو به دست می‌آید. برای Sentinel-2، محصول L2A با نرم‌افزار Sen2Cor ساخته می‌شود و برای Landsat، محصول Collection 2 Level-2.

پایگاه داده مکانی ابری همیشه از **سطح دو** استفاده می‌کند (`sentinel-2-l2a` و `landsat-c2-l2`)؛ پس همه‌ی محصولات و شاخص‌هایی که دریافت می‌کنید بر پایه‌ی بازتاب سطح‌اند و مقایسه‌ی تاریخ‌های مختلف معتبر است.

بدانید

تصحیح اتمسفری چه می‌کند؟ مدل‌های انتقال تابش (مثل 6S یا libRadtran) مسیر نور در جو را شبیه‌سازی می‌کنند. با برآورد مقدار هواویز (از باندهای آبی و هواویز ساحلی) و بخار آب (از باند ۹۴۵ نانومتر)، سهم جو از سیگنال کم و نور جذب‌شده جبران می‌شود. روشی ساده‌تر، **کم کردن پیکسل تیره** (Dark Object Subtraction) است: فرض می‌شود تیره‌ترین پیکسل‌ها (آب عمیق، سایه) بازتاب صفر دارند و هر چه در آن‌ها ثبت شده، سهم جو است.

۳۳-۳ از عدد دیجیتال به بازتاب

مقدار پیکسل در فایل، **عدد دیجیتال** (Digital Number یا DN) صحیح است تا حجم کم بماند. برای رسیدن به بازتاب (بین ۰ و ۱) باید آن را با **ضریب مقیاس** و **جابه‌جایی** تبدیل کرد:

$$\rho = DN \times \text{scale} + \text{offset}$$

مجموعه	scale	offset
Sentinel-2 L2A (از ۲۰۲۲ به بعد)	۰٫۰۰۰۱	-۰٫۱
Landsat Collection 2 Level-2	۰٫۰۰۰۰۲۷۵	-۰٫۲

دروازه‌ی ماهواره‌ی سامانه این ضرایب را برای هر تصویر از فراداده‌ی خود آن می‌خواند و پیش از ساختن شاخص‌ها اعمال می‌کند. محصول «همه‌ی باندها» مقدار **بازتاب × ۱۰۰۰۰** را به صورت عدد صحیح تحویل می‌دهد؛ یعنی عدد ۲۳۵۰ در یک پیکسل یعنی بازتاب ۰٫۲۳۵. وقتی در QGIS با این فایل شاخص می‌سازید، تقسیم بر ۱۰۰۰۰ را فراموش نکنید (برای شاخص‌های نرمال‌شده مثل NDVI مهم نیست، چون در صورت و مخرج ساده می‌شود؛ ولی برای شاخص‌هایی مثل EVI که عدد ثابت دارند لازم است).

با SQL می‌توان این تبدیل را دید. فرض کنید یک پیکسل Sentinel-2 در قرمز ۲۰۴۰ و در فروسرخ نزدیک ۴۴۹۰ است:

```sql
SELECT round(2040 * 0.0001 - 0.1, 4) AS red_reflectance,
       round(4490 * 0.0001 - 0.1, 4) AS nir_reflectance,
       round(((4490 * 0.0001 - 0.1) - (2040 * 0.0001 - 0.1)) / ((4490 *
0.0001 - 0.1) + (2040 * 0.0001 - 0.1)), 3) AS ndvi_reflectance,
       round((4490 - 2040)::numeric / (4490 + 2040), 3) AS ndvi_raw_dn;
```

red_reflectance	nir_reflectance	ndvi_reflectance	ndvi_raw_dn
0.1040	0.3490	0.541	0.375

دو NDVI آخر با هم فرق دارند! چون جابه‌جایی (offset) در صورت و مخرج ساده نمی‌شود. پس حتی برای شاخص‌های نرمال‌شده، روی **بازتاب** حساب کنید، نه عدد خام؛ کاری که سامانه انجام می‌دهد.

۳۳-۴ ابر و سایه

ابر بزرگ‌ترین دشمن سنجش از دور نوری است. پیکسل ابری بازتاب بالایی در همه‌ی باندها دارد و NDVI آن نزدیک صفر است؛ اگر حذف نشود، میانگین شاخص قطعه را به شدت پایین می‌آورد و «افت ناگهانی سلامت مزرعه» نشان می‌دهد که هرگز رخ نداده است. سایه‌ی ابر هم پیکسل‌ها را تیره و نامعتبر می‌کند.

برای حذف، هر محصول سطح دو یک **لایه‌ی ماسک** دارد:

- **Sentinel-2، لایه‌ی SCL** (طبقه‌بندی صحنه): هر پیکسل یک کلاس دارد؛ از جمله ۳ سایه‌ی ابر، ۴ گیاه، ۵ خاک لخت، ۶ آب، ۸ و ۹ ابر با احتمال متوسط و زیاد، ۱۰ سیروس و ۱۱ برف.
- **Landsat، لایه‌ی QA_PIXEL**: هر بیت یک پرچم است (ابر، سایه‌ی ابر، برف، سیروس، …).

سامانه با گزینه‌ی «حذف ابر و سایه» (که پیشنهاد می‌شود همیشه روشن بماند) این پیکسل‌ها را NoData می‌کند: در Sentinel-2 کلاس‌های ۰ (بی‌داده)، ۱ (اشباع)، ۳ (سایه‌ی ابر)، ۸ و ۹ (ابر) و ۱۰ (سیروس)؛ در Landsat بیت‌های ۰ تا ۴ (بی‌داده، ابر پهن‌شده، سیروس، ابر و سایه). برف (کلاس ۱۱) حذف نمی‌شود، چون در مناطق کوهستانی خودش موضوع مطالعه است؛ در تفسیر شاخص‌های زمستانی به آن توجه کنید. در «آمار هر قطعه» ستون `valid_pct` نشان می‌دهد

چند درصد پیکسل‌های هر قطعه پس از حذف ابر باقی مانده‌اند، و پایش خودکار قطعه‌هایی را که کمتر از نیمی از پیکسل‌هایشان معتبر است در مقایسه نمی‌آورد (فصل ۳۷).

> **هشدار**
>
> **درصد ابر کل صحنه، نه محدوده‌ی شما.** درصد ابری که هنگام جست‌وجوی تصویر می‌بینید برای کل صحنه‌ی ۱۱۰ × ۱۱۰ کیلومتری است. تصویری با ۴۰ درصد ابر ممکن است روی مزرعه‌ی شما کاملاً صاف باشد، و تصویری با ۵ درصد ابر ممکن است درست روی آن ابر داشته باشد. پیش‌نمایش را نگاه کنید.

۳۳-۵ ترکیب بی‌ابر

وقتی یک تصویر کاملاً صاف در بازه‌ی مورد نظر پیدا نمی‌شود، یا نقشه‌ای تمیز از یک فصل می‌خواهید، **ترکیب** (composite) راه حل است: چند تصویر از یک بازه را روی هم می‌گذاریم و برای هر پیکسل، از میان مقدارهای **بی‌ابر** آن پیکسل در تصویرهای مختلف، یک مقدار انتخاب می‌کنیم.

سامانه از **میانه** استفاده می‌کند: از کم‌ابرترین تصویرهای بازه (تا شش تصویر)، پس از ماسک ابر و سایه، برای هر پیکسل میانه‌ی مقدارهای معتبر را می‌گیرد. میانه نسبت به میانگین دو مزیت دارد: پیکسل ابری که از ماسک جا مانده (مقدار خیلی بالا) یا سایه‌ی جامانده (مقدار خیلی پایین) را نادیده می‌گیرد، و مقدار نهایی یکی از مقدارهای واقعی مشاهده‌شده است (برای تعداد فرد تصویر).

```sql
-- یک پیکسل در شش تصویر: دو مقدار ابرِ ماسک‌نشده (بالا) و یک سایه (پایین)
SELECT round(avg(v), 3) AS mean,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY v) AS median
FROM (VALUES (0.21), (0.23), (0.22), (0.61), (0.58), (0.05)) t(v);
```

mean	median
0.317	0.225

میانگین با ابر و سایه به ۰٫۳۲ کشیده شد؛ میانه ۰٫۲۲۵ ماند (میانگین دو مقدار وسط، چون تعداد زوج است) که با مقدار واقعی سطح می‌خواند.

نکته

ترکیب برای کدام کار؟ ترکیب برای **نقشه‌ی تمیز** (پوشش زمین، نقشه‌ی پایه، مقایسه‌ی فصل‌ها) و وقتی تصویر صاف نیست عالی است. اما برای پایش **رویداد** (سیل یک روزه، آتش‌سوزی، برداشت محصول در یک تاریخ) مناسب نیست، چون تاریخ‌های مختلف را با هم میانه می‌گیرد و تغییر سریع را محو می‌کند. برای رویداد، تک‌تصویر کم‌ابر نزدیک تاریخ را انتخاب کنید.

۳۳-۶ هندسه: ارتو و بازنمونه‌گیری

تصویرهای سطح یک و دو **ارتوریکتیفای** (orthorectified) شده‌اند: اثر ارتفاع زمین و زاویه‌ی دید با DEM حذف و تصویر روی یک شبکه‌ی منظم در یک سیستم مختصات (معمولاً UTM ناحیه‌ی مربوط) قرار گرفته است. دقت هندسی Sentinel-2 در حد کسری از پیکسل است.

وقتی رستر به سیستم مختصات یا اندازه‌ی پیکسل دیگری برده می‌شود، مقدار پیکسل‌های تازه با **بازنمونه‌گیری** (resampling) تعیین می‌شود:

روش	چگونه	کی
نزدیک‌ترین همسایه	مقدار نزدیک‌ترین پیکسل اصلی	داده‌ی طبقه‌ای (کاربری، SCL)؛ مقدار را عوض نمی‌کند
دوخطی (bilinear)	میانگین وزنی ۴ پیکسل	داده‌ی پیوسته (بازتاب، ارتفاع)؛ نرم‌تر
مکعبی (cubic)	۱۶ پیکسل	نمایش؛ نرم‌ترین، ولی ممکن است مقدار خارج از بازه بسازد

هرگز نقشه‌ی طبقه‌بندی‌شده را با روش دوخطی بازنمونه نکنید: میانگین کلاس ۲ (جنگل) و ۴ (شهر) می‌شود ۳ (آب)!

۳۳-۷ چگونه فقط تکه‌ی لازم دانلود می‌شود: COG و STAC

هر صحنه‌ی Sentinel-2 چند صد مگابایت تا بیش از یک گیگابایت است. اگر فقط یک مزرعه‌ی ۵۰ هکتاری را لازم دارید، دانلود کل صحنه اتلاف است. دو فناوری این مشکل را حل کرده‌اند:

- **STAC** (کاتالوگ دارایی‌های مکانی-زمانی): استاندارد باز برای جست‌وجوی تصویرها با محدوده، تاریخ و درصد ابر. دروازه‌ی ماهواره‌ی سامانه از کاتالوگ عمومی Earth Search استفاده می‌کند.
- **COG** (GeoTIFF بهینه‌شده برای ابر): فایل GeoTIFF که درونش کاشی‌بندی شده و نسخه‌های کوچک‌شده (overview) دارد. نرم‌افزار می‌تواند با درخواست HTTP بخشی (Range) فقط کاشی‌های محدوده‌ی شما را بخواند.

دروازه‌ی ماهواره‌ی سامانه با STAC تصویرهای مناسب را پیدا می‌کند، با COG فقط تکه‌ی لازم از هر باند را می‌خواند، ابر را ماسک، ترکیب و محصول را می‌سازد، و نتیجه را باز هم به شکل COG تحویل می‌دهد. به همین دلیل خروجی مستقیم در QGIS با نشانی HTTP باز می‌شود و فقط بخشی که روی صفحه است دانلود می‌شود (فصل ۳۴).

خلاصه‌ی فصل

- سطح L1C/L1 بازتاب بالای جو و سطح L2A/L2 بازتاب سطح (پس از تصحیح اتمسفری) است؛ سامانه همیشه از سطح دو استفاده می‌کند.
- بازتاب = offset + scale × DN؛ شاخص را روی بازتاب حساب کنید، نه DN. «همه‌ی باندها» = بازتاب × ۱۰۰۰۰.
- ماسک ابر و سایه با SCL (Sentinel-2) و QA_PIXEL (Landsat)؛ درصد ابر جست‌وجو برای کل صحنه است.
- ترکیب بی‌ابر با میانه‌ی تا شش تصویر کم‌ابر؛ برای نقشه‌ی تمیز، نه رویدادهای سریع.
- داده‌ی طبقه‌ای با نزدیک‌ترین همسایه بازنمونه شود؛ پیوسته با دوخطی.
- STAC برای جست‌وجو و COG برای خواندن تکه‌ی لازم.

برای مدرس

زمان پیشنهادی: یک جلسه. یک محدوده را یک بار با «حذف ابر و سایه» و یک بار بدون آن دریافت و NDVI دو نسخه را کنار هم نشان دهید. محاسبه‌ی میانگین و میانه‌ی شش مقدار را روی تخته انجام دهید تا ارزش میانه دیده شود.

تمرین‌ها

۱. سطح ۱ تفاوت بازتاب بالای جو و بازتاب سطح چیست؟ کدام برای مقایسه‌ی دو سال مناسب است؟

۲. سطح ۱ پیکسل Landsat با DN برابر ۱۲۰۰۰ در قرمز و ۲۵۰۰۰ در NIR؛ بازتاب‌ها و NDVI را حساب کنید.

۳. سطح ۲ چرا میانه برای ترکیب بی‌ابر از میانگین بهتر است؟ در چه وضعیتی میانه هم شکست می‌خورد؟

۴. سطح ۲ برای نقشه‌ی پوشش گیاهی بهار یک استان و برای برآورد وسعت سیل دیروز، ترکیب یا تک‌تصویر؟ چرا؟

۵. سطح ۳ از محصول «همه‌ی باندها» یک محدوده در QGIS، EVI را با فرمول فصل ۳۵ بسازید. چرا اینجا تقسیم بر ۱۰۰۰۰ لازم است؟

فصل

۳۴

دریافت تصویر ماهواره‌ای در پایگاه داده مکانی ابری

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۳۱ تا ۳۳**

در این فصل یاد می‌گیرید

- برای یک محدوده، تصویر Sentinel-2 یا Landsat یا مدل ارتفاعی دریافت کنید.
- محدوده را از روی نقشه یا یکی از جدول‌های خودتان تعیین کنید.
- بین تک‌تصویر و ترکیب بی‌ابر و بین محصول‌ها آگاهانه انتخاب کنید.
- تصویر را روی نقشه ببینید، دانلود کنید و مستقیم در QGIS باز کنید.
- سقف‌ها و سهمیه‌ی تصویر را مدیریت کنید.

سرفصل‌ها

۳۴-۱ بخش «تصویر ماهواره‌ای»

۳۴-۲ گام‌به‌گام: دریافت تصویر تازه

۳۴-۳ پس از ثبت

۳۴-۴ در QGIS

۳۴-۵ سقف‌ها و سهمیه

۳۴-۱ بخش «تصویر ماهواره‌ای»

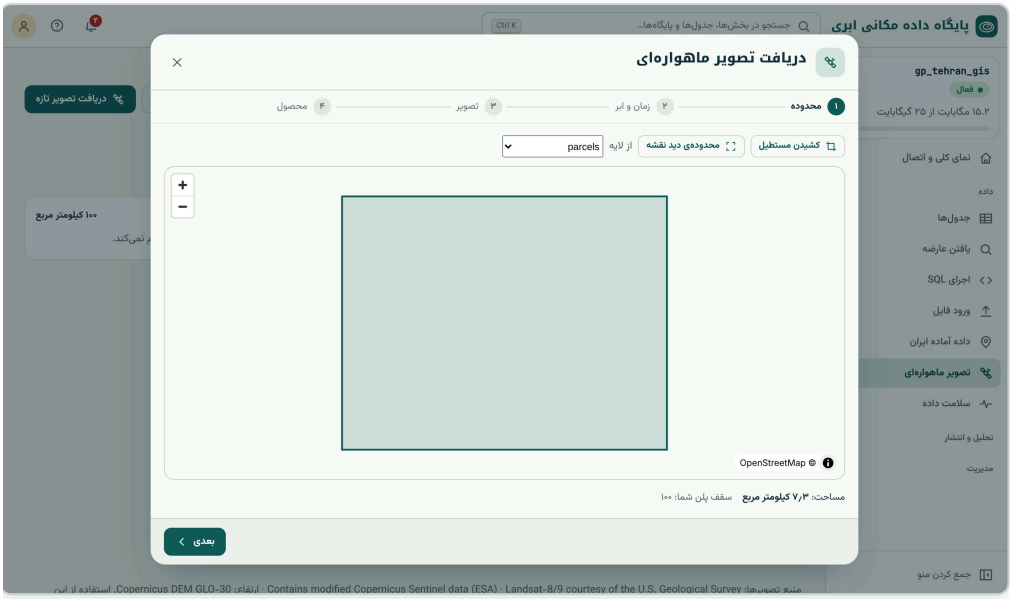
منوی کناری ← داده ← «تصویر ماهواره‌ای». این بخش دو زبانه دارد: **تصویرها** (دریافت و فهرست تصویرهای شما) و **تحلیل و سری زمانی** (آمار هر قطعه و پایش؛ فصل‌های ۳۶ و ۳۷). بالای صفحه سه کارت سهمیه هست: تعداد دریافت در ۳۰ روز اخیر، فضای رستر مصرف‌شده و بزرگ‌ترین محدوده‌ی مجاز برای هر دریافت.

پشت صحنه، کار سنگین (جست‌وجو در کاتالوگ، خواندن تکه‌ی لازم، حذف ابر، ترکیب و ساختن محصول) روی **دروازه‌ی ماهواره**، یک سرور جداگانه، انجام می‌شود و فایل آماده به پایگاه شما تحویل داده می‌شود (فصل ۳۳). کافی است بگویید **کجا**، **کی** و **چه محصولی**.

۳۴-۲ گام‌به‌گام: دریافت تصویر تازه

دکمه‌ی «دریافت تصویر تازه» مودالی چهارمرحله‌ای باز می‌کند.

۱. محدوده



گام اول: محدوده. اینجا محدوده از روی جدول قطعه‌های کشاورزی (parcels) برداشته شده است. مساحت و سقف پلن پایین نقشه نمایش داده می‌شود.

سه راه برای تعیین محدوده هست:

- **کشیدن مستطیل:** دو گوشه را روی نقشه بزنید.
- **محدوده‌ی دید نقشه:** همان چیزی که الان روی نقشه می‌بینید.
- **از لایه:** محدوده‌ی یکی از جدول‌های مکانی خودتان؛ دقیق‌ترین و تکرارپذیرترین راه. برای پایش مزرعه‌ها، جدول قطعه‌ها را انتخاب کنید.

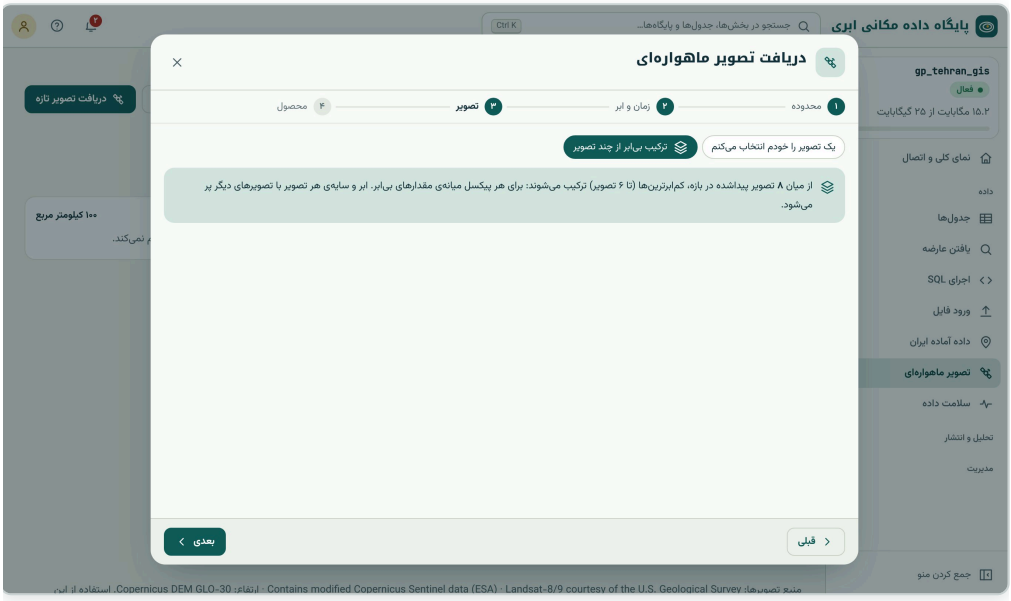
مساحت محدوده و سقف پلن پایین نقشه نوشته می‌شود. محدوده را تا حد امکان کوچک بگیرید: هم زودتر آماده می‌شود و هم فضای رستر کمتری می‌گیرد.

۲. ماهواره، بازه‌ی زمانی و ابر

گام دوم: ماهواره (Sentinel-2 یا Landsat 8/9)، بازه‌ی زمانی و بیشینه‌ی ابر.

- **ماهواره:** Sentinel-2 (۱۰ متر، هر ۵ روز، از ۲۰۱۷) برای بیشتر کارهای کشاورزی و شهری؛ Landsat 8/9 (۳۰ متر، هر ۸ روز، از ۲۰۱۳) برای مقایسه‌ی سال‌های دورتر و محدوده‌های بزرگ‌تر (فصل ۳۲). هر ماهواره فقط وقتی در فهرست می‌آید که دروازه واقعاً به آن دسترسی داشته باشد.
- **بازه‌ی زمانی:** ۷ روز اخیر، ۳۰ روز، ۳ ماه، یک سال یا بازه‌ی دلخواه.
- **بیشینه‌ی ابر:** خیلی صاف (کمتر از ۱۰٪)، تا ۲۰٪، تا ۴۰٪ یا هر مقدار. یادتان باشد این درصد برای کل صحنه است (فصل ۳۳).

۳. تصویر: تک‌تصویر یا ترکیب بی‌ابر



گام سوم: انتخاب یک تصویر از فهرست پیش‌نمایش‌ها یا «ترکیب بی‌ابر» از چند تصویر.

- **یک تصویر را خودم انتخاب می‌کنم:** فهرستی از تصویرهای پیداشده با پیش‌نمایش، تاریخ (شمسی)، درصد ابر و **پوشش محدوده** نمایش داده می‌شود. پوشش کم یعنی تصویر فقط بخشی از محدوده‌ی شما را می‌گیرد (محدوده روی لبه‌ی مسیر ماهواره افتاده است).
- **ترکیب بی‌ابر از چند تصویر:** کم‌ابرترین تصویرهای بازه (تا شش تصویر) برای هر پیکسل میانه‌ی مقدارهای بی‌ابر را می‌دهند (فصل ۳۳). برای نقشه‌ی تمیز و شاخص‌های فصلی بهترین انتخاب است؛ برای رویداد یک‌روزه نه.

۴. محصول

گام چهارم: محصول. NDVI انتخاب شده است؛ پایین، حجم تقریبی و مصرف سهمیه پیش از ثبت نمایش داده می‌شود.

محصول	باندها	نوع داده	کاربرد
رنگ طبیعی	قرمز، سبز، آبی	۸ بیتی	دیدن منطقه همان‌طور که چشم می‌بیند؛ نقشه‌ی پایه
فروسرخ کاذب	NIR، قرمز، سبز	۸ بیتی	گیاه قرمز؛ تفسیر سریع پوشش گیاهی و آب (فصل ۳۱)
شاخص گیاهی NDVI	یک باند	اعشاری ۳۲ بیتی	عدد ۱- تا ۱؛ تحلیل و نقشه‌ی سلامت گیاه (فصل ۳۵)
همه‌ی باندها	۱۰ باند (Landsat: ۶)	صحیح ۱۶ بیتی، بازتاب × ۱۰۰۰۰	کار تخصصی: شاخص‌های دیگر، طبقه‌بندی، در QGIS
ارتفاع (DEM)	یک باند	اعشاری ۳۲ بیتی	ارتفاع ۳۰ متری Copernicus؛ بدون انتخاب تاریخ و تصویر (فصل ۳۸)

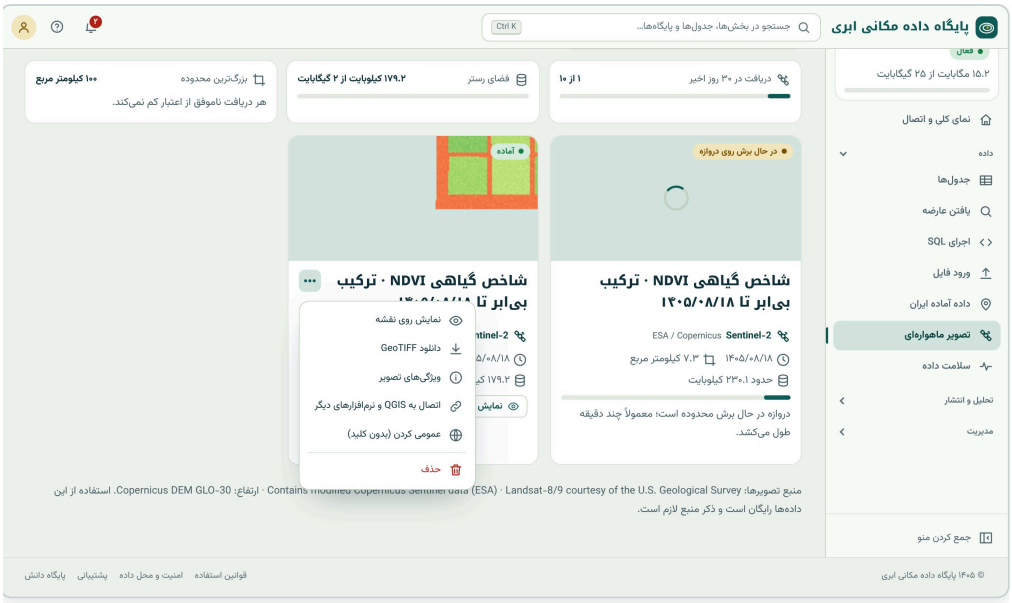
گزینه‌ی **حذف ابر و سایه** را روشن نگه دارید. پیش از ثبت، **حجم تقریبی** و **اعتبار باقی‌مانده** نمایش داده می‌شود. اگر سقفی پر باشد، دکمه به جای غیرفعال شدن، دلیل را با عدد می‌گوید؛ مثلاً «این تصویر حدود ۴۵ مگابایت می‌شود و فقط ۳۰ مگابایت خالی است». با «دریافت تصویر» کار ثبت می‌شود.

۳۴-۳ پس از ثبت

کارت تازه با وضعیت «در حال برش روی دروازه» در فهرست تصویرها می‌آید. پردازش معمولاً چند دقیقه طول می‌کشد و لازم نیست صفحه باز بماند؛ پایان کار با **اعلان** خبر داده می‌شود و کارت «آماده» می‌شود.

کارت تصویر آماده. عنوان، ماهواره، تاریخ (شمسی)، مساحت و حجم، و دکمه‌ی «نمایش روی نقشه».

منوی ⋯ هر کارت:



منوی ⋯ کارت تصویر: نمایش روی نقشه، دانلود GeoTIFF، ویژگی‌های تصویر، اتصال به QGIS و عمومی کردن.

- **نمایش روی نقشه:** تصویر با رنگ‌بندی مناسب محصول (مثلاً طیف قرمز-زرد-سبز برای NDVI).
- **دانلود GeoTIFF:** فایل COG برای کار روی رایانه‌ی خودتان.
- **ویژگی‌های تصویر:** تاریخ، درصد ابر، تفکیک، ابعاد، باندها، سیستم مختصات، حجم، شناسه‌ی صحنه و مجوز استفاده.
- **اتصال به QGIS و نرم‌افزارهای دیگر:** نشانی COG و کاشی XYZ.
- **عمومی کردن:** تصویر بدون کلید API در دسترس می‌شود (مثلاً برای نقشه‌ی عمومی). تصویر خصوصی با افزودن `?key=کلید_API` به نشانی باز می‌شود.
- **حذف:** فضای رستر را فوراً آزاد می‌کند، اما اعتبار دریافت را برنمی‌گرداند.

اگر دریافتی ناموفق شود (مثلاً تصویری با این شرایط پیدا نشود)، دلیل روی کارت نوشته می‌شود و اعتبار برمی‌گردد.

۳۴-۴ در QGIS

تصویر COG را می‌توانید بدون دانلود در QGIS باز کنید و فقط بخشی که روی صفحه است خوانده می‌شود:

۱. منوی ⋯ کارت ← «اتصال به QGIS و نرم‌افزارهای دیگر» و نشانی COG را کپی کنید.

۲. در **Source Type**، **Add Raster Layer** ← **Add Layer** ← **Layer**: QGIS را **Protocol: HTTP(S), cloud, etc.** بگذارید و نشانی را بچسبانید.

۳. برای «همه‌ی باندها»، در **Symbology** ← **Properties** نوع **Multiband color** را انتخاب و ترکیب دلخواه را بسازید؛ مثلاً باندهای ۱۰، ۸، ۱ (NIR، SWIR2 باریک، قرمز) برای مناطق سوخته و زمین‌شناسی.

۴. با **Raster Calculator** شاخص‌های دیگر را بسازید (فصل ۳۵) و با پلاگین‌های طبقه‌بندی، نقشه‌ی پوشش زمین تهیه کنید (فصل ۳۹).

ترتیب باندها در «همه‌ی باندها»ی Sentinel-2: ۱ قرمز، ۲ سبز، ۳ آبی، ۴ ،NIR ۵ تا ۷ لبه‌ی قرمز ۱ تا ۳، ۸ NIR باریک، ۹ SWIR2، ۱۰ SWIR1. در Landsat: ۱ قرمز، ۲ سبز، ۳ آبی، ۴ ،NIR ۵ SWIR2، ۶ SWIR1.

۳۴-۵ سقف‌ها و سهمیه

سقف	توضیح
دریافت در ۳۰ روز گذشته	هر دریافت ۳۰ روز بعد آزاد می‌شود؛ دریافت ناموفق کم نمی‌کند
فضای رستر	جدا از حجم جدول‌های پایگاه داده؛ حذف تصویر آزادش می‌کند
بزرگ‌ترین محدوده‌ی هر دریافت	به کیلومتر مربع، طبق پلن
دریافت هم‌زمان	حداکثر دو دریافت در حال پردازش برای هر پایگاه

وقتی سقفی پر شد: محدوده را کوچک کنید، محصول سبک‌تری بگیرید (NDVI به جای همه‌ی باندها؛ Landsat به جای Sentinel-2)، تصویر قدیمی را حذف کنید یا پلن را ارتقا دهید.

هشدار

ذکر منبع الزامی است. داده‌های Landsat، Sentinel و Copernicus DEM رایگان‌اند، اما هر جا منتشرشان می‌کنید (نقشه، گزارش، مقاله) باید منبع را ذکر کنید: «Contains modified Copernicus Sentinel data (ESA)»، «Landsat-8/9 image courtesy of the U.S. Geological Survey» و «Copernicus DEM GLO-30». سامانه این متن را پایین صفحه‌ی تصویرها و در ویژگی‌های هر تصویر می‌آورد.

نکته

اگر فقط عدد لازم دارید، تصویر نگیرید. اگر هدف شما «میانگین NDVI هر قطعه در چند تاریخ» است، لازم نیست تصویر دریافت کنید؛ زبانه‌ی «تحلیل و سری زمانی» مستقیم عدد هر قطعه را حساب و در جدول پایگاه شما می‌نویسد (فصل ۳۶). این کار سریع‌تر است، فضای رستر نمی‌گیرد و از اعتبار جداگانه‌ی تحلیل استفاده می‌کند.

خودتان امتحان کنید

برای محدوده‌ی جدول `parcels` (یا محله‌ی خودتان)، یک ترکیب بی‌ابر Sentinel-2 در یک سال اخیر با ابر تا ۲۰٪ و محصول «فروسرخ کاذب» بگیرید. روی نقشه ببینید: کدام قطعه‌ها قرمزترند؟ سپس همان محدوده را با محصول NDVI بگیرید و در QGIS باز کنید.

خلاصه‌ی فصل

- «دریافت تصویر تازه» چهار گام دارد: محدوده (مستطیل، دید نقشه یا جدول خودتان)، ماهواره و زمان و ابر، تک‌تصویر یا ترکیب بی‌ابر، محصول.
- محصول‌ها: رنگ طبیعی، فروسرخ کاذب، NDVI، همه‌ی باندها (بازتاب × ۱۰۰۰۰)، DEM.
- پردازش روی دروازه‌ی ماهواره انجام و نتیجه COG تحویل می‌شود؛ پایان کار با اعلان.
- کارت آماده: نمایش، دانلود GeoTIFF، ویژگی‌ها، نشانی QGIS، عمومی کردن.
- سقف دریافت ۳۰روزه، فضای رستر، مساحت هر دریافت و دو دریافت هم‌زمان؛ دریافت ناموفق اعتبار کم نمی‌کند.
- ذکر منبع داده‌ها الزامی است.

برای مدرس

زمان پیشنهادی: یک جلسه‌ی عملی. هر دانشجو برای زادگاه یا محل زندگی خودش یک تصویر رنگ طبیعی و یک فروسرخ کاذب بگیرد و تفاوت‌ها را توضیح دهد. برای کلاس‌های بزرگ، سقف دریافت را در نظر بگیرید و تمرین را گروهی کنید.

تمرین‌ها

۱. سطح ۱ یک تصویر رنگ طبیعی از محل دانشگاه یا محل کار خود دریافت کنید. در «ویژگی‌های تصویر» تاریخ، درصد ابر و سیستم مختصات را بخوانید.

۲. سطح ۱ حجم تقریبی محصول «همه‌ی باندها» را برای یک محدوده با Sentinel-2 و Landsat مقایسه کنید. نسبت آن‌ها چقدر است و چرا؟

۳. سطح ۲ تصویر «همه‌ی باندها» را در QGIS باز و سه ترکیب رنگی (طبیعی، فروسرخ کاذب، SWIR) بسازید. هر کدام چه چیزی را بهتر نشان می‌دهد؟

۴. سطح ۲ برای یک دریاچه، تصویر Landsat یک تابستان پنج سال پیش و تابستان امسال را بگیرید و تغییر سطح آب را با چشم مقایسه کنید.

فصل

۳۵

شاخص‌های طیفی

سطح: **متوسط** | زمان مطالعه: **۱۰۰ دقیقه** | پیش‌نیاز: **فصل‌های ۳۱ و ۳۳**

در این فصل یاد می‌گیرید

- منطق شاخص‌های نرمال‌شده را توضیح دهید و NDVI را از امضای طیفی استخراج کنید.
- شاخص‌های NDRE، EVI، SAVI، NDBI، NBR، NDMI، MNDWI، NDWI، NDVI و BSI را بشناسید و برای هر مسئله شاخص مناسب را انتخاب کنید.
- مقدار شاخص‌ها را تفسیر و محدودیت‌های آن‌ها (اشباع، اثر خاک، ابر) را بیان کنید.
- شاخص‌ها را در سامانه، در QGIS و با SQL محاسبه کنید.
- با تفاضل NBR شدت آتش‌سوزی را طبقه‌بندی کنید.

سرفصل‌ها

۳۵-۱ از امضای طیفی تا یک عدد

۳۵-۲ شاخص تفاضل نرمال‌شده‌ی پوشش گیاهی: NDVI

۳۵-۳ شاخص‌های آب

۳۵-۴ رطوبت پوشش گیاهی: NDMI

۳۵-۵ NBR و شدت آتش‌سوزی

۳۵-۶ سطوح ساخته‌شده: NDBI

۳۵-۷ شاخص‌های پیشرفته‌تر

۳۵-۸ انتخاب شاخص

۳۵-۹ محاسبه در سامانه

۳۵-۱ از امضای طیفی تا یک عدد

در فصل ۳۱ دیدیم که گیاه سالم قرمز را جذب و فروسرخ نزدیک را بازتاب می‌کند، آب فروسرخ را جذب می‌کند و آب داخل برگ SWIR را. **شاخص طیفی** این تفاوت‌ها را در یک عدد خلاصه می‌کند تا به جای نگاه کردن به ده باند، یک نقشه‌ی معنادار داشته باشیم: «سبزی»، «آب»، «رطوبت»، «سوختگی».

پرکاربردترین خانواده، **شاخص‌های نرمال‌شده** است:

$$\mathrm{ND} = \frac{\rho_1 - \rho_2}{\rho_1 + \rho_2}$$

تقسیم بر مجموع دو کار مهم می‌کند: نتیجه را همیشه بین ۱- و ۱ نگه می‌دارد، و اثر **روشنایی کلی** (مثلاً سایه‌ی تپه یا تفاوت زاویه‌ی خورشید) را تا حد زیادی حذف می‌کند، چون روشنایی هر دو باند را با هم کم یا زیاد می‌کند. هر پنج شاخصی که سامانه آماده دارد از این خانواده‌اند.

۳۵-۲ شاخص تفاضل نرمال‌شده‌ی پوشش گیاهی: NDVI

$$\mathrm{NDVI} = \frac{\rho_{NIR} - \rho_{Red}}{\rho_{NIR} + \rho_{Red}}$$

NDVI (روز و همکاران، ۱۹۷۴) پرکاربردترین شاخص سنجش از دور است. با بازتاب‌های نمونه از نمودار امضای طیفی فصل ۳۱:

```sql
SELECT surface, red, nir,
       round((nir - red) / (nir + red), 2) AS ndvi
FROM (VALUES ('0.50 ,0.04      ,'گیاه سالم'),
             ('0.30 ,0.08   ,'گیاه تحت تنش'),
             ('0.24 ,0.21        ,'خاک خشک'),
             ('0.005 ,0.03         ,'آب صاف'),
             ('0.62 ,0.60            ,'ابر')) t(surface, red, nir);
```

surface	red	nir	ndvi
گیاه سالم	0.04	0.50	0.85
گیاه تحت تنش	0.08	0.30	0.58
خاک خشک	0.21	0.24	0.07
آب صاف	0.03	0.005	-0.71
ابر	0.60	0.62	0.02

بازه‌ی NDVI	تفسیر معمول
کمتر از صفر	آب، برف، ابر
۰ تا ۰٫۱	خاک لخت، سنگ، ماسه، سطوح شهری
۰٫۱ تا ۰٫۳	پوشش گیاهی تُنُک، مرتع، بوته‌زار، مزرعه‌ی تازه‌کاشت
۰٫۳ تا ۰٫۶	پوشش گیاهی متوسط، مزرعه در حال رشد
بیش از ۰٫۶	پوشش گیاهی متراکم و سالم، جنگل، مزرعه در اوج رشد

این بازه‌ها **راهنما**ست، نه قانون: NDVI مزرعه‌ی گندم سالم در اسفند با همان مزرعه در اردیبهشت بسیار فرق دارد. معنای واقعی NDVI در **مقایسه** است: با قطعه‌های هم‌کشت، با همان قطعه در سال قبل، یا با تاریخ قبلی (فصل ۳۷).

هشدار

محدودیت‌های NDVI. - **اشباع:** در پوشش گیاهی بسیار متراکم (جنگل، مزرعه‌ی ذرت در اوج)، NDVI به حدود ۰٫۸ تا ۰٫۹ می‌رسد و دیگر افزایش زیست‌توده را نشان نمی‌دهد. EVI و NDRE در این حالت حساس‌ترند. - **اثر خاک:** در پوشش تُنُک، رنگ و رطوبت خاک زیر گیاه NDVI را تغییر می‌دهد. SAVI برای همین ساخته شده است. - **ابر و سایه:** پیکسل ابری NDVI نزدیک صفر دارد و میانگین را پایین می‌کشد؛ همیشه ابر را ماسک کنید (فصل ۳۳).

۳۵-۳ شاخص‌های آب

NDWI (مک‌فیترز، ۱۹۹۶) از جذب کامل فروسرخ توسط آب استفاده می‌کند:

$$\mathrm{NDWI} = \frac{\rho_{Green} - \rho_{NIR}}{\rho_{Green} + \rho_{NIR}}$$

مقدار مثبت معمولاً یعنی آب سطحی. برای تشخیص مرز دریاچه، رود، آبگیر و پهنه‌ی سیل بسیار مناسب است.

MNDWI (شو، ۲۰۰۶) به جای NIR از SWIR1 استفاده می‌کند و در مناطق شهری که سطوح ساخته‌شده در NDWI مثبت کاذب می‌سازند، آب را بهتر جدا می‌کند:

$$\mathrm{MNDWI} = \frac{\rho_{Green} - \rho_{SWIR1}}{\rho_{Green} + \rho_{SWIR1}}$$

۳۵-۴ رطوبت پوشش گیاهی: NDMI

$$\mathrm{NDMI} = \frac{\rho_{NIR} - \rho_{SWIR1}}{\rho_{NIR} + \rho_{SWIR1}}$$

NDMI (گائو، ۱۹۹۶؛ در منابع گاهی با همان نام NDWI آمده و این منشأ سردرگمی است) به **آب داخل برگ** حساس است: آب برگ SWIR را جذب می‌کند، پس برگ پرآب NDMI بالاتری دارد. برای پایش **تنش آبی** پیش از آنکه در NDVI (سبزی) دیده شود، و برای ارزیابی خطر آتش‌سوزی (پوشش خشک) به کار می‌رود. افت NDMI در حالی که NDVI هنوز ثابت است، اغلب **هشدار زودهنگام** کم‌آبی است.

۳۵-۵ NBR و شدت آتش‌سوزی

$$\mathrm{NBR} = \frac{\rho_{NIR} - \rho_{SWIR2}}{\rho_{NIR} + \rho_{SWIR2}}$$

گیاه سالم NIR بالا و SWIR2 پایین دارد؛ سطح سوخته برعکس. **تفاضل NBR** پیش و پس از آتش ($\mathrm{dNBR} = \mathrm{NBR}_{pre} - \mathrm{NBR}_{post}$) شدت سوختگی را می‌دهد. طبقه‌بندی رایج USGS:

```sql
SELECT parcel, nbr_pre, nbr_post,
       round(nbr_pre - nbr_post, 2) AS dnbr,
       CASE WHEN nbr_pre - nbr_post < 0.10 THEN 'نسوخته'
            WHEN nbr_pre - nbr_post < 0.27 THEN 'شدت کم'
            WHEN nbr_pre - nbr_post < 0.44 THEN 'متوسط-کم'
            WHEN nbr_pre - nbr_post < 0.66 THEN 'متوسط-زیاد'
            ELSE 'شدت زیاد' END AS severity
FROM (VALUES ('0.58 ,0.62 ,'الف), ('0.31 ,0.55 ,'ب), ('0.05 ,0.60 ,'ج),
('0.25- ,0.48 ,'د)) t(parcel, nbr_pre, nbr_post);
```

parcel	nbr_pre	nbr_post	dnbr	severity
الف	0.62	0.58	0.04	نسوخته
ب	0.55	0.31	0.24	شدت کم
ج	0.60	0.05	0.55	متوسط-زیاد
د	0.48	-0.25	0.73	شدت زیاد

۳۵-۶ سطوح ساخته‌شده: NDBI

$$\mathrm{NDBI} = \frac{\rho_{SWIR1} - \rho_{NIR}}{\rho_{SWIR1} + \rho_{NIR}}$$

سطوح ساخته‌شده و خاک لخت در SWIR1 بازتاب بیشتری از NIR دارند؛ گیاه برعکس. NDBI برای پایش **گسترش شهر** و ساخت‌وساز در حریم‌ها به کار می‌رود. چون خاک لخت هم NDBI مثبت دارد، معمولاً آن را با NDVI ترکیب می‌کنند (مثلاً NDBI مثبت **و** NDVI کم).

۳۵-۷ شاخص‌های پیشرفته‌تر

این شاخص‌ها در سامانه آماده نیستند، اما با محصول «همه‌ی باندها» در QGIS یا با SQL روی رستر محاسبه می‌شوند:

شاخص	فرمول	کاربرد
SAVI	$1.5 \cdot \frac{NIR - Red}{NIR + Red + 0.5}$	پوشش تُنُک (مناطق خشک ایران)؛ کاهش اثر خاک
EVI	$2.5 \cdot \frac{NIR - Red}{NIR + 6Red - 7.5Blue + 1}$	پوشش متراکم؛ کاهش اشباع و اثر جو
NDRE	$\frac{NIR - RE1}{NIR + RE1}$	مقدار کلروفیل و نیتروژن؛ مرحله‌ی میانی و پایانی رشد
BSI	$\frac{(SWIR1 + Red) - (NIR + Blue)}{(SWIR1 + Red) + (NIR + Blue)}$	خاک لخت؛ فرسایش، تخریب مرتع
NDSI (برف)	$\frac{Green - SWIR1}{Green + SWIR1}$	پوشش برف

ضریب ۰٫۵ در SAVI و عددهای ثابت EVI برای بازتاب (۰ تا ۱) تنظیم شده‌اند؛ پس روی «همه‌ی باندها» که بازتاب × ۱۰۰۰۰ دارد، اول بر ۱۰۰۰۰ تقسیم کنید (فصل ۳۳).

۳۵-۸ انتخاب شاخص

پرسش	شاخص
سلامت و سبزی گیاه، مقایسه‌ی قطعه‌ها	NDVI
پوشش تُنُک در منطقه‌ی خشک	SAVI
جنگل یا مزرعه‌ی متراکم	EVI، NDRE
تنش آبی و خشکسالی کشاورزی	NDMI
آب سطحی، سیل، آبگیر	NDWI؛ در شهر MNDWI
سطح و شدت آتش‌سوزی	NBR و dNBR
گسترش شهر و ساخت‌وساز	NDBI (با NDVI)
خاک لخت و فرسایش	BSI
برف	NDSI

۳۵-۹ محاسبه در سامانه

- **NDVI به‌عنوان تصویر:** محصول «شاخص گیاهی NDVI» در «دریافت تصویر تازه» (فصل ۳۴)، با رنگ‌بندی آماده‌ی قرمز-زرد-سبز.
- **پنج شاخص به‌عنوان عدد هر قطعه:** NBR، NDMI، NDWI، NDVI و NDBI در «تحلیل و سری زمانی» برای هر چندضلعی و هر تاریخ (فصل ۳۶).
- **بقیه:** در QGIS با Raster Calculator روی «همه‌ی باندها». برای Sentinel-2، اگر لایه را `s2` نامیده باشید (باند ۱ قرمز، ۲ سبز، ۳ آبی، ۴ NIR، ۵ لبه‌ی قرمز ۱، ۸ NIR باریک، ۹ SWIR2، ۱۰ SWIR1):

```
NDVI  : ("s2@4" - "s2@1") / ("s2@4" + "s2@1")
SAVI  : 1.5 * ("s2@4" - "s2@1") / ("s2@4" + "s2@1" + 5000)
EVI   : 2.5 * ("s2@4" - "s2@1") / ("s2@4" + 6 * "s2@1" - 7.5 * "s2@3" +
10000)
NDRE  : ("s2@8" - "s2@5") / ("s2@8" + "s2@5")
MNDWI : ("s2@2" - "s2@9") / ("s2@2" + "s2@9")
BSI   : (("s2@9" + "s2@1") - ("s2@4" + "s2@3")) / (("s2@9" + "s2@1") +
("s2@4" + "s2@3"))
```

در SAVI و EVI، به جای تقسیم همه‌ی باندها بر ۱۰۰۰۰، عدد ثابت را در ۱۰۰۰۰ ضرب کرده‌ایم (۰٫۵ × ۱۰۰۰۰ = ۵۰۰۰ و ۱ × ۱۰۰۰۰ = ۱۰۰۰۰) که نتیجه‌ی یکسانی دارد.

۳۵-۱۰ محاسبه با SQL روی رستر

اگر «همه‌ی باندها» را با raster2pgsql وارد پایگاه کنید (فصل ۲۳)، شاخص با `ST_MapAlgebra` دوباندی ساخته می‌شود:

```sql
SELECT ST_MapAlgebra(rast, 4, rast, 1,                    -- باند ۴ و ۱ (قرمز) (NIR)
        'CASE WHEN [rast1] + [rast2] = 0 THEN NULL
              ELSE ([rast1] - [rast2])::float / ([rast1] + [rast2])
END',
        '32BF') AS ndvi
FROM s2_allbands;
```

`CASE` از تقسیم بر صفر (پیکسل‌های بی‌داده) جلوگیری می‌کند و `:: float` نتیجه را اعشاری نگه می‌دارد (فصل ۱۰، تقسیم صحیح).

در دنیای واقعی

آبیاری نامتوازن. در یک باغ ۴۰ هکتاری پسته، نقشه‌ی NDMI از تصویر Sentinel-2 نواری با رطوبت کمتر در شمال باغ نشان داد، در حالی که NDVI هنوز تفاوت چشمگیری نداشت. بازدید میدانی یک لوله‌ی اصلی نیمه‌گرفته را پیدا کرد. ده روز بعد، تفاوت در NDVI هم دیده می‌شد؛ NDMI ده روز زودتر هشدار داده بود.

خلاصه‌ی فصل

- شاخص نرمال‌شده $(\rho_1 - \rho_2)/(\rho_1 + \rho_2)$ بین ۱- و ۱ است و اثر روشنایی را کم می‌کند.
- NDVI = (NIR - قرمز)/(NIR + قرمز): سبزی؛ اشباع در پوشش متراکم، حساس به خاک در پوشش تُنُک.
- NDWI و MNDWI آب؛ NDMI رطوبت برگ (هشدار زودهنگام کم‌آبی)؛ NBR و dNBR آتش؛ NDBI ساخت‌وساز.
- BSI، NDRE، EVI، SAVI و NDSI با «همه‌ی باندها» در QGIS یا SQL؛ ثابت‌ها برای بازتاب ۰ تا ۱ تنظیم‌اند.
- تفسیر شاخص در مقایسه معنا پیدا می‌کند: با قطعه‌های هم‌کشت، با سال قبل، با تاریخ قبل.

برای مدرس

زمان پیشنهادی: دو جلسه. جدول «گیاه سالم، تحت تنش، خاک، آب، ابر» را روی تخته پر کنید و از دانشجوها بخواهید پیش از محاسبه، ترتیب NDVIها را حدس بزنند. در جلسه‌ی دوم هر گروه یک شاخص را برای یک منطقه در QGIS بسازد و تفسیرش را ارائه دهد.

تمرین‌ها

۱. سطح ۱ NDVI پیکسلی با بازتاب قرمز ۰٫۰۶ و NIR ۰٫۳۸ چقدر است؟ تفسیرش چیست؟

۲. سطح ۱ برای هر پرسش شاخص مناسب را بنویسید: وسعت سیل دیروز، خشکسالی باغ‌ها، ساخت‌وساز غیرمجاز در حریم رود، وضعیت مرتع در منطقه‌ی نیمه‌خشک.

۳. سطح ۲ پرسش SQL نمونه‌ی NDVI را گسترش دهید تا NDWI، NDMI و SAVI را هم برای همان پنج سطح حساب کند (بازتاب SWIR1 و سبز را از نمودار فصل ۳۱ بخوانید).

۴. سطح ۲ چرا NDVI ابر نزدیک صفر است، نه منفی؟ این چه خطری برای تحلیل مزرعه دارد؟

۵. سطح ۳ از «همه‌ی باندها»ی یک منطقه‌ی کشاورزی در QGIS، NDVI و EVI بسازید و در پیکسل‌های پرپوشش مقایسه کنید. اشباع NDVI را می‌بینید؟

فصل

۳۶

آمار هر قطعه؛ از تصویر به جدول

سطح: **متوسط** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل‌های ۲۳ و ۳۵**

در این فصل یاد می‌گیرید

- مفهوم آمار ناحیه‌ای و چالش‌های آن (پیکسل لبه، پیکسل مختلط، ابر) را توضیح دهید.
- در سامانه برای یک لایه‌ی چندضلعی، آمار شاخص طیفی را در یک بازه‌ی زمانی بگیرید.
- ساختار جدول نتیجه و نمای «آخرین مقدار» را بشناسید.
- با SQL نتیجه را فیلتر کیفی، رتبه‌بندی و با ویژگی‌های قطعه‌ها ترکیب کنید.
- قطعه‌های غیرعادی را نسبت به قطعه‌های هم‌گروه پیدا کنید.

سرفصل‌ها

۳۶-۱ عدد هر قطعه، نه عکس

۳۶-۲ گام‌به‌گام: تحلیل تازه

۳۶-۳ جدول نتیجه

۳۶-۴ پرسیدن از نتیجه

۳۶-۵ نقشه‌ی سلامت قطعه‌ها

۳۶-۱ عدد هر قطعه، نه عکس

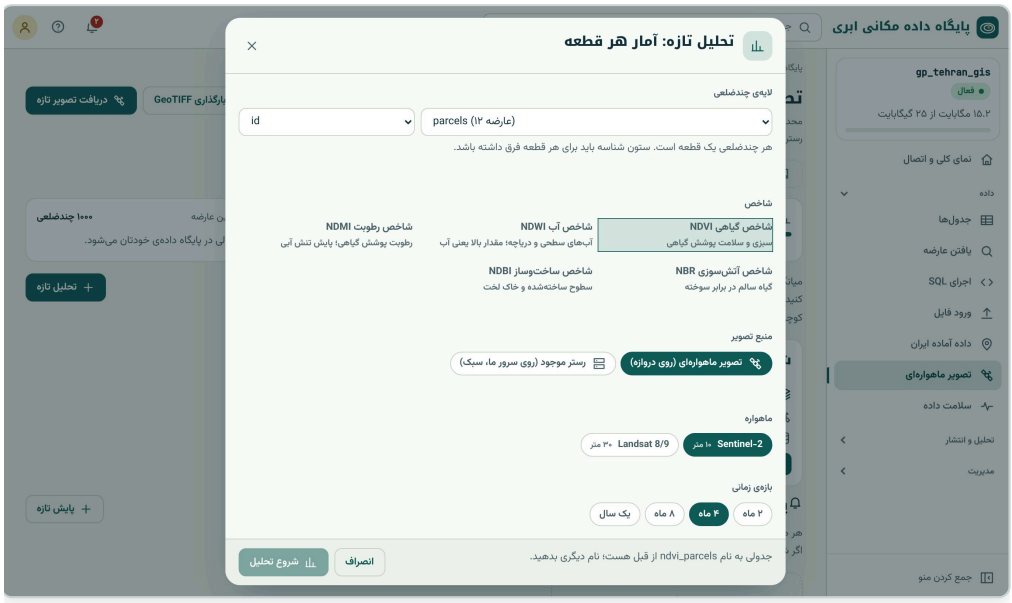
کشاورز، کارشناس بیمه یا مدیر طرح آبیاری معمولاً تصویر نمی‌خواهد؛ می‌خواهد بداند **وضع هر قطعه** چطور است: «کدام باغ این ماه خشک‌تر شده؟»، «میانگین NDVI مزرعه‌ی شماره‌ی ۷ چقدر است؟». **آمار ناحیه‌ای** (zonal statistics) پاسخ همین است: برای هر چندضلعی، آمار پیکسل‌های داخلش (میانگین، میانه، کمینه، بیشینه، انحراف معیار). در فصل ۲۳ همین را با SQL روی رستر داخل پایگاه دیدیم؛ اینجا سامانه آن را مستقیم روی تصویرهای ماهواره‌ای و برای **چند تاریخ** انجام می‌دهد، بدون اینکه تصویری دانلود یا در پایگاه ذخیره کنید.

چالش‌های آمار ناحیه‌ای

- **پیکسل لبه:** پیکسل‌هایی که بخشی‌شان داخل قطعه است. سامانه پیکسل‌هایی را حساب می‌کند که مرکزشان داخل چندضلعی است.
- **پیکسل مختلط:** پیکسل‌های لبه بازتاب جاده یا قطعه‌ی همسایه را هم دارند (فصل ۳۲). قطعه‌ی کوچک‌تر از ده‌ها پیکسل نتیجه‌ی قابل‌اعتمادی ندارد. برای قطعه‌های کوچک، می‌توانید پیش از تحلیل یک **حریم منفی** ۱۰ متری بزنید (فصل ۱۸) تا پیکسل‌های لبه حذف شوند.
- **ابر:** پیکسل‌های ابری حذف می‌شوند و درصد پیکسل‌های معتبر گزارش می‌شود؛ تاریخی که بیشتر پیکسل‌های قطعه‌اش ابری است، قابل اعتماد نیست.
- **میانگین یا میانه:** میانه در برابر پیکسل‌های پرت (یک درخت تنها، یک ساختمان، ابر جامانده) مقاوم‌تر است. هر دو در نتیجه هست.

۳۶-۲ گام‌به‌گام: تحلیل تازه

زبانه‌ی «تحلیل و سری زمانی» در بخش «تصویر ماهواره‌ای» ← «تحلیل تازه»:



مودال «تحلیل تازه: آمار هر قطعه». لایه‌ی چندضلعی، ستون شناسه، شاخص، منبع تصویر، ماهواره و بازه‌ی زمانی.

۱ **لایه‌ی چندضلعی:** جدولی که هر سطرش یک قطعه است (اینجا `parcels`، ۱۲ عارضه). تا ۱۰۰۰ قطعه.

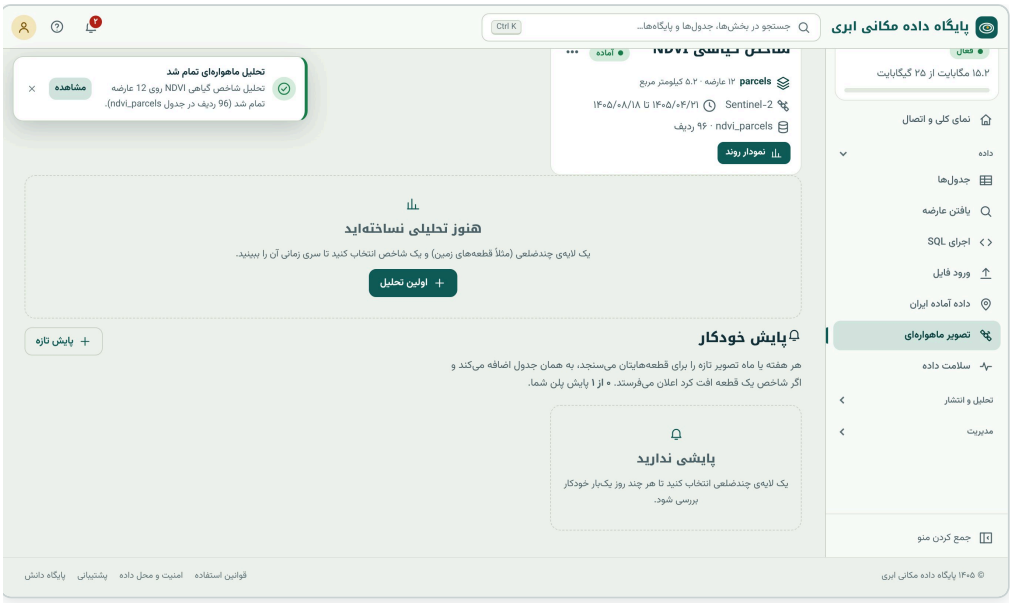
۲ **ستون شناسه:** ستونی که برای هر قطعه یکتاست (اینجا `id`)؛ در جدول نتیجه با نام `feature_id` می‌آید.

۳ **شاخص:** NDVI (سبزی گیاه)، NDWI (آب)، NDMI (رطوبت گیاه)، NBR (سوختگی) یا NDBI (ساخت‌وساز) (فصل ۳۵).

۴ **منبع تصویر:** «تصویر ماهواره‌ای (روی دروازه)» که یک اعتبار تحلیل مصرف می‌کند، یا «رستری که از قبل دارید» (مثلاً NDVI که قبلاً دریافت کرده‌اید) که روی سرور سامانه، بدون اعتبار و فقط وقتی سرور مشغول نیست، اجرا می‌شود.

۵ **ماهواره و بازه‌ی زمانی:** Sentinel-2 یا Landsat؛ ۲ ماه، ۴ ماه، ۸ ماه یا یک سال. هر تصویر کم‌ابر در بازه یک «تاریخ» در نتیجه می‌شود.

۶ «شروع تحلیل». پایان کار با اعلان خبر داده می‌شود.



کارت تحلیل آماده: شاخص، لایه، تعداد قطعه، بازه‌ی تاریخ، ماهواره و نام جدول نتیجه (۹۶ ردیف در ndvi_parcels). اعلان بالای صفحه پایان کار را خبر داده است.

۳۶-۳ جدول نتیجه

تحلیل عکس نمی‌سازد؛ **دو شیء در پایگاه داده‌ی شما** می‌سازد که مثل هر جدول دیگری با QGIS، SQL و استودیوی نقشه قابل استفاده‌اند:

1. **جدول بلند** (مثلاً `ndvi_parcels`): یک سطر برای هر قطعه و هر تاریخ.
2. **نمای «آخرین مقدار»** (`ndvi_parcels_latest`): همان قطعه‌ها با هندسه، و آخرین مقدار شاخص هر قطعه؛ آماده‌ی رنگ‌آمیزی روی نقشه.

```sql
SELECT feature_id, scene_date, round(mean::numeric, 3) AS mean,
round(median::numeric, 3) AS median,
       round(min::numeric, 3) AS min, round(max::numeric, 3) AS max,
round(std::numeric, 3) AS std,
       valid_pct, n_pixels, n_valid
FROM ndvi_parcels
ORDER BY feature_id, scene_date
LIMIT 4;
```

cene_date	mean	median	min	max	std	valid_pct	n_pixels	n_valid
026-07-12	0.300	0.301	0.195	0.394	0.030	100.0	3698	3698
026-07-28	0.451	0.451	0.344	0.546	0.030	100.0	3698	3698
026-08-13	0.600	0.599	0.496	0.704	0.031	100.0	3698	3698
026-08-29	0.680	0.680	0.558	0.784	0.030	100.0	3698	3698

ستون	معنا
`feature_id`	شناسه‌ی قطعه (از ستون شناسه‌ای که انتخاب کردید)
`scene_date`	تاریخ تصویر
`scene`	شناسه‌ی صحنه‌ی ماهواره‌ای
`mean` ، `median` ، `min` ، `max` ، `std`	آمار شاخص روی پیکسل‌های معتبر قطعه
`valid_pct`	درصد پیکسل‌های بی‌ابر
`n_pixels` ، `n_valid`	تعداد کل پیکسل‌ها و پیکسل‌های معتبر

نکته

داده‌ی این فصل. نتیجه‌هایی که در این فصل و فصل بعد می‌بینید از محیط نمایشی کتاب آمده‌اند و برای آموزش ساخته شده‌اند. اگر به تصویر ماهواره‌ای دسترسی ندارید، فایل `ndvi_parcels.sql` همراه کتاب همین جدول را در پایگاه شما می‌سازد تا پرسش‌ها را تمرین کنید.

۳۶-۴ پرسیدن از نتیجه

آخرین وضع هر قطعه، با کیفیت: فقط تاریخ‌هایی که دست‌کم ۸۰ درصد پیکسل‌ها معتبرند:

```sql
SELECT DISTINCT ON (feature_id)
       feature_id, scene_date, round(mean::numeric, 3) AS ndvi,
valid_pct
FROM ndvi_parcels
WHERE valid_pct ⩾ 80
ORDER BY feature_id, scene_date DESC;
```

feature_id	scene_date	ndvi	valid_pct
P01	2026-11-01	0.570	100.0
P02	2026-11-01	0.525	100.0
P03	2026-11-01	0.476	100.0
P04	2026-11-01	0.502	100.0
P05	2026-11-01	0.615	100.0
P06	2026-11-01	0.541	100.0
P07	2026-11-01	0.368	100.0
P08	2026-11-01	0.456	100.0
P09	2026-11-01	0.553	100.0
P10	2026-11-01	0.604	100.0
P11	2026-11-01	0.345	100.0
P12	2026-11-01	0.577	100.0

`DISTINCT ON (feature_id) … ORDER BY feature_id, scene_date DESC` برای هر قطعه فقط تازه‌ترین سطر را نگه می‌دارد؛ همان الگویی که نمای `_latest` با آن ساخته شده است (فصل ۲۴).

رتبه‌ی قطعه‌ها در آخرین تاریخ و مقایسه با میانگین همه:

```sql
WITH last AS (
  SELECT feature_id, mean FROM ndvi_parcels
  WHERE scene_date = (SELECT max(scene_date) FROM ndvi_parcels)
)
SELECT feature_id,
       round(mean::numeric, 3)
AS ndvi,
       rank() OVER (ORDER BY mean DESC)
AS rank,
       round(((mean - avg(mean) OVER ()) / stddev_samp(mean) OVER
())::numeric, 2) AS z_vs_peers
FROM last
ORDER BY mean
LIMIT 5;
```

feature_id	ndvi	rank	z_vs_peers
P11	0.345	12	-1.92
P07	0.368	11	-1.65
P08	0.456	10	-0.63
P03	0.476	9	-0.41
P04	0.502	8	-0.10

ستون `z_vs_peers` نشان می‌دهد هر قطعه چند انحراف معیار از میانگین قطعه‌های دیگر فاصله دارد. قطعه‌ای با z کمتر از ۱٫۵- در همان تاریخ و با همان کشت، کاندید بازدید میدانی است. این مقایسه‌ی **هم‌زمان با هم‌گروه‌ها** اثر فصل و آب‌وهوا را که روی همه یکسان است حذف می‌کند.

پیوند با ویژگی‌ها و هندسه‌ی قطعه‌ها: چون نتیجه یک جدول معمولی است، با جدول قطعه‌ها پیوند می‌خورد (فصل ۱۳):

```sql
SELECT p.id, p.name,
       round((ST_Area(p.geom::geography) / 10000)::numeric, 1) AS
hectares,
       round(avg(r.mean)::numeric, 3)                          AS
season_mean_ndvi,
       round(max(r.mean)::numeric, 3)                          AS
peak_ndvi
FROM parcels p
JOIN ndvi_parcels r ON r.feature_id = p.id
WHERE r.valid_pct ⩾ 80
GROUP BY p.id, p.name, p.geom
ORDER BY season_mean_ndvi
LIMIT 5;
```

id	name	hectares	season_mean_ndvi	peak_ndvi
P08	قطعه 8	37.1	0.445	0.545
P11	قطعه 11	37.0	0.479	0.612
P04	قطعه 4	37.1	0.489	0.598
P02	قطعه 2	37.1	0.511	0.626
P06	قطعه 6	37.1	0.527	0.645

۳۶-۵ نقشه‌ی سلامت قطعه‌ها

نمای `ndvi_parcels_latest` هندسه‌ی قطعه‌ها و ستون‌های `sat_date`، `sat_ndvi` و `sat_valid_pct` دارد. آن را در **استودیوی نقشه‌ی چندلایه** (فصل ۴۲) با طبقه‌بندی عددی روی `sat_ndvi` و طیف قرمز-زرد-سبز نمایش دهید، یا در QGIS با Graduated symbology. هر بار که تحلیل یا پایش تازه‌ای به جدول اضافه شود، نما خودکار به‌روز می‌شود و نقشه هم.

سطح حرفه‌ای

وزن مساحت در تجمیع. اگر چند قطعه متعلق به یک بهره‌بردار است و میانگین NDVI او را می‌خواهید، میانگین ساده‌ی قطعه‌ها قطعه‌ی کوچک و بزرگ را هم‌وزن می‌کند. میانگین درست، وزن تعداد پیکسل معتبر را دارد: `sum(mean * n_valid) / sum(n_valid)`.

در دنیای واقعی

تخصیص آب. شرکت آب منطقه‌ای برای یک شبکه‌ی آبیاری با ۸۰۰ قطعه، هر هفته NDMI همه‌ی قطعه‌ها را با «آمار هر قطعه» می‌گیرد. پرسشی ساده که قطعه‌های با NDMI بسیار پایین‌تر از هم‌کشت‌هایشان را فهرست می‌کند، به ناظران میدانی می‌گوید کجا بروند؛ به جای بازدید از ۸۰۰ قطعه، هر هفته حدود ۴۰ قطعه.

خلاصه‌ی فصل

- آمار ناحیه‌ای = آمار پیکسل‌های داخل هر چندضلعی؛ با پیکسل لبه، پیکسل مختلط و ابر مراقب باشید.
- «تحلیل تازه»: لایه، ستون شناسه، شاخص (NDVI، NDWI، NDMI، NBR، NDBI)، منبع، ماهواره و بازه؛ تا ۱۰۰۰ قطعه.
- نتیجه: جدول بلند (یک سطر برای هر قطعه و تاریخ با mean، median، min، max، std، valid_pct) و نمای `_latest` با هندسه.
- با `valid_pct` تاریخ‌های ابری را کنار بگذارید؛ `DISTINCT ON` آخرین مقدار، تابع پنجره‌ای رتبه و z نسبت به هم‌گروه‌ها.
- نتیجه جدولی معمولی است: پیوند با قطعه‌ها، نقشه در استودیو و QGIS.

برای مدرس

زمان پیشنهادی: یک جلسه. پیش از اجرای تحلیل، از دانشجوها بخواهید روی تصویر فروسرخ کاذب حدس بزنند کدام قطعه ضعیف‌تر است؛ بعد با جدول نتیجه حدس‌ها را بسنجند. پرسش z نسبت به هم‌گروه‌ها پایه‌ی خوبی برای بحث درباره‌ی «مقایسه‌ی منصفانه» است.

تمرین‌ها

۱. سطح ۱ برای جدول `parcels` یک تحلیل NDVI در ۴ ماه اخیر بگیرید (یا فایل `ndvi_parcels.sql` را اجرا کنید). جدول چند سطر دارد؟

۲. سطح ۱ کدام تاریخ کمترین میانگین `valid_pct` را دارد؟ چرا؟

۳. سطح ۲ برای هر قطعه، تاریخ اوج NDVI و مقدار آن را بدهید (`DISTINCT ON` با ترتیب `mean DESC`).

۴. سطح ۲ نمای `ndvi_parcels_latest` را در استودیوی نقشه با طبقه‌بندی پنج‌طبقه‌ای نمایش دهید.

۵. سطح ۳ با حریم منفی ۱۰ متری، لایه‌ی `parcels_inner` بسازید و تحلیل را روی آن تکرار کنید. میانگین‌ها چقدر تغییر کرد؟ چرا؟

فصل

۳۷

سری زمانی، فنولوژی و پایش خودکار

سطح: **متوسط** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل‌های ۱۲ و ۳۶**

در این فصل یاد می‌گیرید

- منحنی فنولوژی گیاه (سبز شدن، اوج، پیری، برداشت) را از سری زمانی NDVI بخوانید.
- نمودار روند سامانه را برای همه‌ی قطعه‌ها و یک قطعه تفسیر کنید.
- با SQL تغییر نسبت به تاریخ قبل، میانگین متحرک، تاریخ اوج و روند را حساب کنید.
- پایش خودکار با هشدار افت شاخص بسازید و منطق آن را با SQL بازسازی کنید.
- خطاهای رایج تفسیر سری زمانی را بشناسید.

سرفصل‌ها

۳۷-۱ زمان، بُعد چهارم

۳۷-۲ فنولوژی

۳۷-۳ نمودار روند در سامانه

۳۷-۴ سری زمانی با SQL

۳۷-۵ پایش خودکار

۳۷-۱ زمان، بُعد چهارم

یک تصویر عکسی از یک لحظه است. اما گیاه زنده است: سبز می‌شود، رشد می‌کند، به اوج می‌رسد، زرد می‌شود و برداشت می‌شود. **سری زمانی** شاخص طیفی این داستان را روایت می‌کند و بسیاری از پرسش‌ها فقط با آن پاسخ می‌گیرند: «این مزرعه کی کاشته شد؟»، «امسال از پارسال بهتر است؟»، «چرا این قطعه یک‌باره افت کرد؟».

۳۷-۲ فنولوژی

فنولوژی (phenology) مطالعه‌ی رویدادهای چرخه‌ی زندگی گیاه در ارتباط با فصل است. منحنی NDVI یک محصول زراعی در یک فصل معمولاً این مراحل را دارد:

1. **پیش از کاشت یا خاک لخت:** NDVI پایین (۰٫۱ تا ۰٫۲).
2. **سبز شدن** (green-up): افزایش تند NDVI با رشد برگ‌ها.
3. **اوج** (peak): بیشترین پوشش برگ؛ برای گندم حدود زمان خوشه‌دهی.
4. **پیری** (senescence): زرد شدن برگ‌ها و کاهش تدریجی NDVI.
5. **برداشت:** افت ناگهانی به سطح خاک.

شکل منحنی برای هر محصول متفاوت است و همین تفاوت، پایه‌ی **تشخیص نوع کشت** از روی سری زمانی است: گندم پاییزه در اسفند و فروردین اوج می‌گیرد، ذرت تابستانه در مرداد، و باغ و یونجه تقریباً همه‌ی سال سبزند. افت ناگهانی وسط فصل رشد، که با چرخه‌ی طبیعی جور نیست، **هشدار** است: کم‌آبی، آفت، سرمازدگی، تگرگ یا برداشت زودهنگام.

۳۷-۳ نمودار روند در سامانه

روی کارت هر تحلیل (فصل ۳۶) دکمه‌ی «نمودار روند» هست. نمودار به‌طور پیش‌فرض **میانگین همه‌ی قطعه‌ها** را با نوار **کمینه تا بیشینه** نشان می‌دهد و با فهرست بالای آن می‌توان **یک قطعه‌ی مشخص** را کنار میانگین دید. تاریخ‌ها شمسی‌اند.

نمودار روند NDVI: میانگین همه‌ی قطعه‌ها (سبزآبی)، نوار کمینه تا بیشینه (خاکستری) و قطعه‌ی P07 (نارنجی). قطعه‌ی P07 تا مهرماه همراه بقیه است، اما در آبان افتی تندتر از میانگین دارد.

منحنی میانگین یک فصل رشد کامل را نشان می‌دهد: سبز شدن در تیر، اوج در شهریور و پیری تدریجی تا آبان. قطعه‌ی P07 تا مهر هم‌پای بقیه است، اما در آخرین تاریخ افتی محسوس دارد؛ دقیقاً همان چیزی که پایش خودکار باید خبر بدهد.

۳۷-۴ سری زمانی با SQL

جدول نتیجه یک سری زمانی «بلند» است (یک سطر برای هر قطعه و تاریخ) و توابع پنجره‌ای (فصل ۱۲) ابزار طبیعی کار با آن‌اند.

تغییر نسبت به تاریخ قبل

```sql
SELECT feature_id, scene_date,
       round(mean::numeric, 3)
AS ndvi,
       round((mean - lag(mean) OVER w)::numeric, 3)
AS change,
       round((100 * (mean - lag(mean) OVER w) / lag(mean) OVER
w)::numeric, 1)              AS change_pct
FROM ndvi_parcels
WHERE feature_id = 'P07'
WINDOW w AS (PARTITION BY feature_id ORDER BY scene_date)
ORDER BY scene_date;
```

feature_id	scene_date	ndvi	change	change_pct
P07	2026-07-12	0.306	NULL	NULL
P07	2026-07-28	0.460	0.153	50.1
P07	2026-08-13	0.613	0.153	33.3
P07	2026-08-29	0.694	0.082	13.4
P07	2026-09-14	0.674	-0.021	-3.0
P07	2026-09-30	0.612	-0.061	-9.1
P07	2026-10-16	0.592	-0.021	-3.4
P07	2026-11-01	0.368	-0.224	-37.9

بند `WINDOW w AS (…)` تعریف پنجره را یک بار می‌نویسد تا در چند ستون تکرار نشود. افت آخرین تاریخ P07 حدود ۳۸ درصد است، در حالی که افت‌های پیش از آن (پیری طبیعی) زیر ۱۰ درصد بودند.

میانگین متحرک و تاریخ اوج

میانگین متحرک نوسان‌های تصادفی (پیکسل ابری جامانده، تفاوت زاویه‌ی دید) را هموار می‌کند:

```sql
WITH s AS (
  SELECT feature_id, scene_date, mean,
         avg(mean) OVER (PARTITION BY feature_id ORDER BY scene_date
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS ma3
  FROM ndvi_parcels
)
SELECT DISTINCT ON (feature_id)
       feature_id, scene_date AS peak_date, round(ma3::numeric, 3) AS
peak_ndvi_ma3
FROM s
ORDER BY feature_id, ma3 DESC;
```

feature_id	peak_date	peak_ndvi_ma3
P01	2026-09-30	0.647
P02	2026-09-30	0.595
P03	2026-09-14	0.680
P04	2026-09-30	0.570
P05	2026-09-30	0.699
P06	2026-09-14	0.615
P07	2026-09-14	0.660
P08	2026-09-30	0.518
P09	2026-09-30	0.628
P10	2026-09-14	0.686
P11	2026-09-30	0.582
P12	2026-09-14	0.654

تاریخ اوج هر قطعه را داریم؛ اگر قطعه‌ای خیلی زودتر یا دیرتر از بقیه به اوج برسد، احتمالاً تاریخ کاشت یا نوع کشتش فرق دارد.

روند کلی

شیب رگرسیون خطی NDVI بر حسب زمان، روند کلی را در یک عدد می‌دهد (اینجا برای دو ماه آخر، یعنی مرحله‌ی پیری):

```sql
SELECT feature_id,
       round((regr_slope(mean, scene_date - date '2026-01-01') *
30)::numeric, 3) AS ndvi_change_per_month,
       round(regr_r2(mean, scene_date - date '2026-01-01')::numeric, 2)
AS r2
FROM ndvi_parcels
WHERE scene_date ⩾ (SELECT max(scene_date) - 60 FROM ndvi_parcels)
GROUP BY feature_id
ORDER BY ndvi_change_per_month
LIMIT 5;
```

feature_id	ndvi_change_per_month	r2
P07	-0.176	0.82
P11	-0.144	0.84
P03	-0.127	0.90
P05	-0.059	0.87
P10	-0.058	0.86

`regr_slope(y, x)` شیب خط را می‌دهد؛ چون x به روز است، در ۳۰ ضرب کرده‌ایم تا «تغییر در ماه» بخوانیم. `regr_r2` نشان می‌دهد خط چقدر داده را توضیح می‌دهد.

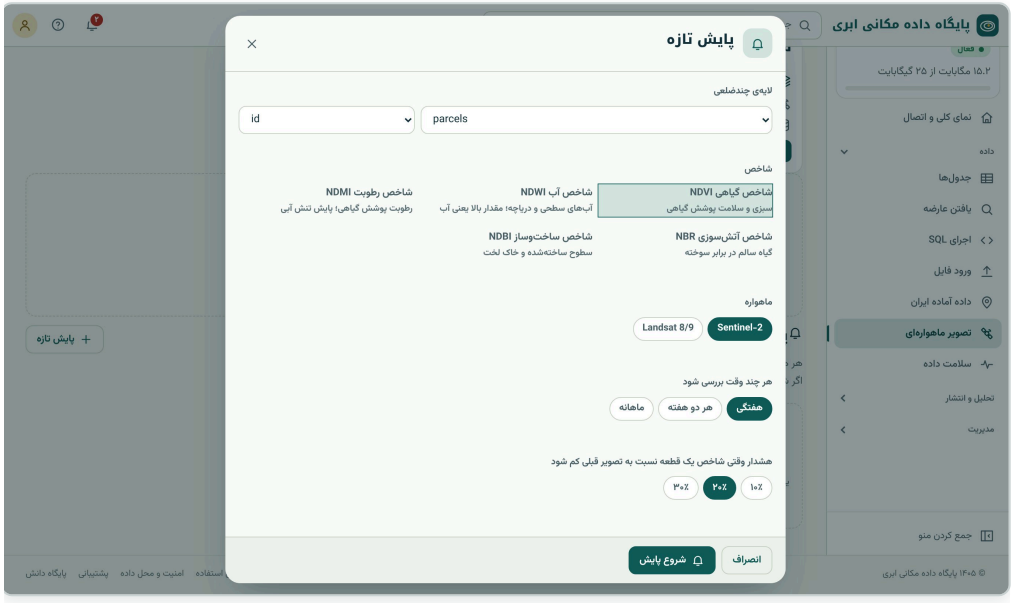
هشدار

خطاهای رایج تفسیر. - **مقایسه‌ی دو تاریخ دلخواه:** «NDVI امروز از ماه پیش کمتر است» ممکن است فقط پیری طبیعی باشد. مقایسه را با قطعه‌های هم‌کشت (فصل ۳۶) یا با همان تاریخ سال قبل انجام دهید. - **تاریخ ابری:** یک تاریخ با پیکسل‌های ابری جامانده افت کاذب می‌سازد؛ `valid_pct` را بسنجید و به میانگین متحرک یا میانه نگاه کنید. - **تغییر ماهواره:** مقدار NDVI حاصل از Landsat و Sentinel-2 برای یک سطح اندکی متفاوت است (باندها کاملاً یکسان نیستند). یک سری زمانی را با یک ماهواره بسازید.

۳۷-۵ پایش خودکار

تحلیل یک‌باره گذشته را نشان می‌دهد؛ **پایش خودکار** آینده را می‌پاید. پایش، لایه‌ی قطعه‌ها را با یک شاخص در فاصله‌های منظم بررسی می‌کند، تصویرهای تازه را به همان جدول اضافه می‌کند

و اگر شاخص قطعه‌ای افت کند، **اعلان** می‌فرستد.



مودال «پایش تازه»: لایه، ستون شناسه، شاخص، ماهواره، تناوب بررسی و درصد افتی که هشدار می‌دهد.

۱. زبانه‌ی «تحلیل و سری زمانی» ← «پایش تازه».
۲. لایه‌ی چندضلعی و ستون شناسه (تا ۱۰۰۰ قطعه).
۳. شاخص: برای سلامت گیاه NDVI، برای تنش آبی NDMI.
۴. ماهواره (برای سری زمانی یکدست، همانی که تحلیل اولیه با آن گرفته شده).
۵. **هر چند وقت بررسی شود:** هفتگی، هر دو هفته یا ماهانه.
۶. **هشدار وقتی شاخص یک قطعه نسبت به تصویر قبل کم شود:** ۱۰، ۲۰ یا ۳۰ درصد.
۷. «شروع پایش».

قاعده‌های پایش:

- هر بار فقط تصویرهای **تازه‌تر** از آخرین تصویر پردازش‌شده سنجیده و به جدول اضافه می‌شوند؛ تاریخچه کامل می‌ماند.
- مقایسه با **تصویر بی‌ابر قبلی** همان قطعه است.

- قطعه‌ای که **کمتر از نیمی** از پیکسل‌هایش بی‌ابر است در مقایسه نمی‌آید (تا ابر هشدار کاذب نسازد).
- پایش از اعتبار تحلیل کم نمی‌کند؛ سقفش «تعداد پایش» پلن است. از منوی ⋯ کارت پایش می‌توان آن را متوقف، از سر گرفت یا همان لحظه اجرا کرد.
- هشدار در **مرکز اعلان‌ها** (زنگوله) با نام قطعه‌ها می‌آید.

منطق پایش با SQL

دانستن منطق پایش کمک می‌کند هشدارها را بفهمید یا قاعده‌ی خودتان را بسازید. همان قاعده‌ی «افت دست‌کم ۲۰ درصد نسبت به تصویر بی‌ابر قبلی، فقط با بیش از نیمی پیکسل معتبر» به SQL:

```sql
WITH clear AS (
  SELECT feature_id, scene_date, mean,
         lag(mean)       OVER (PARTITION BY feature_id ORDER BY
scene_date) AS prev_mean,
         lag(scene_date) OVER (PARTITION BY feature_id ORDER BY
scene_date) AS prev_date
  FROM ndvi_parcels
  WHERE valid_pct ⩾ 50
)
SELECT feature_id, prev_date, scene_date,
       round(prev_mean::numeric, 3) AS before, round(mean::numeric, 3)
AS after,
       round((100 * (prev_mean - mean) / prev_mean)::numeric, 1) AS
drop_pct
FROM clear
WHERE scene_date = (SELECT max(scene_date) FROM ndvi_parcels)
  AND mean ⩽ prev_mean * 0.80
ORDER BY drop_pct DESC;
```

feature_id	prev_date	scene_date	before	after	drop_pct
P07	2026-10-16	2026-11-01	0.592	0.368	37.9
P11	2026-10-16	2026-11-01	0.521	0.345	33.8
P03	2026-10-16	2026-11-01	0.610	0.476	22.0

سه قطعه هشدار می‌گیرند. P07 و P11 همان‌هایی‌اند که در فصل ۳۶ به‌طور محسوسی پایین‌تر از هم‌گروه‌ها بودند (z حدود ۱٫۷- و ۱٫۹-). اما P03 با z حدود ۰٫۴- در آخرین تاریخ هنوز نزدیک میانگین است: افتش بخشی از پیری طبیعی آخر فصل است که در این قطعه کمی تندتر رخ داده. اگر شرط «و z نسبت به هم‌گروه‌ها کمتر از ۱-» (فصل ۳۶) را به قاعده اضافه کنید، P03 کنار می‌رود و فقط دو هشدار واقعی می‌ماند؛ قاعده‌ای که برای آخر فصل رشد دقیق‌تر است (تمرین ۴). با تغییر عدد `0.80` هم حساسیت را بسنجید.

در دنیای واقعی

هشدار سرمازدگی. در یک تعاونی باغداری، پایش هفتگی NDVI روی ۳۵۰ باغ تعریف شده است. یک هفته پس از سرمای دیررس فروردین، اعلان پایش ۲۷ باغ را با افت بیش از ۲۰ درصد فهرست کرد. کارشناسان پیش از آنکه باغداران ادعای خسارت ثبت کنند، از همان ۲۷ باغ بازدید کردند و گزارش خسارت در یک هفته به جای یک ماه آماده شد.

خلاصه‌ی فصل

- سری زمانی چرخه‌ی زندگی گیاه را نشان می‌دهد: خاک، سبز شدن، اوج، پیری، برداشت؛ شکل منحنی نوع کشت را آشکار می‌کند.
- نمودار روند سامانه میانگین قطعه‌ها، نوار کمینه-بیشینه و یک قطعه‌ی مشخص را نشان می‌دهد.
- با SQL: `lag` برای تغییر، میانگین متحرک با `ROWS BETWEEN` ، `DISTINCT ON` برای اوج، `regr_slope` برای روند.
- مقایسه را با هم‌گروه‌ها یا سال قبل انجام دهید؛ ابر و تغییر ماهواره را کنترل کنید.
- پایش خودکار: تناوب (هفتگی، دوهفته، ماهانه) و آستانه‌ی افت (۱۰، ۲۰، ۳۰٪) نسبت به تصویر بی‌ابر قبلی؛ قطعه‌های با کمتر از نیمی پیکسل معتبر کنار گذاشته می‌شوند؛ هشدار در اعلان‌ها.

برای مدرس

زمان پیشنهادی: دو جلسه. منحنی فنولوژی گندم، ذرت و یونجه را روی تخته بکشید و از دانشجوها بخواهید از روی نمودار روند یک قطعه‌ی واقعی حدس بزنند چه کشتی است. پرسش «منطق پایش» را با آستانه‌های مختلف اجرا کنید تا مفهوم هشدار کاذب و از دست رفته (دقت و فراخوانی) ملموس شود.

تمرین‌ها

۱. سطح ۱ نمودار روند تحلیل خود را باز کنید. اوج فصل در چه تاریخی است؟ کدام قطعه با بقیه فرق دارد؟

۲. سطح ۲ برای هر قطعه، بیشترین افت بین دو تاریخ متوالی و تاریخ آن را بدهید.

۳. سطح ۲ قاعده‌ی پایش را با آستانه‌ی ۱۰، ۲۰ و ۳۰ درصد اجرا کنید. چند قطعه در هر حالت هشدار می‌گیرد؟

۴. سطح ۳ قاعده‌ی هشدار تازه‌ای بنویسید که فقط وقتی هشدار دهد که افت نسبت به تاریخ قبل دست‌کم ۱۵ درصد **و** z قطعه نسبت به هم‌گروه‌ها در همان تاریخ کمتر از ۱- باشد.

۵. سطح ۳ با `CREATE VIEW` نمایی بسازید که برای هر قطعه، آخرین NDVI، تغییر نسبت به تاریخ قبل و رتبه‌ی آن را بدهد، و آن را با هندسه‌ی قطعه‌ها روی نقشه ببرید.

فصل

۳۸

مدل رقومی ارتفاع و تحلیل زمین

سطح: **متوسط** ●●○

زمان مطالعه: **۹۰ دقیقه**

پیش‌نیاز: **فصل‌های ۳، ۲۳ و ۳۴**

در این فصل یاد می‌گیرید

- DSM، DEM و DTM را از هم تشخیص دهید و منابع رایج مدل ارتفاعی را بشناسید.
- مرجع ارتفاعی (بیضوی یا ژئوئید) هر مدل را بدانید.
- مشتق‌های زمین (شیب، جهت، سایه‌روشن، انحنا، موقعیت توپوگرافی) را تفسیر کنید.
- پروفیل ارتفاعی و منحنی میزان بسازید.
- مفهوم جهت جریان، تجمع جریان و حوضه‌ی آبریز را توضیح دهید و آن را در QGIS اجرا کنید.

سرفصل‌ها

۳۸-۱ سه مدل ارتفاعی

۳۸-۲ دریافت DEM در سامانه

۳۸-۳ مشتق‌های زمین

۳۸-۴ پروفیل ارتفاعی

۳۸-۵ منحنی میزان

۳۸-۶ هیدرولوژی: آب کجا می‌رود

۳۸-۱ سه مدل ارتفاعی

مدل رقومی ارتفاع (Digital Elevation Model، DEM) رستری است که مقدار هر پیکسل ارتفاع آن نقطه است. این اصطلاح عمومی دو نوع مشخص دارد:

- **مدل رقومی سطح** (DSM): ارتفاع بالاترین سطحی که سنجنده می‌بیند؛ بام ساختمان‌ها، تاج درختان.
- **مدل رقومی زمین** (DTM): ارتفاع زمین لخت، پس از حذف ساختمان و پوشش گیاهی.

اختلاف این دو (DSM منهای DTM) **مدل ارتفاع عوارض** است: ارتفاع ساختمان‌ها و درختان؛ پایه‌ی برآورد حجم ساختمانی شهر یا زیست‌توده‌ی جنگل.

منبع	نوع	تفکیک	فناوری	دسترسی
Copernicus DEM GLO-30	DSM	حدود ۳۰ m	رادار تداخل‌سنجی (TanDEM-X)	رایگان؛ محصول «ارتفاع (DEM)» سامانه
SRTM	DSM	۳۰ و ۹۰ m	رادار شاتل (۲۰۰۰)	رایگان
ALOS World 3D	DSM	۳۰ m	استریو نوری	رایگان
لیدار هوایی	DSM و DTM	زیر ۱ m	لیزر	پروژه‌ای، گران
فتوگرامتری پهپاد	DSM	چند سانتی‌متر	عکس هوایی هم‌پوشان	پروژه‌ای
نقشه‌برداری زمینی	DTM	نقطه‌ای	GNSS، توتال استیشن	پروژه‌ای (فصل ۴۹)

بدانید

ارتفاع نسبت به چه؟ فصل ۳ گفت ارتفاع بیضوی (GNSS) با ارتفاع از سطح دریا (ژئوئید) فرق دارد. ارتفاع Copernicus DEM نسبت به **ژئوئید EGM2008** است، یعنی همان ارتفاع از سطح دریا که روی نقشه‌های توپوگرافی می‌بینید. اگر DEM را با ارتفاع خام GPS مقایسه کنید، چند ده متر اختلاف می‌بینید که خطا نیست؛ ارتفاع ژئوئید است.

۳۸-۲ دریافت DEM در سامانه

در «دریافت تصویر تازه» (فصل ۳۴)، محصول **ارتفاع (DEM)** را انتخاب کنید. چون DEM به تاریخ وابسته نیست، گام‌های ماهواره، زمان و انتخاب تصویر لازم نیست. نتیجه COG اعشاری ۳۲ بیتی با پیکسل ۳۰ متری است که روی نقشه با رنگ‌بندی ارتفاعی نمایش داده می‌شود، دانلود می‌شود یا مستقیم در QGIS باز می‌شود. برای تحلیل با SQL، فایل را با raster2pgsql وارد پایگاه کنید (فصل ۲۳) و حتماً پیش از محاسبه‌ی شیب آن را به UTM ببرید (`gdalwarp -t_srs EPSG:32639`).

۳۸-۳ مشتق‌های زمین

در فصل ۲۳ شیب، جهت و سایه‌روشن را با SQL ساختیم. اینجا تفسیرشان را مرور می‌کنیم و چند مشتق دیگر را می‌شناسیم:

مشتق	معنا	کاربرد
شیب	تندی دامنه (درجه یا درصد)	ساخت‌وساز (معمولاً زیر ۱۵٪)، کشاورزی، فرسایش، زمین‌لغزش
جهت	سمت رو به دامنه (۰ تا ۳۶۰)	تابش خورشید، رطوبت خاک، پوشش گیاهی، نیروگاه خورشیدی
سایه‌روشن	نمایش برجستگی با نور فرضی	زمینه‌ی نقشه و تفسیر بصری
انحنا	محدب یا مقعر بودن سطح	جریان آب (همگرا یا واگرا)، فرسایش
TPI (شاخص موقعیت توپوگرافی)	ارتفاع پیکسل منهای میانگین همسایگی	تشخیص قله و دره، طبقه‌بندی شکل زمین
زبری (TRI)	تغییرپذیری ارتفاع در همسایگی	تحرک، زیستگاه

تبدیل شیب: درصد شیب = tan(زاویه) × ۱۰۰. شیب ۴۵ درجه یعنی ۱۰۰ درصد، و شیب ۱۰ درجه حدود ۱۸ درصد است.

TPI را با SQL روی DEM فصل ۲۳ می‌سازیم: میانگین پنجره‌ی ۵×۵ با `ST_MapAlgebra` همسایگی و کم کردن از ارتفاع پیکسل:

```sql
WITH mean5 AS (
  SELECT rast, ST_MapAlgebra(rast, 1, 'st_mean4ma(double precision[],
int[], text[])'::regprocedure, '32BF', 'FIRST', NULL, 2, 2) AS m
  FROM dem
), tpi AS (
  SELECT ST_MapAlgebra(rast, 1, m, 1, '[rast1] - [rast2]', '32BF') AS t
FROM mean5
)
SELECT round((s).min::numeric, 1) AS tpi_min, round((s).max::numeric,
1) AS tpi_max,
       round((s).stddev::numeric, 2) AS tpi_std
FROM (SELECT ST_SummaryStats(t) AS s FROM tpi) x;
```

tpi_min	tpi_max	tpi_std
-2.3	0.9	0.23

TPI مثبت یعنی پیکسل بالاتر از اطرافش است (قله، یال) و منفی یعنی پایین‌تر (دره، گودی). `st_mean4ma` یکی از توابع آماده‌ی همسایگی PostGIS است و دو عدد ۲ آخر اندازه‌ی پنجره را دو پیکسل به هر طرف (۵×۵) تعیین می‌کنند.

۳۸-۴ پروفیل ارتفاعی

پروفیل نمودار ارتفاع در امتداد یک خط است: مسیر جاده، خط لوله یا مسیر پیاده‌روی. با ارجاع خطی (فصل ۱۸) و `ST_Value` ساده است:

```sql
WITH line AS (SELECT ST_SetSRID(ST_MakeLine(ST_MakePoint(531000,
3953000), ST_MakePoint(535500, 3950500)), 32639) AS g)
SELECT round(f * ST_Length(g))
AS distance_m,
       round(ST_Value(d.rast, ST_LineInterpolatePoint(g, f))::numeric,
1)  AS elevation_m
FROM line, dem d, generate_series(0, 1, 0.125) f
ORDER BY f;
```

distance_m	elevation_m
0.0	1398.7
643.0	1576.9
1287.0	1669.3
1930.0	1587.9
2574.0	1435.8
3217.0	1387.6
3861.0	1442.5
4504.0	1429.5
5148.0	1357.1

مسیر از دامنه‌ی تپه‌ی بزرگ بالا می‌رود، از کنار قله می‌گذرد و پایین می‌آید. در استودیوی نقشه، نوع نمودار **پروفیل ارتفاعی** برای لایه‌هایی که خط سه‌بعدی (با Z) دارند همین را با نمودار نشان می‌دهد (فصل ۴۳)؛ و `ST_3DLength` طول واقعی مسیر را با احتساب شیب می‌دهد (فصل ۲۲).

۳۸-۵ منحنی میزان

منحنی میزان (contour) خطی است که نقطه‌های هم‌ارتفاع را به هم وصل می‌کند؛ قدیمی‌ترین و هنوز خواناترین روش نمایش ارتفاع روی نقشه. PostGIS ۳٫۲ به بعد `ST_Contour` دارد:

```sql
SELECT c.value AS elevation_m, count(*) AS lines,
       round((sum(ST_Length(c.geom)) / 1000)::numeric, 1) AS total_km
FROM dem, ST_Contour(rast, 1, level_interval ⇒ 50) c
GROUP BY c.value
ORDER BY c.value;
```

elevation_m	lines	total_km
1250.0	1	6.4
1300.0	1	7.7
1350.0	1	17.0
1400.0	2	13.4
1450.0	2	9.2
1500.0	1	6.1
… (9 rows)		

فاصله‌ی منحنی‌ها (اینجا ۵۰ متر) را **فاصله‌ی منحنی میزان** می‌گویند و به مقیاس نقشه و ناهمواری منطقه بستگی دارد. منحنی‌های نزدیک به هم یعنی شیب تند. ابزار خط فرمان `gdal_contour -a elev -i 10 dem.tif contours.gpkg` و ابزار Contour در QGIS همین کار را روی فایل انجام می‌دهند. برای منحنی میزان از **نقطه‌های برداشت میدانی** (نه DEM)، فصل ۴۹ روش TIN را آموزش می‌دهد.

۳۸-۶ هیدرولوژی: آب کجا می‌رود

آب از بلندی به پستی می‌رود؛ این ساده‌ترین قانون، پایه‌ی **تحلیل هیدرولوژیک** DEM است:

۱. **پر کردن گودال‌ها** (fill sinks): پیکسل‌هایی که از همه‌ی همسایه‌ها پایین‌ترند (اغلب خطای DEM) پر می‌شوند تا آب «گیر» نکند.

۲. **جهت جریان** (flow direction): برای هر پیکسل، همسایه‌ای که آب به آن می‌رود؛ در روش D8، تندترین شیب رو به پایین از میان هشت همسایه.

۳. **تجمع جریان** (flow accumulation): برای هر پیکسل، تعداد پیکسل‌هایی که آبشان از آن می‌گذرد. پیکسل‌هایی با تجمع زیاد **آبراهه**‌اند.

۴. **آبراهه‌ها:** پیکسل‌هایی با تجمع بیش از یک آستانه؛ به خط تبدیل می‌شوند.

۵. **حوضه‌ی آبریز** (watershed): همه‌ی پیکسل‌هایی که آبشان به یک نقطه‌ی خروجی (مثلاً محل سد یا پل) می‌رسد.

PostGIS ابزار هیدرولوژی ندارد، اما QGIS دارد. با DEM دریافت‌شده از سامانه، در QGIS: **Processing** ← **GRASS** ← **r.watershed** (جهت و تجمع جریان و حوضه‌ها) یا ابزارهای **SAGA** (Channel Network، Fill Sinks و Upslope Area). نتیجه‌ی برداری (آبراهه‌ها و مرز حوضه) را با DB Manager به پایگاه داده برگردانید تا با بقیه‌ی لایه‌ها تحلیل شود؛ مثلاً «کدام روستاها داخل حوضه‌ی بالادست این سداند؟» با یک `ST_Within` (فصل ۱۷).

نکته

دید (Viewshed). «از بالای این دکل، کجا دیده می‌شود؟» تحلیل دید با DEM است و برای جانمایی آنتن، برج دیده‌بانی آتش و ارزیابی منظر به کار می‌رود. در QGIS با ابزار Visibility Analysis یا r.viewshed انجام می‌شود. چون Copernicus DEM یک DSM است، ساختمان‌ها و درختان را هم به‌عنوان مانع در نظر می‌گیرد؛ گاهی مزیت و گاهی مشکل.

در دنیای واقعی

مکان‌یابی سیل‌بند. پس از سیل‌های بهاری، یک استانداری می‌خواست بداند کدام روستاها در آبراهه‌های اصلی یا نزدیک آن‌ها هستند. با DEM سی‌متری استان از سامانه، آبراهه‌ها در QGIS با r.watershed استخراج و به پایگاه داده برگردانده شدند. یک پرسش `ST_DWithin` روستاهای تا ۲۰۰ متری آبراهه‌های با حوضه‌ی بالادست بیش از ۵۰ کیلومتر مربع را فهرست کرد: ۴۳ روستا برای بازدید و طراحی سیل‌بند.

خلاصه‌ی فصل

- DSM ارتفاع بالاترین سطح، DTM ارتفاع زمین لخت؛ Copernicus GLO-30 یک DSM ۳۰ متری با ارتفاع نسبت به ژئوئید EGM2008 است.
- محصول «ارتفاع (DEM)» سامانه بدون انتخاب تاریخ دریافت می‌شود؛ برای شیب، آن را به UTM ببرید.
- مشتق‌ها: شیب، جهت، سایه‌روشن، انحنا، TPI (ارتفاع منهای میانگین همسایگی) و زبری.
- پروفیل با `ST_LineInterpolatePoint` و `ST_Value`؛ منحنی میزان با `ST_Contour` یا gdal_contour.
- هیدرولوژی: پر کردن گودال، جهت جریان (D8)، تجمع جریان، آبراهه و حوضه؛ در QGIS با GRASS یا SAGA و بازگشت نتیجه به پایگاه.

برای مدرس

زمان پیشنهادی: دو جلسه. یک DEM از منطقه‌ای کوهستانی آشنا (مثلاً دامنه‌ی البرز) دریافت کنید. در جلسه‌ی اول شیب، جهت و سایه‌روشن را در QGIS بسازید و با تصویر رنگ طبیعی مقایسه کنید (جنگل روی کدام دامنه است؟). در جلسه‌ی دوم حوضه‌ی آبریز یک رودخانه را استخراج کنید.

تمرین‌ها

۱. سطح ۱ شیب ۲۰ درصد چند درجه است؟ شیب ۳۰ درجه چند درصد؟

۲. سطح ۱ برای محل زندگی خود DEM بگیرید. بلندترین و پست‌ترین نقطه‌ی محدوده چه ارتفاعی دارد؟

۳. سطح ۲ پروفیل ارتفاعی خطی دیگر را روی DEM فصل ۲۳ بسازید و طول سه‌بعدی آن را با `ST_3DLength` (با ساختن خط Z از نقطه‌های پروفیل) حساب کنید.

۴. سطح ۲ منحنی‌های میزان ۲۵ متری DEM نمونه را بسازید، با `CREATE TABLE AS` ذخیره و روی نقشه ببینید.

۵. سطح ۳ در QGIS حوضه‌ی آبریز بالادست یک نقطه‌ی دلخواه را با r.watershed بسازید، به پایگاه وارد و مساحتش را با SQL حساب کنید.

فصل

۳۹

طبقه‌بندی تصویر و آشکارسازی تغییر

سطح: **پیشرفته** ●●●

زمان مطالعه: **۱۲۰ دقیقه**

پیش‌نیاز: **فصل‌های ۲۷، ۳۴ و ۳۵**

در این فصل یاد می‌گیرید

- طبقه‌بندی نظارت‌شده، نظارت‌نشده و شیءگرا را مقایسه کنید.
- نمونه‌ی آموزشی را در پایگاه داده طراحی و نگه‌داری کنید.
- با «همه‌ی باندها» و QGIS نقشه‌ی پوشش زمین بسازید و نتیجه را به پایگاه برگردانید.
- صحت طبقه‌بندی را با ماتریس خطا، صحت کلی، صحت تولیدکننده و کاربر و ضریب کاپا بسنجید.
- تغییر بین دو تاریخ را با تفاضل شاخص و مقایسه‌ی پس از طبقه‌بندی آشکار کنید.

سرفصل‌ها

۳۹-۱ از پیکسل تا کلاس

۳۹-۲ سه رویکرد

۳۹-۳ نمونه‌ی آموزشی در پایگاه داده

۳۹-۴ جریان کار در QGIS

۳۹-۵ ارزیابی صحت

۳۹-۶ آشکارسازی تغییر

۳۹-۱ از پیکسل تا کلاس

طبقه‌بندی تصویر هر پیکسل را به یک **کلاس** پوشش زمین نسبت می‌دهد: آب، جنگل، مزرعه، مرتع، شهر، خاک لخت. نتیجه یک رستر گسسته (فصل ۲) است که پایه‌ی نقشه‌های کاربری اراضی، برآورد سطح زیر کشت و مطالعه‌ی تغییرات است. ایده‌ی اصلی همان امضای طیفی فصل ۳۱ است: پیکسل‌هایی که امضای مشابه دارند، احتمالاً یک نوع پوشش‌اند.

نکته

جایگاه در پایگاه داده مکانی ابری. سامانه طبقه‌بندی‌کننده‌ی آماده ندارد. جریان کار پیشنهادی این است: تصویر «همه‌ی باندها» را از سامانه بگیرید (فصل ۳۴)، نمونه‌های آموزشی را در یک جدول پایگاه داده نگه دارید، طبقه‌بندی را در QGIS انجام دهید و نتیجه را به پایگاه برگردانید تا ارزیابی صحت، آمار مساحت و مقایسه‌ی تغییر با SQL و ابزارهای سامانه انجام شود.

۳۹-۲ سه رویکرد

رویکرد	چگونه	مزیت	محدودیت
نظارت‌نشده (unsupervised)	الگوریتم پیکسل‌ها را بر اساس شباهت طیفی گروه‌بندی می‌کند (k-means، ISODATA)؛ کارشناس بعداً به هر گروه نام می‌دهد	سریع، بدون نمونه	گروه‌ها لزوماً با کلاس‌های مورد نظر یکی نیستند
نظارت‌شده (supervised)	کارشناس برای هر کلاس نمونه‌ی آموزشی معرفی می‌کند؛ الگوریتم از آن‌ها یاد می‌گیرد	کنترل کلاس‌ها، دقت بیشتر	نیاز به نمونه‌ی خوب و کافی
شیءگرا (OBIA)	تصویر ابتدا به **قطعه‌ها**ی همگن بخش‌بندی، سپس هر قطعه با ویژگی‌های طیفی، شکل و بافت طبقه‌بندی می‌شود	نتیجه‌ی تمیز، مناسب تصویر با تفکیک بالا	پیچیده‌تر، حساس به تنظیم بخش‌بندی

طبقه‌بندی‌کننده‌های رایج

- **بیشینه‌ی احتمال** (Maximum Likelihood): فرض می‌کند پیکسل‌های هر کلاس توزیع نرمال دارند؛ کلاسیک و سریع.
- **جنگل تصادفی** (Random Forest): صدها درخت تصمیم که هر کدام روی بخشی از نمونه‌ها و باندها آموزش می‌بینند و رأی می‌دهند؛ امروز پرکاربردترین انتخاب، مقاوم و بدون فرض توزیع.
- **ماشین بردار پشتیبان** (SVM): مرز بهینه بین کلاس‌ها؛ با نمونه‌ی کم هم خوب است.
- **یادگیری عمیق** (شبکه‌های کانولوشنی): برای تصویرهای با تفکیک بالا و الگوهای مکانی پیچیده؛ نیاز به نمونه‌ی بسیار زیاد.

۳۹-۳ نمونه‌ی آموزشی در پایگاه داده

کیفیت طبقه‌بندی نظارت‌شده بیش از هر چیز به **نمونه‌ی آموزشی** بستگی دارد. نگه داشتن نمونه‌ها در پایگاه داده (به جای فایل‌های پراکنده) آن‌ها را چندکاربره، تکرارپذیر و قابل بازبینی می‌کند:

```sql
CREATE TABLE lc_classes (id smallint PRIMARY KEY, name text NOT NULL UNIQUE, color text);
INSERT INTO lc_classes VALUES (1, 'آب','#3a86c9'), (2, 'جنگل و باغ','#e7d4f2'), (3, 'زراعت','#bb04a8'),
                              (4, 'مرتع', '#c9b46a'), (5, 'شهر', '#b23a35'), (6, 'خاک لخت', '#a67c3d');

CREATE TABLE training_samples (
  gid        serial PRIMARY KEY,
  class_id   smallint NOT NULL REFERENCES lc_classes(id),
  purpose    text NOT NULL CHECK (purpose IN ('train', 'validate')),
-- جدا کردن نمونه‌ی آموزش از ارزیابی
  collector  text,
  checked_on date DEFAULT current_date,
  geom       geometry(Polygon, 4326) NOT NULL CHECK (ST_IsValid(geom))
);
CREATE INDEX ON training_samples USING gist (geom);

SELECT id, name FROM lc_classes ORDER BY id;
```

id	name
1	آب
2	جنگل و باغ
3	زراعت
4	مرتع
5	شهر
6	خاک لخت

قاعده‌های نمونه‌ی خوب:

- **نماینده** باشد: از همه‌جای منطقه و همه‌ی گونه‌های هر کلاس (مزرعه‌ی سبز و تازه‌برداشت‌شده هر دو «زراعت»اند).
- **همگن** باشد: چندضلعی آموزشی فقط یک کلاس داشته باشد و از لبه‌ها فاصله بگیرد (پیکسل مختلط، فصل ۳۲).
- **کافی** باشد: قاعده‌ی سرانگشتی دست‌کم ده تا صد برابر تعداد باندها پیکسل برای هر کلاس.
- **جدا** باشد: بخشی از نمونه‌ها (معمولاً ۳۰ درصد) را برای **ارزیابی صحت** کنار بگذارید و در آموزش استفاده نکنید (ستون `purpose`).
- با **بازدید میدانی** یا تصویر با تفکیک بالاتر تأیید شده باشد. برداشت میدانی سامانه (فصل ۴۸) برای جمع‌آوری نمونه با گوشی مناسب است.

۳۹-۴ جریان کار در QGIS

۱. تصویر «همه‌ی باندها»ی Sentinel-2 را از سامانه بگیرید (برای کشاورزی، تاریخ اوج رشد؛ یا ترکیب بی‌ابر فصلی) و در QGIS باز کنید.

۲. لایه‌ی `training_samples` را از پایگاه داده باز کنید (فصل ۶) و چندضلعی‌های نمونه را رقومی کنید؛ هر کارشناس می‌تواند هم‌زمان کار کند.

۳. باندهای کمکی بسازید: NDVI، NDWI و NDBI (فصل ۳۵) اغلب تفکیک کلاس‌ها را بهتر می‌کنند و می‌توان آن‌ها را به‌عنوان باند اضافه به طبقه‌بندی داد.

۴. طبقه‌بندی: با پلاگین **Semi-Automatic Classification (SCP)** یا ابزارهای **Orfeo Toolbox** و **dzetsaka**، جنگل تصادفی را روی نمونه‌های `purpose = 'train'` آموزش و روی تصویر اجرا کنید.

۵. پس‌پردازش: فیلتر اکثریت (Sieve یا Majority) پیکسل‌های تک‌افتاده را حذف می‌کند.

۶. برداری کنید (Polygonize) و نتیجه را با DB Manager به جدول `landcover_1405` در پایگاه وارد کنید؛ یا رستر را با raster2pgsql (فصل ۲۳).

پس از ورود، مساحت هر کلاس با یک پرسش ساده به دست می‌آید: `SELECT class_id, sum(ST_Area(geom::geography)) / 10000 AS ha FROM landcover_1405 GROUP BY class_id;`

۳۹-۵ ارزیابی صحت

نقشه‌ای که صحتش سنجیده نشده، نقشه نیست؛ حدس است. **ماتریس خطا** (confusion matrix) برای نمونه‌های ارزیابی، کلاس واقعی (مرجع) را با کلاس پیش‌بینی‌شده مقایسه می‌کند. فرض کنید پس از پیوند مکانی نمونه‌های `validate` با نقشه‌ی طبقه‌بندی، این شمارش را داریم:

```sql
WITH m(reference, predicted, n) AS (VALUES
  ('آب','آب',48), ('آب','زراعت',1), ('آب','شهر',1),
  ('زراعت','زراعت',87), ('زراعت','مرتع',9), ('زراعت','آب',2),
  ('مرتع','مرتع',61), ('مرتع','زراعت',12), ('مرتع','خاک لخت',7),
  ('شهر','شهر',55), ('شهر','خاک لخت',6),
  ('خاک لخت','خاک لخت',38), ('خاک لخت','شهر',8), ('خاک لخت','مرتع',5))
SELECT reference,
       coalesce(sum(n) FILTER (WHERE predicted = 'آب'), 0)      AS "آب",
       coalesce(sum(n) FILTER (WHERE predicted = 'زراعت'), 0)   AS "زراعت",
       coalesce(sum(n) FILTER (WHERE predicted = 'مرتع'), 0)    AS "مرتع",
       coalesce(sum(n) FILTER (WHERE predicted = 'شهر'), 0)     AS "شهر",
       coalesce(sum(n) FILTER (WHERE predicted = 'خاک لخت'), 0) AS "خاک لخت",
       sum(n) AS total
FROM m GROUP BY reference ORDER BY reference;
```

reference	آب	زراعت	مرتع	شهر	خاک لخت	total
آب	48	1	0	1	0	50
خاک لخت	0	0	5	8	38	51
زراعت	2	87	9	0	0	98
شهر	0	0	0	55	6	61
مرتع	0	12	61	0	7	80

سطرها کلاس واقعی و ستون‌ها کلاس پیش‌بینی‌شده‌اند (`coalesce` خانه‌های خالی را صفر می‌کند). قطر اصلی، پیکسل‌های درست‌طبقه‌بندی‌شده است. از این ماتریس چند معیار استاندارد به دست می‌آید:

- **صحت کلی** (overall accuracy): مجموع قطر ÷ کل.
- **صحت تولیدکننده** (producer's accuracy) برای هر کلاس: درست‌ها ÷ مجموع سطر. «از کل جنگل‌های واقعی، چند درصد را جنگل نقشه کردیم؟» (مکمل آن **خطای حذف** است.)
- **صحت کاربر** (user's accuracy): درست‌ها ÷ مجموع ستون. «اگر روی نقشه جایی جنگل است، چقدر احتمال دارد واقعاً جنگل باشد؟» (مکمل آن **خطای اضافه** است.)
- **ضریب کاپا** (κ، کوهن): توافق فراتر از شانس؛ $\kappa = (p_o - p_e)/(1 - p_e)$ که p_o صحت کلی و p_e توافق مورد انتظار تصادفی است.

```sql
WITH m(reference, predicted, n) AS (VALUES
  ('48','آب','آب'), ('1','زراعت','آب'), ('1','شهر','آب'),
  ('87','زراعت','زراعت'), ('9','مرتع','زراعت'), ('2','آب','زراعت'),
  ('61','مرتع','مرتع'), ('12','زراعت','مرتع'), ('7','خاک لخت','مرتع'),
  ('55','شهر','شهر'), ('6','خاک لخت','شهر'),
  ('38','خاک لخت','خاک لخت'), ('8','شهر','خاک لخت'), ('5','مرتع','خاک لخت')),
tot AS (SELECT sum(n)::numeric AS t FROM m),
cls AS (SELECT DISTINCT reference AS c FROM m),
per AS (
  SELECT c,
         (SELECT coalesce(sum(n), 0) FROM m WHERE reference = c AND
predicted = c) AS correct,
         (SELECT sum(n) FROM m WHERE reference = c)  AS row_total,
         (SELECT coalesce(sum(n), 0) FROM m WHERE predicted = c) AS
col_total
  FROM cls
)
SELECT c AS class,
       round(100.0 * correct / row_total, 1) AS producer_pct,
       round(100.0 * correct / col_total, 1) AS user_pct,
       round(100 * (SELECT sum(correct) FROM per) / t, 1) AS
overall_pct,
       round(((SELECT sum(correct) FROM per) / t - (SELECT
sum(row_total * col_total) FROM per) / t ^ 2)
             / (1 - (SELECT sum(row_total * col_total) FROM per) / t ^
2), 3) AS kappa
FROM per, tot
ORDER BY producer_pct;
```

class	producer_pct	user_pct	overall_pct	kappa
خاک لخت	74.5	74.5	85.0	0.809
مرتع	76.3	81.3	85.0	0.809
زراعت	88.8	87.0	85.0	0.809
شهر	90.2	85.9	85.0	0.809
آب	96.0	96.0	85.0	0.809

صحت کلی حدود ۸۵ درصد و کاپا حدود ۰٫۸ است که برای نقشه‌ی پوشش زمین با Sentinel-2 قابل قبول به شمار می‌رود. اما نگاه به کلاس‌ها مهم‌تر است: مرتع با زراعت و خاک لخت با شهر

بیشترین اشتباه را دارند؛ دو جفت کلاسی که امضای طیفی مشابهی دارند. راه بهبود: نمونه‌ی بیشتر برای این کلاس‌ها، افزودن شاخص‌ها و تصویر چندزمانه (مرتع و زراعت در زمان‌های مختلف سال رفتار متفاوتی دارند).

۳۹-۶ آشکارسازی تغییر

«چه چیزی بین دو تاریخ عوض شده؟» دو رویکرد اصلی دارد:

۱. تفاضل تصویر یا شاخص. شاخص دو تاریخ کم می‌شود و پیکسل‌هایی با تفاضل بیش از آستانه «تغییر» به حساب می‌آیند: dNBR برای آتش (فصل ۳۵)، تفاضل NDVI برای تخریب پوشش گیاهی، تفاضل NDWI برای تغییر آب. ساده و سریع است، اما **نوع** تغییر را نمی‌گوید و به تفاوت فصل و جو حساس است؛ دو تاریخ را از یک فصل انتخاب کنید. در سامانه، سری زمانی «آمار هر قطعه» همین را در سطح قطعه انجام می‌دهد (فصل ۳۷).

۲. مقایسه‌ی پس از طبقه‌بندی. هر تاریخ جداگانه طبقه‌بندی و دو نقشه روی هم گذاشته می‌شوند. نتیجه یک **ماتریس تغییر** «از ← به» است: چند هکتار زراعت به شهر تبدیل شده؟ نوع تغییر معلوم است، اما خطای هر دو طبقه‌بندی در نتیجه جمع می‌شود. برای نقشه‌های برداری، ابزار **«تغییر بین دو نسخه»** سامانه (فصل ۲۷) دقیقاً همین ماتریس را می‌سازد و با نمودار **Sankey** در استودیوی نقشه نمایش می‌دهد (فصل ۴۳). معادل SQL آن، اشتراک دو لایه و گروه‌بندی بر اساس هر دو کلاس است:

```sql
SELECT a.class_id AS from_class, b.class_id AS to_class,
       round(sum(ST_Area(ST_Intersection(a.geom, b.geom)::geography)) /
10000) AS hectares
FROM landcover_1400 a
JOIN landcover_1405 b ON ST_Intersects(a.geom, b.geom)
GROUP BY 1, 2
ORDER BY hectares DESC;
```

در دنیای واقعی

گسترش شهر در حریم رودخانه. با دو نقشه‌ی پوشش زمین ۱۳۹۵ و ۱۴۰۵ (هر دو از Landsat و Sentinel-2 با جنگل تصادفی) و حریم ۱۵۰ متری رودخانه‌ها (فصل ۱۸)، ماتریس تغییر داخل حریم نشان داد ۲۱۰ هکتار باغ و زراعت در ده سال به «شهر» تبدیل شده است. نقشه‌ی همین چندضلعی‌ها، که در استودیوی نقشه منتشر شد، مبنای بازدید مأموران حفاظت رودخانه قرار گرفت.

خلاصه‌ی فصل

- طبقه‌بندی هر پیکسل را به کلاس پوشش زمین نسبت می‌دهد: نظارت‌نشده (k-means)، نظارت‌شده (جنگل تصادفی، SVM، بیشینه‌ی احتمال) یا شیءگرا.
- نمونه‌ی آموزشی را در جدول پایگاه داده با کلید خارجی کلاس و ستون آموزش/ارزیابی نگه دارید؛ نماینده، همگن، کافی و جدا.
- جریان کار: «همه‌ی باندها» ← QGIS (SCP، OTB) ← پس‌پردازش ← برگرداندن به پایگاه.
- ارزیابی: ماتریس خطا، صحت کلی، صحت تولیدکننده و کاربر، کاپا؛ همه با SQL.
- تغییر: تفاضل شاخص (سریع، بی‌نوع) یا مقایسه‌ی پس از طبقه‌بندی (ماتریس «از ← به»؛ ابزار «تغییر بین دو نسخه» و Sankey).

برای مدرس

زمان پیشنهادی: سه جلسه (یک جلسه نظری، یک جلسه نمونه‌گیری گروهی در جدول مشترک، یک جلسه طبقه‌بندی و ارزیابی). نمونه‌گیری گروهی در یک جدول مشترک پایگاه داده، تجربه‌ی کار تیمی واقعی است و مسئله‌ی «نمونه‌ی بد یک نفر کل نقشه را خراب می‌کند» را ملموس می‌کند.

تمرین‌ها

۱. سطح ۲ تفاوت صحت تولیدکننده و صحت کاربر را با یک مثال توضیح دهید. برای سازمانی که می‌خواهد هیچ زمین زراعی را از دست ندهد، کدام مهم‌تر است؟

۲. سطح ۲ در ماتریس نمونه، اگر ۱۰ پیکسل «مرتع» که «زراعت» پیش‌بینی شده بودند درست شوند، صحت کلی و کاپا چقدر می‌شود؟

۳. سطح ۳ برای یک منطقه‌ی کشاورزی، ۵۰ چندضلعی نمونه در شش کلاس جمع کنید (۷۰ درصد آموزش، ۳۰ درصد ارزیابی)، با جنگل تصادفی طبقه‌بندی و صحت را با SQL بسنجید.

۴. سطح ۳ دو تصویر NBR پیش و پس از یک آتش‌سوزی واقعی بگیرید، dNBR را در QGIS بسازید، طبقه‌بندی شدت را انجام دهید و مساحت هر کلاس را گزارش کنید.

فصل

۴۰

رادار و سنجش حرارتی

سطح: **پیشرفته** ●●●

زمان مطالعه: **۹۰ دقیقه**

پیش‌نیاز: **فصل‌های ۳۱ و ۳۲**

در این فصل یاد می‌گیرید

- اصول رادار روزنه‌ی مصنوعی (SAR)، طول موج، قطبش و پس‌پراکنش را توضیح دهید.
- تصویر راداری را تفسیر کنید؛ آب تیره، شهر روشن، اعوجاج‌های هندسی و نویز لکه‌ای.
- کاربردهای Sentinel-1، از پهنه‌بندی سیل تا پایش فرونشست با تداخل‌سنجی را بشناسید.
- اصول سنجش حرارتی، گسیلمندی و دمای سطح زمین را توضیح دهید.
- گام‌های برآورد دمای سطح از Landsat را بدانید.

سرفصل‌ها

۴۰-۱ چرا رادار

۴۰-۲ رادار روزنه‌ی مصنوعی

۴۰-۳ Sentinel-1

۴۰-۴ تداخل‌سنجی و فرونشست

۴۰-۵ سنجش حرارتی

۴۰-۱ چرا رادار

سنجنده‌های نوری دو محدودیت بزرگ دارند: به **نور خورشید** نیاز دارند و از **ابر** عبور نمی‌کنند. در فصل بارانی، درست وقتی سیل رخ داده، آسمان ابری است و تصویر نوری به کار نمی‌آید. **رادار** این دو محدودیت را ندارد: خودش امواج ریزموج می‌فرستد (سنجش فعال، فصل ۳۱) و طول موج چندسانتی‌متری آن از ابر، دود و گردوغبار می‌گذرد. شب و روز، در هر هوایی.

> **نکته**
>
> **جایگاه در کتاب.** پایگاه داده مکانی ابری در حال حاضر تصویر راداری و حرارتی تهیه نمی‌کند. این فصل مفاهیم را آموزش می‌دهد تا بتوانید داده‌ی این سنجنده‌ها را از منابع رایگان (مثلاً Copernicus Data Space برای Sentinel-1 و USGS EarthExplorer برای باند حرارتی Landsat) بگیرید، در QGIS یا SNAP پردازش کنید و نتیجه را برای تحلیل و انتشار به پایگاه داده بیاورید.

۴۰-۲ رادار روزنه‌ی مصنوعی

رادار تصویربرداری روی ماهواره **جانبی** نگاه می‌کند (نه عمودی) و پالس‌های ریزموج را به سطح می‌فرستد. بخشی از انرژی که به سمت سنجنده برمی‌گردد، **پس‌پراکنش** (backscatter) نام دارد و روشنایی هر پیکسل را تعیین می‌کند. برای رسیدن به تفکیک مکانی خوب با آنتنی که روی ماهواره جا می‌شود، رادار حرکت ماهواره را به کار می‌گیرد و سیگنال‌های نقطه‌های پشت‌سرهم مسیر را با هم ترکیب می‌کند، گویی آنتنی چندکیلومتری دارد؛ به همین دلیل **رادار روزنه‌ی مصنوعی** (Synthetic Aperture Radar یا **SAR**) نام دارد.

باند	طول موج	نفوذ	نمونه
X	حدود ۳ cm	کم؛ سطح تاج گیاه	TerraSAR-X
C	حدود ۵٫۶ cm	متوسط	**Sentinel-1**
L	حدود ۲۴ cm	زیاد؛ از تاج جنگل تا زمین	ALOS-2، NISAR

قطبش (polarization): رادار موج را با جهت نوسان افقی (H) یا عمودی (V) می‌فرستد و می‌گیرد. VV و HH (هم‌قطبش) به سطح و زبری حساس‌اند؛ VH و HV (قطبش متقابل) بیشتر از **پراکنش حجمی** پوشش گیاهی می‌آیند. Sentinel-1 در حالت معمول خشکی‌ها VV و VH می‌گیرد.

تفسیر تصویر راداری

روشنایی تصویر راداری به **زبری**، **هندسه** و **رطوبت** سطح بستگی دارد، نه به رنگ:

- **آب آرام: بسیار تیره.** سطح صاف مثل آینه موج را به سمت دیگر بازتاب می‌دهد (بازتاب آینه‌ای) و چیزی به سنجنده برنمی‌گردد. همین ویژگی رادار را بهترین ابزار **پهنه‌بندی سیل** کرده است.
- **شهر: بسیار روشن.** دیوار و زمین یک «گوشه» می‌سازند که موج را دو بار بازتاب و مستقیم به سنجنده برمی‌گرداند (**پراکنش دوگانه**).
- **پوشش گیاهی: متوسط**، به‌ویژه در VH، به دلیل پراکنش حجمی میان برگ‌ها و شاخه‌ها.
- **خاک مرطوب روشن‌تر از خاک خشک**، چون رطوبت ثابت دی‌الکتریک را بالا می‌برد؛ پایه‌ی برآورد رطوبت خاک.

پس‌پراکنش معمولاً به **دسی‌بل** بیان می‌شود، چون بازه‌ی مقدارهایش بسیار گسترده است:

$$\sigma^0_{dB} = 10 \log_{10} (\sigma^0)$$

```sql
SELECT surface, sigma0, round((10 * log(sigma0))::numeric, 1) AS
sigma0_db
FROM (VALUES ('0.002 ,'آب آرام'), ('0.03 ,'مرتع'), ('0.08 ,'جنگل'), ('شهر',
0.6)) t(surface, sigma0);
```

surface	sigma0	sigma0_db
آب آرام	0.002	-27.0
مرتع	0.03	-15.2
جنگل	0.08	-11.0
شهر	0.6	-2.2

`log` در PostgreSQL لگاریتم پایه‌ی ده است. آستانه‌ای حدود ۱۸- تا ۲۰- دسی‌بل در قطبش VV، پیکسل‌های آب را از خشکی جدا می‌کند؛ ساده‌ترین روش پهنه‌بندی سیل.

دشواری‌های رادار

- **نویز لکه‌ای** (speckle): دانه‌دانه بودن ذاتی تصویر راداری که از تداخل بازتاب‌های داخل هر پیکسل می‌آید. با فیلترهای Lee یا Refined Lee یا میانگین زمانی کم می‌شود.
- **اعوجاج هندسی** به دلیل نگاه جانبی: **کوتاه‌شدگی** دامنه‌ی رو به سنجنده، **رویهم‌افتادگی** (layover) قله‌ی کوه روی پای آن، و **سایه‌ی راداری** پشت کوه‌ها که هیچ سیگنالی نمی‌گیرد. در مناطق کوهستانی ایران این اثرها شدیدند و تصحیح زمین (terrain correction) با DEM ضروری است.

۴۰-۳ Sentinel-1

Sentinel-1 بخش راداری برنامه‌ی کوپرنیک است: باند C، حالت تصویربرداری IW با پهنای ۲۵۰ کیلومتر و پیکسل حدود ۱۰ متری، تکرار ۶ تا ۱۲ روزه بسته به تعداد ماهواره‌های فعال، و داده‌ی رایگان. دو محصول رایج: **GRD** (شدت پس‌پراکنش، برای سیل، کشاورزی و یخ) و **SLC** (با فاز موج، برای تداخل‌سنجی).

۴۰-۴ تداخل‌سنجی و فرونشست

تداخل‌سنجی راداری (InSAR) **فاز** موج را در دو تصویر از یک محل در دو زمان مقایسه می‌کند. اگر زمین بین دو تصویر حتی چند میلی‌متر جابه‌جا شده باشد، فاز تغییر می‌کند. با سری زمانی ده‌ها تصویر (روش‌های PS و SBAS)، سرعت جابه‌جایی زمین با دقت میلی‌متر در سال اندازه‌گیری می‌شود.

در دنیای واقعی

فرونشست دشت‌های ایران. برداشت بی‌رویه‌ی آب زیرزمینی باعث فرونشست زمین در بسیاری از دشت‌های ایران شده است. مطالعات InSAR با Sentinel-1 در دشت‌های اطراف تهران، اصفهان، مشهد و کرمان نرخ‌های فرونشست چندسانتی‌متر تا بیش از ده سانتی‌متر در سال را نشان داده‌اند؛ تهدیدی برای راه‌ها، خطوط لوله، ریل و ساختمان‌ها. نتیجه‌ی چنین تحلیلی (نقطه‌هایی با سرعت جابه‌جایی) وقتی وارد پایگاه داده‌ی مکانی شود، با یک پیوند مکانی (فصل ۱۷) می‌گوید کدام بخش از شبکه‌ی آب، کدام مدرسه یا کدام ایستگاه مترو روی مناطق پرخطر است.

۴۰-۵ سنجش حرارتی

هر جسمی که دمایش بالاتر از صفر مطلق باشد، انرژی **گسیل** می‌کند. طبق قانون پلانک، اجسام در دمای زمین (حدود ۳۰۰ کلوین) بیشترین گسیل را در فروسرخ حرارتی، حدود ۱۰ میکرومتر، دارند؛ دلیل اینکه باندهای حرارتی بین ۸ تا ۱۴ میکرومترند (فصل ۳۱).

دو مفهوم کلیدی:

- **دمای درخشندگی** (brightness temperature): دمایی که یک «جسم سیاه» کامل برای گسیل همان انرژی باید داشته باشد؛ مستقیم از سیگنال سنجنده به دست می‌آید.
- **گسیلمندی** (emissivity، ε): کارایی گسیل یک ماده نسبت به جسم سیاه، بین ۰ و ۱. آب و گیاه حدود ۰٫۹۸ تا ۰٫۹۹، خاک لخت حدود ۰٫۹۲ تا ۰٫۹۶ و فلز بسیار کمتر. دو سطح با دمای یکسان ولی گسیلمندی متفاوت، دمای درخشندگی متفاوتی نشان می‌دهند.

دمای سطح زمین (Land Surface Temperature یا LST) با اصلاح دمای درخشندگی برای گسیلمندی به دست می‌آید. گام‌های رایج برای باند ۱۰ Landsat 8/9:

۱. عدد دیجیتال به **تابندگی** بالای جو: $L = M_L \cdot DN + A_L$ (ضرایب در فراداده‌ی صحنه).

۲. تابندگی به **دمای درخشندگی**: $T_B = K_2 / \ln(K_1/L + 1)$، با ثابت‌های K_1 و K_2 باند (برای باند ۱۰ حدود ۷۷۴٫۹ و ۱۳۲۱٫۱).

۳ **گسیلمندی** از NDVI: نسبت پوشش گیاهی $P_v = \left(\frac{NDVI - NDVI_{soil}}{NDVI_{veg} - NDVI_{soil}}\right)^2$ و $\varepsilon = 0.004 P_v + 0.986$.

۴ **دمای سطح:** $LST = \frac{T_B}{1 + (\lambda T_B / \rho) \ln \varepsilon}$ با λ طول موج مؤثر باند (حدود ۱۰٫۹ میکرومتر) و $\rho \approx 14388$ میکرومتر-کلوین.

```sql
WITH p AS (SELECT 10.5 AS radiance, 0.45 AS ndvi),          -- یک پیکسل نمونه
bt AS (SELECT ndvi, 1321.0789 / ln(774.8853 / radiance + 1) AS tb FROM p),
em AS (SELECT tb, 0.004 * power(greatest(least((ndvi - 0.2) / (0.5 - 0.2), 1), 0), 2) + 0.986 AS eps FROM bt)
SELECT round((tb - 273.15)::numeric, 1) AS brightness_c,
       round(eps::numeric, 4) AS emissivity,
       round((tb / (1 + (10.9 * tb / 14388) * ln(eps)) - 273.15)::numeric, 1)  AS lst_c
FROM em;
```

brightness_c	emissivity	lst_c
33.0	0.9888	33.8

دمای سطح کمی بیشتر از دمای درخشندگی است، چون گسیلمندی کمتر از یک است. توجه کنید که این روش ساده‌شده اثر جو را کامل حذف نمی‌کند؛ محصول **Landsat Collection 2 Level-2** دمای سطح تصحیح‌شده (باند ST_B10) را آماده دارد و برای کار جدی بهتر است همان را به کار ببرید.

در دنیای واقعی

جزیره‌ی حرارتی شهری. نقشه‌ی دمای سطح تابستانی یک کلان‌شهر از Landsat معمولاً چند درجه اختلاف بین بافت متراکم و بی‌درخت (یا سطوح بزرگ آسفالت و بام‌های فلزی) و پارک‌ها و باغ‌ها نشان می‌دهد. با وارد کردن LST به پایگاه داده (رستر، فصل ۲۳) و آمار ناحیه‌ای روی محله‌ها (فصل ۳۶)، رابطه‌ی دمای هر محله با سهم فضای سبز (فصل ۱۸) را می‌توان با `corr` (فصل ۱۲) سنجید و محله‌های اولویت‌دار برای درختکاری را مشخص کرد.

خلاصه‌ی فصل

- رادار (SAR) فعال است و از ابر و در شب کار می‌کند؛ باند X، C (Sentinel-1) و L با نفوذ متفاوت؛ قطبش VV و VH.
- روشنایی راداری به زبری، هندسه و رطوبت بستگی دارد: آب آرام تیره، شهر روشن (پراکنش دوگانه)، گیاه متوسط (پراکنش حجمی).
- پس‌پراکنش به دسی‌بل: $10 \log_{10} \sigma^0$؛ آستانه‌ی VV حدود ۱۸- تا ۲۰- دسی‌بل برای آب.
- نویز لکه‌ای و اعوجاج هندسی (کوتاه‌شدگی، رویهم‌افتادگی، سایه) دشواری‌های رادارند.
- InSAR با فاز موج جابه‌جایی میلی‌متری زمین (فرونشست) را می‌سنجد.
- سنجش حرارتی: دمای درخشندگی، گسیلمندی و دمای سطح؛ برآورد LST از Landsat در چهار گام یا محصول آماده‌ی Level-2.

برای مدرس

زمان پیشنهادی: دو جلسه. یک تصویر Sentinel-1 و یک تصویر Sentinel-2 از یک منطقه و یک روز را کنار هم نشان دهید و از دانشجوها بخواهید آب، شهر و مزرعه را در هر کدام پیدا کنند؛ تفاوت منطق «رنگ» و «زبری» خودش آشکار می‌شود. برای سنجش حرارتی، دماسنج فروسرخ دستی و چند سطح (آسفالت، چمن، آب) در حیاط دانشگاه تمرین بسیار ملموسی است.

تمرین‌ها

۱. سطح ۲ چرا آب آرام در تصویر راداری تیره و در تصویر نوری فروسرخ هم تیره است؟ آیا علت فیزیکی یکی است؟

۲. سطح ۲ پس‌پراکنش ۰٫۰۱ و ۰٫۱ چند دسی‌بل است؟ اختلاف ۱۰ دسی‌بل یعنی چند برابر انرژی؟

۳. سطح ۲ در پرسش SQL دمای سطح، NDVI را ۰٫۱ (خاک لخت) و ۰٫۸ (گیاه متراکم) بگذارید. گسیلمندی و LST چگونه تغییر می‌کنند؟

۴. سطح ۳ درباره‌ی یک مطالعه‌ی منتشرشده‌ی InSAR فرونشست در یک دشت ایران تحقیق کنید: چه داده‌ای، چه روشی، چه نرخی؟ چه لایه‌هایی از پایگاه داده‌ی یک شهرداری را با آن پیوند می‌دهید؟

بخش ۷

کارتوگرافی و انتشار

نقشه‌ی خوب داستان داده را می‌گوید. اصول طراحی نقشه، استودیوی چندلایه، نمودار و داشبورد، چاپ و ارائه، و انتشار داده با استانداردهای OGC، کاشی برداری و API.

۴۱ اصول کارتوگرافی

۴۲ استودیوی نقشه‌ی چندلایه

۴۳ نمودار و داشبورد روی نقشه

۴۴ اشتراک، ارائه، چاپ و مقایسه

۴۵ انتشار داده؛ کاشی برداری، OGC API و WMS/WFS

۴۶ ساخت برنامه‌ی نقشه‌ی وب؛ REST API، Leaflet، MapLibre و فرم میدانی

فصل

۴۱

اصول کارتوگرافی

سطح: **مبتدی** | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۲**

در این فصل یاد می‌گیرید

- اجزای یک نقشه‌ی کامل و سلسله‌مراتب بصری را بشناسید.
- متغیرهای بصری برتن را به سطح اندازه‌گیری داده پیوند دهید.
- روش‌های طبقه‌بندی (فاصله‌ی برابر، چندک، شکست طبیعی، انحراف معیار) را مقایسه و مرزها را با SQL حساب کنید.
- طرح رنگ ترتیبی، واگرا و کیفی را درست انتخاب کنید و رنگ‌کوری را در نظر بگیرید.
- از خطاهای رایج نقشه‌ی موضوعی (مقدار مطلق در نقشه‌ی کروپلت، طبقه‌ی گمراه‌کننده) بپرهیزید.

سرفصل‌ها

۴۱-۱ نقشه، استدلال بصری

۴۱-۲ اجزای نقشه و سلسله‌مراتب بصری

۴۱-۳ متغیرهای بصری

۴۱-۴ نقشه‌ی کروپلت و نرمال‌سازی

۴۱-۵ طبقه‌بندی داده

۴۱-۶ رنگ

۴۱-۷ نماد، برچسب و تعمیم

۴۱-۸ چک‌لیست نقشه‌ی خوب

۴۱-۱ نقشه، استدلال بصری

نقشه فقط «نمایش داده» نیست؛ **استدلال** است. هر نقشه‌ی موضوعی پیامی دارد («فضای سبز در جنوب شهر کم است»، «این مزرعه‌ها خشک‌ترند») و انتخاب‌های کارتوگرافیک، از طبقه‌بندی تا رنگ، می‌توانند این پیام را روشن، مبهم یا حتی وارونه کنند. کارتوگرافی دانش و هنر طراحی این استدلال است.

۴۱-۲ اجزای نقشه و سلسله‌مراتب بصری

یک نقشه‌ی کامل (برای چاپ یا گزارش) این اجزا را دارد:

جزء	کار
بدنه‌ی نقشه	خود داده‌ها روی نقشه‌ی پایه
عنوان	پیام نقشه، نه فقط نام لایه («تراکم مدرسه در محله‌ها»، نه «schools»)
راهنما (legend)	معنای نمادها و رنگ‌ها با واحد
مقیاس	نوار مقیاس (در نقشه‌ی رقومی بهتر از کسر عددی)
جهت شمال	به‌ویژه اگر نقشه چرخیده یا مخاطب عام است
منبع و تاریخ	منبع داده، تاریخ، تهیه‌کننده؛ و ذکر مجوز (OSM، Copernicus)
نقشه‌ی موقعیت (اختیاری)	جای منطقه در محدوده‌ی بزرگ‌تر

سلسله‌مراتب بصری یعنی مهم‌ترین چیز، بیشترین توجه را بگیرد: لایه‌ی موضوعی پررنگ و پرکنتراست، نقشه‌ی پایه کم‌رنگ و خاکستری، اجزای کمکی کوچک و آرام. اصل **شکل و زمینه**: موضوع باید از زمینه جدا دیده شود. نقشه‌ی پایه‌ی پررنگ و شلوغ، رایج‌ترین دشمن این اصل است.

۴۱-۳ متغیرهای بصری

ژاک برتن، نقشه‌نگار فرانسوی، در ۱۹۶۷ **متغیرهای بصری** را صورت‌بندی کرد: ویژگی‌هایی از نماد که می‌توان برای نمایش داده عوض کرد. هر متغیر برای سطح اندازه‌گیری خاصی (فصل ۲) مناسب است:

متغیر بصری	اسمی	ترتیبی	کمّی	نمونه
مکان	✔	✔	✔	جای نماد
اندازه		✔	✔	دایره‌ی متناسب با جمعیت
روشنایی (value)		✔	✔ (طبقه‌ای)	طیف روشن به تیره برای تراکم
فام (hue، رنگ)	✔			آبی برای آب، سبز برای پارک
شکل	✔			مربع برای مدرسه، دایره برای بیمارستان
جهت	✔			جهت باد
بافت	✔	✔		هاشور

دو خطای رایج: نمایش داده‌ی **اسمی با طیف روشنایی** (کاربری‌ها با سبز روشن تا تیره، که ترتیب القا می‌کند) و نمایش داده‌ی **کمّی با فام‌های متفاوت** (جمعیت با رنگین‌کمان، که ترتیب را نشان نمی‌دهد).

۴۱-۴ نقشه‌ی کروپلت و نرمال‌سازی

نقشه‌ی کروپلت (choropleth) مناطق را بر اساس مقدارشان رنگ می‌کند و پرکاربردترین نقشه‌ی موضوعی است. قاعده‌ی طلایی آن:

> **در نقشه‌ی کروپلت نسبت یا تراکم نشان دهید، نه مقدار مطلق.**

چرا؟ چون چشم مساحت رنگ را می‌بیند. محله‌ی بزرگ با جمعیت زیاد ممکن است تراکم کمی داشته باشد؛ اگر «جمعیت» را رنگ کنید، محله‌های بزرگ همیشه تیره دیده می‌شوند. برای

مقدار مطلق، **نماد متناسب** (دایره‌ای که مساحتش با مقدار متناسب است) را روی مرکز منطقه بگذارید. تبدیل به نسبت با SQL ساده است:

```sql
SELECT name, population,
       round((ST_Area(geom::geography) / 1e6)::numeric, 1)
AS km2,
       round(population / (ST_Area(geom::geography) / 1e6))
AS people_per_km2
FROM neighborhoods
ORDER BY people_per_km2 DESC;
```

name	population	km2	people_per_km2
هفت‌تیر	65500	16.1	4078.0
جمهوری	56000	16.1	3485.0
شهرک غرب	46500	16.1	2896.0
نارمک	37000	16.1	2304.0
تجریش	27500	16.0	1714.0
ونک	18000	16.1	1121.0

۴۱-۵ طبقه‌بندی داده

برای نقشه‌ی کروپلت، مقدارهای پیوسته معمولاً در **۳ تا ۷ طبقه** دسته‌بندی می‌شوند؛ چشم بیش از هفت روشنایی را به‌سختی از هم جدا می‌کند. روش طبقه‌بندی، ظاهر نقشه را عمیقاً تغییر می‌دهد:

روش	چگونه	مناسب برای	خطر
فاصله‌ی برابر	بازه‌ی کمینه تا بیشینه به طبقه‌های هم‌پهنا	داده‌ی یکنواخت؛ مقایسه‌ی چند نقشه با یک مقیاس	با داده‌ی چوله، بیشتر عارضه‌ها در یک طبقه
چندک (quantile)	هر طبقه تعداد برابری عارضه	داده‌ی چوله؛ رتبه‌بندی	مقدارهای بسیار نزدیک در طبقه‌های جدا
شکست طبیعی (Jenks)	کمینه کردن واریانس درون طبقه‌ها	خوشه‌های طبیعی داده	مرزها برای هر داده فرق می‌کند؛ مقایسه سخت
انحراف معیار	فاصله از میانگین بر حسب انحراف معیار	نمایش بالاتر/پایین‌تر از میانگین (طیف واگرا)	نیاز به توزیع نسبتاً نرمال
دستی	مرزهای معنادار (۰٫۲، ۰٫۴، … یا آستانه‌ی قانونی)	وقتی مرز معنای بیرونی دارد	ذهنی

مرزهای دو روش رایج را با SQL برای تعداد دانش‌آموز مدرسه‌ها حساب می‌کنیم:

```sql
WITH s AS (SELECT min(students) AS lo, max(students) AS hi FROM schools)
SELECT k AS class,
       round(lo + (hi - lo) * (k - 1) / 5.0) AS equal_from,
       round(lo + (hi - lo) * k / 5.0)       AS equal_to,
       (SELECT round(percentile_cont((k - 1) / 5.0) WITHIN GROUP (ORDER BY students)::numeric) FROM schools) AS quantile_from,
       (SELECT round(percentile_cont(k / 5.0)       WITHIN GROUP (ORDER BY students)::numeric) FROM schools) AS quantile_to,
       (SELECT count(*) FROM schools WHERE width_bucket(students, lo, hi + 1, 5) = k)                        AS equal_count
FROM s, generate_series(1, 5) k;
```

class	equal_from	equal_to	quantile_from	quantile_to	equal_count
1	151	300	151	271	43
2	300	450	271	415	35
3	450	599	415	598	30
4	599	749	598	736	40
5	749	898	736	898	32

در فاصله‌ی برابر، تعداد مدرسه‌ی هر طبقه متفاوت است (`equal_count`)؛ در چندک، هر طبقه حدود ۳۶ مدرسه دارد ولی پهنای طبقه‌ها برابر نیست. `width_bucket` تابع آماده‌ی PostgreSQL برای طبقه‌ی فاصله‌ی برابر و `percentile_cont` برای چندک است (فصل ۱۲). استودیوی نقشه‌ی سامانه همین روش‌ها را با پیش‌نمایش زنده دارد (فصل ۴۲).

هشدار

طبقه‌بندی گمراه‌کننده. با انتخاب روش و مرز، می‌توان یک داده را «بحرانی» یا «آرام» نشان داد. اگر نقشه‌ای برای تصمیم‌گیری می‌سازید، روش طبقه‌بندی را در راهنما بنویسید، برای مقایسه‌ی دو سال از **یک مرز ثابت** استفاده کنید، و اگر ممکن است هیستوگرام داده را کنار نقشه بگذارید؛ نمودار هیستوگرام استودیو مرزهای طبقه‌بندی لایه را روی خود هیستوگرام نشان می‌دهد (فصل ۴۳).

۴۱-۶ رنگ

سه نوع **طرح رنگ** برای سه نوع داده:

- **ترتیبی** (sequential): یک فام از روشن به تیره؛ برای مقدار کم به زیاد (تراکم، جمعیت، NDVI). قاعده: **روشن = کم، تیره = زیاد** (روی زمینه‌ی روشن).
- **واگرا** (diverging): دو فام با مرکز خنثی؛ برای مقدارهایی که یک نقطه‌ی میانی معنادار دارند (تغییر مثبت و منفی، بالاتر و پایین‌تر از میانگین، امتیاز z کانون داغ در فصل ۲۱).
- **کیفی** (qualitative): فام‌های متمایز با روشنایی مشابه؛ برای دسته‌ها (کاربری، نوع مرکز درمانی، خط مترو). حداکثر هشت تا ده رنگ.

مجموعه‌ی **ColorBrewer** (سینتیا بروئر) مرجع رایج طرح‌های رنگی آزموده‌شده برای نقشه است و در QGIS و بیشتر نرم‌افزارها موجود است.

> **نکته**
>
> **رنگ‌کوری.** حدود ۸ درصد مردان نوعی کوررنگی (بیشتر قرمز-سبز) دارند. طیف قرمز-سبز (رایج برای NDVI) برای آن‌ها مبهم است. برای مخاطب عام، طرح‌هایی مثل زرد-سبز-آبی یا بنفش-نارنجی را ترجیح دهید، یا به جز رنگ، روشنایی را هم تغییر دهید تا نقشه در چاپ سیاه‌وسفید هم خوانا بماند.

۴۱-۷ نماد، برچسب و تعمیم

- **نماد نقطه‌ای:** اندازه‌ی ثابت برای وجود (مدرسه این‌جاست)، اندازه‌ی متناسب برای مقدار (تعداد دانش‌آموز). برای نماد متناسب، **مساحت** (نه قطر) را متناسب با مقدار بگیرید.
- **برچسب:** نام نقطه بالا-راست نماد (در فارسی بالا-چپ هم رایج است)، نام خط در امتداد آن، نام چندضلعی در مرکز (با `ST_PointOnSurface` ، فصل ۱۵). برچسب‌ها را کم و خوانا نگه دارید؛ هاله‌ی سفید دور متن خوانایی را روی زمینه‌ی شلوغ بیشتر می‌کند.
- **تعمیم** (generalization): در مقیاس کوچک‌تر جزئیات کمتر: ساده‌سازی خط‌ها (فصل ۱۸)، حذف عارضه‌های کوچک، ادغام. کاشی‌های برداری سامانه در هر سطح بزرگ‌نمایی این کار را خودکار انجام می‌دهند.

۴۱-۸ چک‌لیست نقشه‌ی خوب

۱. پیام نقشه در یک جمله چیست؟ عنوان همان را می‌گوید؟

۲. داده نرمال شده است (نسبت، نه مقدار مطلق در کروپلت)؟

۳. متغیر بصری با سطح اندازه‌گیری جور است؟

۴. روش و تعداد طبقه آگاهانه انتخاب و در راهنما ذکر شده؟

۵. طرح رنگ با نوع داده (ترتیبی، واگرا، کیفی) جور و برای رنگ‌کورها خواناست؟

۶. نقشه‌ی پایه آرام است و موضوع را نمی‌پوشاند؟

۷ راهنما با واحد، مقیاس، منبع، تاریخ و مجوز هست؟

خلاصه‌ی فصل

- نقشه استدلال بصری است: عنوان پیام، راهنما با واحد، مقیاس، شمال، منبع و تاریخ.
- متغیرهای بصری برتن: اندازه و روشنایی برای کمّی و ترتیبی؛ فام و شکل برای اسمی.
- کروپلت: نسبت یا تراکم، نه مقدار مطلق؛ برای مقدار مطلق نماد متناسب.
- طبقه‌بندی: فاصله‌ی برابر، چندک، شکست طبیعی، انحراف معیار، دستی؛ ۳ تا ۷ طبقه؛ مرزها با `width_bucket` و `percentile_cont`.
- رنگ: ترتیبی (روشن=کم)، واگرا (مرکز معنادار)، کیفی (دسته‌ها)؛ رنگ‌کوری را در نظر بگیرید.

برای مدرس

زمان پیشنهادی: دو جلسه. یک داده را با چهار روش طبقه‌بندی روی چهار نقشه کنار هم نشان دهید و بپرسید «کدام درست است؟»؛ بحث درباره‌ی اینکه «همه درست‌اند ولی پیام متفاوتی می‌دهند» مهم‌ترین درس کارتوگرافی است. کتاب «چگونه با نقشه دروغ بگوییم» مونمونیر خواندنی تکمیلی خوبی است.

تمرین‌ها

۱. سطح ۱ برای هر داده متغیر بصری مناسب را بنویسید: نوع کاربری، تراکم جمعیت، تعداد تخت بیمارستان، رده‌ی جاده، تغییر NDVI نسبت به سال قبل.

۲. سطح ۱ چرا «تعداد مدرسه» را نباید با کروپلت روی محله‌ها نشان داد؟ چه چیزی را باید نشان داد؟

۳. سطح ۲ مرزهای طبقه‌بندی چندک و فاصله‌ی برابر را برای مساحت پارک‌ها (چهار طبقه) با SQL حساب کنید.

۴. سطح ۲ برای امتیاز z کانون داغ (فصل ۲۱) چه طرح رنگی و چه مرزهایی مناسب است؟ چرا؟

۵. سطح ۳ یک نقشه‌ی چاپی با همه‌ی اجزای جدول «اجزای نقشه» از تراکم جمعیت محله‌ها بسازید (فصل‌های ۴۲ و ۴۴) و با چک‌لیست بسنجید.

فصل

۴۲

استودیوی نقشه‌ی چندلایه

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۸ و ۴۱**

در این فصل یاد می‌گیرید

- نقشه‌ی چندلایه بسازید و لایه اضافه، مرتب، روشن و خاموش کنید.
- برای هر لایه سبک تک‌رنگ، طبقه‌بندی عددی یا دسته‌ای، برچسب و پنجره‌ی اطلاعات تنظیم کنید.
- نمای شروع و نقشه‌ی پایه را تعیین و نقشه را ذخیره کنید.
- سقف‌ها و نکته‌های کارایی نقشه‌ی چندلایه را بدانید.

سرفصل‌ها

۴۲-۱ نمایش با هم و نقشه‌ی ذخیره‌شده

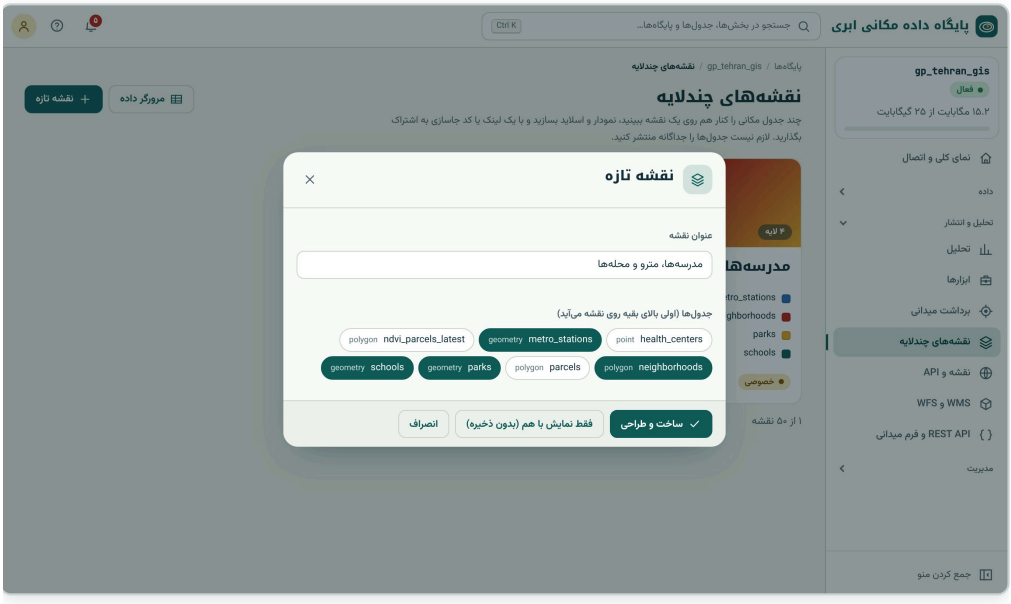
۴۲-۲ استودیو

۴۲-۳ کارایی: کاشی برداری

۴۲-۱ نمایش با هم و نقشه‌ی ذخیره‌شده

دو راه برای دیدن چند جدول روی یک نقشه هست:

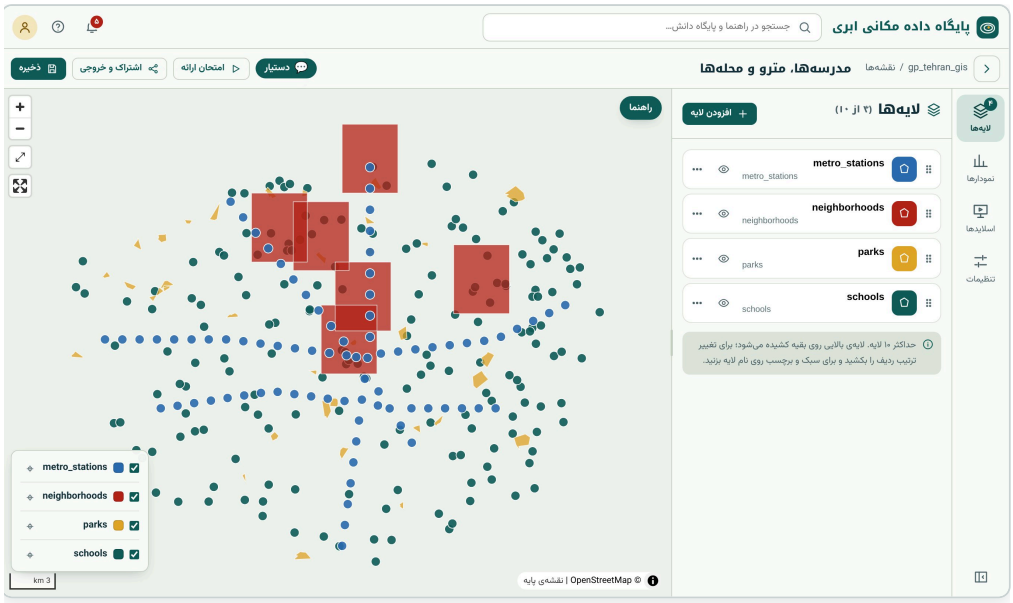
- **نمایش با هم** (بدون ذخیره): در «جدول‌ها» چند جدول مکانی را تیک بزنید و «نمایش با هم روی نقشه»؛ هر لایه رنگ جدا دارد. حداکثر ۱۰ جدول، بدون سقف تعداد.
- **نقشه‌ی چندلایه‌ی ذخیره‌شده:** منوی تحلیل و انتشار ← «نقشه‌های چندلایه» ← «نقشه‌ی تازه». نقشه با سبک، نمودار، اسلاید و نمای شروع ذخیره می‌شود و با لینک یا iframe به اشتراک گذاشته می‌شود. تعداد نقشه و لایه‌ی هر نقشه به پلن بستگی دارد.



مودال «نقشه‌ی تازه»: عنوان و جدول‌ها (اولی بالای بقیه روی نقشه می‌آید). «فقط نمایش با هم» بدون ذخیره باز می‌کند.

۴۲-۲ استودیو

پس از «ساخت و طراحی»، **استودیوی نقشه** باز می‌شود: نقشه در وسط، پنل در کنار با چهار زبانه‌ی **لایه‌ها**، **نمودارها**، **اسلایدها** و **تنظیمات**، و نوار بالا با «ذخیره»، «اشتراک و خروجی» و «اجرای ارائه».

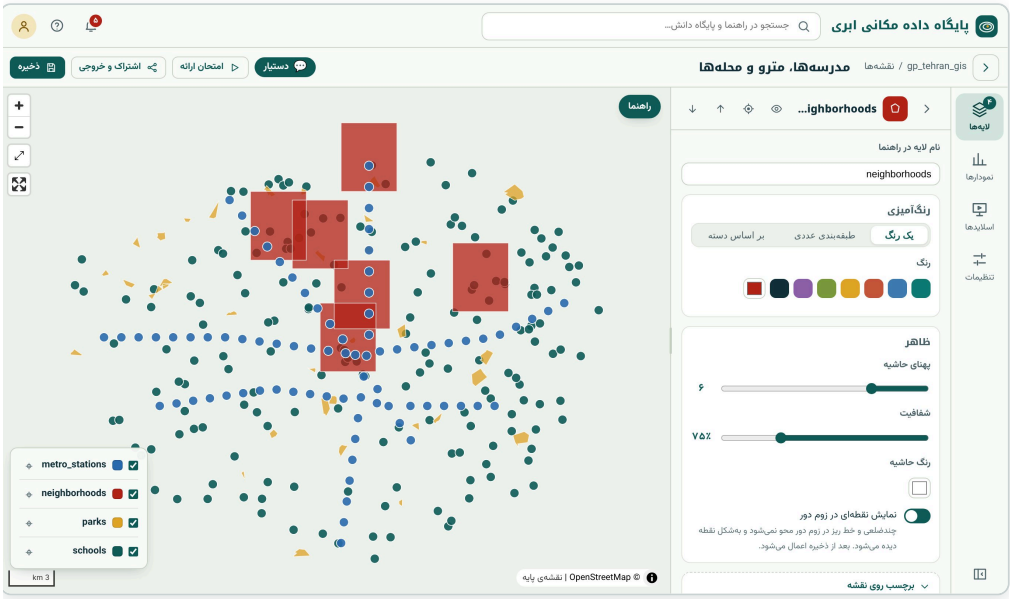


استودیوی نقشه با چهار لایه: ایستگاه‌ها، محله‌ها، پارک‌ها و مدرسه‌ها. راهنمای لایه‌ها پایین نقشه است.

لایه‌ها

- **ترتیب** با کشیدن؛ لایه‌ی بالاتر روی بقیه کشیده می‌شود. قاعده‌ی کلی: نقطه بالای خط، خط بالای چندضلعی.
- **روشن/خاموش** با آیکون چشم.
- ⌖ **رفتن به محدوده‌ی لایه**؛ لایه‌ی روشنی که بیرون از دید است نشان «بیرون از دید» با پیکان جهت می‌گیرد.
- **افزودن لایه** از جدول‌های پایگاه؛ یا از صفحه‌ی هر جدول با «افزودن به نقشه».

سبک هر لایه



سبک لایه‌ی محله‌ها: نام در راهنما، رنگ‌آمیزی (یک رنگ، طبقه‌بندی عددی، بر اساس دسته)، پهنای حاشیه، شفافیت و برچسب.

سبک	کاربرد	تنظیم‌ها
یک رنگ	نمایش وجود و مکان	رنگ، اندازه یا پهنا، شفافیت، حاشیه
طبقه‌بندی عددی	نقشه‌ی کروپلت و نماد درجه‌بندی‌شده	ستون عددی، روش (فاصله‌ی برابر، چندک، ...)، تعداد طبقه، طیف رنگ (فصل ۴۱)
بر اساس دسته	داده‌ی اسمی (نوع، کاربری، خط)	ستون دسته‌ای، رنگ هر دسته
برچسب	نام روی نقشه	ستون، اندازه
پنجره‌ی اطلاعات	کلیک روی عارضه	ستون‌هایی که نمایش داده شوند

نکته

پنجره‌ی اطلاعات و حریم داده. اگر برای پنجره‌ی اطلاعات ستون انتخاب کنید، **فقط همان ستون‌ها** (به‌علاوه‌ی ستون رنگ و برچسب) در کاشی‌های نقشه فرستاده می‌شوند. این یعنی ستون‌های حساس (تلفن، کد ملی، ...) در نقشه‌ی عمومی دیده نمی‌شوند، حتی برای کسی که داده‌ی کاشی را بخواند. پیش از عمومی کردن نقشه، ستون‌های پنجره را بازبینی کنید.

«نمایش نقطه‌ها در زوم دور»: چندضلعی و خط ریز در زوم دور به شکل نقطه دیده می‌شوند تا گم نشوند.

تنظیمات

نقشه‌ی پایه (از جمله نقشه‌ی پایه‌ی فارسی سامانه؛ فصل ۳۰)، **نمای شروع** (نمایی که نقشه با آن باز می‌شود؛ «ذخیره‌ی نمای فعلی») و توضیح نقشه.

۴۳-۳ کارایی: کاشی برداری

نقشه‌ی چندلایه داده را به شکل **کاشی برداری** (MVT) می‌فرستد: در هر بزرگ‌نمایی فقط کاشی‌های روی صفحه، ساده‌شده برای همان مقیاس (فصل ۱۸)، بارگذاری می‌شوند. برای همین سقف ۲۰۰۰ عارضه‌ی زبانه‌ی نقشه‌ی جدول (فصل ۸) اینجا وجود ندارد و جدول‌های صدهزار عارضه‌ای هم روان نمایش داده می‌شوند، به شرطی که شاخص مکانی داشته باشند (فصل ۲۰).

خودتان امتحان کنید

نقشه‌ای با محله‌ها (طبقه‌بندی عددی روی تراکم جمعیت؛ ابتدا ستون تراکم را با SQL فصل ۴۱ به جدول اضافه کنید)، پارک‌ها (سبز، نیمه‌شفاف) و مدرسه‌ها (بر اساس مقطع با سه رنگ) بسازید. برای مدرسه‌ها پنجره‌ی اطلاعات با نام و تعداد دانش‌آموز تنظیم کنید، نمای شروع را ذخیره و نقشه را ذخیره کنید.

خلاصه‌ی فصل

- «نمایش با هم» برای دیدن سریع (تا ۱۰ جدول)؛ «نقشه‌ی چندلایه» برای نقشه‌ی ماندگار با سبک، نمودار و اسلاید.
- استودیو: لایه‌ها، نمودارها، اسلایدها، تنظیمات؛ ترتیب با کشیدن.
- سبک: یک رنگ، طبقه‌بندی عددی، بر اساس دسته؛ برچسب و پنجره‌ی اطلاعات (فقط ستون‌های انتخابی فرستاده می‌شوند).
- کاشی برداری محدودیت تعداد عارضه ندارد؛ شاخص مکانی لازم است.

تمرین‌ها

۱. سطح ۱ نقشه‌ای از ایستگاه‌های مترو بر اساس خط (چهار رنگ) با برچسب نام بسازید.

۲. سطح ۲ نمای `ndvi_parcels_latest` (فصل ۳۶) را با طبقه‌بندی عددی پنج‌طبقه‌ای و طیف مناسب پوشش گیاهی نمایش دهید.

۳. سطح ۲ نتیجه‌ی کانون داغ فصل ۲۱ را با طرح رنگ واگرا نمایش دهید.

فصل

۴۳

نمودار و داشبورد روی نقشه

سطح: **متوسط** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۱۲ و ۴۲**

در این فصل یاد می‌گیرید

- نوع نمودار مناسب هر پرسش را انتخاب کنید.
- در استودیوی نقشه نمودار بسازید که با نقشه هم‌گام است.
- چیدمان داشبورد و نمودار در نقشه‌ی عمومی را بشناسید.
- بدانید هر نمودار پشت صحنه چه SQL تجمیعی است.

سرفصل‌ها

۴۳-۱ نقشه و نمودار، دو چشم یک داده

۴۳-۲ ساختن نمودار

۴۳-۳ کدام نمودار، برای کدام پرسش

۴۳-۴ داشبورد و نقشه‌ی عمومی

۲۳-۱ نقشه و نمودار، دو چشم یک داده

نقشه می‌گوید «کجا» و نمودار می‌گوید «چقدر» و «چگونه توزیع شده». کنار هم، تصویری کامل می‌سازند: نقشه‌ی تراکم مدرسه‌ها با هیستوگرام تعداد دانش‌آموز، یا نقشه‌ی سلامت مزرعه‌ها با سری زمانی NDVI. در استودیوی نقشه، نمودارها به نقشه **وصل‌اند**: کلیک روی یک میله یا دسته عارضه‌هایش را روی نقشه برجسته می‌کند و گزینه‌ی «فقط محدوده‌ی دیده‌شده» نمودار را با جابه‌جایی نقشه به‌روز می‌کند.

۲۳-۲ ساختن نمودار

زبانه‌ی «نمودارها» در استودیو ← «نمودار تازه». مودال سه بخش دارد: **لایه**، **نوع نمودار** و **تنظیمات**، با پیش‌نمایش زنده.

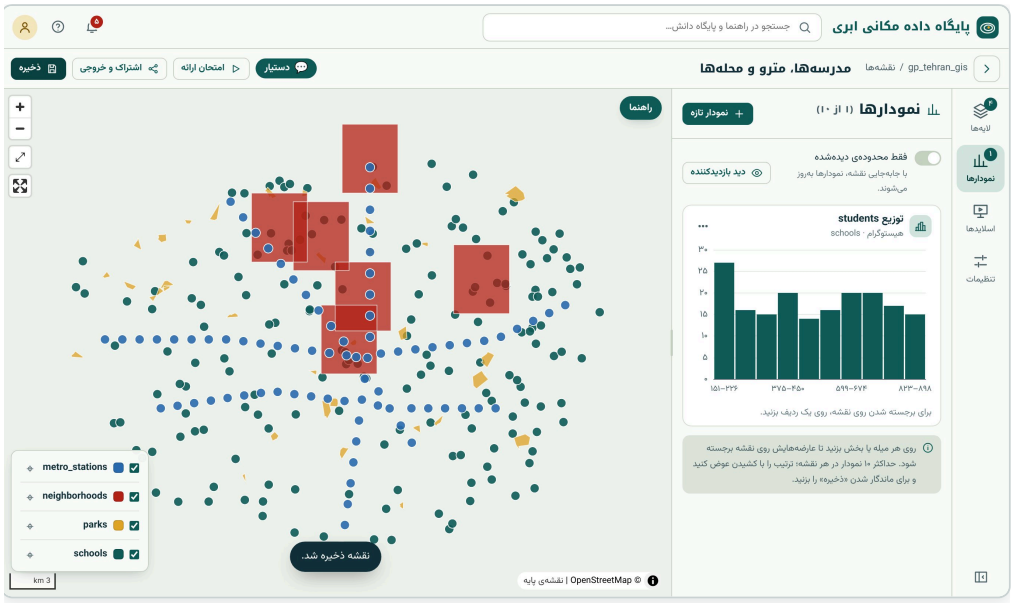
مودال «نمودار تازه»: لایه‌ی مدرسه‌ها، نوع «میله‌ای» و تنظیم ستون دسته و محاسبه. پیش‌نمایش زنده در سمت چپ.

۴۳-۳ کدام نمودار، برای کدام پرسش

نوع	پرسش	نمونه	تجمیع SQL (فصل ۱۲)
کارت عددی	یک عدد کلیدی	کل دانش‌آموزان	`sum(students)`
میله‌ای	مقایسه‌ی دسته‌ها	مدرسه به تفکیک مقطع	`count(*) GROUP BY level`
دایره‌ای	سهم از کل (دسته‌های کم)	سهم هر نوع مرکز درمانی	`count(*) GROUP BY type`
هیستوگرام	توزیع یک متغیر عددی	توزیع تعداد دانش‌آموز	`width_bucket`
سری زمانی	روند در زمان (تاریخ شمسی)	NDVI قطعه‌ها	`GROUP BY date_trunc(…)`
ستونی انباشته / ۱۰۰٪	ترکیب دسته‌ها در گروه‌ها	سهم کاربری در هر محله	دو ستون گروه‌بندی
پراکنش	رابطه‌ی دو متغیر	جمعیت و تعداد مدرسه	نمونه‌ی ۲۰۰۰ نقطه
رادار	مقایسه‌ی چند شاخص	پروفایل محله‌ها	چند تجمیع
جریان (Sankey)	از ← به	تغییر کاربری (فصل ۲۷)	خروجی «تغییر بین دو نسخه»
پروفیل ارتفاعی	ارتفاع در امتداد خط	مسیر کوهستانی	خط Z (فصل ۳۸)

هیستوگرام تعداد دانش‌آموز مدرسه‌ها با ده بازه. مرزهای هیستوگرام با طبقه‌بندی رنگی لایه هم‌مرز است.

همه‌ی تجمیع‌ها **در PostGIS** انجام می‌شود، نه در مرورگر؛ پس نمودار روی جدول‌های بزرگ هم سریع است. هیستوگرام می‌تواند **درصد تجمعی** هم نشان دهد و هم‌مرز با طبقه‌بندی رنگی لایه است؛ ابزاری عالی برای دیدن اینکه طبقه‌بندی چطور داده را بُریده است (فصل ۴۱).



نقشه‌ی ذخیره‌شده با نمودار در پنل. با کلیک روی هر میله، مدرسه‌های آن بازه روی نقشه برجسته می‌شوند.

۴۳-۴ داشبورد و نقشه‌ی عمومی

- **چیدمان داشبورد:** نمودارها را در کنار نقشه به شکل داشبورد بچینید (تا سقف نمودار پلن).
- **نقشه‌ی عمومی و iframe:** نمودارها در نقشه‌ی عمومی (فصل ۴۴) هم دیده می‌شوند. داده‌ی هر نمودار نشانی JSON خودش را دارد که برای برنامه‌های دیگر هم قابل استفاده است؛ هر بار خواندنش از سقف درخواست روزانه کم می‌شود.
- **از تحلیل به نمودار:** خروجی ابزارهایی مثل «سهم همپوشانی» و «شمارش در چندضلعی» با دکمه‌ی «نقشه و نمودار»، نقشه با نمودار پیشنهادی می‌سازد (فصل ۲۵).

هشدار

نمودار دایره‌ای را کم به کار ببرید. چشم زاویه و مساحت را بسیار بدتر از طول مقایسه می‌کند. برای بیش از چهار پنج دسته یا مقایسه‌ی دقیق، نمودار میله‌ای مرتب‌شده تقریباً همیشه بهتر است. نمودار دایره‌ای فقط وقتی پیام «سهم از کل» است و دسته‌ها کم‌اند.

خلاصه‌ی فصل

- نمودارهای استودیو به نقشه وصل‌اند: کلیک روی دسته عارضه‌ها را برجسته می‌کند؛ «فقط محدوده‌ی دیده‌شده» نمودار را با نقشه به‌روز می‌کند.
- انواع: کارت، میله‌ای، دایره‌ای، هیستوگرام (هم‌مرز با طبقه‌بندی)، سری زمانی شمسی، انباشته، پراکنش، رادار، Sankey، پروفیل ارتفاعی.
- تجمیع در PostGIS؛ نمودار در نقشه‌ی عمومی و iframe هم هست.

تمرین‌ها

۱. سطح ۱ نمودار میله‌ای تعداد مدرسه به تفکیک مقطع و کارت عددی کل دانش‌آموزان را به نقشه اضافه کنید.

۲. سطح ۲ SQL معادل هیستوگرام ده‌بازه‌ای تعداد دانش‌آموز را با `width_bucket` بنویسید و با نمودار مقایسه کنید.

۳. سطح ۲ برای جدول `ndvi_parcels` نمودار سری زمانی بسازید.

فصل

۴۴

اشتراک، ارائه، چاپ و مقایسه

سطح: **مبتدی** ●●○

زمان مطالعه: **۴۵ دقیقه**

پیش‌نیاز: **فصل ۴۲**

در این فصل یاد می‌گیرید

- نقشه را عمومی یا خصوصی کنید و با لینک یا iframe به اشتراک بگذارید.
- با اسلایدها از نقشه یک ارائه‌ی روایی بسازید.
- نقشه را با اجزای کامل به PDF و PNG چاپ کنید.
- دو لایه را با پرده‌ی کشویی مقایسه کنید و پروژه‌ی QGIS و سرویس WMS/WFS بگیرید.

سرفصل‌ها

۴۴-۱ اشتراک و خروجی

۴۴-۲ ارائه با اسلاید

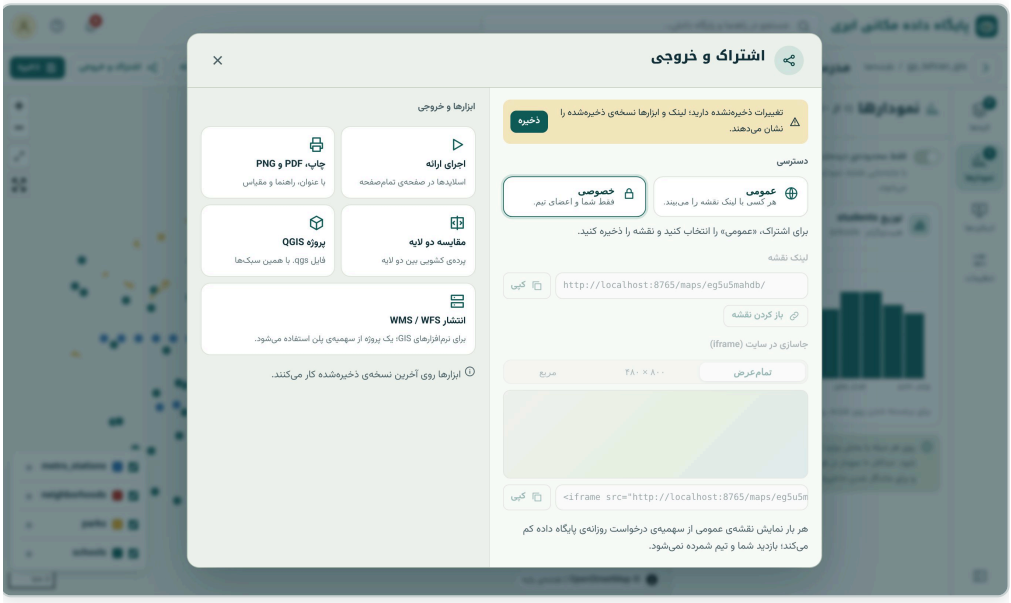
۴۴-۳ چاپ، PDF و PNG

۴۴-۴ مقایسه‌ی دو لایه

۴۴-۵ WMS/WFS و QGIS

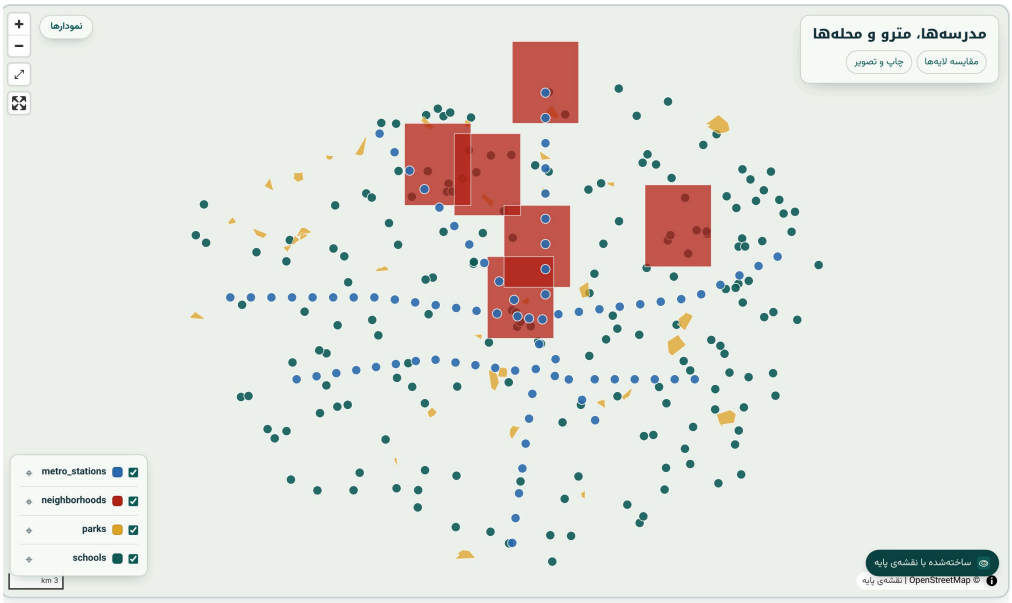
۴۴-۱ اشتراک و خروجی

دکمه‌ی «اشتراک و خروجی» در استودیو همه‌ی راه‌های بیرون بردن نقشه را یک‌جا دارد:



مودال «اشتراک و خروجی»: دسترسی (عمومی یا خصوصی)، لینک نقشه، کد iframe با اندازه، و ابزارهای ارائه، چاپ، مقایسه، پروژه‌ی QGIS و انتشار WMS/WFS.

- **دسترسی:** «خصوصی» فقط برای شما و اعضای تیم (فصل ۵۰)؛ «عمومی» برای هر کسی که لینک را دارد. خواندن نقشه‌ی عمومی از سقف درخواست روزانه‌ی پلن کم می‌شود.
- **لینک نقشه** و **کد جاسازی (iframe)** با اندازه‌ی دلخواه برای وب‌سایت یا وبلاگ.
- ابزارها روی **آخرین نسخه‌ی ذخیره‌شده** کار می‌کنند؛ پیش از اشتراک ذخیره کنید.

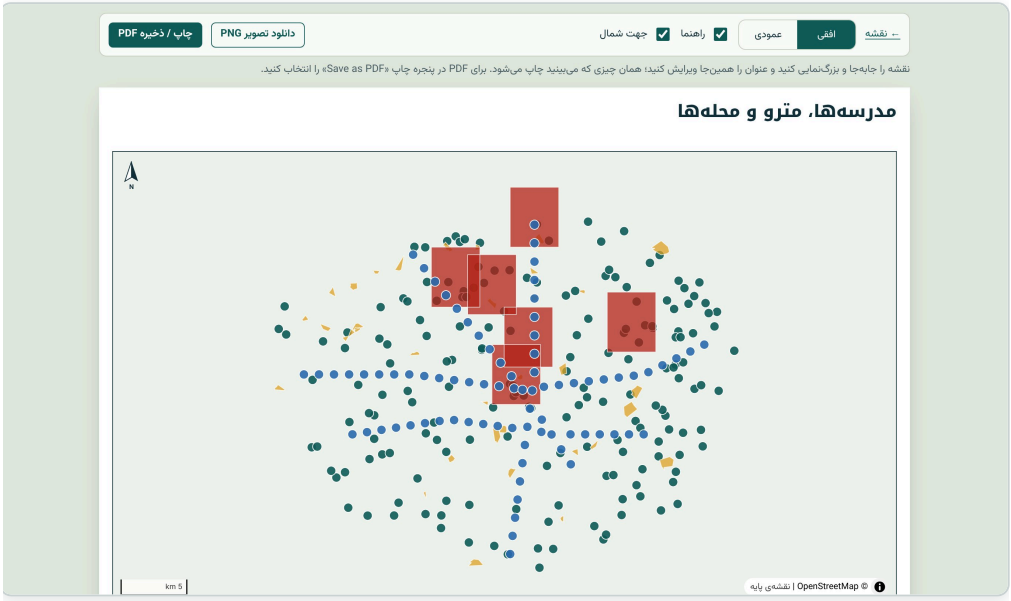


نقشه‌ی عمومی با راهنمای لایه‌ها و دکمه‌های «مقایسه‌ی لایه‌ها» و «چاپ و تصویر».

۴۴-۲ ارائه با اسلاید

در زبانه‌ی «اسلایدها»، «اسلاید از نمای فعلی» نما و لایه‌های روشن را با یک عنوان و متن نگه می‌دارد (تا ۳۰ اسلاید). «اجرای ارائه» نقشه را تمام‌صفحه می‌کند و با کلیدهای جهت، نقشه به نرمی از اسلایدی به اسلاید بعد می‌رود. این روش برای **روایت مکانی** (story map) عالی است: «اینجا مشکل است ← این علتش است ← این راه حل پیشنهادی است». صفحه‌ی ارائه‌ی نقشه‌ی عمومی بدون ورود باز می‌شود.

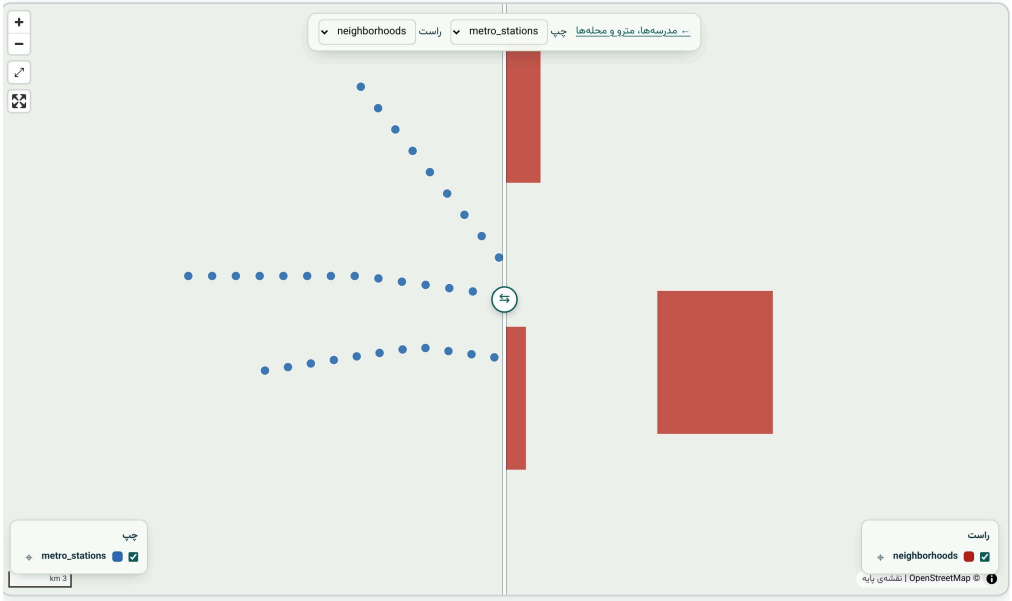
۴۴-۳ چاپ، PDF و PNG



صفحه‌ی چاپ: جهت افقی یا عمودی، راهنما، جهت شمال، عنوان؛ «چاپ / ذخیره‌ی PDF» و «دانلود تصویر PNG».

برگه‌ی چاپ A4 افقی یا عمودی است با عنوان، راهنما، مقیاس، جهت شمال، تاریخ شمسی و ذکر منبع نقشه‌ی پایه؛ همه‌ی اجزای «نقشه‌ی کامل» فصل ۴۱. پیش از چاپ می‌توانید نقشه را جابه‌جا و بزرگ‌نمایی کنید. «چاپ / ذخیره‌ی PDF» از پنجره‌ی چاپ مرورگر و «دانلود تصویر PNG» با کیفیت چاپ است.

۴۴-۴ مقایسه‌ی دو لایه



مقایسه‌ی کشویی: لایه‌ی چپ (ایستگاه‌ها) و راست (محله‌ها) روی دو نقشه‌ی هم‌گام با پرده‌ی وسط.

دو لایه روی دو نقشه‌ی هم‌گام با **پرده‌ی کشویی** نمایش داده می‌شوند؛ برای مقایسه‌ی قبل و بعد (کاربری ۱۴۰۰ و ۱۴۰۵)، دو دوره‌ی NDVI یا دو سبک. صفحه‌ی مقایسه برای نقشه‌ی عمومی با لینک باز است.

۴۴-۵ QGIS و WMS/WFS

- **دانلود پروژه‌ی QGIS:** فایل `.qgs` با همان ترتیب، رنگ‌آمیزی و برچسب لایه‌ها؛ اتصال به پایگاه داده بدون رمز ذخیره می‌شود و QGIS بار اول رمز را می‌پرسد. برای کارتوگرافی چاپی پیشرفته (Print Layout در QGIS) عالی است.
- **انتشار به‌صورت WMS/WFS:** همان پروژه با یک کلیک سرویس استاندارد OGC می‌شود (فصل ۴۵) و از سهمیه‌ی پروژه‌ی WMS/WFS پلن کم می‌کند.

خلاصه‌ی فصل

- «اشتراک و خروجی»: عمومی/خصوصی، لینک، iframe؛ روی آخرین نسخه‌ی ذخیره‌شده.
- اسلایدها (تا ۳۰) و «اجرای ارائه» برای روایت مکانی.
- چاپ A4 با عنوان، راهنما، مقیاس، شمال، تاریخ شمسی و منبع؛ PDF و PNG.
- مقایسه‌ی کشویی دو لایه؛ پروژه‌ی QGIS؛ انتشار WMS/WFS.

تمرین‌ها

۱. سطح ۱ نقشه‌ی فصل ۴۲ را عمومی کنید و در یک صفحه‌ی HTML ساده با iframe بگذارید.

۲. سطح ۱ یک ارائه‌ی پنج‌اسلایدی درباره‌ی دسترسی مدرسه‌ها به مترو بسازید.

۳. سطح ۲ نقشه‌ی تراکم جمعیت را با همه‌ی اجزای چک‌لیست فصل ۴۱ به PDF چاپ کنید.

۴. سطح ۲ پروژه‌ی QGIS نقشه را دانلود و در Print Layout یک برگه‌ی A3 بسازید.

فصل

۴۵

انتشار داده؛ کاشی برداری، OGC API و WMS/WFS

سطح: **متوسط** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل‌های ۴ و ۴۲**

در این فصل یاد می‌گیرید

- تفاوت کاشی برداری، OGC API Features، GeoJSON و WMS/WFS را بدانید و برای هر کار انتخاب کنید.
- یک جدول را به لایه‌ی وب عمومی یا خصوصی تبدیل کنید.
- کلید API بسازید، بفرستید و ایمن نگه دارید.
- پروژه‌ی QGIS را به سرویس WMS/WFS تبدیل کنید.
- سقف درخواست روزانه را مدیریت کنید.

سرفصل‌ها

۴۵-۱ چرا انتشار

۴۵-۲ کدام قالب، برای کدام کار

۴۵-۳ انتشار لایه

۴۵-۴ کلید API

۴۵-۵ QGIS Server با WFS و WMS

۴۵-۶ سقف درخواست روزانه

۴۵-۱ چرا انتشار

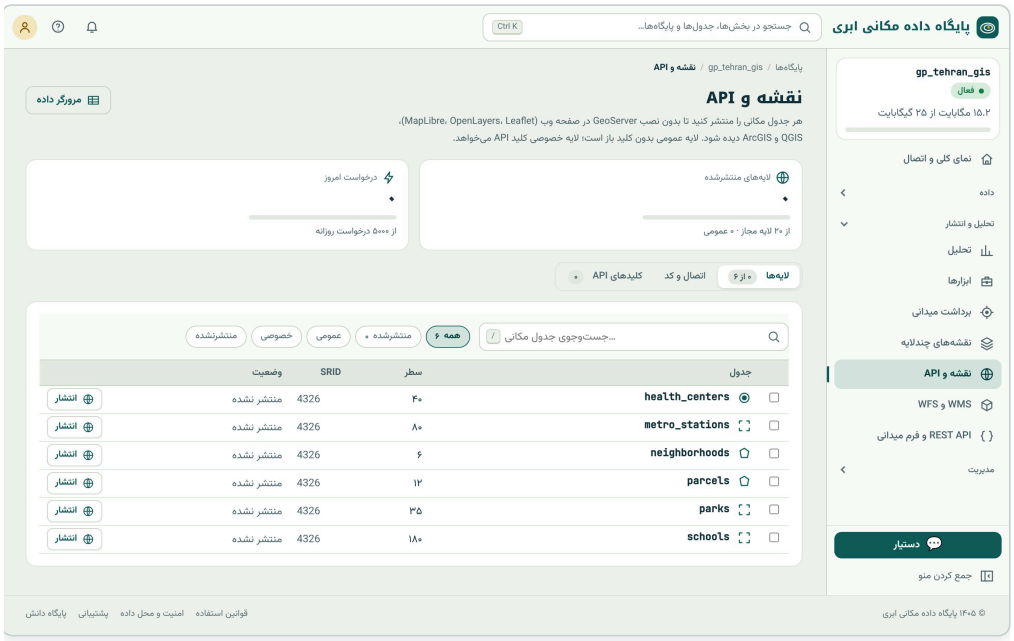
پایگاه داده‌ی شما را نرم‌افزارهایی می‌خوانند که رمز آن را دارند (فصل ۶). اما برای نقشه‌ی وب، برنامه‌ی موبایل، همکار سازمانی دیگر یا عموم مردم، **نباید** رمز پایگاه داده را بدهید. **انتشار** یک جدول را به سرویس وب استاندارد تبدیل می‌کند که فقط **خواندن** همان جدول را ممکن می‌کند، با کنترل عمومی یا خصوصی بودن و سقف استفاده (فصل ۴، استانداردهای OGC).

۴۵-۲ کدام قالب، برای کدام کار

قالب	چه می‌دهد	مناسب برای
کاشی برداری (MVT)	داده در کاشی‌های فشرده، فقط محدوده‌ی روی صفحه	نقشه‌ی وب سریع با هر حجم داده (MapLibre، OpenLayers، QGIS)
GeoJSON	همه‌ی عارضه‌ها یک‌جا	داده‌ی کم (تا چند صد عارضه) و پردازش در برنامه
OGC API – Features	عارضه‌ها با صفحه‌بندی و فیلتر محدوده، JSON	برنامه‌های مدرن، QGIS، تبادل استاندارد
WMS	تصویر نقشه با سبک پروژه‌ی QGIS	لایه‌ی زمینه در نرم‌افزارها، نقشه‌ی ثابت
WFS	عارضه‌ها (GML/GeoJSON) با استاندارد قدیمی‌تر	نرم‌افزارهای سازمانی قدیمی‌تر
REST	سطرهای جدول به JSON با فیلتر (فصل ۴۶)	برنامه‌نویسی

۴۵-۳ انتشار لایه

منوی تحلیل و انتشار ← **«نقشه و API»**:



صفحه‌ی «نقشه و API»: لایه‌های منتشرشده از سقف پلن، درخواست امروز از سقف روزانه، و فهرست جدول‌های مکانی با دکمه‌ی «انتشار».

۱ کنار جدول «انتشار» را بزنید (یا چند جدول را تیک بزنید و یک‌جا منتشر کنید).

۲ **عمومی یا خصوصی:** پیش‌فرض **خصوصی** است؛ فقط با کلید API خوانده می‌شود. فقط داده‌ای را عمومی کنید که می‌خواهید همه ببینند.

۳ در صورت نیاز ستون‌هایی را که منتشر می‌شوند محدود کنید.

۴ لایه نشانی‌های آماده دارد که در زبانه‌ی «اتصال و کد» نمایش داده می‌شود.

نشانی‌های یک لایه‌ی منتشرشده با نام `schools` :

نشانی	کاربرد
`/tiles/schools.json`	TileJSON (فرادادهٔ کاشی‌ها) برای MapLibre و QGIS
`/tiles/schools/{z}/{x}/{y}.pbf`	خود کاشی‌های برداری
`/ogc/schools/collections`	فهرست مجموعه‌های OGC API
`/ogc/schools/collections/<id>/items?bbox=…&limit=…`	عارضه‌ها به GeoJSON
`/m/schools/` و `/m/schools/embed/`	نقشهٔ عمومی تک‌لایه و نسخهٔ جاسازی (نقشه‌ساز)

در QGIS، کاشی برداری را با **New Generic Connection** ← **Vector Tiles** و نشانی TileJSON، و OGC API را با **WFS / OGC API – Features** ← **Data Source Manager** باز کنید.

بدانید

کاشی برداری چرا سریع است؟ نقشهٔ وب به هرم کاشی‌ها تقسیم می‌شود: در زوم ۰ یک کاشی همه‌ی جهان است و در هر زوم بعد، هر کاشی به چهار کاشی تقسیم می‌شود. برای هر کاشی فقط عارضه‌های داخل آن، **ساده‌شده برای همان زوم** (فصل ۱۸) و با مختصات فشرده فرستاده می‌شود. مرورگر فقط کاشی‌های روی صفحه را می‌خواهد. PostGIS این کاشی‌ها را با `ST_AsMVTGeom` و `ST_AsMVT` مستقیم از جدول می‌سازد:

```sql
SELECT ST_AsMVT(t, 'schools') FROM (
  SELECT name, level, ST_AsMVTGeom(ST_Transform(geom, 3857),
ST_TileEnvelope(14, 10528, 6449)) AS geom
  FROM schools WHERE geom && ST_Transform(ST_TileEnvelope(14,
10528, 6449), 4326)
) t;
```

۴۵-۴ کلید API

لایه‌ی خصوصی فقط با **کلید API** خوانده می‌شود. کلید را در زبانه‌ی «کلیدهای API» بسازید؛ با `gpk_` شروع می‌شود و **فقط یک بار** کامل نمایش داده می‌شود. چهار راه فرستادن:

```
Authorization: Bearer gpk_…
X-API-Key: gpk_…
Basic auth (کلید به‌جای رمز)
?key=gpk_…   (در نشانی؛ ساده ولی در گزارش‌های سرور و تاریخچه‌ی مرورگر می‌ماند)
```

> **هشدار**
>
> **کلید در کد مرورگر، کلید عمومی است.** هر چیزی در جاوااسکریپت صفحه‌ی وب برای همه قابل دیدن است. اگر کلید را در صفحه‌ی عمومی بگذارید، لایه‌ی خصوصی عملاً عمومی شده است. برای نقشه‌ی عمومی، لایه را **عمومی** کنید؛ کلید را فقط در **سرور** خودتان نگه دارید. کلید لورفته را **فوراً باطل** کنید (منوی ⋯ کلید) و کلید تازه بسازید.

۴۵-۵ QGIS Server با WFS و WMS

برای کسانی که سبک‌دهی دقیق QGIS یا استاندارد WMS لازم دارند، منوی «WMS و WFS» یک پروژه‌ی QGIS را به سرویس OGC تبدیل می‌کند:

- فایل پروژه‌ی QGIS (`.qgz` / `.qgs`) را آپلود کنید، یا پروژه‌ای را که در خود پایگاه داده ذخیره شده وارد کنید، یا از نقشه‌ی چندلایه «انتشار به‌صورت WMS/WFS» بزنید (فصل ۴۴).
- سامانه نشانی `/ows/<نام>` را برای WMS و در صورت فعال بودن WFS می‌دهد.
- لایه‌های ناسازگار (مثلاً فایل محلی که روی سرور نیست) با هشدار نمایش داده می‌شوند.
- هر پروژه یکی از سهمیه‌ی «پروژه‌ی WMS/WFS» پلن را می‌گیرد.

برای امکانات کامل GeoServer (سبک SLD، گروه لایه، کش کاشی)، امکانات سازمانی **GeoServer اختصاصی** را دارد (فصل ۵۲).

۴۵-۶ سقف درخواست روزانه

همه‌ی لایه‌های منتشرشده، نقشه‌های عمومی و نمودارهای آن‌ها با هم یک **سقف درخواست روزانه** دارند که در کارت «درخواست امروز» دیده می‌شود. هر کاشی یک درخواست است؛ پس نقشه‌ی پربازدید سریع مصرف می‌کند. پس از سقف، تا روز بعد خطای محدودیت برگردانده می‌شود. راه‌ها: کش سمت برنامه‌ی خودتان، محدود کردن بازه‌ی زوم، یا ارتقای پلن.

خلاصه‌ی فصل

- انتشار جدول را به سرویس فقط‌خواندنی وب تبدیل می‌کند؛ هرگز رمز پایگاه را در وب نگذارید.
- کاشی برداری برای نقشه‌ی سریع با هر حجم؛ GeoJSON برای داده‌ی کم؛ OGC API برای تبادل استاندارد؛ WMS/WFS با QGIS Server.
- لایه پیش‌فرض خصوصی است؛ کلید API (`gpk_…`) با Bearer، X-API-Key، Basic یا `?key=`؛ کلید در کد مرورگر عمومی است.
- سقف درخواست روزانه برای همه‌ی لایه‌ها و نقشه‌های عمومی مشترک است.

تمرین‌ها

۱. سطح ۱ جدول `metro_stations` را عمومی منتشر و در QGIS به‌عنوان Vector Tile باز کنید.

۲. سطح ۲ جدول `health_centers` را خصوصی منتشر کنید، کلید بسازید و با `curl -H "X-API-Key: …"` عارضه‌ها را از OGC API بخوانید.

۳. سطح ۲ یک پروژه‌ی QGIS با دو لایه از پایگاه بسازید، به‌صورت WMS منتشر و در QGIS دیگری به‌عنوان WMS اضافه کنید.

فصل

۴۶

ساخت برنامه‌ی نقشه‌ی وب؛ MapLibre، Leaflet، REST API و فرم میدانی

سطح: **پیشرفته** ●●●○○ | زمان مطالعه: **۹۰ دقیقه** | پیش‌نیاز: **فصل ۴۵**

در این فصل یاد می‌گیرید

- با MapLibre یک لایه‌ی کاشی برداری را در صفحه‌ی وب نمایش دهید.
- با Leaflet و OpenLayers داده‌ی منتشرشده را بخوانید.
- از REST API سامانه با فیلتر و صفحه‌بندی داده بخوانید و با کلید نوشتن، داده بنویسید.
- فرم جمع‌آوری میدانی بدون ورود بسازید.

سرفصل‌ها

۴۶-۱ صفحه‌ی وب با MapLibre

۴۶-۲ OpenLayers و Leaflet

۴۶-۳ REST API

۴۶-۴ فرم جمع‌آوری میدانی

۴۶-۱ صفحه‌ی وب با MapLibre

MapLibre GL JS کتابخانه‌ی متن‌باز نقشه‌ی وب برداری است که کاشی برداری را مستقیم ترسیم می‌کند. این صفحه‌ی کامل، لایه‌ی منتشرشده‌ی مدرسه‌ها را با رنگ بر اساس مقطع و پنجره‌ی اطلاعات نشان می‌دهد (`HOST` را با نشانی سامانه عوض کنید):

```
<!doctype html>
<html lang="fa" dir="rtl">
<head>
  <meta charset="utf-8">
  <link href="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.css"
rel="stylesheet">
  <script src="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-
gl.js"></script>
  <style>html, body, #map { margin: 0; height: 100%; }</style>
</head>
<body>
<div id="map"></div>
<script>
const map = new maplibregl.Map({
  container: "map",
  style: "https://HOST/basemap/style.json",           // نقشه‌ی پایه‌ی فارسی
سامانه
  center: [51.40, 35.70], zoom: 11
});
map.on("load", () => {
  map.addSource("schools", { type: "vector", url:
"https://HOST/tiles/schools.json" });
  map.addLayer({
    id: "schools", type: "circle", source: "schools", "source-layer":
"schools",
    paint: {
      "circle-radius": 5,
      "circle-color": ["match", ["get", "level"],
        "2#" ,"ابتدایی"e7d4f", "متوسطه اول", "#b8741a", "#0f6e6e"]
    }
  });
  map.on("click", "schools", (e) => {
    const p = e.features[0].properties;
    new maplibregl.Popup().setLngLat(e.lngLat).setHTML(`<b>${p.name}
</b><br>${p.students} دانش‌آموز`).addTo(map);
  });
});
</script>
</body>
</html>
```

`source-layer` باید هم‌نام لایه‌ی منتشرشده باشد. عبارت `["match", …]` همان سبک «بر اساس دسته» استودیو (فصل ۴۲) است. برای لایه‌ی خصوصی، کلید را به نشانی TileJSON بیفزایید، ولی هشدار فصل ۴۵ را به خاطر داشته باشید: کلید در صفحه‌ی عمومی، عمومی است.

۴۶-۲ Leaflet و OpenLayers

Leaflet ساده‌ترین کتابخانه‌ی نقشه‌ی وب است و با GeoJSON کار می‌کند؛ برای داده‌ی کم، از OGC API بخوانید:

```
const map = L.map("map").setView([35.70, 51.40], 11);
fetch("https://HOST/ogc/metro_stations/collections/metro_stations/items?limit=1000")
  .then((r) ⇒ r.json())
  .then((fc) ⇒ L.geoJSON(fc, { onEachFeature: (f, l) ⇒ l.bindPopup(f.properties.name) }).addTo(map));
```

OpenLayers کتابخانه‌ی کامل‌تری است که کاشی برداری را مستقیم با `VectorTileLayer` و `MVT` می‌خواند؛ برای برنامه‌های سازمانی با نیاز به سیستم مختصات گوناگون و ابزارهای ویرایش مناسب است. قاعده‌ی انتخاب: **داده‌ی زیاد ← کاشی برداری (MapLibre/OpenLayers)؛ داده‌ی کم و سادگی ← GeoJSON (Leaflet).**

۴۶-۳ REST API

منوی «REST API و فرم میدانی» جدول‌ها را به سرویس REST تبدیل می‌کند. نشانی `/rest/<جدول>/<سرویس>` سطرها را به JSON برمی‌گرداند:

پارامتر	کار	نمونه
`limit` ، `offset`	صفحه‌بندی (پیش‌فرض ۱۰۰، حداکثر ۱۰۰۰)	`?limit=50&offset=100`
`order`	مرتب‌سازی	`?order=-students`
`bbox`	محدوده	`?bbox=51.3,35.6,51.5,35.8`
ستون `__`عملگر	فیلتر	`?students__gt=600&level=ابتدایی`

```
curl -H "Authorization: Bearer gpk_…" \
  "https://HOST/rest/myservice/schools/?
level=ابتدایی&students__gt=600&order=-students&limit=5"
```

نوشتن: فقط کلیدی که مجوز «نوشتن» دارد، و فقط روی جدولی که نوشتن برایش مجاز شده، می‌تواند سطر بسازد (`POST`)، ویرایش کند (`PATCH`) یا حذف کند (`DELETE`). کلید نوشتن را **فقط** در سرور خودتان نگه دارید؛ هرگز در برنامه‌ی موبایل یا صفحه‌ی وب.

سطح حرفه‌ای

معادل SQL هر درخواست. درخواست بالا پشت صحنه تقریباً همین پرسش است: `SELECT … FROM schools WHERE level = 'ابتدایی' AND students > 600 ORDER BY students DESC LIMIT 5;` با پارامترهای ایمن (فصل ۶)، نقشی که فقط همان جدول را می‌بیند و سقف درخواست روزانه. این همان «سرور میانی» است که فصل ۶ برای برنامه‌های وب توصیه کرد، آماده و بدون برنامه‌نویسی.

۴۶-۴ فرم جمع‌آوری میدانی

فرم میدانی صفحه‌ای روی `/f/<فرم>` است که کاربران **بدون ورود** روی نقشه نقطه می‌گذارند و ویژگی‌ها را پر می‌کنند؛ داده مستقیم در جدول شما می‌نشیند. روی گوشی مثل برنامه نصب می‌شود. کاربردها: ثبت شکایت شهروندی، گزارش چاله، پرسش‌نامه‌ی مکانی، ثبت درخت‌های خیابانی توسط داوطلبان. ستون‌های فرم از ساختار جدول ساخته می‌شوند؛ پس پیش از ساختن

فرم، جدول را با قیدهای مناسب (فصل ۱۴) طراحی کنید تا داده‌ی بد وارد نشود. برای برداشت حرفه‌ای با GNSS و کار آفلاین، بخش «برداشت میدانی» (فصل‌های ۴۷ تا ۴۹) مناسب‌تر است.

خلاصه‌ی فصل

- MapLibre کاشی برداری را با `source` (TileJSON) و `source-layer` هم‌نام لایه نمایش می‌دهد؛ سبک با عبارت‌های `match` و `interpolate`.
- Leaflet برای GeoJSON و داده‌ی کم؛ OpenLayers برای برنامه‌های کامل.
- REST: `limit`، `offset`، `order`، `bbox` و فیلتر `ستون_عملگر`؛ نوشتن فقط با کلید نوشتن و فقط از سرور.
- فرم میدانی `/f/<نام>` جمع‌آوری بدون ورود و مستقیم در جدول.

تمرین‌ها

۱. سطح ۲ صفحه‌ی MapLibre این فصل را برای لایه‌ی منتشرشده‌ی خودتان بسازید و رنگ مدرسه‌ها را بر اساس تعداد دانش‌آموز (`interpolate`) تغییر دهید.

۲. سطح ۲ با REST API، ده مرکز درمانی با بیشترین تخت را بخوانید.

۳. سطح ۳ جدول «گزارش چاله» با قیدهای مناسب بسازید، فرم میدانی آن را راه بیندازید و با گوشی سه گزارش ثبت کنید؛ سپس گزارش‌ها را روی نقشه‌ی چندلایه ببینید.

بخش ۸

برداشت میدانی و نقشه‌برداری

GNSS، تصحیح RTK و NTRIP، برداشت آفلاین، محاسبات COGO، سطح و منحنی میزان و حجم؛ از گوشی تا گیرنده‌ی دوفرکانسه.

۴۷ GNSS؛ موقعیت‌یابی ماهواره‌ای، RTK و NTRIP

۴۸ پروژه‌ی برداشت میدانی

۴۹ محاسبات نقشه‌برداری؛ COGO، سطح، منحنی میزان و حجم

فصل

۴۷

GNSS؛ موقعیت‌یابی ماهواره‌ای، RTK و NTRIP

سطح: **متوسط** | زمان مطالعه: **۷۵ دقیقه** | پیش‌نیاز: **فصل ۳**

در این فصل یاد می‌گیرید

- اصول موقعیت‌یابی ماهواره‌ای و سامانه‌های GPS، گلوناس، گالیله و بیدو را توضیح دهید.
- منابع خطای موقعیت و نقش هندسه‌ی ماهواره‌ها (DOP) را بشناسید.
- تفاوت موقعیت‌یابی مستقل، DGPS و RTK و حالت‌های fixed و float را بدانید.
- با NTRIP و RTCM تصحیح بگیرید و جمله‌های NMEA را بخوانید.
- برای هر کار سطح دقت و تجهیزات مناسب را انتخاب کنید.

سرفصل‌ها

۴۷-۱ ماهواره‌ها چگونه موقعیت می‌دهند

۴۷-۲ منابع خطا

۴۷-۳ سطوح دقت

۴۷-۴ RTCM و NTRIP

۴۷-۵ جمله‌های NMEA

۴۷-۶ کدام دقت، برای کدام کار

۴۷-۱ ماهواره‌ها چگونه موقعیت می‌دهند

GNSS (سامانه‌ی ماهواره‌ای ناوبری جهانی) نام عمومی چند منظومه‌ی ماهواره‌ای است: **GPS** آمریکا، **گلوناس** روسیه، **گالیله** اروپا و **بیدو** چین. هر کدام ده‌ها ماهواره در ارتفاع حدود ۲۰ هزار کیلومتری دارند و گیرنده‌های امروزی هم‌زمان از چند منظومه استفاده می‌کنند؛ ماهواره‌ی بیشتر یعنی موقعیت پایدارتر، به‌ویژه بین ساختمان‌ها.

اصل کار ساده است: هر ماهواره پیوسته زمان دقیق (از ساعت اتمی) و موقعیت مداری‌اش را پخش می‌کند. گیرنده از اختلاف زمان فرستادن و رسیدن سیگنال، **فاصله** تا ماهواره را حساب می‌کند (فاصله = سرعت نور × زمان سفر). با فاصله تا دست‌کم **چهار** ماهواره، سه مختصات (x، y، z) و خطای ساعت ارزان گیرنده به دست می‌آید. به این روش **ترفیع فاصله‌ای** (trilateration) می‌گویند. چون ساعت گیرنده دقیق نیست، فاصله‌ی اندازه‌گیری‌شده را «شبه‌فاصله» (pseudorange) می‌نامند.

۴۷-۲ منابع خطا

منبع	توضیح	مقدار تقریبی
یون‌سپهر	لایه‌ی بالایی جو سیگنال را کند می‌کند	چند متر؛ گیرنده‌ی دوفرکانسه بیشترش را حذف می‌کند
وردسپهر	بخار آب و هوای پایین	کمتر از یک متر
ساعت و مدار ماهواره	خطای کوچک اطلاعات پخش‌شده	حدود یک متر
چندمسیری (multipath)	بازتاب سیگنال از ساختمان و سطوح	چند متر؛ دشمن کار شهری
هندسه‌ی ماهواره‌ها (DOP)	ماهواره‌های جمع در یک سمت آسمان	ضریب بزرگ‌کننده‌ی همه‌ی خطاها
ارتفاع آنتن و مرکز فاز	اندازه‌گیری نادرست ارتفاع ژالون	خطای مستقیم در ارتفاع

DOP (رقیق‌شدگی دقت) نشان می‌دهد هندسه‌ی ماهواره‌ها چقدر خطا را بزرگ می‌کند: HDOP برای افقی، VDOP برای ارتفاع. HDOP زیر ۲ خوب و بالای ۵ ضعیف است. در دره، کنار ساختمان

بلند یا زیر درخت، ماهواره‌های قابل دید کم و DOP زیاد می‌شود.

هشدار

ارتفاع GNSS ارتفاع بیضوی است. گیرنده ارتفاع را نسبت به بیضوی WGS84 می‌دهد؛ برای ارتفاع از سطح دریا باید ارتفاع ژئوئید کم شود (فصل ۳). همچنین ارتفاع آنتن (طول ژالون تا مرکز فاز) را درست وارد کنید؛ اشتباه در آن، ارتفاع همه‌ی نقطه‌ها را جابه‌جا می‌کند.

۳-۴۷ سطوح دقت

روش	چگونه	دقت افقی	تجهیزات
مستقل (گوشی)	شبه‌فاصله با یک گیرنده	۳ تا ۱۰ متر	گوشی، تبلت
مستقل دوفرکانسه	حذف بیشتر خطای یون‌سپهر	حدود ۱ متر	گوشی‌های جدید، گیرنده‌ی GIS
DGPS	تصحیح از ایستگاه مرجع با شبه‌فاصله	زیر ۱ متر	گیرنده با تصحیح
RTK	تصحیح لحظه‌ای با **فاز حامل** از ایستگاه مرجع نزدیک	۱ تا ۳ سانتی‌متر	گیرنده‌ی نقشه‌برداری + تصحیح
استاتیک (پس‌پردازش)	ثبت طولانی و پردازش بعدی	میلی‌متر تا سانتی‌متر	شبکه‌ی کنترل ژئودزی

RTK (کینماتیک لحظه‌ای) دقت سانتی‌متری را از **فاز موج حامل** سیگنال می‌گیرد: اندازه‌گیری‌ای بسیار دقیق‌تر از کد، که فقط باید تعداد طول موج‌های کامل («ابهام») آن معلوم شود. یک **ایستگاه مرجع** با مختصات معلوم، مشاهده‌هایش را لحظه‌ای برای گیرنده‌ی سیار (rover) می‌فرستد. دو حالت مهم:

- **Fixed** (ثابت): ابهام حل شده؛ دقت سانتی‌متری. فقط نقطه‌های fixed را برای نقشه‌برداری دقیق بپذیرید.
- **Float** (شناور): ابهام هنوز حل نشده؛ دقت ده‌ها سانتی‌متر.

۴۷-۴ NTRIP و RTCM

ایستگاه مرجع می‌تواند گیرنده‌ی دوم خودتان باشد، اما امروز بیشتر از **شبکه‌ی ایستگاه‌های دائمی** (CORS) استفاده می‌شود که تصحیح را از اینترنت پخش می‌کنند. پروتکل این پخش **NTRIP** است: گیرنده (یا نرم‌افزار واسط) به یک **کستر** (caster) با نشانی، پورت، نام کاربری، رمز و **mountpoint** (نام جریان تصحیح) وصل می‌شود و داده‌ی تصحیح را در قالب استاندارد **RTCM** می‌گیرد. NTRIP در حالت شبکه‌ای، موقعیت تقریبی گیرنده را هم (در جمله‌ی GGA) برای کستر می‌فرستد تا تصحیح مناسب همان محل ساخته شود.

در برداشت میدانی سامانه، کارت «تصحیحات RTK (NTRIP)» همین کار را انجام می‌دهد: مشخصات کستر را ذخیره و «دریافت» را می‌زنید؛ سرور تصحیح RTCM را از کستر می‌گیرد و مرورگر آن را روی اتصال سریال یا بلوتوث به گیرنده می‌نویسد (فصل ۴۸).

۴۷-۵ جمله‌های NMEA

گیرنده‌ها موقعیت را با قالب متنی استاندارد **NMEA 0183** گزارش می‌دهند. مهم‌ترین جمله، **GGA** است:

```
$GNGGA,083215.00,3542.1234,N,05123.4567,E,4,18,0.7,1210.35,M,-15.20,M,1
.0,0001*5B
        زمان UTC   عرض (ddmm.mmmm)  طول (dddmm.mmmm)  کیفیت تعداد HDOP
ارتفاع ژئوئید  سن تصحیح
```

ستون **کیفیت** مهم‌ترین عدد است: ۱ مستقل، ۲ DGPS، **۴ RTK fixed**، **۵ RTK float**، ۶ تخمینی. عرض و طول به شکل «درجه و دقیقه‌ی اعشاری» است (۳۵ درجه و ۴۲٫۱۲۳۴ دقیقه)، نه درجه‌ی اعشاری؛ تبدیلش با فرمول فصل ۳:

```sql
SELECT round(35 + 42.1234 / 60, 7) AS lat_dd, round(51 + 23.4567 /
60, 7) AS lon_dd,
       1210.35 - (-15.20)          AS ellipsoidal_height_m;
```

lat_dd	lon_dd	ellipsoidal_height_m
35.7020567	51.3909450	1225.55

در GGA، ارتفاع ستون نهم نسبت به **ژئوئید** (سطح دریا) و ستون یازدهم ارتفاع ژئوئید است؛ جمع آن‌ها ارتفاع بیضوی است.

۴۷-۶ کدام دقت، برای کدام کار

کار	دقت لازم	روش
ثبت مکان شکایت، درخت، تیر برق برای GIS عمومی	چند متر	گوشی
نقشه‌ی دارایی شهری، مرز مزرعه برای پایش ماهواره‌ای	زیر متر	گوشی دوفرکانسه یا DGPS
نقشه‌برداری عمرانی، پیاده‌کردن، حجم خاک	سانتی‌متر	RTK
شبکه‌ی کنترل، پایش جابه‌جایی سازه	میلی‌متر	استاتیک

خلاصه‌ی فصل

- GNSS (GPS، گلوناس، گالیله، بیدو) با فاصله تا دست‌کم چهار ماهواره موقعیت و خطای ساعت را حساب می‌کند.
- خطاها: یون‌سپهر، وردسپهر، مدار و ساعت، چندمسیری؛ DOP بزرگ‌کننده‌ی خطاست (HDOP زیر ۲ خوب).
- دقت: گوشی چند متر، DGPS زیر متر، RTK با فاز حامل سانتی‌متر (فقط fixed)، استاتیک میلی‌متر.
- NTRIP تصحیح RTCM را از کستر با mountpoint می‌آورد؛ GGA کیفیت (۴ fixed، ۵ float)، HDOP و ارتفاع را گزارش می‌دهد.
- ارتفاع GNSS بیضوی است؛ ارتفاع آنتن را درست وارد کنید.

تمرین‌ها

۱. سطح ۱ چرا دست‌کم چهار ماهواره لازم است، نه سه؟

۲. سطح ۲ جمله‌ی

`GA,101530.00,3239.4200,N,05140.6800,E,5,12,1.4,1575.2,M,-12.8,M,,*7A`

را بخوانید: موقعیت به درجه‌ی اعشاری، کیفیت و ارتفاع بیضوی چیست؟

۳. سطح ۲ برای برداشت مرز قطعه‌های کشاورزی برای «آمار هر قطعه» (فصل ۳۶)، چه دقتی کافی است؟ با توجه به پیکسل ۱۰ متری Sentinel-2 پاسخ دهید.

فصل

۴۸

پروژه‌ی برداشت میدانی

سطح: **مبتدی** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۷ و ۴۷**

در این فصل یاد می‌گیرید

- پروژه‌ی برداشت با سیستم مختصات و کدهای عارضه بسازید.
- با GPS گوشی یا گیرنده‌ی خارجی (و تصحیح RTK) نقطه، خط و چندضلعی برداشت کنید.
- بدون اینترنت کار کنید و داده را بعداً همگام کنید.
- ساختار جدول‌های پروژه را بشناسید و داده را وارد و صادر کنید.

سرفصل‌ها

۴۸-۱ برداشت میدانی در پایگاه داده

۴۸-۲ ساختن پروژه

۴۸-۳ برداشت

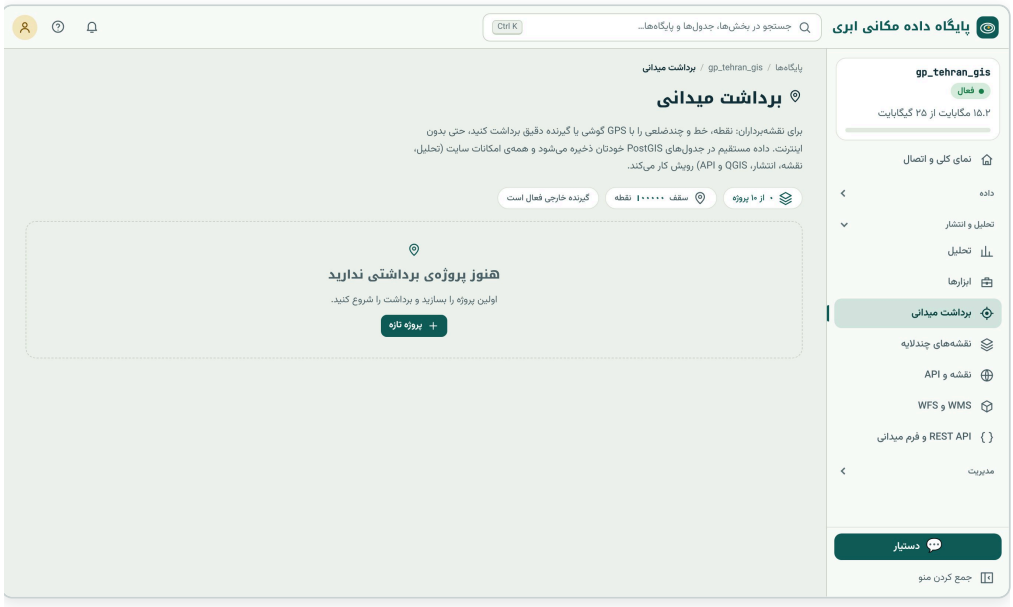
۴۸-۴ کار بدون اینترنت

۴۸-۵ ورود و خروج

۴۸-۶ داده‌ی برداشت در بقیه‌ی سامانه

۴۸-۱ برداشت میدانی در پایگاه داده

در روش سنتی، نقشه‌بردار نقطه‌ها را در دفترچه‌ی گیرنده ثبت می‌کند، در دفتر به رایانه منتقل می‌کند، به CAD می‌برد و بعد کسی آن را به GIS وارد می‌کند. در بخش **«برداشت میدانی»** پایگاه داده مکانی ابری، نقطه از همان لحظه‌ی ثبت در **جدول PostGIS** پروژه می‌نشیند و همه‌ی امکانات کتاب (SQL، نقشه، انتشار، تحلیل) رویش کار می‌کند.



بخش «برداشت میدانی»: فهرست پروژه‌ها و دکمه‌ی «پروژه‌ی تازه».

۴۸-۲ ساختن پروژه

منوی تحلیل و انتشار ← «برداشت میدانی» ← «پروژه‌ی تازه»: **نام**، **سیستم مختصات** (UTM ناحیه‌ی ۳۸ تا ۴۱ برای کار متری و نقشه‌برداری؛ یا WGS84) و **قالب کدهای عارضه** (فهرست کدهایی مثل GR برای زمین، TR برای درخت، BL برای ساختمان که هنگام برداشت انتخاب می‌کنید). هر پروژه یک **schema** جداگانه با نام `s_` و هشت حرف در پایگاه شما می‌سازد، با شش جدول ثابت:

جدول	محتوا
`sessions`	جلسه‌های برداشت (تاریخ، کاربر، منبع موقعیت)
`points`	نقطه‌ها با شماره، کد، توضیح، ارتفاع، دقت، کیفیت تصحیح
`lines`	خط‌ها (مسیر، لبه‌ی جاده)
`polygons`	چندضلعی‌ها (قطعه، ساختمان)
`control_points`	نقطه‌های کنترل با مختصات معلوم
`codes`	کدهای عارضه‌ی پروژه

۴۸-۳ برداشت

استودیوی برداشت روی گوشی، تبلت یا رایانه باز می‌شود. منبع موقعیت را انتخاب کنید:

- **GPS گوشی / رایانه:** بدون سخت‌افزار اضافه؛ مرورگر مجوز موقعیت می‌خواهد. دقت چند متر (فصل ۴۷)؛ برای GIS عمومی کافی، برای نقشه‌برداری دقیق نه.
- **گیرنده‌ی خارجی (کابل یا بلوتوث سریال)** و **گیرنده‌ی خارجی (بلوتوث BLE):** هر گیرنده‌ای که NMEA می‌دهد؛ فقط در Chrome یا Edge (که Web Serial و Web Bluetooth دارند) و در پلن‌های پولی.

در زبانه‌ی «برداشت»، نوع عارضه (نقطه، خط، چندضلعی) را بزنید و «ثبت نقطه». تنظیم‌های مهم:

- **شماره و کد و توضیح** هر نقطه.
- **ارتفاع آنتن** (فصل ۴۷).
- **میانگین‌گیری:** چند ثانیه موقعیت را میانگین می‌گیرد تا نوسان کم شود.
- **حداکثر خطای مجاز:** نقطه‌ای که دقت گزارش‌شده‌اش بدتر است ثبت نمی‌شود؛ برای RTK مثلاً ۳ سانتی‌متر و فقط fixed.

برای دقت سانتی‌متری، کارت **«تصحیحات (RTK (NTRIP»** را با نشانی کستر، پورت، کاربر، رمز و mountpoint پر و «دریافت» را بزنید (فصل ۴۷).

۴۸-۴ کار بدون اینترنت

استودیو یک **برنامه‌ی آفلاین** (PWA) است: در میدان بدون اینترنت، نقطه‌ها روی دستگاه ذخیره می‌شوند و وقتی اینترنت برگشت، با «ارسال» (یا خودکار) به پایگاه داده می‌روند. هر نقطه یک شناسه‌ی یکتای سمت دستگاه (`client_uuid`) دارد؛ پس ارسال دوباره، سطر تکراری نمی‌سازد (همان منطق upsert فصل ۱۴). برای زمینه‌ی نقشه در میدان، در «ابزار ← نقشه‌ی آفلاین» محدوده‌ی کار را با سطح جزئیات دلخواه روی دستگاه ذخیره کنید (حداکثر ۸٬۰۰۰ کاشی در هر دانلود).

۴۸-۵ ورود و خروج

در زبانه‌ی «فایل»:

- **ورود:** نقطه‌های CSV، TXT، XYZ یا GeoJSON با انتخاب ترتیب ستون‌ها (مثلاً «شماره، شرقی، شمالی، ارتفاع، کد»)؛ حداکثر ۵ مگابایت. برای آوردن برداشت توتال استیشن یا گیرنده‌ی دیگر.
- **خروجی:** **DXF** (برای اتوکد و نرم‌افزارهای عمرانی)، GeoJSON، CSV و **گزارش چاپی / PDF**.

برای همگام‌سازی از نرم‌افزار دیگر، API پروژه آیتم‌های JSON را به `/api/v1/survey/<نام-پروژه>/sync` می‌گیرد (تا ۱۰۰۰ آیتم در هر درخواست، با کلید API دارای اجازه‌ی نوشتن).

۴۸-۶ داده‌ی برداشت در بقیه‌ی سامانه

چون داده در جدول‌های معمولی PostGIS است:

```sql
-- نقطه‌های امروز با کیفیت RTK fixed
SELECT number, code, ST_Z(geom) AS h FROM s_ab12cd34.points
WHERE captured_at::date = current_date AND fix_quality = 4;
```

(نام schema و ستون‌ها را از زبانه‌ی «ستون‌ها»ی جدول در مرورگر داده ببینید.) نقطه‌ها در QGIS باز می‌شوند، در نقشه‌ی چندلایه نمایش داده می‌شوند، منتشر می‌شوند و ورودی ابزارهای تحلیل‌اند؛ و برای محاسبات نقشه‌برداری، فصل ۴۹.

خلاصه‌ی فصل

- هر پروژه‌ی برداشت یک schema (s_…) با شش جدول: lines، points، sessions، codes، control_points، polygons.
- منبع موقعیت: GPS گوشی (چند متر) یا گیرنده‌ی خارجی NMEA با سریال/BLE در Chrome/Edge؛ RTK با NTRIP.
- تنظیم‌ها: کد، ارتفاع آنتن، میانگین‌گیری، حداکثر خطای مجاز.
- آفلاین با همگام‌سازی بدون تکرار (client_uuid) و نقشه‌ی پایه‌ی آفلاین.
- ورود CSV/TXT/XYZ/GeoJSON؛ خروجی PDF، GeoJSON، CSV، DXF؛ API همگام‌سازی.

تمرین‌ها

۱. سطح ۱ یک پروژه‌ی برداشت در UTM ناحیه‌ی شهر خود بسازید و با GPS گوشی ده درخت حیاط دانشگاه را با کد TR ثبت کنید.

۲. سطح ۲ یک نقطه را پنج بار بدون میانگین‌گیری و پنج بار با میانگین‌گیری ده‌ثانیه‌ای ثبت کنید؛ پراکندگی دو گروه را با SQL (انحراف معیار مختصات) مقایسه کنید.

۳. سطح ۲ حالت هواپیما را روشن کنید، سه نقطه ثبت کنید، دوباره آنلاین شوید و ارسال را بررسی کنید.

فصل

۴۹

محاسبات نقشه‌برداری؛ COGO، سطح، منحنی میزان و حجم

سطح: **متوسط** زمان مطالعه: **۹۰ دقیقه**

پیش‌نیاز: **فصل‌های ۱۶، ۱۸ و ۴۸**

در این فصل یاد می‌گیرید

- فاصله، آزیموت، مساحت و محیط را از نقطه‌های برداشت حساب کنید.
- پیاده‌کردن نقطه را بفهمید.
- از نقطه‌های دارای ارتفاع سطح TIN بسازید.
- حجم خاک‌برداری و خاک‌ریزی نسبت به یک تراز و تغییر حجم بین دو برداشت را حساب کنید.
- همه‌ی این محاسبات را با SQL هم انجام دهید.

سرفصل‌ها

۴۹-۱ داده‌ی این فصل

۴۹-۲ هندسه‌ی مختصاتی :COGO

۴۹-۳ پیاده‌کردن

۴۹-۴ سطح TIN

۴۹-۵ منحنی میزان از TIN

۴۹-۶ حجم خاک‌برداری و خاک‌ریزی

۴۹-۱ داده‌ی این فصل

برای کدهای این فصل، ۶۴ نقطه‌ی برداشت با ارتفاع (شبکه‌ای ۸×۸ با فاصله‌ی ۲۵ متر در UTM ناحیه‌ی ۳۹) را در جدول `survey_points` می‌سازیم؛ فایل `survey_points.sql` همراه کتاب است:

```sql
SELECT count(*) AS points,
       round(min(ST_Z(geom))::numeric, 2) AS h_min,
round(max(ST_Z(geom))::numeric, 2) AS h_max,
       ST_SRID(geom) AS srid
FROM survey_points
GROUP BY ST_SRID(geom);
```

points	h_min	h_max	srid
64	1181.54	1201.19	32639

۴۹-۲ هندسه‌ی مختصاتی: COGO

COGO (هندسه‌ی مختصاتی) محاسبات پایه‌ی نقشه‌برداری از روی مختصات است. در استودیوی برداشت، در زبانه‌ی «نقاط» دو نقطه را تیک بزنید تا **فاصله و آزیموت** بیاید؛ سه نقطه یا بیشتر (به ترتیب) **مساحت و محیط** می‌دهد. همین با SQL، در سیستم UTM که متری است (فصل ۳):

```sql
SELECT a.name AS from_pt, b.name AS to_pt,
       round(ST_Distance(a.geom, b.geom)::numeric, 3)            AS
horizontal_m,
       round(ST_3DDistance(a.geom, b.geom)::numeric, 3)          AS
slope_m,
       round((ST_Z(b.geom) - ST_Z(a.geom))::numeric, 3)          AS
dh_m,
       round(degrees(ST_Azimuth(a.geom, b.geom))::numeric, 4)    AS
azimuth_deg
FROM survey_points a, survey_points b
WHERE a.name = '1' AND b.name = '64';
```

from_pt	to_pt	horizontal_m	slope_m	dh_m	azimuth_deg
1	64	247.487	247.487	0.140	45.0000

آزیموت **شبکه** است (نسبت به شمال شبکه‌ی UTM)، نه شمال جغرافیایی؛ اختلاف این دو (همگرایی نصف‌النهارها) نزدیک نصف‌النهار مرکزی ناچیز و در لبه‌ی ناحیه تا حدود دو درجه است.

مساحت چندضلعی از نقطه‌های مرزی (مثلاً چهار گوشه‌ی محدوده):

```sql
SELECT round(ST_Area(poly)::numeric, 2) AS area_m2,
round(ST_Perimeter(poly)::numeric, 2) AS perimeter_m
FROM (SELECT ST_MakePolygon(ST_MakeLine(ARRAY(
        SELECT ST_Force2D(geom) FROM survey_points WHERE name IN ('1',
'8', '64', '57', '1')
        ORDER BY array_position(ARRAY['1', '8', '64', '57'], name)) ||
ST_Force2D((SELECT geom FROM survey_points WHERE name = '1')))) AS
poly) x;
```

area_m2	perimeter_m
30625.00	700.00

۴۹-۳ پیاده‌کردن

پیاده‌کردن (stakeout) عکس برداشت است: نقطه‌ای با مختصات طراحی (مثلاً گوشه‌ی پی ساختمان) را روی زمین پیدا و میخ‌کوبی می‌کنیم. در استودیو، «ابزار ← پیاده‌سازی نقطه» جهت و فاصله تا نقطه‌ی هدف را **زنده** نشان می‌دهد تا نقشه‌بردار به سمتش برود و وقتی فاصله به حد مجاز رسید، نقطه‌ی پیاده‌شده را ثبت کند. پایه‌ی محاسبه همان فاصله و آزیموت بالا بین موقعیت لحظه‌ای و نقطه‌ی هدف است.

۴۹-۴ سطح TIN

برای ارتفاع **بین** نقطه‌های برداشت، سطح پیوسته لازم است. روش استاندارد نقشه‌برداری، **شبکه‌ی نامنظم مثلثی** (TIN) است: نقطه‌ها با **مثلث‌بندی دلونی** (فصل ۱۸) به هم وصل می‌شوند، طوری که دایره‌ی محیطی هیچ مثلثی نقطه‌ی دیگری را در بر نگیرد (مثلث‌های تا حد

ممکن «چاق»). هر مثلث یک صفحه‌ی تخت است و ارتفاع هر نقطه‌ی داخلش با درون‌یابی خطی به دست می‌آید. برخلاف رستر (فصل ۳۸)، TIN نقطه‌های واقعی برداشت را دقیقاً نگه می‌دارد.

```sql
SELECT count(*)                                  AS triangles,
       round(sum(ST_Area(t))::numeric)           AS area_m2,
       round(min(ST_Area(t))::numeric, 1)        AS smallest_m2
FROM (SELECT (ST_Dump(ST_DelaunayTriangles(ST_Collect(geom)))).geom AS
t FROM survey_points) tin;
```

triangles	area_m2	smallest_m2
98	30625	312.5

در استودیوی برداشت، «ابزار ← سطح و منحنی میزان» این سطح را می‌سازد. **خط شکست** (breakline، مثل لبه‌ی جوی یا تاج دیوار) باید به‌عنوان قید در مثلث‌بندی بیاید تا مثلثی از روی آن نگذرد؛ وگرنه سطح لبه‌ی تیز را «صاف» می‌کند.

۴۹-۵ منحنی میزان از TIN

منحنی میزان با بریدن هر مثلث با صفحه‌های افقی (هر ۱ متر، هر ۰٫۵ متر) و وصل کردن قطعه‌ها ساخته می‌شود. استودیو منحنی‌ها را با فاصله‌ی دلخواه می‌سازد و می‌توانید آن‌ها را در جدول ذخیره و به **DXF** برای اتوکد صادر کنید. برای منحنی میزان از DEM رستری، فصل ۳۸.

۴۹-۶ حجم خاک‌برداری و خاک‌ریزی

پرسش رایج پروژه‌های عمرانی: «برای تسطیح این محدوده تا تراز ۱۱۹۰، چقدر خاک باید برداشت (cut) و چقدر ریخت (fill)؟». با TIN، حجم هر مثلث تقریباً منشوری است با قاعده‌ی مساحت افقی مثلث و ارتفاعِ میانگینِ اختلاف سه رأس با تراز:

$$V \approx \sum_{i} A_i \cdot (\bar{z}_i - z_{level})$$

که مثبت‌ها خاک‌برداری و منفی‌ها خاک‌ریزی‌اند:

```sql
WITH tin AS (SELECT
(ST_Dump(ST_DelaunayTriangles(ST_Collect(geom)))).geom AS t FROM
survey_points),
tri AS (
  SELECT ST_Area(t) AS a,
         (ST_Z(ST_PointN(ST_ExteriorRing(t), 1)) +
ST_Z(ST_PointN(ST_ExteriorRing(t), 2))
          + ST_Z(ST_PointN(ST_ExteriorRing(t), 3))) / 3 AS z_mean
  FROM tin
)
SELECT 1190 AS level_m,
       round(sum(a * greatest(z_mean - 1190, 0))::numeric) AS cut_m3,
       round(sum(a * greatest(1190 - z_mean, 0))::numeric) AS fill_m3,
       round(sum(a * (z_mean - 1190))::numeric)            AS net_m3
FROM tri;
```

level_m	cut_m3	fill_m3	net_m3
1190	69900	57711	12189

حجم خاک‌برداری کمی بیشتر از خاک‌ریزی است؛ یعنی با تراز ۱۱۹۰ حدود ۱۲ هزار متر مکعب خاک اضافه می‌ماند. ترازی که خاک‌برداری و خاک‌ریزی را برابر کند (تراز **متعادل**) کمی بالاتر است؛ تمرین ۳. این روش «میانگین سه رأس» در مثلث‌هایی که تراز از وسطشان می‌گذرد تقریبی است؛ نرم‌افزارهای دقیق (و ابزار سطح استودیو) این مثلث‌ها را با خط تراز می‌برند.

تغییر حجم بین دو برداشت (مثلاً دپوی معدن در ابتدا و پایان ماه) با همین منطق، اما به جای تراز ثابت، سطح برداشت اول به‌عنوان مرجع: تفاضل ارتفاع دو سطح در هر نقطه ضرب در مساحت. ابزار سطح استودیو این مقایسه را بین دو جلسه‌ی برداشت انجام می‌دهد.

در دنیای واقعی

پرداخت پیمانکار. در یک پروژه‌ی راه‌سازی، حجم خاک‌برداری هر ماه از تفاضل سطح TIN برداشت آغاز و پایان ماه حساب می‌شود و مبنای صورت‌وضعیت پیمانکار است. با داده‌ی برداشت در جدول‌های PostGIS، هر دو طرف (کارفرما و پیمانکار) به همان داده و همان پرسش دسترسی دارند و اختلاف بر سر عدد کمتر می‌شود.

خلاصه‌ی فصل

- COGO: فاصله‌ی افقی و مایل، اختلاف ارتفاع، آزیموت شبکه، مساحت و محیط؛ در UTM و با توابع `3D` .
- پیاده‌کردن: جهت و فاصله‌ی زنده تا نقطه‌ی طراحی.
- TIN با مثلث‌بندی دلونی (`ST_DelaunayTriangles`)؛ خط شکست برای لبه‌های تیز.
- منحنی میزان از برش مثلث‌ها؛ خروجی DXF.
- حجم ≈ Σ مساحت × (ارتفاع میانگین - تراز)؛ خاک‌برداری، خاک‌ریزی، خالص؛ تغییر حجم بین دو برداشت.

تمرین‌ها

۱. سطح ۲ فاصله و آزیموت بین نقطه‌های ۸ و ۵۷ را حساب کنید.

۲. سطح ۲ حجم خاک‌برداری و خاک‌ریزی را برای ترازهای ۱۱۸۵، ۱۱۹۰ و ۱۱۹۵ در یک پرسش با `generate_series` بگیرید.

۳. سطح ۳ تراز متعادل (خالص صفر) را با جست‌وجو در ترازهای ۰٫۱ متری پیدا کنید. چه رابطه‌ای با میانگین وزنی ارتفاع مثلث‌ها دارد؟

۴. سطح ۳ ارتفاع همه‌ی نقطه‌ها را در یک جدول کپی ۵۰ سانتی‌متر کم کنید (برداشت دوم) و تغییر حجم بین دو برداشت را حساب کنید.

بخش ۹

حرفه‌ای و سازمانی

تیم و نقش‌ها، امنیت، بکاپ و بازگردانی، کارایی، برنامه‌نویسی با Python و Node.js، امکانات سازمانی، تدریس با کلاس و پروژه‌های جامع پایانی.

۵۰ تیم، نقش‌ها و امنیت داده

۵۱ بکاپ، بازگردانی، مصرف و کارایی

۵۲ امکانات سازمانی

۵۳ برنامه‌نویسی با پایگاه داده‌ی مکانی؛ تابع، تریگر، Python و خودکارسازی

۵۴ پایگاه داده مکانی ابری در کلاس

۵۵ پروژه‌های جامع

فصل

۵۰

تیم، نقش‌ها و امنیت داده

سطح: **متوسط** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۵ و ۶**

در این فصل یاد می‌گیرید

- اعضای تیم را با نقش مناسب (مشاهده‌گر، ویرایشگر) دعوت کنید.
- برای نرم‌افزارها کاربر جداگانه‌ی پایگاه داده با کمترین دسترسی بسازید.
- اصل کمترین دسترسی را با GRANT و REVOKE در SQL پیاده کنید.
- حریم خصوصی داده‌ی مکانی و امنیت رمز و کلید را رعایت کنید.

سرفصل‌ها

۵۰-۱ دو نوع دسترسی

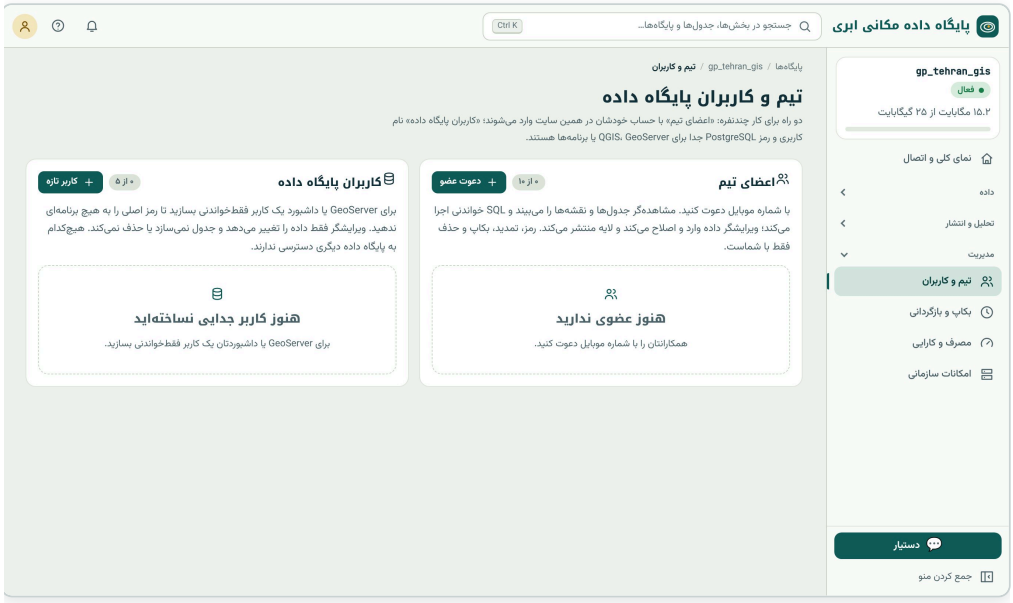
۵۰-۲ کمترین دسترسی با SQL

۵۰-۳ حریم خصوصی داده‌ی مکانی

۵۰-۴ فهرست کنترل امنیت

۵۰-۱ دو نوع دسترسی

صفحه‌ی «تیم و کاربران» (منوی مدیریت) دو راه جداگانه برای کار چندنفره دارد:



صفحه‌ی «تیم و کاربران پایگاه داده»: اعضای تیم (دعوت با موبایل) و کاربران پایگاه داده (برای GeoServer و برنامه‌ها).

	اعضای تیم	کاربران پایگاه داده
کیست؟	**آدم**‌ها با حساب خودشان در سامانه	**نرم‌افزار**ها: GeoServer، داشبورد، برنامه
ورود	با موبایل خودشان، در همین سامانه	نام کاربری و رمز PostgreSQL (فصل ۶)
کاربرد	دیدن و ویرایش در مرورگر، نقشه، SQL	اتصال مستقیم با کمترین دسترسی

اعضای تیم و نقش‌ها

همکاران را با شماره‌ی موبایل دعوت کنید و نقش بدهید:

- **مشاهده‌گر:** جدول‌ها و نقشه‌ها را می‌بیند و SQL فقط‌خواندنی اجرا می‌کند.
- **ویرایشگر:** به‌علاوه، داده وارد و ویرایش می‌کند و لایه منتشر می‌کند.
- **مالک** (شما): به‌علاوه‌ی کارهای حساس که فقط با خود شماست: رمز، تمدید، بکاپ و بازگردانی، حذف، کلید API.

تعداد اعضای تیم به پلن بستگی دارد. نقشه‌های خصوصی برای اعضای تیم باز است (فصل ۴۴).

کاربران پایگاه داده

برای GeoServer، داشبورد یا هر برنامه‌ای که مستقیم وصل می‌شود، **رمز اصلی خودتان را ندهید.** یک کاربر جداگانه بسازید: **فقط‌خواندنی** یا **ویرایشگر جدول‌های مشخص**. این کاربرها نه جدول تازه می‌سازند و نه جدولی را حذف می‌کنند و به پایگاه‌های دیگر دسترسی ندارند. اگر رمز یکی لو رفت، فقط همان را حذف یا رمزش را عوض می‌کنید.

۵۰-۲ کمترین دسترسی با SQL

پشت صحنه، این کاربرها **نقش**‌های PostgreSQL با مجوزهای محدودند. همین را در SQL با `GRANT` و `REVOKE` (زبان کنترل دسترسی، فصل ۱۰) می‌توان دقیق‌تر هم ساخت:

```sql
-- نقش فقط‌خواندنی برای یک داشبورد، فقط روی دو جدول
CREATE ROLE dashboard_reader LOGIN PASSWORD '…';
GRANT USAGE ON SCHEMA public TO dashboard_reader;
GRANT SELECT ON schools, neighborhoods TO dashboard_reader;

-- نمای عمومی بدون ستون‌های حساس، و دسترسی فقط به نما
CREATE VIEW public_health_centers AS SELECT gid, name, type, geom FROM
health_centers;
GRANT SELECT ON public_health_centers TO dashboard_reader;

-- پس گرفتن دسترسی
REVOKE SELECT ON schools FROM dashboard_reader;
```

سطح حرفه‌ای

امنیت سطر. PostgreSQL **امنیت سطح سطر** (Row-Level Security) هم دارد: هر نقش فقط سطرهایی را می‌بیند که شرطی را دارند؛ مثلاً هر کارشناس منطقه فقط قطعه‌های منطقه‌ی خودش را، که با یک شرط مکانی هم ممکن است: `CREATE POLICY own_district ON parcels USING (ST_Intersects(geom, (SELECT geom FROM districts WHERE manager = current_user)));` و سپس `ALTER TABLE parcels ENABLE ROW LEVEL SECURITY;`.

۵۰-۳ حریم خصوصی داده‌ی مکانی

داده‌ی مکانی ممکن است **شخصی** باشد، حتی بدون نام: نقطه‌ی خانه‌ی یک بیمار، مسیر روزانه‌ی یک راننده، محل یک شکایت. چند نقطه‌ی مکانی برای شناسایی یک فرد کافی است. اصول:

- **حداقل داده:** فقط آنچه لازم است جمع و نگه دارید؛ دقت مکانی بیش از نیاز هم «داده‌ی اضافه» است.
- **تجمیع پیش از انتشار:** به جای نقطه‌ی هر بیمار، تعداد در شش‌ضلعی یا محله (فصل‌های ۲۱ و ۲۶)؛ خانه‌هایی با تعداد خیلی کم را پنهان کنید.
- **ستون‌های حساس را منتشر نکنید:** در پنجره‌ی اطلاعات نقشه و لایه‌ی منتشرشده فقط ستون‌های لازم (فصل‌های ۴۲ و ۴۵)؛ برای همکاران، نما بدون ستون حساس.
- **قانون و مجوز:** شرایط داده‌ی منبع (ODbL، مجوزهای سازمانی) و قوانین حفاظت داده‌ی شخصی را رعایت کنید.

۵۰-۴ فهرست کنترل امنیت

۱. رمز پایگاه داده قوی و در مدیر رمز است؛ در کد، ایمیل یا پیام‌رسان نیست.

۲. هر نرم‌افزار کاربر جدا با کمترین دسترسی دارد؛ رمز اصلی فقط دست شماست.

۳. اعضای تیم با نقش درست؛ عضو رفته از تیم حذف شده است.

۴. کلید API فقط در سرور؛ کلید لورفته باطل شده؛ لایه‌ها پیش‌فرض خصوصی (فصل ۴۵).

۵ اتصال همیشه با SSL (فصل ۶).

۶ بکاپ مهم روی رایانه‌ی خودتان هم هست (فصل ۵۱).

خلاصه‌ی فصل

- اعضای تیم (آدم‌ها، با موبایل، نقش مشاهده‌گر/ویرایشگر) و کاربران پایگاه داده (نرم‌افزارها، با کمترین دسترسی) دو چیز جدایند.
- کارهای حساس (رمز، تمدید، بکاپ، حذف، کلید) فقط با مالک است.
- `GRANT` / `REVOKE` و نما برای کمترین دسترسی؛ امنیت سطح سطر برای «هر کس فقط داده‌ی خودش».
- داده‌ی مکانی می‌تواند شخصی باشد: حداقل داده، تجمیع پیش از انتشار، عدم انتشار ستون‌های حساس.

تمرین‌ها

۱. سطح ۱ یک همکار را با نقش مشاهده‌گر دعوت کنید و بررسی کنید که نمی‌تواند جدول را ویرایش کند.

۲. سطح ۲ یک کاربر فقط‌خواندنی برای GeoServer بسازید و با آن از QGIS وصل شوید؛ آزمایش کنید که نمی‌تواند سطری را حذف کند.

۳. سطح ۳ نمایی از مراکز درمانی بسازید که به جای نقطه‌ی دقیق، فقط تعداد مراکز هر محله را بدهد، و برای انتشار عمومی آماده کنید.

فصل

۵۱

بکاپ، بازگردانی، مصرف و کارایی

سطح: **متوسط** | زمان مطالعه: **۶۰ دقیقه** | پیش‌نیاز: **فصل‌های ۵ و ۲۰**

در این فصل یاد می‌گیرید

- بکاپ خودکار و دستی را بشناسید، دانلود و بازگردانی کنید.
- بکاپ را با pg_dump و pg_restore روی رایانه‌ی خودتان بگیرید و بازگردانید.
- صفحه‌ی «مصرف و کارایی» را بخوانید و جدول‌ها و پرسش‌های پرهزینه را پیدا کنید.
- سقف اتصال را با PgBouncer و pool مدیریت کنید.

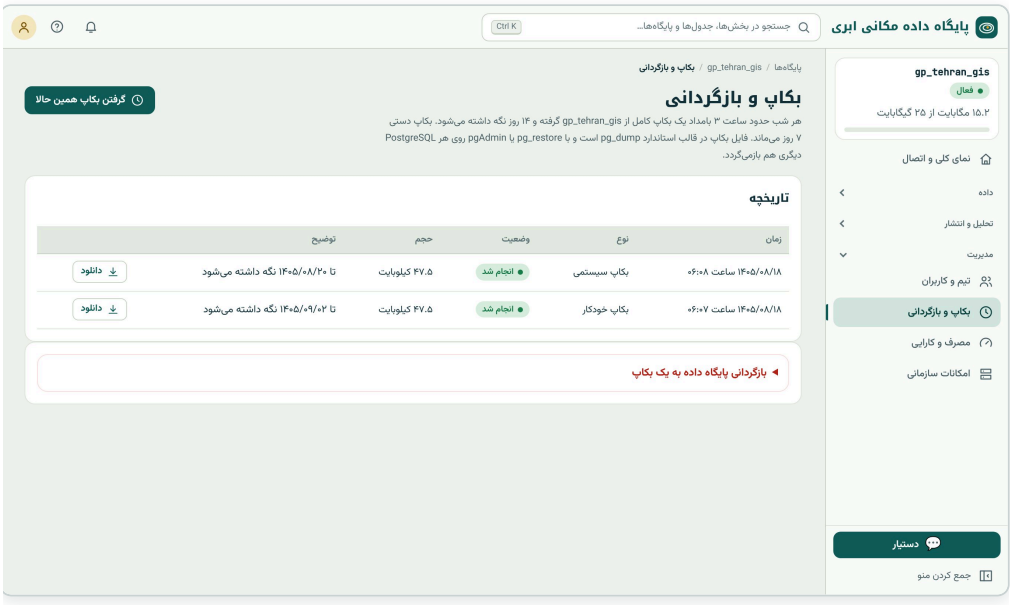
سرفصل‌ها

۵۱-۱ بکاپ

۵۱-۲ مصرف و کارایی

۵۱-۳ اتصال‌ها و PgBouncer

۵۱-۱ بکاپ



صفحه‌ی «بکاپ و بازگردانی»: زمان، نوع (خودکار، سیستمی، دستی)، وضعیت، حجم و تاریخ نگه‌داری هر بکاپ، با دکمه‌ی دانلود؛ «گرفتن بکاپ همین حالا» و بازگردانی.

- **خودکار:** هر شب، حدود ساعت ۳ بامداد به وقت تهران، برای هر پایگاه داده‌ی فعال؛ به اندازه‌ی روزهای پلن نگه داشته می‌شود.
- **دستی:** «گرفتن بکاپ همین حالا»، پیش از هر تغییر بزرگ (حذف جدول، `UPDATE` گسترده، ورود با جایگزینی).
- **ایمنی:** پیش از هر بازگردانی، سامانه خودکار یک بکاپ ایمنی از وضع فعلی می‌گیرد (سه روز نگه داشته می‌شود)؛ پس اگر بازگردانی اشتباه بود، راه برگشت هست.
- بکاپ‌ها به‌طور دوره‌ای **آزمون بازگردانی** می‌شوند تا مطمئن شویم سالم‌اند.

فایل بکاپ با قالب استاندارد `pg_dump` است؛ پس با **pg_restore** یا pgAdmin روی هر PostgreSQL دیگری هم بازمی‌گردد: داده‌ی شما زندانی سامانه نیست.

هشدار

بکاپ مهم را بیرون هم نگه دارید. پایگاه داده‌ی منقضی یا حذف‌شده بکاپ تازه نمی‌گیرد و پس از حذف، بکاپ‌هایش هم در دسترس نیستند (فصل ۵). قاعده‌ی ۳-۲-۱: سه نسخه، روی دو نوع رسانه، یکی بیرون از محل. بکاپ‌های مهم را دانلود و جای دیگری هم نگه دارید.

بکاپ با خط فرمان

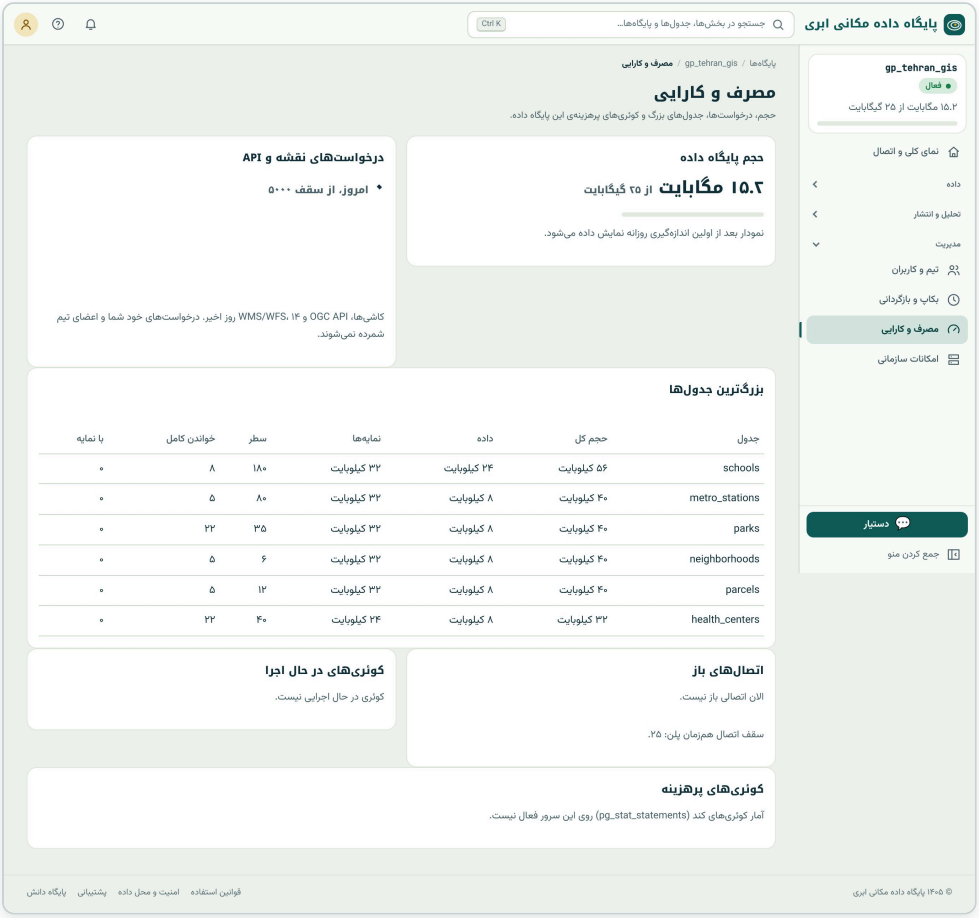
```
# بکاپ کامل در قالب فشرده‌ی سفارشی
pg_dump "host=… port=… dbname=gp_tehran_gis user=gp_tehran_gis_user
sslmode=require" -Fc -f tehran_gis.dump

# فقط یک جدول
pg_dump "…" -Fc -t schools -f schools.dump

# بازگردانی در پایگاه دیگر (مثلاً PostgreSQL محلی با PostGIS)
pg_restore -d "host=localhost dbname=test" --no-owner tehran_gis.dump
```

`--no-owner` لازم است چون نقش مالک در سرور دیگر وجود ندارد. نسخه‌ی `pg_dump` باید دست‌کم هم‌نسخه‌ی سرور باشد.

۵۱-۲ مصرف و کارایی



صفحه‌ی «مصرف و کارایی»: حجم پایگاه از سقف پلن، درخواست‌های نقشه و API، بزرگ‌ترین جدول‌ها (حجم کل، داده، شاخص‌ها، سطر، خواندن کامل)، اتصال‌های باز و کوئری‌های در حال اجرا و پرهزینه.

این صفحه پاسخ «چرا کند است؟» و «حجم کجا رفت؟» را می‌دهد:

- **بزرگ‌ترین جدول‌ها:** حجم داده و شاخص‌ها. ستون **«خواندن کامل»** (Seq Scan) مهم است: جدول بزرگی که زیاد کامل خوانده می‌شود، احتمالاً شاخص لازم دارد (فصل ۲۰).
- **اتصال‌های باز** در برابر سقف اتصال هم‌زمان پلن.
- **کوئری‌های در حال اجرا** (برای یافتن پرسش گیرکرده) و **کوئری‌های پرهزینه** (اگر افزونه‌ی `pg_stat_statements` روی سرور فعال باشد).

همین اطلاعات با SQL:

```sql
SELECT relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
       seq_scan, idx_scan, n_live_tup AS live_rows
FROM pg_stat_user_tables
WHERE schemaname = 'public' AND relname <> 'spatial_ref_sys'
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 6;
```

table_name	total_size	seq_scan	idx_scan	live_rows
schools	104 kB	2025	6511	180
parks	96 kB	503	307	35
parcels	80 kB	118	336	12
ndvi_parcels	80 kB	134	167	96
metro_stations	40 kB	910	15396	80
neighborhoods	40 kB	6000734	6749	6

```sql
-- اتصال‌ها و پرسش‌های فعال من
SELECT pid, state, now() - query_start AS running_for, left(query, 60)
AS query
FROM pg_stat_activity WHERE usename = current_user AND state <> 'idle';
-- متوقف کردن یک پرسش گیرکرده
SELECT pg_cancel_backend(12345);
```

برای کاهش حجم: جدول‌های موقت و نتیجه‌های قدیمی تحلیل را حذف کنید، ستون‌های بی‌استفاده را بردارید، هندسه‌ها را برای کاربردتان ساده کنید (فصل ۱۸) و پس از حذف انبوه `VACUUM` بزنید (فصل ۲۰).

۵۱-۳ اتصال‌ها و PgBouncer

هر پنجره‌ی QGIS، هر اسکریپت و هر کارگر برنامه‌ی وب یک اتصال می‌گیرد. برنامه‌ی وب پرترافیک به سرعت از **سقف اتصال همزمان** پلن عبور می‌کند و خطای `too many`

`connections` می‌دهد. **PgBouncer** اتصال‌ها را **تجمیع** (pool) می‌کند: صدها درخواست برنامه با چند اتصال واقعی به پایگاه سرویس می‌گیرند.

اتصال	برای
مستقیم	کارهای مدیریتی، QGIS، pgAdmin، pg_dump، ogr2ogr
PgBouncer (پورت جدا، در «اطلاعات اتصال»)	وب‌سایت و برنامه با ترافیک زیاد

PgBouncer در حالت **transaction** کار می‌کند: دستورهای وابسته به نشست (`SET` بیرون از تراکنش، `LISTEN`، جدول موقت بین تراکنش‌ها) کار نمی‌کنند و prepared statement سمت سرور را باید در درایور خاموش کنید. در برنامه، pool کوچک خود برنامه (مثلاً `max: 3` در Node.js، فصل ۶) را هم فراموش نکنید.

خلاصه‌ی فصل

- بکاپ خودکار شبانه، دستی پیش از تغییر بزرگ، و بکاپ ایمنی پیش از بازگردانی؛ قالب pg_dump قابل بازگردانی در هر PostgreSQL.
- بکاپ مهم را دانلود و بیرون هم نگه دارید (۳-۲-۱).
- «مصرف و کارایی»: بزرگ‌ترین جدول‌ها، خواندن کامل (نیاز به شاخص)، اتصال‌ها و پرسش‌های پرهزینه؛ معادل SQL با `pg_stat_*` .
- PgBouncer برای برنامه‌های پرترافیک؛ مستقیم برای QGIS و کارهای مدیریتی.

تمرین‌ها

۱. سطح ۱ یک بکاپ دستی بگیرید و دانلود کنید.

۲. سطح ۲ با pg_dump فقط جدول `schools` را بکاپ و در یک PostgreSQL محلی بازگردانید.

۳. سطح ۲ کدام جدول نمونه بیشترین «خواندن کامل» را دارد؟ آیا شاخص لازم دارد؟ (راهنمایی: اندازه‌ی جدول، فصل ۲۰)

فصل

۵۲

امکانات سازمانی

سطح: **پیشرفته** ●●●

زمان مطالعه: **۴۵ دقیقه**

پیش‌نیاز: **فصل‌های ۴۵ و ۵۱**

در این فصل یاد می‌گیرید

- GeoServer اختصاصی و کاربرد آن را بشناسید.
- بکاپ ساعتی و کپی آزمایشی پایگاه داده را برای کار امن به کار ببرید.
- رسترهای خودتان را با سرویس رستر منتشر کنید.

سرفصل‌ها

۵۲-۱ امکانات سازمانی در یک نگاه

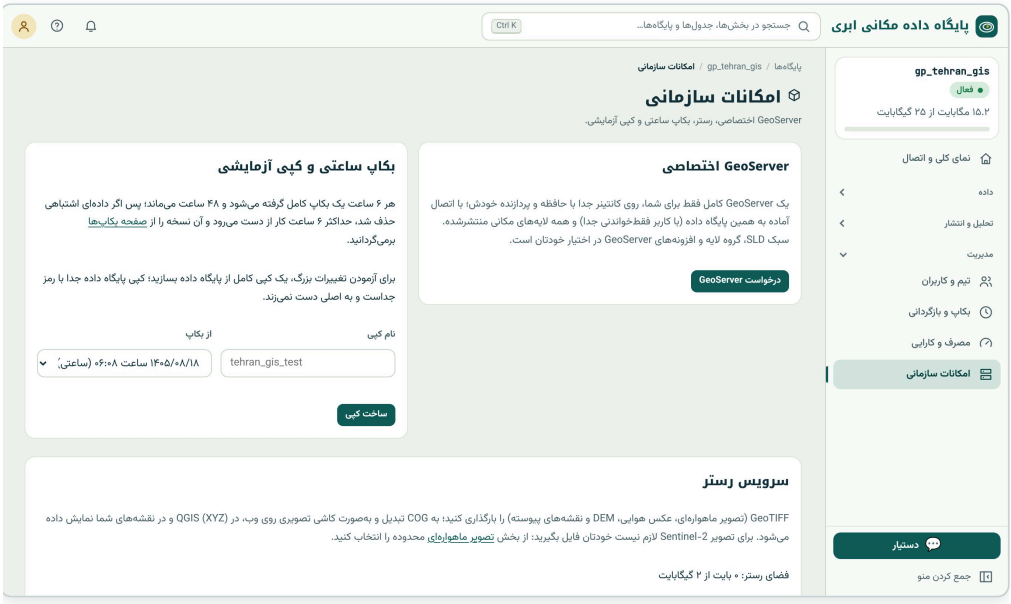
۵۲-۲ اختصاصی GeoServer

۵۲-۳ بکاپ ساعتی و کپی آزمایشی

۵۲-۴ سرویس رستر

۵۲-۱ امکانات سازمانی در یک نگاه

پلن‌های سازمانی، منوی «امکانات سازمانی» را دارند:



صفحه‌ی «امکانات سازمانی»: GeoServer اختصاصی، بکاپ ساعتی و کپی آزمایشی، و سرویس رستر.

۵۲-۲ اختصاصی GeoServer

یک **GeoServer** کامل فقط برای شما، روی کانتینر جدا با حافظه و پردازنده‌ی خودش، با اتصال آماده به همین پایگاه داده (با کاربر فقط‌خواندنی جدا؛ فصل ۵۰) و همه‌ی لایه‌های مکانی منتشرشده. سبک **SLD**، گروه لایه، کش کاشی و همه‌ی افزونه‌های GeoServer در اختیار شماست. برای سازمان‌هایی که زیرساخت GIS خود را بر GeoServer بنا کرده‌اند یا WMS/WFS با پیکربندی پیشرفته لازم دارند. برای نیازهای ساده‌تر، WMS/WFS با QGIS Server (فصل ۴۵) کافی است.

۵۲-۳ بکاپ ساعتی و کپی آزمایشی

- **بکاپ هر ۶ ساعت** (نگه‌داری ۴۸ ساعت) کنار بکاپ شبانه؛ اگر داده‌ای اشتباهی حذف شد، حداکثر ۶ ساعت کار از دست می‌رود.
- **کپی آزمایشی:** یک کپی کامل از یک بکاپ، به‌عنوان **پایگاه داده‌ی جداگانه** با رمز خودش. برای آزمودن تغییرهای بزرگ (مهاجرت ساختار، اسکریپت پاک‌سازی، به‌روزرسانی انبوه) روی کپی، بدون هیچ خطری برای پایگاه اصلی. پس از اطمینان، همان اسکریپت را روی اصلی اجرا کنید.

در دنیای واقعی

مهاجرت ساختار بی‌خطر. یک شهرداری می‌خواست ستون کاربری ۸۰۰ هزار قطعه را از متن آزاد به کد استاندارد تبدیل کند. اسکریپت `UPDATE` روی کپی آزمایشی سه بار اجرا و اصلاح شد تا همه‌ی حالت‌ها پوشش داده شوند؛ سپس روی پایگاه اصلی، داخل یک تراکنش (فصل ۱۴)، در چند دقیقه اجرا شد.

۵۲-۴ سرویس رستر

GeoTIFF خودتان (عکس هوایی، ارتوفتو پهپاد، DEM، نقشه‌ی پیوسته) را بارگذاری کنید؛ به **COG** تبدیل و به‌صورت **کاشی تصویری روی وب** (XYZ) منتشر می‌شود تا در QGIS و نقشه‌های شما نمایش داده شود. تصویرهای ماهواره‌ای بخش «تصویر ماهواره‌ای» (فصل ۳۴) هم در همین فضای رستر قرار می‌گیرند. فضای رستر جدا از حجم جدول‌های پایگاه داده است. رستری که این‌جا دارید، منبع «آمار هر قطعه» هم می‌تواند باشد (فصل ۳۶، منبع «رستری که از قبل دارید»).

خلاصه‌ی فصل

- GeoServer اختصاصی: کانتینر جدا، اتصال آماده، SLD و همه‌ی امکانات GeoServer.
- بکاپ هر ۶ ساعت و کپی آزمایشی برای آزمودن تغییرهای بزرگ بدون خطر.
- سرویس رستر: COG ← GeoTIFF ← کاشی XYZ؛ منبع آمار هر قطعه.

تمرین‌ها

۱. سطح ۲ یک کپی آزمایشی بسازید، روی آن ستونی را حذف کنید و ببینید پایگاه اصلی دست‌نخورده است.

۲. سطح ۳ ارتوفتوی پهپاد یک محدوده را در سرویس رستر بارگذاری و در نقشه‌ی چندلایه زیر لایه‌های برداری نمایش دهید.

فصل

۵۳

برنامه‌نویسی با پایگاه داده‌ی مکانی؛ تابع، تریگر، Python و خودکارسازی

سطح: **پیشرفته** ●●●

زمان مطالعه: **۱۰۰ دقیقه**

پیش‌نیاز: **فصل‌های ۶، ۱۴ و ۱۸**

در این فصل یاد می‌گیرید

- منطق تکراری را در تابع SQL ذخیره کنید.
- با تریگر، ستون‌های محاسباتی و قواعد کیفیت را خودکار کنید.
- با Python (psycopg و GeoPandas) داده بخوانید، تحلیل کنید و بنویسید.
- کارهای دوره‌ای (ورود، تحلیل، گزارش) را خودکار کنید.

سرفصل‌ها

۵۳-۱ منطق در پایگاه داده

۵۳-۲ تابع SQL

۵۳-۳ تریگر: کار خودکار پس از هر تغییر

۵۳-۴ Python

۵۳-۵ خودکارسازی

۵۳-۱ منطق در پایگاه داده

تا اینجا پرسش‌ها را هر بار نوشتیم. اما منطقی که بارها تکرار می‌شود (محاسبه‌ی ناحیه‌ی UTM، به‌روز کردن مساحت پس از هر ویرایش) بهتر است **یک بار** در خود پایگاه داده ذخیره شود، تا برای همه‌ی نرم‌افزارها (QGIS، مرورگر سامانه، برنامه‌ها) یکسان اجرا شود.

۵۳-۲ تابع SQL

```sql
CREATE FUNCTION utm_srid(g geometry) RETURNS integer
LANGUAGE sql IMMUTABLE AS $$
  SELECT 32600 + floor((ST_X(ST_Centroid(ST_Transform(g, 4326))) + 180)
/ 6)::int + 1
$$;

SELECT name, utm_srid(geom) AS srid,
       round(ST_Area(ST_Transform(geom, utm_srid(geom)))) AS
area_m2_in_own_zone
FROM neighborhoods ORDER BY gid LIMIT 3;
```

name	srid	area_m2_in_own_zone
ونک	32639	16042536.0
تجریش	32639	16033710.0
نارمک	32639	16047824.0

تابع `utm_srid` همان فرمول فصل ۳ است و حالا هر جا لازم است صدا زده می‌شود؛ حتی برای داده‌ای که در چند ناحیه پخش است، هر عارضه در ناحیه‌ی خودش اندازه گرفته می‌شود. کلیدواژه‌ی `IMMUTABLE` به PostgreSQL می‌گوید نتیجه فقط به ورودی بستگی دارد؛ پس می‌توان روی آن شاخص ساخت و برنامه‌ریز بهتر بهینه می‌کند. توابع پیچیده‌تر با **PL/pgSQL** (زبان روالی PostgreSQL با متغیر، شرط و حلقه) نوشته می‌شوند.

۵۳-۳ تریگر: کار خودکار پس از هر تغییر

تریگر (trigger) تابعی است که پیش یا پس از `INSERT`، `UPDATE` یا `DELETE` روی هر سطر خودکار اجرا می‌شود. مثال کلاسیک GIS: ستون مساحت که با هر ویرایش هندسه، از هر نرم‌افزاری، به‌روز بماند:

```sql
ALTER TABLE parks ADD COLUMN area_m2 numeric(12,1), ADD COLUMN
updated_at timestamptz;

CREATE FUNCTION parks_before_write() RETURNS trigger LANGUAGE plpgsql
AS $$
BEGIN
  IF NOT ST_IsValid(NEW.geom) THEN
    NEW.geom := ST_CollectionExtract(ST_MakeValid(NEW.geom), 3);   --
قاعده‌ی کیفیت (فصل ۲۲)
  END IF;
  NEW.area_m2    := ST_Area(NEW.geom::geography);
  NEW.updated_at := now();
  RETURN NEW;
END $$;

CREATE TRIGGER parks_area BEFORE INSERT OR UPDATE OF geom ON parks
FOR EACH ROW EXECUTE FUNCTION parks_before_write();

UPDATE parks SET geom = ST_Buffer(geom::geography, 20)::geometry WHERE
park_id = 1;   -- ویرایش یک هندسه
SELECT park_id, name, area_m2, updated_at IS NOT NULL AS stamped FROM
parks WHERE park_id IN (1, 2) ORDER BY park_id;
```

park_id	name	area_m2	stamped
1	پارک 1	163592.9	True
2	پارک 2	NULL	False

پارک ۱ که هندسه‌اش عوض شد، مساحت و زمان به‌روزرسانی گرفت؛ پارک ۲ دست‌نخورده ماند. حالا مهم نیست کسی از QGIS، از ویرایشگر مرورگر یا از اسکریپت هندسه را عوض کند؛ مساحت همیشه درست است. کاربردهای دیگر تریگر: نوشتن تاریخچه‌ی تغییرها در جدول

جداگانه (audit)، نسبت دادن خودکار محله به هر نقطه‌ی تازه (فصل ۱۴)، جلوگیری از حذف رکوردهای قفل‌شده.

> **هشدار**
>
> **تریگر را ساده نگه دارید.** تریگر روی هر سطر اجرا می‌شود؛ تریگر سنگین (مثلاً پیوند مکانی با جدولی بزرگ) ورود انبوه را بسیار کند می‌کند. برای عملیات سنگین، به جای تریگر، کار دوره‌ای (پایین) یا نمای مادی‌شده (فصل ۱۴) را در نظر بگیرید. ستون محاسباتی ساده را با `GENERATED ALWAYS AS (…) STORED` (فصل ۲۰) هم می‌توان ساخت.

۵۳-۴ Python

psycopg درایور اصلی PostgreSQL در پایتون است و **GeoPandas** جدول‌های مکانی را به DataFrame مکانی تبدیل می‌کند:

```
import os, psycopg
import geopandas as gpd
from sqlalchemy import create_engine

DSN = os.environ["DATABASE_URL"]            # رمز در متغیر محیطی، نه در کد

# ۱) پرسش پارامتری با psycopg
with psycopg.connect(DSN) as conn:
    rows = conn.execute(
        "SELECT name, students FROM schools WHERE level = %s AND
students > %s ORDER BY students DESC LIMIT 5",
        ("600 ,"ابتدایی")).fetchall()

# ۲) خواندن، تحلیل و نوشتن با GeoPandas
engine = create_engine(DSN.replace("postgresql://",
"postgresql+psycopg://"))
schools = gpd.read_postgis("SELECT * FROM schools", engine,
geom_col="geom")
hoods = gpd.read_postgis("SELECT * FROM neighborhoods", engine,
geom_col="geom")
joined = gpd.sjoin(schools, hoods[["name", "geom"]],
predicate="within")
summary = joined.groupby("name_right").students.agg(["count", "sum"])
summary.to_csv("schools_by_neighborhood.csv")
joined.to_postgis("schools_with_hood", engine, if_exists="replace",
index=False)
```

قاعده‌ی کلی: **کار مکانی سنگین را به پایگاه داده بسپارید** (SQL)، و پایتون را برای چیزهایی که SQL ندارد: کتابخانه‌های آماری و یادگیری ماشین (scikit-learn، PySAL برای آمار فضایی فصل ۲۱)، نمودارهای پیچیده، فراخوانی سرویس‌های بیرونی، و هماهنگی گام‌ها. داده را تا حد ممکن با `WHERE` و تجمیع در SQL کوچک کنید و بعد به پایتون بیاورید.

۵۳-۵ خودکارسازی

کارهای تکراری را زمان‌بندی کنید (با cron روی سرور خودتان، GitHub Actions یا هر زمان‌بند دیگر):

کار	ابزار
ورود شبانه‌ی داده‌ی تازه	`ogr2ogr -append` (فصل ۷) یا REST API (فصل ۴۶)
تحلیل دوره‌ای	اسکریپت `psql -f analysis.sql` با `CREATE TABLE AS` یا `REFRESH MATERIALIZED VIEW`
گزارش هفتگی	پایتون: پرسش، نمودار، PDF یا ایمیل
همگام‌سازی برداشت میدانی	API همگام‌سازی پروژه (فصل ۴۸)
پایش ماهواره‌ای	پایش خودکار خود سامانه (فصل ۳۷)؛ نیازی به اسکریپت نیست

```
# نمونه‌ی cron: هر شب ساعت ۲، ورود گزارش‌های تازه و به‌روزرسانی نمای آماری
0 2 * * *  ogr2ogr -append -f PostgreSQL PG:"$PGDSN"
/data/reports_today.gpkg -nln reports && \
           psql "$PGDSN" -c "REFRESH MATERIALIZED VIEW
mv_report_density;"
```

برای اسکریپت‌ها هم کاربر پایگاه داده‌ی جداگانه با کمترین دسترسی بسازید (فصل ۵۰) و از PgBouncer برای اتصال‌های پرتعداد استفاده کنید (فصل ۵۱).

خلاصه‌ی فصل

- تابع SQL منطق تکراری را یک‌جا نگه می‌دارد؛ `IMMUTABLE` برای شاخص و بهینه‌سازی.
- تریگر `BEFORE INSERT OR UPDATE` ستون‌های محاسباتی و قواعد کیفیت را برای همه‌ی نرم‌افزارها خودکار می‌کند؛ ساده نگهش دارید.
- Python: psycopg با پارامتر، GeoPandas با `read_postgis` و `to_postgis`؛ کار سنگین مکانی در SQL.
- خودکارسازی با cron یا زمان‌بند: REFRESH ، `psql -f`، ogr2ogr، گزارش پایتون.

تمرین‌ها

۱. سطح ۲ تابعی بنویسید که هندسه بگیرد و مساحت را به هکتار با دو رقم اعشار برگرداند؛ آن را روی پارک‌ها به کار ببرید.

۲. سطح ۳ تریگری بنویسید که با درج هر مدرسه‌ی تازه، نام محله‌اش را در ستون `neighborhood` بنویسد (فصل ۱۴).

۳. سطح ۳ جدول `parks_audit` و تریگری بسازید که هر تغییر هندسه را با زمان و کاربر (`current_user`) ثبت کند.

۴. سطح ۳ با GeoPandas تعداد مدرسه‌های هر محله را حساب کنید و با نتیجه‌ی SQL فصل ۱۳ مقایسه کنید.

فصل

۵۴

پایگاه داده مکانی ابری در کلاس

سطح: **مبتدی** زمان مطالعه: **۴۵ دقیقه** پیش‌نیاز: **ندارد**

در این فصل یاد می‌گیرید

- چرخه‌ی کلاس (قرارداد، ساخت کلاس، ورود دانشجو، تمرین، تحویل، نمره، پایان ترم) را بشناسید.
- تمرینی طراحی کنید که با پایگاه داده‌ی شخصی هر دانشجو قابل انجام و بررسی باشد.
- این کتاب را با بخش «کلاس» به یک درس کامل تبدیل کنید.

سرفصل‌ها

۵۴-۱ چرا کلاس

۵۴-۲ چرخه‌ی کلاس

۵۴-۳ طراحی تمرین خوب

۵۴-۴ این کتاب به‌عنوان درس

۵۴-۱ چرا کلاس

تدریس GIS معمولاً با چند مشکل همیشگی روبه‌روست: نصب نرم‌افزار روی رایانه‌های سایت، داده‌ای که هر دانشجو جدا کپی می‌کند، تحویل تمرین با فایل‌های پراکنده و نمره‌دهی‌ای که استاد باید فایل هر دانشجو را جدا باز کند. بخش **«کلاس»** این چرخه را ساده می‌کند: هر دانشجو **پایگاه داده‌ی PostGIS شخصی** خودش را دارد، تمرین‌ها و تحویل‌ها یک‌جا هستند و استاد داده‌ی هر دانشجو را مستقیم می‌بیند.

۵۴-۲ چرخه‌ی کلاس

۱. **قرارداد:** دانشگاه یا آموزشگاه برای تعداد دانشجو و بازه‌ی ترم قرارداد می‌بندد و شماره‌ی استاد در آن ثبت می‌شود.

۲. **ساخت کلاس:** استاد با حساب خودش عنوان، ظرفیت (در حد صندلی‌های آزاد قرارداد) و **داده‌ی آغازین** را تعیین می‌کند: داده‌ی نمونه (همان داده‌ی این کتاب) یا داده‌ی OSM یک شهر (فصل ۳۰). سامانه یک **کد ورود شش‌حرفی** می‌دهد.

۳. **ورود دانشجو:** دانشجو حساب می‌سازد (فصل ۵)، کد کلاس را وارد و نام و شماره‌ی دانشجویی را ثبت می‌کند. پایگاه داده‌ی شخصی‌اش با داده‌ی آغازین ساخته می‌شود و رمزش یک بار نمایش داده می‌شود.

۴. **تمرین:** استاد «تمرین جدید» را با صورت تمرین، مهلت، سقف نمره و در صورت نیاز **نقشه‌ی همراه** می‌سازد.

۵. **انجام و تحویل:** دانشجو در پایگاه داده‌ی خودش کار می‌کند (SQL، ابزارها، نقشه) و در صفحه‌ی تمرین پاسخ (متن، نام جدول، نقشه) را تحویل می‌دهد؛ تا پیش از نمره می‌تواند دوباره تحویل دهد.

۶. **نمره:** در «بررسی تحویل‌ها» استاد وضعیت همه را می‌بیند، پاسخ، نقشه و **پایگاه داده‌ی هر دانشجو** را بررسی، نمره و بازخورد می‌دهد یا برای اصلاح برمی‌گرداند. نمره‌ها خروجی CSV دارند.

۷ **پایان ترم:** پایگاه داده‌های کلاس بایگانی و غیرفعال می‌شوند و از هر کدام یک بکاپ برای ۱۸۰ روز نگه داشته می‌شود. به دانشجوها بگویید پیش از پایان ترم کارشان را بکاپ بگیرند (فصل ۵۱).

اشتراک نقشه با کلاس

استاد می‌تواند یکی از نقشه‌های چندلایه‌ی خودش را با کلاس به اشتراک بگذارد (با توضیح یا به‌عنوان تمرین). دانشجوها آن را حتی اگر خصوصی باشد می‌بینند، ارائه‌اش را اجرا می‌کنند و با **«کپی در پایگاه داده‌ی من»** همان نقشه را روی جدول‌های هم‌نام پایگاه خودشان می‌سازند. لایه‌ها و نمودارهایی که جدول یا ستون هم‌نام ندارند کنار گذاشته و اعلام می‌شوند. اگر همه‌ی دانشجوها با داده‌ی نمونه شروع کرده باشند، کپی کامل است.

۵۴-۳ طراحی تمرین خوب

تمرینی که **خروجی‌اش در پایگاه داده** است، بررسی‌اش آسان و تقلب در آن سخت است:

خوب	کمتر خوب
«جدول `hw3_buffer` را با حریم ۵۰۰ متری ایستگاه‌ها بسازید و تعداد مدرسه‌های داخلش را بنویسید.»	«درباره‌ی حریم توضیح دهید.»
خروجی با نام مشخص که استاد با یک پرسش می‌سنجد	خروجی فقط به صورت تصویر
پرسشی که پاسخ عددی قابل بررسی دارد	پرسش کاملاً باز
گام آخر تفسیری («این عدد برای برنامه‌ریز چه معنایی دارد؟»)	فقط اجرای دستور

استاد می‌تواند با دسترسی به پایگاه دانشجو، درستی جدول را با SQL بسنجد: تعداد سطر، نوع هندسه، SRID، مجموع مساحت.

۵۴-۴ این کتاب به‌عنوان درس

پیشنهاد سرفصل ترمی و کارگاهی در «راهنمای استفاده از کتاب» آمده است. تمرین‌های پایان هر فصل سه سطح دارند و همه روی داده‌ی نمونه‌اند؛ یعنی اگر کلاس با داده‌ی نمونه شروع شود، هر تمرین کتاب بدون هیچ آماده‌سازی قابل واگذاری است. کادرهای «برای مدرس» در هر فصل زمان پیشنهادی و فعالیت کلاسی را می‌گویند. پروژه‌های فصل ۵۵ برای پروژه‌ی پایانی ترم مناسب‌اند.

خلاصه‌ی فصل

- قرارداد، ساخت کلاس با داده‌ی آغازین و کد شش‌حرفی، ورود دانشجو با پایگاه شخصی، تمرین، تحویل، نمره (با دیدن پایگاه دانشجو)، بایگانی پایان ترم (بکاپ ۱۸۰ روزه).
- اشتراک نقشه با کلاس و «کپی در پایگاه داده‌ی من» روی جدول‌های هم‌نام.
- تمرین خوب خروجی با نام مشخص در پایگاه داده دارد که با SQL سنجیده می‌شود.
- داده‌ی نمونه‌ی کتاب همان داده‌ی آغازین کلاس است؛ تمرین‌های کتاب مستقیم قابل واگذاری‌اند.

فصل

۵۵

پروژه‌های جامع

سطح: **پیشرفته** ●●●

زمان مطالعه: **هر پروژه چند جلسه**

پیش‌نیاز: **بخش‌های مربوط**

در این فصل یاد می‌گیرید

- یک مسئله‌ی واقعی را از پرسش تا نقشه‌ی منتشرشده با همه‌ی ابزارهای کتاب حل کنید.
- جریان کار GIS (فصل ۱) را در عمل اجرا کنید.
- نتیجه را برای تصمیم‌گیر قابل فهم گزارش کنید.

سرفصل‌ها

۵۵-۱ ساختار هر پروژه

۵۵-۲ پروژه‌ی ۱: دسترسی دانش‌آموزان به مدرسه

۵۵-۳ پروژه‌ی ۲: پایش خشکسالی کشاورزی یک دشت

۵۵-۴ پروژه‌ی ۳: تغییر کاربری در حریم رودخانه

۵۵-۵ پروژه‌ی ۴: پوشش اورژانس و آتش‌نشانی

۵۵-۶ پروژه‌ی ۵: کانون‌های تصادف و پیشنهاد اصلاح

۵۵-۷ پروژه‌ی ۶: برداشت و حجم‌سنجی یک محوطه‌ی عمرانی

۵۵-۸ معیار ارزیابی پروژه (برای مدرس)

۵۵-۹ پایان سخن

۵۵-۱ ساختار هر پروژه

شش پروژه‌ی این فصل هر کدام یک مسئله‌ی واقعی را با همان جریان کار فصل ۱ حل می‌کنند: **پرسش** ← **داده** ← **آماده‌سازی** ← **تحلیل** ← **نمایش و انتشار** ← **گزارش.** برای هر گام، فصل‌های مرتبط آمده است. پروژه‌ها را می‌توان با داده‌ی نمونه، داده‌ی OSM شهر خودتان (فصل ۳۰) یا داده‌ی سازمان خودتان انجام داد. هر پروژه با یک **گزارش دوصفحه‌ای** برای مدیر (نه برای متخصص GIS) و یک **نقشه‌ی منتشرشده** پایان می‌یابد.

۵۵-۲ پروژه‌ی ۱: دسترسی دانش‌آموزان به مدرسه

پرسش: کدام محله‌ها کمبود مدرسه دارند و مدرسه‌ی تازه کجا ساخته شود؟

1. **داده:** مدرسه‌ها، محله‌ها با جمعیت، شبکه‌ی معابر (فصل‌های ۷ و ۳۰).
2. **آماده‌سازی:** اصلاح همپوشانی محله‌ها (فصل‌های ۱۷ و ۱۸)، سلامت داده (فصل ۷).
3. **تحلیل:** نسبت جمعیت به مدرسه برای هر محله (فصل ۱۳)؛ حوزه‌ی دسترسی پیاده‌ی ۱ کیلومتری هر مدرسه روی شبکه (فصل‌های ۲۴ و ۲۸)؛ بخش‌هایی از شهر که بیرون از همه‌ی حوزه‌هاست (فصل ۲۷)؛ کانون‌های جمعیت بی‌پوشش (فصل ۲۱).
4. **مکان‌یابی:** سه زمین کاندید را با «پوشش چند مرکز» مقایسه کنید: هر کدام چند نفر تازه را زیر پوشش می‌برد؟
5. **نمایش:** کروپلت نسبت جمعیت به مدرسه، لایه‌ی بی‌پوشش، سه کاندید با نمودار مقایسه (فصل‌های ۴۱ تا ۴۳)؛ ارائه‌ی پنج‌اسلایدی (فصل ۴۴).

۵۵-۳ پروژه‌ی ۲: پایش خشکسالی کشاورزی یک دشت

پرسش: کدام مزرعه‌ها در این فصل تنش آبی دارند و وضع امسال نسبت به پارسال چیست؟

1. **داده:** مرز قطعه‌ها (برداشت میدانی یا رقومی‌سازی روی تصویر؛ فصل‌های ۹ و ۴۸)، تصویر Sentinel-2.

2. **تحلیل:** آمار هر قطعه برای NDVI و NDMI در دو فصل رشد (فصل ۳۶)؛ منحنی فنولوژی و تاریخ اوج (فصل ۳۷)؛ z هر قطعه نسبت به هم‌کشت‌ها (فصل ۳۶)؛ مقایسه‌ی اوج امسال و پارسال.
3. **پایش:** پایش هفتگی NDMI با هشدار ۲۰ درصد (فصل ۳۷).
4. **نمایش:** نقشه‌ی `_latest` با طیف مناسب، نمودار سری زمانی، مقایسه‌ی کشویی دو سال (فصل‌های ۴۲ تا ۴۴).
5. **گزارش:** فهرست قطعه‌های اولویت‌دار برای بازدید.

۵۵-۴ پروژه‌ی ۳: تغییر کاربری در حریم رودخانه

پرسش: در ده سال گذشته چقدر از حریم رودخانه ساخت‌وساز شده است؟

1. **داده:** خط رودخانه (OSM، فصل ۳۰)، تصویرهای Landsat و Sentinel-2 در دو تاریخ هم‌فصل.
2. **تحلیل:** حریم ۱۵۰ متری (فصل ۲۵)؛ طبقه‌بندی پوشش زمین هر تاریخ در QGIS (فصل ۳۹) با ارزیابی صحت؛ ماتریس تغییر داخل حریم (فصل ۲۷)؛ کمکی: NDBI دو تاریخ (فصل ۳۵).
3. **نمایش:** نمودار Sankey تغییرها، نقشه‌ی «باغ و زراعت ← شهر» (فصل ۴۳).

۵۵-۵ پروژه‌ی ۴: پوشش اورژانس و آتش‌نشانی

پرسش: کدام بخش‌های شهر بیش از ۸ دقیقه با نزدیک‌ترین ایستگاه فاصله دارند؟

1. **داده:** ایستگاه‌ها (OSM یا سازمانی)، شبکه‌ی معابر با سرعت.
2. **تحلیل:** SQL مسیریابی با هزینه‌ی زمانی (فصل ۲۴)؛ حوزه‌ی دسترسی ۴ و ۸ دقیقه‌ای هر ایستگاه؛ نقطه‌های کور؛ جمعیت (یا ساختمان‌ها) داخل نقطه‌های کور (فصل ۲۶).
3. **سناریو:** افزودن یک ایستگاه در سه جای کاندید و مقایسه‌ی کاهش نقطه‌ی کور.
4. **نمایش:** حلقه‌های زمانی، نقطه‌های کور، جدول سناریوها.

۵۵-۶ پروژه‌ی ۵: کانون‌های تصادف و پیشنهاد اصلاح

پرسش: کانون‌های معنادار تصادف کجاست و چه ویژگی‌ای دارند؟

1. **داده:** نقطه‌ی تصادف‌ها (CSV با مختصات، فصل ۷)، معابر OSM.
2. **آماده‌سازی:** چسباندن هر تصادف به نزدیک‌ترین معبر (فصل ۱۹)؛ یکدست کردن تاریخ و نوع.
3. **تحلیل:** شبکه‌ی شش‌ضلعی و Gi* (فصل ۲۱)؛ DBSCAN برای کانون‌های نقطه‌ای؛ ویژگی‌های معابر داخل کانون‌ها (رده، سرعت مجاز؛ فصل ۳۰)؛ روند زمانی با توابع پنجره‌ای (فصل ۱۲).
4. **نمایش:** نقشه‌ی کانون داغ با طیف واگرا، نمودار ساعت و روز، پیشنهادهای اصلاح.

۵۵-۷ پروژه‌ی ۶: برداشت و حجم‌سنجی یک محوطه‌ی عمرانی

پرسش: برای تسطیح محوطه تا تراز طراحی، حجم خاک‌برداری و خاک‌ریزی چقدر است و تراز متعادل کجاست؟

1. **داده:** برداشت شبکه‌ای نقطه‌ها با RTK (فصل‌های ۴۷ و ۴۸)؛ کمکی: DEM برای محدوده‌ی بزرگ‌تر (فصل ۳۸).
2. **تحلیل:** COGO و کنترل نقطه‌ها (فصل ۴۹)؛ TIN، منحنی میزان، حجم نسبت به تراز طراحی و تراز متعادل (فصل ۴۹)؛ پروفیل طولی محور اصلی (فصل ۳۸).
3. **خروجی:** DXF منحنی‌ها برای طراح، گزارش حجم، نقشه‌ی منتشرشده برای کارفرما.

۵۵-۸ معیار ارزیابی پروژه (برای مدرس)

معیار	وزن پیشنهادی
تبدیل مسئله به پرسش مکانی دقیق	۱۵٪
کیفیت و آماده‌سازی داده (SRID، اعتبار، فراداده)	۱۵٪
درستی تحلیل (روش مناسب، SQL یا ابزار درست، کنترل نتیجه)	۳۰٪
کارتوگرافی و نمودار (چک‌لیست فصل ۴۱)	۲۰٪
گزارش برای تصمیم‌گیر (روشن، کوتاه، با محدودیت‌ها)	۲۰٪

۵۵-۹ پایان سخن

اگر به این‌جا رسیده‌اید، مسیری را پیموده‌اید که از «مکان در همه‌چیز هست» شروع شد و به SQL مکانی حرفه‌ای، تحلیل تصویر ماهواره‌ای، برداشت میدانی و انتشار نقشه‌ی وب رسید. ابزارها عوض می‌شوند، اما سه چیزی که این کتاب کوشید بسازد می‌ماند: **فهم علمی** پدیده‌ها، **زبان SQL** که همه‌جا کار می‌کند، و **عادت کنترل نتیجه**: هر عدد را با یک پرسش ساده‌ی دیگر بسنجید، هر نقشه را با چک‌لیست، و هر هشدار ماهواره‌ای را با بازدید میدانی. موفق باشید؛ و اگر این کتاب را درس می‌دهید، تجربه‌ی کلاس‌تان بهترین منبع ویرایش بعدی است.

پیوست الف ـ واژه‌نامه

واژه‌های کلیدی کتاب به ترتیب الفبای فارسی، با معادل انگلیسی و فصلی که مفصل‌تر در آن آمده است.

Synthetic Aperture Radar SAR
رادار روزنه‌ی مصنوعی (فصل ۴۰).

Spatial Reference ID SRID
شناسه‌ی سیستم مختصات؛ کد EPSG (فصل ۳).

Universal Transverse Mercator UTM
سیستم تصویر ناحیه‌ای متری؛ ایران ناحیه‌های ۳۸ تا ۴۱ (فصل ۳).

Well-Known Text / Binary WKT / WKB
قالب متنی و دودویی استاندارد هندسه (فصل ۱۵).

آمار ناحیه‌ای Zonal statistics
آمار پیکسل‌های رستر داخل هر چندضلعی (فصل‌های ۲۳ و ۳۶).

ارجاع خطی Linear referencing
بیان مکان با فاصله در امتداد خط (فصل ۱۸).

اعتبار هندسه Geometry validity
رعایت قواعد OGC در هندسه؛ شرط تحلیل درست (فصل ۲۲).

افزونه Extension
بسته‌ی گسترش PostgreSQL مثل PostGIS و pgRouting (فصل ۴).

امضای طیفی Spectral signature
نمودار بازتاب یک ماده در طول موج‌ها (فصل ۳۱).

بازتاب طیفی Reflectance
کسر انرژی بازتاب‌شده در یک طول موج؛ ۰ تا ۱ (فصل ۳۱).

بازنمونه‌گیری Resampling
تعیین مقدار پیکسل‌ها هنگام تغییر شبکه‌ی رستر (فصل ۳۳).

باند Band
یک لایه‌ی طیفی یا داده‌ای در رستر (فصل ۲).

برنامه‌ی اجرا Query plan
روشی که پایگاه داده برای اجرای پرسش برمی‌گزیند؛ با EXPLAIN (فصل ۲۰).

بیضوی Ellipsoid
مدل ریاضی شکل زمین (فصل ۳).

تراکنش Transaction
مجموعه دستور همه‌یاهیچ؛ ACID (فصل ۱۴).

ترکیب بی‌ابر Cloud-free composite
میانه‌ی پیکسل‌های بی‌ابر چند تصویر (فصل ۳۳).

تصحیح اتمسفری Atmospheric correction
حذف اثر جو؛ از بازتاب بالای جو به سطح (فصل ۳۳).

تصویر نقشه Map projection
قاعده‌ی بردن سطح خمیده به صفحه (فصل ۳).

تفکیک Resolution
مکانی، طیفی، زمانی و رادیومتری (فصل ۳۲).

توپولوژی Topology
قواعد رابطه‌ی عارضه‌ها: همسایگی، اتصال (فصل‌های ۲ و ۲۲).

حریم Buffer
ناحیه‌ی تا فاصله‌ی معین از هندسه (فصل ۱۸).

حوزه‌ی دسترسی Isochrone / Service area
جاهای قابل رسیدن در زمان یا فاصله‌ی معین روی شبکه (فصل ۲۴).

خودهمبستگی فضایی Spatial autocorrelation
شباهت مقادیر نزدیک به هم؛ شاخص موران (فصل ۲۱).

داده‌نما Datum
جایگاه بیضوی نسبت به زمین؛ WGS84، ED50 (فصل ۳).

درخت R R-tree
ساختار شاخص مکانی از مستطیل‌های تودرتو (فصل ۲۰).

رستر Raster
شبکه‌ی منظم پیکسل‌ها (فصل ۲).

سری زمانی Time series
مقادیر یک کمیت در تاریخ‌های پیاپی (فصل ۳۷).

سطح پردازش Processing level
L1C بازتاب بالای جو، L2A بازتاب سطح (فصل ۳۳).

شاخص Index
ساختار کمکی برای سرعت پرسش؛ در PostGIS از نوع GiST (فصل ۲۰).

شاخص طیفی Spectral index
ترکیب ریاضی باندها؛ NDVI، NDWI، … (فصل ۳۵).

طبقه‌بندی نظارت‌شده Supervised classification
طبقه‌بندی با نمونه‌ی آموزشی (فصل ۳۹).

عارضه Feature
یک پدیده با هندسه و ویژگی؛ یک سطر جدول (فصل ۲).

فرادادە Metadata
داده درباره‌ی داده (فصل ۲).

فنولوژی Phenology
چرخه‌ی زندگی گیاه در طول فصل (فصل ۳۷).

قید Constraint
قاعده‌ای که پایگاه داده همیشه اعمال می‌کند (فصل ۱۴).

لایه Layer
مجموعه‌ی عارضه‌های یک نوع؛ در پایگاه داده یک جدول (فصل ۱).

ماتریس خطا Confusion matrix
مقایسه‌ی کلاس واقعی و پیش‌بینی‌شده (فصل ۳۹).

مدل رقومی ارتفاع DEM / DSM / DTM
رستر ارتفاع؛ سطح یا زمین (فصل ۳۸).

منحنی میزان Contour
خط هم‌ارتفاع (فصل‌های ۳۸ و ۴۹).

نرمال‌سازی Normalization
طراحی جدول‌ها به‌گونه‌ای که هر واقعیت یک جا ذخیره شود (فصل ۱۰).

نزدیک‌ترین همسایه k-Nearest Neighbors (KNN)
یافتن نزدیک‌ترین عارضه‌ها با <-> (فصل ۱۹).

نما View
پرسش ذخیره‌شده‌ای که مثل جدول خوانده می‌شود (فصل ۱۴).

هندسه Geometry
شکل و مکان عارضه؛ نوع داده‌ی PostGIS (فصل‌های ۲ و ۱۵).

ورونوی (تیسن) Voronoi (Thiessen)
تقسیم فضا به نزدیک‌ترین نقطه (فصل ۱۸).

پراکنش دوگانه Double bounce
بازتاب دوباره‌ی موج رادار بین دیوار و زمین؛ شهر روشن (فصل ۴۰).

پوسته‌ی محدب Convex hull
کوچک‌ترین چندضلعی محدب دربرگیرنده (فصل ۱۸).

پیوند مکانی Spatial join
پیوند دو جدول با شرط مکانی (فصل‌های ۱۳ و ۱۷).

پیکسل مختلط Mixed pixel

پیکسلی که بازتاب چند پوشش را با هم دارد (فصل ۳۲).

ژئوئید Geoid

سطح هم‌پتانسیل گرانش؛ مرجع ارتفاع از سطح دریا (فصل ۳).

کاشی برداری Vector tile (MVT)

داده‌ی برداری برش‌خورده برای نقشه‌ی وب سریع (فصل ۴۵).

کانون داغ Hot spot

خوشه‌ی معنادار مقادیر بالا؛ آماره‌ی Gi* (فصل ۲۱).

کلید اصلی Primary key

ستون یکتاکننده‌ی هر سطر (فصل ۱۰).

کلید خارجی Foreign key

ستون اشاره‌گر به کلید اصلی جدول دیگر (فصل‌های ۱۰ و ۱۴).

گراف Graph

مدل شبکه از گره‌ها و یال‌ها (فصل ۲۴).

گزاره‌ی مکانی Spatial predicate

تابع درست/نادرست بین دو هندسه (فصل ۱۷).

گسیلمندی Emissivity

کارایی گسیل حرارتی نسبت به جسم سیاه (فصل ۴۰).

پیوست ب ـ مرجع توابع پرکاربرد PostGIS

حدود هفتاد تابعی که کار روزمره‌ی بیشتر کارشناسان را پوشش می‌دهد، به ترتیب کاربرد. ستون آخر فصلی است که تابع در آن آموزش داده شده است. برای فهرست کامل، مستندات رسمی PostGIS را ببینید.

ساختن هندسه

تابع	کار	فصل
`ST_Point(x, y, srid)`	نقطه با SRID (PostGIS ۳٫۲+)	۱۵
`ST_MakePoint(x, y [, z])`	نقطه بدون SRID	۱۵
`ST_SetSRID(g, srid)`	برچسب SRID (بدون تبدیل)	۳
`ST_GeomFromText(wkt, srid)`	از WKT	۱۵
`ST_GeomFromGeoJSON(json)`	از GeoJSON	۱۵
`ST_MakeLine(a, b)` / `ST_MakeLine(g ORDER BY t)`	خط از نقطه‌ها؛ تجمیعی برای مسیر	۱۵
`ST_MakePolygon(ring)`	چندضلعی از حلقه‌ی بسته	۱۵
`ST_MakeEnvelope(xmin, ymin, xmax, ymax, srid)`	مستطیل	۱۵
`ST_Collect(g)`	چندبخشی (تجمیعی)	۱۲ و ۱۸

خروجی و قالب

تابع	کار	فصل
`ST_AsText(g)` / `ST_AsEWKT(g)`	WKT / با SRID	۱۵
`ST_AsGeoJSON(g, digits)`	GeoJSON؛ با سطر کامل، Feature	۱۵
`ST_AsKML` ، `ST_AsGML` ، `ST_AsSVG`	قالب‌های دیگر	۱۵
`ST_AsMVT` ، `ST_AsMVTGeom`	کاشی برداری	۴۵
`ST_GeoHash(g, n)`	رشته‌ی geohash	۱۵

خواندن اجزا

تابع	کار	فصل
`GeometryType(g)`، `ST_GeometryType(g)`	نوع هندسه	۱۵
`ST_SRID(g)`، `Find_SRID(schema, table, col)`	SRID	۳
`ST_X`، `ST_Y`، `ST_Z`	مختصات نقطه	۱۵
`ST_NPoints`، `ST_NumGeometries`، `ST_GeometryN`	رأس‌ها و اجزا	۱۵
`ST_StartPoint`، `ST_EndPoint`، `ST_PointN`	نقطه‌های خط	۱۵ و ۲۴
`ST_ExteriorRing`، `ST_InteriorRingN`	حلقه‌ها	۱۵ و ۲۲
`ST_Dump(g)`، `ST_DumpPoints(g)`	شکافتن به سطرها	۱۵
`ST_Envelope`، `ST_Extent` (تجمیعی)	محدوده	۱۲ و ۱۵
`ST_Centroid`، `ST_PointOnSurface`	مرکز ثقل / نقطه‌ی تضمینی داخل	۱۵

تبدیل مختصات

تابع	کار	فصل
`ST_Transform(g, srid)`	تبدیل واقعی مختصات	۳
`ST_Transform(g, 'متن PROJ')`	تبدیل به تصویر سفارشی	۱۶
`g::geography`	محاسبه روی بیضوی (متر)	۳ و ۱۶
`ST_FlipCoordinates(g)`	جابه‌جایی x و y	۱۸

اندازه‌گیری

تابع	کار	فصل
`ST_Distance(a, b)`	کوتاه‌ترین فاصله	۱۶
`ST_DistanceSphere(a, b)`	فاصله‌ی کروی سریع (متر)	۱۶
`ST_Length(g)`، `ST_3DLength(g)`	طول	۱۶ و ۲۲
`ST_Area(g)`، `ST_Perimeter(g)`	مساحت و محیط	۱۶
`ST_Azimuth(a, b)`، `ST_Project(p, d, az)`	جهت / نقطه در جهت	۱۶
`ST_ShortestLine(a, b)`، `ST_ClosestPoint(a, b)`	خط و نقطه‌ی نزدیک‌ترین	۱۶ و ۱۹
`ST_MaxDistance`، `ST_HausdorffDistance`	دورترین فاصله / شباهت شکل	۱۶

روابط مکانی (درست/نادرست)

تابع	درست وقتی	فصل
`ST_Intersects(a, b)`	نقطه‌ی مشترک دارند	۱۷
`ST_Contains(a, b)` ، `ST_Within(a, b)`	کاملاً داخل (مرز حساب نمی‌شود)	۱۷
`ST_Covers(a, b)` ، `ST_CoveredBy(a, b)`	کاملاً داخل (با مرز)	۱۷
`ST_Touches` ، `ST_Crosses` ، `ST_Overlaps`	تماس، گذر، همپوشانی جزئی	۱۷
`ST_Equals(a, b)`	از نظر مکانی یکسان	۱۷
`ST_Disjoint(a, b)`	جدا	۱۷
`ST_DWithin(a, b, d)`	تا فاصله‌ی d (با شاخص)	۸، ۱۷ و ۱۹
`ST_Relate(a, b [, pattern])`	ماتریس DE-9IM / الگو	۱۷
`a && b`	برخورد مستطیل‌ها (عملگر شاخص)	۲۰
`a <-> b`	فاصله برای نزدیک‌ترین همسایه (با شاخص)	۱۹

پردازش هندسه

تابع	کار	فصل
`ST_Buffer(g, d [, opts])`	حریم	۱۸
`ST_Union(g)` (تجمیعی) / `ST_Union(a, b)`	ادغام	۱۸
`ST_Intersection`، `ST_Difference`، `ST_SymDifference`	همپوشانی	۱۸
`ST_Simplify`، `ST_SimplifyPreserveTopology`، `ST_SimplifyVW`	ساده‌سازی	۱۸
`ST_SnapToGrid`، `ST_ReducePrecision`، `ST_Snap`	گرد کردن / چسباندن	۱۸
`ST_ConvexHull`، `ST_ConcaveHull(g, ratio)`	پوسته	۱۸
`ST_VoronoiPolygons`، `ST_DelaunayTriangles`	ورونوی / دلونی	۱۸
`ST_Subdivide(g, n)`	خرد کردن برای کارایی	۱۸ و ۲۰
`ST_Split(g, blade)`	بریدن با خط	۱۸
`ST_LineInterpolatePoint`، `ST_LineLocatePoint`، `ST_LineSubstring`	ارجاع خطی	۱۸ و ۳۸
`ST_LineMerge(g)`	پیوستن خط‌ها	۲۴
`ST_GeneratePoints(poly, n)`	نقطه‌ی تصادفی	۱۸
`ST_Multi`، `ST_CollectionExtract(g, type)`، `ST_Force2D`	تبدیل نوع	۱۸ و ۲۲

اعتبار و توپولوژی

تابع	کار	فصل
`ST_IsValid`، `ST_IsValidReason`، `ST_IsValidDetail`	بررسی اعتبار	۷ و ۲۲
`ST_MakeValid(g [, 'method=structure'])`	اصلاح	۲۲
`ST_IsEmpty(g)`	هندسه‌ی خالی	۱۸
`topology.CreateTopology`، `toTopoGeom`	توپولوژی ذخیره‌شده	۲۲

شبکه‌ی منظم و خوشه‌بندی

تابع	کار	فصل
`ST_HexagonGrid(size, bounds)`، `ST_SquareGrid`	شبکه‌ی شش‌ضلعی و مربعی	۲۱
`ST_ClusterDBSCAN(g, eps, minpoints) OVER ()`	خوشه‌های چگال	۲۱
`ST_ClusterKMeans(g, k) OVER ()`	k گروه	۲۱
`ST_ClusterWithin(g, d)`	زنجیره‌ی فاصله	۲۱

رستر

تابع	کار	فصل
`ST_MetaData`، `ST_SummaryStats`، `ST_ValueCount`	فراداده و آمار	۲۳
`ST_Value(rast, point)`	مقدار در نقطه	۲۳
`ST_Clip(rast, geom)`	برش با چندضلعی	۲۳
`ST_MapAlgebra`	جبر نقشه	۲۳ و ۳۵
`ST_Reclass`	طبقه‌بندی دوباره	۲۳
`ST_Slope`، `ST_Aspect`، `ST_HillShade`	تحلیل ارتفاع	۲۳ و ۳۸
`ST_Contour`	منحنی میزان	۳۸
`ST_DumpAsPolygons`، `ST_AsRaster`	رستر ↔ برداری	۲۳
`ST_AsPNG`، `ST_AsGDALRaster`	خروجی	۲۳

سه‌بعدی (SFCGAL)

تابع	کار	فصل
`ST_3DDistance`، `ST_3DLength`	فاصله و طول سه‌بعدی	۲۲
`ST_Extrude(g, dx, dy, dz)`	حجم از ردپا	۲۲
`ST_Volume(ST_MakeSolid(g))`، `ST_3DArea`	حجم و سطح	۲۲

pgRouting

تابع	کار	فصل
`pgr_dijkstra(edges_sql, from, to)`	کوتاه‌ترین مسیر	۲۴
`pgr_aStar` ، `pgr_bdDijkstra` ، `pgr_KSP`	مسیر با حدس، دوطرفه، k مسیر	۲۴
`pgr_drivingDistance(edges_sql, from, cost)`	حوزه‌ی دسترسی	۲۴
`pgr_dijkstraCost` ، `pgr_dijkstraCostMatrix`	ماتریس هزینه	۲۴
`pgr_TSP`	فروشنده‌ی دوره‌گرد	۲۴
`pgr_extractVertices` ، `pgr_connectedComponents`	توپولوژی و بخش‌های شبکه	۲۴

توابع SQL عمومی که در کار مکانی زیاد لازم‌اند

تابع	کار	فصل
`coalesce(a, b)`، `nullif(a, b)`	جایگزین NULL / جلوگیری از تقسیم بر صفر	۱۰ و ۱۳
`round(x::numeric, n)`	گرد کردن (توجه به `::numeric`)	۱۵
`percentile_cont(p) WITHIN GROUP (ORDER BY x)`	میانه و صدک	۱۲
`count(*) FILTER (WHERE …)`	شمارش شرطی	۱۲
`rank()`، `row_number()`، `lag()`، `lead()` با `OVER`	توابع پنجره‌ای	۱۲ و ۳۷
`DISTINCT ON (…)`	اولین سطر هر گروه	۲۴ و ۳۶
`generate_series(a, b[, step])`	دنباله‌ی عدد یا تاریخ	۱۸ و ۲۰
`regr_slope`، `regr_r2`، `corr`	رگرسیون و همبستگی	۱۲ و ۳۷
`translate(s, from, to)`	یکدست کردن رقم فارسی و «ی/ک»	۱۱
`similarity(a, b)`، `a % b` (pg_trgm)	تشابه متن	۱۱

پیوست ج ـ سیستم‌های مختصات ایران

انتخاب سریع

کار	SRID پیشنهادی
ذخیره‌ی عمومی، GPS، تبادل، GeoJSON	`4326`
محاسبه‌ی فاصله و مساحت در یک شهر یا استان	ناحیه‌ی مربوط UTM (`32638` تا `32641`)
محاسبه در مقیاس کشور	`::geography` یا تصویر آلبرز ایران (پایین)
نمایش روی نقشه‌ی وب	`3857` (فقط نمایش)
داده‌ی قدیمی نقشه‌های توپوگرافی	معمولاً ED50 / UTM (`23038` تا `23041`)؛ پیش از تحلیل به WGS84 تبدیل شود

ناحیه‌های UTM ایران

ناحیه	محدوده‌ی طول	نصف‌النهار مرکزی	SRID (WGS84)	SRID (ED50)
۳۸	۴۲ تا ۴۸ درجه‌ی شرقی	۴۵	`32638`	`23038`
۳۹	۴۸ تا ۵۴	۵۱	`32639`	`23039`
۴۰	۵۴ تا ۶۰	۵۷	`32640`	`23040`
۴۱	۶۰ تا ۶۶	۶۳	`32641`	`23041`

شماره‌ی ناحیه برای هر طول: $\lfloor(\lambda + 180)/6\rfloor + 1$؛ و SRID در نیمکره‌ی شمالی: ۳۲۶۰۰ + شماره‌ی ناحیه.

```sql
SELECT city, lon, floor((lon + 180) / 6) + 1 AS utm_zone, 32600 +
floor((lon + 180) / 6)::int + 1 AS srid
FROM (VALUES ('46.29','تبریز'), ('47.07','کرمانشاه'), ('48.67','اهواز'),
('49.59','رشت'), ('51.39','تهران'), ('51.67','اصفهان'),
            ('52.53','شیراز'), ('54.36','یزد'), ('56.27','بندرعباس'),
('57.08','کرمان'), ('59.60','مشهد'), ('60.86','زاهدان')) t(city, lon)
ORDER BY lon;
```

city	lon	utm_zone	srid
تبریز	46.29	38	32638
کرمانشاه	47.07	38	32638
اهواز	48.67	39	32639
رشت	49.59	39	32639
تهران	51.39	39	32639
اصفهان	51.67	39	32639
… (12 rows)			

مرکز استان‌ها و ناحیه‌ی UTM

ناحیه‌ی ۳۸	ناحیه‌ی ۳۹	ناحیه‌ی ۴۰	ناحیه‌ی ۴۱
تبریز، ارومیه، سنندج، کرمانشاه، ایلام	اردبیل، زنجان، همدان و خرم‌آباد (هر چهار لبه‌ی غربی)، اهواز، رشت، قزوین، کرج، تهران، قم، اراک، اصفهان، شهرکرد، یاسوج، شیراز، بوشهر، ساری، سمنان (لبه)	یزد، کرمان، بندرعباس، مشهد، بجنورد، بیرجند، گرگان (لبه)	زاهدان

«لبه» یعنی شهر نزدیک مرز دو ناحیه است؛ برای داده‌ای که از مرز ناحیه عبور می‌کند، ناحیه‌ای را انتخاب کنید که بیشتر داده در آن است، یا با `geography` کار کنید.

تصویر هم‌مساحت برای کل ایران

برای آمار مساحت در سطح کشور، این تعریف PROJ (مخروطی هم‌مساحت آلبرز با مدارهای استاندارد ۲۷ و ۳۸ درجه) را در `ST_Transform` به کار ببرید (فصل ۱۶):

```
+proj=aea +lat_1=27 +lat_2=38 +lat_0=32 +lon_0=54 +x_0=0 +y_0=0
+datum=WGS84 +units=m +no_defs
```

تشخیص سیستم مختصات از روی مقدارها

مقدارهای مختصات	سیستم محتمل
x بین ۴۴ تا ۶۴ و y بین ۲۵ تا ۴۰	جغرافیایی درجه‌ای (`4326`)
x و y برعکس بالا	جغرافیایی با طول و عرض جابه‌جا
x بین حدود ۱۶۰٬۰۰۰ تا ۸۴۰٬۰۰۰ و y بین ۲٬۷۰۰٬۰۰۰ تا ۴٬۴۰۰٬۰۰۰	UTM؛ ناحیه را از محل داده بیابید
x بین ۴٬۹۰۰٬۰۰۰ تا ۷٬۱۰۰٬۰۰۰ و y بین ۲٬۹۰۰٬۰۰۰ تا ۴٬۹۰۰٬۰۰۰	Web Mercator (`3857`)
مقدارهای کوچک (صدها یا هزارها)	سیستم محلی؛ از تهیه‌کننده بپرسید

اگر داده UTM است ولی ناحیه معلوم نیست، آن را با هر ناحیه‌ی ممکن به ۴۳۲۶ تبدیل کنید و ببینید کدام در محل درست می‌نشیند. همیشه پیش از تحلیل، چند عارضه را روی نقشه‌ی پایه بررسی کنید (فصل‌های ۳ و ۷).

از اولین نقشه تا SQL مکانی حرفه‌ای و تحلیل تصویر ماهواره‌ای

این کتاب شما را از نقطه‌ی صفر، بدون پیش‌زمینه‌ی برنامه‌نویسی یا نقشه‌برداری، تا سطح متخصصی می‌برد که پایگاه داده‌ی مکانی می‌سازد، با SQL مکانی پرسش‌های پیچیده می‌پرسد، تصویر ماهواره‌ای را تحلیل می‌کند و نتیجه را روی نقشه‌ی وب منتشر می‌کند.

هر فصل سه لایه دارد: مفهوم علمی مستقل از نرم‌افزار، اجرای گام‌به‌گام در پایگاه داده مکانی ابری با تصویر واقعی رابط، و همان کار با SQL. همه‌ی کدهای کتاب روی داده‌ی نمونه اجرا و نتیجه‌شان کنار کد چاپ شده است.

در این کتاب

- مبانی GIS، سیستم مختصات و داده‌نمای ایران
- طراحی پایگاه داده‌ی مکانی و ورود داده
- SQL از صفر تا پیشرفته
- SQL مکانی و PostGIS
- تحلیل شبکه و مسیریابی
- سنجش از دور و شاخص‌های طیفی
- پایش زمانی با Sentinel-2 و Landsat
- کارتوگرافی، نمودار و داشبورد
- انتشار OGC، کاشی برداری و API
- برداشت GNSS، RTK و COGO
- امنیت، بکاپ و کارایی
- پروژه‌های جامع صنعتی

برای چه کسانی

دانشجو · مدرس و استاد · کارشناس شهری و منابع طبیعی · کشاورزی و آب · نقشه‌بردار · توسعه‌دهنده‌ی وب · مدیر داده

```
SELECT n.name, count(s.*) AS schools
FROM neighborhoods n
JOIN schools s ON ST_Contains(n.geom, s.geom)
GROUP BY n.name ORDER BY schools DESC;
```

شابک:

ویرایش نخست · ۱۴۰۵